Learning Rust from Zero

Andy Shiue

Generated on September 27, 2026



https://creativecommons.org/licenses/by-nc-nd/4.0/

Learning Rust from Zero

Hello! The main goal of this tutorial is to help complete beginners — people who have never written a program before — understand the many concepts in the Rust programming language. There are already plenty of Rust tutorials out there, but they all seem to be written for learners who already know at least one other programming language, so I hope this tutorial can fill that gap. Rust is a language with a very distinctive character: it is powerful, programs written in Rust run fast, and when you use Rust, it's easier to catch mistakes early on, while you're still writing the code. C++, which everyone has heard of, is likewise powerful and fast, but it sacrifices quite a lot when it comes to safety. Because of these characteristics, Rust also plays a pivotal role in this era of AI-assisted programming.

The approach of this tutorial is therefore aligned with how development works in the current AI era: it doesn't assign "homework" on the assumption that you must be able to write algorithms to a particular specification. Instead, this tutorial aims to get you to the point where you can read Rust code and understand roughly what it's doing, and hopefully also understand how the architecture of a real piece of software gets designed. I'd even say that if you can't be bothered to use a computer, simply reading this tutorial without actually writing any code is a viable way to learn Rust too.

That said, I still recommend reading this tutorial chapter by chapter. If you have some background in a statically typed language, you can probably skip Chapter 1, but what I'd recommend even more is spending just a few minutes skimming Chapter 1 before moving on. If you're a complete beginner, that goes without saying. Of course, if you're not worried about missing anything, you can also jump straight to whatever interests you, or use the search feature to read ahead to explanations that only come later in the tutorial — these are all perfectly workable approaches. Oh, and one more thing: there's a chapter called "Appendix I" — despite the name, I recommend reading all of it as well. By the time you finish Part I, including Appendix I, you should be equipped to learn other advanced Rust features on your own. For readers' convenience and for completeness, however, Part II still covers those features.

If, while reading, you run into a passage you don't understand, want practice problems, or your program won't run, and you'd like to ask an AI for help, I've prepared a zip archive for AIs to read, so that the AI can respond to your needs more precisely. To use it, just upload the entire archive to the AI, tell it "Please read the `GUIDE.md` inside the archive before answering me," let the AI know which chapter and episode you're currently on, and then state your request. The download link for the archive is below:

> `rust-book-src.zip`: https://andyshiue.github.io/learning-rust-from-zero/en/rust-book-src.zip

I strongly recommend using a good AI model to read the archive! A free AI might not even bother to read what's inside.

A little slow on the uptake, I only realized later that many readers might hand the entire tutorial

to an AI instead of reading it themselves. If you've diligently made it this far into the foreword — or if your AI has reached this part — here are my thoughts for readers who choose that approach:

1. I had considered that some people might use the tutorial this way, but I didn't expect so many to do so. It is, of course, a perfectly valid approach.
2. Even so, the tutorial itself exists for a reason; otherwise, I wouldn't have written it. Asking an AI only when you encounter code you don't understand may leave you able to read programs but unable to write them, though I admit that readers in exactly that position were part of my intended audience from the beginning.
3. The value of the tutorial itself is that it lets you gradually build a comprehensive understanding of Rust's features.
4. Even if you use AI to help you read, that doesn't mean you can't come back to the tutorial itself whenever you need it.
5. You don't actually need to go to GitHub and `git clone` my project; the `.zip` file above already contains the entire tutorial (you can extract it yourself and take a look). I recommend giving the archive to an AI instead of having it read the website directly because the archive contains plain text, which is easier for an AI to process.
6. I only realized later that there may not be all that many true "complete beginners" these days, so some readers may want to skip Chapter 1. As noted above, that may be workable, but if you find that you don't quite grasp the most basic concepts, the episodes in Chapter 1 are all very short. Reading them really won't take much time.
7. Pay special attention to the first half of the chapter on asynchronous programming. Its essence lies in practicing how to read and understand code, and that remains true even if you use AI to read it. I especially recommend the book *Asynchronous Programming in Rust*. I wrote this tutorial's chapter on asynchronous programming in my own way, using that book as a reference.

This tutorial also has a PDF version available for download:

> PDF version: https://andyshiue.github.io/learning-rust-from-zero/en/book.pdf

However, I may not maintain the PDF version in the long run, so I recommend reading the interactive web version instead. If you happen to be reading the PDF version right now, here's the URL of the web version:

> Web version: https://andyshiue.github.io/learning-rust-from-zero/en/

Finally, let me mention the interactive features I referred to above, since otherwise I'm afraid nobody would notice them: you can run the code directly inside the web version of this tutorial. There are a few buttons at the top-right corner of each code snippet in the text — press them and you'll see what happens. That's about it for now.....

Apart from the outline, the first draft of this tutorial was written by AI and revised by humans:

- Models: Claude 4.5 ~ 5 / GPT-5.5 ~ 6
- Harnesses: OpenClaw / Claude Code / ChatGPT (Codex)

Contents

Learning Rust from Zero i

Part I 1

1 The Basics 2

1.1 Installing Rust . . . 2
1.2 Your First Program . . . 3
1.3 Variables and Output . . . 5
1.4 Comments . . . 6
1.5 Arithmetic Operators . . . 7
1.6 Operator Precedence . . . 8
1.7 Comparison Operators . . . 9
1.8 `if` . . . 11
1.9 Scope . . . 12
1.10 `else` . . . 13
1.11 `else if` . . . 14
1.12 Logical Operators . . . 15
1.13 `let mut` . . . 16
1.14 Compound Assignment Operators . . . 17
1.15 `stdin` . . . 19
1.16 `parse` . . . 20
1.17 Practice Problems . . . 22
1.18 `loop` + `break` . . . 23
1.19 `while` . . . 24
1.20 `for` + Ranges . . . 26
1.21 Nested Loops . . . 27
1.22 `continue` . . . 29
1.23 Types (the Basics) . . . 30
1.24 Types (Numbers in Detail) . . . 31
1.25 `char` . . . 34
1.26 Escape Characters . . . 35
1.27 `if` as an Expression . . . 37

2 Functions, Arrays, and Slices 40

2.1 `const` . . . 40
2.2 Shadowing . . . 41
2.3 Underscore Variables . . . 43

2.4 Tuples . . . 46
2.5 `{:?}` and the `Debug` Format . . . 47
2.6 Simple Functions . . . 49
2.7 Function Parameters . . . 50
2.8 Function Return Values . . . 52
2.9 Early `return` . . . 54
2.10 Recursion . . . 57
2.11 Array Basics . . . 59
2.12 Iterating over Arrays . . . 60
2.13 Slices: `&[T]` . . . 62
2.14 Slices as Parameters . . . 65
2.15 String Slices: `&str` . . . 67

3 Structs, Enums, and Pattern Matching 70
3.1 `struct` (Named Fields) . . . 70
3.2 Tuple `structs` and Unit `structs` . . . 72
3.3 `enum` (C-style) . . . 73
3.4 `match` on C-style `enums` . . . 74
3.5 `match` as an Expression . . . 76
3.6 Block Expressions . . . 77
3.7 `enum` with Tuple Variants . . . 79
3.8 `enum` with `struct` Variants . . . 80
3.9 Destructuring Tuple Variants with `match` . . . 81
3.10 Destructuring `struct` Variants with `match` . . . 83
3.11 Field Shorthand . . . 85
3.12 Tuple Patterns . . . 86
3.13 Slice Patterns . . . 88
3.14 Nested Pattern Matching . . . 89
3.15 The _ Wildcard . . . 91
3.16 Ignoring Multiple Values with `..` . . . 92
3.17 Range Patterns . . . 95
3.18 Multiple Values with | . . . 97
3.19 `@` Bindings . . . 99
3.20 `match` Guards . . . 101
3.21 Destructuring Tuples with `let` . . . 104
3.22 Destructuring `structs` with `let` . . . 105
3.23 Destructuring in `for` Loops . . . 107
3.24 Destructuring in Function Parameters . . . 108
3.25 `if let` . . . 111
3.26 `while let` . . . 113
3.27 `let else` . . . 115
3.28 Associated Functions . . . 117
3.29 Methods . . . 119

3.30 Capital `Self` . . . 121

4 Ownership and Borrowing 124

4.1 Ownership (the Keychain Analogy) . . . 124
4.2 A Brief Introduction to `traits` . . . 125
4.3 Moves and `Clone` . . . 128
4.4 `Copy` . . . 130
4.5 Borrowing: `&` . . . 133
4.6 Mutable Borrowing: `&mut` . . . 136
4.7 The Borrowing Rules . . . 139
4.8 `self` vs `&self` vs `&mut self` . . . 142
4.9 The Stack and the Heap . . . 145
4.10 `String` . . . 147
4.11 `String` vs `&str` . . . 149
4.12 `Vec` Basics . . . 152
4.13 `Vec` and Ownership . . . 154

5 Generics, Trait Bounds, and Lifetimes 158

5.1 Generic Functions . . . 158
5.2 Generic `structs` . . . 159
5.3 Generic `enums` . . . 161
5.4 The Turbofish Syntax . . . 162
5.5 The Placeholder Type `_` . . . 164
5.6 Type Aliases . . . 165
5.7 Generic `impl` . . . 167
5.8 `Option<T>` . . . 169
5.9 Common `Option` Methods . . . 171
5.10 `Result<T, E>` . . . 172
5.11 The `?` Operator . . . 174
5.12 `traits` with Multiple Methods and Default Implementations . . . 177
5.13 `trait` Bounds . . . 179
5.14 `use` Basics . . . 181
5.15 The `Display trait` . . . 182
5.16 Multiple `trait` Bounds and `where` . . . 184
5.17 The `impl Trait` Syntax . . . 186
5.18 Multi-parameter `traits` . . . 188
5.19 `From<T>` / `Into<T>` . . . 190
5.20 `Drop` . . . 193
5.21 `Box<T>` . . . 196
5.22 `Rc<T>` . . . 198
5.23 `Deref` . . . 201
5.24 `Cell<T>` . . . 206
5.25 `RefCell<T>` . . . 208
5.26 Lifetime Basics . . . 210

5.27 Lifetime Elision Rules . . . 214
5.28 Lifetimes on Types . . . 216
5.29 Lifetime Bounds . . . 219
5.30 Supertraits . . . 221
5.31 Common `derive` `traits` . . . 224
5.32 Associated Types . . . 226
5.33 `Cow<'a, B>` . . . 231

6 Closures and Iterators 234
6.1 Function Pointers . . . 234
6.2 Closures in Action . . . 236
6.3 Implementing Closures by Hand . . . 239
6.4 How Closure Kinds Are Inferred . . . 244
6.5 `FnOnce` / `FnMut` / `Fn` . . . 246
6.6 `move` Closures . . . 249
6.7 Closure Methods on `Option` / `Result` . . . 252
6.8 The `Iterator` `trait` . . . 256
6.9 What `for` Loops Really Are . . . 260
6.10 `into_iter` / `iter_mut` / `iter` . . . 262
6.11 Collecting . . . 265
6.12 Aggregation . . . 267
6.13 Combining and Slicing . . . 270
6.14 Transforming and Filtering . . . 272
6.15 Lazy Evaluation . . . 275

7 Cargo, Crates, and the Module System 280
7.1 Cargo and crates.io . . . 280
7.2 `mod` . . . 282
7.3 File `mods` . . . 284
7.4 `pub` Visibility . . . 287
7.5 `use` . . . 292
7.6 `cargo test` . . . 296
7.7 `pub use` . . . 300
7.8 The Orphan Rule . . . 302
7.9 Doc Comments . . . 306
7.10 `cargo publish` . . . 310

8 Appendix I 315
8.1 Number Literal Formats . . . 315
8.2 Short-circuiting of && and || . . . 317
8.3 `break` with a Value . . . 318
8.4 Multiline Strings & Raw String Literals . . . 321
8.5 Format Strings in Depth . . . 323
8.6 `structs` / `enums` inside `fns` . . . 326
8.7 `struct` Update Syntax . . . 328

8.8 `ref` Patterns and `match` Ergonomics . . . 330
8.9 `panic!` / `todo!` / `unimplemented!` / `unreachable!` . . . 332
8.10 `let` Chains . . . 335
8.11 `Rc` Cycles and `Weak` . . . 337
8.12 Fully Qualified Syntax . . . 341
8.13 A Brief Introduction to DSTs . . . 344
8.14 `extern crate` . . . 348

Part II **351**

9 Multithreading **352**
9.1 Pointers . . . 352
9.2 `thread::spawn` . . . 354
9.3 `Send` / `Sync` . . . 357
9.4 `RefCell` under Multithreading . . . 360
9.5 `Rc` under Multithreading . . . 362
9.6 `Arc<T>` . . . 363
9.7 Atomic Types . . . 365
9.8 `Mutex<T>` . . . 368
9.9 `RwLock<T>` . . . 371
9.10 Poisoning . . . 374
9.11 `mpsc` . . . 377
9.12 Deadlocks . . . 379
9.13 A Brief Introduction to `thread::scope` . . . 381

10 Advanced Language Features **385**
10.1 `dyn Trait` Basics . . . 385
10.2 `dyn` Compatibility . . . 390
10.3 `const fn` . . . 393
10.4 Associated `consts` . . . 396
10.5 `const` Generics . . . 398
10.6 Default Parameters . . . 400
10.7 Operator Overloading . . . 402
10.8 Type Conversion with `as` . . . 405
10.9 `enum` Discriminants . . . 407
10.10 An Overview of Attributes . . . 410
10.11 The `cfg!` Macro . . . 412
10.12 `macro_rules!` . . . 414
10.13 Proc Macros . . . 418
10.14 `unsafe` . . . 420
10.15 `static` Variables . . . 424
10.16 `LazyLock` . . . 425
10.17 `extern` Blocks . . . 426
10.18 `union` . . . 428
10.19 The Never Type `!` . . . 430

11 Advanced Standard Library Topics **433**
11.1 `AsRef<T>` / `AsMut<T>` . . . 433
11.2 `Ordering` and Sorting . . . 435
11.3 `HashMap<K, V>` . . . 438
11.4 `HashSet<T>` . . . 441
11.5 A Brief Tour of Other Collections . . . 444
11.6 `std::env` / `std::process` . . . 446
11.7 `std::path` . . . 448
11.8 String Methods . . . 450
11.9 I/O: `stdin` and File Access . . . 452
11.10 The `Error` `trait` . . . 455
11.11 A Brief Introduction to `thiserror` / `anyhow` . . . 458
11.12 `catch_unwind` . . . 461
12 Async **464**
12.1 Your First `async` Program . . . 464
12.2 Why We Need `async` . . . 467
12.3 `async fn` Returns a `Future` . . . 468
12.4 `async` as an Effect . . . 470
12.5 `async` Blocks . . . 473
12.6 The `Future` `trait` and the Most Bare-bones Executor . . . 476
12.7 Writing a `Delay` `Future` by Hand . . . 478
12.8 Waiting for `Delay` with `.await` . . . 481
12.9 Writing join by Hand . . . 483
12.10 Waking the Executor with `Threads` and `Wakers` . . . 487
12.11 `spawn` and the Ready Queue . . . 491
12.12 `spawn` and `JoinHandle` . . . 496
12.13 `mio` . . . 502
12.14 Writing a Reactor by Hand . . . 505
12.15 The State Machine behind `async fn` . . . 513
12.16 Self-referential `Futures` . . . 516
12.17 Why `poll` Needs `Pin` . . . 519
12.18 `Unpin` . . . 523
12.19 `pin!` . . . 526
12.20 `async` Recursion . . . 528
12.21 Back to Tokio . . . 530
12.22 `spawn_blocking` . . . 532
12.23 `join!` . . . 534
12.24 `Semaphore` and Backpressure . . . 536
12.25 `mpsc` . . . 538
12.26 `oneshot`, `watch`, and `broadcast` . . . 540
12.27 `AsyncRead` and `AsyncWrite` . . . 542
12.28 `select!` . . . 544

12.29 `async` `Mutex`, `RwLock`, and `Notify` . . . 548
12.30 `Stream` . . . 551
12.31 `JoinSet` and `FuturesUnordered` . . . 553
12.32 Graceful Shutdown . . . 555
12.33 Testing `async` Code . . . 560
12.34 Runtimes Other Than Tokio . . . 562

13 Appendix II **564**
13.1 `'a: 'b` . . . 564
13.2 HRTB . . . 570
13.3 Variance . . . 575
13.4 `PhantomData` / `PhantomPinned` . . . 579
13.5 GAT . . . 585
13.6 `async` Closures . . . 589
13.7 `use` Bounds . . . 596

Afterword **602**

Part I

Chapter 1

The Basics

In this chapter, this tutorial will walk you through the basic program control flow found in most programming languages. For example, if we want a program to make decisions based on the current temperature — say, turning on the air conditioner when the temperature exceeds 30 degrees — then we need to check a condition: is the current temperature greater than some number? If the answer is yes, turn on the air conditioner; if the answer is no, do something else. Although this chapter won't teach you how to actually hook your program up to an air conditioner, you will at least learn how to write conditional checks in the language. Or perhaps you'd like to make an LED blink once per second. In that case, what we need is to tell the computer to repeat an action over and over: light on, light off. Likewise, this chapter won't teach you how to connect your program to an LED — it won't even teach you how to wait one second — but you will learn how to tell the computer to perform an operation repeatedly.

1.1 Installing Rust

1.1.1 Goal of This Episode

Get Rust installed on your computer and make sure it works.

1.1.2 Main Text

Hello! Welcome to this Rust tutorial series!

This is episode one, and we won't write any code yet — we'll just get the tools set up. If you want to cook, you need a pot first, right?

1.1.2.1 Installing rustup

Rust has an official installation tool called **rustup**, which installs everything you need in one go.

Open your browser and head to this URL:

```
https://rustup.rs
```

- **Windows users**: Download `rustup-init.exe`, double-click to run it, and just keep pressing enter to accept the defaults.
- **Mac / Linux users**: Open a terminal and paste in this command:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

It will ask whether you want to use the default settings — just press enter.

1.1.2.2 Confirming the Installation Succeeded

Once it's done, open a **new** terminal (this matters — the old one may not pick up the changes yet), and type:

```
rustc --version
```

If you see something like this:

```
rustc 1.XX.X (xxxxxxx 20XX-XX-XX)
```

Congratulations! Rust is installed!

`rustc` is the Rust compiler. Its job is to turn the code you write into something the computer can run. As for what a compiler is, we'll get to that little by little later — for now, all you need to know is "it's installed, and it works."

1.1.3 Recap

- Install Rust with **rustup**, which sets up all the tools you need in one go.
- After installing, open a new terminal and confirm the installation with `rustc --version`.
- `rustc` is the Rust compiler, which turns your code into something the computer can run.

1.2 Your First Program

1.2.1 Goal of This Episode

Create a project with Cargo and run the very first Rust program of your life.

1.2.2 Main Text

Last episode we got Rust installed, so today let's write our first program!

1.2.2.1 Creating a Project with Cargo

Rust comes with a wonderfully handy tool called **Cargo**, which handles building Rust projects and managing their dependencies. You can think of it as a butler that organizes your code, compiles it, and runs it — it takes care of everything.

Open a terminal and type:

```
cargo new hello
```

This creates a folder named `hello` for you, with a basic file structure already set up inside.

1.2.2.2 Opening It in VS Code

Next, open the `hello` folder in VS Code (or your favorite editor). You'll see two important things:

1. **Cargo.toml** — This is the project's configuration file. It records things like your project's name and version. You don't need to worry about it right now; just know it exists.
2. **src/main.rs** — This is your code! Open it up and take a look:

```rust
fn main() {
    println!("Hello, world!");
}
```

This is the first program Rust generated for you automatically. `fn main()` is the entry point of the program — every program starts running from here. `println!` is the command for printing things to the screen. Throughout Chapter 1, we'll only ever write code inside the curly braces that follow `fn main()`.

1.2.2.3 What Is Compiling?

Before we run the program, let's cover an important concept.

The `.rs` files we write contain code meant for humans to read — computers can't actually understand it. So we need a translation step that turns the code we write into a file the computer can execute directly. This translation step is called **compiling**.

The tool responsible for this is called a **compiler**, and Rust's compiler is the `rustc` we installed last episode.

The good news is that you don't need to invoke `rustc` yourself: the `cargo run` command we're about to use will automatically compile and then run your program, all in one step.

1.2.2.4 Let's Run It

Back in the terminal, first go into the `hello` folder:

```
cd hello
```

Then type:

```
cargo run
```

You should see this printed on the screen:

```
Hello, world!
```

Fantastic! Your first Rust program is up and running!

1.2.2.5 Change It and Run It Again

Now go back to your favorite text editor and change the text inside `println!` to:

```rust
fn main() {
    println!("Hello, Rust!");
}
```

Save the file, then go back to the terminal and run `cargo run` again:

```
Hello, Rust!
```

See that? Whatever you change it to, that's what it prints. That's what programming is all about — you tell the computer what to do, and it does exactly that.

1.2.3 Recap

- **Cargo** builds Rust projects and manages their dependencies; use `cargo new` to create a new project.
- Inside a project, `Cargo.toml` is the configuration file and `src/main.rs` is the main code.
- **Compiling** means translating human-readable code into a file the computer can run.

- Use `cargo run` to compile and run in one step.
- `fn main()` is the program's entry point, and `println!` prints things to the screen.

1.3 Variables and Output

1.3.1 Goal of This Episode

Learn to create variables with `let`, then print them out with `println!`.

1.3.2 Main Text

Last episode we successfully printed "Hello, Rust!", but that text was hard-coded into the program. What if we want to be a bit more flexible? That's where **variables** come in.

1.3.2.1 What Is a Variable?

A variable gives a value a name, so you can use that value later by that name.

Let's see how to use one:

```rust
fn main() {
    let x = 5;
    println!("{}", x);
}
```

Here, `let x = 5;` binds the name `x` to the value `5`.

Then, in `println!("{}", x);`, the `{}` is a placeholder — it means "at this spot, please fill in the value of `x`."

1.3.2.2 Text Variables

A variable name can be bound to more than just a number — it can be bound to text too:

```rust
fn main() {
    let name = "Rust";
    println!("Hello, {}!", name);
}
```

See that? The `{}` was replaced by the value of `name`, which is `"Rust"`.

Try changing `"Rust"` to your own name and see what gets printed!

1.3.2.3 A `let` Variable Doesn't Have to Be Assigned Right Away

When you declare a variable with `let`, you don't have to give it a value immediately. You can declare first and assign later:

```rust
fn main() {
    let x;
    x = 5;
    println!("{}", x);
}
```

This is perfectly legal, but you must assign to it exactly once before using it — using it without assigning causes a compile error.

1.3.3 Recap

- `let` creates a variable.
- Text is wrapped in `"double quotes"`.
- `println!("{}", variable)` prints out the value of a variable.
- `{}` is a placeholder that gets replaced by the value that follows.
- A variable declared with `let` doesn't have to be assigned right away, but it must be assigned exactly once before use.

1.4 Comments

1.4.1 Goal of This Episode

Learn to write notes (comments) inside your code, so you and others can tell what you're doing.

1.4.2 Main Text

When writing code, sometimes you'll want to jot a note beside it, reminding yourself "here's what this part does." That's what **comments** are for.

Comments are never executed by the computer — they exist purely for humans to read.

1.4.2.1 Single-line Comments

Start with `//`, and the rest of the line becomes a comment:

```
fn main() {
    // This is a comment; the computer ignores this line
    let x = 5; // You can also put one after code
    println!("{}", x);
}
```

Running this still prints just 5 — those two comments have no effect on the program at all.

1.4.2.2 Multi-line Comments

If you want to write a longer note, you can wrap it in `/* */`:

```
fn main() {
    /*
        This is a multi-line comment
        It can span several lines
        The computer ignores all of it
    */
    let x = 10;
    println!("{}", x);
}
```

1.4.2.3 When Should You Write Comments?

- When the logic of a piece of code isn't obvious.
- When you're worried you'll forget what it does when you come back a few days later.
- When you want to temporarily stop a line of code from running (i.e., "comment it out").

```rust
fn main() {
    let x = 5;
    // println!("{}", x); // Not printing for now, but don't want to delete it
    println!("Program finished");
}
```

Now `println!("{}", x);` won't be executed, but you can bring it back to life at any time by removing the `//`.

1.4.2.4 A Small Reminder

You don't need a comment on every line! Good code should be clear enough on its own. Comments are for the "non-obvious" spots — not for explaining every single line.

1.4.3 Recap

- `//` is a single-line comment; `/* */` is a multi-line comment.
- Comments are for humans; the computer ignores them completely.
- You can use comments to temporarily "switch off" a line of code without deleting it.
- Good code should be clear on its own; save comments for the non-obvious parts.

1.5 Arithmetic Operators

1.5.1 Goal of This Episode

Learn to add, subtract, multiply, divide, and take remainders in Rust.

1.5.2 Main Text

Today we're doing math! Don't worry — it's just arithmetic.

1.5.2.1 The Four Basic Operations

First, create two variables:

```rust
fn main() {
    let a = 10;
    let b = 3;

    println!("{} + {} = {}", a, b, a + b); // 13
    println!("{} - {} = {}", a, b, a - b); // 7
    println!("{} * {} = {}", a, b, a * b); // 30
    println!("{} / {} = {}", a, b, a / b); // 3
    println!("{} % {} = {}", a, b, a % b); // 1
}
```

1.5.2.2 Wait — How Is 10 / 3 Equal to 3?

Good question! Because `a` and `b` are both integers, Rust performs **integer division**: everything after the decimal point simply gets chopped off. 10 divided by 3 is 3.333..., and chopping off the decimals gives 3.

1.5.2.3 What Is %?

`%` is the **remainder** operator (the modulo operation). 10 divided by 3 is 3 with a remainder of 1, so `10 % 3` is `1`.

You can think of it as: "How many 3s fit inside 10? Three of them, with 1 left over." That leftover is the remainder.

1.5.2.4 Using Multiple {}s

Did you notice? We put three {}s inside `println!`:

```
fn main() {
    let a = 10;
    let b = 3;
    println!("{} + {} = {}", a, b, a + b);
}
```

Rust fills in the values in order:

- The first {} → the value of `a` (10).
- The second {} → the value of `b` (3).
- The third {} → the value of `a + b` (13).

The number of {}s matches the number of values that follow, and the order must line up.

1.5.3 Recap

- The five arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (remainder).
- Integer division discards the decimal part (`10 / 3` is `3`, not `3.333`).
- `%` takes the remainder: `10 % 3` is the `1` left over after dividing 10 by 3.
- `println!` can contain multiple {}s, matched in order with the values that follow.

1.6 Operator Precedence

1.6.1 Goal of This Episode

Understand Rust's order of operations — multiplication and division before addition and subtraction — and how to change the order with parentheses.

1.6.2 Main Text

Last episode we learned addition, subtraction, multiplication, and division — but what if we mix them together? Which one does the computer compute first?

1.6.2.1 Multiplication and Division First, Then Addition and Subtraction

```
fn main() {
    println!("{}", 2 + 3 * 4);
}
```

What do you think the answer is?

If you thought 20 (first 2 + 3 = 5, then times 4), that's wrong!

The answer is **14**. Just like in math, Rust does **multiplication and division before addition and subtraction**. So it first computes `3 * 4 = 12`, then `2 + 12 = 14`.

1.6.2.2 Changing the Order with Parentheses

What if you really do want the addition to happen first? Just add parentheses:

```
fn main() {
    println!("{}", (2 + 3) * 4);
}
```

This time the answer is **20**. Whatever is inside the parentheses gets computed first: `2 + 3 = 5`, then `5 * 4 = 20`.

1.6.2.3 A Little Tip

When you're not sure about the order, just add parentheses. Parentheses don't just change the order — sometimes they also make code easier to read. Even when the order is already correct, there's no harm in adding parentheses to make your intent clearer.

```
fn main() {
    // These two lines give the same result, but the second is clearer
    println!("{}", 2 + 3 * 4);
    println!("{}", 2 + (3 * 4));
}
```

1.6.3 Recap

- Rust's operator precedence works just like math: multiplication and division come before addition and subtraction.
- Parentheses `()` force a different order of operations.
- When operator precedence isn't obvious, parentheses can make the intended order clearer.

1.7 Comparison Operators

1.7.1 Goal of This Episode

Learn to use comparison operators to compare sizes and check equality.

1.7.2 Main Text

So far we've been doing math, but there's another very important kind of operation in programming — **comparison**.

The result of a comparison isn't a number; it's `true` or `false`.

1.7.2.1 == Equal To

```
fn main() {
    println!("{}", 5 == 5);
}
```

Is 5 equal to 5? Yes, so it's `true`.

Careful here: it's **two equals signs** ==, not one. A single equals sign = is for assigning values to variables (`let x = 5;`); two equals signs == are for comparing.

1.7.2.2 != Not Equal To

```
fn main() {
    println!("{}", 5 != 3);
}
```

Is 5 not equal to 3? Correct.

1.7.2.3 < Less Than

```
fn main() {
    println!("{}", 3 < 5);
}
```

3 is less than 5.

1.7.2.4 > Greater Than

```
fn main() {
    println!("{}", 10 > 7);
}
```

10 is greater than 7.

1.7.2.5 <= Less Than or Equal To

```
fn main() {
    println!("{}", 5 <= 5);
}
```

Is 5 less than or equal to 5? Being equal counts too.

1.7.2.6 >= Greater Than or Equal To

```
fn main() {
    println!("{}", 8 >= 10);
}
```

Is 8 greater than or equal to 10? No.

1.7.2.7 At a Glance

Operator	Meaning	Example	Result
==	equal to	`5 == 5`	`true`
!=	not equal to	`5 != 3`	`true`
<	less than	`3 < 5`	`true`
>	greater than	`10 > 7`	`true`
<=	less than or equal to	`5 <= 5`	`true`
>=	greater than or equal to	`8 >= 10`	`false`

1.7.3 Recap

- The six comparison operators: ==, !=, <, >, <=, >=.
- The result of a comparison is `true` or `false`.
- == (two equals signs) compares; = (one equals sign) assigns. Don't mix them up.

1.8 `if`

1.8.1 Goal of This Episode

Use `if` to let a program decide whether to do something based on a condition.

1.8.2 Main Text

So far, our programs have run from top to bottom, one line at a time. But real programs need to make "judgments" — if such-and-such, then do something.

That's what `if` is for.

1.8.2.1 Basic Usage

```
fn main() {
    let x = 7;
    if x > 3 {
        println!("Greater than 3");
    }
}
```

The logic is simple: x is 7. Is 7 greater than 3? Yes, so the code inside the curly braces {} runs.

1.8.2.2 What If the Condition Doesn't Hold?

Try changing x to 1:

```
fn main() {
    let x = 1;
    if x > 3 {
        println!("Greater than 3");
    }
}
```

Run it and... nothing. Because 1 is not greater than 3, the condition is `false`, so the code inside the curly braces gets skipped.

1.8.3 Recap

- `if` is followed by a condition; if the condition is `true`, the code inside the curly braces runs.
- If the condition is `false`, the whole block is skipped.
- Rust's `if` conditions don't need parentheses around them.

1.9 Scope

1.9.1 Goal of This Episode

Understand the "region" created by curly braces `{}`, and why a variable can't be used once you're outside its braces.

1.9.2 Main Text

This episode covers a very important concept — **scope**.

1.9.2.1 What Is a Scope?

You can think of a pair of curly braces `{}` as a room. Things created inside the room can't be seen once you leave it.

Take a look at this example:

```
fn main() {
    {
        let y = 10;
        println!("{}", y);
    }
}
```

So far so good.

1.9.2.2 What Happens Outside the Braces?

Now try using `y` outside the braces:

```
fn main() {
    {
        let y = 10;
        println!("{}", y);
    }
    println!("{}", y); // This line causes an error!
}
```

You'll get a compile error — Rust is telling you: "I can't find this thing called `y`."

Why? Because `y` was created inside that pair of curly braces, and the moment you step outside them, `y` is gone. It's like putting a chair in a room: once the door is closed, you can't see that chair from the hallway.

1.9.2.3 Why Have Scopes at All?

This is actually a good thing. It keeps your variables from wandering into places they shouldn't be. Imagine if every variable were usable anywhere in the program — once the program got big, it would be utter chaos. Scopes keep things neat and organized for you.

1.9.2.4 Not Just Standalone Braces

The `if` we learned last episode has curly braces too, right? Well, the braces of an `if` also form a scope — variables inside can't be seen from outside. When you see `{}`, the inside is often a scope. It's a very consistent rule in Rust.

1.9.3 Recap

- Curly braces {} often enclose a scope.
- A variable created inside a scope disappears once you leave the {} — it can't be used anymore.
- An `if` forms its own scope.

1.10 `else`

1.10.1 Goal of This Episode

Use `else` to make the program do something different when the condition doesn't hold.

1.10.2 Main Text

When we learned `if`, the program simply did nothing if the condition didn't hold. But often we want to say: "If this, do A; otherwise, do B." That's what `else` is for.

1.10.2.1 Basic Usage

```
fn main() {
    let x = 2;
    if x > 5 {
        println!("big");
    } else {
        println!("small");
    }
}
```

x is 2. Is 2 greater than 5? No, so the `if` block is skipped and the `else` block runs, printing "small".

1.10.2.2 Try a Different Value

Change x to 8:

```
fn main() {
    let x = 8;
    if x > 5 {
        println!("big");
    } else {
        println!("small");
    }
}
```

This time it prints "big", because 8 is greater than 5 — the condition holds, so the `if` side runs.

1.10.2.3 In Plain Words

You can think of `if...else...` as:

> **If** the condition holds, do this; **otherwise**, do that.

Exactly one side will run — never both, and never neither.

1.10.3 Recap

- `else` follows an `if` and handles what to do when the condition doesn't hold.
- `if...else...` is an either-or: exactly one side runs, never both and never neither.

1.11 `else if`

1.11.1 Goal of This Episode

Use `else if` to handle multiple condition branches — not just two-way choices, but three-way, four-way, and beyond.

1.11.2 Main Text

The `if...else...` from last episode only handles "pick one of two." But what if there are more cases? Say, letter grades: A, B, C, F... That's when you need `else if`.

1.11.2.1 Example: Letter Grades

```
fn main() {
    let score = 85;

    if score >= 90 {
        println!("A");
    } else if score >= 80 {
        println!("B");
    } else if score >= 70 {
        println!("C");
    } else {
        println!("F");
    }
}
```

1.11.2.2 How Does It Decide?

Rust goes **from top to bottom**, checking the conditions one by one:

1. `score >= 90`? Is 85 >= 90? No, skip.
2. `score >= 80`? Is 85 >= 80? Yes! Print `"B"`, then stop.
3. Everything after that is never looked at.

This is important: **as soon as one condition holds, all the rest are skipped**.

1.11.2.3 Try Other Scores

- `score = 95` → prints `"A"`.
- `score = 73` → prints `"C"`.
- `score = 50` → prints `"F"` (nothing above holds, so it falls through to `else`).

1.11.2.4 The Structure

```
if condition1 {
    ...
} else if condition2 {
```

```
    ...
} else if condition3 {
    ...
} else {
    ... (when none of the above holds)
}
```

You can have as many `else if`s as you like. The final `else` is optional (though it's usually a good idea to include it, in case some situation slips through).

1.11.3 Recap

- `else if` handles multiple condition branches — more than just two-way choices.
- Rust checks conditions from top to bottom; the first one that holds gets executed, and all the rest are skipped.
- The final `else` is optional; it handles the case where "none of the above holds."

1.12 Logical Operators

1.12.1 Goal of This Episode

Learn to combine multiple conditions with && (and), || (or), and ! (not).

1.12.2 Main Text

Over the last few episodes we learned `if`, but our conditions were all simple — just one at a time. In real life, you often need to consider several conditions at once, such as "at least 18 years old **and** a student." That's where logical operators come in.

1.12.2.1 && — AND

Both conditions must hold for the result to be `true`:

```
fn main() {
    let age = 24;
    let is_student = true;

    if age >= 18 && is_student {
        println!("An adult student");
    }
}
```

Since 24 >= 18 is `true` and `is_student` is also `true`, both hold, so the whole thing is `true`.

If you change `age` to 15, then 15 >= 18 is `false`, and no matter whether `is_student` is `true` or not, the whole thing is `false`, so nothing is printed.

1.12.2.2 || — OR

As long as **either** condition holds, the result is `true`:

```
fn main() {
    let is_weekend = false;
    let is_holiday = true;
```

```
    if is_weekend || is_holiday {
        println!("No work today!");
    }
}
```

Although `is_weekend` is `false`, `is_holiday` is `true` — one `true` is all it takes.

1.12.2.3 `!` — NOT

Turns `true` into `false` and `false` into `true`:

```
fn main() {
    let raining = false;

    if !raining {
        println!("Let's go out for a walk!");
    }
}
```

`raining` is `false`; with `!` in front it becomes `true`, so the condition holds.

You can read it as: "If it's **not** raining, go out for a walk."

1.12.3 Recap

- && (and): `true` only when both sides are `true`.
- || (or): `true` as long as either side is `true`.
- `!` (not): turns `true` into `false` and `false` into `true`.
- Combine multiple conditions with logical operators to write more precise checks.

1.13 `let mut`

1.13.1 Goal of This Episode

Understand that Rust variables are immutable by default, and that you need `mut` to change their values.

1.13.2 Main Text

Today let's talk about one of Rust's most distinctive design decisions — **variables are immutable by default**.

1.13.2.1 First, See What Happens

```
fn main() {
    let x = 5;
    x = 10;
    println!("{}", x);
}
```

Do you think it prints 10? It doesn't. You get a compile error. Rust is telling you: "x is immutable — you can't give it a new value."

1.13.2.2 Wait, Why Not?

In many programming languages, variables can be changed freely. But Rust's attitude is: **if you don't intend to change it, don't let it be changeable**.

Why? Because if you know a value never changes, you don't have to worry about it being modified behind your back while reading the code. This matters a lot in large programs.

1.13.2.3 To Change It, Add `mut`

If you really do need to change the value, add `mut` (short for "mutable"):

```
fn main() {
    let mut x = 5;
    println!("x was originally {}", x);
    x = 10;
    println!("x is now {}", x);
}
```

This time it works! By writing `let mut`, you've told Rust: "I'm going to change this variable later."

1.13.2.4 Quick Summary

```
    let x = 5;     // Immutable; can't be changed later
    let mut x = 5; // Mutable; can be changed later
```

Rust isn't forbidding you from changing variables — it just wants you to **say so explicitly**. This is part of Rust's design philosophy: **make choices consciously**.

1.13.3 Recap

- Rust variables are **immutable** by default; they can't be reassigned.
- To make a variable changeable, add `mut` when declaring it: `let mut x = 5;`.
- To modify a mutable variable's value, just write `x = new_value;` (no `let` needed again).
- This is Rust's design philosophy: make you choose explicitly, rather than silently allowing modification.

1.14 Compound Assignment Operators

1.14.1 Goal of This Episode

Learn to update a variable's value using shorthand like `+=` and `-=`.

1.14.2 Main Text

Last episode we learned `let mut`, which makes variables changeable. Today let's learn a lazier way to write updates.

1.14.2.1 What Does `x = x + 5` Mean?

First, a very important idea. Suppose you have a variable x that's 10, and you want to add 5 to it:

```rust
fn main() {
    let mut x = 10;
    x = x + 5;
    println!("{}", x); // 15
}
```

Here x appears on both the left and right sides of =. This is not the mathematical statement "x equals x + 5" (which makes no sense in math, right?). In programming it means: **first compute the right side, x + 5 (that is, 10 + 5 = 15), then store the result back into the x on the left**. So x goes from 10 to 15.

1.14.2.2 The += Shorthand

That line actually has a shorter way of being written:

```rust
fn main() {
    let mut x = 10;
    x += 5;
    println!("{}", x); // 15
}
```

x += 5 means exactly x = x + 5, just more concise.

1.14.2.3 The Other Compound Assignment Operators

Subtraction, multiplication, division, and remainder all have corresponding shorthands:

```rust
fn main() {
    let mut a = 20;

    a -= 3;
    println!("20 - 3 = {}", a); // 17

    a *= 2;
    println!("17 * 2 = {}", a); // 34

    a /= 4;
    println!("34 / 4 = {}", a); // 8 (integer division)

    a %= 3;
    println!("8 % 3 = {}", a);  // 2
}
```

1.14.2.4 At a Glance

Shorthand	Equivalent to
x += 5	x = x + 5
x -= 5	x = x - 5
x *= 5	x = x * 5
x /= 5	x = x / 5
x %= 5	x = x % 5

1.14.2.5 A Small Reminder

To use these operators, the variable must be declared with `let mut`, because you are **changing** its value.

1.14.3 Recap

- The compound assignment operators: `+=`, `-=`, `*=`, `/=`, `%=`.
- `x += 5` is shorthand for `x = x + 5` — compute the right side first, then store it back on the left.
- Requirement: the variable must be declared with `let mut`.

1.15 stdin

1.15.1 Goal of This Episode

Make the program read the user's keyboard input — copy the code as-is for now; you don't need to fully understand every line.

1.15.2 Main Text

So far, the values in our programs have all been hard-coded. But what if we want the user to enter values themselves? For example, letting the user enter their name so the program can greet them?

1.15.2.1 First, Copy This Code As-Is

```
fn main() {
    println!("Please enter your name:");

    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("failed to read input");

    println!("Hello, {}!", input.trim());
}
```

What it looks like when run:

```
Please enter your name:
Andy
Hello, Andy!
```

1.15.2.2 What Is This Doing?

I know this looks a bit intimidating, but don't worry — for now, treat it as a **black box**. All you need to know is that it reads user input.

Roughly speaking:

1. `let mut input = String::new();` → Create an empty text variable, ready to receive input.
2. `std::io::stdin().read_line(&mut input).expect("failed to read input");` → Read one line of text from the keyboard and store it in `input`.

3. `input.trim()` → Strip off the extra whitespace and the newline character.

As for what `String::new()`, `&mut`, and `.expect()` mean — we'll get to those gradually. For now, just copy them as-is.

1.15.2.3 Why No Explanation Yet?

Because explaining this code requires several concepts we haven't learned yet. Rather than force-feeding you a pile of incomprehensible explanations, it's better to learn to use it first — understanding will come naturally later.

It's like learning to ride a bike as a kid: you didn't need to study mechanics and gyroscopic effects first — you just got on and rode.

1.15.2.4 The Important Bit

Whenever you need to read user input, grab these three lines:

```
    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("failed to read input");
    let name = input.trim(); // Strip the trailing newline
```

1.15.3 Recap

- The three-line boilerplate for reading user input: `String::new()` → `stdin().read_line(&mut input)` → `.trim()`.
- Treat it as a black box and copy it for now; the underlying concepts will come gradually later.
- `.trim()` strips the newline character off the end of the input.

1.16 parse

1.16.1 Goal of This Episode

Learn to convert text the user typed in into a number.

1.16.2 Main Text

Just like last episode, feel free to copy the syntax in this episode as-is — you don't need to fully understand what every line does. We'll come back and explain once we've learned more concepts.

Last episode we learned to read the user's input, but what comes in is **text**. If the user types 42, to Rust that's a piece of text, not the number 42.

You can't do arithmetic on text, so we need to "convert" it into a number.

1.16.2.1 Text to Number

```
fn main() {
    println!("Please enter a number:");

    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("failed to read input");
```

```
    let num = input.trim().parse::<i32>().expect("not a number");

    println!("The number you entered is {}", num);
}
```

Running it:

```
Please enter a number:
42
The number you entered is 42
```

1.16.2.2 The Key Line Is This One

```
    let num = input.trim().parse::<i32>().expect("not a number");
```

Breaking it down:

1. `input.trim()` → Strip whitespace and newlines from both ends.
2. `.parse::<i32>()` → **Parse** the text into an integer (`i32` is one of the integer types).
3. `.expect("not a number")` → If the conversion fails (say, the user typed "abc"), print this error message and end the program.

1.16.2.3 A Complete Interactive Example

```
fn main() {
    println!("Please enter a number:");

    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("failed to read input");

    let num = input.trim().parse::<i32>().expect("not a number");

    println!("{} times 2 is {}", num, num * 2);
}
```

```
Please enter a number:
7
7 times 2 is 14
```

Now you can read numbers and compute with them!

1.16.3 Recap

- What the user types in is text; use `.parse::<i32>()` to turn it into an integer before doing arithmetic.
- `.expect("error message")` prints the message and ends the program if the conversion fails.
- The full pipeline: `input.trim().parse::<i32>().expect("not a number")`.

1.17 Practice Problems

1.17.1 Goal of This Episode

Combine what we've learned so far into a small "enter a score → get a grade" program.

1.17.2 Main Text

Congratulations on making it this far! Today we're going to string together everything we've learned into a genuinely useful little program.

1.17.2.1 The Goal

Let the user enter a score, and have the program determine the letter grade and print it.

1.17.2.2 The Complete Code

```
fn main() {
    println!("Please enter your score:");

    // Read user input
    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("failed to read input");

    // Convert the text into a number
    let score = input.trim().parse::<i32>().expect("not a number");

    // Determine the grade
    if score >= 90 {
        println!("Your grade is A");
    } else if score >= 80 {
        println!("Your grade is B");
    } else if score >= 70 {
        println!("Your grade is C");
    } else {
        println!("Your grade is F");
    }
}
```

1.17.2.3 Give It a Run

```
Please enter your score:
85
Your grade is B
```

```
Please enter your score:
92
Your grade is A
```

```
Please enter your score:
45
Your grade is F
```

1.17.2.4 A Look Back at the Techniques We Used

1. `println!` → printing a prompt message (Episode 2)

2. `let mut` + `String::new()` → getting ready to receive input (Episode 15)
3. `stdin().read_line(&mut input)` → reading keyboard input (Episode 15)
4. `.trim().parse::<i32>()` → converting text to a number (Episode 16)
5. `if` / `else if` / `else` → conditional logic (Episodes 8, 10, 11)

See that? By combining various features, you can build an interactive little program. That's the charm of programming — piece small bits of knowledge together and you can make something useful.

1.17.2.5 Challenges

If you'd like more practice, try these:

- Add a D grade (scores 60 ~ 69).
- If the score is above 100 or below 0, print "Invalid score".

1.17.3 Recap

- Combining the `stdin`, `parse`, `if`, and `else if` we learned earlier gives you an interactive program.
- The charm of programming: piecing small bits of knowledge together makes something useful.
- Beyond learning new syntax, practicing how to combine things matters a lot too.

1.18 `loop` + `break`

1.18.1 Goal of This Episode

Create an infinite loop with `loop`, then use `break` to jump out at the right moment.

1.18.2 Main Text

Up to now, our programs run once and end. But what if something needs to be done repeatedly? Like a countdown: 5, 4, 3, 2, 1, liftoff!

That calls for a **loop**.

1.18.2.1 `loop` — the Infinite Loop

`loop` just keeps running and running, forever:

```
fn main() {
    loop {
        println!("I can't stooooop");
    }
}
```

If you actually run this program, it will print and print and print... You'll have to force-stop it with `Ctrl + C`.

So we need an "exit."

1.18.2.2 `break` — Jumping Out of the Loop

```
fn main() {
    let mut count = 5;
    loop {
        if count == 0 {
            println!("Liftoff!");
            break;
        }
        println!("{}", count);
        count -= 1;
    }
}
```

1.18.2.3 How Does It Work?

1. `count` starts at 5.
2. Enter the `loop`. First check: is `count == 0`? No, so print 5, then `count -= 1` makes count 4.
3. Back to the top of the `loop`. Is `count == 0`? No, print 4, count becomes 3.
4. ...and so on...
5. When `count` reaches 0, `if count == 0` holds: print "Liftoff!", then `break` out of the loop.
6. The program ends.

1.18.3 Recap

- `loop` repeats the code inside the curly braces forever.
- `break` jumps out of the loop so the program can continue on.
- A `loop` without a `break` is an infinite loop — remember to have an exit.

1.19 `while`

1.19.1 Goal of This Episode

Rewrite the countdown with a `while` loop, and compare it to the `loop` + `break` version.

1.19.2 Main Text

Last episode we built a countdown with `loop` + `break`. Today we'll learn another kind of loop — `while` — which makes the same logic cleaner to write.

1.19.2.1 Rewriting the Countdown with `while`

```
fn main() {
    let mut count = 5;
    while count > 0 {
        println!("{}", count);
        count -= 1;
    }
    println!("Liftoff!");
}
```

Exactly the same result!

1.19.2.2 Comparing with `loop` + `break`

Last episode's version:

```
fn main() {
    let mut count = 5;
    loop {
        if count == 0 {
            println!("Liftoff!");
            break;
        }
        println!("{}", count);
        count -= 1;
    }
}
```

The `while` version:

```
fn main() {
    let mut count = 5;
    while count > 0 {
        println!("{}", count);
        count -= 1;
    }
    println!("Liftoff!");
}
```

See the difference? `while` merges the "condition check" and the "loop" into one. No need to write your own `if` and `break` — just tell `while`: "As long as this condition holds, keep going."

1.19.2.3 `while` in Plain Words

> **While** the condition holds, keep doing what's inside the curly braces.

`while count > 0` → as long as `count` is greater than 0, keep running. Once `count` hits 0, the condition no longer holds, and it stops automatically.

1.19.2.4 When to Use `loop`, and When `while`?

- **`while`**: You know whether to continue before the loop iteration starts (check the condition first, then decide whether to run).
- **`loop` + `break`**: You can only decide whether to stop somewhere in the middle of the loop.

Both get the job done — it's just that `while` is more concise in many situations.

1.19.3 Recap

- A `while` loop keeps running as long as the condition holds, and stops automatically when it doesn't.
- More concise than `loop` + `break`; good for when you know whether to continue before each iteration.
- `loop` + `break` suits cases where the stopping decision happens mid-loop.

1.20 `for` + Ranges

1.20.1 Goal of This Episode

Use a `for` loop together with a range to repeat things, without managing a counter yourself.

1.20.2 Main Text

In the last two episodes we learned `loop` and `while`, both of which required managing a counter by hand (`count -= 1` and so on). Today we'll learn an even simpler way — the `for` loop.

1.20.2.1 `for` + a Range

```
fn main() {
    for i in 0..5 {
        println!("{}", i);
    }
}
```

1.20.2.2 What Is `0..5`?

`0..5` is called a **range**. It means "starting at 0, up to just before 5." Note: **5 is not included**!

So `0..5` gives the five numbers 0, 1, 2, 3, 4.

You can read `for i in 0..5` as: "Let `i` run from 0 to 4 in order, doing what's inside the curly braces each time."

Notice that `i` here doesn't need a `let` — `for` declares it for you automatically. And `i` is usable anywhere inside the curly braces `{}` that follow the `for`.

1.20.2.3 Want to Include the End? Use `0..=5`

```
fn main() {
    for i in 0..=5 {
        println!("{}", i);
    }
}
```

`0..=5` has an extra `=`, meaning "including 5."

1.20.2.4 Comparison

Syntax	Meaning	Numbers produced
`0..5`	0 to 4	0, 1, 2, 3, 4
`0..=5`	0 to 5	0, 1, 2, 3, 4, 5
`1..4`	1 to 3	1, 2, 3
`1..=4`	1 to 4	1, 2, 3, 4

1.20.2.5 `while` and `for` Can Use `break` Too

We learned `break` inside `loop` earlier, but it also works in `while` and `for`. One thing to note: `break` only jumps out of a **loop** — it doesn't jump out of control structures like `if`. So in the

code below, `break` exits the `for` loop, not the `if`:

```
fn main() {
    for i in 0..10 {
        if i == 5 {
            println!("Found 5 — stopping here!");
            break;
        }
        println!("{}", i);
    }
}
```

Running this prints only 0~4; when it hits 5, `break` exits the loop.

1.20.3 Recap

- `for i in 0..5` runs `i` from 0 to 4 (end excluded); `0..=5` includes 5.
- Compared to `while`, `for` doesn't need manual counter increments or condition checks — more concise and less error-prone.
- `break` works inside `loop`, `while`, and `for`.

1.21 Nested Loops

1.21.1 Goal of This Episode

Put a loop inside a loop — use nested loops to print the multiplication table.

1.21.2 Main Text

Last episode we learned the `for` loop. Today we'll try something more advanced — putting one loop **inside another loop**.

1.21.2.1 What Are Nested Loops?

"Nested" means "one layer inside another," like Russian nesting dolls. Each time the outer loop runs once, the inner loop runs all the way through.

1.21.2.2 The Multiplication Table

Let's take on a challenge: print the 9×9 multiplication table with nested loops:

```
fn main() {
    for i in 1..=9 {
        for j in 1..=9 {
            print!("{} x {} = {}   ", i, j, i * j);
        }
        println!(); // New line
    }
}
```

1.21.2.3 How Does It Work?

1. The outer loop runs `i` from 1 to 9.
2. When `i = 1`, the inner loop runs `j` from 1 to 9 → printing 1×1, 1×2, ... 1×9.

3. After the inner loop finishes, `println!()` starts a new line.
4. The outer loop moves to `i = 2`, and the inner loop runs 1 to 9 again → printing 2×1, 2×2, ... 2×9.
5. And so on...

1.21.2.4 `print!` vs `println!`

We used something new here: `print!`. It's a lot like `println!`, except that `print!` **doesn't start a new line** after printing, whereas `println!` does.

1.21.2.5 Visualizing It

One outer-loop iteration = one row:

```
i=1 → [j=1, j=2, j=3, ... j=9] → new line
i=2 → [j=1, j=2, j=3, ... j=9] → new line
...
i=9 → [j=1, j=2, j=3, ... j=9] → new line
```

1.21.2.6 `break` Only Exits the Innermost Layer

Using `break` inside nested loops only exits the **innermost** loop — the outer one keeps going:

```
fn main() {
    for i in 1..=3 {
        for j in 1..=3 {
            if j == 2 {
                break; // Only exits the inner loop
            }
            println!("i={}, j={}", i, j);
        }
    }
}
```

Each time `j` reaches 2 it `break`s, but the outer `i` still runs through 1, 2, 3.

1.21.2.7 Loop Labels: Breaking Out of a Specific Layer

What if you want to jump straight out of the outer loop? Use a **loop label**:

```
fn main() {
    'outer: for i in 1..=3 {
        for j in 1..=3 {
            if j == 2 {
                break 'outer; // Exits the outer loop
            }
            println!("i={}, j={}", i, j);
        }
    }
    println!("Done!");
}
```

`'outer:` is a **label**, placed in front of a loop. `break 'outer` says "exit the loop labeled `'outer`." Note that label names start with `'` (a single quote).

1.21.3 Recap

- Nested loops are loops inside loops: each outer-loop iteration runs the inner loop all the way through.
- The difference between `print!` and `println!`: `print!` doesn't start a new line.
- In nested loops, `break` only exits the innermost loop.
- Use a loop label (`'outer:` + `break 'outer`) to break out of a specific outer loop.

1.22 `continue`

1.22.1 Goal of This Episode

Use `continue` to skip certain iterations of a loop.

1.22.2 Main Text

Earlier we learned that `break` jumps out of a loop. Today it's `continue` — which doesn't exit the loop, but **skips the current iteration and goes straight to the next one**.

1.22.2.1 Printing Only Odd Numbers

```
fn main() {
    for i in 0..10 {
        if i % 2 == 0 {
            continue;
        }
        println!("{}", i);
    }
}
```

1.22.2.2 How Does It Work?

The loop runs `i` from 0 to 9:

- `i = 0` → Is `0 % 2 == 0`? Yes (even), `continue`! Skip it, don't print.
- `i = 1` → Is `1 % 2 == 0`? No (odd), keep going, print 1.
- `i = 2` → Even, `continue`, skip.
- `i = 3` → Odd, print 3.
- ...and so on.

1.22.2.3 `break` vs `continue`

- **`break`**: The whole loop ends; no more iterations.
- **`continue`**: Skip this iteration, but the loop continues with the next one.

Like `break`, `continue` only acts on **loops** — it doesn't skip control structures like `if`. In the code above, `continue` skips the current iteration of the `for` loop, not the `if`.

1.22.2.4 Another Example

Skip 5 and don't print it:

```
fn main() {
    for i in 1..=10 {
```

```
        if i == 5 {
            continue;
        }
        println!("{}", i);
    }
}
```

5 gets skipped; everything else prints normally.

1.22.2.5 `continue` + Loop Labels

Last episode we learned that `break 'outer` can exit a specific loop layer. `continue` works with labels too:

```
fn main() {
    'outer: for i in 1..=3 {
        for j in 1..=3 {
            if j == 2 {
                continue 'outer; // Skip to the outer loop's next iteration
            }
            println!("i={}, j={}", i, j);
        }
    }
}
```

Each time `j` reaches 2, `continue 'outer` jumps straight to the outer loop's next iteration, so `j=2` and `j=3` never get printed.

1.22.3 Recap

- `continue` skips the current iteration and goes straight to the next one.
- `break` means "stop the whole loop"; `continue` means "skip the current iteration and run the next one."
- Pair it with `if` to selectively skip particular cases.
- `continue 'outer` works with loop labels to skip the current iteration of an outer loop.

1.23 Types (the Basics)

1.23.1 Goal of This Episode

Meet Rust's basic types.

1.23.2 Main Text

Until now, when we wrote `let x = 5;` we never said anything about what "type" x is. Today let's formally meet the concept of types.

1.23.2.1 What Is a Type?

A type tells Rust: "Here's what kind of thing this variable holds."

Is it an integer? A decimal? Text? Or `true` / `false`? Different types represent different kinds of data.

1.23.2.2 Annotating Types by Hand

You can specify a type by adding `: type` after the variable name:

```
fn main() {
    let x: i32 = 5;
    let negative: i32 = -10;
    let y: f64 = 3.14;
    let z: bool = true;

    println!("x = {}", x);
    println!("negative = {}", negative);
    println!("y = {}", y);
    println!("z = {}", z);
}
```

- `i32` → an integer (32-bit).
- `f64` → a floating-point number (64-bit), used for values that can have a fractional part, such as `3.14` or `0.5`.
- `bool` → a boolean, which is only ever `true` or `false`.

1.23.2.3 Then Why Didn't We Annotate Before?

Because Rust is smart! It looks at the value you provide and **infers** the type automatically:

```
    let x = 5;    // Rust figures out: this is an i32
    let y = 3.14; // Rust figures out: this is an f64
    let z = true; // Rust figures out: this is a bool
```

This is called **type inference**. Most of the time, Rust can work it out on its own and you don't need to annotate.

1.23.3 Recap

- Three basic types: `i32` (integer), `f64` (floating-point number), `bool` (boolean).
- Rust has **type inference**; most of the time you don't need to annotate types by hand.
- When needed, specify a type manually with `let x: i32 = 5;`.

1.24 Types (Numbers in Detail)

1.24.1 Goal of This Episode

Get to know all of Rust's numeric types, plus how numeric suffixes work.

1.24.2 Main Text

Last episode we briefly met `i32` and `f64`. Today let's go through all of Rust's numeric types.

1.24.2.1 Integer Types

Rust's integer types come in **signed** (can be negative) and **unsigned** (only positive and zero) flavors:

Signed	Unsigned	Bits	Range (signed)
`i8`	`u8`	8	-128 ~ 127
`i16`	`u16`	16	-32,768 ~ 32,767
`i32`	`u32`	32	roughly ±2.1 billion
`i64`	`u64`	64	enormous
`i128`	`u128`	128	astronomical
`isize`	`usize`	system-dependent	64 bits on a 64-bit system

- `i` = integer, `u` = unsigned.
- The number says how many bits are used for storage — more bits means bigger numbers can be stored.
- The size of `isize` and `usize` depends on whether your system is 32-bit or 64-bit (nearly everything is 64-bit these days).

For everyday use, `i32` is enough. When in doubt, use `i32`.

1.24.2.2 Floating-point Types

Floating-point numbers are used for values that can have a fractional part. Rust has two floating-point types:

Type	Precision
`f32`	single precision (about 7 significant digits)
`f64`	double precision (about 15 significant digits)

For everyday use, `f64` is enough. It's also Rust's default floating-point type.

1.24.2.3 Floating-point Arithmetic

Back in Episode 5, when we covered arithmetic, we used integers throughout. Floating-point numbers work with `+ - * / %` too, but there's one important difference — **floating-point division can produce a fractional result**:

```
fn main() {
    let a = 10.0;
    let b = 3.0;
    println!("{}", a / b); // 3.3333333333333335
    println!("{}", a % b); // 1
}
```

Remember how `10 / 3` in Episode 5 gave `3` (integer division truncates)? Floating-point doesn't truncate: `10.0 / 3.0` gives `3.3333...`.

Also note that `a % b` prints as `1`, not `1.0`. Don't be fooled — its type is still `f64`; it's just that when `{}` prints a float whose fractional part is zero, it doesn't tack on a `.0`.

That said, floating-point has one classic pitfall — **precision issues**:

```
fn main() {
    println!("{}", 0.1 + 0.2); // 0.30000000000000004
}
```

`0.1 + 0.2` is not `0.3`! This isn't a bug in Rust — it's a floating-point precision limitation found in almost every programming language. Computers store decimals in binary, and some decimal fractions simply can't be represented exactly. Just knowing this exists is enough; don't worry about it too much.

1.24.2.4 How Does Rust Infer Numeric Types?

When you write `let x = 5;`, Rust treats it as `i32` by default. When you write `let y = 3.14;`, Rust treats it as `f64` by default.

But Rust doesn't just look at the number itself — it also infers the type from **how you use the variable**. Sometimes, based on context, Rust will infer an integer type other than `i32`. This will become clearer when we run into it later.

Fundamentally though: integers default to `i32`, and floating-point numbers default to `f64`.

1.24.2.5 Numeric Suffixes (Literal Suffixes)

If you want to specify a type, besides `let x: i64 = 5;` there's an even more concise way — append the type name directly to the number:

```
fn main() {
    let a = 5i32;      // i32
    let b = 5u8;       // u8
    let c = 3.14f64;   // f64
    let d = 2.0f32;    // f32
    let e = 100000i64; // i64

    println!("{} {} {} {} {}", a, b, c, d, e);
}
```

`5i32` means "the number 5, with type `i32`." No space between the number and the type — they're joined directly.

1.24.2.6 A Small Reminder

Numbers of different types **can't be mixed in arithmetic directly**:

```
fn main() {
    let a: i32 = 5;
    let b: i64 = 10;
    println!("{}", a + b); // ✕ Compile error! i32 and i64 can't be added directly
}
```

This is a safety-minded design: Rust generally doesn't convert types for you automatically.

1.24.3 Recap

- Integers come in signed (`i8`-`i128`) and unsigned (`u8`-`u128`); `i32` is enough for everyday use.
- The size of `isize` and `usize` depends on the system (64 bits on 64-bit systems).
- Floating-point types are `f32` and `f64`; use `f64` day-to-day (Rust's default).
- Numeric suffixes (like `5i32`, `3.14f64`) specify the type directly.
- Floating-point division can produce fractional results, but has precision issues (`0.1 + 0.2`

`≠ 0.3`).

- Rust generally doesn't convert types for you automatically.

1.25 `char`

1.25.1 Goal of This Episode

Meet the `char` type — the type for holding "a single character."

1.25.2 Main Text

We've used strings before (text wrapped in double quotes `"`). Today let's meet a smaller unit — the **character** (`char`).

1.25.2.1 What Is a `char`?

A `char` is **one character**. Note: "one," not a string of them.

```
fn main() {
    let c = 'A';
    let c2 = '你';
    let c3 = '🦀';

    println!("{}", c);
    println!("{}", c2);
    println!("{}", c3);
}
```

1.25.2.2 Single Quotes vs Double Quotes

This matters:

- **Single quotes** `'` → `char`; holds exactly **one** character.
- **Double quotes** `"` → a string; can hold many characters.

```
fn main() {
    let c = 'A';     // char, one character
    let s = "Hello"; // string, five characters
}
```

If you put more than one character inside single quotes, Rust reports an error:

```
    let c = 'AB'; // ✕ Error! A char can hold only one character
```

1.25.2.3 Unicode

Rust's `char` supports **Unicode**, so it's not just English letters — Chinese, Japanese, even emoji all work:

```
fn main() {
    let letter = 'R';
    let chinese = '美';
    let japanese = 'の';
    let emoji = '😊';

    println!("{} {} {} {}", letter, chinese, japanese, emoji);
```

```
}
```

Each of these is a legal `char`.

1.25.2.4 Type Annotation

If you want to annotate the type explicitly:

```
fn main() {
    let c: char = 'Z';
    println!("{}", c);
}
```

Usually there's no need, though — Rust sees single quotes and knows it's a `char`.

1.25.3 Recap

- `char` is the "one character" type, wrapped in **single quotes**: `'A'`, `'你'`, `'🦀'`.
- It supports Unicode: Chinese, Japanese, and emoji are all legal `char`s.
- Single quote `'` = `char` (one character); double quote `"` = string (a sequence of characters). Don't mix them up.

1.26 Escape Characters

1.26.1 Goal of This Episode

Learn to use the backslash \ to insert newlines, tabs, and other special characters into strings.

1.26.2 Main Text

Sometimes you want to put something "special" in a string — a newline, a tab, or a double quote itself. That's when you need **escape characters**.

1.26.2.1 `\n` — Newline

```
fn main() {
    println!("First line\nSecond line");
}
```

`\n` tells Rust: "Break the line here." It doesn't literally print the two characters `\n` — it produces an actual line break.

1.26.2.2 `\t` — Tab

```
fn main() {
    println!("Name\tScore");
    println!("Ming\t85");
    println!("Hua\t92");
}
```

`\t` inserts a tab space.

1.26.2.3 \\ — the Backslash Itself

What if you want to print the backslash \ itself? Since \ is already taken as the start of escape sequences, you need two backslashes:

```
fn main() {
    println!("File path: C:\\Users\\Andy");
}
```

1.26.2.4 \" — Double Quote

Strings are wrapped in ", so what if the string needs a " inside it?

```
fn main() {
    println!("He said: \"Hello!\"");
}
```

\" tells Rust: "This double quote is part of the string's content, not the end of the string."

1.26.2.5 Using Them in a `char`

Escape characters work inside a `char` too:

```
fn main() {
    let newline: char = '\n';
    let tab: char = '\t';
    let backslash: char = '\\';

    print!("A{}B{}C{}", newline, tab, backslash);
}
```

1.26.2.6 \' — Single Quote

Inside a `char`, if you want to represent the single quote itself, you have to escape it:

```
fn main() {
    let quote: char = '\'';
    println!("{}", quote);
}
```

Because a `char` is wrapped in ', putting a ' inside requires \'.

1.26.2.7 When You Don't Need to Escape

Inside a string (""), single quotes don't need escaping — use them directly:

```
fn main() {
    println!("It's a test"); // ' needs no escaping inside a string
}
```

Likewise, inside a `char` (''), double quotes don't need escaping:

```
fn main() {
    let c: char = '"'; // " needs no escaping inside a char
    println!("{}", c);
}
```

In short: **only the symbol doing the wrapping needs escaping; the other one doesn't.**

1.26.2.8 At a Glance

Escape	Effect
`\n`	newline
`\t`	tab
`\\`	backslash \
`\"`	double quote "
`\'`	single quote '

1.26.3 Recap

- Escape characters start with `\` and stand for special characters: `\n` (newline), `\t` (tab), `\\` (backslash).
- `\"` represents a double quote inside a string; `\'` represents a single quote inside a `char`.
- Escape characters work in both strings and `char`s.
- Rule of thumb: only the wrapping symbol needs escaping; the other one doesn't.

1.27 `if` as an Expression

1.27.1 Goal of This Episode

Learn to treat `if` as an "expression" and use it directly to assign a value to a variable.

1.27.2 Main Text

This is the last episode of Chapter 1! Today I'll introduce one of Rust's cool features — `if` isn't just for making decisions; it can also **return a value**.

1.27.2.1 First, the Usual Way

Suppose you want to give a variable different values based on a condition. You might write:

```
fn main() {
    let condition = true;
    let x;

    if condition {
        x = 1;
    } else {
        x = 2;
    }

    println!("{}", x);
}
```

Nothing wrong with that — but Rust has a more concise way.

1.27.2.2 `if` as an Expression

```
fn main() {
    let condition = true;
    let x = if condition { 1 } else { 2 };
```

```
    println!("{}", x);
}
```

Running it prints 1.

See that? The whole `if condition { 1 } else { 2 }` sits on the right side of `let x =`, assigning its result directly to x.

If `condition` is `true`, x is 1; if it's `false`, x is 2.

1.27.2.3 Note: No Semicolons Inside the Braces

```
fn main() {
    let condition = true;
    let x = if condition { 1 } else { 2 };
    //                     ^          ^
    //                no semicolon  no semicolon
}
```

Those values (1 and 2) have **no semicolon** after them. In Rust, a value without a semicolon is the "return value." This is Rust's expression syntax — we'll go into more detail when we learn about functions.

1.27.2.4 Both Sides Must Have the Same Type!

```
fn main() {
    let condition = true;
    let x = if condition { 1 } else { "hello" }; // ✕ Error!
}
```

This fails, because 1 is an integer and `"hello"` is a string. Rust won't allow x to be sometimes a number and sometimes a string — it needs one definite type.

The values inside the two sets of braces **must have the same type**:

```
    // ✓ Both sides are integers
    let x = if condition { 1 } else { 2 };

    // ✓ Both sides are strings
    let msg = if condition { "good" } else { "bad" };

    // ✕ One side an integer, the other a string
    let bad = if condition { 1 } else { "hello" };
```

1.27.2.5 What's the Benefit?

1. x doesn't need to be `mut`.
2. The code is more concise.
3. It reflects Rust's design philosophy: many things are "expressions" that can return values.

1.27.3 Recap

- In Rust, `if` is an **expression** and can return a value directly: `let x = if condition { 1 } else { 2 };`.

- The part inside the braces that serves as the return value takes **no semicolon**.
- The `if` and `else` sides must have matching types.

Congratulations on finishing Chapter 1! 🎉 You've learned Rust's basic syntax: variables, arithmetic, conditionals, loops, types, and more. In the next chapter, we'll start learning more of Rust's distinctive features!

Chapter 2

Functions, Arrays, and Slices

In this chapter, we'll learn how to modularize a program in the most basic way. Beyond that, you'll also learn how to work with multiple values at once — storing them in hard-coded form in your program and viewing some of the values within.

2.1 `const`

2.1.1 Goal of This Episode

Declare a never-changing constant with `const`, and understand how it differs from `let`.

2.1.2 Main Text

In Chapter 1 we learned to declare variables with `let`. Today let's meet its good friend — `const`.

`const` means "constant," as in: **this value never changes, and it's already determined at compile time.**

Here's the syntax:

```
fn main() {
    const MAX_SCORE: i32 = 100;
    println!("The highest score is: {}", MAX_SCORE);
}
```

Looks a lot like `let`, right? But there are several important differences:

2.1.2.1 Difference 1: `const` Must Have a Type Annotation

```
fn main() {
    const MAX_SCORE: i32 = 100; // ✅ The : i32 is required
    let max_score = 100;        // ✅ let can omit it; the compiler infers it
}
```

With `const`, you can't lazily skip the type — the compiler will complain.

2.1.2.2 Difference 2: The Naming Convention Is ALL CAPS with Underscores

```
fn main() {
    const MAX_SCORE: i32 = 100;    // ✅ All caps, separated by underscores
    const PI_VALUE: f64 = 3.14159; // ✅ Like this
    const maxScore: i32 = 100;     // ⚠️ Compiles, but the compiler will warn you
}
```

This is the Rust community convention: constants use `SCREAMING_SNAKE_CASE`. Your program still runs if you don't follow it, but the compiler will grumble.

2.1.2.3 Difference 3: `const` Can't Take `mut`

```
    const mut MAX: i32 = 100; // ✕ No such thing exists
    let mut x = 5;            // ✓ This is fine
```

A constant is a constant — unchangeable means unchangeable. There's no such contradiction as a "mutable constant."

2.1.2.4 Difference 4: `const` Can Live Outside `fn`

```
const MAX_PLAYERS: i32 = 10;

fn main() {
    println!("At most {} players", MAX_PLAYERS);
}
```

`const` can be declared at the outermost level of a program; `let` cannot.

2.1.2.5 When to Use `const`?

When you have a value that's **fixed and unchanging**, and you already know what it is while writing the program, use `const`. For example:

```
const TAX_RATE: f64 = 0.05;
const MAX_RETRY: i32 = 3;
```

2.1.3 Recap

- `const` declares a compile-time constant whose value never changes.
- The type annotation is mandatory (can't be omitted).
- The naming convention is all caps with underscores, like `MAX_SCORE`.
- No `mut` allowed.
- It can be placed outside `fn` so other parts of the program can use it.

2.2 Shadowing

2.2.1 Goal of This Episode

Re-declare a variable of the same name with `let` (shadowing), and see the key difference from `mut`.

2.2.2 Main Text

Rust has a really interesting feature called **shadowing**. In short: you can use `let` to declare a variable with the same name again, and the new one "covers up" the old one.

```
fn main() {
    let x = 5;
    let x = x + 1;
    println!("x = {}", x);
}
```

The second line, `let x = x + 1;`, is really saying: "I want to create a **brand-new** x, whose

value is the old x plus 1." The old x gets covered up, and from then on x is 6.

You can even shadow several times in a row:

```
fn main() {
    let x = 1;
    let x = x + 1; // x = 2
    let x = x * 3; // x = 6
    println!("x = {}", x);
}
```

2.2.2.1 Shadowing vs `mut`: The Biggest Difference

"Wait — how is this different from `mut`? Aren't both just changing the value?"

The biggest difference: **shadowing can change the type; `mut` can't.**

```
fn main() {
    // Shadowing: can go from a number to a string
    let x = 5;
    let x = "hello";
    println!("x = {}", x);
}
```

Perfectly legal! Because the second `let x` is a **brand-new variable** that just happens to share the name.

But try it with `mut`:

```
fn main() {
    let mut x = 5;
    x = "hello"; // ✕ Compile error! Can't stuff a string into an i32
}
```

`mut` only lets you change the "value" — the type stays locked. Shadowing creates an entirely new variable, so the type can be completely different.

2.2.2.2 Practical Use

The most common use of shadowing is "converting the type while keeping the name":

```
fn main() {
    let input = "42"; // This is a string
    let input = input.trim().parse::<i32>().expect("not a number"); // now i32, same name
    println!("input + 1 = {}", input + 1);
}
```

Without shadowing, you'd have to invent two different names, like `input_str` and `input_num` — a bit long-winded.

2.2.2.3 Shadowing and Scope

Remember scopes from Chapter 1, Episode 9? Shadowing works inside curly braces {} too — and once you leave the braces, the shadowing ends and the old variable "comes back":

```
fn main() {
    let x = 1;
    {
```

```
        let x = 2; // From this line on inside the block, x is shadowed as 2
        println!("Inside the block, x = {}", x); // 2
    }
    println!("Outside the block, x = {}", x);     // 1
}
```

The `let x = 2;` inside the braces creates a new x, shadowing the outer x. But this shadowing is only effective inside the braces — the moment you leave them, the new x vanishes and the original x (with value 1) is usable again.

This is completely different from `mut`. If you change the value in a block with `mut`, the value really has changed once you leave the block:

```
fn main() {
    let mut x = 1;
    {
        x = 2; // Directly changing the value; not shadowing
    }
    println!("x = {}", x); // 2
}
```

So, once more with emphasis: shadowing **creates a new variable**, while `mut` **changes the old variable's value**. The difference is especially clear where scopes are involved.

2.2.3 Recap

- Re-declaring a variable of the same name with `let` is called shadowing.
- The new variable covers up the old one.
- The biggest difference from `mut`: **shadowing can change the type**.
- In reality, each `let` creates an entirely new variable — the names just match.
- A variable shadowed inside braces vanishes once you exit them, and the original "comes back."

2.3 Underscore Variables

2.3.1 Goal of This Episode

Use variable names starting with an underscore _ to tell the compiler "I know this is unused — stop nagging me."

2.3.2 Main Text

Rust's compiler is very considerate (sometimes a bit annoying). If you declare a variable but never use it, it gives you a warning:

```
fn main() {
    let x = 5;
    // x is never used
}
```

The program still runs, but that yellow warning is unpleasant to look at. How do we get rid of it?

2.3.2.1 Method 1: Add an Underscore Prefix

Put an underscore _ in front of the variable name:

```
fn main() {
    let _x = 5;
    // _x is never used, but the compiler no longer warns
}
```

Now the compiler understands: "Oh, you're leaving it unused on purpose. Fine."

Note that _x is still a normal variable — you can still use it if you want:

```
fn main() {
    let _x = 5;
    println!("{}", _x);  // Still usable
}
```

2.3.2.2 Method 2: A Lone Underscore _

If you don't even want to bother naming it, just use a single underscore:

```
fn main() {
    let _ = 42;
}
```

This means "I don't care about this value at all." It has no name, so you can't use it afterward either.

2.3.2.3 The Difference between _x and _

- _x: has a name; the value is kept and can be used later.
- _: does not give the value a name; you cannot use _ to access it later.

In most situations either one works.

2.3.2.4 Underscores Work in `for` Loops Too

Last chapter we learned `for i in 0..5`, where the loop variable `i` takes the values 0, 1, 2, 3, 4 in turn. But what if you just want to do something five times and don't care which iteration you're on? That's when _ comes in:

```
fn main() {
    for _ in 0..5 {
        println!("Five times!");
    }
}
```

`for _ in 0..5` means "run five times, but I don't need to know which round it currently is."

2.3.2.5 A Practical Use: Guess the Number

Here's a slightly more complete example — challenge the player to guess a number within five tries:

```
fn main() {
    let secret = 67;
    let mut success = false;
```

```
    println!("Guess a number from 1~100:");

    for _ in 0..5 {
        let mut input = String::new();
        std::io::stdin().read_line(&mut input).expect("failed to read input");
        let guess = input.trim().parse::<i32>().expect("not a number");

        if guess == secret {
            success = true;
            break;
        }

        println!("Not it......");
    }

    if success {
        println!("Congratulations, you guessed it within five tries!");
    } else {
        println!("Five tries and no luck......");
    }
}
```

If you guess right:

```
Guess a number from 1~100:
50
Not it......
70
Not it......
67
Congratulations, you guessed it within five tries!
```

If all five guesses miss:

```
Guess a number from 1~100:
50
Not it......
75
Not it......
60
Not it......
80
Not it......
90
Not it......
Five tries and no luck......
```

Here `for _ in 0..5` means "at most five guesses." We don't need to know which attempt we're on — we just need the loop to run five times. On a correct guess, we set `success` to `true` and `break` out of the loop.

2.3.3 Recap

- Rust's compiler warns you about unused variables.
- Prefixing the name with _ (like _x) silences the warning.

- A lone _ means “I don’t care about this value at all.”
- _x can still be used; _ cannot.
- When you don’t need the loop variable, `for _ in 0..5` simply repeats five times.

2.4 Tuples

2.4.1 Goal of This Episode

Use a tuple to bundle several values of different types into one, and learn how to get the values back out.

2.4.2 Main Text

So far, one variable has held one value. But what if I want to tie together “an integer, a decimal, and a boolean”? That’s what a **tuple** is for.

2.4.2.1 Creating a Tuple

```
fn main() {
    let t = (1, 3.14, true);
    println!("{}", t.0); // 1
    println!("{}", t.1); // 3.14
    println!("{}", t.2); // true
}
```

Wrap the values in parentheses `()`, separate them with commas, and you’ve got a tuple.

To get values out, use **a dot plus an index**: `t.0`, `t.1`, `t.2`. Note that indexing starts at 0!

2.4.2.2 The Unit Type — the Empty Tuple

Rust has one special tuple with nothing inside:

```
fn main() {
    let _u: () = ();
}
```

This `()` is called the **unit type**. It’s “a type with only one value” — and that value is also written `()`.

2.4.2.3 Annotating the Type

If you want to spell out a tuple’s type explicitly:

```
fn main() {
    let t: (i32, f64, bool) = (1, 3.14, true);
    println!("{} {} {}", t.0, t.1, t.2);
}
```

Each position’s type must correspond.

2.4.2.4 Single-element Tuples — Don’t Forget the Comma!

If you want a tuple with just one element, remember the comma:

```
fn main() {
    let not_a_tuple = (5);      // This is just the number 5, wrapped in parentheses
    let a_tuple = (5,);         // THIS is a tuple! Note the comma

    println!("{}", a_tuple.0); // 5
}
```

`(5)` is merely a number in parentheses, not a tuple. `(5,)` is. That comma matters!

The same goes for the type:

```
fn main() {
    let t: (i32,) = (5,);
    println!("{}", t.0);
}
```

`(i32)` is just `i32` in parentheses; `(i32,)` is the type of a single-element tuple.

2.4.2.5 Modifying Values inside a Tuple

If the tuple is declared with `let mut`, you can modify the values inside:

```
fn main() {
    let mut t = (1, 2, 3, 4);
    println!("Before: {}", t.1); // 2
    t.1 = 99;
    println!("After: {}", t.1); // 99
}
```

Same as any other `mut` variable — no `mut`, no changes.

2.4.3 Recap

- A tuple packs values of different types together with `()`.
- Get values with `t.0`, `t.1`, `t.2`... (indexing starts at 0).
- `()` is the unit type, representing "no meaningful value."
- Single-element tuples need the comma: `(5,)` is a tuple; `(5)` is just a number.
- A `let mut` tuple allows modifying inner values with `t.0 = new_value`.

2.5 {:?} and the Debug Format

2.5.1 Goal of This Episode

Use `{:?}` to print tuples and other things that "can't be printed with `{}`."

2.5.2 Main Text

So far we've printed everything with `{}`:

```
fn main() {
    let x = 42;
    println!("{}", x); // ✅ 42
}
```

Numbers, `bool`s, and other basic types work fine with `{}`. But try printing a tuple with `{}`:

```
fn main() {
    let t = (1, 2, 3);
    println!("{}", t); // ✕ Compile error!
}
```

The compiler spits out a pile of error messages. In short: "This type doesn't implement `Display` — I don't know how to print it in a 'nice-looking way.'"

2.5.2.1 The Fix: Use `{:?}`

```
fn main() {
    let t = (1, 2, 3);
    println!("{:?}", t); // ✓ (1, 2, 3)
}
```

`{:?}` is the **`Debug` format**. It's not a "pretty format" for end users — it's a "debugging format" for developers.

2.5.2.2 `Display` `{}` vs `Debug` `{:?}`

For simple types like numbers and `bools`, `{}` and `{:?}` print much the same thing. So what's the point of `{:?}`?

It can print things `{}` can't — like tuples. Tuples only have a `Debug` format, not a `Display` format.

2.5.2.3 The Pretty Version: `{:#?}`

If the data is complex (say, tuples nested in tuples), you can use `{:#?}` to print a "prettified `Debug` format":

```
fn main() {
    let data = ((1, 2), (3, 4), (5, 6));
    println!("{:#?}", data);
}
```

2.5.2.4 A Handy Trick: The `dbg!` Macro

Rust also has a very convenient debugging tool, `dbg!`:

```
fn main() {
    let x = 5;
    dbg!(x);
    dbg!(x + 1);
}
```

It prints the file name, the line and column, and the value — super convenient:

```
[src/main.rs:3:5] x = 5
[src/main.rs:4:5] x + 1 = 6
```

2.5.3 Recap

- `{}` is the `Display` format, meant for end users, but not every type supports it.
- `{:?}` is the `Debug` format, meant for developers; compound types like tuples support it.
- `{:#?}` is the prettified `Debug` format — clearer for complex data.

- `dbg!` is a great helper for quick debugging; it prints the file name plus the line and column.

2.6 Simple Functions

2.6.1 Goal of This Episode

Define your own function with `fn` and call it from `main`.

2.6.2 Main Text

Up to now, almost all our code has lived inside `main`. But as programs grow, cramming everything together gets messy. That's when we can "package" a piece of code into a **function** and just call it whenever we need it.

2.6.2.1 Defining a Function

```
fn greet() {
    println!("Hello! Welcome to the world of Rust!");
}

fn main() {
    greet();
}
```

Breaking down the syntax:

- `fn` → tells Rust "I'm defining a function."
- `greet` → the function's name.
- `()` → the parameter list (empty for now; next episode covers this).
- `{ ... }` → what the function does.

Then writing `greet();` inside `main` calls it.

2.6.2.2 Functions Can Be Called Many Times

```
fn greet() {
    println!("Hi there!");
}

fn main() {
    greet();
    greet();
    greet();
}
```

That's the beauty of functions — write once, use many times.

2.6.2.3 Functions Can Call Each Other

It's not just `main` that can call functions — functions can call one another. `main` is merely the program's **entry point** (where execution begins), but functions it calls can call other functions in turn:

```rust
fn say_name() {
    println!("I'm Rust!");
}

fn greet() {
    say_name();
}

fn main() {
    greet(); // main calls greet, and greet calls say_name
}
```

2.6.2.4 Where to Define Functions: Above or Below, Both Fine

In some languages, a function must be defined before it's used. Not in Rust!

```rust
fn main() {
    greet(); // ✅ Called first
}

fn greet() { // Defined later
    println!("Hello!");
}
```

Totally fine. The Rust compiler scans the whole file first, so whether you put a function above or below `main`, it will be found.

2.6.2.5 Function Naming Convention

Rust function names, like variables created with `let`, use **snake_case**: all lowercase, with words separated by underscores `_`.

```rust
fn say_hello() { // ✅ snake_case
    println!("Hello!");
}

fn sayHello() {  // ⚠ Runs, but the compiler warns
    println!("Hello!");
}
```

2.6.3 Recap

- Define a function with `fn name() { ... }`.
- Call a function with `name();`.
- `main` is the program's entry point, but functions can call each other.
- Function definitions can go above or below `main`.
- The naming convention is snake_case (all lowercase with underscores).

2.7 Function Parameters

2.7.1 Goal of This Episode

Add parameters to a function so it can receive data passed in from outside.

2.7.2 Main Text

Last episode's `greet` function could only ever print the same thing — a bit boring. If we want functions to be more flexible — say, "give me two numbers and I'll add them for you" — we need **parameters**.

2.7.2.1 Adding Parameters

```rust
fn add(a: i32, b: i32) {
    println!("{} + {} = {}", a, b, a + b);
}

fn main() {
    add(3, 4);
    add(10, 20);
}
```

Syntax breakdown:

- `a: i32` → the first parameter is named `a`, with type `i32`.
- `b: i32` → the second parameter is named `b`, also of type `i32`.
- Parameters are separated by commas.

When calling, `add(3, 4)` passes 3 to `a` and 4 to `b`.

2.7.2.2 Parameters Must Have Type Annotations

In Rust, function parameters **must be annotated with types** — no slacking:

```rust
fn add_v1(a, b) {           // ✕ Compile error! No type annotations
    println!("{}", a + b);
}

fn add_v2(a: i32, b: i32) { // ✓ Required
    println!("{}", a + b);
}
```

"But doesn't `let x = 5;` get to skip the annotation?"

True — `let` lets the compiler infer the type. But function parameters must have type annotations: a function's definition must be clear, and Rust checks the code inside the function using those declared types.

2.7.2.3 Multiple Parameters, Different Types

Parameters can have different types:

```rust
fn describe(x: i32, is_positive: bool) {
    println!("Is {} positive? {}", x, is_positive);
}

fn main() {
    describe(5, true);
    describe(-3, false);
}
```

2.7.2.4 One Parameter Works Too

```
fn double(x: i32) {
    println!("Twice {} is {}", x, x * 2);
}

fn main() {
    double(5);
    double(100);
}
```

2.7.3 Recap

- Function parameters go inside the parentheses: `fn name(param: type)`.
- Multiple parameters are separated by commas.
- **Parameters must have type annotations** — a hard rule in Rust.
- When calling, just pass in the corresponding values.

2.8 Function Return Values

2.8.1 Goal of This Episode

Make a function return a value, and learn Rust's distinctive "no semicolon means return value" style.

2.8.2 Main Text

Last episode's functions just printed their results. But often what we want is: "Once you've computed it, **hand the answer back** — I'll decide what to do with it."

2.8.2.1 Basic Syntax

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    let result = add(3, 4);
    println!("3 + 4 = {}", result);
}
```

The key points:

1. `-> i32` after the parameters tells Rust "this function returns an `i32`." The `i32` here is the function's **return type**.
2. The function's last line, `a + b`, has **no semicolon** → that's the return value.

The declaration `fn add(a: i32, b: i32) -> i32` is called the function's **signature**. It describes the function's name, parameters and their types, and return type. The part inside the braces `{ ... }` is the **function body**, which is not part of the function signature.

2.8.2.2 No Semicolon = Return Value

This is one of Rust's most distinctive designs. If the last line of a function has **no semicolon**, its value automatically becomes the return value:

```
fn double(x: i32) -> i32 {
    x * 2 // ✅ No semicolon — this is the return value
}
```

2.8.2.3 What If You Add a Semicolon?

If you accidentally add one:

```
fn double(x: i32) -> i32 {
    x * 2; // ❌ Semicolon added
}
```

The compiler reports an error. Why? With the semicolon, the result of `x * 2` gets thrown away, and the function ends without leaving any meaningful value. In that case, what's actually returned is `()` (the unit type — remember Episode 4?). But you promised to return an `i32`; the types don't match, so the compiler complains.

2.8.2.4 Functions with No Declared Return Value

Look back at the `greet` function from Episode 6 of this chapter — it has no `->` return type:

```
fn greet() {
    println!("Hello!");
}
```

In Rust, every function has a return value. Omitting `->` is the same as writing `-> ()`:

```
fn greet() -> () {
    println!("Hello!");
}
```

It's just that `-> ()` is usually left out. `println!("Hello!");` ends with a semicolon, its result is discarded, and the function returns `()` — exactly matching the declaration.

2.8.2.5 Catching the Return Value

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    let result = add(3, 4);
    println!("Result: {}", result);

    // You can also use it directly inside an expression
    println!("Plus 10 more: {}", add(3, 4) + 10);
}
```

2.8.2.6 Returning Multiple Values with a Tuple

A function can only return "one" value — so what if you want to return several? Just pack them in a tuple:

```
fn swap(a: i32, b: i32) -> (i32, i32) {
    (b, a)
}

fn main() {
    let result = swap(1, 2);
    println!("First: {}, second: {}", result.0, result.1);
}
```

`-> (i32, i32)` means returning a tuple containing two `i32`s. After the call, use `.0` and `.1` to pull the values out.

Here's a more practical example:

```
fn min_max(a: i32, b: i32) -> (i32, i32) {
    if a < b {
        (a, b)
    } else {
        (b, a)
    }
}

fn main() {
    let result = min_max(7, 3);
    println!("Smallest: {}, largest: {}", result.0, result.1);
}
```

2.8.3 Recap

- Declare a function's return type with `-> type`.
- A function signature describes the function's name, parameters and their types, and return type; the part inside the braces `{ ... }` is the function body.
- The last line **without a semicolon** is the return value (the idiomatic Rust style).
- With a semicolon, it becomes an ordinary statement, and `()` gets returned instead.
- A function without a declared return type actually returns `()`.
- Want to return multiple values? Wrap them in a tuple: `-> (i32, i32)`, retrieved with `.0` and `.1`.

2.9 Early `return`

2.9.1 Goal of This Episode

Use the `return` keyword to send a value back partway through a function, without waiting for the last line.

2.9.2 Main Text

Last episode we learned that a function's final line without a semicolon is the return value. But sometimes you want to return **in the middle** of a function — bailing out early when some condition is met. That's what the `return` keyword is for.

2.9.2.1 Basic Example: Absolute Value

```
fn abs(x: i32) -> i32 {
    if x >= 0 {
        return x; // If x is positive or zero, return immediately
    }
    -x            // Reaching here means x is negative; return -x
}

fn main() {
    println!("abs(5) = {}", abs(5));
    println!("abs(-3) = {}", abs(-3));
    println!("abs(0) = {}", abs(0));
}
```

Look closely:

- `return x;` → uses the `return` keyword.
- The last line `-x` → no semicolon; that's the "natural return."

2.9.2.2 `return` vs No Semicolon

Comparing the two ways to return:

```
// Way 1: using return (usually for "leaving early")
fn abs_v1(x: i32) -> i32 {
    if x >= 0 {
        return x;
    }
    -x
}

// Way 2: pure expressions (the whole if-else is the return value)
fn abs_v2(x: i32) -> i32 {
    if x >= 0 {
        x
    } else {
        -x
    }
}
```

Both are correct! The Rust community's convention:

- **Use `return` when you mean "I want to leave early"** (Way 1).
- **Use expressions the rest of the time** (Way 2).

2.9.2.3 A Practical Scenario: Blocking Invalid Input Early

```
fn divide(a: f64, b: f64) -> f64 {
    if b == 0.0 {
        println!("Error: can't divide by zero!");
        return 0.0; // Leave early
    }

    // Possibly lots of other work here......
```

```
    a / b
}

fn main() {
    println!("{}", divide(10.0, 3.0));
    println!("{}", divide(10.0, 0.0));
}
```

This "check first, bail out if something's wrong" style is called a **guard clause**, and it's extremely common in practice.

2.9.2.4 `return` for Functions Returning `()`

If the function returns `()` (e.g., no `-> type` written), `return` doesn't need a value after it:

```
fn check_age(age: i32) {
    if age < 0 {
        println!("Age can't be negative!");
        return; // Same as return ();
    }
    println!("Your age is {}", age);
}

fn main() {
    check_age(25);
    check_age(-3);
}
```

`return;` is shorthand for `return ();` — since what's returned is `()` (the unit type), leaving it out is cleaner.

2.9.2.5 Don't Use `return` Everywhere

Writing `return` for every return value does run, but in Rust it's not good style:

```
// Not very Rust-y
fn add_v1(a: i32, b: i32) -> i32 {
    return a + b; // Runs, but unnecessary
}

// Idiomatic Rust
fn add_v2(a: i32, b: i32) -> i32 {
    a + b         // The last line serves as the return value
}
```

Save `return` for the "leaving early" scenarios.

2.9.3 Recap

- `return value;` returns early from the middle of a function.
- The natural, semicolon-less return on the last line is idiomatic Rust.
- `return` is most often used in guard clauses: check a condition and bail out early if it's wrong.
- In functions returning `()`, `return;` is shorthand for `return ();`.
- Don't write `return` for every return value — only when leaving early.

2.10 Recursion

2.10.1 Goal of This Episode

Have a function call itself to solve a problem — a technique called "recursion."

2.10.2 Main Text

Have you ever wondered: can a function call itself from inside itself?

The answer is **yes**, and the technique is called **recursion**. It sounds mystical, but the concept is actually simple.

2.10.2.1 The Classic Example: Factorial

"The factorial of 5" is written `5!`, meaning `5 × 4 × 3 × 2 × 1 = 120`.

Thinking recursively:

- `5! = 5 × 4!`
- `4! = 4 × 3!`
- `3! = 3 × 2!`
- `2! = 2 × 1!`
- `1! = 1` (stop here)

See it? Each step is "myself times the factorial one size smaller than me," stopping once we reach 1.

```
fn factorial(n: u32) -> u32 {
    if n <= 1 {
        1
    } else {
        n * factorial(n - 1)
    }
}

fn main() {
    println!("5! = {}", factorial(5));
    println!("3! = {}", factorial(3));
    println!("1! = {}", factorial(1));
}
```

2.10.2.2 The Two Keys to Recursion

Every recursive function needs two things:

1. The base case: when to stop

```
if n <= 1 {
    1 // Stop! No more calling myself
}
```

2. The recursive case: how to shrink the problem

```
n * factorial(n - 1) // Shrink the problem: n becomes n - 1
```

If you forget the base case, the function calls itself endlessly and the program eventually blows

up.

2.10.2.3 Tracing the Execution

Let's trace how `factorial(5)` executes:

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120
```

Like Russian nesting dolls: unfold layer by layer, hit the bottom, then fold back up layer by layer.

2.10.2.4 Another Example: Countdown

```rust
fn countdown(n: u32) {
    if n == 0 {
        println!("Liftoff! 🚀");
        return;
    }
    println!("{}...", n);
    countdown(n - 1);
}

fn main() {
    countdown(5);
}
```

2.10.2.5 Recursion vs Loops

All the examples above could be written with loops. So when to use recursion, and when loops?

- **Simple repetition** → loops are more intuitive.
- **The problem itself has a recursive structure** → recursion is more natural.

For now, just knowing how to write recursion is enough — the right scenarios will come along later.

2.10.3 Recap

- Recursion is a function calling itself.
- There must be a **base case** (stopping condition), or you get infinite recursion.
- Each call must make the problem **smaller**, moving toward the base case.

2.11 Array Basics

2.11.1 Goal of This Episode

Use an array to line up multiple values of the same type, and learn how to access and create them.

2.11.2 Main Text

We've learned that tuples can package values of different types together. Today let's meet another good friend — the **array**. An array is "a bunch of values of the **same type**, lined up in a row."

2.11.2.1 Creating an Array

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    println!("{:?}", arr);
}
```

We used `{:?}` (the `Debug` format) to print the array — remember Episode 5 of this chapter? Like tuples, arrays only have the `Debug` format; `{}` doesn't work.

Wrap the values in square brackets `[]`, separated by commas. Note: the values in an array **must all be the same type**.

```
    let arr = [1, "hello", 3.14]; // ✗ No! Different types
```

Want to mix types? Use the tuple from a few episodes back.

2.11.2.2 Accessing by Index

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    println!("First: {}", arr[0]);
    println!("Third: {}", arr[2]);
    println!("Last: {}", arr[4]);
}
```

Key point: **indexing starts at 0**! So the indices of 5 elements are 0, 1, 2, 3, 4.

2.11.2.3 Going Out of Bounds Panics

If you access an index that doesn't exist:

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    println!("{}", arr[10]); // 💥 index out of bounds!
}
```

The program **crashes** (panics) and prints an error. Rust won't let you sneak a read of memory you shouldn't touch. Compared to silently handing you a garbage value, crashing outright is actually safer — at least you know immediately where things went wrong.

2.11.2.4 The Type of an Array

An array's type is written `[element_type; length]`:

```
fn main() {
    let arr: [i32; 5] = [1, 2, 3, 4, 5];
    println!("{:?}", arr);
}
```

`[i32; 5]` means "an array holding 5 `i32`s." Note that **the length is part of the type** — `[i32; 3]` and `[i32; 5]` are different types!

Most of the time Rust infers this automatically, no annotation needed. But knowing how to write the type will be useful later.

2.11.2.5 Quick Creation: The Repeat Syntax

If you want an array of "5 zeros":

```
fn main() {
    let zeros = [0; 5];
    println!("{:?}", zeros);
}
```

`[0; 5]` means "the value 0, repeated 5 times." Before the semicolon is the value; after it, the count.

A few more examples:

```
fn main() {
    let ones = [1; 10];    // ten 1s
    let flags = [true; 3]; // three trues
    println!("{:?}", ones);
    println!("{:?}", flags);
}
```

2.11.3 Recap

- Create arrays with `[value1, value2, ...]`; all elements must share a type.
- Indexing starts at 0; access values with `arr[0]`.
- Accessing an out-of-range index panics (crashes the program).
- Array types are written `[type; length]`, e.g. `[i32; 5]` (the length is part of the type).
- `[value; count]` quickly creates a repeated array, e.g. `[0; 5]`.
- Print whole arrays with `{:?}`.

2.12 Iterating over Arrays

2.12.1 Goal of This Episode

Use a `for` loop to walk through every element in an array.

2.12.2 Main Text

Last episode we learned to fetch values one at a time with `arr[0]`, `arr[1]`. But if an array has 100 elements, you can't write 100 lines, right? That's when we use a `for` loop to **iterate** over

the whole array.

2.12.2.1 Basic Syntax

```
fn main() {
    let arr = [1, 2, 3, 4, 5];

    for x in arr {
        println!("{}", x);
    }
}
```

`for x in arr` means: "Take the elements of `arr` out one by one, putting each into `x`, then run the code in the curly braces."

2.12.2.2 Computing with the Elements

```
fn main() {
    let scores = [80, 95, 72, 88, 100];

    for score in scores {
        if score >= 90 {
            println!("{} points → Excellent!", score);
        } else {
            println!("{} points → Keep it up!", score);
        }
    }
}
```

2.12.2.3 Summing All the Elements

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    let mut total = 0;

    for x in arr {
        total += x;
    }

    println!("Total: {}", total);
}
```

First create a mutable accumulator with `let mut total = 0;`, then add each value onto it in the loop.

2.12.2.4 `for in` a Range vs `for in` an Array

The `for i in 0..5` from Chapter 1 iterates over a **range of numbers**. This episode's `for x in arr` iterates over an **array**. Same syntax — only the thing after `in` differs:

```
fn main() {
    let arr = [10, 20, 30];

    // Iterating over a range: i is 0, 1, 2 in turn
    for i in 0..3 {
```

```
        println!("Index {}: {}", i, arr[i]);
    }

    // Iterating over the array: x is 10, 20, 30 in turn
    for x in arr {
        println!("Value: {}", x);
    }
}
```

When walking an array, `for x in arr` is cleaner, safer, and possibly faster than using indices — no worrying about going out of bounds. For when you need both the index and the value, we'll learn a better way later.

2.12.3 Recap

- `for x in arr { ... }` iterates over each element of the array.
- Inside the loop you can compute, test, and accumulate with each element.
- `for x in arr` (over an array) and `for i in 0..n` (over a range) share the same syntax; only the thing after `in` differs.
- When iterating over an array, `for x in arr` beats indexing: cleaner, safer, possibly faster.

2.13 Slices: `&[T]`

2.13.1 Goal of This Episode

Use a slice to grab part of an array — like looking at what's inside through a window.

2.13.2 Main Text

Sometimes you don't need the whole array, just a stretch of it. Say, out of a 5-element array, you only want elements 2 through 4. That's when you use a **slice**.

2.13.2.1 Basic Syntax

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    let slice = &arr[1..4];
    println!("{:?}", slice);
}
```

Like arrays and tuples, slices can only be printed with `{:?}`, not `{}`.

`&arr[1..4]` means: "Starting at index 1, up to just **before** index 4."

- Index 1 → value 2 (included).
- Index 2 → value 3 (included).
- Index 3 → value 4 (included).
- Index 4 → value 5 (**not included**).

So the result is `[2, 3, 4]`.

2.13.2.2 The Range Notations

```
fn main() {
    let arr = [1, 2, 3, 4, 5];

    let a = &arr[0..3];  // [1, 2, 3]       from 0 up to 3 (3 excluded)
    let b = &arr[0..=2]; // [1, 2, 3]       from 0 to 2 (2 included)
    let c = &arr[2..];   // [3, 4, 5]       from 2 to the end
    let d = &arr[..3];   // [1, 2, 3]       from the start up to 3 (3 excluded)
    let e = &arr[..];    // [1, 2, 3, 4, 5] the whole array

    println!("{:?}", a);
    println!("{:?}", b);
    println!("{:?}", c);
    println!("{:?}", d);
    println!("{:?}", e);
}
```

- `1..4` → from 1 up to 4 (4 excluded).
- `1..=3` → from 1 to 3 (3 **included**) — remember `..=` from Chapter 1, Episode 20? Same usage.
- `2..` → from 2 to the end.
- `..3` → from the start up to 3 (3 excluded).
- `..` → the whole thing.

2.13.2.3 A Slice Is a "Window," Not a "Copy"

Here's an important idea: a slice does **not** copy the data out. It "points at a stretch of the original array." Like looking at things in a room through a window — the things are still in the room; you're just viewing them through the window.

```
fn main() {
    let arr = [10, 20, 30, 40, 50];
    let slice = &arr[1..4];

    println!("Array: {:?}", arr);
    println!("Slice: {:?}", slice);
}
```

2.13.2.4 What's That &?

You may have noticed the & in front of the slice. That symbol stands for "borrowing," one of Rust's most important concepts. No need to dig deep right now — just remember "slices need &," and we'll explain in detail later.

For now, all you need to know: when writing a slice, put & in front.

2.13.2.5 The Type of a Slice

Remember that an array's type is `[i32; 5]` (the type includes the length)? A slice's type is `&[i32]` — **no length**:

```
fn main() {
    let arr: [i32; 5] = [1, 2, 3, 4, 5];
    let slice: &[i32] = &arr[1..4];
```

```
    println!("{:?}", slice);
}
```

`&[i32]` means "a slice of `i32`s," regardless of length. This is the biggest difference between slices and arrays — an array's length is part of its type (`[i32; 3]` and `[i32; 5]` are different types), but a slice doesn't care about length: `&[i32]` can point to a contiguous stretch of any length.

2.13.2.6 Iterating over a Slice

Slices can be walked with `for` too:

```
fn main() {
    let arr = [1, 2, 3, 4, 5];
    let slice = &arr[1..4];

    for x in slice {
        println!("{}", x);
    }
}
```

2.13.2.7 Compound Types

Having learned slices, let's take stock: so far we've met two categories of types.

Primitive types: `i32`, `f64`, `bool`, `char`, and so on — each value is a single standalone thing.

Compound types: types that combine other types. We've now learned three:

- **Tuple**: `(i32, f64, bool)` — can hold different types.
- **Array**: `[i32; 5]` — one type, fixed length.
- **Slice**: `&[i32]` — one type, any length.

The types inside a compound type don't have to be primitive — compound types can nest inside other compound types:

```
fn main() {
    // An array holding tuples
    let pairs: [(i32, bool); 3] = [(1, true), (2, false), (3, true)];
    println!("{:?}", pairs);

    // A tuple holding arrays
    let t: ([i32; 3], [i32; 3]) = ([1, 2, 3], [4, 5, 6]);
    println!("{:?}", t);

    // An array holding arrays
    let grid: [[i32; 2]; 3] = [[1, 2], [3, 4], [5, 6]];
    println!("{:?}", grid);

    // Slices are compound types too
    let arr: [i32; 5] = [10, 20, 30, 40, 50];
    let slice: &[i32] = &arr[1..4];
    println!("{:?}", slice);

    // A tuple holding slices
    let pair: (&[i32], &[i32]) = (&arr[..2], &arr[3..]);
```

```
    println!("{:?}", pair);
}
```

2.13.3 Recap

- A slice takes part of an array with `&arr[start..end]` or `&arr[start..=end]`.
- `start..end` means "start included, end excluded."
- `start..=end` means "both start and end included."
- A slice is a "window" onto the array, not a copy.
- A slice's type is `&[i32]` (no length); an array's type is `[i32; 5]` (with length).
- The leading `&` stands for borrowing — detailed explanation coming later.
- Slices can be iterated with `for` as well.
- Tuples, arrays, and slices are all **compound types** — they can contain other types, including compound ones.

2.14 Slices as Parameters

2.14.1 Goal of This Episode

Use a slice `&[i32]` as a function parameter, so arrays of any length can be passed in.

2.14.2 Main Text

Last episode we learned slices. This episode covers a super-practical application: **using slices as function parameters**.

2.14.2.1 First, the Problem

Suppose you want to write a function that sums an array. If you use an array as the parameter:

```
fn sum(nums: [i32; 5]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}

fn main() {
    let a = [1, 2, 3, 4, 5];
    println!("{}", sum(a)); // ✅ Works

    let b = [1, 2, 3];
    println!("{}", sum(b)); // ❌ Nope! b has 3 elements, but the function wants 5
}
```

The problem is `[i32; 5]` — you've hard-coded the length as 5. A 3-element array can't get in.

2.14.2.2 The Solution: Use a Slice

```
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
```

```
    for x in nums {
        total += x;
    }
    total
}

fn main() {
    let a = [1, 2, 3, 4, 5];
    let b = [10, 20, 30];
    let c = [7];

    println!("Sum of a: {}", sum(&a)); // 15
    println!("Sum of b: {}", sum(&b)); // 60
    println!("Sum of c: {}", sum(&c)); // 7
}
```

Change the parameter type from `[i32; 5]` to `&[i32]`, and the function accepts slices of **any length**!

When calling, add `&`: `sum(&a)` means "pass in a slice of a."

2.14.2.3 You Can Pass Part of a Slice Too

Since the parameter is `&[i32]`, you can pass not only whole arrays but also slices of them:

```
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}

fn main() {
    let arr = [1, 2, 3, 4, 5];

    println!("All: {}", sum(&arr));            // 15
    println!("First three: {}", sum(&arr[..3])); // 6
    println!("Last three: {}", sum(&arr[2..]));  // 12
}
```

That's the power of slices — one function, many uses.

2.14.2.4 Why Are Slices Better Than Fixed-length Arrays?

Fixed length `[i32; 5]`	Slice `&[i32]`
Accepts exactly 5 elements only	Any length works
Changing length means rewriting the function	One function covers all

In practice, nearly every function that takes an array uses a slice parameter.

2.14.3 Recap

- Use `&[i32]` instead of `[i32; 5]` for parameters to accept any length.

- When calling, &arr and &arr[1..4] both work.
- Slice parameters make functions more flexible and more general.
- This is the most common style in real-world Rust.

2.15 String Slices: &str

2.15.1 Goal of This Episode

Meet the &str type — it turns out the strings we've been using all along are slices!

2.15.2 Main Text

In recent episodes we learned array slices, &[i32]. Today let's meet another kind of slice — the **string slice**. As it happens, the "hello" we've been writing all along is a string slice.

2.15.2.1 The True Face of Strings

```
fn main() {
    let s = "hello";
    println!("{}", s);
}
```

You've seen this kind of code countless times. But what is the type of s?

The answer: **&str** (a string slice).

```
fn main() {
    let s: &str = "hello"; // Type spelled out explicitly
    println!("{}", s);
}
```

&str is read "string slice." Just like the array slice &[i32], it's a "window pointing at a stretch of data."

2.15.2.2 Comparison with Array Slices

Array slice	String slice
&[i32]	&str
Points at a stretch of i32 data	Points at a stretch of text data
let s = &arr[1..4];	let s = "hello";

Exactly the same concept! One is a slice of numbers, the other a slice of text.

2.15.2.3 String Slices Can Take Substrings Too

```
fn main() {
    let s = "hello world";
    let hello = &s[0..5];
    let world = &s[6..11];
    println!("{}", hello); // hello
    println!("{}", world); // world
}
```

`&s[0..5]` takes the first 5 bytes of `s` (note: bytes, not characters).

As with array slices, `..=` includes the end:

```
fn main() {
    let s = "hello world";
    let hello = &s[0..=4]; // Includes index 4; same as &s[0..5]
    println!("{}", hello); // hello
}
```

2.15.2.4 ⚠ Careful When Slicing Chinese Strings!

An English letter takes 1 byte, but a Chinese character usually takes **3 bytes**. If your cut lands right in the "middle" of a Chinese character, the program crashes outright:

```
fn main() {
    let s = "你好";
    let first = &s[0..3]; // ✅ "你" (exactly 3 bytes)
    println!("{}", first);
}
```

But try slicing `&s[0..1]`:

```
fn main() {
    let s = "你好";
    let oops = &s[0..1];  // ✕ Program crashes!
    println!("{}", oops);
}
```

Because "你" occupies 3 bytes (indices 0, 1, 2), cutting at index 1 lands in the middle of the character, and Rust won't allow it.

In short: slicing English strings is safe, but when slicing strings with Chinese (or other multi-byte) characters, make sure the cut lands exactly on a character boundary. When unsure, hold off on using `&s[start..end]` with such strings.

2.15.2.5 `&str` as a Function Parameter

Now that you know strings are `&str`, you can use it as a function parameter:

```
fn greet(name: &str) {
    println!("Hi, {}!", name);
}

fn main() {
    greet("Andy");
    greet("Ming");
}
```

`"Andy"` is itself of type `&str`, so it can be passed straight in.

2.15.3 Recap

- The type of `"hello"` is `&str` — a string slice.
- `&str` shares the concept of the array slice `&[i32]` — both are "windows onto a stretch of data."
- Write `&str` for parameters to accept strings.

- You can take substrings with `&s[start..end]`, but beware: indices are byte positions, not character positions — cutting through the middle of a multi-byte character (like Chinese) panics.

Congratulations on finishing Chapter 2! 🎉 In this chapter we learned more ways to organize programs — functions, arrays, slices, and assorted techniques for clearer code. In the next chapter we'll start defining custom types, describing your own data with `struct` and `enum`!

Chapter 3

Structs, Enums, and Pattern Matching

This chapter teaches you how to write your own types, and how to take apart and analyze both existing types and the ones you create. The knowledge in this chapter may not let you write more complex algorithms, but it does mark your first step down the road of real software engineering. Even though the types you create are composed of other, simpler types that already exist, they can be given logically richer meaning. And as one type becomes part of the next type in turn, we gain the ability to manage more complex logic and model reality — or people's imaginations — with greater precision.

3.1 `struct` (Named Fields)

3.1.1 Goal of This Episode

Learn to use a `struct` to group several related values together into a custom type.

3.1.2 Concept

So far, every type we've used has been built into Rust: `i32`, `f64`, `bool`, `char`, plus tuples, arrays, and slices. But in real programming, you'll need to **define new types of your own**.

A `struct` is one of the ways Rust lets you define a new type. Defining a `struct` tells Rust: "I want a new type, and it contains these fields."

For example, a "point" has an `x` coordinate and a `y` coordinate. We could represent it with a tuple `(i32, i32)`, but tuples only offer `.0` and `.1` — you can't tell which is `x` and which is `y`. With a `struct`, every field gets a name.

The syntax for defining a `struct`:

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {}
```

`struct` definitions generally go outside `fn main()`, so other functions can use them too. Above or below is fine (like functions, they're not restricted by definition order).

To create a `struct` value, use the `TypeName { field_name: value }` syntax. To read a value, use `.field_name`. To modify a `struct`'s fields, the variable must be `mut`.

3.1.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 3, y: 7 };
    println!("The x coordinate is {}", p.x);
    println!("The y coordinate is {}", p.y);

    // Point is a type, just like i32, and can be used in type annotations
    let p2: Point = Point { x: 100, y: 200 };
    println!("p2's coordinates are ({}, {})", p2.x, p2.y);

    // With mut, a struct's values can be modified
    let mut q = Point { x: 0, y: 0 };
    q.x = 10;
    q.y = 20;
    println!("q's coordinates are ({}, {})", q.x, q.y);
}
```

3.1.4 Extra: Trailing Commas

Notice that in the `struct` definition, even the last field has a comma after it:

```
struct Point {
    x: i32,
    y: i32, // ← This comma is optional
}

fn main() {}
```

Rust allows a comma after the last item in `struct` definitions, `struct` creation, function calls, and more. This is called a **trailing comma**. Adding it is never an error, and the benefit is that when you add a field later, you don't have to go back and add a comma to the previous line — and the git diff stays cleaner.

The Rust community convention is to **include the trailing comma**.

3.1.5 Recap

- A `struct` lets you define a custom type with named fields.
- `struct` definitions generally go outside `fn main()`; above or below both work (like functions).
- Syntax for creating a `struct` value: `Point { x: 1, y: 2 }`.
- Read a field's value with `.field_name`, e.g. `p.x`.
- To modify a `struct`'s fields, the variable must be `mut`.
- The comma after the last field (trailing comma) is optional; convention is to include it.

3.2 Tuple `structs` and Unit `structs`

3.2.1 Goal of This Episode

Learn to define `structs` without field names using tuple `structs`, and `structs` with no fields at all using unit `structs`.

3.2.2 Concept

Last episode's `structs` gave every field a name. But sometimes the fields' meanings are already obvious, and no naming is needed. That's when you can use a **tuple `struct`** — it looks like a hybrid of a tuple and a `struct`.

```
struct Point(i32, i32);

fn main() {}
```

Create a value with `Point(3, 7)` — note that `Point` here is both the **name of the type** and the **name used when creating values**. Access values with `.0`, `.1`, just like a tuple.

The named-field `struct` from last episode works the same way: `Point` is both the type name and the name used when writing `Point { x: 1, y: 2 }` to create a value.

There's an even more extreme case: a `struct` with no fields at all, called a **unit `struct`**. It's usually used as a "marker" — signaling some identity or role while carrying no data of its own.

```
struct Marker;

fn main() {}
```

3.2.3 Example Code

```
// Tuple struct: fields have no names; access by position
struct Point(i32, i32);

// Another tuple struct example
struct Color(i32, i32, i32);

// Unit struct: no fields at all
struct Marker;

fn main() {
    let p: Point = Point(3, 7);
    println!("x = {}, y = {}", p.0, p.1);

    let red: Color = Color(255, 0, 0);
    println!("R={}, G={}, B={}", red.0, red.1, red.2);

    // Creating a unit struct needs no parentheses or braces
    let _m: Marker = Marker;
    println!("Marker created! (It carries no data)");
}
```

3.2.4 Recap

- **Tuple `struct`**: `struct Point(i32, i32);`, with values accessed via `.0`, `.1`.
- **Unit `struct`**: `struct Marker;`, with no fields whatsoever.
- Tuple `structs` suit cases where field meanings are obvious and names are unnecessary.
- Unit `structs` work as markers, carrying no data.
- Even if two tuple `structs` have exactly the same field types, they are different types (e.g. `Point(i32, i32)` and `Size(i32, i32)` are not interchangeable).

3.3 `enum` (C-style)

3.3.1 Goal of This Episode

Learn to use an `enum` to define a fixed set of options, so a variable can only be one of those values.

3.3.2 Concept

Last episode we learned `struct` — for defining new types that "group several values together." This episode covers another way to define a new type: `enum`.

Sometimes we want to express "this thing can only be one of a few options." For instance, a traffic light can only be red, yellow, or green.

An `enum` (enumeration) is for defining exactly this kind of "pick one of several" type. Like a `struct`, defining an `enum` tells Rust: "I want a new type whose value can only be one of these options." The simplest `enum` looks like this:

```
enum Color {
    Red,
    Green,
    Blue,
}

fn main() {}
```

Each option is called a **variant**. To create an `enum` value, use the `TypeName::VariantName` syntax:

```
enum Color {
    Red,
    Green,
    Blue,
}

fn main() {
    let c = Color::Red;
}
```

Note the two colons `::` in the middle — in Rust this is the "path operator," meaning "the `Red` under `Color`."

This most basic kind of `enum` — where no variant carries any extra data — is sometimes called

a C-style `enum`, because that's what `enum`s are like in the C language.

3.3.3 Example Code

```
enum Direction {
    Up,
    Down,
    Left,
    Right,
}

fn main() {
    let _dir = Direction::Up;

    // We can't do much with enums yet
    // Next episode we'll learn match, which lets us act on an enum's value
    // For now, here's how to create different enum values

    let _d1 = Direction::Down;
    let _d2 = Direction::Left;
    let _d3 = Direction::Right;

    println!("Direction is set!");
    println!("(Next episode, match lets us act based on the direction)");
}
```

3.3.4 Recap

- Like `struct`, an `enum` is a way to define a new type.
- `struct`: groups several values together; `enum`: picks one from several options.
- Use `::` to name a specific variant, e.g. `Direction::Up`.
- Every variant of a C-style `enum` carries no extra data.
- Like `struct`s, `enum` definitions generally go outside `fn main()`, above or below.
- We can't directly print an `enum`'s value yet (we can once we learn `match` next episode).

3.4 `match` on C-style `enum`s

3.4.1 Goal of This Episode

Learn to use `match` to run different code based on an `enum`'s value, and understand the idea of "exhaustiveness."

3.4.2 Concept

Last episode we defined an `enum` but couldn't do different things based on its value. Time to learn `match` — Rust's most powerful **pattern matching** tool.

The basic syntax of `match`:

```
match variable {
    pattern1 => do something,
    pattern2 => do something else,
```

```
    pattern3 => do a third thing,
}
```

Each line is called an "arm." Rust checks from top to bottom and runs the code for the first pattern that matches.

The most important rule: `match` must exhaustively cover every possible value. If your `enum` has three variants, you must handle all three. Leave one out, and the compiler reports an error. This is Rust catching bugs for you — making sure you never forget to handle a case.

As with `structs` and `enums`, the last arm of a `match` can take a trailing comma. The Rust community convention is to include it.

3.4.3 Example Code

```
enum Color {
    Red,
    Green,
    Blue,
}

fn main() {
    let c = Color::Green;

    match c {
        Color::Red => println!("Red"),
        Color::Green => println!("Green"),
        Color::Blue => println!("Blue"),
    }

    // One more example
    let light = Color::Red;

    match light {
        Color::Red => println!("Stop!"),
        Color::Green => println!("Go!"),
        Color::Blue => println!("This traffic light is a bit odd..."),
    }
}
```

3.4.4 Recap

- `match` compares a value against patterns and runs the corresponding arm.
- Each arm separates the pattern from the code with `=>`.
- **`match` must cover every variant** — missing one fails compilation.
- Arms are checked top to bottom; the first match runs.
- `match` is the most fundamental way Rust handles `enums`.

3.5 `match` as an Expression

3.5.1 Goal of This Episode

Learn to use `match` as an expression, so it returns a value.

3.5.2 Concept

Remember from Chapter 1 that `if` can be an expression?

```
fn main() {
    let condition = true;
    let x = if condition { 1 } else { 2 };
}
```

`match` can too! You can put a whole `match` on the right side of a `let`, with each arm returning a value:

```
enum Color {
    Red,
    Green,
    Blue,
}

fn main() {
    let c = Color::Red;
    let msg = match c {
        Color::Red => "red",
        Color::Green => "green",
        Color::Blue => "blue",
    };
}
```

Note the semicolon `;` at the very end — the whole `let msg = match ... { ... };` is one `let` statement.

The values returned by all the arms must have matching types. If the first arm returns `&str`, every other arm must return `&str` too.

3.5.3 Example Code

```
enum Season {
    Spring,
    Summer,
    Autumn,
    Winter,
}

fn main() {
    // match as an expression, returning &str
    let season = Season::Autumn;
    let name = match season {
        Season::Spring => "spring",
        Season::Summer => "summer",
        Season::Autumn => "autumn",
```

```
        Season::Winter => "winter",
    };
    println!("It's {} now", name);

    // Another example: match returning i32
    let weather = Season::Summer;
    let temp = match weather {
        Season::Spring => 22,
        Season::Summer => 35,
        Season::Autumn => 18,
        Season::Winter => 8,
    };
    println!("About {} degrees", temp);
}
```

3.5.4 Recap

- `match` can be an expression; the whole `match` returns a value.
- Usage: `let x = match ... { ... };` (don't forget the final semicolon).
- All arms must return values of matching types.
- Same idea as the `if` expression — lots of things in Rust are expressions.

3.6 Block Expressions

3.6.1 Goal of This Episode

Learn to create block expressions with curly braces `{}`, running several lines of code inside and returning a value.

3.6.2 Concept

In Rust, a pair of curly braces `{}` isn't just a scope — it's also an **expression** in its own right, capable of returning a value. The rule is simple: if the last line inside the block has no semicolon, that line's value is the whole block's return value.

```
fn main() {
    let x = {
        let y = 5;
        y + 1 // No semicolon → this is the block's return value
    };
    // x is now 6
}
```

This concept is especially useful in `match`. Our `match` arms so far were all one-liners, but if you want to do several things in one arm, use a block:

```
    match c {
        Color::Red => {
            println!("It's red!");
            "red"
        }
        // ...
```

```
    }
```

Inside the block you can declare variables and do calculations; the last line without a semicolon is the return value.

Note: if a `match` arm uses a block `{}`, the comma after it can be omitted. The `}` is already an unambiguous end marker, so Rust doesn't need the comma as a separator. But if the arm is a single line (no block), the comma can't be dropped — with one exception: the very last arm of the `match` is at the end, with nothing after it to separate, so its trailing comma is optional.

```
    match s {
        Season::Summer => {
            println!("So hot!");
            "a scorching summer"
        } // ← No comma; OK
        Season::Autumn => "a cool autumn", // ← Single-line arm; comma required
        // ...
    }
```

3.6.3 Example Code

```
enum Season {
    Spring,
    Summer,
    Autumn,
    Winter,
}

fn main() {
    // Basic use of a block expression
    let result = {
        let a = 10;
        let b = 20;
        a + b // Last line without a semicolon → the return value
    };
    println!("result = {}", result);

    // Using a block in a match arm
    let s = Season::Summer;

    let description = match s {
        Season::Spring => {
            let temp = 22;
            println!("Spring is in the air");
            if temp > 20 {
                "a warm spring"
            } else {
                "a still-chilly spring"
            }
        }
        Season::Summer => {
            println!("So hot!");
            "a scorching summer"
        }
```

```
        Season::Autumn => "a cool autumn",
        Season::Winter => "a cold winter",
    };
    println!("{}", description);
}
```

3.6.4 Recap

- A {} block is itself an expression; the last line without a semicolon is its return value.
- `let x = { ... };` runs multiple lines inside the block and assigns the result to x.
- A `match` arm can use `=> { ... }` to run multiple lines, and no comma is needed after the block.
- Variables declared inside a block only live within the block (scope).
- Block expressions are extremely common in Rust, and they are an important basic concept to understand.

3.7 `enum` with Tuple Variants

3.7.1 Goal of This Episode

Learn to make `enum` variants carry extra data, tuple-style.

3.7.2 Concept

In the C-style `enums` we learned earlier, each variant was just a name carrying no data. But often, different options need to carry different data.

For instance, a "shape" might be a circle or a rectangle. A circle needs a radius; a rectangle needs a width and a height — they need different data. In Rust, you can have each variant carry data:

```
enum Shape {
    Circle(f64),         // Carries one f64 (the radius)
    Rectangle(i32, i32), // Carries two i32s (width, height)
}
```

This style looks like appending tuple fields to the variant's name, so it's called a **tuple variant**.

Creating a value looks like calling a function — put the data in the parentheses:

```
    let s = Shape::Circle(3.14);
    let r = Shape::Rectangle(10, 20);
```

Note: we now know how to create `enums` that carry data, but to "extract" the data inside, we need `match` — which we'll learn in Episode 9.

3.7.3 Example Code

```
enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}
```

```
enum Message {
    Quit,           // No data (just like C-style)
    Echo(i32),      // Carries one i32
    Move(i32, i32), // Carries two i32s
}

fn main() {
    let s1 = Shape::Circle(5.0);
    let s2 = Shape::Rectangle(10, 20);

    let m1 = Message::Quit;
    let m2 = Message::Echo(42);
    let m3 = Message::Move(3, 7);

    // For now, we just create the values
    // Episode 9 covers extracting the data with match
    println!("Shapes and messages created!");

    // Within one enum, different variants can carry different amounts and types of data
    // Some variants can even carry nothing at all (like Message::Quit)
}
```

3.7.4 Recap

- `enum` variants can carry data: `Circle(f64)` means Circle carries one `f64`.
- Create a data-carrying variant with `Shape::Circle(5.0)`.
- Within one `enum`, different variants can carry different data.
- Some variants carry nothing, some one value, some several — very flexible.
- Extracting a variant's data requires `match` (coming in Episode 9).

3.8 `enum` with `struct` Variants

3.8.1 Goal of This Episode

Learn to give `enum` variants named fields, `struct`-style.

3.8.2 Concept

Last episode's tuple variants had unnamed fields, distinguished by position. But if a variant carries a lot of data, having no names makes things easy to mix up.

Rust lets you write variants in a style similar to named-field `structs`, giving every field a name:

```
enum Shape {
    Circle { radius: f64 },
    Rectangle { width: i32, height: i32 },
}
```

Creating a value works just like creating a `struct`:

```
    let s = Shape::Circle { radius: 5.0 };
    let r = Shape::Rectangle { width: 10, height: 20 };
```

Within a single `enum`, some variants can use the tuple form, some the `struct` form, and some can carry nothing at all — mixing and matching is completely fine.

3.8.3 Example Code

```
enum Shape {
    Circle { radius: f64 },
    Rectangle { width: i32, height: i32 },
    Dot, // A data-free variant can be mixed in too
}

fn main() {
    let s1 = Shape::Circle { radius: 5.0 };
    let s2 = Shape::Rectangle { width: 10, height: 20 };
    let s3 = Shape::Dot;

    // We can't extract the fields directly yet
    // Episode 10 covers extracting struct-variant data with match
    println!("All three shapes created!");

    // A more everyday example
    let event = Event::Click { x: 100, y: 200 };
    println!("Event created!");
}

enum Event {
    Click { x: i32, y: i32 },
    KeyPress(char), // The tuple form mixes in fine
    Quit,           // Carrying nothing works too
}
```

3.8.4 Recap

- Variants can carry named fields in `struct` form: `Circle { radius: f64 }`.
- Create a value: `Shape::Circle { radius: 5.0 }`.
- One `enum` can mix and match: tuple-form variants, `struct`-form variants, and data-free ones.
- The `struct` form's advantage: named fields sometimes make the code easier to read.
- Extracting field data requires `match` (coming in Episode 10).

3.9 Destructuring Tuple Variants with `match`

3.9.1 Goal of This Episode

Learn to destructure `enum` tuple variants with `match`, extracting the data they carry.

3.9.2 Concept

In Episode 7 we learned to create `enum` variants that carry data, but we've had no way to get the data back out. Now we finally can!

Inside a `match` pattern, you can use a variable name to "catch" the data inside a variant:

```
    match s {
        Shape::Circle(r) => println!("The radius is {}", r),
        Shape::Rectangle(w, h) => println!("Width {}, height {}", w, h),
    }
```

The `r` in `Shape::Circle(r)` isn't a fixed name — you can pick anything. It means: "If `s` is a `Circle`, take the `f64` inside and call it `r`."

This move is called **destructuring** — taking a compound thing apart and pulling out its pieces. `match` doesn't just check "which variant is it"; it can simultaneously destructure the data inside for you to use.

3.9.3 Example Code

```
enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}

fn main() {
    let s = Shape::Circle(5.0);

    match s {
        Shape::Circle(r) => {
            println!("This is a circle");
            println!("The radius is {}", r);
            let area = r * r * 3.14159;
            println!("The area is roughly {}", area);
        }
        Shape::Rectangle(w, h) => {
            println!("This is a rectangle");
            println!("Width {}, height {}", w, h);
            let area = w * h;
            println!("The area is {}", area);
        }
    }

    // One more example
    let action = Action::Move(3, -2);

    match action {
        Action::Stop => println!("Standing still"),
        Action::Move(dx, dy) => {
            println!("Moving {} along x and {} along y", dx, dy);
        }
    }
}

enum Action {
    Stop,
    Move(i32, i32),
}
```

3.9.4 Recap

- **Destructuring**: taking a compound thing apart to get at the pieces inside.
- In a `match` pattern, variable names inside the parentheses destructure a tuple variant.
- `Shape::Circle(r)` → take the value inside `Circle` and call it `r`.
- `Shape::Rectangle(w, h)` → call the two values inside `Rectangle` `w` and `h`.
- The variable names are yours to choose.
- `match` still has to cover every variant.

3.10 Destructuring `struct` Variants with `match`

3.10.1 Goal of This Episode

Learn to destructure `enum` `struct` variants with `match`, extracting the named fields inside.

3.10.2 Concept

Episode 9 covered destructuring tuple variants (by position); now let's destructure `struct` variants (by field name).

The syntax uses `field_name: variable_name` inside the pattern:

```
    match s {
        Shape::Circle { radius: r } => println!("Radius {}", r),
        Shape::Rectangle { width: w, height: h } => println!("{}x{}", w, h),
    }
```

`radius: r` means "take the value of the `radius` field and call it `r`." Left of the colon is the field name; right of it is a variable name of your choosing.

This looks a lot like the syntax for creating a `struct` variant, just in the opposite direction: creating "puts values in," while `match` "takes values out."

3.10.3 Example Code

```
enum Shape {
    Circle { radius: f64 },
    Rectangle { width: i32, height: i32 },
}

fn main() {
    let s = Shape::Rectangle { width: 10, height: 5 };

    match s {
        Shape::Circle { radius: r } => {
            println!("This is a circle, radius = {}", r);
            let area = r * r * 3.14159;
            println!("Area roughly {}", area);
        }
        Shape::Rectangle { width: w, height: h } => {
            println!("This is a rectangle");
            println!("Width = {}, height = {}", w, h);
            let area = w * h;
```

```
            println!("Area = {}", area);
            let perimeter = 2 * (w + h);
            println!("Perimeter = {}", perimeter);
        }
    }
}
```

3.10.4 Ordinary `structs` Work the Same Way

It's not just `enum` `struct` variants — ordinary named-field `structs` can be destructured the same way:

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 3, y: 0 };

    match p {
        Point { x: 0, y: 0 } => println!("The origin"),
        Point { x: a, y: 0 } => println!("On the x-axis, x = {}", a),
        Point { x: 0, y: b } => println!("On the y-axis, y = {}", b),
        Point { x: a, y: b } => println!("At ({}, {})", a, b),
    }
}
```

Exactly the same syntax — `TypeName { field_name: variable_name }`.

Notice the patterns above mix **fixed values** and **variables**: in `Point { x: 0, y: b }`, `x: 0` is a fixed value (it only matches when `x` equals 0), while `y: b` is a variable (take y's value and call it `b`). This trick is very common in `match`. `match` compares the patterns top to bottom in order. As soon as one matches, the code on the right runs, and then execution leaves the whole `match` — no further comparisons.

3.10.5 Recap

- In a `match`, destructure `struct` variants with `field_name: variable_name`.
- `Shape::Circle { radius: r }` → take the `radius` field and call it `r`.
- Left of the colon is the field name (must match the definition); right of it is your chosen variable name.
- Ordinary named-field `structs` can be destructured in a `match` the same way.
- Patterns can mix fixed values and variables: `Point { x: 0, y: b }` means "x must be 0; take y out as `b`."
- `match` compares top to bottom; on the first success it runs that arm and exits the `match`.
- All fields must be written out (for now — we'll learn how to ignore them later).

3.11 Field Shorthand

3.11.1 Goal of This Episode

Learn to simplify `struct` creation and pattern matching with field shorthand.

3.11.2 Concept

Last episode we wrote `radius: r` in a match, meaning "take the `radius` field out and call it `r`." But what if you want the variable to be called `radius` itself? Following the earlier style you'd write `radius: radius` — the field name and variable name repeated, a bit wordy.

Rust offers a shorthand: if the variable name matches the field name, write it once:

```
    // Full form
    Shape::Circle { radius: radius };
    // Shorthand (field shorthand)
    Shape::Circle { radius };
```

This shorthand isn't just for `match` — **it works when creating a `struct` too**:

```
    let x = 3;
    let y = 7;
    // Full form
    let p = Point { x: x, y: y };
    // Shorthand
    let p = Point { x, y };
```

Whenever the variable name matches the field name, the `: variable_name` part can be dropped.

3.11.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

enum Shape {
    Circle { radius: f64 },
    Rectangle { width: i32, height: i32 },
}

fn main() {
    // Field shorthand when creating a struct
    let x = 5;
    let y = 10;
    let p = Point { x, y }; // Same as Point { x: x, y: y }
    println!("The point's coordinates: ({}, {})", p.x, p.y);

    // Works when creating an enum struct variant too
    let radius = 3.5;
    let s = Shape::Circle { radius }; // Same as Shape::Circle { radius: radius }

    // Field shorthand works in match as well
```

```
    match s {
        Shape::Circle { radius } => {
            println!("Circle, radius = {}", radius);
        }
        Shape::Rectangle { width, height } => {
            println!("Rectangle {}x{}", width, height);
        }
    }
}
```

3.11.4 Recap

- When the variable name matches the field name, write it once: `Point { x, y }` equals `Point { x: x, y: y }`.
- This shorthand is called **field shorthand**.
- Usable when creating `structs` / `enum` variants.
- Usable in `match` patterns too.
- It's very common — real Rust code uses the shorthand all the time.

3.12 Tuple Patterns

3.12.1 Goal of This Episode

Learn to destructure ordinary tuples and tuple `structs` inside a `match`.

3.12.2 Concept

Episode 9 covered destructuring `enum` variants in `match`. But it's not just `enums` — we can use `match` to destructure ordinary tuples too!

```
fn main() {
    let point = (3, 7);

    match point {
        (0, 0) => println!("The origin"),
        (x, 0) => println!("On the x-axis, x = {}", x),
        (0, y) => println!("On the y-axis, y = {}", y),
        (x, y) => println!("At ({}, {})", x, y),
    }
}
```

`match` compares top to bottom:

- `(0, 0)` → matches only when both values are 0.
- `(x, 0)` → the second value is 0, the first is anything (captured as `x`).
- `(0, y)` → the first value is 0, the second is anything.
- `(x, y)` → matches everything (the last arm acts as the "default").

Just like Episode 10, patterns can mix "fixed values" and "variables." Fixed values do the comparing; variables catch the data.

3.12.3 Example Code

```
fn main() {
    let point = (2, 0);

    match point {
        (0, 0) => println!("The origin"),
        (x, 0) => println!("On the x-axis, x = {}", x),
        (0, y) => println!("On the y-axis, y = {}", y),
        (x, y) => println!("An ordinary point ({}, {})", x, y),
    }

    // Simple classification with match and tuples
    let score = (85, 90);

    match score {
        (100, 100) => println!("Double perfect score!"),
        (a, b) => {
            println!("Literature {}, math {}", a, b);
            let total = a + b;
            println!("Total {}", total);
        }
    }
}
```

3.12.4 Tuple `structs` Work the Same Way

Remember the tuple `struct` from Episode 2? Its pattern matching works exactly like ordinary tuples:

```
struct Point(i32, i32);

fn main() {
    let p = Point(3, 0);

    match p {
        Point(0, 0) => println!("The origin"),
        Point(x, 0) => println!("On the x-axis, x = {}", x),
        Point(0, y) => println!("On the y-axis, y = {}", y),
        Point(x, y) => println!("At ({}, {})", x, y),
    }
}
```

The only difference is the type name in front of the pattern, `Point(...)`, whereas ordinary tuples are written directly as `(...)`.

3.12.5 Recap

- Ordinary tuples can be `match`ed too.
- Tuple `structs` pattern-match the same way, just with the type name in front: `Point(x, y)`.

3.13 Slice Patterns

3.13.1 Goal of This Episode

Learn to destructure arrays and slices with slice patterns.

3.13.2 Concept

3.13.2.1 Pattern Matching on Arrays

Last episode we learned to destructure tuples — and we can pattern-match arrays and slices too! Use a slice pattern like `[a, b, c]` to compare each element of an array:

```
fn main() {
    let rgb = [255, 128, 0];

    match rgb {
        [255, 0, 0] => println!("Pure red"),
        [0, 255, 0] => println!("Pure green"),
        [0, 0, 255] => println!("Pure blue"),
        [r, g, b] => println!("Custom color: R={}, G={}, B={}", r, g, b),
    }
}
```

As in recent episodes, patterns can mix “fixed values” and “variables.” Fixed values do the comparing; variables catch the data.

3.13.2.2 Slices Work Too

It’s not just fixed-length arrays — slices (`&[T]`) can use slice patterns as well. The difference is that a slice’s length is unknown at compile time, so you can match with patterns of different lengths. The _ in the last arm below means “everything else”; Episode 15 covers it properly:

```
fn describe(numbers: &[i32]) {
    match numbers {
        [] => println!("Empty"),
        [x] => println!("Just one element: {}", x),
        [x, y] => println!("Two elements: {} and {}", x, y),
        [x, y, z] => println!("Three elements: {}, {}, {}", x, y, z),
        _ => println!("More than three elements"),
    }
}
```

A fixed-length array always has its fixed length — for an `[i32; 3]`, arms like `[]` or `[x]` can never match. Only slices need to account for varying lengths.

3.13.3 Example Code

```
fn describe(data: &[i32]) {
    match data {
        [] => println!("An empty slice"),
        [only] => println!("Just one element: {}", only),
        [first, second] => println!("Two elements: {} and {}", first, second),
        _ => println!("Many elements; the first is {}", data[0]),
    }
```

```
}

fn main() {
    // Slice pattern on a fixed-length array
    let rgb = [255, 128, 0];

    match rgb {
        [255, 0, 0] => println!("Pure red"),
        [0, 255, 0] => println!("Pure green"),
        [0, 0, 255] => println!("Pure blue"),
        [r, g, b] => println!("Custom color: R={}, G={}, B={}", r, g, b),
    }

    println!("---");

    // Slice patterns on slices — matching different lengths
    describe(&[]);
    describe(&[42]);
    describe(&[1, 2]);
    describe(&[10, 20, 30, 40, 50]);
}
```

3.13.4 Recap

- With arrays, `match` can use slice patterns like `[a, b, c]`, similar to tuple patterns.
- Slices `&[T]` have no fixed length, so patterns of different lengths can match (`[]`, `[x]`, `[x, y]`...).

3.14 Nested Pattern Matching

3.14.1 Goal of This Episode

Learn to destructure deeper structures within a `match` — nested pattern matching.

3.14.2 Concept

So far our `match`es have destructured just one layer. But what if the data structure is nested? Say, a tuple wrapping an `enum`, or an `enum` wrapping another `struct`?

Rust's pattern matching can destructure several layers at once — like peeling an onion, reaching in layer by layer.

For instance, given a tuple `(i32, Shape)`, you can destructure both the tuple and the `Shape` inside it in one `match`:

```
enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}

fn main() {
    let data = (666, Shape::Circle(42.0));
    match data {
```

```
        (id, Shape::Circle(r)) => println!("#{} is a circle with radius {}", id, r),
        (id, Shape::Rectangle(w, h)) => println!("#{} is a rectangle {}x{}", id, w, h),
    }
}
```

In a single pattern, the outer layer destructures the tuple to get `id` and the `Shape`, and the inner layer destructures the `Shape` to get the data inside. All in one line!

3.14.3 Example Code

```
enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}

struct Point {
    x: i32,
    y: i32,
}

fn main() {
    // Example 1: a tuple wrapping an enum
    let data = (1, Shape::Circle(5.0));

    match data {
        (id, Shape::Circle(r)) => {
            println!("Shape #{} is a circle with radius {}", id, r);
        }
        (id, Shape::Rectangle(w, h)) => {
            println!("Shape #{} is a rectangle {}x{}", id, w, h);
        }
    }

    // Example 2: a tuple wrapping a struct
    let item = ("origin", Point { x: 0, y: 0 });

    match item {
        (name, Point { x, y }) => {
            println!("{}: coordinates ({}, {})", name, x, y);
        }
    }
}
```

3.14.4 Recap

- Rust's pattern matching can destructure multiple layers of nesting.
- One pattern can destructure tuple + `enum`, tuple + `struct`, and so on, all at once.
- Nested destructuring saves you from writing multiple `matches` — everything comes out in one go.
- The syntax simply nests patterns layer by layer, mirroring the data's structure.

3.15 The _ Wildcard

3.15.1 Goal of This Episode

Learn to use _ to ignore values you don't care about, and to build a default arm in a `match`.

3.15.2 Concept

Sometimes in a `match` we only care about a few cases and want to "ignore" the rest. Rust offers _ (the underscore) as a **wildcard** — it matches any value without binding it to a variable.

The two most common uses:

1. The default arm: `_ => ...`

Placed at the end of a `match`, it means "every other case goes here":

```
fn main() {
    let score = 95;
    match score {
        100 => println!("Perfect score!"),
        _ => println!("Not a perfect score"),
    }
}
```

2. Ignoring a value at some position

In a tuple or `enum` pattern, use _ to hold a position you don't need:

```
fn main() {
    let point = (5, 5);
    match point {
        (0, _) => println!("On the y-axis"), // Don't care about the second value
        (_, 0) => println!("On the x-axis"), // Don't care about the first value
        (_, _) => println!("Somewhere else"),
    }
}
```

3.15.3 Example Code

```
enum Direction {
    Up,
    Down,
    Left,
    Right,
}

fn main() {
    // _ as the default arm
    let dir = Direction::Left;

    match dir {
        Direction::Up => println!("Going up"),
        _ => println!("Not up (maybe down, left, or right)"),
    }
```

```
    // _ ignoring a value inside a tuple
    let record = ("Alice", 95, 'A');

    match record {
        (name, _, _) => println!("The name is {}", name),
    }

    // Mixed usage
    let data = (1, Direction::Up);

    match data {
        (_, Direction::Up) => println!("The direction is up (whatever the number)"),
        (id, _) => println!("Number {} (direction isn't up)", id),
    }

    // _ as the default on an i32
    let score = 87;

    match score {
        100 => println!("Perfect score!"),
        0 => println!("Zero..."),
        _ => println!("Scored {} points", score),
    }
}
```

3.15.4 Recap

- _ is the wildcard: it matches any value without binding a variable.
- `_ => ...` at the end of a `match` is the "default arm," handling every unlisted case.
- Use _ in patterns to ignore fields you don't need.
- With _, a `match` no longer has to spell out every variant.

3.16 Ignoring Multiple Values with `..`

3.16.1 Goal of This Episode

Learn to use `..` to ignore multiple unneeded values in a `struct` or tuple at once.

3.16.2 Concept

Last episode we ignored one value with _. But what if a `struct` has many fields and you only care about one or two? Writing _ for every unwanted field is tedious.

Rust provides `..` (two dots), meaning "I don't want any of the rest."

3.16.2.1 In a `match` on a `struct`

```
struct Player {
    id: i32,
    hp: i32,
    mp: i32,
    level: i32,
}
```

```
fn main() {
    let p = Player { id: 1, hp: 0, mp: 50, level: 10 };

    match p {
        Player { hp: 0, .. } => println!("This player is down!"),
        Player { level, .. } => println!("Level {}", level),
    }
}
```

`Player { hp: 0, .. }` means "hp is 0; I don't care about the other fields." No need to write _ for every unwanted field.

`enum` `struct` variants can be matched this way too — exactly the same usage.

3.16.2.2 In a `match` on a Tuple

```
fn main() {
    let scores = (90, 85, 78, 92, 88);

    match scores {
        (first, ..) => println!("First subject: {}", first),
    }

    match scores {
        (.., last) => println!("Last subject: {}", last),
    }

    match scores {
        (first, .., last) => println!("First subject {}, last subject {}", first, last),
    }
}
```

`(first, ..)` takes only the first, `(.., last)` only the last, and `(first, .., last)` the head and tail.

Tuple `structs` and `enum` tuple variants can be matched similarly, e.g. `MyStruct(first, ..)` or `MyEnum::Variant(first, ..)`.

3.16.2.3 In Arrays and Slices

Episode 13 covered slice patterns; `..` works just as nicely in arrays and slices:

```
fn main() {
    let data: &[i32] = &[10, 20, 30, 40, 50];

    match data {
        [first, .., last] => println!("Head = {}, tail = {}", first, last),
        [only] => println!("Just one: {}", only),
        [] => println!("Empty"),
    }
}
```

3.16.2.4 Note: `..` Can Appear Only Once

`..` can appear only once within one layer of a pattern — with two, Rust wouldn't know how to distribute the values in between.

3.16.3 Example Code

```
struct Player {
    id: i32,
    hp: i32,
    mp: i32,
    level: i32,
}

fn main() {
    // .. on a struct
    let p1 = Player { id: 1, hp: 100, mp: 50, level: 10 };

    match p1 {
        Player { hp, .. } => println!("HP = {}", hp),
    }

    let p2 = Player { id: 2, hp: 0, mp: 30, level: 5 };

    match p2 {
        Player { hp: 0, .. } => println!("This player is already down!"),
        Player { level, .. } => println!("Level {}", level),
    }

    // .. on a tuple
    let scores = (90, 85, 78, 92, 88);

    match scores {
        (first, ..) => println!("First subject: {}", first),
    }

    match scores {
        (.., last) => println!("Last subject: {}", last),
    }

    match scores {
        (first, .., last) => println!("First subject {}, last subject {}", first, last),
    }

    // .. on a slice
    let data: &[i32] = &[10, 20, 30, 40, 50];

    match data {
        [first, .., last] => println!("Head = {}, tail = {}", first, last),
        [only] => println!("Just one: {}", only),
        [] => println!("Empty"),
    }
}
```

3.16.4 Recap

- `..` ignores multiple fields or values at once.
- On a `struct` in `match`: `Player { hp, .. }` takes only `hp` and ignores the rest; same for `enum` `struct` variants.
- On a tuple: `(first, ..)` takes only the first, `(.., last)` only the last; tuple `structs` and `enum` tuple variants use similar syntax.
- In arrays and slices: `[first, ..]` takes the first; `[first, .., last]` takes head and tail.
- `..` can appear only once per layer of a pattern.

3.17 Range Patterns

3.17.1 Goal of This Episode

Learn to match numeric values with ranges inside a `match`.

3.17.2 Concept

When we learned `match`, we compared values one at a time. But what if you want to match "any number between 1 and 5"? Surely not five separate arms.

Rust offers the **range pattern**, letting you match against ranges in a `match`:

```
fn main() {
    let score = 12;
    match score {
        1..=5 => println!("Low score"),
        _ => {}
    }
}
```

`1..=5` means 1, 2, 3, 4, 5 (both ends included). This `..=` means much the same as the `for i in 0..=5` from Chapter 1.

Besides `..=` (end included), you can use `..` (end excluded):

```
fn main() {
    let score = 65;
    match score {
        0..50 => println!("Failing"),   // 0 through 49
        50..=100 => println!("Passing"), // 50 through 100 (inclusive)
        _ => {}
    }
}
```

3.17.2.1 Careful: Don't Confuse the Two Kinds of `..`!

Last episode's `..` and this episode's `..` look identical but mean completely different things:

- **Last episode**: `Point { x, .. }` → ignore remaining fields; `..` means "I don't care about the rest."
- **This episode**: `0..50` → a numeric range; `..` means "from one number to another."

The Rust compiler tells them apart from context and never confuses them. But as a beginner,

take care to distinguish the two.

3.17.2.2 One-sided Ranges

Range patterns also support writing just one side:

```
fn main() {
    let temperature = 25;
    match temperature {
        ..0 => println!("Below zero"),   // Less than 0
        0..=30 => println!("Ordinary"),  // 0 through 30
        31.. => println!("Very hot"),    // 31 and up
    }
}
```

3.17.2.3 `char` Works Too

Range patterns aren't just for numbers — they work on `char` too:

```
fn main() {
    let c = '哼';
    match c {
        'a'..='z' => println!("A lowercase English letter"),
        'A'..='Z' => println!("An uppercase English letter"),
        '0'..='9' => println!("A digit"),
        _ => println!("Some other character"),
    }
}
```

3.17.3 Example Code

```
fn main() {
    // Grading scores with range patterns
    let score = 78;

    match score {
        90..=100 => println!("A"),
        80..90 => println!("B"),
        70..80 => println!("C"),
        60..70 => println!("D"),
        0..60 => println!("F"),
        _ => println!("Score out of range"),
    }

    // One-sided ranges
    let temperature = -5;

    match temperature {
        ..0 => println!("Below zero — freezing!"),
        0..=35 => println!("Tolerable"),
        36.. => println!("Too hot!"),
    }

    // Range patterns on char
    let c = 'G';
```

```
    match c {
        'a'..='z' => println!("'{}' is a lowercase letter", c),
        'A'..='Z' => println!("'{}' is an uppercase letter", c),
        '0'..='9' => println!("'{}' is a digit", c),
        _ => println!("'{}' is some other character", c),
    }
}
```

3.17.4 Recap

- `..` and `..=` work not only in `for` loops but also as patterns.
- `1..=5` → both ends included (1, 2, 3, 4, 5).
- `0..50` → start included, end excluded (0 through 49).
- `..0` → less than 0; `31..` → 31 and up (one-sided ranges).
- `char` supports range patterns too: `'a'..='z'`.
- This `..` is a "numeric range" — a different thing from last episode's field-ignoring `..`; don't mix them up.

3.18 Multiple Values with |

3.18.1 Goal of This Episode

Learn to match several possible values in one `match` arm.

3.18.2 Concept

Sometimes you want several values to run the same code. For instance, Saturday and Sunday are both days off — no need for two separate arms.

Rust uses | (the pipe symbol) to mean "or":

```
fn main() {
    let day = 1;
    match day {
        6 | 7 => println!("Day off"),
        _ => println!("Workday"),
    }
}
```

6 | 7 means "6 or 7." You can chain as many values as you like with |:

```
fn main() {
    let n = 3;
    match n {
        1 | 2 | 3 => println!("Top three"),
        _ => println!("Other"),
    }
}
```

It works with enums too:

```
enum Color {
    Red,
```

```
    Green,
    Blue,
}

fn main() {
    let color = Color::Red;
    match color {
        Color::Red | Color::Blue => println!("Warm or cool color"),
        Color::Green => println!("Green"),
    }
}
```

3.18.3 Example Code

```
enum Season {
    Spring,
    Summer,
    Autumn,
    Winter,
}

fn main() {
    // Matching multiple numbers
    let month = 7;

    match month {
        3 | 4 | 5 => println!("Spring"),
        6 | 7 | 8 => println!("Summer"),
        9 | 10 | 11 => println!("Autumn"),
        12 | 1 | 2 => println!("Winter"),
        _ => println!("Invalid month"),
    }

    // Matching multiple enum variants
    let s = Season::Autumn;

    let is_hot = match s {
        Season::Summer => true,
        Season::Spring | Season::Autumn | Season::Winter => false,
    };
    println!("Is the weather hot? {}", is_hot);

    // Combining range patterns with |
    let ch = '5';

    match ch {
        'a'..='z' | 'A'..='Z' => println!("A letter"),
        '0'..='9' => println!("A digit"),
        ' ' | '\t' | '\n' => println!("Whitespace"),
        _ => println!("Other"),
    }
}
```

3.18.4 Recap

- In `match`, `|` means "or," letting one arm match multiple values.
- Syntax: `pattern1 | pattern2 | pattern3 => ...`.
- Works with `enum` variants.
- Works with range patterns too: `'a'..='z' | 'A'..='Z'`.
- When several values need the same handling, `|` beats writing multiple arms.

3.19 @ Bindings

3.19.1 Goal of This Episode

Learn to use `@` to bind the matching value to a variable while matching a pattern.

3.19.2 Concept

We have already learned range patterns: `0..=100` matches values from 0 through 100. Now suppose the `SetVolume` variant of a `Command` carries a volume. We want to check that volume's range and print its exact value inside the arm:

```
enum Command {
    SetVolume(i32),
    SetBrightness(i32),
    Quit,
}

fn main() {
    let command = Command::SetVolume(72);

    match command {
        Command::SetVolume(level @ 0..=100) => {
            println!("Set the volume to {}", level);
        }
        Command::SetVolume(level) => {
            println!("Volume {} is out of range", level);
        }
        Command::SetBrightness(level) => {
            println!("Set the brightness to {}", level);
        }
        Command::Quit => println!("Quit"),
    }
}
```

Inside `Command::SetVolume(level @ 0..=100)`, the range `0..=100` on the right performs the match, while `level` on the left binds the exact volume. When the value is `Command::SetVolume(72)`, the pattern matches and `level` is 72 inside the arm.

This is the syntax of an `@` binding:

```
variable_name @ pattern
```

The left side creates a binding, and the right side performs the match. After the pattern matches, the variable on the left can be used inside the arm.

`@` can be used with other patterns too, including `|`:

In the example below, the first arm uses `('a' | 'e' | 'i' | 'o' | 'u')` to match a lowercase vowel, then binds the matching character to `key`. When the pattern to the right of `@` uses `|`, that group must be wrapped in parentheses.

The `MouseClick` arm demonstrates an `@` binding inside a field of a `struct` variant. `0..=10` matches the range of the `x` field, and `horizontal` binds the exact coordinate that matched.

3.19.3 Example Code

```
enum Event {
    KeyPress(char),
    MouseClick { x: i32, y: i32 },
    Quit,
}

fn main() {
    let event = Event::MouseClick { x: 6, y: 30 };

    match event {
        Event::KeyPress(key @ ('a' | 'e' | 'i' | 'o' | 'u')) => {
            println!("Pressed the lowercase vowel '{}'", key);
        }
        Event::KeyPress(key @ 'a'..='z') => {
            println!("Pressed another lowercase letter '{}'", key);
        }
        Event::KeyPress(key) => {
            println!("Pressed another key '{}'", key);
        }
        Event::MouseClick {
            x: horizontal @ 0..=10,
            y,
        } => {
            println!("Clicked in the left area: ({}, {})", horizontal, y);
        }
        Event::MouseClick { x, y } => {
            println!("Clicked elsewhere: ({}, {})", x, y);
        }
        Event::Quit => println!("Quit"),
    }
}
```

3.19.4 Recap

- `variable_name @ pattern` checks the pattern on the right; after a successful match, the actual value is bound to the variable on the left.
- `Command::SetVolume(level @ 0..=100)` both restricts the volume range and captures the exact volume.
- `@` can be used inside nested data such as `enum` variants and `struct` fields.
- `@` works with ranges, `|`, and other patterns. With `|`, write `value @ (pattern1 | pattern2)`.

3.20 `match` Guards

3.20.1 Goal of This Episode

Learn to add extra conditional checks (guards) to `match` arms.

3.20.2 Concept

Patterns are good at checking the shape of data, fixed values, and ranges. However, a pattern does not evaluate comparisons between fields such as `from == to`, and a variable created with `let` cannot be used as the bound of a range pattern. A `match` guard handles these extra computations.

A **`match` guard** adds an `if condition` after a pattern:

```
pattern if condition => ...
```

For example, a sensor reading carries a room number and a measured value. A pattern can first destructure those fields, and a guard can then check whether the measurement has crossed a warning level:

```rust
enum Reading {
    Temperature { room: i32, celsius: i32 },
    Humidity { room: i32, percent: i32 },
    Offline { room: i32 },
}

fn main() {
    let reading = Reading::Temperature {
        room: 3,
        celsius: 34,
    };
    let heat_warning = 30;

    match reading {
        Reading::Temperature { room, celsius } if celsius >= heat_warning => {
            println!("Room {} is too hot: {} degrees", room, celsius);
        }
        Reading::Temperature { room, celsius } => {
            println!("Room {} has a normal temperature: {} degrees", room, celsius);
        }
        Reading::Humidity { room, percent } if percent > 70 => {
            println!("Room {} is too humid: {}%", room, percent);
        }
        Reading::Humidity { room, percent } => {
            println!("Room {} has normal humidity: {}%", room, percent);
        }
        Reading::Offline { room } => {
            println!("The sensor in room {} is offline", room);
        }
    }
}
```

The first arm happens in two steps:

1. `Reading::Temperature { room, celsius }` first confirms that the value is a `Temperature` reading and binds its two fields to `room` and `celsius`.
2. `if celsius >= heat_warning` then uses the newly bound `celsius` in an extra check.

After the pattern matches, `room` and `celsius` are available in both the guard and the code on the right. A guard can also use variables that existed before the pattern, such as `heat_warning` above.

3.20.3 A Failed Guard Tries the Next Arm

A matching pattern does not necessarily mean that its arm runs. If the guard is `false`, Rust continues trying the later arms.

For the temperature example:

- A `Temperature` with `celsius >= heat_warning` runs the first arm.
- If it is a `Temperature` below the warning level, the first guard fails and the second `Temperature` arm handles it.
- If it is not a `Temperature` at all, neither of those two patterns matches, so Rust continues looking for another variant.

An arm with a guard therefore usually goes before the more general arm that handles its remaining cases.

A guard can compare not only one field with a limit, but also multiple fields bound by the same pattern:

In the example below, the first guard compares `from` with `to`, while the second compares `amount` with the outer `daily_limit`. Conditions involving calculations between fields or values chosen at runtime are where guards are more appropriate than plain patterns.

3.20.4 Example Code

```
enum Request {
    Transfer {
        from: i32,
        to: i32,
        amount: i32,
    },
    CheckBalance {
        account: i32,
    },
}

fn main() {
    let request = Request::Transfer {
        from: 1001,
        to: 2002,
        amount: 1500,
    };
    let daily_limit = 1000;

    match request {
```

```
        Request::Transfer { from, to, amount } if from == to => {
            println!(
                "Account {} does not need to transfer {} to itself",
                from, amount
            );
        }
        Request::Transfer { from, to, amount } if amount > daily_limit => {
            println!(
                "Transfer {} from account {} to account {}: extra confirmation required",
                amount, from, to
            );
        }
        Request::Transfer { from, to, amount } => {
            println!(
                "Transfer {} from account {} to account {}",
                amount, from, to
            );
        }
        Request::CheckBalance { account } => {
            println!("Check the balance of account {}", account);
        }
    }
}
```

3.20.5 Guards and Exhaustiveness

Suppose we write two arms for `Temperature`:

- One guard is `celsius >= heat_warning`.
- The other guard is `celsius < heat_warning`.

We can see that every temperature must satisfy one of those conditions, so the two arms logically cover every possibility. However, Rust's exhaustiveness check may not be able to infer from the logical relationship between the guards that every temperature has been handled. Even with both arms present, the compiler may still consider `Temperature` not fully covered.

To make the coverage clear to the compiler as well, keep a **pattern without a guard**. That is why the second `Temperature` arm in the earlier example has no guard: the first arm handles temperatures at or above the warning level, and the second catches every remaining temperature. The `Humidity` and `Transfer` arms follow the same arrangement.

3.20.6 Recap

- A `match` guard has the syntax `pattern if condition => ...`.
- Rust first matches the pattern and creates its bindings, then checks the guard.
- A guard can use variables bound by the same pattern as well as variables that already exist outside it.
- If the pattern matches but the guard is `false`, Rust continues trying the later arms.
- Even when several guards logically cover every possibility, the compiler may still need an arm without a guard to confirm that the `match` is exhaustive.
- Guards are especially useful for comparisons between fields, calculations, and comparisons with runtime limits.

3.21 Destructuring Tuples with `let`

3.21.1 Goal of This Episode

Learn to use `let` to break a tuple apart directly, assigning its values to separate variables.

3.21.2 Concept

We've learned to destructure tuples inside `match`, like `(x, y) => ...`. But actually, **you don't need `match` — `let` can destructure directly**!

```
    let (x, y) = (1, 2);
```

This one line does two things:

1. Creates the tuple `(1, 2)`.
2. Takes the first value out as `x` and the second as `y`.

Back in Chapter 2, we always used `t.0` and `t.1` to access tuples. Now, with destructuring, one line splits all the values apart, each with a readable name.

The `_` and `..` we learned earlier work in `let` destructuring too.

3.21.3 `mut` on Bindings

In Chapter 1 we learned `let mut x = 5;`. In fact, `mut` isn't part of the type — it's a modifier on the **binding**.

Since `let` destructuring is doing bindings, you can naturally put `mut` on individual variables:

```
    let (mut a, b) = (1, 2);
    a += 10; // OK, a is mutable
    b += 10; // Error, b is immutable
```

Within one pattern, some variables can take `mut` and others not — each independent.

This rule isn't limited to `let`: **anywhere a variable is bound, `mut` can be added**:

- match arms: `Some(mut x) => { x += 1; }`.
- for loops: `for mut x in [1, 2, 3] { ... }`.
- function parameters: `fn foo(mut x: i32) { x += 1; }`.

The same goes for every binding construct we'll learn later. It's all one thing — `mut` modifies the binding, not the type.

3.21.4 Example Code

```
fn main() {
    // Basic let destructuring
    let (x, y) = (10, 20);
    println!("x = {}, y = {}", x, y);

    // Three-value tuples work too
    let (name, score, grade) = ("Ming", 95, 'A');
    println!("{} scored {} points, grade {}", name, score, grade);
```

```
    // Combine with _ to ignore one value
    let (_, second, _) = (1, 2, 3);
    println!("Just the second: {}", second);

    // Combine with .. to ignore several values
    let (first, ..) = (100, 200, 300, 400);
    println!("Just the first: {}", first);

    // mut on individual bindings
    let (mut a, b) = (1, 2);
    a += 10;
    println!("a = {}, b = {}", a, b);

    // A function returning a tuple, destructured directly
    let (min, max) = min_max(7, 3);
    println!("Smallest {}, largest {}", min, max);
}

fn min_max(a: i32, b: i32) -> (i32, i32) {
    if a < b {
        (a, b)
    } else {
        (b, a)
    }
}
```

3.21.5 Recap

- `let (x, y) = (1, 2);` breaks a tuple apart directly.
- Destructuring a tuple sometimes reads better than `.0` and `.1`.
- Combine with _ to ignore single values, or `..` to ignore several.
- `mut` modifies the binding, not the type — any binding position can take `mut`.
- When a function returns a tuple, `let` destructuring extracts every value at once.

3.22 Destructuring `structs` with `let`

3.22.1 Goal of This Episode

Learn to use `let` to break a `struct`'s fields apart directly, assigning them to variables.

3.22.2 Concept

Last episode we destructured tuples with `let`; now let's destructure `structs`. The idea is exactly the same — one `let` splits the `struct`'s fields apart:

```
    let Point { x, y } = p;
```

This line puts the value of `p.x` into the variable `x` and `p.y` into `y`. It uses field shorthand (from Episode 11), so `x` is both the field name and the variable name.

If you want a variable name different from the field name, use the `field_name: variable_name` form:

```
    let Point { x: px, y: py } = p;
    // The variables are now called px and py
```

The `..` from earlier works too, taking only the fields you need:

```
    let Point { x, .. } = p;
    // Take only x; ignore the other fields
```

Tuple `structs` can be destructured too — nearly identical to tuple patterns, just with the type name in front:

```
struct Pair(i32, i32);

fn main() {
    let p = Pair(1, 2);
    let Pair(a, b) = p;
}
```

3.22.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

struct Rectangle {
    x: i32,
    y: i32,
    width: i32,
    height: i32,
}

fn main() {
    let p = Point { x: 5, y: 10 };

    // let destructuring a struct (with field shorthand)
    let Point { x, y } = p;
    println!("x = {}, y = {}", x, y);

    // With different variable names
    let p2 = Point { x: 3, y: 7 };
    let Point { x: px, y: py } = p2;
    println!("px = {}, py = {}", px, py);

    // With .. to take only some fields
    let rect = Rectangle { x: 0, y: 0, width: 100, height: 50 };
    let Rectangle { width, height, .. } = rect;
    println!("Width {}, height {}", width, height);
    let area = width * height;
    println!("Area = {}", area);
}
```

3.22.4 Recap

- `let Point { x, y } = p;` splits a `struct`'s fields into separate variables.
- `..` ignores fields you don't need.
- Tuple `structs` destructure too: `let Pair(a, b) = p;`.
- `let` destructuring is extremely handy for pulling data out of a `struct`.

3.23 Destructuring in `for` Loops

3.23.1 Goal of This Episode

Learn to destructure tuples or `structs` directly in the variable position of a `for` loop.

3.23.2 Concept

We've already seen `let` destructure tuples and `structs`. It turns out the variable position of a `for` loop can too — just write the same destructuring syntax right there.

Iterating over an array of tuples:

```
fn main() {
    let pairs = [(1, "one"), (2, "two"), (3, "three")];

    for (num, name) in pairs {
        println!("{} = {}", num, name);
    }
}
```

`(num, name)` is the pattern. Each element of the array is a tuple, and the loop splits it into `num` and `name`.

Iterating over `structs` works the same way:

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let points = [
        Point { x: 0, y: 0 },
        Point { x: 1, y: 2 },
        Point { x: 3, y: 4 },
    ];

    for Point { x, y } in points {
        println!("({}, {})", x, y);
    }
}
```

Think of it as `let` destructuring combined with a `for` loop: each time the loop takes out an element, it splits it apart with `let`-destructuring syntax.

3.23.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    // Iterate over a tuple array, destructuring
    let scores = [("Alice", 85), ("Bob", 92), ("Carol", 78)];
    for (name, score) in scores {
        println!("{}: {}", name, score);
    }

    // Iterate over a struct array, destructuring
    let points = [
        Point { x: 0, y: 0 },
        Point { x: 3, y: 4 },
        Point { x: -1, y: 2 },
    ];
    for Point { x, y } in points {
        println!("({}, {})", x, y);
    }

    // Use .. to ignore unwanted fields
    let more_points = [
        Point { x: 1, y: 10 },
        Point { x: 2, y: 20 },
    ];
    for Point { x, .. } in more_points {
        println!("x = {}", x);
    }
}
```

3.23.4 Recap

- The variable position of a `for` loop accepts destructuring patterns directly.
- Iterating over an array of tuples: `for (a, b) in pairs`.
- Iterating over an array of `structs`: `for Point { x, y } in points`.

3.24 Destructuring in Function Parameters

3.24.1 Goal of This Episode

Learn to destructure tuples or `structs` directly in a function's parameter position.

3.24.2 Concept

We've learned to destructure in `let`, `match`, and `for`. Well, **function parameters can destructure too**!

Suppose you have a function that receives a tuple `(i32, i32)` representing coordinates. Rather than splitting it apart inside the function, split it right in the parameter position:

```
fn print_point((x, y): (i32, i32)) {
    println!("({}, {})", x, y);
}
```

Note the syntax: `(x, y)` is the pattern, and `: (i32, i32)` is the type annotation. Pattern and type are separated by `:`.

Calling works as usual — pass a tuple in:

```
fn print_point((x, y): (i32, i32)) {
    println!("({}, {})", x, y);
}
fn main() {
    print_point((3, 7));
}
```

`structs` can be destructured in the parameter position too:

```
fn print_point_struct(Point { x, y }: Point) {
    println!("({}, {})", x, y);
}
```

3.24.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

// A function destructuring tuples in its parameters
fn add_coordinates((x1, y1): (i32, i32), (x2, y2): (i32, i32)) -> (i32, i32) {
    (x1 + x2, y1 + y2)
}

// A function destructuring a struct in its parameter
// Of course, you could also choose to use match here
fn describe_point(Point { x, y }: Point) {
    if x == 0 && y == 0 {
        println!("The origin");
    } else if x == 0 {
        println!("On the y-axis, y = {}", y);
    } else if y == 0 {
        println!("On the x-axis, x = {}", x);
    } else {
        println!("An ordinary point ({}, {})", x, y);
    }
}

fn main() {
    // Passing tuples to the function
    let a = (1, 2);
    let b = (3, 4);
    let result = add_coordinates(a, b);
    println!("({}, {}) + ({}, {}) = ({}, {})", a.0, a.1, b.0, b.1, result.0, result.1);
```

```
    // Passing structs to the function
    let p = Point { x: 0, y: 5 };
    describe_point(p);

    let origin = Point { x: 0, y: 0 };
    describe_point(origin);
}
```

3.24.4 Why Can Tuples and `structs` Be Destructured with `let`?

You might wonder: why is it that tuples and `structs` can be destructured directly in `let`, `for`, and function parameters?

```
    let (x, y) = (1, 2);    // OK
    let Point { x, y } = p; // OK
    let Shape::Circle { radius } = s; // Not allowed!
```

The answer: **the tuple and `struct` patterns used in this episode cannot fail to match**. Any `(i32, i32)` matches `(x, y)`, and any `Point` matches `Point { x, y }`.

`enums` are different. A `Shape` might be a `Circle` or a `Rectangle`. If you write `let Shape::Circle { radius } = s;` but `s` is actually a `Rectangle`, it fails. Rust doesn't allow patterns that can fail in a `let`.

Patterns that always succeed are called **irrefutable patterns**; ones that can fail are **refutable patterns**. `let`, `for`, and function parameters accept only irrefutable patterns.

What other irrefutable patterns are there?

```
fn main() {
    let arr = [1, 2, 3];
    let [head, ..] = arr;
    println!("The first element is {}", head);
}
```

`arr` has type `[i32; 3]`; the compiler can see at a glance that it always has three elements, so `[head, ..]` always matches — it's also an irrefutable pattern, and `let` destructuring works. Conversely, with a slice `&[i32]` it wouldn't: a slice might be empty, making `[head, ..]` refutable on slices, and `let` can't take it.

Want to handle refutable patterns? Next episode covers `if let`.

3.24.5 Recap

- Function parameters can destructure with patterns directly: `fn foo((x, y): (i32, i32))`.
- Both tuples and `structs` can be destructured in the parameter position.
- Calls look the same as always; destructuring is the function's internal business.
- `let`, `for`, and function parameters accept only patterns that can't fail (irrefutable patterns), so this episode's `(x, y)` and `Point { x, y }` work, but `Shape::Circle { radius }` doesn't.

3.25 if let

3.25.1 Goal of This Episode

Learn to use `if let` to simplify a `match` where you only care about one pattern.

3.25.2 Concept

Sometimes you only care about one variant of an `enum`, and the rest don't matter. With `match`, you must handle every case, even when you only want to handle one:

```
    match c {
        Color::Red => println!("It's red!"),
        _ => {} // Do nothing in every other case
    }
```

That `_ => {}` looks redundant. Rust offers the `if let` syntax to simplify this situation:

```
    if let Color::Red = c {
        println!("It's red!");
    }
```

`if let pattern = value` means "if this value fits this pattern, run the code in the braces."

You can add an `else` to handle the non-matching case:

```
    if let Color::Red = c {
        println!("It's red!");
    } else {
        println!("Not red");
    }
```

Note: the `=` in `if let` is a single equals sign, not two. This isn't a comparison — it's "pattern matching."

3.25.3 Example Code

```
enum Color {
    Red,
    Green,
    Blue,
}

enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}

fn main() {
    let c = Color::Red;

    // Checking whether it's Red with if let
    if let Color::Red = c {
        println!("It's red!");
    }
```

```
    // With else
    let c2 = Color::Blue;

    if let Color::Red = c2 {
        println!("It's red!");
    } else {
        println!("Not red");
    }

    // if let can extract a variant's data too
    let s = Shape::Circle(5.0);

    if let Shape::Circle(r) = s {
        println!("It's a circle! Radius = {}", r);
        let area = r * r * 3.14159;
        println!("Area roughly {}", area);
    }

    // If it's not a Circle, the if-let body won't run
    let s2 = Shape::Rectangle(10, 20);

    if let Shape::Circle(r) = s2 {
        println!("This line never runs, because s2 is a Rectangle");
        println!("Radius {}", r);
    } else {
        println!("Not a circle");
    }
}
```

3.25.4 `if let` Guards

`if let` can also appear in a `match` guard position (the `match` guards from Episode 20). The syntax is `pattern if let pattern2 = expression =>`:

```
enum Wrapper {
    Value(i32),
    Empty,
}

fn lookup(key: i32) -> Wrapper {
    if key > 0 { Wrapper::Value(key * 10) } else { Wrapper::Empty }
}

fn main() {
    let items = [1, -2, 3];

    for item in items {
        match item {
            x if let Wrapper::Value(v) = lookup(x) => {
                println!("{} found: {}", x, v);
            }
            x => println!("{} not found", x),
        }
    }
```

```
}
```

`x if let Wrapper::Value(v) = lookup(x)` means: first bind the value to `x`, then pattern-match again on the result of `lookup(x)` — the arm applies only when that result is `Wrapper::Value(v)`.

This example could be written with an ordinary `if let` too. But when the logic gets more complex — say the outer `match` is already comparing other patterns and you need to pattern-match another value within one arm — an `if let` guard can sometimes read better than nesting another `if let` inside a `match` arm.

3.25.5 Recap

- `if let pattern = value { ... }` is shorthand for a `match` with just one arm.
- The braces run only when the value fits the pattern.
- An `else` can handle the non-matching case.
- Patterns can extract data, as in `if let Shape::Circle(r) = s`.
- Compared to `match` + `_ => {}`, `if let` is more concise.
- `if let` also works in `match` guards: `pattern if let pattern2 = expression => ...`.

3.26 while let

3.26.1 Goal of This Episode

Learn to use `while let` to keep pattern matching in a loop until the pattern no longer fits.

3.26.2 Concept

Last episode we learned `if let` — "if it matches, run once." `while let` is "as long as it matches, keep running" — the loop version of `if let`.

Syntax:

```
while let pattern = value {
    // Loop body
}
```

Before each iteration, Rust checks "does the value fit the pattern?" If yes, keep going; if not, stop.

To demonstrate `while let`, we'll use a custom `enum` to simulate a "maybe there's a value, maybe we're done" situation:

```
enum Step {
    Value(i32),
    Done,
}
```

3.26.3 Example Code

```
enum Step {
    Value(i32),
    Done,
}

fn get_step(index: i32) -> Step {
    if index < 5 {
        Step::Value(index * 10)
    } else {
        Step::Done
    }
}

fn main() {
    let mut i = 0;

    // while let: keep going as long as get_step returns Value
    while let Step::Value(v) = get_step(i) {
        println!("Step {}, value = {}", i, v);
        i += 1;
    }
    println!("Done! Ran {} steps in total", i);

    println!();

    // Another example: a countdown
    let mut count = 5;

    // Simulating a countdown with a custom enum
    while let Countdown::Tick(n) = get_countdown(count) {
        println!("Counting down {}...", n);
        count -= 1;
    }
    println!("Liftoff! 🚀");
}

enum Countdown {
    Tick(i32),
    Launch,
}

fn get_countdown(n: i32) -> Countdown {
    if n > 0 {
        Countdown::Tick(n)
    } else {
        Countdown::Launch
    }
}
```

3.26.4 Recap

- `while let pattern = value { ... }` is the loop version of `if let`.
- As long as the value fits the pattern, the loop keeps running.

- When the value no longer fits, the loop ends automatically.

3.27 let else

3.27.1 Goal of This Episode

Learn to use `let...else...` to bail out early when a pattern doesn't match, writing flatter code.

3.27.2 Concept

3.27.2.1 The Flip Side of `if let`

Last episode was `while let`, and before that `if let` — "if the match succeeds, do something." But sometimes you want the reverse: "if the match **fails**, leave early; if it succeeds, keep going."

Suppose we have this `enum`:

```
enum Color {
    Red,
    Green,
    Blue,
    Custom(i32, i32, i32),
}
```

Written with `if let`:

```
fn describe(color: Color) {
    if let Color::Custom(r, g, b) = color {
        println!("Custom color: {} {} {}", r, g, b);
    } else {
        println!("Not a custom color; ending");
        return;
    }
    // We'd like to use r, g, b here... but they're already out of scope!
}
```

`r`, `g`, and `b` live only inside the `if let`'s `{}` — the code afterward can't touch them.

3.27.2.2 The `let...else...` Syntax

`let...else...` makes the bound variables live in the code that follows, rather than only inside `{}`:

```
fn describe(color: Color) {
    let Color::Custom(r, g, b) = color else {
        println!("Not a custom color; ending");
        return;
    };
    // r, g, b are directly usable here!
    println!("Red: {}, green: {}, blue: {}", r, g, b);
}
```

Meaning:

1. Try to match `color` against the pattern.

2. On success, `r`, `g`, `b` are bound and the program continues downward.
3. On failure, the code inside `else` runs.

3.27.2.3 The `else` Must Leave

The `else` block can't just "do a bit of work and continue" — it must make the program leave the current flow. Legal options include:

- `return` — leave the function
- `break` — leave the loop
- `continue` — skip to the loop's next iteration

Why? Because if the pattern doesn't match, the variables were never bound. If the program kept running after the `else`, those variables would be undefined — and Rust doesn't allow that.

3.27.2.4 Comparison with `if let`

- `if let`: enter the {} block only on a successful match; bound variables live only inside.
- `let...else...`: leave on a failed match; bound variables live in all the code that follows.

`let...else...` keeps code flatter — no extra level of indentation.

3.27.3 Example Code

```
enum Shape {
    Circle(f64),
    Rectangle(i32, i32),
}

fn print_circle_info(shape: Shape) {
    let Shape::Circle(radius) = shape else {
        println!("Not a circle; skipping");
        return;
    };
    // radius is directly usable here
    println!("Circle, radius = {}", radius);
}

fn main() {
    print_circle_info(Shape::Circle(3.14));
    print_circle_info(Shape::Rectangle(10, 20));

    // With continue inside a loop
    let shapes = [
        Shape::Rectangle(3, 4),
        Shape::Circle(1.0),
        Shape::Rectangle(5, 6),
        Shape::Circle(2.5),
    ];

    println!("\nPrinting only the circles:");
    for shape in shapes {
        let Shape::Circle(r) = shape else {
            continue;  // Not a circle; skip this round
        };
```

```
        println!("Radius: {}", r);
    }
}
```

3.27.4 Recap

- `let pattern = expr else { return / break / continue };` leaves early when the match fails.
- The `else` must exit the current flow (`return` / `break` / `continue`).
- On a successful match, the bound variables remain usable in the code that follows.
- Better suited than `if let` to "fail → leave, succeed → continue" scenarios — the code stays flatter.

3.28 Associated Functions

3.28.1 Goal of This Episode

Learn to define associated functions for a `struct` or `enum` with `impl`, and call them with `::`.

3.28.2 Concept

So far, all our functions have been "standalone" — defined at the top level, unrelated to any type. But often, certain functions are closely tied to a specific type. For example, "create a new `Point`" relates directly to the `Point` type.

Rust uses `impl` blocks to let you "attach" functions to a type:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x, y }
    }
}
```

A function defined this way is called an **associated function**, because it's "associated" with the `Point` type. Call it with `::`:

```
    let p = Point::new(3, 7);
```

Does `Point::new` look a little familiar? We used `::` with `enum`s too — like `Color::Red`. It's the same concept: `::` means "something under a type."

The most common use of associated functions is `new` — a "constructor" for creating values of the type.

3.28.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

impl Point {
```

```
    // Associated function: create a new Point
    fn new(x: i32, y: i32) -> Point {
        Point { x, y }
    }

    // Other associated functions can be defined too
    fn origin() -> Point {
        Point { x: 0, y: 0 }
    }
}

// enums can have impl too!
enum Color {
    Red,
    Green,
    Blue,
}

impl Color {
    fn from_number(n: i32) -> Color {
        match n {
            0 => Color::Red,
            1 => Color::Green,
            _ => Color::Blue,
        }
    }
}

fn main() {
    // Calling associated functions with ::
    let p1 = Point::new(3, 7);
    println!("p1 = ({}, {})", p1.x, p1.y);

    let p2 = Point::origin();
    println!("p2 = ({}, {})", p2.x, p2.y);

    // An enum's associated function
    let c = Color::from_number(1);
    match c {
        Color::Red => println!("Red"),
        Color::Green => println!("Green"),
        Color::Blue => println!("Blue"),
    }
}
```

3.28.4 Recap

- `impl TypeName { ... }` defines associated functions for a type.
- Associated functions are called with `TypeName::function_name()`.
- The most common use is a `new` function serving as a constructor.
- Both `structs` and `enums` can have `impl` blocks.

3.29 Methods

3.29.1 Goal of This Episode

Learn to define methods with `self`, so functions can be called on a value with `.`.

3.29.2 Concept

Last episode we learned associated functions, called with `::` and tied to the "type." But sometimes we want to operate on a value that **already exists** — say, "compute this `Point`'s `x + y`."

That's a **method** — the first slot in the parameter list is `self`, standing for "the value this method was called on":

```
impl Point {
    fn sum(self) -> i32 {
        self.x + self.y
    }
}
```

Call it with `.` rather than `::`:

```
    let p = Point::new(3, 7);
    let s = p.sum(); // Calling the method with .
```

Note: when calling `p.sum()`, **you don't pass `self` manually**. The `p` before the `.` automatically becomes the method's `self`. So although the definition says `fn sum(self)`, the call is just `p.sum()`, not `p.sum(p)`.

3.29.2.1 Methods Can Take Other Parameters

Besides `self`, a method can take one or more other parameters — just like an ordinary function:

```
impl Point {
    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}
```

When calling, `self` comes automatically from the value before the `.`; you only pass the remaining parameters:

```
    let p1 = Point::new(1, 2);
    let p2 = Point::new(3, 4);
    let p3 = p1.add(p2); // p1 is self, p2 is other
```

3.29.2.2 The Difference between Associated Functions and Methods:

- Associated function: no `self`, called with `::` → `Point::new(3, 7)`.
- Method: first parameter is `self`, called with `.` → `p.sum()`.

3.29.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

impl Point {
    // Associated function (no self)
    fn new(x: i32, y: i32) -> Point {
        Point { x, y }
    }

    // Method (first parameter is self)
    fn sum(self) -> i32 {
        self.x + self.y
    }

    // Methods can take parameters beyond self
    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }

    // Another method
    fn is_origin(self) -> bool {
        self.x == 0 && self.y == 0
    }
}

enum Direction {
    Up,
    Down,
    Left,
    Right,
}

impl Direction {
    // enums can have methods too
    fn is_horizontal(self) -> bool {
        match self {
            Direction::Left => true,
            Direction::Right => true,
            Direction::Up => false,
            Direction::Down => false,
        }
    }
}

fn main() {
    let p = Point::new(3, 7); // :: calls the associated function
    let s = p.sum();          // . calls the method
    println!("3 + 7 = {}", s);
```

```
    // A method with an extra parameter
    let a = Point::new(1, 2);
    let b = Point::new(10, 20);
    let c = a.add(b); // a is self, b is other
    println!("After adding: ({}, {})", c.x, c.y);

    let origin = Point::new(0, 0);
    println!("Is it the origin? {}", origin.is_origin());

    // An enum's method
    let dir = Direction::Left;
    let horizontal = dir.is_horizontal();
    println!("Is it horizontal? {}", horizontal);
}
```

3.29.4 Recap

- A method's first parameter is `self`, standing for the value itself.
- Methods are called with `.`: `p.sum()` — the value before the `.` automatically becomes `self`; no manual passing.
- Methods can take parameters beyond `self`: `fn add(self, other: Point) -> Point`; when calling, the parentheses hold only the non-`self` arguments.
- Both `structs` and `enums` can have methods.

3.30 Capital `Self`

3.30.1 Goal of This Episode

Learn to use capital `Self` as an alias for "the type currently being `impl`ed," making code more concise.

3.30.2 Concept

Last episode we wrote code like this inside an `impl`:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x, y }
    }
}
```

Notice that the name `Point` appears three times: `impl Point`, `-> Point`, `Point { x, y }`. If the type name were long (say `Rectangle`), repeating it would get wordy.

Rust provides capital `Self` (note the capital S!), which inside an `impl` block stands for "the type currently being `impl`ed." So the code above can become:

```
impl Point {
    fn new(x: i32, y: i32) -> Self {
        Self { x, y }
    }
}
```

`Self` is an alias for `Point`. Two benefits:

1. More concise, especially with long type names.
2. If you rename the type later, nothing inside the `impl` needs changing.

Keep these apart:

- Lowercase `self`: "this value itself" (a method's first parameter).
- Capital `Self`: "the current type."

3.30.3 Example Code

```
struct Point {
    x: i32,
    y: i32,
}

impl Point {
    // Self in place of Point
    fn new(x: i32, y: i32) -> Self {
        Self { x, y }
    }

    fn origin() -> Self {
        Self { x: 0, y: 0 }
    }

    // Self works in methods too
    fn flip(self) -> Self {
        Self { x: self.y, y: self.x }
    }

    fn sum(self) -> i32 {
        self.x + self.y
    }
}

// enums can use Self too
enum Light {
    Red,
    Yellow,
    Green,
}

impl Light {
    fn next(self) -> Self {
        match self {
            Self::Red => Self::Green,
            Self::Green => Self::Yellow,
            Self::Yellow => Self::Red,
        }
    }

    fn is_stop(self) -> bool {
        match self {
```

```
            Self::Red => true,
            Self::Yellow => true,
            Self::Green => false,
        }
    }
}

fn main() {
    // A struct using Self
    let p = Point::new(3, 7);
    println!("Original: ({}, {})", p.x, p.y);

    let p2 = Point::new(3, 7);
    let flipped = p2.flip();
    println!("Flipped: ({}, {})", flipped.x, flipped.y);

    let p3 = Point::origin();
    println!("Origin: ({}, {})", p3.x, p3.y);

    // An enum using Self
    let light = Light::Red;
    let stop = light.is_stop();
    println!("Need to stop? {}", stop);

    let light2 = Light::Red;
    let next_light = light2.next();
    let stop2 = next_light.is_stop();
    println!("Does the next light require stopping? {}", stop2);
}
```

3.30.4 Recap

- Capital `Self` inside an `impl` block stands for "the current type," so it can appear where a type is needed.
- `Self` works in parameter types, return types `-> Self`, construction `Self { ... }`, and `enum` variants `Self::Red`.
- Lowercase `self` = the value itself; capital `Self` = the type itself.
- `Self` works in both `struct` and `enum` `impl`s.
- Using `Self` makes code more concise and easier to maintain.

Congratulations on finishing Chapter 3! 🐉 In this chapter you learned `structs`, `enums`, pattern matching (`match`, `if let`, `while let`, `let...else...`), destructuring, associated functions, and methods. You can now organize data and behavior with Rust's type system. Next chapter, we enter Rust's most central and most distinctive concept — ownership!

Chapter 4

Ownership and Borrowing

This chapter approaches the Rust language from an angle very different from the previous chapters. Besides its high-level-language ability to build abstractions, Rust also aims for the performance of low-level languages that sit closer to the hardware. To fulfill this design philosophy, Rust must build numerous restrictions into the language to guarantee runtime efficiency.

4.1 Ownership (the Keychain Analogy)

4.1.1 Goal of This Episode

Use an everyday keychain analogy to understand Rust's most central concept — ownership.

4.1.2 Main Text

No code this episode. Let's first talk about Rust's most important concept: **ownership**.

4.1.2.1 The Keychain Analogy

Imagine you have a keychain. It might carry a few small charms (light, always on you), and it might carry a key — a key that opens a safe. The rule is simple:

> **Each keychain can be in only one person's hands.**

That's Rust's ownership rule. While you hold the keychain, the charms and the key on it are yours.

4.1.2.2 A Move = Once You Hand It Over, It's Gone

If someone says, "Give me your keychain," then after you hand the whole thing over, you're left with nothing. You can't open the safe with that key anymore, because the key is no longer in your hands.

In Rust, this is called a **move**. When you give a value away (say, by assigning it to another variable), the original variable can no longer be used.

4.1.2.3 Why Can't We Copy the Key?

You might think: "Couldn't I just get a copy of the key made?"

Here's the problem: if two people each hold a key to the same safe, things can go wrong —

- A is organizing the things inside the safe.
- B opens the safe at the same time and takes something out.
- A turns around: "Huh? Where did my stuff go?"

This illustrates conflicting access to the same data. When such a conflict happens across mul-

tiple `Threads` without synchronization, it is called a **data race**. Rust's ownership rules exist to **prevent this kind of problem at the root**.

4.1.2.4 `clone` = Get a New Keychain That Works Just Like the Original, and Make Sure It Causes No Trouble

But what if I really do need a second usable keychain?

Rust's answer is called **`clone`**. It means: **get a new keychain that works just like the one in your hand, while making sure that doing so causes no trouble.**

The most common way to "make sure" is to leave the original key alone — buy a new safe, put a `clone` of everything inside into it, and hang a brand-new key on the new keychain. In the simplest case — plain data in the safe — each person ends up with their own safe and their own things, without interfering with each other: two fully independent sets.

Not every type does it this way, though. Later you'll meet types that really do "just cut an extra key," relying on other mechanisms for safety. For now, though, every `clone` you encounter can be understood as "buy a new safe."

4.1.2.5 Why Is Rust So Strict?

Most programming languages don't police any of this — copy freely, share freely, and deal with the bugs later. Rust is different: it stands guard while you're writing the code, ensuring no two parties ever mess with the same data at once.

That's Rust's core philosophy: **prevent errors at compile time, rather than waiting for things to blow up at runtime.**

4.1.3 Recap

- Every value has one "owner," just as every keychain is in exactly one person's hands.
- **Move**: hand the keychain to someone else, and you no longer have it.
- You can't simply copy a key to open the same safe — that risks conflicting access to the same data.
- **`clone`**: get a new keychain that works just like the original, while making sure it causes no trouble — usually by buying a new safe + `clone`-ing the contents + cutting a new key; in the simplest case, two fully independent sets.
- Rust enforces ownership rules at compile time, preventing this kind of conflicting access.

4.2 A Brief Introduction to `traits`

4.2.1 Goal of This Episode

Learn to define a `trait` and implement it for a type, and meet `#[derive]`, the shortcut that auto-generates implementations.

4.2.2 Concept

4.2.2.1 What Is a `trait`?

Before we get into ownership proper, let's learn an important tool: the **`trait`**. It has no direct connection to last episode's keychain analogy, but we'll need it when discussing `Clone`, `Copy`, and friends — so let's pick it up first.

In Chapter 3 we learned to add methods to `structs` and `enums` with `impl`. But what if we want to require that "certain types must all have a certain capability"?

Say I want to require: "these types must all be able to say hello." That's what a **`trait`** is for — it defines a set of "capabilities" or "behaviors," and different types can each implement those behaviors in their own way.

A `trait` is like a "spec sheet" that says: "To meet this spec, you must provide these features."

4.2.2.2 Defining a `trait`

Use the `trait` keyword:

```
trait Greet {
    fn greet(self);
}
```

This code means: "Any type that implements the `Greet` `trait` must have a `greet` method."

4.2.2.3 Implementing a `trait` for a Type

```
impl Greet for Cat {
    fn greet(self) {
        println!("Meow~");
    }
}
```

Earlier we wrote `impl Cat { ... }` to add methods to `Cat` directly. Now, `impl Greet for Cat { ... }` says "`Cat` meets the `Greet` spec," and inside we provide the methods `Greet` demands.

4.2.2.4 `derive`: The Shortcut That Auto-generates Implementations

Some `traits` have very formulaic implementations that the Rust compiler can generate for you. That's when you use `#[derive(...)]`:

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}
```

Remember using `{:?}` to print tuples and arrays in Chapter 2? That `{:?}` is actually using the `Debug` `trait`. Tuples and arrays come with `Debug` built in, but the `structs` and `enums` we define ourselves don't — so we add `#[derive(Debug)]` to have Rust generate the `Debug` implementation automatically.

4.2.3 Example Code

```rust
// Define a trait: every implementer must be able to "say hello"
trait Greet {
    fn greet(self);
}

// Define two kinds of animals
struct Cat;
struct Dog;

// Implement Greet for Cat
impl Greet for Cat {
    fn greet(self) {
        println!("I'm a cat, meow~");
    }
}

// Implement Greet for Dog
impl Greet for Dog {
    fn greet(self) {
        println!("I'm a dog, woof!");
    }
}

// Use derive to have Rust auto-generate a Debug implementation
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let cat = Cat;
    let dog = Dog;

    // Calling the trait methods
    cat.greet();
    dog.greet();

    // Printing the struct with {:?} (thanks to #[derive(Debug)])
    let p = Point { x: 3, y: 7 };
    println!("{:?}", p);
}
```

4.2.4 Recap

- A **`trait`** is a spec defining a set of behaviors — like a "capability checklist."
- Implement a `trait` for a type with `impl TraitName for TypeName` (e.g. `impl Greet for Cat`).
- `#[derive(Debug)]` has Rust automatically implement the `Debug` `trait` for your `struct` / `enum`.
- With `#[derive(Debug)]` added, `{:?}` can print your custom `struct` / `enum`.

4.3 Moves and `Clone`

4.3.1 Goal of This Episode

Understand Rust's move semantics — both assignment and passing into a function transfer ownership — and replicate data with `Clone`.

4.3.2 Concept

4.3.2.1 Move: Hand It Over and It's Gone

Last episode we learned `traits`; now let's see what ownership looks like in code.

In Rust, when you assign a `struct` value to another variable, the original variable **can no longer be used**. This is the "handing over the keychain" from Episode 1:

```
    let p1 = Point { x: 1, y: 2 };
    let p2 = p1; // Ownership of p1 moves to p2
    // From here on, p1 can't be used anymore!
```

This behavior is called a **move**. The Rust compiler checks this at compile time — if you try to use the original variable after a move, the compiler reports an error outright.

4.3.2.2 Passing into a Function Is Also a Move

It's not just assignment — passing a value into a function moves it too:

```
struct Point {
    x: i32,
    y: i32,
}

fn print_point(p: Point) {
    println!("({}, {})", p.x, p.y);
}

fn main() {
    let p1 = Point { x: 1, y: 2 };
    print_point(p1); // p1 gets moved into the function
    // p1 can't be used anymore!
}
```

Because a function's parameter is like a new variable — the value gets "handed" to it.

4.3.2.3 `Clone`

If you need to keep the original value and also want a replica, use **`Clone`**.

First, add `#[derive(Clone)]` to your type (throwing in `Debug` too, why not):

```
#[derive(Debug, Clone)]
struct Point {
    x: i32,
    y: i32,
}
```

Then replicate the value with `.clone()`:

```rust
#[derive(Debug, Clone)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p1 = Point { x: 1, y: 2 };
    let p2 = p1.clone();  // Replicate p1; p1 survives
    println!("{:?}", p1); // OK! p1 is still usable
    println!("{:?}", p2); // p2 is an independent replica
}
```

Recall Episode 1's analogy: `clone` means "get a new keychain that works just like the original, while making sure it causes no trouble." For `Point`, the new keychain is a complete replica — and that's exactly what the `clone` generated by `#[derive(Clone)]` does: `clone` every field. Each variable owns its own `clone`.

4.3.2.4 Integers Don't Move?

You may notice integers behave differently:

```rust
fn main() {
    let a = 42;
    let b = a;
    println!("{}", a); // This actually works!
}
```

Why don't integers move? We'll answer that next episode.

4.3.3 Example Code

```rust
#[derive(Debug, Clone)]
struct Point {
    x: i32,
    y: i32,
}

fn print_point(p: Point) {
    println!("The function received the point: ({}, {})", p.x, p.y);
}

fn main() {
    let p1 = Point { x: 10, y: 20 };

    // Use clone to make a replica so p1 doesn't get moved away
    let p2 = p1.clone();
    println!("p1 = {:?}", p1);
    println!("p2 = {:?}", p2);

    // Passing into a function moves too, so clone first
    print_point(p1.clone());
    println!("p1 is still here: {:?}", p1);

    // Without cloning, passing it in moves p1 away
```

```
    print_point(p1);
    // Uncommenting the line below makes the compiler report an error:
    // println!("p1 is gone: {:?}", p1);
}
```

4.3.4 Recap

- `let p2 = p1;` **moves** — afterward `p1` can't be used.
- Passing a value into a function is also a move.
- `#[derive(Clone)]` + `.clone()` calls `clone` on every field — for a type like `Point`, that means an independent replica.
- After a `clone`, the original variable remains usable.
- Integers (`i32` and friends) don't move — next episode explains why.

4.4 Copy

4.4.1 Goal of This Episode

Understand the `Copy trait` — why integers, floats, booleans, and characters don't move on assignment.

4.4.2 Concept

4.4.2.1 Last Episode's Question

Last episode we found that `struct` values move when assigned or passed into functions, but integers don't:

```
fn main() {
    let a = 42;
    let b = a;
    println!("{}", a); // Completely fine!
}
```

Why? The answer is the **`Copy trait`**.

4.4.2.2 What Is `Copy`?

`Copy` is a special `trait`. If a type implements `Copy`, then on assignment or when passed into a function, Rust automatically makes a copy instead of moving.

`Copy` means copying the value is a simple, mechanical operation, so Rust can do it automatically on assignment or when passing the value into a function — no need to write `.clone()`.

4.4.2.3 Which Types Have `Copy` Automatically?

These types are born with `Copy`:

- Integers: `i8`, `i16`, `i32`, `i64`, `i128`, `u8`, `u16`, `u32`, `u64`, `u128`, `isize`, `usize`.
- Floats: `f32`, `f64`.
- Booleans: `bool`.
- Characters: `char`.
- ...and plenty of other types.

Additionally, **tuples** and **arrays** are `Copy` as a whole if every element inside is `Copy`:

```
fn main() {
    let t = (1, true, 'a');  // (i32, bool, char) → all Copy → the tuple is Copy too
    let t2 = t;
    println!("{:?}", t);     // OK!

    let arr = [1, 2, 3];     // [i32; 3] → i32 is Copy → the array is Copy too
    let arr2 = arr;
    println!("{:?}", arr);   // OK!
}
```

That's why in the code you wrote in earlier chapters, integers, tuples, and arrays could be freely assigned to multiple variables and passed into multiple functions without any trouble.

Beyond `Copy`: when every type in a tuple implements `Clone`, the tuple automatically implements `Clone` too. In fact, tuples behave this way for some other `traits` — for those `traits`, if all the elements implement one, the tuple as a whole does too. We won't belabor this point again.

4.4.2.4 Your Own Types Can Take `Copy` Too

If every value in your type has a `Copy` type, your type can take `#[derive(Copy, Clone)]`:

```
#[derive(Debug, Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}
```

Note: `Copy` requires `Clone`. When using `derive`, the usual approach is `#[derive(Copy, Clone)]`; writing only `#[derive(Copy)]` without a `Clone` implementation is a compile error.

Why? Because Rust decrees: anything that can be copied must also be `clone`-able. `Copy` is "automatic copying"; `Clone` is "calling `.clone()` by hand." If something can't even be `cloned` manually, it certainly shouldn't be copied automatically. So `Copy` requires `Clone` first.

Once added, `Point` behaves just like an integer — assignment doesn't move:

```
#[derive(Debug, Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p1 = Point { x: 1, y: 2 };
    let p2 = p1; // Automatically copied; p1 survives!
    println!("{:?}", p1); // OK
}
```

4.4.2.5 The Difference between copy and `clone`

	copy	`clone`
Triggered by	Automatic (assignment, passing into functions)	Manual (`.clone()`)

	copy	`clone`
Can customize its behavior	No	Yes, through the implementation of `.`
Prerequisite to implement	All contents must be `Copy`	None

In short: **copy is automatic duplication; `clone` is called by hand, and what it does is up to the type.**

4.4.3 Example Code

```
#[derive(Debug, Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}

fn print_point(p: Point) {
    println!("The function received: ({}, {})", p.x, p.y);
}

fn double(n: i32) -> i32 {
    n * 2
}

fn main() {
    // Integers copy automatically
    let a = 42;
    let b = a;
    println!("a = {}, b = {}", a, b); // Both usable

    // bool is Copy too
    let flag = true;
    let flag2 = flag;
    println!("flag = {}, flag2 = {}", flag, flag2);

    // Passing an integer into a function doesn't move it
    let x = 10;
    let result = double(x);
    println!("x = {}, result = {}", x, result);

    // A custom struct with Copy added doesn't move either
    let p1 = Point { x: 3, y: 7 };
    let p2 = p1;     // Automatically copied
    print_point(p1); // p1 remains usable
    println!("p1 = {:?}", p1); // Still works!
    println!("p2 = {:?}", p2);
}
```

4.4.4 Don't Slap `Copy` on Your Types Casually

Having read this episode, you might think: "So why don't I just add `#[derive(Copy, Clone)]` to every `struct` from now on?"

Please don't. Here's why: once `Copy` is on, code using your type comes to depend on the "auto-copy on assignment" behavior. If one day you need to modify the `struct` and add a non-`Copy` field, you'll have to remove `Copy`.

And then the trouble starts: with `Copy` gone, every `let p2 = p1;` flips from "automatic copy" to "move," and `p1` stops being usable. All the code using this type may break — potentially in many, scattered places.

So the good habit is: **only add `Copy` when you're sure the type will never gain a non-`Copy` field.** Something like `Point { x: i32, y: i32 }` is a great fit. When unsure, add only `Clone` — write `.clone()` manually when you need it, and future changes won't ripple through other code.

4.4.5 Recap

- **`Copy`** is a `trait` that makes a type copy automatically on assignment and function calls, instead of moving.
- Primitive types like `i32`, `f64`, `bool`, `char` have `Copy` innately.
- Tuples and arrays are `Copy` when all their elements are.
- Tuples behave this way for some `traits` (`Copy`, `Clone`, etc.): for those `traits`, if all elements implement one, the tuple does too.
- Custom `structs` can take `#[derive(Copy, Clone)]`, but every field must be a `Copy` type.
- `Copy` requires `Clone`; when using `derive`, both are usually listed together.
- **`Copy` = automatic copying; `Clone` = calling `.clone()` by hand**
- Don't add `Copy` casually — removing it later breaks all the code that relied on auto-copying. When unsure, just add `Clone`.

4.5 Borrowing: &

4.5.1 Goal of This Episode

Learn to borrow values with & — letting others read your data without a move or a `clone`.

4.5.2 Concept

4.5.2.1 Both move and `clone` Have Costs

So far we've learned two ways to deal with ownership:

- **Move**: hand it over and it's gone; the original variable can't be used.
- **`clone`**: replicate the data (true for every type we've met so far) — but if the data is large, replication is wasteful.

Is there a way to **neither hand it over nor replicate it — just lend it out for a look**?

Yes! That's **borrowing**, using the & symbol.

4.5.2.2 & Means "Borrow"

```
let p = Point { x: 1, y: 2 };
let r: &Point = &p; // r is a reference to p; p remains the owner
```

`&p` means: "I'm not taking ownership of `p` — I'm just borrowing it for a look." `p` is still there and remains usable afterward; as for the restrictions during a borrow, we'll lay those out later.

The thing `&p` produces (that is, `r`) is called a **reference**, with the type written `&Point`. And "looking at someone's data through a reference without taking ownership" is what we call **borrowing**. Borrowing and references are two sides of one coin: **borrowing** is the act of "taking something for a look," and a **reference** is the pass that act hands you — holding it lets you go look at the data. From here on, the word "reference" usually means a value borrowed with `&`.

4.5.2.3 Function Parameters with `&` Don't Move

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn print_point(p: &Point) {
    println!("({}, {})", p.x, p.y);
}

fn main() {
    let p1 = Point { x: 10, y: 20 };
    print_point(&p1); // Passing &p1 — just borrowing, not moving
    println!("{:?}", p1); // p1 is still here!
}
```

Note two places:

1. The parameter type is written `&Point` (with a leading `&`).
2. The call passes `&p1` (also with `&`).

The function merely "borrows" `p1` for a look and hands it back when done — `p1`'s ownership never changes.

4.5.2.4 Those Earlier `&`s Were References All Along!

Remember `&[i32]` (slices) and `&str` (string slices)? At the time, we said not to dig too deep. Now we can explain — those `&`s are borrows!

- `&[i32]` is a reference to a stretch of array data; it doesn't own it.
- `&str` is a reference to a stretch of string data; it doesn't own it.

So for a function like this:

```
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}
```

`for x in nums` walks every element of the slice, just like iterating an array before. The function only borrows a slice of the array — it never moves the whole array away.

4.5.2.5 * Dereferencing

& is "borrow"; conversely, * is "follow the reference back to the original value," called **dereferencing**:

```
fn main() {
    let x = 42;
    let r = &x;
    println!("{}", *r); // 42, same as x
}
```

Most of the time, though, you won't write * by hand — Rust dereferences automatically when you access fields with `.`, call methods, or use `println!`. Knowing it exists is enough for now; next episode will use it.

Note: the `&[T]` and `&str` we met earlier are special — you can't use * on them to get a value out. The reason comes later; just know it for now.

4.5.2.6 Every `&T` Is `Copy`

Last episode we learned `Copy` — some types copy automatically on assignment rather than moving. Whatever `T` is, `&T` is `Copy`. After all, a reference is just a borrow — copying a reference doesn't affect the original data; it just means one more onlooker:

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let s = Point { x: 0, y: 0 };
    let r1 = &s;
    let r2 = r1; // Copies the reference; not a move
    println!("{:?}, {:?}", r1, r2); // Both r1 and r2 are usable
}
```

Note: `Point` itself isn't `Copy` (assignment moves it), but `&Point` is `Copy`.

4.5.2.7 & References Are Usually Read-only

When borrowing with &, you can **usually only read, not modify directly**. If you want to borrow something in order to change it directly — that's next episode.

4.5.3 Example Code

```
#[derive(Debug, Clone)]
struct Point {
    x: i32,
    y: i32,
}

// Borrowing; no move
fn print_point(p: &Point) {
    println!("({}, {})", p.x, p.y);
}
```

```rust
// A slice parameter is a reference
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}

fn main() {
    let p1 = Point { x: 10, y: 20 };

    // Borrowing: pass &p1, and p1 isn't moved
    print_point(&p1);
    print_point(&p1); // You can borrow many times!
    println!("p1 is still here: {:?}", p1);

    // Array slices are borrows too
    let numbers = [1, 2, 3, 4, 5];
    let total = sum(&numbers);
    println!("Total = {}", total);
    println!("numbers is still here: {:?}", numbers);

    // &str is a borrow as well
    let greeting: &str = "Hello";
    println!("{}", greeting);
    println!("{}", greeting); // Usable many times
}
```

4.5.4 Recap

- & is borrowing — **no ownership transfer**; the original variable stays usable.
- Write parameters as `&Type` and pass `&value` at the call site.
- Borrowing can happen many times, unlike a move which happens once.
- `*` is dereferencing — following a reference to the original value (though Rust usually does it for you).
- `&[T]` and `&str` are special references; `*` can't extract a value from them.
- Every `&T` is `Copy` — copying a reference doesn't affect the original data.
- & references are **usually read-only**; you can't directly modify what you borrowed.

4.6 Mutable Borrowing: `&mut`

4.6.1 Goal of This Episode

Learn to borrow a value with `&mut` and modify it — changing someone's data without a move.

4.6.2 Concept

4.6.2.1 Last Episode's Limitation

Last episode we learned & borrowing, but the resulting reference was read-only — look, don't touch. What if we want to borrow someone's thing in order to modify it?

4.6.2.2 `&mut` Is "Borrow to Modify"

```
fn main() {
    let mut x = 10;
    let r: &mut i32 = &mut x; // A mutable reference
    *r = 20;                  // Modify x's value through r
}
```

Key points:

1. The original variable must be `let mut` (you're going to change it).
2. Borrow with `&mut x`.
3. To modify the value through the reference, write `*r` (last episode's dereferencing — following the reference to the original value).

4.6.2.3 `&mut` in Function Parameters

The more common usage is in functions:

```
fn add_ten(n: &mut i32) {
    *n += 10;
}

fn main() {
    let mut x = 5;
    add_ten(&mut x);
    println!("{}", x); // 15
}
```

The function receives an `&mut i32` — a **mutable reference**. Through `*n` it can modify the original value. The call passes `&mut x`.

4.6.2.4 Mutable Borrows of `structs`

Same story with `structs`:

```
fn move_right(p: &mut Point) {
    p.x += 1; // No * needed for struct fields; Rust handles it
}
```

Note: when modifying a `struct`'s fields, you don't write `(*p).x += 1` — just `p.x += 1`. As mentioned last episode, Rust auto-dereferences.

4.6.3 Example Code

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
```

```rust
}

// Modifying an integer through a mutable reference
fn add_ten(n: &mut i32) {
    *n += 10;
}

// Modifying a struct's fields through a mutable reference
fn move_right(p: &mut Point) {
    p.x += 1;
}

fn move_up(p: &mut Point) {
    p.y += 1;
}

fn main() {
    // Modifying an integer
    let mut score = 80;
    println!("Before: {}", score);
    add_ten(&mut score);
    println!("After: {}", score);

    // Modifying a struct
    let mut pos = Point { x: 0, y: 0 };
    println!("Starting position: {:?}", pos);

    move_right(&mut pos);
    move_right(&mut pos);
    move_up(&mut pos);
    println!("After moving: {:?}", pos);

    // Modifying directly with &mut
    let mut val = 100;
    let r = &mut val;
    *r += 50;
    println!("val = {}", val);
}
```

4.6.4 Recap

- `&mut` is a mutable borrow — once borrowed, the original value can be modified.
- The original variable must be `let mut`.
- Write parameters as `&mut Type` and pass `&mut value` when calling.
- Modifying `struct` fields works directly as `r.field` (auto-dereferencing).
- Next episode covers the important restrictions on mutable borrows.

4.7 The Borrowing Rules

4.7.1 Goal of This Episode

Understand Rust's borrowing rules: at any one time, either one `&mut` or many `&`s; why a borrowed value can't be moved; plus the problem of dangling references.

4.7.2 Concept

4.7.2.1 Why Do We Need Rules?

Last episode we learned `&mut` mutable borrowing. But what would happen if Rust allowed multiple mutable references at once?

Imagine your keychain. Lending it to many people **to look at** (`&`) is fine — everyone just looks; nothing on the keychain changes. But lend it to two people **to modify** at once (`&mut`) — A is adding a new key while B is removing it — and the result becomes unpredictable.

That is conflicting access to the same data, and it can lead to all kinds of weird bugs. So Rust lays down strict borrowing rules.

4.7.2.2 Rule 1: Only One `&mut` at a Time

At any single moment, a value can have at most **one** mutable reference:

```
    let mut x = 10;
    let r1 = &mut x;
    let r2 = &mut x; // Compile error! There's already a &mut
    *r1 += 1;
```

4.7.2.3 Rule 2: `&` and `&mut` Can't Coexist

If someone is reading (`&`), no one may be modifying (`&mut`) — and vice versa:

```
    let mut x = 10;
    let r1 = &x;     // Read-only borrow
    let r2 = &mut x; // Compile error! There's already a &, so no &mut
    println!("{}", r1);
```

4.7.2.4 Rule 3: Multiple `&`s Can Coexist

Many simultaneous readers is no problem at all:

```
fn main() {
    let x = 10;
    let r1 = &x;
    let r2 = &x;
    let r3 = &x;
    println!("{} {} {}", r1, r2, r3); // Totally fine
}
```

4.7.2.5 You Can't Move a Value That's Borrowed

We learned earlier that a move makes the original variable unusable. So while a value is still borrowed by a reference, you can't move it away:

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    let r = &p;

    let p2 = p; // Compile error! p is still borrowed by r
    println!("{:?}", r);
}
```

`r` still uses the borrowed value later, but `let p2 = p;` would move `p` away, making `p` unusable from that line on. Rust won't let you keep a reference that will still be used while invalidating the original variable.

That said, a borrow doesn't live from the reference's creation all the way to the closing brace. Once the reference is used for the last time, the borrow ends — and moving is fine again:

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    let r = &p;

    println!("{:?}", r); // r's last use

    let moved = p; // OK: the borrow has already ended
    println!("{:?}", moved);
}
```

There's a second case: you can't move a non-`Copy` value out from behind a reference.

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    let r = &p;

    let moved = *r; // Compile error! Can't move a Point out from behind a &Point
    println!("{:?}", moved);
}
```

`*r` follows the reference to the original value. `Point` doesn't implement `Copy`, so storing it into `moved` is a move, not a copy.

This one differs from the case above: there, waiting until `r` was done with the value was enough; here it makes no difference whether `r` is ever used again. `p` is the owner of that value, and `r` only borrowed a look at it — what you borrow, you may look at but not carry away.

Of course, if the value behind `*r` were a `Copy` type like `i32`, it's a different story: Rust copies it instead of moving, so no rule is broken.

4.7.2.6 Dangling References

One more important rule: **a reference must point to a value that's still valid**. If a reference will still be used but the value it points to has become invalid, it becomes a **dangling reference** — pointing at a place that no longer exists. Rust stops this at compile time.

The most common dangling reference happens when a reference "escapes" from an inner scope:

```
    let r;
    {
        let x = 42;
        r = &x; // x lives only inside these braces
    } // x is dropped here
    println!("{}", r); // Compile error! The x that r points to no longer exists
```

`x` is dropped when the braces close, yet `r` tries to use it outside — Rust says no.

Another common case is a function trying to return a reference to a local variable:

```
fn bad() -> &i32 {
    let x = 42;
    &x // x is dropped when the function ends; the reference would point to a vanished value
}
```

Same reasoning: `x` disappears after the function ends, and the returned reference would point to a value that doesn't exist.

As for how Rust tracks "is this reference still valid" — that's the concept of **lifetimes**, coming later. For now, remember: **a reference must point to a value that's still valid**.

4.7.3 Example Code

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    // Multiple immutable borrows: OK
    let p = Point { x: 1, y: 2 };
    let r1 = &p;
    let r2 = &p;
    println!("r1 = {:?}, r2 = {:?}", r1, r2);

    // One mutable reference: OK
    let mut p2 = Point { x: 10, y: 20 };
```

```
    let r3 = &mut p2;
    r3.x += 5;
    println!("After modifying: {:?}", r3);

    // Once r3 is used for the last time the mutable borrow ends, so & borrowing is allowed now
    let r4 = &p2;
    println!("Read-only borrow: {:?}", r4);

    // Demo: multiple simultaneous read-only borrows
    let nums = [10, 20, 30, 40, 50];
    let slice1 = &nums[0..3];
    let slice2 = &nums[2..5];
    println!("slice1 = {:?}", slice1);
    println!("slice2 = {:?}", slice2);
}
```

4.7.4 Recap

- Unrestricted borrowing would also allow conflicting access to the same data, so Rust lays down borrowing rules.
- **Only one `&mut` at a time** — two simultaneous mutable references are forbidden.
- **`&` and `&mut` can't coexist** — either everyone reads, or exactly one person modifies.
- **Multiple `&`s can coexist** — many simultaneous readers are fine.
- **You can't move a value that's borrowed**: while the reference will still be used, the original can't be moved away; once its last use is past, the borrow ends and moving is fine again.
- **Nor can you move a non-`Copy` value out from behind a reference**: what you borrow, you may look at but not carry away — and that holds whether or not the reference is used again.
- **Dangling references**: a reference must point to a valid value and can't outlive the value it points to — whether the value left its scope or a function returned a reference to a local.
- These rules let Rust prevent conflicting access at compile time; later we'll learn lifetimes for tracking reference validity more precisely.

4.8 `self` vs `&self` vs `&mut self`

4.8.1 Goal of This Episode

Learn to choose between `self`, `&self`, and `&mut self` in methods, and how to pick `T` / `&T` / `&mut T` for function parameters.

4.8.2 Concept

4.8.2.1 Recap: Chapter 3's `self`

In Chapter 3 we learned `impl` and methods, where every method took `self` by value:

```
impl Cat {
    fn meow(self) {
        println!("Meow~");
    }
```

```
}
```

But taking `self` by value **consumes** the value — after the call, the original variable can't be used anymore (it was moved).

Now that we know borrowing, we can be smarter!

4.8.2.2 The Three Kinds of `self`

Form	Meaning	Effect
`self`	Takes ownership	After the call, the original variable is unusable (move)
`&self`	Read-only borrow of itself	The original stays usable afterward, but the borrow can't modify
`&mut self`	Mutable borrow of itself	The original stays usable afterward, and the borrow can modify

4.8.2.3 How to Choose?

- **Just reading data** → use `&self` (the most common!)
- **Modifying your own fields** → use `&mut self`.
- **Transferring ownership (original unusable after the call)** → use `self`.

Most methods use `&self`, because you usually just want to "look at this thing's state" without consuming it.

4.8.2.4 A Real Example: `Clone`

A great example is the `Clone` `trait`. Its simplified definition looks like this:

```
trait Clone {
    fn clone(&self) -> Self;
}

fn main() {}
```

`clone` takes `&self` — merely borrowing itself, not consuming — then returns a new `Self` (capital `Self`, taught in Chapter 3's last episode: the type implementing this `trait`). This explains why you can call `.clone()` on the same variable over and over — `clone` only borrows, never moving the original value.

If `clone`'s signature were `fn clone(self) -> Self`, every `clone` would consume the original — defeating the whole point of `clone`.

4.8.2.5 Function Parameters Follow the Same Logic

It's not just methods' `self` — ordinary function parameters follow the same logic:

Parameter type	Meaning
`p: Point`	Takes ownership (move)
`p: &Point`	Read-only borrow
`p: &mut Point`	Mutable borrow

Same selection principle:

- Read only → &T.

- Modify → `&mut T`.
- Consume → `T`.

4.8.3 Example Code

```rust
#[derive(Debug)]
struct Counter {
    id: i32,
    count: i32,
}

impl Counter {
    // Associated function: create a new Counter
    fn new(id: i32) -> Self {
        Counter { id, count: 0 }
    }

    // &self: read-only
    fn get_count(&self) -> i32 {
        self.count
    }

    // &self: read-only, printing info
    fn display(&self) {
        println!("Counter {}: current count = {}", self.id, self.count);
    }

    // &mut self: mutable borrow, modifying count
    fn increment(&mut self) {
        self.count += 1;
    }

    // self: takes ownership, returning the final result
    fn finish(self) -> i32 {
        println!("Counter {} finished! Final count = {}", self.id, self.count);
        self.count
    }
}

// Ordinary functions follow the same logic
fn print_counter(c: &Counter) {
    println!("(Function version) Counter {}: {}", c.id, c.count);
}

fn reset_counter(c: &mut Counter) {
    c.count = 0;
}

fn main() {
    let mut c = Counter::new(1);

    // &self: read-only
    c.display();
    println!("Currently: {}", c.get_count());
```

```
    // &mut self: modifying
    c.increment();
    c.increment();
    c.increment();
    c.display();

    // &T and &mut T with ordinary functions
    print_counter(&c);
    reset_counter(&mut c);
    c.display();
    c.increment();
    c.increment();

    // self: taking ownership
    let final_count = c.finish();
    println!("Returned final count: {}", final_count);
    // finish took c's ownership; uncommenting the line below is a compile error:
    // c.display();
}
```

4.8.4 No Manual `&` or `&mut` at the Call Site

You may have noticed — when calling, we just write `c.display()` and `c.increment()`, not `(&c).display()` or `(&mut c).increment()`. Rust automatically adds the `&` or `&mut` based on the method's `self` parameter. You could write `(&c).display()` or `(&mut c).increment()`, but there's no need.

4.8.5 Recap

- `&self`: read-only borrow — the most common; the value stays usable after the call.
- `&mut self`: mutable borrow — can modify fields; the value stays usable after the call.
- `self`: consumes ownership — the variable is unusable after the call.
- Selection principle: **read** → **`&self`**, **modify** → **`&mut self`**, **consume** → **`self`**.
- `Clone`'s method is defined as `fn clone(&self) -> Self` — borrowing itself to produce a new `Self`, so `clone` never consumes the original.
- Ordinary function parameters likewise: **read** → **`&T`**, **modify** → **`&mut T`**, **consume** → **`T`**.
- Call methods simply as `c.method()`.

4.9 The Stack and the Heap

4.9.1 Goal of This Episode

Understand the difference between the stack and the heap, and unveil what Episode 1's key-chain analogy really meant.

4.9.2 Concept

4.9.2.1 Two Common Places in Memory

While a program runs, its data lives in memory. For now, let's look at two common places where data can be stored: the stack and the heap.

The stack:

- When a function is called, the stack is commonly used to store local variables whose sizes are known at compile time.
- The types we've learned so far — integers, floats, booleans, `char`s, fixed-length arrays, tuples, and `struct`s containing only these kinds of data — are commonly stored directly on the stack when used as local variables.
- Data whose size is known at compile time is not necessarily small; what matters is that the compiler already knows how much space it needs.
- When the function ends, the stack space used by that call is reclaimed together.

The heap:

- A program can request additional heap space as needed while it runs.
- Data is often stored separately on the heap when its amount is known only at runtime or may grow while the program runs. For example, if a program needs to store every number a user enters, it may not know beforehand how many there will be.
- The program remembers how to find the data stored there later.
- Rust's ownership system determines when that space can be returned.

4.9.2.2 The Keychain Analogy, Unveiled!

Remember Episode 1's keychain analogy? Time to reveal what it really meant:

- **The key** = the information that lets the program find the safe later.
- **The safe** = data stored separately on the heap.
- **The charms on the keychain** = data carried directly on the keychain.

So when we said "a move is handing over the keychain":

- The key and the charms are handed to the new owner together.
- The safe itself stays where it is; it doesn't need to be moved or recreated.

4.9.2.3 Why Are Integers `Copy`?

Integers (`i32` and so on) are like the charms on the keychain. Copying an integer is a simple, mechanical operation, so integers implement `Copy`.

Some types are also responsible for managing data stored separately on the heap, so they can't be copied automatically in the same way. Assigning such a value moves it; creating a `clone` requires an explicit `.clone()`. The next few episodes will show concrete examples.

4.9.3 Example Code

```
#[derive(Debug, Copy, Clone)]
struct StackData {
    x: i32,
```

```
    y: i32,
    active: bool,
}

fn main() {
    // These local variable sizes are known at compile time,
    // so the values can be stored directly on the stack.
    let a = 42;    // i32, 4 bytes
    let b = 3.14;  // f64, 8 bytes
    let c = true;  // bool, 1 byte
    let ch = '🦀'; // char, 4 bytes
    println!("Integer: {}, float: {}, boolean: {}, character: {}", a, b, c, ch);

    // The struct stores all its fields directly, so it can be stored on the stack too
    let data = StackData { x: 10, y: 20, active: true };
    let data2 = data; // Copy! data stays usable
    println!("data = {:?}", data);
    println!("data2 = {:?}", data2);

    // A fixed-length array can be stored directly on the stack too
    let arr = [1, 2, 3, 4, 5];
    println!("Array: {:?}", arr);

    // A tuple can be stored directly on the stack too
    let t = (42, true, 'A');
    println!("tuple: {:?}", t);
}
```

4.9.4 Recap

- **Stack**: local variables whose sizes are known at compile time — such as integers, fixed-length arrays, tuples, and `structs` made from these kinds of data — are commonly stored directly here; the space used by a function call is reclaimed when the function ends.
- **Heap**: commonly stores data whose amount is known only at runtime or may grow; the program remembers how to find the data stored there.
- Keychain analogy unveiled: key = the information used to find the data, safe = separately stored heap data, charms = directly carried data.
- Integers implement `Copy` because copying them is a simple, mechanical operation.
- Types that manage separately stored heap data move on assignment; creating a `clone` requires an explicit `.clone()`.

4.10 `String`

4.10.1 Goal of This Episode

Meet Rust's `String` type — a string that owns its data and can be modified.

4.10.2 Concept

4.10.2.1 The Strings We Had Were All Borrowed

Since Chapter 1, we've been using the `&str` type:

```
    let greeting: &str = "Hello";
```

The string `"Hello"` is written directly in the code; its data gets compiled into the program itself. `&str` is a reference — you're only looking at the text; you **don't own** it and **can't modify** it.

4.10.2.2 `String`: A String You Own

`String` is a string type you can own and modify. Its data lives mainly on the heap.

Create one with `String::from()`:

```
    let s = String::from("Hello");
```

`String::from` is an associated function (called with `::`, as in Chapter 3). It copies the `&str`'s contents onto the heap, creating a `String` you own.

4.10.2.3 `push_str`: Appending Text

A `String` can be modified! Use `push_str` to tack on more text:

```
fn main() {
    let mut s = String::from("Hello");
    s.push_str(", world!");
    println!("{}", s); // Hello, world!
}
```

Note the variable must be declared `let mut`, since we're modifying it.

4.10.2.4 `format!`: Combining Values into a String

`format!` works exactly like `println!`, except it doesn't print — it returns a `String`:

```
fn main() {
    let name = "Ming";
    let age = 20;
    let msg = format!("My name is {} and I'm {} years old", name, age);
    println!("{}", msg);
}
```

4.10.2.5 `String` Follows the Ownership Rules Too

Because a `String`'s data lives mainly on the heap, it is **not `Copy`**. Assignment and passing into functions both move:

```
    let s1 = String::from("hello");
    let s2 = s1; // Move! s1 can't be used anymore
```

Same as before — to keep `s1`, use `.clone()` or borrow with `&`.

4.10.3 Example Code

```
fn main() {
    // Creating a String
    let mut greeting = String::from("Hello");
    println!("{}", greeting);

    // push_str: appending more text
```

```
    greeting.push_str(", world");
    greeting.push_str("!");
    println!("{}", greeting);

    // format!: combining several values
    let name = "Hana";
    let score = 95;
    let report = format!("Student {} scored {} points", name, score);
    println!("{}", report);

    // String moves (it's not Copy)
    let s1 = String::from("Rust");
    // let s2 = s1; // Writing this would move s1 away, making it unusable
    let s2 = s1.clone(); // Make a replica with clone; s1 survives
    println!("s1 = {}", s1);
    println!("s2 = {}", s2);

    // Passing into a function: borrowing avoids the move
    let s3 = String::from("Hi there");
    print_string(&s3);
    println!("s3 is still here: {}", s3);

    // The Debug format works too
    let s4 = String::from("debug test");
    println!("{:?}", s4);
}

fn print_string(s: &String) {
    println!("The function received: {}", s);
}
```

4.10.4 Recap

- `String` is a string type that owns its data, which lives mainly on the heap.
- `String::from("...")` creates a new `String`.
- `push_str` appends more text to the string (requires `let mut`).
- `format!` shares `println!`'s syntax but returns a `String` instead of printing.
- `String` is **not `Copy`** — assignment and passing into functions move it.
- To keep the original `String`, use `.clone()` or borrow with `&`.

4.11 `String` vs `&str`

4.11.1 Goal of This Episode

Get the difference between `String` and `&str` straight, and figure out which one function parameters should use.

4.11.2 Concept

4.11.2.1 Two Kinds of Strings — What Exactly Differs?

	`String`	`&str`
Owns the data?	✔ Yes	✖ Just borrowing
Where's the data?	Mainly on the heap	Possibly in the code itself, or borrowing a `String`'s data
Modifiable?	✔ Yes (`push_str`, etc.)	✖ No
Moves?	✔ Yes	✖ No (it's just a reference)

4.11.2.2 `&String` Converts to `&str` Automatically

When you have a `String` and want to pass its reference to a function that accepts `&str`, Rust converts for you:

```
fn greet(name: &str) {
    println!("Hello, {}!", name);
}

fn main() {
    let s = String::from("Ming");
    greet(&s); // &String converts to &str automatically — totally fine!
}
```

Why does this work? We'll learn later. For now, just know: **where you pass an `&String` and the parameter type is `&str`, Rust handles it automatically**.

4.11.2.3 Function Parameters Prefer `&str`

If your function only needs to "read" a piece of text without owning it, make the parameter `&str`:

```
fn count_chars(s: &str) -> i32 {
    let mut count = 0;
    for _c in s.chars() {
        count += 1;
    }
    count
}
```

The `.chars()` used here is a method — implemented on both `String` and `&str`. It splits the string into individual characters for you to iterate over.

The benefits of this approach:

1. Passing an `&str` (string literal) works.
2. Passing an `&String` works too (automatic conversion).
3. Nothing gets moved.

That's why the Rust community broadly recommends: **use `&str` for function parameters, not `&String`**.

4.11.2.4 When to Use `String`?

- You need to **own** the text (storing it in a `struct`, returning it to the caller).
- You need to **modify** the text (`push_str`, etc.).

4.11.3 Example Code

```rust
// Parameter as &str: accepts both &str and &String
fn greet(name: &str) {
    println!("Hello, {}!", name);
}

fn char_count(s: &str) -> i32 {
    let mut count = 0;
    for _c in s.chars() {
        count += 1;
    }
    count
}

fn main() {
    // &str: a string literal
    let literal = "world";
    greet(literal);

    // String: an owned string
    let owned = String::from("Hana");
    greet(&owned); // &String auto-converts to &str

    // Both can be passed to a function accepting &str
    println!("\"{}\" has {} characters", literal, char_count(literal));
    println!("\"{}\" has {} characters", owned, char_count(&owned));

    // String can be modified; &str can't
    let mut s = String::from("Rust");
    s.push_str(" is fun");
    println!("{}", s);

    // String moves
    let s1 = String::from("hello");
    let s2 = s1; // move
    // println!("{}", s1); // Compile error!
    println!("{}", s2);

    // &str doesn't move (it's a reference by nature)
    let greeting: &str = "Hi";
    let greeting2 = greeting;  // This is a Copy! (&str is Copy)
    println!("{}", greeting);  // OK
    println!("{}", greeting2); // OK
}
```

4.11.4 Recap

- **`String`** owns its data (mainly on the heap), can be modified, and moves.
- **`&str`** is a reference — owns nothing, can't be modified, doesn't move.
- `&String` converts to `&str` automatically.
- Prefer `&str` for function parameters — it accepts more (both `&str` and `&String` can be passed).
- Use `String` only when you need to own or modify the text.

4.12 `Vec` Basics

4.12.1 Goal of This Episode

Learn to use `Vec` — an array that can grow dynamically.

4.12.2 Concept

4.12.2.1 The Limits of Arrays

In Chapter 2 we learned arrays, `[i32; 5]` — but an array's size is fixed, settled at declaration, with no adding or removing afterward.

What if we need a collection whose **size can change**? Say, a user entering data record by record, or a program accumulating results as it runs.

That calls for **`Vec`**. A `Vec` is like a **stretchable array**, with its data mainly on the heap.

4.12.2.2 Creating a `Vec`

The simplest way is the `vec!` macro:

```
    let nums = vec![1, 2, 3, 4, 5];
```

That creates a `Vec` holding five `i32`s. Rust infers the type from the values you put in.

Like the array's `[0; 5]`, `vec!` supports the "repeat N times" form too:

```
    let zeros = vec![0; 10]; // ten 0s
```

You can also create an empty `Vec` and add items one by one:

```
    let mut nums = Vec::new();
    nums.push(10);
    nums.push(20);
```

Rust infers the type at your first `push`.

4.12.2.3 Indexing and Iterating

Indexing a `Vec` works like an array, with `[i]`:

```
fn main() {
    let nums = vec![10, 20, 30];
    println!("{}", nums[0]); // 10
    println!("{}", nums[2]); // 30
}
```

Iterating also works like arrays, with `for`:

```
fn main() {
    let nums = vec![10, 20, 30];
    for n in &nums {
        println!("{}", n);
    }
}
```

Note: we iterate with `&nums` (borrowing) so `nums` doesn't get moved away. Details next episode.

4.12.2.4 `push`: Adding New Elements

```
fn main() {
    let mut fruits = Vec::new();
    fruits.push("apple");
    fruits.push("banana");
    fruits.push("cherry");
    println!("{:?}", fruits);
}
```

`push` appends the new element at the end. Note that the `Vec` must be `let mut` to `push`.

4.12.2.5 `len`: Getting the Length

```
fn main() {
    let nums = vec![1, 2, 3];
    println!("Length: {}", nums.len());
}
```

4.12.3 Example Code

```
fn main() {
    // Creating with vec!
    let scores = vec![85, 92, 78, 95, 88];
    println!("Scores: {:?}", scores);
    println!("First entry: {}", scores[0]);
    println!("{} entries in total", scores.len());

    // An empty Vec, filled with push
    let mut names = Vec::new();
    names.push("Ming");
    names.push("Hana");
    names.push("Wang");
    println!("Roster: {:?}", names);

    // Iterating
    println!("Listing one by one:");
    for name in &names {
        println!("  - {}", name);
    }

    // Iterating with for and summing
    let nums = vec![10, 20, 30, 40, 50];
    let mut total = 0;
    for x in &nums {
        total += x;
    }
    println!("Total = {}", total);

    // A Vec can keep growing
    let mut growing = Vec::new();
    for i in 0..5 {
        growing.push(i * 10);
    }
    println!("Built dynamically: {:?}", growing);
```

```
}
```

4.12.4 Recap

- **`Vec`** is a dynamically growable array whose data lives mainly on the heap.
- `vec![1, 2, 3]` creates a `Vec` with initial values; `vec![0; 10]` creates ten 0s (like the array's `[0; 10]`).
- `Vec::new()` creates an empty `Vec`.
- `push` appends an element at the end (requires `let mut`).
- Index with `v[0]`, `v[1]`, etc.; get the length with `v.len()` (a method returning the element count).
- Iterate with `for x in &v` (borrowing, no move).
- `Vec` handles a lot like an array, but its size can change.

4.13 `Vec` and Ownership

4.13.1 Goal of This Episode

Understand `Vec`'s ownership behavior, and its symmetry with `String` / `&str`.

4.13.2 Concept

4.13.2.1 `Vec` and `String` Are a Pair

In recent episodes we learned the relationship between `String` and `&str`:

Owned version	Borrowed version
`String`	`&str`

`Vec` has exactly the same correspondence:

Owned version	Borrowed version
`Vec`	`&[T]` (a slice)

A `String` owns a piece of text; an `&str` borrows a piece of text. A `Vec` owns a set of elements; an `&[T]` borrows a set of elements. **Perfectly symmetric concepts.**

4.13.2.2 `Vec` Moves

A `Vec`'s data lives mainly on the heap, so it's not `Copy`. Assignment and passing into functions both move:

```
    let v1 = vec![1, 2, 3];
    let v2 = v1; // Move! v1 can't be used anymore
```

Exactly like `String`.

4.13.2.3 Use Slices &[T] for Function Parameters

Same advice as with String / &str — if a function only needs to read the contents of a Vec of i32, use a slice &[i32]:

```
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}

fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let total = sum(&v); // &Vec of i32 auto-converts to &[i32]
    println!("Total: {}", total);
    println!("v is still here: {:?}", v);
}
```

Just as &String auto-converts to &str, an &Vec of i32 auto-converts to &[i32].

4.13.2.4 for Loops and Ownership

This point matters a lot: when a for loop iterates a Vec, you choose between move and borrow:

for x in v — move!

```
fn main() {
    let v = vec![1, 2, 3];
    for x in v {
        println!("{}", x);
    }
    // v was moved away; it can't be used anymore!
}
```

for x in v consumes the whole Vec. After the loop, v no longer exists.

for x in &v — borrow!

```
fn main() {
    let v = vec![1, 2, 3];
    for x in &v {
        println!("{}", x); // x has type &i32
    }
    println!("v is still here: {:?}", v); // OK!
}
```

for x in &v merely borrows; v isn't consumed.

One detail matters here: x is not an i32; it is a reference, with type &i32. Because the loop iterates over the borrowed &v, each element it receives is borrowed too, rather than moved out of the Vec. Similarly, the earlier function parameter nums: &[i32] is already a borrowed slice, so the x in for x in nums is also an &i32.

Most of the time you should use for x in &v, unless you're certain you won't need the Vec again.

4.13.3 Example Code

```rust
// Slice parameters: &Vec of i32 auto-converts to &[i32]
fn sum(nums: &[i32]) -> i32 {
    let mut total = 0;
    for x in nums {
        total += x;
    }
    total
}

fn print_all(nums: &[i32]) {
    let mut first = true;
    for x in nums {
        if first {
            first = false;
        } else {
            print!(", ");
        }
        print!("{}", x);
    }
    println!();
}

fn main() {
    // Vec moves
    let v1 = vec![10, 20, 30];
    let v2 = v1.clone(); // clone keeps v1
    println!("v1 = {:?}", v1);
    println!("v2 = {:?}", v2);

    // Functions with slice parameters (borrowing)
    let scores = vec![85, 92, 78, 95, 88];
    println!("Total = {}", sum(&scores));
    print_all(&scores);
    println!("scores is still here: {:?}", scores);

    // Slice operations
    let slice = &scores[1..4]; // Borrowing a part
    println!("The middle three: {:?}", slice);
    println!("Total of the middle three = {}", sum(slice));

    // for x in &v: borrowing iteration
    println!("Listing one by one (borrowed):");
    for s in &scores {
        println!("  {}", s);
    }
    println!("scores is still here: {:?}", scores);

    // for x in v: moving iteration (gone after use)
    let temp = vec![1, 2, 3];
    println!("Consuming iteration:");
    for x in temp {
        println!("  {}", x);
    }
```

```
    // temp has been moved; the line below would be a compile error:
    // println!("{:?}", temp);

    // The symmetry, summarized
    // String  ↔ &str     (own ↔ borrow, text)
    // Vec     ↔ &[T]     (own ↔ borrow, a set of values)
    println!("--- The symmetry ---");
    let s = String::from("hello");
    let s_ref: &str = &s; // &String → &str
    println!("String: {}, &str: {}", s, s_ref);

    let v = vec![1, 2, 3];
    let v_ref: &[i32] = &v; // &Vec of i32 → &[i32]
    println!("Vec: {:?}, slice: {:?}", v, v_ref);
}
```

4.13.4 How Do You Write the Type of "a `Vec` of `i32`"?

Throughout this episode we kept saying "a `Vec` of `i32`" — but you may have noticed the code never once spelled that type out. Variable types were all inferred by Rust, and function parameters only used the slice `&[i32]`. What if you someday need to write it by hand (say, as a parameter or return type)? And what exactly can that `T` in `&[T]` from the table above be? Both questions have the same answer — and the next chapter spends a great deal of time on it.

4.13.5 Recap

- **`Vec` and `String` have perfectly symmetric ownership behavior**: both keep their data mainly on the heap, both move, both can `clone`.
- `String` ↔ `&str` mirrors `Vec` ↔ `&[T]` (own ↔ borrow).
- `&Vec` auto-converts to `&[T]` (just like `&String` to `&str`).
- Prefer slice parameters `&[T]` over `&Vec`.
- `for x in v`: **move** — consumes the whole `Vec`.
- `for x in &v`: **borrow** — the `Vec` survives; in this example, `x` is a reference with type `&i32`.
- Mostly use `for x in &v`, unless you're sure you're done with the `Vec`.
- We never wrote out the type of "a `Vec` of `i32`" by hand — how to write it, and what the `T` in `&[T]` is, gets revealed next chapter.

Congratulations on finishing Chapter 4! 🎉 In this chapter you learned Rust's most central concepts — ownership, moves, `clone`, `Copy`, borrowing — plus `String` and `Vec`, the two most commonly used non-`Copy` types. These concepts are Rust's biggest departure from other languages, and the key to how Rust guarantees memory safety without sacrificing performance. Next chapter, we move into generics, `trait` bounds, and lifetimes — letting your code handle arbitrary types while staying type-safe!

Chapter 5

Generics, Trait Bounds, and Lifetimes

If Chapter 3 was about building high-level abstractions for humans to understand, and Chapter 4 about imposing restrictions in step with modern computer hardware to achieve efficiency, then this chapter extends and combines the two. On the abstraction side, we introduce types that can take types as parameters — generics. Then we'll learn about `traits`, which can constrain generics. Finally we'll come to see that lifetimes not only deliver the performance of a low-level language, but also integrate perfectly with the type system, as in a high-level one.

5.1 Generic Functions

5.1.1 Goal of This Episode

Learn to define generic functions with <T>, so one function can handle different types.

5.1.2 Concept

In Chapter 4 we learned `Vec` and used it to store a bunch of `i32`s. But did you notice we always wrote `vec![1, 2, 3]` and let Rust infer the type?

In truth, `Vec` isn't a complete type. Its full form is `Vec<i32>`, `Vec<String>`, `Vec<bool>` — the angle brackets <> hold "what type of thing this `Vec` stores."

Chapter 4 deliberately kept quiet about the angle brackets, because we hadn't learned generics yet. Now it's time to lift the veil.

5.1.2.1 What Are Generics?

Suppose you want a function that takes two values and returns the first:

```
fn first_i32(a: i32, b: i32) -> i32 {
    a
}
```

What if you also need to handle `f64`? Surely not a whole separate `first_f64`?

Generics solve this. We use a "type parameter" `T` in place of a concrete type:

```
fn first<T>(a: T, b: T) -> T {
    a
}
```

The <T> after the function name says "this function has a type parameter named `T`." The parameters `a` and `b` both have type `T`, and so does the return value.

When you call `first(10, 20)`, Rust sees `10` is an `i32` and knows `T = i32`. Calling `first(3.14,`

`2.71)` makes `T = f64`. One function definition, automatically fitting different types.

5.1.2.2 Naming Convention

Type parameters usually use single capital letters: `T` (Type), `U`, `V`. Longer, meaningful names appear when there's semantic weight, but `T` is fine for now.

5.1.3 Example Code

```
// A generic function: return the first of two values
fn first<T>(a: T, _b: T) -> T {
    a
}

// Multiple type parameters are allowed
fn make_pair<T, U>(a: T, b: U) -> (T, U) {
    (a, b)
}

fn main() {
    // T is inferred as i32
    let x = first(10, 20);
    println!("{}", x);

    // T is inferred as &str
    let y = first("hello", "world");
    println!("{}", y);

    // But both arguments must share a type, since first's parameters are both T
    // let bad = first(1, "a"); // Compile error! 1 is i32 but "a" is &str

    // T = i32, U = &str
    let pair = make_pair(42, "hello");
    println!("{:?}", pair);
}
```

5.1.4 Recap

- `Vec`'s full form is `Vec<T>`, with a type parameter in the angle brackets — Chapter 4 deliberately omitted this; now it's official.
- Generic functions declare type parameters with `<T>`, letting one function handle different types.
- Rust infers what `T` is from the values passed in.
- Multiple type parameters are allowed: `<T, U>`.
- Type parameters conventionally use capital letters: `T`, `U`, `V`.

5.2 Generic `structs`

5.2.1 Goal of This Episode

Learn to define `structs` with type parameters, so one structure can hold data of different types.

5.2.2 Concept

Last episode we learned generic functions. Well, `structs` can have type parameters too!

For example, `Vec<T>` is a generic `struct`: `Vec<i32>` and `Vec<String>` use the same `struct` definition, just holding different types. We can define generic `structs` of our own the same way.

5.2.2.1 Defining a Generic `struct`

```
struct Pair<T> {
    first: T,
    second: T,
}
```

The `<T>` after the `struct` name says "`Pair` has one type parameter `T`." Both `first` and `second` have type `T`, so they must be the same type.

In use:

```
    let p = Pair { first: 1, second: 2 };       // T = i32
    let q = Pair { first: "hi", second: "yo" }; // T = &str
```

5.2.2.2 Multiple Type Parameters

If you want `first` and `second` to be different types, use two type parameters:

```
struct MixedPair<T, U> {
    first: T,
    second: U,
}
```

Exactly the same idea as last episode's `make_pair<T, U>`.

5.2.3 Example Code

```
// The two fields must share a type
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

// The two fields may differ in type
#[derive(Debug)]
struct MixedPair<T, U> {
    first: T,
    second: U,
}

fn main() {
    let int_pair = Pair { first: 10, second: 20 };
    println!("{:?}", int_pair);

    let str_pair = Pair { first: "hello", second: "world" };
    println!("{:?}", str_pair);
```

```
    // Pair<T>'s two fields must share a type; this would be a compile error:
    // let bad = Pair { first: 42, second: "oops" };

    let mixed = MixedPair { first: 42, second: "answer" };
    println!("{:?}", mixed);
}
```

5.2.4 Recap

- A `struct` can declare type parameters with `<T>`, making one definition fit many types.
- `Pair<T>`'s two fields are both `T`, so they must share a type.
- When different types are needed, use multiple type parameters: `MixedPair<T, U>`.
- As with generic functions, Rust infers type parameters from usage.

5.3 Generic `enum`s

5.3.1 Goal of This Episode

Learn to define `enum`s with type parameters.

5.3.2 Concept

Last episode was generic `struct`s; this one is generic `enum`s. The idea is exactly the same — add `<T>` after the `enum` name, and the data variants carry can be of any type.

5.3.2.1 Defining a Generic `enum`

Suppose we want a "maybe there's a value" type — it might hold something, or be empty:

```
enum Maybe<T> {
    Something(T),
    Nothing,
}
```

`Something(T)` carries a value of type `T`; `Nothing` carries nothing.

Generic `enum`s can have multiple type parameters too. Say, an "either-or" type:

```
enum Either<L, R> {
    Left(L),
    Right(R),
}
```

An `Either<L, R>` is either `Left(L)` or `Right(R)` — the two types fully independent.

5.3.3 Example Code

```
// Our own generic enum
#[derive(Debug)]
enum Maybe<T> {
    Something(T),
    Nothing,
}
```

```rust
// A generic enum with two type parameters
#[derive(Debug)]
enum Either<L, R> {
    Left(L),
    Right(R),
}

fn main() {
    let a: Maybe<i32> = Maybe::Something(42);
    let b: Maybe<i32> = Maybe::Nothing;

    println!("{:?}", a);
    println!("{:?}", b);

    // Extracting the value with match
    match a {
        Maybe::Something(val) => println!("There's something inside: {}", val),
        Maybe::Nothing => println!("Empty"),
    }

    // Two type parameters
    let x: Either<i32, &str> = Either::Left(100);
    let y: Either<i32, &str> = Either::Right("hello");

    println!("{:?}", x);
    println!("{:?}", y);
}
```

5.3.4 Recap

- `enum`s can take type parameters too: `enum Maybe<T> { ... }`.
- The data a variant carries can be generalized with `T`.
- Multiple type parameters are allowed: `enum Either<L, R> { Left(L), Right(R) }`.
- The standard library has many important generic `enum`s — we'll meet them in due course.

5.4 The Turbofish Syntax

5.4.1 Goal of This Episode

Learn to specify type parameters manually with the `::<>` turbofish syntax, and understand its relationship to generic definitions.

5.4.2 Concept

Over the past few episodes we've learned generics — functions, `struct`s, and `enum`s can all take type parameters `<T>`. Most of the time Rust infers what `T` is, but sometimes the compiler can't work it out, and we must tell it ourselves.

5.4.2.1 What's a Turbofish?

Remember writing this back in Chapter 1 when learning `parse`?

```rust
fn main() {
    let input = "1";
    let num = input.trim().parse::<i32>().expect("not a number");
}
```

Back then we copied `::<i32>` as a black box. Now, with generics learned, we can finally understand it!

`.parse()` is a generic method with a type parameter `T`, meaning "what type you want to turn the string into." But from `input.trim().parse()` alone, the compiler can't tell whether you want an `i32`, an `f64`, or something else.

So we manually specify `T = i32` with `::<i32>`. This `::<>` syntax is called the **turbofish** (because `::<>` looks like a fish 🐟).

5.4.2.2 The Essence of the Turbofish

The turbofish is "manually filling in the type parameters declared in the generic definition's angle brackets":

- Generic definition: `fn parse<T>(...)` — the `<T>` here is the declaration.
- Turbofish: `.parse::<i32>()` — the `::<i32>` here fills it in.

Functions, methods, and types can all take a turbofish:

```rust
// Turbofish on a function
func::<i32>(arg);

// Turbofish on a type
Vec::<i32>::new();
```

5.4.2.3 What Does `.parse()` Do?

While we're at it, the full story on `parse`: it converts a string into the type you specify. The conversion can fail (e.g. `"abc"` can't become a number), so it pairs with `.expect()` to handle failure — as we did back in Chapter 1.

5.4.3 Example Code

```rust
fn first<T>(a: T, _b: T) -> T {
    a
}

fn main() {
    // Usually Rust infers on its own; no turbofish needed
    let x = first(10, 20);
    println!("{}", x);

    // Manually specifying the type with a turbofish
    let y = first::<f64>(3.14, 2.71);
    println!("{}", y);

    // Turbofish on Vec
    let v = Vec::<i32>::new();
    println!("{:?}", v);
```

```
    // Turbofish on parse — echoing Chapter 1's black box
    let input = "42";
    let num = input.parse::<i32>().expect("not a number");
    println!("{}", num);

    let pi = "3.14".parse::<f64>().expect("not a number");
    println!("{}", pi);
}
```

5.4.4 Recap

- The **turbofish** `::<>` is the syntax for specifying generic type parameters manually.
- Most of the time Rust infers automatically and no turbofish is needed.
- When the compiler can't infer the type (e.g. `.parse()`), use the turbofish.
- Chapter 1's `.parse::<i32>()` was a turbofish all along — now we understand why it's written that way.
- `.parse()` converts a string into the given type; conversion can fail, hence the `.expect()`.

5.5 The Placeholder Type _

5.5.1 Goal of This Episode

Learn to use _ in type annotations to let the compiler infer part of a type.

5.5.2 Concept

Last episode's turbofish specifies all the type parameters by hand. But sometimes you only want to specify some of them and let Rust infer the rest. That's when _ serves as a type-level wildcard.

5.5.2.1 _ as a Type Placeholder

Take this example:

```
fn main() {
    let v: Vec<_> = vec![1, 2, 3];
}
```

We're telling Rust "this is a `Vec`," while the element type _ says "you figure it out." Rust sees `1, 2, 3` are integers and infers _ = `i32`.

_ works inside a turbofish too:

```
    let v = Vec::<_>::new();
```

Though written this way it's really no different from `Vec::new()` with full inference. _ shines when you need to specify the outer type but let Rust infer the inner one.

5.5.2.2 When Is It Useful?

When a type has several parameters and you only want to annotate some. The power of _ grows with the type's complexity — you'll feel it naturally once we meet more standard-library types.

5.5.3 Example Code

```rust
fn main() {
    // Let Rust infer the Vec's element type with _
    let v: Vec<_> = vec![1, 2, 3];
    println!("{:?}", v);

    // _ works in a turbofish too
    let v2 = Vec::<_>::new(); // Same as Vec::new(); _ lets Rust infer
    let v2: Vec<i32> = v2;    // The type is inferred from later usage
    println!("{:?}", v2);

    // Comparison: no annotation at all vs partial annotation with _
    let a = vec![true, false]; // Rust infers everything: Vec<bool>
    let b: Vec<_> = vec![true, false]; // Tell Rust it's a Vec; element type inferred
    println!("{:?}", a);
    println!("{:?}", b);
}
```

5.5.4 Recap

- _ can stand in as a placeholder in type annotations, letting Rust infer that position's type.
- Good for "I know the outer type; let Rust infer the inner one."
- Both turbofish and `let` annotations can use _.

5.6 Type Aliases

5.6.1 Goal of This Episode

Learn to create type aliases with `type`, making complex generic types easier to read.

5.6.2 Concept

Now that we know generics, types will get increasingly complex. For example, a three-dimensional data structure:

```rust
Vec<Vec<Vec<i32>>>
```

Writing the full type every time is tiring, and hard to read. Rust offers the `type` keyword for creating **type aliases**:

```rust
type Grid3D = Vec<Vec<Vec<i32>>>;
```

From then on, `Grid3D` and `Vec<Vec<Vec<i32>>>` are the same type — just under a different name. It doesn't create a new type; it's purely shorthand.

5.6.2.1 A Simple Alias

```rust
type Name = String;
```

`Name` and `String` are fully equivalent, usable interchangeably.

5.6.2.2 Type Aliases with Parameters

Type aliases can take generic parameters too:

```
type Pair<T> = (T, T);
```

Now `Pair<i32>` equals `(i32, i32)`, and `Pair<String>` equals `(String, String)`.

5.6.2.3 Note

A type alias is only shorthand, not a new type. `Name` and `String` are freely interchangeable — the compiler treats them as one and the same type.

5.6.3 Example Code

```
// A simple type alias
type Name = String;

// Simplifying a complex nested type
type Grid3D = Vec<Vec<Vec<i32>>>;

// An alias with a generic parameter
type Pair<T> = (T, T);

fn main() {
    // Name IS String
    let greeting: Name = String::from("Hello");
    println!("{}", greeting);

    // A 3D Vec is much tidier with an alias
    let mut grid: Grid3D = vec![vec![vec![0; 3]; 3]; 3];
    grid[1][1][1] = 42;
    println!("grid[1][1][1] = {}", grid[1][1][1]);

    // Pair<i32> IS (i32, i32)
    let point: Pair<i32> = (3, 7);
    println!("{:?}", point);

    let coords: Pair<f64> = (1.5, 3.7);
    println!("{:?}", coords);
}
```

5.6.4 Recap

- `type Name = ExistingType;` creates a type alias — shorthand only, not a new type.
- Type aliases can take generic parameters: `type Pair<T> = (T, T);`.
- Common use: simplifying complex nested types (like `Vec<Vec<Vec<i32>>>`).
- An alias is fully equivalent to the original type, usable interchangeably.

5.7 Generic impl

5.7.1 Goal of This Episode

Learn to implement methods for a generic struct, and understand what the two Ts in the impl<T> syntax mean.

5.7.2 Concept

In Episode 2 we defined the generic struct Pair<T>. This episode we give it an impl.

Recall from Chapter 3, impl on a struct looks like this:

```
impl Point {
    fn sum(&self) -> i32 {
        self.x + self.y
    }
}
```

What about a generic struct?

5.7.2.1 The impl<T> Syntax

```
impl<T> Pair<T> {
    fn new(first: T, second: T) -> Pair<T> {
        Pair { first, second }
    }
}
```

Note there are **two** Ts in different positions, playing different roles:

1. The <T> in impl<T>: **declares** a type parameter T. It tells Rust "I'm about to use a type parameter named T."
2. The <T> in Pair<T>: **uses** the just-declared T. It tells Rust "the type I'm implementing for is Pair<T>."

In other words: impl<T> declares T, then hands T to Pair<T> — "for any type T, implement the following methods for Pair<T>."

If you wrote only impl Pair<T> without the impl<T>, Rust would assume T is a concrete type name (like i32 or String), fail to find a type called T, and report an error.

Conversely, writing impl Pair<i32> (no impl<T> needed) adds methods only to Pair<i32> — Pair<String> and the rest get nothing.

5.7.2.2 Using T inside Methods

Once T is declared, the whole impl block can use it:

```
impl<T> Pair<T> {
    fn new(first: T, second: T) -> Pair<T> {
        Pair { first, second }
    }

    fn first(&self) -> &T {
        &self.first
```

```
    }
}
```

5.7.2.3 Implementing `traits` Works the Same Way

When Chapter 4 taught `traits`, we implemented them for concrete types, like `impl Greet for Cat`. To implement a `trait` for a generic type, the syntax is the same — declare the type parameter after `impl`:

```
impl<T> SomeTrait for Pair<T> {
    // ...
}
```

Again: "for any type `T`, implement this `trait` for `Pair<T>`."

5.7.3 Example Code

```
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

impl<T> Pair<T> {
    // Associated function
    fn new(first: T, second: T) -> Pair<T> {
        Pair { first, second }
    }

    // Method: return a reference to first
    fn first(&self) -> &T {
        &self.first
    }

    // Method: return a reference to second
    fn second(&self) -> &T {
        &self.second
    }
}

fn main() {
    let p = Pair::new(10, 20);
    println!("first = {}", p.first());
    println!("second = {}", p.second());
    println!("{:?}", p);

    let q = Pair::new("hello", "world");
    println!("first = {}", q.first());
    println!("second = {}", q.second());
}
```

5.7.4 Recap

- To implement methods for a generic `struct`, write `impl<T> Pair<T> { ... }`.

- The <T> in impl<T> **declares** T; the <T> in Pair<T> **uses** T.
- Bottom line: impl<T> declares T, then hands it to Pair<T>.
- Once declared, every method in the impl block can use T.
- Implementing a trait for a generic type is similar: impl<T> SomeTrait for Pair<T> { ... }.

5.8 Option<T>

5.8.1 Goal of This Episode

Meet the most important generic enum in Rust's standard library — Option<T> — and understand how it replaces null and prevents runtime errors.

5.8.2 Concept

5.8.2.1 The Problem with null

In some programming languages, any variable can be null (empty). This causes a classic problem: you assume a variable has a value, use it, and the program blows up at runtime — the "null pointer exception." Tony Hoare, null's inventor, even called it his "billion-dollar mistake."

Rust's solution is simple: **there is no null.**

In its place stands a generic enum: Option<T>.

5.8.2.2 The Definition of Option

Option<T> looks like this (the standard library defines it for you):

```
enum Option<T> {
    Some(T),
    None,
}
```

Doesn't it look a lot like the Maybe<T> we wrote ourselves in Episode 3? Exactly! Same concept:

- Some(T) means "there's a value of type T."
- None means "no value."

5.8.2.3 Forced Handling of None

The brilliance of Option: the compiler **forces** you to handle the "no value" case. You can't use an Option<i32> directly as an i32 — you must first check whether it's Some or None.

That's where match comes in:

```
    match maybe_value {
        Some(v) => println!("There's a value: {}", v),
        None => println!("No value"),
    }
```

5.8.2.4 No Full Path Needed for Option

Because Option, Some, and None are so commonly used, Rust brings them into every file by default. So there's no need to write Option::Some(42) — just Some(42).

5.8.2.5 The Zero-cost Secret: Niche Optimization

A fun bit of trivia: `Option<&T>` occupies exactly as much memory as a plain reference `&T`!

Since a reference `&T` can never be null, Rust cleverly uses null in memory to represent `None` — no extra space needed. This is called **niche optimization**: exploiting a type's "impossible values" to squeeze in extra information.

5.8.3 Example Code

```
// Find the first even number in a slice; return None if there isn't one
fn find_even(numbers: &[i32]) -> Option<i32> {
    for n in numbers {
        if n % 2 == 0 {
            return Some(*n);
        }
    }
    None
}

fn main() {
    let nums = vec![1, 3, 5, 8, 11];
    let result = find_even(&nums);

    // Extracting the Option's value with match
    match result {
        Some(n) => println!("Found an even number: {}", n),
        None => println!("No even numbers"),
    }

    let odds = vec![1, 3, 5, 7];
    let result2 = find_even(&odds);

    match result2 {
        Some(n) => println!("Found an even number: {}", n),
        None => println!("No even numbers"),
    }
}
```

5.8.4 Recap

- `Option<T>` is Rust's generic `enum` for "possibly no value," replacing other languages' null.
- `Some(T)` means a value exists; `None` means it doesn't.
- The compiler forces you to handle the `None` case — no null pointer exceptions at runtime.
- `Option`, `Some`, and `None` are so common Rust imports them by default; no extra path needed.
- Niche optimization: `Option<&T>` is the same size as `&T` — zero extra cost.

5.9 Common Option Methods

5.9.1 Goal of This Episode

Learn Option's common methods — unwrap, expect, unwrap_or, flatten — plus extracting values with if let.

5.9.2 Concept

Last episode we handled Option with match, the safest way. But writing match every time can be long-winded. Rust offers some convenient methods.

5.9.2.1 unwrap: Brute-force Extraction

```
    let x: Option<i32> = Some(42);
    let value = x.unwrap(); // 42
```

If it's Some, you get the value inside directly. But if it's None, the program panics (crashes)! So use unwrap with care — usually when you're certain it can't be None.

5.9.2.2 expect: unwrap with a Message

```
    let x: Option<i32> = None;
    let value = x.expect("this shouldn't be None"); // Panics, printing your message
```

Same as unwrap, but the panic prints your custom message — handy for debugging.

5.9.2.3 unwrap_or: Providing a Default

```
    let x: Option<i32> = None;
    let value = x.unwrap_or(0); // 0
```

If it's Some, extract the value; if None, use the default you supplied. No panics — very safe.

5.9.2.4 flatten: Squashing Nested Options

Sometimes you run into the nested structure Option<Option<T>>:

```
    let nested: Option<Option<i32>> = Some(Some(42));
    let flat: Option<i32> = nested.flatten(); // Some(42)
```

flatten squashes two layers of Option into one. If either the outer or inner layer is None, the result is None.

5.9.3 Example Code

```
fn find_even(numbers: &[i32]) -> Option<i32> {
    for n in numbers {
        if n % 2 == 0 {
            return Some(*n);
        }
    }
    None
}
```

```rust
fn main() {
    let nums = [1, 3, 5, 7];
    let has_even = [2, 4, 6];

    // unwrap_or: safely provide a default
    let result = find_even(&nums).unwrap_or(0);
    println!("Even number (0 if not found): {}", result);

    // expect: for when you're sure there's a value
    let result2 = find_even(&has_even).expect("there should be an even number");
    println!("Found an even number: {}", result2);

    // if let: the syntax from Chapter 3
    if let Some(n) = find_even(&has_even) {
        println!("Extracted with if let: {}", n);
    }

    // flatten: squashing a nested Option
    let nested: Option<Option<i32>> = Some(Some(42));
    let flat = nested.flatten();
    println!("{:?}", flat);

    let nested_none: Option<Option<i32>> = Some(None);
    let flat_none = nested_none.flatten();
    println!("{:?}", flat_none);

    let outer_none: Option<Option<i32>> = None;
    let flat_outer = outer_none.flatten();
    println!("{:?}", flat_outer);
}
```

5.9.4 Recap

- `unwrap()`: extracts the `Some` value; panics on `None` — handle with care.
- `expect("message")`: like `unwrap`, but the panic prints your custom message.
- `unwrap_or(default)`: returns the default on `None`; never panics.
- `flatten()`: squashes an `Option<Option<T>>` into an `Option<T>`.
- Pairing with `if let Some(x) = ...` (from Chapter 3) is convenient too.

5.10 `Result<T, E>`

5.10.1 Goal of This Episode

Learn to handle fallible operations with `Result<T, E>`, and understand its symmetry with `Option`.

5.10.2 Concept

The last two episodes covered `Option<T>` — "maybe there's a value, maybe not." But sometimes "no value" isn't enough — you also need to know **why** there isn't one.

Take parsing a number: on failure, you want to know whether it was "bad format" or "number

too large." That's what `Result<T, E>` is for.

5.10.2.1 The Definition of `Result`

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- `Ok(T)` means success, wrapping the successful value.
- `Err(E)` means failure, wrapping the error information.

Like `Option`, `Result`, `Ok`, and `Err` are imported into every file by default.

5.10.2.2 The Symmetry between `Option` and `Result`

Option	Result
Some(T)	Ok(T)
None	Err(E)

`Option` only knows "there is or there isn't"; `Result` also knows "why there isn't."

5.10.2.3 Revisiting Chapter 1's Black Box

Remember Chapter 1's `.expect("failed to read input")` and `.parse::<i32>().expect("not a number")`?

What `.parse()` returns is a `Result`. And `expect` behaves exactly like `Option`'s `expect` — on success, extract the `Ok` value; on failure, panic and print your message.

Now we can finally understand Chapter 1's "black box" code in full.

5.10.2.4 Common Methods

Like `Option`, `Result` has:

- `.unwrap()`: extract the value on success; panic on failure.
- `.expect("message")`: like `unwrap`, with a custom panic message.
- `.unwrap_or(default)`: use the default on failure.

5.10.3 Example Code

```
fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("The divisor can't be zero"))
    } else {
        Ok(a / b)
    }
}

fn main() {
    // Handling a Result with match
    let result = divide(10, 3);
    match result {
```

```
        Ok(value) => println!("10 / 3 = {}", value),
        Err(msg) => println!("Error: {}", msg),
    }

    // The division-by-zero case
    let bad = divide(10, 0);
    match bad {
        Ok(value) => println!("Result: {}", value),
        Err(msg) => println!("Error: {}", msg),
    }

    // unwrap_or: a default on failure
    let safe = divide(10, 0).unwrap_or(0);
    println!("The safe result: {}", safe);

    // Back to Chapter 1: parse returns a Result
    let input = "42";
    let num: Result<i32, _> = input.parse();
    match num {
        Ok(n) => println!("Parsed successfully: {}", n),
        Err(e) => println!("Parse failed: {:?}", e),
    }

    // expect: for when you're sure it won't fail
    let num2 = "100".parse::<i32>().expect("this shouldn't fail");
    println!("{}", num2);
}
```

5.10.4 Recap

- `Result<T, E>` expresses "success (`Ok`) or failure (`Err`)" — `Option` plus error information.
- `Ok(T)` corresponds to success; `Err(E)` to failure.
- Like `Option`, `Result`, `Ok`, and `Err` come pre-imported in every file.
- `unwrap`, `expect`, and `unwrap_or` work exactly as they do with `Option`.
- Chapter 1's `.parse().expect(...)` was using `Result` all along — now we understand.

5.11 The ? Operator

5.11.1 Goal of This Episode

Learn to streamline error propagation with the `?` operator, avoiding `match` after `match`.

5.11.2 Concept

Last episode's `Result` was handled with `match` for success and failure. But what if one function has several fallible operations?

```
fn do_stuff() -> Result<i32, String> {
    let a = match "42".parse::<i32>() {
        Ok(n) => n,
        Err(e) => return Err(format!("{:?}", e)),
    };
```

```
    let b = match "10".parse::<i32>() {
        Ok(n) => n,
        Err(e) => return Err(format!("{:?}", e)),
    };
    Ok(a + b)
}
```

A `match` for every `parse` — far too wordy. The `?` operator solves exactly this.

5.11.2.1 The Essence of `?`

Placed after a `Result`, `?` does the following:

- If it's `Ok(v)`, extract `v` and keep going.
- If it's `Err(e)`, return the `Err`, leaving the function early.

So `?` is shorthand for `match` + early `return`.

5.11.2.2 Note: Error Types Must Line Up

When using `Result`, the type inside the `Err` must match the function's declared `Err` type, or be related to it in a certain way (we'll cover exactly which way later). Without that relationship, `?` can't be used directly — you must first convert the error to the right type.

For instance, `.parse()`'s error type is `std::num::ParseIntError`, but your function returns `Result<_, String>`. You can convert it yourself with `match` and a manual `return`:

```
    let n = match input.parse::<i32>() {
        Ok(v) => Ok(v),
        Err(e) => return Err(format!("{:?}", e)),
    };
```

Or wrap a helper function that converts the error first; once that helper returns, `?` works directly — the example code below does exactly that.

Later we'll cover more convenient ways to handle this situation, without hand-converting error types every time.

5.11.2.3 `?` Works on `Option` Too

`?` isn't just for `Result` — it works on `Option` as well: on `None`, it simply `return None`s.

5.11.2.4 `main` Can Return a `Result` Too

If the `main` function returns `Result<(), String>`, you can use `?` inside `main`.

5.11.3 Example Code

```
// A helper function that converts the error type by hand
fn parse_i32(input: &str) -> Result<i32, String> {
    match input.parse::<i32>() {
        Ok(n) => Ok(n),
        Err(e) => Err(format!("Failed to parse '{}': {:?}", input, e)),
    }
}

// Streamlining error propagation with ?
```

```rust
fn add_two_strings(a: &str, b: &str) -> Result<i32, String> {
    let x = parse_i32(a)?; // Extract on Ok; return early on Err
    let y = parse_i32(b)?;
    Ok(x + y)
}

// ? on an Option: is the first element positive?
fn first_is_positive(numbers: &[i32]) -> Option<bool> {
    // If the slice is empty, .first() returns None, and ? returns None immediately
    let first = numbers.first()?;
    Some(*first > 0)
}

// main can return a Result too, enabling ?
fn main() -> Result<(), String> {
    let result = add_two_strings("42", "10")?;
    println!("42 + 10 = {}", result);

    // The error case
    let bad = add_two_strings("42", "abc");
    match bad {
        Ok(n) => println!("Result: {}", n),
        Err(e) => println!("Error: {}", e),
    }

    let nums = [3, 7, 2];
    match first_is_positive(&nums) {
        Some(true) => println!("The first element is positive"),
        Some(false) => println!("The first element isn't positive"),
        None => println!("An empty slice"),
    }

    let empty: &[i32] = &[];
    match first_is_positive(empty) {
        Some(b) => println!("Result: {}", b),
        None => println!("Empty slice; no first element"),
    }

    Ok(())
}
```

5.11.4 Recap

- `?` is shorthand for `match` + early `return`.
- `?` on a `Result`: extract on `Ok`, return early on `Err`.
- `?` on an `Option`: extract on `Some`, return early on `None`.
- When using `?`, the error type must match the function's return type or be related to it; otherwise you convert it yourself.
- `fn main() -> Result<(), String>` lets `main` use `?` too.

5.12 `traits` with Multiple Methods and Default Implementations

5.12.1 Goal of This Episode

Learn to define multiple methods in a `trait`, and use default implementations so implementers only override what they need.

5.12.2 Concept

When Chapter 4 introduced `traits`, ours had just one method each. In fact a `trait` can have many methods, and some can come with a **default implementation** — a pre-written "generic version" that implementers can override if they don't like it.

5.12.2.1 Multiple Methods

```
trait Describe {
    fn name(&self) -> String;
    fn description(&self) -> String;
}
```

When implementing, every method must be provided:

```
impl Describe for Cat {
    fn name(&self) -> String { ... }
    fn description(&self) -> String { ... }
}
```

5.12.2.2 Default Implementations

Some methods can ship with a sensible default version:

```
trait Describe {
    fn name(&self) -> String;

    fn description(&self) -> String {
        let n = self.name();
        let mut result = String::from("I am ");
        result.push_str(&n);
        result
    }
}
```

`description` has a default implementation that calls `.name()` to build the string. When implementing `Describe`, you only need to supply `.name()` — `description()` automatically uses the default.

Of course, you can also override the default with your own version.

5.12.3 Example Code

```
trait Describe {
    // A method that must be implemented
    fn name(&self) -> String;

    // A default implementation: usable as-is, or overridable
```

```
    fn description(&self) -> String {
        let n = self.name();
        let mut result = String::from("I am ");
        result.push_str(&n);
        result
    }
}

struct Cat {
    nickname: String,
}

struct Dog {
    nickname: String,
}

// Cat implements only name; description uses the default
impl Describe for Cat {
    fn name(&self) -> String {
        self.nickname.clone()
    }
}

// Dog overrides description
impl Describe for Dog {
    fn name(&self) -> String {
        self.nickname.clone()
    }

    fn description(&self) -> String {
        let n = self.name();
        let mut result = String::from("Woof! My name is ");
        result.push_str(&n);
        result.push_str(", and I'm a dog!");
        result
    }
}

fn main() {
    let cat = Cat { nickname: String::from("Tangerine") };
    let dog = Dog { nickname: String::from("Shiba") };

    // Cat uses the default description
    println!("{}", cat.name());
    println!("{}", cat.description());

    // Dog uses its custom description
    println!("{}", dog.name());
    println!("{}", dog.description());
}
```

5.12.4 Recap

- A `trait` can define multiple methods.

- Methods can have a **default implementation** — write `{ ... }` after the method instead of `;`.
- Default implementations can call other methods of the same `trait`.
- When implementing a `trait`, methods with defaults may be skipped (using the default) or overridden.

5.13 `trait` Bounds

5.13.1 Goal of This Episode

Learn to constrain a generic parameter's capabilities with `trait` bounds, and add methods to qualifying types with conditional `impl`.

5.13.2 Concept

Back in Episode 1's generic functions, we wrote `fn first<T>(a: T, b: T) -> T`. But what if you want to `clone` a value inside a generic function?

```
fn duplicate<T>(x: &T) -> (T, T) {
    (x.clone(), x.clone()) // Compile error!
}
```

The compiler complains: "Not every `T` has a `.clone()` method."

Fair enough — `T` could be any type. What if some type doesn't implement `Clone`?

5.13.2.1 `trait` Bounds: Constraining What `T` Can Do

The fix is a **`trait` bound**, telling Rust "`T` must implement `Clone`":

```
fn duplicate<T: Clone>(x: &T) -> (T, T) {
    (x.clone(), x.clone())
}

fn main() {}
```

`T: Clone` means "`T` must implement the `Clone` `trait`." Now Rust knows `x.clone()` is always callable.

5.13.2.2 `trait` Bounds Go Everywhere

`trait` bounds aren't just for functions. Nearly anywhere a generic parameter appears can take one — `struct`, `enum`, and `impl` definitions included:

```
struct Wrapper<T: Clone> {
    value: T,
}
```

5.13.2.3 Conditional `impl`

The most practical spot is on an `impl` block. This is a **conditional `impl`** — providing certain methods only when the type parameter meets certain conditions.

```
impl<T: Clone> Pair<T> {
    fn to_tuple(&self) -> (T, T) {
```

```
        (self.first.clone(), self.second.clone())
    }
}
```

This says: only when `T` implements `Clone` does `Pair<T>` have the `to_tuple` method.

5.13.2.4 The Effect in Practice

```
    let p1 = Pair::new(1, 2); // i32 has Clone
    let t = p1.to_tuple();    // Callable ✓

    let p2 = Pair::new(Pair::new(1, 2), Pair::new(3, 4)); // Pair doesn't derive Clone
    p2.to_tuple(); // Compile error! Pair<i32> doesn't implement Clone
```

`Pair<Pair<i32>>` can't call `.to_tuple()`, because `Pair<i32>` doesn't implement `Clone` (we never derived `Clone` for it).

5.13.3 Example Code

```
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

// Every Pair<T> has new
impl<T> Pair<T> {
    fn new(first: T, second: T) -> Pair<T> {
        Pair { first, second }
    }
}

// Only Pair<T> with T: Clone has to_tuple
impl<T: Clone> Pair<T> {
    fn to_tuple(&self) -> (T, T) {
        (self.first.clone(), self.second.clone())
    }
}

// Generic function + trait bound
fn duplicate<T: Clone>(x: &T) -> (T, T) {
    (x.clone(), x.clone())
}

fn main() {
    // i32 has Clone, so Pair<i32> has to_tuple
    let p = Pair::new(10, 20);
    let t = p.to_tuple();
    println!("{:?}", t);

    // The generic function works too
    let pair = duplicate(&42);
    println!("{:?}", pair);
```

```
    let pair2 = duplicate(&String::from("hello"));
    println!("{:?}", pair2);

    // Pair<Pair<i32>> can't call to_tuple
    // because Pair<i32> doesn't derive Clone
    let nested = Pair::new(Pair::new(1, 2), Pair::new(3, 4));
    println!("{:?}", nested);
    // nested.to_tuple(); // Compile error! Pair<i32> doesn't implement Clone
}
```

5.13.4 Recap

- The `trait` bound `T: Clone` requires `T` to implement a specific `trait`.
- `trait` bounds can go on functions, `structs`, `enums`, `impls` — any generic parameter.
- Without a `trait` bound, a generic function or method can't assume `T` has any capability.
- Conditional `impl`: `impl<T: Clone> Pair<T> { ... }` provides methods only when `T` qualifies.

5.14 use Basics

5.14.1 Goal of This Episode

Learn to shorten long paths with `use`, and understand why we could use `Option`, `Vec`, and friends without any `use` before.

5.14.2 Concept

Rust ships with a great many built-in functions, types, and `traits`. To organize them, the standard library sorts things into modules. Every type has a full **path** describing which module it lives in, with segments separated by `::` — like `std::string::String` (`String` lives in `std`'s `string` module), `std::vec::Vec`, `std::fmt::Display`. Normally, using a type means writing out its full path.

But strangely, we've been using `Vec`, `String`, `Option`, and `Result` all along without ever writing full paths like `std::vec::Vec`. Why?

Because Rust has a mechanism called the **prelude** — Rust imports the most commonly used functions, types, and `traits` into every file by default. `Vec`, `String`, `Option`, `Result`, `Some`, `None`, `Ok`, `Err`, plus common `traits` like `Clone` and `Copy`, are all in the prelude, so no full paths are needed.

But not everything is in the prelude. The `trait` `std::fmt::Display`, for instance, isn't. To use it, you either write the full path — or bring it in with `use`.

5.14.2.1 The use Syntax

```
use std::fmt::Display;
```

This line means: "Bring `std::fmt::Display` into the current scope; from now on, just write `Display`."

`use` brings an existing name into the current scope, which lets you write a shorter path. Without `use`, you write `std::fmt::Display`; with it, just `Display`.

5.14.3 Example Code

```
use std::cmp::max;

fn main() {
    // Without use, the full path is needed:
    println!("The smaller is: {}", std::cmp::min(3, 7));

    // With use, just max will do:
    println!("The larger is: {}", max(3, 7));
    println!("The larger is: {}", max(10, -2));
}
```

`std::cmp::max` and `std::cmp::min` are standard-library functions returning the larger or smaller of two values. They're not in the prelude, so it's either the full path or a `use`.

5.14.4 Recap

- `use std::fmt::Display;` shortens the long path; afterward, just write `Display`.
- `use` brings an existing name into the current scope, which lets you write a shorter path.
- Rust's compiler imports the prelude's common types and `traits` by default (`Vec`, `String`, `Option`, `Clone`, etc.).
- Things outside the prelude (like `Display`) need the full path or a `use`.

5.15 The `Display trait`

5.15.1 Goal of This Episode

Learn to implement the `Display trait` for custom types, the difference between `Display` and `Debug`, and the relationship between `Display` and `ToString`.

5.15.2 Concept

In Chapter 2 we learned `{:?}` for printing tuples, arrays, and `structs` with `#[derive(Debug)]`. But `{:?}` is the developer-facing "`Debug` format." To print a custom type with `{}`, you need to implement the `Display trait`.

5.15.2.1 `Display` vs `Debug`

- **`Debug`** (`{:?}`): a format for developers, auto-generated with `#[derive(Debug)]`.
- **`Display`** (`{}`): a format for end users, which **must be implemented by hand** — it can't be `derived`.

Why keep them separate? Developers need to see all the fields and type information (the `Debug` format), while users just need readable text. Different needs, so one `trait` can't serve both.

5.15.2.2 Implementing Display

```
use std::fmt::Display;
use std::fmt::Formatter;
use std::fmt::Result;

impl Display for Point {
    fn fmt(&self, f: &mut Formatter) -> Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}
```

The `fmt` method receives a `&mut Formatter`, and you write your desired format into it with the `write!` macro. `write!` works almost exactly like `println!`, except its first argument is the `&mut Formatter`.

5.15.2.3 The Relationship between Display and ToString

Rust has a `ToString` trait with just one method:

```
fn to_string(&self) -> String;
```

Here's the point — **you never implement `ToString` yourself**. The standard library contains this code:

```
impl<T: Display> ToString for T {
    fn to_string(&self) -> String {
        // Internally uses Display's fmt method to produce the string
        // ...
    }
}
```

It means: "For **every** type `T` that implements `Display`, automatically implement `ToString`." This is called a **blanket implementation** — like a blanket, covering every qualifying type.

So implement `Display`, and your type automatically gains the `.to_string()` method — nothing extra to do.

5.15.3 Example Code

```
use std::fmt::Display;
use std::fmt::Formatter;

#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

// Implementing Display by hand
impl Display for Point {
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}
```

```rust
#[derive(Debug)]
struct Color {
    r: u8,
    g: u8,
    b: u8,
}

impl Display for Color {
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "R{}G{}B{}", self.r, self.g, self.b)
    }
}

fn main() {
    let p = Point { x: 3, y: 7 };

    // The Debug format (for developers)
    println!("Debug: {:?}", p);

    // The Display format (for users)
    println!("Display: {}", p);

    // Display grants .to_string() automatically
    let s = p.to_string();
    println!("to_string: {}", s);

    let c = Color { r: 255, g: 128, b: 0 };
    println!("Debug: {:?}", c);
    println!("Display: {}", c);
    println!("to_string: {}", c.to_string());
}
```

5.15.4 Recap

- The `Display` `trait` lets your type print with the `{}` format.
- `Debug` (`{:?}`) is for developers and can be `derived`; `Display` (`{}`) is for users and must be hand-implemented.
- How: `impl Display for MyType`, writing the format with `write!` inside `fmt`.
- Implementing `Display` grants `.to_string()` automatically (a blanket implementation).

5.16 Multiple `trait` Bounds and `where`

5.16.1 Goal of This Episode

Learn to combine multiple `trait` bounds with `+`, and make complex bounds more readable with `where` clauses.

5.16.2 Concept

Episode 13 gave us `T: Clone`, requiring `T` to implement `Clone`. But what if you want `T` to implement several `traits` at once?

5.16.2.1 Multiple `trait` Bounds

Chain them with +:

```
fn show_clone<T: Clone + std::fmt::Display>(x: &T) {
    let cloned = x.clone();
    println!("Original: {}", x);
    println!("Clone: {}", cloned);
}
```

`T: Clone + Display` means `T` must implement both `Clone` and `Display`.

5.16.2.2 `where` Clauses

When `trait` bounds get long, cramming them into the <> gets crowded. Rust offers the `where` clause, placed after the function signature:

```
fn show_clone<T>(x: &T)
where
    T: Clone + std::fmt::Display,
{
    let cloned = x.clone();
    println!("Original: {}", x);
    println!("Clone: {}", cloned);
}
```

The two forms are completely equivalent; `where` just reads better.

5.16.2.3 `where` Is More Flexible Than Angle Brackets

What sits before the colon in a `where` clause isn't limited to `T` — it can be something more complex, such as a tuple type:

```
fn clone_pair<T, U>(pair: &(T, U)) -> (T, U)
where
    (T, U): Clone,
{
    pair.clone()
}
```

`(T, U): Clone` requires the tuple `(T, U)` to be `clone`-able. This form can only appear in a `where` clause, not inside <> — that's where `where`'s extra flexibility lies.

5.16.3 Example Code

```
use std::fmt::Display;

// Multiple trait bounds: Clone + Display
// Print the original, then return a clone of it
fn clone_and_show<T: Clone + Display>(x: &T) -> T {
    println!("About to clone: {}", x);
    x.clone()
}

// With a where clause: sometimes more readable
fn show_pair<T, U>(a: &T, b: &U)
where
```

```
    T: Display,
    U: Display,
{
    println!("a = {}, b = {}", a, b);
}

fn main() {
    // Multiple trait bounds
    let cloned = clone_and_show(&42);
    println!("The clone received: {}", cloned);

    let cloned2 = clone_and_show(&String::from("hello"));
    println!("The clone received: {}", cloned2);

    // A where clause
    show_pair(&10, &"world");
}
```

5.16.4 Where Else Can `where` Go

`where` isn't just for functions. It works in many other generic spots, such as `impl` blocks:

```
impl<T> Pair<T>
where
    T: Clone + Display,
{
    // Method definitions
}
```

Furthermore, `where` can appear on `struct`, `enum`, and `trait` definitions too. Just knowing this is enough for now — it'll come back to you when needed.

5.16.5 Recap

- Combine multiple `trait` bounds with `+`: `T: Clone + Display`.
- The `where` clause is another way to write `trait` bounds — more readable.
- `where` is more flexible than angle brackets: complex types like tuples can precede the colon (e.g. `(T, U): Clone`).
- `where` works beyond functions — on `impl`, `struct`, `enum`, `trait`, anywhere generics go.

5.17 The `impl Trait` Syntax

5.17.1 Goal of This Episode

Learn `impl Trait` as shorthand for `trait` bounds, and its different meanings in parameter versus return position.

5.17.2 Concept

We've learned `trait` bounds: `fn foo<T: Display>(x: &T)`. Rust also offers a more concise form: `impl Trait`.

5.17.2.1 `impl Trait` in Parameter Position

```
fn show(x: &impl Display) {
    println!("{}", x);
}
```

This is almost fully equivalent to `fn show<T: Display>(x: &T)` — both say "x's type must implement `Display`," and it's terser.

5.17.2.2 Each `impl Trait` Is an Independent Type

Important idea: each `impl Trait` in the parameters stands for an **independent** type.

```
fn show_two(a: &impl Display, b: &impl Display) {
    println!("{} {}", a, b);
}
```

`a` and `b` may be **different types** — as long as both implement `Display`. Say, `a` an `i32` and `b` a `String`.

If you require `a` and `b` to be **the same type**, use a named type parameter:

```
fn show_same<T: Display>(a: &T, b: &T) {
    println!("{} {}", a, b);
}
```

5.17.2.3 `impl Trait` in Return Position

`impl Trait` also works on return values:

```
fn greeting() -> impl Display {
    String::from("Hello")
}
```

This says "I'll return some type implementing `Display`, without telling you which one." The caller knows only that the return value supports `Display`'s methods (like `println!("{}", greeting())`), not whether it's a `String` or something else.

5.17.3 Example Code

```
use std::fmt::Display;

// impl Trait in parameter position
fn show(x: &impl Display) {
    println!("Showing: {}", x);
}

// Each impl Trait is an independent type; a and b may differ
fn show_pair(a: &impl Display, b: &impl Display) {
    println!("{} and {}", a, b);
}

// Requiring the same type: use generics
fn show_same<T: Display>(a: &T, b: &T) {
    println!("{} and {}", a, b);
}
```

```rust
// impl Trait in return position
fn make_greeting(name: &str) -> impl Display {
    let mut s = String::from("Hello, ");
    s.push_str(name);
    s.push_str("!");
    s
}

fn main() {
    // Parameter position
    show(&42);
    show(&String::from("hello"));

    // The two parameters may have different types
    show_pair(&42, &"hello");

    // Requiring the same type
    show_same(&10, &20);
    // show_same(&10, &"hello"); // Compile error! i32 and &str differ

    // Returning impl Trait
    let greeting = make_greeting("world");
    println!("{}", greeting);

    // greeting's type is `impl Display`, not `String`
    // So you can't use it as a String:
    // greeting.push_str("!!!"); // Compile error! impl Display has no push_str method
    // We know it is a String, but the compiler sees only Display
}
```

5.17.4 Recap

- `fn foo(x: &impl Display)` is shorthand for `fn foo<T: Display>(x: &T)`.
- Each `impl Trait` parameter is its own type — two `impl Displays` may differ.
- To force the same type, use a named type parameter `<T: Display>`.
- `-> impl Trait` in return position hides the concrete type; callers know only which `trait` it implements.

5.18 Multi-parameter `traits`

5.18.1 Goal of This Episode

Learn to define `traits` with extra type parameters, so one type can implement the same `trait` for different target types.

5.18.2 Concept

Our `traits` so far have been fairly simple — `Describe`, `Clone`, `Display`, no extra type parameters. But sometimes the behavior you want to define relates to **another type**.

Take "conversion": an `i32` can turn into an `f64`, or into a `String`. Same type, different targets, different logic.

5.18.2.1 traits with Extra Type Parameters

```
trait Convert<T> {
    fn convert(self) -> T;
}
```

Convert<T> means: "can be converted into type T." One type can implement Convert<f64>, Convert<String>, and other versions.

5.18.2.2 Implementing a Multi-parameter trait

```
impl Convert<(i32,)> for i32 {
    fn convert(self) -> (i32,) {
        (self,)
    }
}
```

Here i32 implements Convert<(i32,)> — turning itself into a single-element tuple.

The same type can implement it multiple times, as long as the type parameters differ:

```
impl Convert<String> for i32 {
    fn convert(self) -> String {
        // Using the ToString trait (i32 already has it)
        self.to_string()
    }
}
```

5.18.2.3 The Difference from traits without Extra Parameters

- Clone (no extra parameters): a type can implement Clone only once.
- Convert<T> (with a parameter): a type can implement several versions — Convert<String>, Convert<(i32,)>, and so on.

5.18.3 Example Code

```
// Defining a trait with a type parameter
trait Convert<T> {
    fn convert(self) -> T;
}

// i32 into a single-element tuple
impl Convert<(i32,)> for i32 {
    fn convert(self) -> (i32,) {
        (self,)
    }
}

// i32 into String
impl Convert<String> for i32 {
    fn convert(self) -> String {
        self.to_string()
    }
}
```

```rust
// bool into i32
impl Convert<i32> for bool {
    fn convert(self) -> i32 {
        if self {
            1
        } else {
            0
        }
    }
}

fn main() {
    // i32 -> (i32,)
    let x: i32 = 42;
    let tuple: (i32,) = x.convert();
    println!("{:?}", tuple);

    // i32 -> String
    let y: i32 = 100;
    let s: String = y.convert();
    println!("{}", s);

    // bool -> i32
    let b = true;
    let n: i32 = b.convert();
    println!("{}", n);
}
```

5.18.4 Recap

- A `trait` can take extra type parameters: `trait Convert<T> { ... }`.
- One type can implement the same `trait` for different `T`s (e.g. `Convert<String>` and `Convert<(i32,)>`).
- Unlike `trait`s without extra parameters, which a type can implement only once.
- Multi-parameter `trait`s give a unified home to "behavior involving another type."

5.19 `From<T>` / `Into<T>`

5.19.1 Goal of This Episode

Learn type conversion with the standard library's `From` and `Into` `trait`s, and understand the "implement `From`, get `Into` free" mechanism.

5.19.2 Concept

Last episode we defined our own `Convert<T>` `trait`. As it happens, Rust's standard library already has a more complete conversion mechanism: `From` and `Into`.

5.19.2.1 From

The definition of `From<T>` (simplified):

```
trait From<T> {
    fn from(value: T) -> Self;
}
```

It means: "I can be converted from a T."

You've certainly seen this:

```
    let s = String::from("hello");
```

That's String implementing From<&str> — converting from &str to String.

5.19.2.2 Into

Into<T> is From in the opposite direction:

```
trait Into<T> {
    fn into(self) -> T;
}
```

The key point: **implement From, and you get Into automatically.** No need to implement Into yourself.

Another blanket implementation — Rust has a rule saying "if Y: From<X>, then X automatically implements Into<Y>."

5.19.2.3 TryFrom / TryInto

Some conversions can fail — say, turning a huge i64 into an i32 might overflow. For those, use TryFrom and TryInto, which return a Result instead of a bare value.

As with From / Into, implementing TryFrom grants TryInto automatically.

5.19.3 Example Code

```
use std::fmt::Display;
use std::fmt::Formatter;

struct Celsius {
    value: f64,
}

struct Fahrenheit {
    value: f64,
}

impl Display for Celsius {
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "{}°C", self.value)
    }
}

impl Display for Fahrenheit {
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "{}°F", self.value)
    }
```

```rust
}

// Implementing From: converting Celsius into Fahrenheit
impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Fahrenheit {
        Fahrenheit {
            value: c.value * 1.8 + 32.0,
        }
    }
}

fn main() {
    // String::from — what we've used all along
    let s = String::from("hello");
    println!("{}", s);

    // Our custom From
    let boiling = Celsius { value: 100.0 };
    println!("Celsius: {}", boiling);
    let f = Fahrenheit::from(Celsius { value: 100.0 });
    println!("Fahrenheit: {}", f);

    // Into comes free (no separate implementation needed)
    let body_temp = Celsius { value: 37.0 };
    let f2: Fahrenheit = body_temp.into();
    println!("Body temperature: {}", f2);

    // A TryFrom example: i32 to u8 can fail
    let big: i32 = 300;
    let result = u8::try_from(big);
    match result {
        Ok(n) => println!("Conversion succeeded: {}", n),
        Err(e) => println!("Conversion failed: {:?}", e),
    }

    let small: i32 = 42;
    let ok = u8::try_from(small);
    match ok {
        Ok(n) => println!("Conversion succeeded: {}", n),
        Err(e) => println!("Conversion failed: {:?}", e),
    }
}
```

5.19.4 Recap

- `From<T>` defines "converted from a T": `String::from("hello")` is exactly this.
- Implement `From` and `Into` comes automatically — no separate implementation.
- `.into()` is `.from()`'s reverse direction: `let f: Fahrenheit = celsius.into();`.
- `TryFrom` / `TryInto` handle fallible conversions, returning a `Result`.
- Implementing `TryFrom` grants `TryInto` automatically too.

5.20 Drop

5.20.1 Goal of This Episode

Learn to discard a value early with `drop(value)`, understand how Rust automatically `drops` the contents a value owns, and use the `Drop` `trait` to perform an extra action before the contained values are `dropped`.

5.20.2 Concept

5.20.2.1 Discarding a Value Early with `drop(value)`

Rust normally `drops` a value automatically when it leaves scope. If you do not want to wait until the scope ends, call `drop(value)` to `drop` it early:

```
fn main() {
    let message = String::from("hello");
    println!("{}", message);

    drop(message);
    println!("message has been dropped");

    // println!("{}", message); // Compile error! message's value was moved
}
```

`drop` is an ordinary function provided by the prelude, so it needs no additional `use`. It takes ownership of the value passed to it, which means you cannot use the original value after `drop(message)`.

To emphasize the distinction, it is the **value** bound to `message` that is `dropped`, not the variable name itself. The variable's original scope has not become shorter; its value has been moved into `drop` and then discarded.

5.20.2.2 Contained Values Are `dropped` Automatically

When a value is `dropped`, Rust continues by `dropping` the fields or elements that the value owns:

```
struct Message {
    title: String,
    body: String,
}

fn main() {
    let message = Message {
        title: String::from("Greeting"),
        body: String::from("Hello!"),
    };

    drop(message);
}
```

We only need to `drop` `message`; Rust automatically `drops` its `title` and `body` too. The same mechanism applies to values owned by tuples, `enums`, arrays, `Vecs`, and other types. You do not need to clean them up one by one yourself.

5.20.2.3 Using `Drop` to Act Before `dropping`

Sometimes we want to do something before Rust automatically `drops` the contained values, such as closing a connection, returning a resource, or printing a log message. We can implement the `Drop trait` for the type:

```
impl Drop for Resource {
    fn drop(&mut self) {
        println!("Releasing resource: {}", self.name);
    }
}
```

When a `Resource` is `dropped`, Rust runs `Drop`'s `.drop()` method first and then automatically `drops` its fields. This method adds an action to the `drop` process; it does not replace Rust's automatic handling of the contained values.

Although `Drop`'s `.drop()` method and the `drop(value)` function are both named `drop`, they are used differently:

- `drop(value)` is an ordinary function that you can call to `drop` a value early.
- `Drop`'s `.drop()` method is run automatically by Rust when a value is `dropped`. You cannot call it manually as `value.drop()`.

Why can't you call `value.drop()` manually? Because this method only receives `&mut self`; it does not take ownership of the value. If a manual call were allowed, the value would still exist afterward. When the value was actually `dropped` later, Rust would run the same method again, potentially releasing the same resource twice. Rust therefore rejects this syntax.

`drop(value)` is different: it takes ownership of the value, so the original value cannot be used afterward. This lets Rust complete the entire `drop` process early without leaving behind a value that must be `dropped` again later.

5.20.3 Example Code

```
struct Resource {
    name: String,
}

impl Drop for Resource {
    fn drop(&mut self) {
        println!("Releasing resource: {}", self.name);
    }
}

struct Worker {
    name: String,
    resource: Resource,
}

impl Drop for Worker {
    fn drop(&mut self) {
        println!(
            "Stopping worker: {} (releasing {} next)",
            self.name,
```

```
            self.resource.name,
        );
    }
}

fn main() {
    let worker = Worker {
        name: String::from("downloader"),
        resource: Resource {
            name: String::from("network connection"),
        },
    };

    println!("Worker is running");
    drop(worker);
    println!("Worker was dropped early");

    {
        let temporary = Resource {
            name: String::from("temporary file"),
        };
        println!("Using temporary resource: {}", temporary.name);
    } // temporary is dropped automatically here
}
```

5.20.4 Types That Implement `Drop` Cannot Be Partially Moved

If a type implements `Drop`, you cannot move a value out of one of its fields:

```
struct Resource {
    name: String,
    id: i32,
}

impl Drop for Resource {
    fn drop(&mut self) {
        println!("Releasing {} (ID {})", self.name, self.id);
    }
}

fn main() {
    let resource = Resource {
        name: String::from("database connection"),
        id: 1,
    };

    let name = resource.name; // Compile error! A partial move is not allowed
}
```

`Drop`'s `.drop()` method receives a complete `&mut self`, so it might access any field. If `name` could be moved out first, `resource` would no longer be complete when this method later ran. Rust therefore rejects this operation.

If a field is itself a `struct`, moving a value out of one of its deeper fields would also leave the outer value incomplete, so that is rejected as well.

The restriction applies to **moving** a value out of a field. You can still:

- Move the entire `resource`.
- Borrow a field, such as `&resource.name`.
- Copy a field that implements `Copy`, such as `resource.id`.

5.20.5 Recap

- Rust automatically `drop`s a value when it leaves scope.
- `drop(value)` takes ownership of a value and lets you `drop` it before the scope ends.
- When an outer value is `drop`ped, the contained values it owns are `drop`ped automatically too.
- `Drop`'s `.drop()` method lets you perform an extra action before contained values are `drop`ped. Rust runs it automatically, so you cannot call it directly.
- A type that implements `Drop` cannot be partially moved, but you can still move the whole value, borrow its fields, or copy fields that implement `Copy`.

5.21 `Box<T>`

5.21.1 Goal of This Episode

Learn to put data on the heap with `Box<T>`, and understand why it's necessary for recursive types.

5.21.2 Concept

Remember Chapter 4's safe analogy? A key hangs on the keychain, the key opens a safe, and the safe holds the real goods.

`Box<T>` is the key to that safe — the data lives on the heap, and the `Box` value on the stack lets Rust reach it.

5.21.2.1 Why Do We Need `Box`?

Most of the time, Rust putting data straight on the stack is fine. But two situations call for `Box`:

1. The data is too big

If a `struct` has many fields and takes lots of space, the stack may not be a great place for it (stack space is limited). `Box` moves it to the heap, leaving only the "key" on the stack. This kind of information used to find data stored elsewhere is called a pointer.

2. Recursive types

The more important reason. Suppose you want to define a linked list:

```
enum List {
    Node(i32, List), // Compile error!
    Empty,
}
```

Rust needs to know every type's size at compile time. But here's the problem: to know `List`'s size, you need to know how big `Node` is. `Node` holds an `i32` and a `List` — so you need `List`'s

size. But `List` contains another `List`...

Expanding it: `List`'s size = `i32` + `List`'s size = `i32` + `i32` + `List`'s size = ... it never terminates. The compiler flat-out errors: "recursive type has infinite size."

The fix is `Box`:

```
enum List {
    Node(i32, Box<List>),
    Empty,
}
```

`Box<List>` has a fixed size (a pointer's size), and the problem is solved.

5.21.2.2 Using a `Box`

```
fn main() {
    let x = Box::new(42);
    println!("{}", x); // Usable directly; Rust fetches the inner value automatically
}
```

`Box::new(value)` moves the value onto the heap. The `Box` owns its contents and releases them automatically at scope exit (since `Box` implements `Drop`).

5.21.3 Example Code

```
// A recursive type via Box: a linked list
enum List {
    Node(i32, Box<List>),
    Empty,
}

// Printing the list
fn print_list(list: &List) {
    match list {
        List::Node(value, next) => {
            print!("{} -> ", value);
            print_list(next);
        }
        List::Empty => {
            println!("end");
        }
    }
}

fn main() {
    // Basic Box usage
    let x = Box::new(42);
    println!("The value in the Box: {}", x);

    // Building a linked list step by step: 3 -> 2 -> 1 -> end
    // Starting from the tail
    let list = List::Empty;                    // end
    let list = List::Node(1, Box::new(list)); // 1 -> end
    let list = List::Node(2, Box::new(list)); // 2 -> 1 -> end
```

```
    let list = List::Node(3, Box::new(list)); // 3 -> 2 -> 1 -> end

    print_list(&list);

    // A Box is a single key — the key isn't Copy, so let b = a is a move
    let a = Box::new(String::from("hello"));
    let b = a; // The key passes from a to b, leaving a empty
    // println!("{}", a); // Compile error! a has been moved
    println!("{}", b);
}
```

5.21.4 Recap

- `Box<T>` puts data on the heap, leaving only a pointer on the stack (the "key" from the safe analogy).
- Its most important use: **recursive types** (like linked lists) need `Box` to break the infinite-size problem.
- `Box::new(value)` creates the `Box`; it's released automatically at scope exit.
- A `Box` is a single key; moving it follows the same rules as moving other non-`Copy` values.

5.22 `Rc<T>`

5.22.1 Goal of This Episode

Learn to let several `Rc<T>` values share the same heap data, and understand how reference counting keeps that data alive.

5.22.2 Concept

Last episode was `Box<T>`: one safe, one key. The `Box` value is the key, and the heap data is the thing inside the safe.

But sometimes several parts of your program need to use the **same** heap data without creating another independent heap value.

5.22.2.1 The Problem: Moving a `Box`

With `Box<T>`, there is only one key. Assigning it to another variable moves that key:

```
    let a = Box::new(String::from("hello"));
    let b = a; // Move! a can't be used anymore
```

After this move, `b` has the key. The heap data was not cloned, and `a` can no longer use it.

What if you want both `a` and `b` to keep using the same text?

You might think: "Just call `.clone()`, right?"

```
    let a = Box::new(String::from("hello"));
    let b = a.clone(); // Creates a new Box with another String
```

That does let both `a` and `b` work. But in this `Box<String>` example, `.clone()` creates another `Box` holding a new `String` with the same contents. If the inner value is large, that can be a real

expense. And a and b now use **two separate String values**, not the same one.

If what you need is "several values using the **same** heap data," Box's .clone() is the wrong tool.

5.22.2.2 Rc's .clone() Really Does Just Cut an Extra Key

Remember Chapter 4's definition of .clone()? — **"get a new keychain that works just like the original, while making sure that doing so causes no trouble."** For simple owned types like String or Vec<i32>, the way to "make sure" is to create another key to another safe with the same contents.

Rc is the exception Chapter 4 foreshadowed: its .clone() really does just cut an extra key. You get a new Rc value that opens the same safe. The data itself is not replicated, which is why calling .clone() on an Rc stays cheap when the data is large.

But if several keys can open the same safe, Rust needs a way to know when the safe can be released. That's where the counter comes in.

5.22.2.3 Rc: Reference Counting

Rc<T> stands for **reference counting**.

Think of Rc<T> as a safe that allows several keys. Rc keeps a counter for that safe.

That counter tracks how many Rc<T> values can still open the safe:

- Creating the first Rc: count = 1.
- .clone(): count +1 — you get another key to the same safe, not another independent set of the data.
- An Rc value leaving scope: count -1.
- Only when the count reaches zero is the heap data released.

What the counter handles is when the safe can be released: as long as someone still holds a key, the safe stays, so nobody is left holding a key to a safe that is already gone.

This also means Rc's Clone cannot be derived. Rc's .clone() has two jobs: cut another key, and bump the counter on the safe. derive only calls .clone() on every field — it can do the first job, not the second, so you would end up with an extra key while the count stays put. That's why the standard library hand-writes Clone for Rc.

5.22.2.4 Two Layers to Keep Separate

This distinction matters:

```
    let a = Rc::new(String::from("hello"));
    let b = a.clone();
```

There are two layers here:

- a owns one Rc<String> value.
- b owns another Rc<String> value.
- Both Rc<String> values open the same heap data.
- The counter tracks how many Rc<String> values still exist.

So Rc does not mean ordinary Rust values stop following the ownership rules. Each Rc value

is still an ordinary value: it can move, it can be `dropped`, and after moving it the old variable cannot be used. The special part is the heap data behind those `Rc` values: it stays alive until the last key is gone.

5.22.2.5 `Rc` Is Read-only

`Rc<T>` by itself provides shared read access. Several `Rc` values can read the same heap data, but they cannot freely modify it.

If you need shared data that can also be modified, `RefCell<T>` comes later.

5.22.2.6 Wait — Didn't Chapter 4 Warn Against Duplicating Keys?

Chapter 4's worry was very concrete: two people each hold a key to the same safe, A is tidying up what's inside, and B walks off with the contents. So what earns `Rc` the right to cut several keys?

Because `Rc` blocks both of the dangers in that scene:

1. **Nobody can touch what's inside**: `Rc<T>` only grants shared read access. Chapter 4's fear was one person tidying while another takes things away; if everyone can only look, that cannot happen.
2. **The safe is not released while someone still holds a key**: that's the counter's job. The data is freed only once the last key is gone.

The second point is not free: the counter must update on every `clone` and `drop` — overhead `Box<T>` doesn't have. `Box<T>` is the simple one-key heap setup with no counter; `Rc<T>` adds runtime bookkeeping so several `Rc` values can open the same safe. That sharing is useful, but it is not free.

And when we get to multithreading, we'll see that `Rc` has other restrictions there. Just know this for now.

5.22.3 Example Code

```
use std::rc::Rc;

fn main() {
    // Create an Rc; count = 1
    let a = Rc::new(String::from("shared data"));
    println!("Created a, count = {}", Rc::strong_count(&a));

    // .clone() creates another Rc for the same heap data
    let b = a.clone();
    println!("Cloned to b, count = {}", Rc::strong_count(&a));

    let c = a.clone();
    println!("Cloned to c, count = {}", Rc::strong_count(&a));

    // a, b, and c can all read the same data
    println!("a = {}", a);
    println!("b = {}", b);
    println!("c = {}", c);
```

```
    {
        let _d = a.clone();
        println!("Inside the scope, count = {}", Rc::strong_count(&a));
    } // _d is dropped; count -1
    println!("After leaving the scope, count = {}", Rc::strong_count(&a));

    // A practical use: several values sharing the same name
    let shared_name = Rc::new(String::from("Rust"));

    let greeting1 = shared_name.clone();
    let greeting2 = shared_name.clone();

    println!("1: {}", greeting1);
    println!("2: {}", greeting2);
}
```

5.22.4 Recap

- `Rc<T>` lets several `Rc` values share the same heap data.
- `Rc::new(value)` starts the count at 1.
- `.clone()` creates another `Rc` value for the same heap data: count +1, no new independent inner value.
- Dropping an `Rc` decrements the count; the heap data is released when the count reaches zero.
- `Rc<T>` by itself provides shared read access, not unrestricted mutation.
- `Rc` shares safely thanks to two things: read-only access, plus a counter that keeps the data alive until the last `Rc` is gone.
- `Rc<T>` has restrictions; it is not the general answer for every sharing problem.
- Each `Rc` value still follows ordinary ownership rules: it moves and `drops` like any other non-`Copy` value.
- Check the current reference count with `Rc::strong_count(&x)`.

5.23 `Deref`

5.23.1 Goal of This Episode

Understand the `Deref` `trait`, the `DerefMut` `trait`, Rust's `deref` coercion, and why smart pointers can often be used like the values inside them.

5.23.2 Concept

5.23.2.1 Using `*` on an `Rc`

So far we've used `*` mostly on ordinary references (`&T`). But `*` also works on some smart pointers:

```
use std::rc::Rc;

fn main() {
    let value = Rc::new(42);
    let number: i32 = *value;
```

```
    println!("{}", number); // 42
}
```

`Rc<i32>` is not an `i32`, but Rust can use the key to reach the `i32` inside. Here, `i32` is `Copy`, so assigning `*value` into `number` creates another `i32` value.

If the inner value isn't `Copy` — say a `String` — you can't move it out this way, just as you'd expect from how borrowing worked in Chapter 4:

```
use std::rc::Rc;

fn main() {
    let text = Rc::new(String::from("hello"));
    let moved: String = *text; // Compile error!
}
```

There may be other `Rc` values that open the same heap data. Moving the inner `String` out would leave those `Rc` values with a key to an empty safe, so Rust forbids it.

5.23.2.2 The `Deref` `trait`

The mechanism behind this is the `Deref` `trait`. We do not need its exact definition yet; the important idea is simpler:

`Deref` tells Rust how to borrow through a value. For example, `Rc<i32>` can borrow through to the `i32` inside, producing an `&i32`.

That reference is the important part. `Deref` gives Rust a **reference** to the inner value. It does not, by itself, give ownership of the inner value.

`Rc<T>` and `Box<T>` both implement `Deref`. Types like these — whose whole job is to be the key to some inner value, managing it and letting Rust reach it through `Deref` — are commonly called **smart pointers**. Some other standard library types (like `String` and `Vec<T>`) implement `Deref` too, even though being a key isn't their main job; in this episode we focus on smart pointers.

5.23.2.3 What Happens behind `*v`

When you use `*` on a type implementing `Deref`, the useful mental model is:

```
*v
// roughly: borrow through v, then follow that reference
```

For the earlier `Rc<i32>` example:

```
let value = Rc::new(42);

*value
// roughly:
// borrow through value to get &i32
// then follow that &i32
```

Because `i32` is `Copy`, this can produce another `i32` value. If the inner value is not `Copy`, like `String`, ordinary `Deref` does not let you move it out.

5.23.2.4 `deref` Coercion

`deref` coercion is Rust's mechanism for automatically converting reference types through `Deref` when needed.

For example, this function expects an `&i32`:

```
use std::rc::Rc;

fn show(n: &i32) {
    println!("{}", n);
}

fn main() {
    let value = Rc::new(42);

    show(&value); // &Rc<i32> automatically becomes &i32
}
```

`show` needs `&i32`, but `&value` is `&Rc<i32>`. Since `Rc<i32>` implements `Deref` in a way that lets Rust borrow the inner `i32`, Rust can convert:

```
&Rc<i32> -> &i32
```

This conversion happens at the reference level. No ownership moves.

`deref` coercion can also chain:

```
use std::rc::Rc;

fn show(n: &i32) {
    println!("{}", n);
}

fn main() {
    let value = Rc::new(Box::new(42));

    show(&value); // &Rc<Box<i32>> -> &Box<i32> -> &i32
}
```

Rust first goes through the `Rc`, then through the `Box`, until the reference type matches what the function expects.

5.23.2.5 Auto-dereferencing in Method Calls

Method calls have their own auto-dereferencing behavior. When you call a method with `.`, Rust tries the outer type first. If it cannot find a matching method there, it goes one layer inward and tries again.

For example:

```
use std::rc::Rc;

fn main() {
    let numbers = Rc::new(vec![10, 20, 30]);

    println!("{}", numbers.len()); // calls Vec<i32>'s .len()
}
```

`Rc<Vec<i32>>` itself does not define `.len()`, but `Vec<i32>` does. Rust can use the `Rc` key, borrow the inner `Vec<i32>`, and call `.len()` on that.

With multiple layers, Rust can go inward one layer at a time:

```
let numbers = Rc::new(Box::new(vec![10, 20, 30]));

numbers.len()
// Rust can go through Rc, then Box, then find Vec's .len()
```

This is why smart pointers often feel like the value inside them: method calls can automatically borrow through the smart pointer.

5.23.2.6 `DerefMut`

`DerefMut` is the mutable version of `Deref`. It tells Rust how to borrow through a value mutably: from a mutable smart pointer to a mutable reference of the inner value.

`Rc<T>` does not implement `DerefMut`, because there may be other `Rc` values that open the same heap data. Ordinary `Rc<T>` provides shared read access, not unrestricted mutable access. `Rc<T>` cannot prove that it is the only key to the heap data; if `DerefMut` were allowed, the same heap data could end up with several `&mut T` references at the same time.

`Box<T>`, however, has one key and no reference counter, so a mutable `Box<T>` can provide mutable access to the inner value:

```
fn main() {
    let mut text = Box::new(String::from("hello"));

    text.push_str(" world");
    println!("{}", text);

    *text = String::from("replaced");
    println!("{}", text);
}
```

The call to `.push_str()` mutably borrows the inner `String`. The assignment through `*text` replaces the inner `String`. Both are normal `DerefMut` behavior: Rust gets a `&mut String` through the `Box`.

5.23.2.7 Priority When Method Names Collide

Rust searches for methods from the outside in. The outer smart pointer's own methods take priority over the inner type's methods.

A common example is `.clone()`. `Rc` itself has a `.clone()` method: it creates another `Rc` for the same heap data and increments the reference count. The inner value may also have its own `.clone()` method.

Calling `.clone()` directly creates another `Rc`:

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(String::from("hello"));
    let b = a.clone(); // Rc's .clone(): bumps the count, doesn't create a new String
```

```
}
```

If you want the inner `String`'s own `.clone()`, spell that out:

```
    let c = (*a).clone(); // String's .clone(): creates a new String
```

5.23.2.8 One Special Thing about `Box<T>`

Everything above treats `Deref` as borrowing through a smart pointer. That is the right general model.

`Box<T>` has one extra ability: when you own the `Box<T>`, Rust lets you move the inner `T` out with `*box_value`:

```
fn main() {
    let boxed = Box::new(String::from("owned"));
    let text: String = *boxed; // OK: moves the String out of the Box

    println!("{}", text);
}
```

This is special support for `Box<T>`. It is **not** what ordinary `Deref` types can do:

```
use std::rc::Rc;

fn main() {
    let shared = Rc::new(String::from("shared"));
    let text: String = *shared; // Compile error!
}
```

So keep the general rule simple: `Deref` lets Rust borrow through a value. Moving a non-`Copy` value out with `*` is a special `Box<T>` ability.

5.23.3 Example Code

```
use std::rc::Rc;

fn show(n: &i32) {
    println!("value: {}", n);
}

fn main() {
    // Rc<i32>: * reaches the i32. Since i32 is Copy, this creates another i32 value.
    let shared = Rc::new(42);
    let number: i32 = *shared;
    println!("number: {}", number);

    // Deref coercion: &Rc<i32> -> &i32
    show(&shared);

    // Deref coercion can chain: &Rc<Box<i32>> -> &Box<i32> -> &i32
    let nested = Rc::new(Box::new(99));
    show(&nested);

    // Method-call auto-deref: Rc<Vec<i32>> can call Vec<i32>'s methods.
    let numbers = Rc::new(vec![10, 20, 30]);
```

```
    println!("length: {}", numbers.len());

    // DerefMut: Box<String> can mutably borrow the inner String.
    let mut text = Box::new(String::from("hello"));
    text.push_str(" world");
    println!("{}", text);

    *text = String::from("replaced");
    println!("{}", text);

    // Method-name priority: Rc's .clone() wins over String's .clone().
    let a = Rc::new(String::from("shared"));
    let b = a.clone();    // Rc .clone(): bumps the count
    let c = (*a).clone(); // String .clone(): creates a new String
    println!("a = {}, b = {}, c = {}", a, b, c);
    println!("Rc count = {}", Rc::strong_count(&a)); // 2, not 3

    // Box<T> special case: owning a Box lets you move T out.
    let boxed = Box::new(String::from("owned"));
    let owned: String = *boxed;
    println!("moved out of Box: {}", owned);
}
```

5.23.4 Recap

- `Deref` is mainly about borrowing through a value: it lets Rust get a reference to the inner value.
- `*v` on a `Deref` type is "borrow, then follow the reference"; whether you can copy, mutate, or move afterward is decided by the inner type and how the expression is used.
- `deref` coercion automatically converts references such as `&Rc<i32>` to `&i32`; it can chain through multiple layers.
- Method-call auto-dereferencing lets smart pointers call methods of the inner value.
- `DerefMut` gives mutable access to the inner value; `Box<T>` supports it, while `Rc<T>` does not.
- On method-name collisions, the outer type wins; `Rc`'s `.clone()` is chosen before the inner value's `.clone()`.
- Moving a non-`Copy` value out with `*box_value` is special support for `Box<T>`, not ordinary `Deref` behavior.

5.24 `Cell<T>`

5.24.1 Goal of This Episode

Learn to modify values through shared references with `Cell<T>`, and understand its limitations.

5.24.2 Concept

Chapter 4 taught the borrowing rules: either one `&mut` or many `&`s, never both at once. Safe — but sometimes, holding only a `&` (shared reference), you still want to modify a value.

5.24.2.1 The Idea of `Cell`

`Cell<T>` provides interior mutability — even through a shared reference, it can copy values out with `.get()` and replace them with `.set(v)`, **no mutable reference required**.

```
use std::cell::Cell;

fn main() {
    let x = Cell::new(42);
    x.set(100);              // No mut needed!
    println!("{}", x.get()); // 100
}
```

But `.get()` comes with one important restriction:

5.24.2.2 `T` Must Be `Copy` to Use `.get()`

`Cell<T>`'s `.get()` **copies** the value out (rather than borrowing). So `T` must implement `Copy` to use `.get()`.

You can't call `.get()` on a `Cell<String>`, since `String` isn't `Copy`. Only `Copy` types work with `.get()` (`i32`, `f64`, `bool`, etc.).

5.24.2.3 Why Not Just Use `mut`?

Sometimes getting a `&mut` isn't convenient. Say a `struct` is shared by reference in several places (`&self`), but you want to bump a counter inside it. `Cell` fits that scenario nicely.

5.24.2.4 `Rc` Is Built on `Cell`

The `Rc<T>` we learned about earlier needs a reference counter — +1 on every `clone`, -1 on every `drop`. But look at `Clone`'s signature: `fn clone(&self) -> Self`. It only gets `&self` (a shared reference), yet it must bump the count. How? With `Cell`! The counter inside `Rc` is a `Cell<usize>`, so the count can update even through `&self`.

5.24.3 Example Code

```
use std::cell::Cell;

struct Counter {
    count: Cell<i32>,
    name: String,
}

impl Counter {
    fn new(name: String) -> Counter {
        Counter {
            count: Cell::new(0),
            name,
        }
    }

    // Note: only &self needed, not &mut self
    fn increment(&self) {
        let current = self.count.get();
        self.count.set(current + 1);
```

```
    }

    fn get_count(&self) -> i32 {
        self.count.get()
    }
}

fn main() {
    // Basic usage
    let x = Cell::new(42);
    println!("Original value: {}", x.get());

    x.set(100);
    println!("After modifying: {}", x.get());

    // Using Cell inside a struct
    let counter = Counter::new(String::from("visit count"));

    // Only &counter (a shared reference), yet count can be modified
    counter.increment();
    counter.increment();
    counter.increment();

    println!("Count for {}: {}", counter.name, counter.get_count());
}
```

5.24.4 Recap

- `Cell<T>` lets you modify a value without needing `&mut`.
- `.get()` copies the value out; `.set(v)` writes a new one.
- **`T` must be `Copy` to use `.get()`** — because `get` copies rather than borrows.
- Great for "I only have `&self` but want to modify a field" scenarios.

5.25 `RefCell<T>`

5.25.1 Goal of This Episode

Learn to have the borrowing rules checked at runtime with `RefCell<T>`, and combine it with `Rc` for mutable shared data.

5.25.2 Concept

Last episode's `Cell<T>` uses `.get()` to read the inner value, but `.get()` requires `T` to be `Copy`. What if you want to modify a `String` or a `Vec` and borrow the inner value to read or write it?

5.25.2.1 `RefCell`: Runtime Borrow Checking

`RefCell<T>` is like `Cell` — letting you modify values without needing `&mut`. The differences:

- `Cell<T>`: uses `.get()` / `.set()`; `.get()` requires `T` to be `Copy`, with **zero cost** (compiles down to the same as direct access).

- `RefCell<T>`: uses `.borrow()` and `.borrow_mut()` to obtain values that behave like shared and mutable references, `T` **doesn't need** `Copy`, but there's a **runtime cost** (every borrow gets checked against the rules).

```
use std::cell::RefCell;

fn main() {
    let x = RefCell::new(String::from("hello"));
    x.borrow_mut().push_str(" world"); // Modify the String inside
    println!("{}", x.borrow());        // Borrow to read
}
```

5.25.2.2 Runtime Checking

Ordinary & and `&mut` have the borrowing rules checked at **compile time**. `RefCell` moves that check to **runtime**. The rules are identical (one `&mut` or many &s) — but a violation isn't a compile error, it's a **panic**.

```
    let x = RefCell::new(42);
    let a = x.borrow();     // An immutable borrow
    let b = x.borrow_mut(); // Panic! An immutable borrow already exists
```

So `RefCell` doesn't "bypass" the borrowing rules — it "defers the check."

5.25.2.3 `Rc` + `RefCell`: Mutable Shared Data

`Rc<T>` gives you shared reading: several `Rc` values can read the same data, but none of them can change it. `RefCell<T>` gives you interior mutability: you can modify through a shared reference, but getting the data into several places still takes an `Rc`. Put them together:

```
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    let shared = Rc::new(RefCell::new(vec![1, 2, 3]));
}
```

Now several `Rc` values share the same data, and `borrow_mut()` allows modifying it.

5.25.3 Example Code

```
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    // Basic RefCell usage
    let data = RefCell::new(String::from("hello"));

    // An immutable borrow
    {
        let borrowed = data.borrow();
        println!("Reading: {}", borrowed);
    } // borrowed leaves scope, releasing the borrow

    // A mutable borrow
```

```
    {
        let mut borrowed_mut = data.borrow_mut();
        borrowed_mut.push_str(" world");
    } // borrowed_mut leaves scope, releasing the borrow

    println!("After modifying: {}", data.borrow());

    // Violating the borrowing rules → panic!
    // Uncommenting the block below panics at runtime
    // {
    //     let r1 = data.borrow();      // An immutable borrow
    //     let r2 = data.borrow_mut();  // A simultaneous mutable borrow → panic!
    // }

    // Rc + RefCell: mutable shared data
    let shared = Rc::new(RefCell::new(vec![1, 2, 3]));

    let a = shared.clone();
    let b = shared.clone();

    // Modify through a
    a.borrow_mut().push(4);

    // The change is visible through b
    println!("Reading through b: {:?}", b.borrow());

    // Modify through b
    b.borrow_mut().push(5);

    // Visible through a too
    println!("Reading through a: {:?}", a.borrow());
}
```

5.25.4 Recap

- `RefCell<T>` is like `Cell` — modifying values without needing `&mut`.
- `RefCell<T>` moves the borrowing-rule check from compile time to runtime.
- `.borrow()` obtains a value that behaves like a shared reference; `.borrow_mut()` obtains one that behaves like a mutable reference.
- `.borrow()` and `.borrow_mut()` don't require `T` to be `Copy`, unlike `Cell<T>`'s `.get()`.
- `Cell` is zero-cost; `RefCell` pays a runtime check on every borrow.
- Violating the borrowing rules **panics** (not a compile error).
- The `Rc<RefCell<T>>` combo: mutable shared data.

5.26 Lifetime Basics

5.26.1 Goal of This Episode

Understand why lifetime annotations `'a` are needed, and learn to annotate lifetimes when a function returns a reference.

5.26.2 Concept

When Chapter 4 covered borrowing, we planted a seed: "you can't return a reference to a local variable." Time to face this properly.

5.26.2.1 Problem 1: Returning a Reference to a Local Variable

```
fn make_greeting() -> &str {
    let s = String::from("Hi there");
    &s // Compile error!
} // s is released here; the returned reference would point to memory that no longer exists
```

This one's easy to grasp — `s` is gone once the function ends, so returning a reference to it is meaningless. Rust blocks it outright.

5.26.2.2 Problem 2: Several References — Which One Gets Returned?

But this case is subtler:

```
fn longer(a: &str, b: &str) -> &str {
    if a.len() > b.len() {
        a
    } else {
        b
    }
}
```

This fails to compile too. `a` and `b` are references passed in from outside — they don't vanish when the function ends. So why not?

Because when Rust checks the **call site**, it needs to know how long the returned reference can "live." Consider:

```
    let s1 = String::from("hello world");
    let result;
    {
        let s2 = String::from("hi");
        result = longer(&s1, &s2);
    } // s2 is released here
    println!("{}", result); // Can result still be used or not?
```

If `longer` returned `a` (i.e. `&s1`), `result` is safe — `s1` is still alive. But if it returned `b` (i.e. `&s2`), `result` is a dangling reference — `s2` has been released.

The catch: **when checking `longer`'s call site, the compiler does not look at `longer`'s body**. It reads only the signature. And the signature says `-> &str` — nothing tells it whose lifespan the return value is tied to.

5.26.2.3 The Lifetime Annotation `'a`

The fix is a lifetime annotation, explicitly describing the relationship between the return value and the parameters:

```
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() {
        a
```

```
    } else {
        b
    }
}
```

`'a` is a **lifetime parameter** (like a type parameter `T`, but starting with `'`). The signature tells Rust: "`a`, `b`, and the return value are all annotated with the same `'a`. So the return value's lifespan can't exceed the **shorter** of `a` and `b`." Note that lifetime parameters go inside `<>` just like type parameters. When both are present, **lifetimes come first**: `fn foo<'a, T>(x: &'a T) -> &'a T`.

5.26.2.4 Why the Shorter One?

Because `a` and `b` share one `'a`, Rust takes their **intersection** — the stretch of time during which both are still alive.

Back to the example:

```
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() {
        a
    } else {
        b
    }
}

fn main() {
    let s1 = String::from("hello world"); // s1 lives longer
    let result;
    {
        let s2 = String::from("hi");      // s2 lives shorter
        result = longer(&s1, &s2);
        println!("{}", result);           // ✅ Both s1 and s2 are alive here
    } // s2 is released here
    // println!("{}", result);            // ❌ No! 'a also ends when s2 dies
}
```

`'a` gets inferred as `s2`'s lifespan (the shorter one), so `result` is usable only while `s2` is still alive.

5.26.2.5 `&'a mut T`

Mutable references can carry lifetime annotations too, written `&'a mut T` — the `'a` slots between `&` and `mut`. `'a` likewise describes how long the reference may live.

```
fn replace<'a>(target: &'a mut String, new_value: &str) {
    target.clear();
    target.push_str(new_value);
}
```

5.26.2.6 Lifetimes Don't Change Lifespans

Key idea: lifetime annotations never make any reference live longer or shorter. They only **describe** relationships that already exist, helping the compiler check. Just as a type annotation doesn't change a value's contents.

5.26.2.7 Not Every Function Needs Annotations

If a function has just one reference parameter, Rust can usually infer on its own (next episode covers the details):

```
fn first_byte(s: &str) -> &str {
    &s[..1] // The return value obviously lives as long as s; no manual annotation needed
}
```

5.26.2.8 The `'static` Lifetime

One special lifetime exists: `'static`, meaning "lives until the program ends."

String literals are `'static` — the type of `"hello"` is `&'static str`, because string literals are baked into the code and exist for the program's entire run.

5.26.3 Example Code

```
// Returning a reference requires lifetime annotations
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() {
        a
    } else {
        b
    }
}

// The return value relates only to a; b has no bearing
fn always_first<'a>(a: &'a str, _b: &str) -> &'a str {
    a
}

fn main() {
    // Example 1: both parameters live equally long
    let s1 = String::from("a rather long string");
    let s2 = String::from("short");
    let result = longer(&s1, &s2);
    println!("The longer one is: {}", result);

    // Example 2: the parameters have different lifespans
    let s3 = String::from("hello world");
    let r;
    {
        let s4 = String::from("hi");
        r = longer(&s3, &s4);
        println!("Inside the scope: {}", r); // ✅ Both s3 and s4 are alive
    }
    // println!("{}", r); // ✕ Compile error! s4 was released; r's lifetime is too short

    // Example 3: the return value borrows only one parameter
    let s5 = String::from("I get returned");
    let r2;
    {
        let s6 = String::from("I don't");
        r2 = always_first(&s5, &s6);
    }
```

```
    // r2 borrows only s5, so s6 being released doesn't matter
    println!("{}", r2); // ✅ s5 is alive; r2 is usable

    // The 'static lifetime
    let s: &'static str = "I'm a static string, alive until the program ends";
    println!("{}", s);
}
```

5.26.4 Recap

- When a function returns a reference, Rust needs to know how long it can live — that's what lifetime annotations are for.
- `'a` is a lifetime parameter describing lifespan relationships between references.
- When several parameters share one `'a`, Rust takes the intersection (the shorter one).
- Lifetime annotations **don't change lifespans** — they only describe existing relationships.
- `'static` means "lives until the program ends" — string literals have type `&'static str`.

5.27 Lifetime Elision Rules

5.27.1 Goal of This Episode

Understand Rust's lifetime elision rules, and why most of the time you needn't write lifetime annotations by hand.

5.27.2 Concept

Last episode, we only wrote `'a` by hand for cases like `longer`, where the return value might come from different reference parameters. What about other functions that return references? Do they all need annotations too?

Good news: mostly not. Rust has a set of **elision rules** that fill in lifetime annotations for you automatically.

5.27.2.1 The Three Elision Rules

The Rust compiler tries inferring lifetimes with these three rules:

Rule 1: each position in the parameters that can hold a lifetime gets its own independent lifetime

```
fn foo(a: &str, b: &str)
// The compiler sees: fn foo<'a, 'b>(a: &'a str, b: &'b str)
```

Rule 2: if after Rule 1 there is exactly one input lifetime, the return value's lifetime equals it

```
fn first_word(s: &str) -> &str
// Rule 1: fn first_word<'a>(s: &'a str) -> &str
// Rule 2: only one input lifetime 'a → fn first_word<'a>(s: &'a str) -> &'a str
```

That's why `first_word` above needs no `'a` — with just one input lifetime, Rule 2 handles it.

Note that one parameter can carry several input lifetimes — e.g. `&'a &'b T` (a reference to a

reference) has two (`'a` and `'b`). With two or more input lifetimes, Rule 2 no longer applies.

Rule 3: if there's a `&self` or `&mut self` parameter, the return value's lifetime equals `self`'s

```
impl MyStruct {
    fn name(&self) -> &str { ... }
    // The compiler sees: fn name<'a>(&'a self) -> &'a str
}
```

5.27.2.2 When Do the Rules Fall Short?

When there are several reference parameters and it's unclear which one the return value's lifetime binds to — exactly the situation of last episode's `longer` function. That's when manual annotation becomes mandatory.

5.27.2.3 Summary

- One reference parameter → almost never needs writing.
- A method returning part of `&self` → no writing needed.

5.27.3 Example Code

```
// Rule 2: one input lifetime, inferred automatically
fn trim_hello(s: &str) -> &str {
    if s.len() >= 5 {
        &s[5..]
    } else {
        s
    }
}

struct Article {
    title: String,
    content: String,
}

impl Article {
    fn new(title: String, content: String) -> Article {
        Article { title, content }
    }

    // Rule 3: with a &self parameter, the return's lifetime binds to self
    fn title(&self) -> &str {
        &self.title
    }

    fn summary(&self) -> &str {
        &self.content
    }
}

// Multiple reference parameters + returning a reference → manual annotation needed
fn pick_longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() {
        a
```

```
    } else {
        b
    }
}

fn main() {
    // Rule 2: no lifetime to write
    let greeting = String::from("Hello, world!");
    let trimmed = trim_hello(&greeting);
    println!("{}", trimmed);

    // Rule 3: methods taking references need no lifetimes
    let article = Article::new(
        String::from("Rust lifetimes"),
        String::from("Not as scary as they seem"),
    );
    println!("Title: {}", article.title());
    println!("Summary: {}", article.summary());

    // Multiple reference parameters: manual annotation
    let a = String::from("hello");
    let b = String::from("hi");
    let result = pick_longer(&a, &b);
    println!("The longer one: {}", result);
}
```

5.27.4 Recap

- Rust has three **elision rules** that fill in lifetime annotations automatically most of the time.
- Rule 1: each lifetime-capable position in the parameters gets its own independent lifetime.
- Rule 2: exactly one input lifetime → the return value's lifetime automatically equals it.
- Rule 3: a method with `&self` or `&mut self` → the return value's lifetime automatically equals `self`'s.

5.28 Lifetimes on Types

5.28.1 Goal of This Episode

Learn to annotate lifetimes on `structs` and `enums` that contain references, and simplify annotations with the anonymous lifetime `'_`.

5.28.2 Concept

Until now, our `structs` and `enums` have owned their own data (`String`, `i32`, etc.). But sometimes you want them to borrow someone else's data — say, storing an `&str` instead of a `String`.

5.28.2.1 References inside Types

```
struct Excerpt {
    text: &str, // Compile error!
}
```

This errors, because Rust needs to know: "How long can this `&str` live?" If the borrowed data is released, the reference in the `struct` becomes dangling.

The fix is a lifetime parameter:

```
struct Excerpt<'a> {
    text: &'a str,
}
```

`'a` tells Rust: "This `struct` may not outlive the data it borrows."

Same for `enum`s — if a variant carries a reference, it needs a lifetime:

```
enum Token<'a> {
    Word(&'a str),
    Number(i32),
}
```

`Token::Word` borrows a piece of text, so a `Token` can't outlive that text. `Token::Number` contains no references, but since it shares the `enum` with `Word`, `Token::Number(42)` still has the type `Token<'a>` — the compiler infers that `'a` for you, and it has no practical effect for `Number`.

5.28.2.2 Using a Type with a Lifetime

```
    let novel = String::from("A very long story...");
    let excerpt = Excerpt { text: &novel };
```

`excerpt` borrows `novel`'s data, so `excerpt` can't live longer than `novel`.

5.28.2.3 The Anonymous Lifetime `'_`

When a lifetime can be inferred, you can simplify with `'_`:

```
fn print_excerpt(e: &Excerpt<'_>) {
    println!("{}", e.text);
}
```

`'_` tells Rust "I know a lifetime belongs here — infer it yourself." Remember the type placeholder `_` from Episode 5? `'_` is its lifetime counterpart.

5.28.2.4 `impl` for a `struct` with a Lifetime

```
impl<'a> Excerpt<'a> {
    fn text(&self) -> &str {
        self.text
    }
}
```

Just like `impl` on a generic `struct` — `impl<'a>` declares the lifetime parameter, and `Excerpt<'a>` uses it.

Note that `fn text(&self) -> &str` needs no lifetime annotations at all — last episode's third elision rule kicks in: with `&self` on a method, the return value's lifetime automatically equals `self`'s.

5.28.2.5 Lifetime-carrying Types as Function Parameters

If a function receives a type carrying a lifetime, `'_` lets the compiler infer:

```
fn into_text(e: Excerpt<'_>) -> &str {
    e.text
}
```

Written out in full:

```
fn into_text<'a>(e: Excerpt<'a>) -> &'a str {
    e.text
}
```

The elision rules see `Excerpt<'_>` carrying one input lifetime, and Rule 2 sets the return value's lifetime to the same one.

Note that `e` itself isn't a reference — `e` gets `dropped` when the function ends. But the returned `&'a str` doesn't borrow `e`; it borrows the text stored inside `e` — text whose lifespan is `'a`, unrelated to `e`'s own.

5.28.3 Example Code

```
// A struct holding a reference needs a lifetime annotation
struct Excerpt<'a> {
    text: &'a str,
    page: i32,
}

impl<'a> Excerpt<'a> {
    fn new(text: &'a str, page: i32) -> Excerpt<'a> {
        Excerpt { text, page }
    }

    fn text(&self) -> &str {
        self.text
    }

    fn summary(&self) -> String {
        let mut s = String::from("Page ");
        let page_str = self.page.to_string();
        s.push_str(&page_str);
        s.push_str(": ");
        s.push_str(self.text);
        s
    }
}

// Using the anonymous lifetime '_
fn print_excerpt(e: &Excerpt<'_>) {
    println!("[p.{}] {}", e.page, e.text);
```

```
}

fn main() {
    let novel = String::from("A long, long time ago, there was a programmer...");

    // excerpt borrows novel's data
    let excerpt = Excerpt::new(&novel[..15], 1);
    println!("{}", excerpt.text());
    println!("{}", excerpt.summary());

    // A function using the anonymous lifetime
    print_excerpt(&excerpt);

    // excerpt can't outlive novel
    // If novel were dropped, excerpt would become unusable
}
```

5.28.4 Recap

- A `struct` holding references must annotate lifetimes: `struct Excerpt<'a> { text: &'a str }`.
- The lifetime guarantees the `struct` won't outlive the data it borrows.
- `'_` is the anonymous lifetime, letting the compiler infer (the lifetime version of _).
- `impl` for a lifetime-carrying `struct`: `impl<'a> Excerpt<'a> { ... }`.

5.29 Lifetime Bounds

5.29.1 Goal of This Episode

Learn lifetime bounds like `T: 'a`, and understand why `&'a T` requires every reference inside `T` to outlive `'a`.

5.29.2 Concept

5.29.2.1 The Problem: `T` Might Contain References

So far, our generic functions have mostly handled types owning their own data — `i32`, `String`. But `T` could also be `&str`, or some other type containing references.

Take this `struct`:

```
struct Ref<'a, T> {
    value: &'a T,
}
```

If `T` is `&'x str`, then `value` is `&'a &'x str` — a reference pointing at another reference. In that case, `'x` must live at least as long as `'a`, or the inner `&'x str` might expire while the outer `&'a` is still alive.

5.29.2.2 What `T: 'a` Means

`T: 'a` is a **lifetime bound**, meaning "every reference inside `T` outlives `'a`."

If `T` is `i32` (no references), `T: 'a` is satisfied automatically. If `T` is `&'x str`, then `T: 'a` requires `'x` to live at least as long as `'a`.

5.29.2.3 When Do You Write It?

In many cases, the compiler sees `&'a T` and knows `T: 'a` is needed, adding it for you. But in certain `trait` definitions or more intricate generic structures, you may need to write it by hand:

```
struct Ref<'a, T: 'a> {
    value: &'a T,
}
```

The `T: 'a` here is actually redundant (the compiler derives it from `&'a T`), but writing it out isn't wrong, and it makes the intent clearer.

5.29.2.4 References to Lifetime-carrying Types

The same reasoning extends to any type carrying a lifetime. If you have `&'b A<'a>` — a reference living for `'b`, pointing at an `A<'a>` — then the whole `A<'a>` must remain valid throughout `'b`. That means the data `A` borrows must outlive `'b`; in other words, `'a` must be at least as long as `'b`.

The reason is intuitive: holding a `&'b` reference, you can reach all the data `A` borrows. If `A`'s borrowed data expired before your reference did, you could touch memory that's already been reclaimed. So Rust requires `'a` to live at least as long as `'b`.

5.29.3 Example Code

```
struct Excerpt<'a> {
    text: &'a str,
}

// T: 'a ensures the references inside T outlive 'a
struct Ref<'a, T: 'a> {
    value: &'a T,
}

impl<'a, T: 'a> Ref<'a, T> {
    fn new(value: &'a T) -> Ref<'a, T> {
        Ref { value }
    }

    fn get(&self) -> &T {
        self.value
    }
}

fn main() {
    // T = i32 (no references; T: 'a automatically satisfied)
    let num = 42;
    let r = Ref::new(&num);
    println!("Ref<i32>: {}", r.get());
```

```
    // T = &str (T is itself a reference)
    let text = String::from("hello");
    let slice: &str = &text;
    let r2 = Ref::new(&slice);
    println!("Ref<&str>: {}", r2.get());

    // An example of &'b A<'a>
    let novel = String::from("A very long story...");
    let excerpt = Excerpt { text: &novel };
    let r3 = &excerpt; // &'b Excerpt<'a>
    // Here 'a is novel's lifespan, and 'b is how long r3 borrows excerpt
    // novel lives at least as long as r3, so 'a outlives 'b — condition satisfied
    println!("Reading through the reference: {}", r3.text);

    // T = String (owns its data, no references; T: 'a automatically satisfied)
    let s = String::from("world");
    let r3 = Ref::new(&s);
    println!("Ref<String>: {}", r3.get());
}
```

5.29.4 Recap

- `T: 'a` means every reference inside `T` outlives `'a`.
- If `T` holds no references (like `i32`, `String`), `T: 'a` is automatically satisfied.
- For `&'a T` to be legal, `T: 'a` is required — usually inferred by the compiler.
- Understanding lifetime bounds is key to reading the standard library's more intricate generics.

5.30 Supertraits

5.30.1 Goal of This Episode

Learn to define dependencies between `traits` with supertraits, and understand the design reasoning behind `Copy: Clone` and `DerefMut: Deref`.

5.30.2 Concept

Sometimes one `trait` needs to build on top of another.

5.30.2.1 Supertrait Syntax

```
trait Summarize: std::fmt::Display {
    fn summary(&self) -> String;
}
```

`Summarize: Display` means: "To implement `Summarize`, you must first implement `Display`." `Display` is `Summarize`'s **supertrait**; conversely, `Summarize` is `Display`'s **subtrait**.

The benefit: inside `Summarize`'s default implementations, or in user code, you can rely on `self` implementing `Display`.

Note: **implementing `Summarize` does not implement `Display` for you automatically**. You

must implement `Display` by hand before you can implement `Summarize`. A supertrait is a "prerequisite," not a "free bonus."

5.30.2.2 `Copy: Clone`

Chapter 4 covered `Copy` and `Clone`. The relationship between them is exactly a supertrait:

```
trait Copy: Clone { }
```

This says: **to implement `Copy`, you must first implement `Clone`.**

Why? Because `Copy` is an "automatic copying" ability, while `Clone` is "manual cloning." Logically, if you can copy automatically, you can surely `clone` manually. So `Copy` demands `Clone` as its prerequisite.

That's why `#[derive(Copy, Clone)]` lists both. In this example, writing only `derive(Copy)` would fail because `Point` has no other `Clone` implementation. If `Clone` were implemented manually, `Copy` could be derived on its own.

5.30.2.3 `DerefMut: Deref`

Episode 23's `DerefMut` follows the same reasoning — `DerefMut`'s supertrait is `Deref`. To dereference mutably, you must first be able to dereference immutably. So any type implementing `DerefMut` necessarily implements `Deref` too.

5.30.3 Example Code

```
use std::fmt::Display;
use std::fmt::Formatter;

// Defining a supertrait: Summarize requires Display
trait Summarize: Display {
    fn summary(&self) -> String {
        // Display is required by the supertrait bound,
        // so Rust also provides .to_string() through ToString
        let full = self.to_string();
        // Collect the chars into a Vec so we measure length in characters
        // (.len() on a string counts bytes)
        let mut chars = Vec::new();
        for c in full.chars() {
            chars.push(c);
        }
        if chars.len() > 10 {
            let mut s = String::new();
            // Take the first 10 characters
            for c in &chars[..10] {
                s.push(*c);
            }
            s.push_str("...");
            s
        } else {
            full
        }
    }
}
```

```rust
struct Article {
    title: String,
    content: String,
}

// Display (the supertrait) must be implemented first
impl Display for Article {
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "{}: {}", self.title, self.content)
    }
}

// Only then can Summarize be implemented
impl Summarize for Article {}

// Demonstrating Copy: Clone
#[derive(Debug, Clone, Copy)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let article = Article {
        title: String::from("Rust"),
        content: String::from("A wonderful programming language, well worth learning"),
    };

    // Using Display (the supertrait)
    println!("Full: {}", article);

    // Using Summarize's default implementation:
    // .to_string() comes from ToString, made available by Display
    println!("Summary: {}", article.summary());

    // Demonstrating that Copy requires Clone
    let p = Point { x: 1, y: 2 };
    let p2 = p; // copy (automatic)
    let p3 = p.clone(); // clone (manual) works too
    println!("{:?} {:?} {:?}", p, p2, p3);
}
```

5.30.4 Recap

- `trait A: B` means "implementing `A` requires implementing `B` first" — `B` is `A`'s supertrait, `A` is `B`'s subtrait.
- `Copy: Clone` — implementing `Copy` requires a `Clone` implementation; the two are commonly derived together, but `Clone` may be implemented manually.
- `DerefMut: Deref` — mutable dereferencing presupposes immutable dereferencing.
- Implementing a subtrait doesn't auto-implement the supertrait — you must write `impl Supertrait` yourself first.
- A subtrait's default implementations may rely on capabilities guaranteed by the super-

trait.

5.31 Common `derive` traits

5.31.1 Goal of This Episode

Learn the uses of, and differences between, the common `derive` traits: `PartialEq`, `Eq`, `PartialOrd`, `Ord`, `Default`, and friends.

5.31.2 Concept

Chapter 4 covered `Debug`, `Clone`, and `Copy`. Rust's standard library has other `derive`-able `traits` — today we meet the most common ones.

5.31.2.1 `PartialEq` and `Eq`

`PartialEq` lets your type be compared with == and `!=`.

```
#[derive(PartialEq)]
struct Point { x: i32, y: i32 }
```

`Eq` builds on top of `PartialEq` — `PartialEq` is `Eq`'s supertrait (last episode's topic). What `Eq` adds is a guarantee of **reflexivity** — every value equals itself.

"Wait, what value doesn't equal itself?" — `f64::NAN`! In the floating-point standard, `NAN != NAN`. That's why `f64` has only `PartialEq`, not `Eq`.

If your type contains no floats, you can usually `derive` both `PartialEq` and `Eq`.

5.31.2.2 `PartialOrd` and `Ord`

`PartialOrd` lets your type be compared with <, >, <=, >=.

`Ord` is a total ordering — guaranteeing any two values can be ranked. Because of `NAN`, `f64` has only `PartialOrd`, not `Ord`.

`NAN` compared with anything using <, >, <= or >=, or with ==, is `false` — itself included. Only `!=` is the exception: it is always `true`:

```
fn main() {
    let nan = f64::NAN;
    println!("{}", nan < 1.0);  // false
    println!("{}", nan > 1.0);  // false
    println!("{}", nan == nan); // false
    println!("{}", nan <= nan); // false
}
```

That's why `f64` can't have `Ord` — there's no way to place `NAN` in a sorted order, since it stands in no ordering relation to anything; no position makes sense for it.

5.31.2.3 The Full Relationship among the Four `traits`

First their definitions (simplified):

```
pub trait PartialEq { ... }
pub trait Eq: PartialEq { }
```

```
pub trait PartialOrd: PartialEq { ... }
pub trait Ord: PartialOrd + Eq { ... }
```

Organized as an inheritance picture:

- `Eq: PartialEq` — total equality presupposes partial equality.
- `PartialOrd: PartialEq` — comparing sizes presupposes comparing equality (since `<=` subsumes `==`).
- `Ord: PartialOrd + Eq` — total ordering presupposes partial ordering **and** total equality.

Why does `PartialOrd` require `PartialEq`? Because "comparing sizes" implicitly involves "judging equality" — if `a <= b` and `b <= a`, then `a == b`.

Why does `Ord` require `Eq`? Because a total ordering must compare any two values, equal ones included. And `Ord` guarantees every value a definite position, so values like `NAN` that "aren't equal to themselves" aren't allowed.

That's why `f64` can only walk one side (`PartialEq` + `PartialOrd`) and never reach the other (`Eq` + `Ord`).

5.31.2.4 `Default`

The `Default` `trait` provides a "default value." Numbers default to `0`, `bool` to `false`, `String` to the empty string, `Vec` to an empty `Vec`.

If every field of a `struct` has `Default`, you can `derive` it:

```
#[derive(Debug, Default)]
struct Config {
    width: i32,
    height: i32,
    title: String,
}

fn main() {
    let config = Config::default();
    // Config { width: 0, height: 0, title: "" }
}
```

5.31.3 Example Code

```
#[derive(Debug, PartialEq, Eq, PartialOrd, Ord, Clone, Default)]
struct Student {
    grade: i32,
    name: String,
}

fn main() {
    let alice = Student { grade: 90, name: String::from("Alice") };
    let bob = Student { grade: 85, name: String::from("Bob") };
    let alice2 = Student { grade: 90, name: String::from("Alice") };

    // PartialEq: == and !=
    println!("alice == alice2: {}", alice == alice2);
```

```
    println!("alice == bob: {}", alice == bob);
    println!("alice != bob: {}", alice != bob);

    // PartialOrd: < > <= >=
    // The derived Ord compares fields in declaration order (grade first, then name)
    println!("alice > bob: {}", alice > bob);
    println!("bob < alice: {}", bob < alice);

    // Sorting requires Ord
    let mut students = vec![
        Student { grade: 70, name: String::from("Charlie") },
        Student { grade: 90, name: String::from("Alice") },
        Student { grade: 85, name: String::from("Bob") },
    ];

    students.sort();
    for s in &students {
        println!("{}: {}", s.name, s.grade);
    }

    // f64's special case: NAN
    let nan = f64::NAN;
    println!("NAN == NAN: {}", nan == nan); // false!
    println!("NAN < 1.0: {}", nan < 1.0);   // false!
    println!("NAN > 1.0: {}", nan > 1.0);   // false!

    // f64 has no Ord, so .sort() won't work
    // let mut floats = vec![1.0, 2.0, f64::NAN];
    // floats.sort(); // Compile error! f64 doesn't implement Ord

    // Default
    let default_student = Student::default();
    println!("Default student: {:?}", default_student);
    // Student { grade: 0, name: "" }
}
```

5.31.4 Recap

- `PartialEq`: `==`, `!=` comparisons; `Eq`: guarantees reflexivity (`NAN` being the exception).
- `PartialOrd`: <, >, <=, >= comparisons; `Ord`: guarantees a total ordering.
- Because of `NAN`, `f64` has only the `Partial` versions, never the total ones.
- The derived `Ord` compares fields one by one in declaration order.
- `Default`: provides default values (numbers `0`, `bool` `false`, `String` empty).

5.32 Associated Types

5.32.1 Goal of This Episode

Learn to define associated types in a `trait`, and understand how they differ from generic parameters.

5.32.2 Concept

In Episode 18 we learned multi-parameter `traits`: `trait Convert<T>`. But sometimes a type parameter isn't "open" — a given type will only ever have one sensible implementation.

5.32.2.1 The Problem: Multi-parameter `traits` Are Too Permissive

Picture a "container" `trait`. What element type does the container hold? With a multi-parameter `trait`:

```
trait Container<T> {
    fn first(&self) -> Option<&T>;
}
```

But that means one type could implement both `Container<i32>` and `Container<String>` at the same time — while a container usually has just one element type.

5.32.2.2 Associated Types: A One-to-one Relationship

Associated types solve this:

```
trait Container {
    type Item;
    // To use Self's associated type, write Self::Type
    fn first(&self) -> Option<&Self::Item>;
}
```

`type Item;` declares an associated type. Implementations must specify what it is:

```
impl Container for NumberList {
    type Item = i32;
    fn first(&self) -> Option<&i32> {
        self.data.first()
    }
}
```

Once `Self` (`NumberList`) and the angle-bracket parameters (none here) are fixed, `Item` is uniquely determined as `i32` — no ambiguity.

5.32.2.3 The Difference from Generic Parameters

You can picture a `trait` as a function that takes some "inputs" and determines some "outputs":

- **Inputs**: `Self` (who implements the `trait`) and the type parameters in angle brackets (<T>).
- **Outputs**: associated types (`type Item`).

Inputs determine outputs — once "who" (`Self`) and "the angle-bracket parameters" are fixed, the associated type is uniquely determined.

For example, the `T` in `Convert<T>` is an input, so the same `Self` with different `T`s can have different implementations: `i32` can implement both `Convert<String>` and `Convert<(i32,)>`.

But `Container`'s `Item` is an output. Once `Self` is fixed as `NumberList`, `Item` can have **only one answer** — `i32`.

Which to use? If "once all inputs are fixed, only one sensible answer remains," put it in an associated type (output). If "the same inputs can pair with several different answers," put it in

the angle brackets (input).

5.32.2.4 `Deref` Has an Associated Type Too

The `Deref trait` from Episode 23 uses an associated type:

```
trait Deref {
    type Target;
    fn deref(&self) -> &Self::Target;
}
```

`type Target` is the type reached by dereferencing.

For example, if `p` is a `Box<i32>` and we write:

```
    let n = *p;
```

Rust treats the `*p` part roughly like this:

```
    let n = *p.deref();
```

`deref` takes `&self`, so `p.deref()` borrows the smart pointer first, then returns a reference to the inner value: `&Self::Target`. Finally, the outer `*` follows that reference.

For `Box<i32>`, `Self::Target` is `i32`, so `.deref()` returns `&i32`. Since `i32` is `Copy`, `let n = *p;` can create another `i32`.

This is the same reasoning as `Container`'s `type Item`: once `Self` is fixed as `Box<i32>`, `Target` has only one answer — `i32`. That is why `Target` is an associated type rather than a generic parameter.

5.32.2.5 `DerefMut` Uses the Same `Target`

The mutable version is `DerefMut`. In simplified form, it looks like this:

```
trait DerefMut: Deref {
    fn deref_mut(&mut self) -> &mut Self::Target;
}
```

`DerefMut` does not declare another associated type. It uses `Self::Target` from `Deref`.

That matters because immutable dereferencing and mutable dereferencing must reach the same kind of inner value. If `Box<String>` has `Target = String`, then `.deref()` returns `&String`, and `.deref_mut()` returns `&mut String`.

For example:

```
    *p = String::from("world");
```

Since the left side is `*p`, Rust needs mutable access to the inner value. It treats that part roughly like this:

```
    *p.deref_mut() = String::from("world");
```

`deref_mut` takes `&mut self`, so `p.deref_mut()` mutably borrows the smart pointer first, then returns a mutable reference to the inner value: `&mut Self::Target`. Finally, the outer `*` follows that mutable reference, so the assignment can replace the inner `String`.

5.32.2.6 Specifying Associated Types in `trait` Bounds

You can specify an associated type's concrete type inside a `trait` bound:

```
fn print_first<C: Container<Item = i32>>(c: &C) { ... }
```

`Container<Item = i32>` means "implements `Container`, with `Item` being `i32`."

If you only want to require the associated type to implement some `trait`, you can write the `trait` bound right after it:

```
fn print_first<C: Container<Item: Display>>(c: &C) { ... }
```

`Item: Display` means "`Container`'s `Item` must implement `Display`."

5.32.3 Example Code

```
use std::fmt::Display;

// A container trait defined with an associated type
trait Container {
    type Item;

    fn first(&self) -> Option<&Self::Item>;
    fn last(&self) -> Option<&Self::Item>;
    fn len(&self) -> usize;
}

struct NumberList {
    data: Vec<i32>,
}

impl Container for NumberList {
    type Item = i32; // Specifying the associated type

    fn first(&self) -> Option<&i32> {
        self.data.first()
    }

    fn last(&self) -> Option<&i32> {
        self.data.last()
    }

    fn len(&self) -> usize {
        self.data.len()
    }
}

struct WordList {
    words: Vec<String>,
}

impl Container for WordList {
    type Item = String; // A different type, a different Item

    fn first(&self) -> Option<&String> {
```

```
        self.words.first()
    }

    fn last(&self) -> Option<&String> {
        self.words.last()
    }

    fn len(&self) -> usize {
        self.words.len()
    }
}

// Using the associated type in a trait bound
fn print_first_item<C>(c: &C)
where
    C: Container,
    C::Item: Display,
{
    match c.first() {
        Some(item) => println!("The first element: {}", item),
        None => println!("The container is empty"),
    }
}

fn main() {
    let nums = NumberList { data: vec![10, 20, 30] };
    let words = WordList {
        words: vec![
            String::from("hello"),
            String::from("world"),
        ],
    };

    println!("Number container length: {}", nums.len());
    print_first_item(&nums);

    println!("Word container length: {}", words.len());
    print_first_item(&words);

    // last
    match nums.last() {
        Some(n) => println!("The last number: {}", n),
        None => println!("Empty"),
    }
}
```

5.32.4 Recap

- `type Item;` defines an associated type in a `trait`.
- The `Self::Item` syntax reads `Self`'s associated type within the `trait` definition.
- Implementations specify the concrete type with `type Item = i32;`.
- **Input vs output**: `Self` and the angle-bracket parameters are inputs; associated types are outputs. Inputs determine outputs.

- `Deref`'s `type Target` is an associated type too — `Box<T>` has `Target = T`, meaning dereferencing reaches `T`.
- `DerefMut` uses the same `Self::Target` and returns `&mut Self::Target`.
- In a `trait` bound, `Container<Item = i32>` specifies the associated type.
- A `trait` bound can also require the associated type to implement a `trait`: `Container<Item: Display>`.

5.33 `Cow<'a, B>`

5.33.1 Goal of This Episode

Learn to use `Cow<'a, str>` for a flexible "borrow when possible, `clone` only when needed" strategy.

5.33.2 Concept

Some functions can sometimes return borrowed data directly, yet other times must return owned data.

5.33.2.1 An Example

Suppose you have a function that prepends a greeting to a string. If the string already starts with "Hello", just return the original (borrowed). If not, a new string must be built (owned).

Should the return type be `&str` or `String`? Neither is quite right.

5.33.2.2 `Cow` to the Rescue

`Cow` stands for **`Clone` on write**. It lives in the `std::borrow` module. Here's a simplified definition to show the core structure; it omits pieces needed by the full standard-library type, so this sketch is not a drop-in working implementation:

```
enum Cow<'a, B>
where
    B: 'a + ToOwned,
{
    Borrowed(&'a B),
    Owned(B::Owned), // ToOwned's associated type
}
```

Line by line:

- **`'a`**: the lifetime parameter — the lifespan of the borrowed data.
- **`B: 'a`**: a lifetime bound (from recent episodes); references inside `B` must outlive `'a`.
- **`B: ToOwned`**: a `trait` bound; `B` must implement `ToOwned`.
- **`Borrowed(&'a B)`**: the borrowed version, holding an `&'a B`.
- **`Owned(...)`**: the owning version, whose type is decided by `ToOwned`'s associated type `Owned`.

`ToOwned` is a `trait` with an associated type `Owned`, representing "the owning version of the type."

For `str`:

- `str` implements `ToOwned` with `type Owned = String`.
- So `Cow<'a, str>` = `Borrowed(&'a str)` or `Owned(String)`.

For `[T]`:

- `[T]` implements `ToOwned` with `type Owned = Vec<T>`.
- So `Cow<'a, [T]>` = `Borrowed(&'a [T])` or `Owned(Vec<T>)`.

5.33.2.3 `Cow` Implements `Deref`

The most crucial point when using `Cow`: `Cow<'a, B>` implements `Deref<Target = B>`. Meaning: whether it holds `Borrowed(&str)` or `Owned(String)`, you can use `&Cow<'_, str>` directly as an `&str` — calling `&str`'s methods or passing it to functions accepting `&str`, without ever caring whether it's borrowed or owned.

```
use std::borrow::Cow;

fn main() {
    let cow: Cow<'_, str> = Cow::Owned(String::from("hello"));
    // Used directly as &str; Deref handles it
    println!("Length: {}", cow.len());
    println!("Uppercase: {}", cow.to_uppercase());
}
```

Thanks to `Deref`, callers usually needn't care whether the inside is borrowed or owned — just use it as an `&str`.

5.33.2.4 Common Methods

- **`.to_mut()`**: if it's `Borrowed`, first `clone`s into `Owned`, then returns a mutable reference. If already `Owned`, returns its mutable reference directly. This is the heart of "`clone` on write."
- **`.into_owned()`**: converts either variant into an owned value. `Borrowed` gets `cloned`; `Owned` is taken as-is.

5.33.3 Example Code

```
use std::borrow::Cow;

// If the string already starts with "Hello", return it borrowed
// Otherwise build a new String
fn ensure_greeting(s: &str) -> Cow<'_, str> {
    if s.starts_with("Hello") {
        // No modification needed; borrow directly
        Cow::Borrowed(s)
    } else {
        // Modification needed; build a new string
        let mut greeting = String::from("Hello, ");
        greeting.push_str(s);
        Cow::Owned(greeting)
    }
}

fn main() {
    // Already starts with "Hello" → borrowed, zero cost
    let s1 = "Hello world";
```

```
    let result1 = ensure_greeting(s1);
    println!("{}", result1);

    // Doesn't start with "Hello" → a new string is built
    let s2 = "Rust";
    let result2 = ensure_greeting(s2);
    println!("{}", result2);

    // You can check whether it's borrowed or owned
    match ensure_greeting(s1) {
        Cow::Borrowed(s) => println!("Borrowed: {}", s),
        Cow::Owned(s) => println!("Owned: {}", s),
    }

    match ensure_greeting(s2) {
        Cow::Borrowed(s) => println!("Borrowed: {}", s),
        Cow::Owned(s) => println!("Owned: {}", s),
    }

    // to_mut: clone on write
    let mut cow: Cow<'_, str> = Cow::Borrowed("hello");
    // It's Borrowed now; calling to_mut clones it into Owned first
    cow.to_mut().push_str(" world");
    println!("{}", cow); // "hello world"

    // into_owned: converting into an owned String
    let cow2: Cow<'_, str> = Cow::Borrowed("bye");
    let owned: String = cow2.into_owned();
    println!("{}", owned);
}
```

5.33.4 Recap

- `Cow<'a, str>` can be borrowed (`&str`) or owned (`String`), as circumstances demand.
- `Cow` uses the `ToOwned` `trait`'s associated type to decide the owning version's type (`str` → `String`, `[T]` → `Vec<T>`).
- `Cow` implements `Deref`: whether `Borrowed` or `Owned`, a `&Cow<'_, str>` can be used directly as an `&str` — its greatest strength.
- `.to_mut()`: `clone` on write (`Borrowed` → `clone` into `Owned` → return a mutable reference).
- `.into_owned()`: converts either variant into an owned value.
- Suited to "mostly no modification, occasional modification" scenarios.

Congratulations on finishing Chapter 5! 🎉 This chapter was truly packed — from generics, `trait` bounds, and lifetimes, through smart pointers like `Box` and `Rc` and the `Deref` machinery, to the interior mutability of `Cell` and `RefCell`, plus `Display`, associated types, and `Cow`. These are the most powerful weapons in Rust's type system, and the foundation for reading the standard library's source. Next chapter, we enter closures and iterators — Rust's most elegant style of functional programming!

Chapter 6

Closures and Iterators

This chapter teaches closures and iterators. These features combine many of the language capabilities covered earlier and are among the more advanced, harder-to-grasp topics — which is why they come after Chapter 5. Even so, they're extremely common in real-world Rust code, so if you want to read genuine Rust, understanding closures and iterators is a must.

6.1 Function Pointers

6.1.1 Goal of This Episode

Meet the function pointer type, and learn to pass and store function names as values.

6.1.2 Concept

In Rust, functions can be more than called — they can be passed around like values, stored in variables, and put into `Vec`s. To do that, we need to meet the **function pointer** type.

6.1.2.1 How Function Pointers Are Written

Suppose you have a function:

```
fn add_one(x: i32) -> i32 {
    x + 1
}
```

This function's pointer type is `fn(i32) -> i32`. Note the lowercase `fn` — it denotes the function pointer type, not the keyword for defining functions.

6.1.2.2 Storing a Function in a Variable

You can assign a function's name directly to a variable:

```
    let f: fn(i32) -> i32 = add_one;
```

Afterward, calling `f(10)` works the same as calling `add_one(10)` directly.

6.1.2.3 Passing Functions as Arguments

One of the most common uses of function pointers is "passing one function to another":

```
fn apply(f: fn(i32) -> i32, value: i32) -> i32 {
    f(value)
}
```

Now `apply` accepts any function with the signature `fn(i32) -> i32` — very flexible.

6.1.2.4 Multiple Parameters and Different Return Types

A function pointer's type is determined by its parameters and return value:

- No parameters, no return value: `fn()`.
- Two parameters: `fn(i32, i32) -> i32`.
- Returning a `String`: `fn(&str) -> String`.

6.1.2.5 Function Pointers vs Next Episode's Closures

The function pointer `fn(...) -> ...` is a concrete type with a fixed size. But a function can't reach the local variables inside another function — to use one, you have to pass it in as a parameter. Next episode introduces closures, which are different: **wherever a closure is written, whatever local variables sit next to it are simply available**, without being passed in one by one.

6.1.3 Example Code

```rust
fn add_one(x: i32) -> i32 {
    x + 1
}

fn double(x: i32) -> i32 {
    x * 2
}

fn apply(f: fn(i32) -> i32, value: i32) -> i32 {
    f(value)
}

fn pick_function(use_double: bool) -> fn(i32) -> i32 {
    if use_double {
        double
    } else {
        add_one
    }
}

fn main() {
    // Storing a function in a variable
    let f: fn(i32) -> i32 = add_one;
    println!("f(5) = {}", f(5));

    // Passing functions as arguments
    println!("apply(add_one, 10) = {}", apply(add_one, 10));
    println!("apply(double, 10) = {}", apply(double, 10));

    // Functions can be return values too
    let chosen = pick_function(true);
    println!("chosen(7) = {}", chosen(7));

    let chosen2 = pick_function(false);
    println!("chosen2(7) = {}", chosen2(7));
```

```
    // Putting functions in a Vec
    let operations: Vec<fn(i32) -> i32> = vec![add_one, double];
    for op in &operations {
        println!("op(3) = {}", op(3));
    }
}
```

6.1.4 Recap

- The function pointer type is written `fn(param_types) -> return_type` — note the lowercase `fn`.
- A function's name works directly as a value: assign it to variables, pass it to other functions, store it in containers like `Vec`.
- A function can't reach the local variables inside another function; anything it needs must be passed in as a parameter. Next episode's closures can use the local variables sitting next to where they're written.

6.2 Closures in Action

6.2.1 Goal of This Episode

Learn basic closure syntax, see how closures capture outside variables, and look at real standard-library cases that use closures.

6.2.2 Concept

6.2.2.1 Closure Syntax

Last episode's function pointers are handy, but a function can't reach the local variables inside another function — anything it needs has to be passed in as a parameter. Closures are different: wherever one is written, it can use the local variables right there — which is exactly why they exist.

A closure's basic syntax wraps the parameters in `|`:

```
    let add_one = |x| x + 1;
```

You can add type annotations, explicit like a function:

```
    let add_one = |x: i32| -> i32 { x + 1 };
```

Calling a closure works like calling an ordinary function — just `add_one(5)`, no special syntax needed.

6.2.2.2 When Are Braces Required?

The rule is simple:

- With **a single expression**, the braces can be dropped: `|x| x + 1`.
- With **multiple lines** or **statements like `let`**, wrap them in braces:

```
    let process = |x: i32| {
        let doubled = x * 2;
```

```
        println!("Computing: {}", doubled);
        doubled + 1
    };
```

As with functions, the last line inside the braces without a semicolon is the return value.

Also, with a return type annotation (`-> i32`), the braces become mandatory:

```
    let add_one = |x: i32| -> i32 { x + 1 }; // With -> the {} are required
    let add_one = |x: i32| x + 1;            // Without -> the {} can be dropped
```

6.2.2.3 Closures Capture Outside Variables

This is the biggest difference from function pointers:

```
fn main() {
    let offset = 10;
    let add_offset = |x| x + offset; // Captures offset
    println!("{}", add_offset(5));   // 15
}
```

The closure `add_offset` "remembers" the outer `offset`, using it on every call. An ordinary function can't do that.

6.2.2.4 Not All Closures Are Alike

Depending on **how** a closure uses its captured variables, Rust sorts closures into different kinds — some callable only once, others many times. This episode shows two examples to get a feel; the next few episodes dig deeper.

6.2.2.5 `Result`'s `map` — a `FnOnce` Example

Many standard-library methods take closures. Remember `Result<T, E>` from Chapter 5? It has a `map` method that transforms the value inside an `Ok`. `map` only needs to call the closure once, so it accepts `FnOnce` — "callable at least once" suffices.

That means you can hand it a closure that **consumes a captured variable**:

```
fn main() {
    let prefix = String::from("The result is: ");
    let result: Result<i32, String> = Ok(42);
    let message = result.map(|x| {
        // prefix gets moved in; this closure can only be called once
        let mut s = prefix; // Move!
        s.push_str(&x.to_string());
        s
    });
    println!("{:?}", message); // Ok("The result is: 42")
}
```

This closure moves `prefix` in; after one call, `prefix` is gone. That's fine — `map` was only ever going to call the received function once.

6.2.2.6 `Vec`'s `retain` — a `FnMut` Example

`Vec<T>`'s `retain` method keeps elements meeting a condition and removes the rest. It takes a closure receiving `&T` (a reference to each element) and returning `bool` (`true` keeps, `false` re-

moves). Since `retain` must call it once per element, it demands `FnMut` — "callable repeatedly."

You can pass a closure that **modifies a captured variable**:

```
fn main() {
    let mut numbers = vec![1, 2, 3, 4, 5, 6];
    let mut removed_count = 0;
    numbers.retain(|x| {
        if x % 2 == 0 {
            true // Keep the evens
        } else {
            removed_count += 1; // Modifying an outer variable
            false
        }
    });
    println!("{:?}, removed {}", numbers, removed_count);
    // [2, 4, 6], removed 3
}
```

This closure modifies `removed_count` each time it's called — it's `FnMut`. Note it moves nothing (it only modifies the outer variable through `&mut`), so it can be called many times.

6.2.2.7 What If a `FnOnce` Goes to `retain`?

Could the variable-moving closure we gave `Result`'s `map` be passed to `retain`?

```
    let mut items = vec![1, 2, 3];
    let header = String::from("Removing: ");
    items.retain(|x| {
        if *x <= 1 {
            let mut log = header; // Moves header
            log.push_str(&x.to_string());
            log.push(' ');
        }
        *x > 1
    }); // Compile error!
```

This closure moves `header` away the first time it removes an element; by the second removal, `header` no longer exists. It's callable only once (`FnOnce`), but `retain` needs repeated calls (`FnMut`). So the compiler objects.

6.2.2.8 Capture-free Closures → Convertible to Function Pointers

If a closure captures no outer variables, it's not much different from an ordinary function. Rust allows it to convert automatically into a function pointer `fn`:

```
    let add_one: fn(i32) -> i32 = |x| x + 1; // No captures; convertible to fn
```

But once it captures an outer variable, that conversion is off the table.

6.2.3 Example Code

```
fn apply_fn_pointer(f: fn(i32) -> i32, value: i32) -> i32 {
    f(value)
}
```

```rust
fn main() {
    // Basic closure syntax
    let square = |x: i32| -> i32 { x * x };
    println!("square(4) = {}", square(4));

    // Capturing an outer variable
    let base = 100;
    let add_base = |x| x + base;
    println!("add_base(7) = {}", add_base(7));

    // Result's map (FnOnce)
    let result: Result<i32, String> = Ok(21);
    let doubled = result.map(|x| x * 2);
    println!("doubled = {:?}", doubled);

    let err_result: Result<i32, String> = Err(String::from("oops"));
    let still_err = err_result.map(|x| x * 2);
    println!("still_err = {:?}", still_err);

    // Vec's retain (FnMut)
    let mut scores = vec![55, 72, 88, 43, 91, 60];
    scores.retain(|s| *s >= 60);
    println!("Passing scores: {:?}", scores);

    // A capture-free closure converts to a function pointer
    let triple: fn(i32) -> i32 = |x| x * 3;
    println!("apply_fn_pointer(triple, 5) = {}", apply_fn_pointer(triple, 5));

    // A capturing closure can't convert to a function pointer
    // let offset = 10;
    // let bad: fn(i32) -> i32 = |x| x + offset; // Compile error!
}
```

6.2.4 Recap

- Closures use the `|params| expression` syntax; type annotations can be omitted for Rust to infer.
- A closure's defining feature is **capturing outer variables** — something function pointers can't do.
- `Result`'s `map` accepts `FnOnce` closures — one call needed.
- `Vec`'s `retain` accepts `FnMut` closures — repeated calls needed.
- A once-only closure (`FnOnce`) can't go to a method that needs repeated calls.
- Capture-free closures convert automatically into function pointers `fn`.

6.3 Implementing Closures by Hand

6.3.1 Goal of This Episode

We'll manually model closures as `structs` + methods, so you can understand what the compiler does behind the scenes. You'll see one `struct`-based example for each of the three closure kinds, and why calling a closure is really calling a method.

6.3.2 Concept

6.3.2.1 A Closure = an Anonymous `struct` + a Method

Last episode we saw closures capture outside variables. But how do they "remember" them?

The answer is direct — the compiler does two things for you:

1. **Creates an anonymous `struct`**, storing the captured variables as fields.
2. **`impl`s a method on that `struct`**, whose body is what you wrote after the `||`.

In other words, the closure body you write (the code inside `{ ... }`) is that method's implementation.

Today we'll do the compiler's job by hand, simulating each of the three closure kinds.

6.3.2.2 Calling a Closure = Calling a Method

When you write `f()` to call a closure, the compiler actually turns it into a method call on the `struct`:

- **`FnOnce`**: `f()` → `f.call_once()` — takes `self`, consuming the whole `struct`.
- **`FnMut`**: `f()` → `f.call_mut()` — takes `&mut self`, a mutable reference to the `struct`.
- **`Fn`**: `f()` → `f.call()` — takes `&self`, a shared reference to the `struct`.

See it? These are the three `self` parameter forms from Chapter 4: `self`, `&mut self`, `&self`. The three closure kinds are, at bottom, the three ways a method can receive `self`.

Last episode introduced `FnOnce` (consumes captured values, one call only) and `FnMut` (modifies captured values, repeatable calls). **`Fn` didn't appear last episode** — it's the third kind: calling it neither consumes the closure nor requires a mutable reference to it, so it can be called any number of times.

Next we'll simulate all three by hand with `struct`s. Note: **the three examples below use different field types**. These are the field types used in these examples, not fixed requirements of the three closure kinds.

6.3.2.3 An `FnOnce` Example: The `struct` Stores Owned Values, the Method Takes `self`

Suppose we have this closure:

```
fn main () {
    let name = String::from("Alice");
    let greet = || {
        let s = name; // The closure body moves name away
        println!("Hello, {}!", s);
    };
    greet();
    // greet(); // Compile error! name was moved; no second call
}
```

The compiler generates something like:

```
struct GreetOnce {
    name: String, // Owns name
}
```

```
// Creating the closure = stuffing the captures into the struct
// let greet = GreetOnce { name };

impl GreetOnce {
    // Calling the closure = calling the struct's method
    fn call_once(self) {
        let s = self.name; // Move name out of the struct
        println!("Hello, {}!", s);
    }
}
```

Since the method takes `self`, the whole `struct` is consumed on the call — hence one call only. That's `FnOnce`.

6.3.2.4 An `FnMut` Example: The `struct` Stores Mutable References, the Method Takes `&mut self`

Suppose the closure modifies a captured variable:

```
fn main() {
    let mut name = String::from("Alice");
    let mut greet = || {
        name.push_str("!");
        println!("Hello, {}", name);
    };
    greet();
    greet(); // Repeated calls are fine
}
```

The compiler's product:

```
struct GreetMut<'a> {
    name: &'a mut String, // A mutable reference to name
}

// let mut greet = GreetMut { name: &mut name };

impl<'a> GreetMut<'a> {
    fn call_mut(&mut self) {
        self.name.push_str("!");
        println!("Hello, {}", self.name);
    }
}
```

Why does the `struct` store `&mut` while the method also takes `&mut self`? Because a closure may capture several variables. If a closure modifies `a`, `b`, and `c`, the `struct` has three fields:

```
struct SomeClosure<'a> {
    a: &'a mut i32,
    b: &'a mut String,
    c: &'a mut Vec<i32>,
}
```

The method takes `&mut self` rather than `self` because `self` would consume it in one call — making it `FnOnce`. `FnMut` needs repeated calls, so it can only take a mutable reference to the `struct`.

6.3.2.5 An `Fn` Example: The `struct` Stores Shared References, the Method Takes `&self`

If the closure only reads captured variables, never modifying:

```
fn main() {
    let name = String::from("Alice");
    let greet = || {
        println!("Hello, {}!", name);
    };
    greet();
    greet(); // Repeated calls, no problem at all
}
```

The compiler's product:

```
struct GreetRef<'a> {
    name: &'a String, // A shared reference to name
}

// let greet = GreetRef { name: &name };

impl<'a> GreetRef<'a> {
    fn call(&self) {
        println!("Hello, {}!", self.name);
    }
}
```

Since the method takes `&self`, the `struct` is neither consumed nor modified — callable any number of times. That's `Fn`.

6.3.2.6 The Comparison Table

`self` kind	Corresponding kind	What the fields hold in this example	What it can do
`self`	`FnOnce`	Owned values (like `String`)	Consumes captures; one call only
`&mut self`	`FnMut`	Mutable references (like `&mut String`)	Modifies captures; repeatable calls
`&self`	`Fn`	Shared references (like `&String`)	Reads only; repeatable calls

6.3.2.7 Wrapping Up: What Is a Closure, Really?

Stringing it all together:

1. The compiler builds an anonymous `struct` for you, storing the captures inside.
2. The closure body you write is the implementation of a method on that `struct`.
3. When you write `f()`, the compiler — depending on the closure's kind — calls the `struct`'s `.call_once()` / `.call_mut()` / `.call()`.

Every time you write a closure, the compiler is backstage doing "build a `struct` → `impl` a method → call the method."

Having grasped that "the closure body is just a method's content," here's a bonus thought: what happens if you write `return` inside a closure? Since the closure body really is some method's implementation, `return` exits that method — i.e. **it exits only the innermost closure**, never the enclosing function. Much like `break`'s default effect — `break` also exits only the innermost loop, not every nested loop at once.

6.3.3 Example Code

The complete code below collects the three examples above. Each `struct` simulates one closure kind with the field types and `self` parameter form used in that example:

```
// === Simulating FnOnce ===
// The struct owns the value; the method takes self
struct GreetOnce {
    name: String,
}

impl GreetOnce {
    fn call_once(self) {
        // The closure body: move name away
        let s = self.name;
        println!("[FnOnce] Hello, {}!", s);
        // self has been consumed; no more use
    }
}

// === Simulating FnMut ===
// The struct stores a mutable reference; the method takes &mut self
struct GreetMut<'a> {
    name: &'a mut String,
}

impl<'a> GreetMut<'a> {
    fn call_mut(&mut self) {
        // The closure body: modify the captured variable
        self.name.push_str("!");
        println!("[FnMut] Hello, {}", self.name);
    }
}

// === Simulating Fn ===
// The struct stores a shared reference; the method takes &self
struct GreetRef<'a> {
    name: &'a String,
}

impl<'a> GreetRef<'a> {
    fn call(&self) {
        // The closure body: read only, no modification
        println!("[Fn] Hello, {}!", self.name);
    }
}

fn main() {
    // --- FnOnce: consumed after one call ---
    let name1 = String::from("Alice");
    let greet_once = GreetOnce { name: name1 };
    greet_once.call_once();
    // greet_once.call_once(); // Compile error! The struct has been consumed

    // --- FnMut: repeatable calls, modifying each time ---
```

```
    let mut name2 = String::from("Bob");
    {
        let mut greet_mut = GreetMut { name: &mut name2 };
        greet_mut.call_mut(); // Bob!
        greet_mut.call_mut(); // Bob!!
        greet_mut.call_mut(); // Bob!!!
    } // greet_mut leaves scope; its mutable reference is no longer in use
    println!("name2 is now: {}", name2);

    // --- Fn: read-only; call as many times as you like ---
    let name3 = String::from("Charlie");
    let greet_ref = GreetRef { name: &name3 };
    greet_ref.call();
    greet_ref.call();
    greet_ref.call();
}
```

6.3.4 Recap

- Behind a closure is an anonymous `struct`; the captured variables become its fields.
- **The three closure kinds differ in how the method receives `self`**: `self` (`FnOnce`), `&mut self` (`FnMut`), `&self` (`Fn`).
- The closure body is the implementation of the `struct`'s method.
- `f()` gets compiled into a method call: `f.call_once()` / `f.call_mut()` / `f.call()`.
- `Fn`: calling it only requires a shared reference to the closure value, so it can be called repeatedly.
- Since a closure body is just a method's content, a `return` inside a closure exits only the innermost closure, not the enclosing function — like `break` exiting only the innermost loop by default.
- Next episode: how the compiler **automatically decides** whether a closure counts as `FnOnce`, `FnMut`, or `Fn`.

6.4 How Closure Kinds Are Inferred

6.4.1 Goal of This Episode

Understand how Rust automatically infers whether a closure is `FnOnce`, `FnMut`, or `Fn` from the closure body's contents.

6.4.2 Main Text

Last episode we hand-simulated the three closure kinds with `structs`, corresponding to `self`, `&mut self`, and `&self`. Yet when writing closures you never tell Rust "this is `FnOnce`" or "this is `FnMut`" — Rust decides automatically.

6.4.2.1 The Inference Rules

Rust looks at **what the closure body does with the captured variables**:

1. **If the body moves a captured variable** (e.g. `let s = captured_string;`) → the closure is **`FnOnce`** — once moved, it's gone; one call only.

2. **If the body needs mutable access to its captured state** (e.g. `count += 1;`) → the closure is **`FnMut`** — repeatable calls, but needing `&mut`.
3. **If the body only needs shared access to its captured state** (e.g. `println!("{}", name);`) → the closure is **`Fn`** — only `&self` needed.

Rust picks **the kind permitting the most usage patterns** — shared access gets `Fn` (an `Fn` closure also works as `FnMut` and `FnOnce`). Needing mutable access makes it `FnMut`. A move makes it `FnOnce`.

6.4.2.2 Examples Side by Side

```
    let name = String::from("Alice");

    // Only reads name → Fn
    let greet = || println!("Hi, {}!", name);

    // Modifies count → FnMut
    let mut count = 0;
    let mut increment = || { count += 1; };

    // Moves name → FnOnce
    let consume = || { let s = name; };
```

No markers needed — Rust reads the closure body and knows.

6.4.2.3 What about Capturing Several Variables?

A closure may capture several variables at once, using each differently:

```
    let name = String::from("Alice");
    let mut count = 0;
    let closure = || {
        count += 1;           // Modifies count → needs &mut
        println!("{}", name); // Only reads name → needs just &
    };
```

Pictured as a `struct`, this closure's anonymous `struct` has two fields: `count` (needing `&mut`) and `name` (needing only `&`). But a closure call has just one `self` — and `&mut self` can perform `&` operations, though not vice versa — so the whole closure is `FnMut` (`&mut self`). Just like a method taking `&mut self` that doesn't necessarily modify every field:

```
struct Data<'a> {
    count: &'a mut i32,
    name: &'a String,
}

impl<'a> Data<'a> {
    fn increment_and_greet(&mut self) {
        *self.count += 1;                  // Modifies count
        println!("Hello, {}!", self.name); // Only reads name
    }
}
```

Closures work the same way.

Likewise, `FnOnce`'s `self` can of course take `&` or `&mut` of the values inside — owning a value includes being able to borrow it.

6.4.2.4 What If Nothing Is Captured?

A capture-free closure is automatically `Fn`, since it needs no outside state:

```
    let add_one = |x: i32| x + 1; // Fn
```

Episode 2's note that "capture-free closures convert to function pointers" follows from this too — such a closure doesn't even need the anonymous `struct`.

6.4.3 Recap

- Rust infers a closure's kind from its body: move → `FnOnce`, mutable access → `FnMut`, shared access → `Fn`.
- No manual markers; the compiler picks the kind permitting the most usage patterns.
- Capture-free closures are `Fn`, convertible to function pointers.
- An `Fn` closure can go where `FnMut` or `FnOnce` is wanted; `FnMut` can go where `FnOnce` is wanted; never the reverse.

6.5 `FnOnce` / `FnMut` / `Fn`

6.5.1 Goal of This Episode

Understand that `FnOnce`, `FnMut`, and `Fn` are `traits` rather than types, grasp their inheritance relationships, and learn to choose the right closure `trait`.

6.5.2 Concept

6.5.2.1 They're `traits`, Not Types

We've been saying `FnOnce`, `FnMut`, and `Fn` for several episodes without formally explaining — they are in fact **`traits`**. Like the `Clone` and `Display` you've already met, `FnOnce` / `FnMut` / `Fn` are `traits` defined in the standard library. Each closure's anonymous `struct` automatically `impls` the corresponding `traits` (last episode's inference rules decide which).

So what do these `traits` look like?

- `FnOnce(Args) -> Ret`: callable once (the call may consume the closure, so further calls are not guaranteed).
- `FnMut(Args) -> Ret`: callable repeatedly, with mutable access to captured state.
- `Fn(Args) -> Ret`: callable repeatedly, without mutable access to captured state.

Watch out! `fn(i32) -> i32` (lowercase) is the function pointer **type**, while `Fn(i32) -> i32` (capitalized) is a **`trait`**. Two entirely different things.

6.5.2.2 The Inheritance Relationships

The three `traits` form supertrait relationships:

```
Fn : FnMut : FnOnce
```

Meaning:

- Everything implementing Fn automatically implements FnMut and FnOnce.
- Everything implementing FnMut automatically implements FnOnce.
- But FnOnce doesn't imply FnMut, nor FnMut imply Fn.

Why this direction?

- **Fn → FnMut**: if a closure runs with just &self, using &mut self certainly works too (it simply uses a mutable reference where a shared one would have sufficed).
- **FnMut → FnOnce**: if a closure runs with &mut self, handing it self (full ownership) certainly works — owning a thing includes being able to modify it. It's just that after the call the struct is consumed, so no second call.

The reverse doesn't hold — a closure that must consume itself (FnOnce) can't promise repeated calls (FnMut).

6.5.2.3 Accepting Closures with impl Trait

Remember Chapter 5's impl Trait? Use it to accept closure parameters:

```
fn call_once(f: impl FnOnce() -> String) -> String {
    f()
}

fn call_many_times(mut f: impl FnMut()) {
    f();
    f();
    f();
}

fn call_twice(f: impl Fn() -> i32) -> i32 {
    f() + f()
}
```

Note the mut on the FnMut parameter — calling an FnMut closure needs &mut self, so f itself must be mut.

6.5.2.4 Design Principle: Pick the Bound Accepting the Most Closures

When designing a function that takes a closure, choose the trait bound **accepting the widest range of closures**:

1. Try FnOnce first — if you only call it once.
2. Move to FnMut — if you need repeated calls.
3. Only then Fn — if you need repeated calls without a mutable reference to the closure value.

Why? Because FnOnce accepts every closure (every closure implements FnOnce), while Fn accepts only closures callable through a shared reference. The widest bound gives callers maximum freedom.

In practice Fn is rarely needed — most functions calling a closure repeatedly do fine with FnMut (which also accepts Fn closures). Use Fn when the function needs to call the closure without a mutable reference to the closure value.

6.5.2.5 Function Pointers Implement All Three `traits` Too

Ordinary functions (and function pointers `fn`) automatically implement `Fn`, `FnMut`, and `FnOnce`. So a function name can be passed anywhere these three `traits` are accepted.

6.5.3 Example Code

```
// Only one call needed → FnOnce (accepts the most closures)
fn consume_and_print(f: impl FnOnce() -> String) {
    let result = f();
    println!("Result: {}", result);
}

// Repeated calls needed → FnMut
fn repeat_three_times(mut f: impl FnMut()) {
    f();
    f();
    f();
}

// Repeated calls without a mutable reference to the closure value → Fn
fn sum_two_calls(f: impl Fn(i32) -> i32, x: i32) -> i32 {
    f(x) + f(x)
}

fn main() {
    // FnOnce: the closure consumes a captured value
    let name = String::from("Rust");
    consume_and_print(|| {
        let s = name; // Moves name
        format!("Hello, {}!", s)
    });

    // FnMut: the closure modifies a captured variable
    let mut count = 0;
    repeat_three_times(|| {
        count += 1;
        println!("Call number {}", count);
    });
    println!("Called {} times in total", count);

    // Fn: the closure only reads
    let multiplier = 3;
    let result = sum_two_calls(|x| x * multiplier, 5);
    println!("sum_two_calls result: {}", result);

    // Ordinary functions can be passed in too
    fn double(x: i32) -> i32 {
        x * 2
    }
    let result2 = sum_two_calls(double, 10);
    println!("With an ordinary function: {}", result2);

    // An Fn closure also fits an FnOnce parameter (every Fn closure implements FnOnce)
    let greeting = String::from("Hi");
```

```
    consume_and_print(|| {
        format!("{}, world!", greeting) // Only reads greeting — it's Fn
    });
    // greeting survives, since the closure merely borrowed it
    println!("greeting is still here: {}", greeting);
}
```

6.5.4 Recap

- FnOnce, FnMut, Fn are **traits**, not types; fn is the function pointer type.
- The inheritance: Fn ⊂ FnMut ⊂ FnOnce (FnOnce accepts every closure).
- Accept closure parameters with `impl FnOnce()` / `impl FnMut()` / `impl Fn()`.
- FnMut parameters need mut.
- Design principle for closure-taking functions: **start with FnOnce**, switch to FnMut for repeated calls, and use Fn when calls must not require a mutable reference to the closure value.
- Function pointers automatically implement Fn + FnMut + FnOnce.

6.6 move Closures

6.6.1 Goal of This Episode

Learn to force a closure to capture outer variables by value with the move keyword, and understand when that fixes lifetime problems.

6.6.2 Concept

6.6.2.1 The Default Capture Behavior

Rust's closures are clever, automatically picking the "lightest" way to capture:

- Only reading a variable → capture by &T (borrow).
- Needing to modify → by &mut T (mutable borrow).
- Needing to consume → by T (move).

Usually that's great. But sometimes borrowing creates lifetime problems.

6.6.2.2 The Problem Scenario: Returning a Closure

Suppose you want a function that returns a closure:

```
fn make_greeter(name: String) -> impl Fn() {
    || println!("Hello, {}!", name) // Compile error!
}
```

This does not compile because the closure captures name by borrow (&name) by default, but name is a local variable of the function, discarded when the function ends. The borrow inside the closure becomes a dangling reference — our old friend from Chapter 4.

6.6.2.3 The move Keyword

Adding move solves it:

```rust
fn make_greeter(name: String) -> impl Fn() {
    move || println!("Hello, {}!", name)
}

fn main() {}
```

`move` tells Rust to capture every used outer variable **by value**. Here, the closure captures the `String` itself, so `name` now belongs to the closure; however the original scope ends, the closure keeps its `name`.

6.6.2.4 The Anonymous `struct` of a move Closure

Recall recent episodes — a closure is an anonymous `struct`. Without `move`, the `struct`'s fields may be references to outer variables (`&T` or `&mut T`); with `move`, the closure captures those variables **by value**:

```rust
    // Without move: the closure borrows name; the struct stores a reference
    let name = String::from("Alice");
    let greet = || println!("{}", name);
    // name stays usable, since the closure only borrows

    // With move: name is moved into the struct; the closure owns it
    let name = String::from("Alice");
    let greet = move || println!("{}", name);
    // name can't be used anymore; it's been moved into the closure
```

In this example, the captured variable is a `String`, so the closure owns the string and no longer borrows the local variable `name`. That is why it can be returned from the function safely.

But capturing by value does not turn a reference into an owned version of the data it points to. If the captured variable is itself a reference, the closure stores that reference unchanged:

```rust
fn make_printer<'a>(text: &'a str) -> impl Fn() + 'a {
    // text itself is an &'a str; move captures that reference by value
    move || println!("{}", text)
}

fn main() {
    let message = String::from("hello");
    let print = make_printer(&message);
    print();
}
```

Here `text` is an `&'a str`. Since shared references are `Copy`, `move` copies that reference value into the closure; it does not give the closure ownership of the string data. In the `struct` analogy, the closure's field is still `text: &'a str`. The `+ 'a` on the return type makes this lifetime relationship explicit: when `text` refers to a local string, the returned closure cannot be used after that string is dropped. In other words, `move` does not automatically make a closure `'static`.

6.6.2.5 `move` Doesn't Affect Which `Fn` `trait` the Closure Gets

A common confusion: a `move` closure is not automatically `FnOnce`!

`move` affects only **how it captures**, not **how it uses**:

```rust
fn main() {
    let name = String::from("Alice");
    let greet = move || println!("Hello, {}!", name);
    // name was moved into the closure, but the closure only "reads" name
    // So this closure is Fn, callable repeatedly
    greet();
    greet();
}
```

6.6.2.6 The `traits` Closures Implement Automatically

Until now, we've mostly treated closures as things you call. There wasn't a good place to ask a different ownership question: can the closure value itself be moved, copied, or `cloned`?

This episode is finally about ownership around closures, so this is the right place to answer that. Moving a closure value is allowed like moving other values, but whether it can be copied or `cloned` depends on the values it actually captures — much like a tuple: if every value stored inside is copyable, the whole is:

- All captured values implement `Copy` → the closure is `Copy` too.
- All captured values implement `Clone` → the closure is `Clone` too.
- The same holds for certain other `traits`.

```rust
fn main() {
    let x = 42;
    let f = move || x + 1; // x is i32 (Copy), so f is Copy too
    let g = f; // f was copied
    println!("{}", f()); // f is still usable
    println!("{}", g());
}
```

6.6.3 Example Code

```rust
// Returning a closure usually needs move
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}

fn make_counter(start: i32) -> impl FnMut() -> i32 {
    let mut count = start;
    move || {
        count += 1;
        count
    }
}

fn main() {
    // move gives the closure ownership of its captures, safe to return
    let add_five = make_adder(5);
    println!("10 + 5 = {}", add_five(10));
    println!("20 + 5 = {}", add_five(20));

    // move + FnMut: the closure owns count and modifies it each time
    let mut counter = make_counter(0);
```

```
    println!("Count: {}", counter());
    println!("Count: {}", counter());
    println!("Count: {}", counter());

    // move doesn't mean FnOnce
    let name = String::from("Bob");
    let greet = move || {
        println!("Hi, {}!", name); // Only reads name, so it's Fn
    };
    greet();
    greet(); // Repeatable calls — not FnOnce

    // A closure capturing Copy types can be Copied
    let factor = 3;
    let multiply = move |x: i32| x * factor;
    let multiply_copy = multiply; // Copied
    println!("multiply(4) = {}", multiply(4)); // The original still works
    println!("multiply_copy(4) = {}", multiply_copy(4));

    // A move closure capturing a String (non-Copy) can't be Copied
    let label = String::from("result");
    let show = move |x: i32| {
        println!("{}: {}", label, x);
    };
    // let show2 = show; // This would move show — not a Copy
    show(42);
}
```

6.6.4 Recap

- `move` forces the closure to capture every used outer variable by value. If a captured variable is itself a reference, it remains a reference and keeps its lifetime.
- Returning a closure often requires `move` so it captures local variables by value, but any references captured by value must still live long enough.
- `move` **does not affect** whether the closure is `FnOnce` / `FnMut` / `Fn` — that depends on **how it uses** the captured values.
- Whether a closure can `clone` / copy depends on whether all the values it actually captures are `Clone` / `Copy`.

6.7 Closure Methods on `Option` / `Result`

6.7.1 Goal of This Episode

Meet the common closure-taking methods on `Option` and `Result`, and feel how closures make code cleaner and more fluent.

6.7.2 Concept

In Chapter 5 we handled `Option` and `Result` with `match`, spelling out two arms every time. With closures learned, many operations shrink to one line.

6.7.2.1 Option's Closure Methods

The following methods are defined on Option<T>; the T in the signatures is Option<T>'s type parameter.

map — Transforming the Value inside Some

```
    // A method on Option<T>:
    // fn map<U>(self, f: impl FnOnce(T) -> U) -> Option<U>
    let x: Option<i32> = Some(5);
    let y = x.map(|v| v * 2); // Some(10)
```

On None, map does nothing and returns None as-is. No match needed.

and_then — Chaining (Possibly Failing) Operations

map's closure returns a plain value — but what if your transformation can itself return None? Use and_then:

```
    // A method on Option<T>:
    // fn and_then<U>(self, f: impl FnOnce(T) -> Option<U>) -> Option<U>
    let x: Option<i32> = Some(5);
    let y = x.and_then(|v| if v > 3 { Some(v * 2) } else { None });
```

and_then's closure returns an Option, avoiding the nested Option<Option<T>> problem. In fact, and_then equals map followed by flatten — map would produce Option<Option<U>>, and flatten squashes it into Option<U>. and_then does it in one step.

unwrap_or_else — a Closure Computing the Default

```
fn main() {
    // A method on Option<T>:
    // fn unwrap_or_else(self, f: impl FnOnce() -> T) -> T
    let x: Option<i32> = None;
    let y = x.unwrap_or_else(|| {
        println!("No value; computing a default...");
        42
    });
}
```

Unlike unwrap_or, unwrap_or_else computes its default **lazily** — the closure runs only when it's actually None.

filter — Conditional Filtering

```
    // A method on Option<T>:
    // fn filter(self, predicate: impl FnOnce(&T) -> bool) -> Option<T>
    let x: Option<i32> = Some(4);
    let y = x.filter(|v| v % 2 == 0); // Some(4), since 4 is even
    let z = x.filter(|v| v % 2 != 0); // None, since 4 isn't odd
```

6.7.2.2 Result's Closure Methods

Result has a similar set. The following are defined on Result<T, E>, where T is the Ok type and E the Err type.

map — Transforming the Ok Value

```
    // A method on Result<T, E>:
    // fn map<U>(self, f: impl FnOnce(T) -> U) -> Result<U, E>
    let r: Result<i32, String> = Ok(10);
    let doubled = r.map(|v| v * 2); // Ok(20)
```

`map_err` — Transforming the `Err` Value

The mirror of `map` — `map` acts on `Ok` and leaves `Err` alone; `map_err` acts on `Err` and leaves `Ok` alone.

```
    // A method on Result<T, E>:
    // fn map_err<F>(self, f: impl FnOnce(E) -> F) -> Result<T, F>
    let r: Result<i32, String> = Err(String::from("not found"));
    let r2 = r.map_err(|e| format!("Error: {}", e));
```

`and_then` — Chaining

```
    // A method on Result<T, E>:
    // fn and_then<U>(self, f: impl FnOnce(T) -> Result<U, E>) -> Result<U, E>
    let r: Result<i32, String> = Ok(5);
    let r2 = r.and_then(|v| {
        if v > 0 {
            Ok(v * 10)
        } else {
            Err(String::from("Must be positive"))
        }
    });
```

As with `Option`, `and_then` equals `map` then `flatten`.

`unwrap_or_else` — Computing a Default from the `Err`

```
fn main() {
    // A method on Result<T, E>:
    // fn unwrap_or_else(self, f: impl FnOnce(E) -> T) -> T
    let r: Result<i32, String> = Err(String::from("oops"));
    let value = r.unwrap_or_else(|e| {
        println!("An error occurred: {}; using the default", e);
        0
    });
}
```

6.7.2.3 Comparison with `match`

With `match`:

```
    let result = match opt {
        Some(v) => Some(v * 2),
        None => None,
    };
```

With the closure method:

```
    let result = opt.map(|v| v * 2);
```

One line, and the intent is clearer — "transform the value inside the `Some`."

6.7.3 Example Code

```
fn parse_and_double(input: &str) -> Result<i32, String> {
    input
        .parse::<i32>()
        .map_err(|e| format!("Parse failed: {}", e))
        .and_then(|n| {
            if n >= 0 {
                Ok(n * 2)
            } else {
                Err(String::from("Negative numbers not accepted"))
            }
        })
}

fn find_even(numbers: &[i32]) -> Option<i32> {
    for n in numbers {
        if n % 2 == 0 {
            return Some(*n);
        }
    }
    None
}

fn main() {
    // Option's map
    let maybe_num: Option<i32> = Some(21);
    let doubled = maybe_num.map(|n| n * 2);
    println!("map: {:?}", doubled);

    // Option's and_then
    let result = maybe_num.and_then(|n| {
        if n > 10 { Some(n - 10) } else { None }
    });
    println!("and_then: {:?}", result);

    // Option's filter
    let even = maybe_num.filter(|n| n % 2 == 0);
    println!("filter(even): {:?}", even);

    // Option's unwrap_or_else
    let none_value: Option<i32> = None;
    let default = none_value.unwrap_or_else(|| {
        println!("Computing a default...");
        99
    });
    println!("unwrap_or_else: {}", default);

    // Chained Result operations
    println!("\n--- Chained Result operations ---");
    let good = parse_and_double("21");
    println!("parse_and_double(\"21\") = {:?}", good);

    let bad_parse = parse_and_double("abc");
    println!("parse_and_double(\"abc\") = {:?}", bad_parse);
```

```
    let negative = parse_and_double("-5");
    println!("parse_and_double(\"-5\") = {:?}", negative);

    // Result's unwrap_or_else
    let safe_value = parse_and_double("oops").unwrap_or_else(|e| {
        println!("Handling the error: {}", e);
        0
    });
    println!("Safely obtained value: {}", safe_value);

    // Combining Option methods
    println!("\n--- Chained Option operations ---");
    let numbers = vec![1, 3, 5, 8, 11];
    let result = find_even(&numbers)
        .filter(|n| *n > 5)
        .map(|n| n * 10);
    println!("First even number, times 10 only if > 5: {:?}", result);
}
```

6.7.4 Recap

- `Option`'s and `Result`'s `map` transform the inner value; nothing runs on `None` / `Err`.
- `and_then` is for closures that themselves return `Option` / `Result`, avoiding nesting.
- `unwrap_or_else` computes the default lazily — the closure runs only on `None` / `Err`.
- `Option`'s `filter` keeps the `Some` or turns it into `None` based on a condition.
- `Result`'s `map_err` converts the error type — handy in error-handling chains.
- These methods chain, reading far cleaner than stacked `match`es.
- You may have noticed: the type signature alone tells you what a method does (`Option<T>`'s `map` takes `FnOnce(T) -> U`, returns `Option<U>`). A hallmark of functional programming — the types are the documentation.

6.8 The `Iterator` `trait`

6.8.1 Goal of This Episode

Meet the heart of the `Iterator` `trait` — implement just the `next` method, and dozens of useful methods come free.

6.8.2 Concept

6.8.2.1 The Definition of `Iterator`

The core of the `Iterator` `trait` couldn't be simpler:

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

That's it. One required method, `next`, which on each call returns:

- `Some(value)` — there's a next element.

- None — the iteration is over.

Remember associated types from Chapter 5? type Item is one — "the element type this iterator produces."

6.8.2.2 Calling next by Hand

You can call .next() manually to fetch elements one at a time:

```
fn main() {
    let v = vec![10, 20, 30];
    let mut iter = v.iter();

    println!("{:?}", iter.next()); // Some(&10)
    println!("{:?}", iter.next()); // Some(&20)
    println!("{:?}", iter.next()); // Some(&30)
    println!("{:?}", iter.next()); // None
}
```

Note iter must be mut, since every .next() advances internal state.

6.8.2.3 Implement Only next; the Rest Come Free

The Iterator trait supplies a wealth of **default implementations** (remember Chapter 5?). Since every iteration operation boils down to "keep calling next until None," implementing next alone makes dozens of methods — map, filter, count, sum, and more — automatically available.

6.8.2.4 A Custom Iterator

Let's build our own iterator. Say we want a "countdown timer":

```
struct Countdown {
    value: i32,
}

impl Iterator for Countdown {
    type Item = i32;

    fn next(&mut self) -> Option<i32> {
        if self.value > 0 {
            let current = self.value;
            self.value -= 1;
            Some(current)
        } else {
            None
        }
    }
}
```

With next implemented, map, filter, sum, collect, and dozens more become automatically available. The coming episodes cover them one by one.

6.8.2.5 The Standard Library's Iterator Factories

The standard library offers convenient functions for creating iterators:

- std::iter::repeat(value) — repeats one value endlessly

- `std::iter::from_fn(closure)` — a closure decides what each `.next()` returns.

```
use std::iter;

fn main() {
    // Endlessly producing 42
    let mut repeater = iter::repeat(42);
    println!("{:?}", repeater.next()); // Some(42)
    println!("{:?}", repeater.next()); // Some(42) (never None)

    // Producing increasing numbers with a closure
    let mut n = 0;
    let mut counter = iter::from_fn(move || {
        n += 1;
        Some(n)
    });
    println!("{:?}", counter.next()); // Some(1)
    println!("{:?}", counter.next()); // Some(2)
}
```

Note that iterators from `repeat` and `from_fn` may be **infinite** — never returning `None`. Episode 15 explores this property in depth.

6.8.3 Example Code

```
use std::iter;

// A custom iterator: the Fibonacci sequence (infinite!)
struct Fibonacci {
    a: u64,
    b: u64,
}

impl Fibonacci {
    fn new() -> Fibonacci {
        Fibonacci { a: 0, b: 1 }
    }
}

impl Iterator for Fibonacci {
    type Item = u64;

    fn next(&mut self) -> Option<u64> {
        let current = self.a;
        let new_b = self.a + self.b;
        self.a = self.b;
        self.b = new_b;
        Some(current) // Never returns None
    }
}

fn main() {
    // Calling .next() of a Vec's .iter() by hand
    let names = vec!["Alice", "Bob", "Charlie"];
    let mut name_iter = names.iter();
```

```
    println!("First: {:?}", name_iter.next());
    println!("Second: {:?}", name_iter.next());
    println!("Third: {:?}", name_iter.next());
    println!("Done: {:?}", name_iter.next());

    // A custom Iterator: Fibonacci (calling next by hand)
    println!("\nFibonacci:");
    let mut fib = Fibonacci::new();
    println!("{:?}", fib.next()); // Some(0)
    println!("{:?}", fib.next()); // Some(1)
    println!("{:?}", fib.next()); // Some(1)
    println!("{:?}", fib.next()); // Some(2)
    println!("{:?}", fib.next()); // Some(3)
    println!("{:?}", fib.next()); // Some(5)
    // Never None — an infinite iterator

    // std::iter::repeat: endless repetition
    let mut threes = iter::repeat(3);
    println!("\nrepeat(3):");
    println!("{:?}", threes.next()); // Some(3)
    println!("{:?}", threes.next()); // Some(3)
    println!("{:?}", threes.next()); // Some(3) (never None)

    // std::iter::from_fn: a closure controls the output
    let mut n = 0;
    let mut squares = iter::from_fn(|| {
        n += 1;
        if n <= 3 {
            Some(n * n)
        } else {
            None
        }
    });
    println!("\nfrom_fn (the first 3 squares):");
    println!("{:?}", squares.next()); // Some(1)
    println!("{:?}", squares.next()); // Some(4)
    println!("{:?}", squares.next()); // Some(9)
    println!("{:?}", squares.next()); // None
}
```

6.8.4 Recap

- The core of the `Iterator` trait is `next(&mut self) -> Option<Self::Item>`.
- Implement `.next()` alone and receive dozens of default implementations free (covered in coming episodes).
- Implementing `Iterator` for your own type is easy — define `type Item` and `next`.
- `std::iter::repeat(value)` builds an endlessly repeating iterator.
- `std::iter::from_fn(closure)` controls each produced value with a closure.
- Iterators may be infinite (never returning `None`).

6.9 What `for` Loops Really Are

6.9.1 Goal of This Episode

Unmask the `for` loop, and understand how it works through `IntoIterator` + `while let`.

6.9.2 Concept

6.9.2.1 `for` Loops Aren't Magic

We've been using `for` loops since Chapter 1:

```
fn main() {
    let v = vec![1, 2, 3];
    for x in v {
        println!("{}", x);
    }
}
```

Looks simple, right? But what's actually happening underneath?

6.9.2.2 A Conceptual Rewrite

Conceptually, you can think of the `for` loop above as this:

```
fn main() {
    let v = vec![1, 2, 3];
    let mut iter = v.into_iter();
    while let Some(x) = iter.next() {
        println!("{}", x);
    }
}
```

Three steps:

1. Call `v.into_iter()` to turn `v` into an iterator.
2. Call `iter.next()` repeatedly.
3. Destructure with `while let Some(x)` (remember Chapter 3's `while let`?), ending when `None` arrives.

6.9.2.3 The `IntoIterator` `trait`

`IntoIterator` is a `trait` defining "how to turn oneself into an iterator":

```
trait IntoIterator {
    type Item;
    type IntoIter: Iterator<Item = Self::Item>;
    fn into_iter(self) -> Self::IntoIter;
}
```

Any type implementing `IntoIterator` works with `for` loops. `Vec`, arrays, a string slice's `.chars()`... all thanks to implementing this `trait`.

6.9.2.4 `Iterator` Implements `IntoIterator` Too

A very convenient design: every `Iterator` automatically implements `IntoIterator` (its `into_iter()` simply returns itself). So an iterator can be thrown straight into `for`:

```rust
fn main() {
    let v = vec![1, 2, 3];
    let iter = v.iter(); // This is an Iterator
    for x in iter {      // Iterators implement IntoIterator too
        println!("{}", x);
    }
}
```

6.9.3 Example Code

```rust
fn main() {
    // A regular for loop
    let fruits = vec!["apple", "banana", "orange"];
    println!("--- The for loop ---");
    for fruit in fruits {
        println!("Fruit: {}", fruit);
    }

    // Manually rewritten as while let (conceptually equivalent)
    let fruits = vec!["apple", "banana", "orange"];
    println!("\n--- Rewritten by hand ---");
    let mut iter = fruits.into_iter();
    while let Some(fruit) = iter.next() {
        println!("Fruit: {}", fruit);
    }

    // A custom iterator (Iterator auto-implements IntoIterator, so for works)
    println!("\n--- A custom Iterator ---");
    let countdown = Countdown { value: 5 };
    for n in countdown {
        print!("{} ", n);
    }
    println!("Liftoff!");

    // An iterator itself can go into for
    println!("\n--- An Iterator straight into for ---");
    let numbers = vec![10, 20, 30, 40, 50];
    for n in numbers.iter() {
        if *n > 20 {
            println!("Greater than 20: {}", n);
        }
    }

    // Ranges implement IntoIterator too
    println!("\n--- Range ---");
    for i in 1..=5 {
        print!("{} ", i);
    }
    println!();
}

// A custom iterator
struct Countdown {
    value: i32,
```

```
}

impl Iterator for Countdown {
    type Item = i32;

    fn next(&mut self) -> Option<i32> {
        if self.value > 0 {
            let current = self.value;
            self.value -= 1;
            Some(current)
        } else {
            None
        }
    }
}
```

6.9.4 Recap

- `for x in v` is shorthand; conceptually, it works like `v.into_iter()` + `while let Some(x) = iter.next()`.
- The `IntoIterator` `trait` defines "how to turn oneself into an iterator."
- Any type implementing `IntoIterator` works with `for` loops.
- Every `Iterator` implements `IntoIterator` automatically.
- Writing `for i in 1..5` or `for i in 1..=5` works precisely because ranges implement `IntoIterator`.

6.10 `into_iter` / `iter_mut` / `iter`

6.10.1 Goal of This Episode

Get the three iteration modes straight — consuming, mutable borrowing, and borrowing — and their relationship to the ownership system.

6.10.2 Concept

6.10.2.1 Three Ways to Iterate

We touched earlier on the difference between `for x in v` and `for x in &v`. Today, we'll complete the picture by introducing `Vec`'s three iteration methods:

Method	Produced type	Meaning	Is the `Vec` still usable after?
`.into_iter()`	`T`	Consumes the whole collection	✗ No
`.iter_mut()`	`&mut T`	Mutably borrows each element	✓ Yes (now modified)
`.iter()`	`&T`	Borrows each element	✓ Yes

6.10.2.2 `.into_iter()` — Taking Everything

```
    let names = vec![String::from("Alice"), String::from("Bob")];
    for name in names.into_iter() {
        println!("{}", name); // name is a String (owned)
```

```
    }
    println!("{:?}", names); // Compile error! names was consumed
```

`.into_iter()` hands over each element's ownership. The collection itself is consumed, unusable afterward.

In fact, `for name in names` equals `for name in names.into_iter()`.

6.10.2.3 `.iter_mut()` — Borrowing to Modify

```
fn main() {
    let mut scores = vec![60, 70, 80];
    for score in scores.iter_mut() {
        *score += 10; // score is a &mut i32
    }
    println!("{:?}", scores); // [70, 80, 90]
}
```

`.iter_mut()` returns an iterator of `&mut T`, letting you modify each element in place.

6.10.2.4 `.iter()` — Just Looking

```
fn main() {
    let names = vec![String::from("Alice"), String::from("Bob")];
    for name in names.iter() {
        println!("{}", name); // name is a &String
    }
    println!("names is still here: {:?}", names); // Fine — only borrowed
}
```

`.iter()` returns an iterator of `&T`. The collection is untouched, still there afterward.

6.10.2.5 The Correspondence

These three methods map onto the three ownership operations from Chapter 4:

Ownership concept	Iteration method	for shorthand
T (moved ownership)	`.into_iter()`	`for x in v`
`&mut T` (mutable borrow)	`.iter_mut()`	`for x in &mut v`
`&T` (borrow)	`.iter()`	`for x in &v`

6.10.2.6 The `IntoIterator` behind It

Last episode showed `for x in something` calls `something.into_iter()`. So how do the three `for` forms work?

Because `Vec<T>`, `&mut Vec<T>`, and `&Vec<T>` each implement `IntoIterator`:

```
impl<T> IntoIterator for Vec<T> {
    type Item = T;
    fn into_iter(self) -> ... { /* Consumes the Vec, producing T */ }
}

impl<'a, T> IntoIterator for &'a mut Vec<T> {
    type Item = &'a mut T;
```

```
    fn into_iter(self) -> ... { /* Same as .iter_mut(), producing &mut T */ }
}

impl<'a, T> IntoIterator for &'a Vec<T> {
    type Item = &'a T;
    fn into_iter(self) -> ... { /* Same as .iter(), producing &T */ }
}
```

So `for x in v`, `for x in &mut v`, and `for x in &v` use the `IntoIterator` implementations for `Vec<T>`, `&mut Vec<T>`, and `&Vec<T>`, respectively, ultimately yielding `T`, `&mut T`, and `&T`.

Most collection types (`Vec`, arrays...) follow this pattern — implementing `IntoIterator` three times, for themselves, `&mut self`, and `&self`.

6.10.2.7 Which to Choose?

- Taking ownership of the elements → `.into_iter()`.
- Modifying in place → `.iter_mut()`.
- Only reading → `.iter()` (most common).

Choose the iteration method that gives you only the access you need.

6.10.3 Example Code

```
fn main() {
    // .into_iter() — consuming ownership
    let words = vec![
        String::from("hello"),
        String::from("world"),
    ];
    println!("--- .into_iter() (consuming) ---");
    for word in words.into_iter() {
        println!("Received: {}", word); // word is a String (owned)
    }
    // println!("{:?}", words); // Compile error! words was consumed

    // .iter_mut() — mutable borrowing, in-place modification
    let mut prices = vec![100, 200, 300];
    println!("\n--- .iter_mut() (modifying) ---");
    println!("Before the discount: {:?}", prices);
    for price in prices.iter_mut() {
        *price = *price * 8 / 10; // 20% off
    }
    println!("After the discount: {:?}", prices);

    // .iter() — borrowing
    let animals = vec![
        String::from("cat"),
        String::from("dog"),
        String::from("rabbit"),
    ];

    println!("\n--- .iter() (borrowing) ---");
    for animal in animals.iter() {
```

```
        println!("Animal: {}", animal);
    }
    println!("animals is still here: {:?}", animals);

    // The shorthand correspondences
    println!("\n--- The shorthands ---");
    let owned = vec![1, 2, 3];

    // for x in owned equals for x in owned.into_iter()
    for x in owned {
        print!("{} ", x);
    }
    println!("← owned (consuming)");
    // owned is no longer usable

    let mut mutable = vec![1, 2, 3];
    // for x in &mut mutable equals for x in mutable.iter_mut()
    for x in &mut mutable {
        *x *= 10;
    }
    println!("{:?} ← &mut mutable (mutable borrowing)", mutable);

    let borrowed = vec![1, 2, 3];
    // for x in &borrowed equals for x in borrowed.iter()
    for x in &borrowed {
        print!("{} ", x);
    }
    println!("← &borrowed (borrowing)");
    println!("borrowed is still here: {:?}", borrowed);
}
```

6.10.4 Recap

- `.into_iter()` produces `T`, consuming the whole collection and taking ownership.
- `.iter_mut()` produces `&mut T`, allowing in-place modification.
- `.iter()` produces `&T`, borrowing elements; the collection is unaffected.
- `for x in v` = `.into_iter()`, `for x in &mut v` = `.iter_mut()`, `for x in &v` = `.iter()`.
- Choose the iteration method that gives you only the access you need — `.into_iter()` to consume, `.iter_mut()` to modify, `.iter()` to read.

6.11 Collecting

6.11.1 Goal of This Episode

Learn to collect an iterator into various collection types with `.collect()`.

6.11.2 Concept

We wouldn't ordinarily spend this many episodes introducing methods, but iterators are simply too important — among the tools most used in everyday Rust — so the next few episodes take their time. Even then, plenty of methods will inevitably be missed. When you need more,

consult the official documentation's `Iterator trait` page.

6.11.2.1 `.collect()` — the `Iterator`'s Terminus

We've been building iterators, but an iterator is itself **lazy** (Episode 15 goes deep on this) — nothing actually runs until someone "pulls" on it. `.collect()` is the most common pull: gather all the iterator's elements into a collection.

```
    let v: Vec<i32> = (1..=5).into_iter().collect();
```

6.11.2.2 Collecting into a `String`

`.collect()` isn't limited to `Vec`. If the iterator produces `chars` or `&str`s, it can collect straight into a `String`:

```
fn main() {
    let chars = vec!['R', 'u', 's', 't'];
    let word: String = chars.into_iter().collect();
    println!("{}", word); // "Rust"
}
```

6.11.2.3 `.last()` — Taking the Final Element

`.last()` consumes the whole iterator and returns the final element (an `Option<T>`):

```
fn main() {
    let v = vec![10, 20, 30];
    let last = v.iter().last();
    println!("{:?}", last); // Some(&30)
}
```

Depending on the iterator, `.last()` may need to walk through every remaining element. Some iterators can override the default behavior with a more efficient implementation.

6.11.3 Example Code

```
fn main() {
    // Basic collect — Range into Vec
    let numbers: Vec<i32> = (1..=10).into_iter().collect();
    println!("1 through 10: {:?}", numbers);

    // The turbofish syntax
    let numbers2 = (1..=5).into_iter().collect::<Vec<i32>>();
    println!("turbofish: {:?}", numbers2);

    // Collecting into a String
    let greeting: String = vec!['h', 'e', 'l', 'l', 'o'].into_iter().collect();
    println!("String: {}", greeting);

    // .last()
    let last_num = (1..=100).into_iter().last();
    println!("\nThe last of 1..=100: {:?}", last_num);

    let empty: Vec<i32> = vec![];
    let last_empty = empty.iter().last();
    println!("last of an empty Vec: {:?}", last_empty);
```

```
}
```

6.11.4 Recap

- `.collect()` gathers an iterator's elements into a target collection type.
- Tell Rust the target type with an annotation `let v: Vec<i32>` or the turbofish `.collect::<Vec<i32>>()`.
- Collection targets include `Vec`, `String`, and many other types.
- `.last()` consumes the whole iterator, returning the final element wrapped in `Some`.

6.12 Aggregation

6.12.1 Goal of This Episode

Learn to "fold" an entire sequence into one value with the iterator's aggregation methods.

6.12.2 Concept

6.12.2.1 What Is Aggregation?

Recent episodes covered creating iterators and `collecting` them into collections. But sometimes you don't want a collection — you want a **single value**: a sum, a maximum, a count... That's aggregation.

6.12.2.2 `.count()` — How Many Are There

```
    let names = vec!["Alice", "Bob", "Charlie"];
    let count = names.iter().count(); // 3
```

6.12.2.3 `.sum()` and `.product()`

```
    let total: i32 = (1..=10).into_iter().sum();         // 55
    let factorial: i64 = (1..=10).into_iter().product(); // 3628800
```

Like `.collect()`, `.sum()` and `.product()` need the return type specified — usually via a type annotation.

6.12.2.4 `.min()` and `.max()`

```
    let v = vec![3, 1, 4, 1, 5, 9, 2, 6];
    let smallest = v.iter().min(); // Some(&1)
    let largest = v.iter().max();  // Some(&9)
```

They return `Option`, since the iterator might be empty (returning `None` if so).

6.12.2.5 `.fold(init, f)` — the Most General Aggregation

`fold` is the "boss" of all aggregation methods. Its type:

```
fn fold<B>(self, init: B, f: impl FnMut(B, Self::Item) -> B) -> B;
```

It takes an initial value `init` (of type `B`) and a closure; each step combines the "accumulated value" and the "current element" into a new accumulated value:

```
    let sum = (1..=5).into_iter().fold(0, |acc, x| acc + x);
    // Steps: 0+1=1, 1+2=3, 3+3=6, 6+4=10, 10+5=15
```

In fact, every other method in this episode can be built from `fold`:

```
    // count = fold from 0, +1 each step
    let count = (1..=5).into_iter().fold(0, |acc, _x| acc + 1);

    // sum = fold from 0, adding each element
    let sum = (1..=5).into_iter().fold(0, |acc, x| acc + x);

    // product = fold from 1, multiplying by each element
    let product = (1..=5).into_iter().fold(1, |acc, x| acc * x);

    // min / max are left to reduce below — fold makes them awkward
```

`fold` can do more flexible things. String numbers together? Track multiple values at once? All possible:

```
    let text = (1..=5).into_iter().fold(String::new(), |mut acc, x| {
        if !acc.is_empty() {
            acc.push_str(", ");
        }
        acc.push_str(&x.to_string());
        acc
    });
    // "1, 2, 3, 4, 5"
```

6.12.2.6 `.reduce(f)` — `fold` without an Initial Value

`reduce` resembles `fold`, but uses the first element as the initial value:

```
    let product = vec![2, 3, 4].into_iter().reduce(|acc, x| acc * x);
    // Some(24): 2*3=6, 6*4=24
```

Since there may be no first element (an empty iterator), `reduce` returns an `Option`.

Implementing `min` and `max` with `reduce` is very natural:

```
    let min = vec![3, 1, 4, 1, 5].into_iter()
        .reduce(|a, b| if a < b { a } else { b });
    // Some(1)

    let max = vec![3, 1, 4, 1, 5].into_iter()
        .reduce(|a, b| if a > b { a } else { b });
    // Some(5)
```

Since `reduce` itself returns `Option`, an empty iterator automatically gets `None` — whereas `fold` requires special handling for the empty case.

6.12.3 Example Code

```
fn main() {
    let scores = vec![85, 92, 78, 95, 88, 76, 91];

    // .count()
```

```
    let total = scores.iter().count();
    println!("{} scores in total", total);

    // .sum()
    let sum: i32 = scores.iter().sum();
    println!("Total: {}", sum);

    // .min() / .max()
    let min = scores.iter().min();
    let max = scores.iter().max();
    println!("Lowest: {:?}, highest: {:?}", min, max);

    // .product()
    let factorial: i64 = (1..=10).into_iter().product();
    println!("\n10! = {}", factorial);

    // .fold() — computing an average
    let (count2, sum2) = scores.iter().fold((0, 0), |(c, s), &score| {
        (c + 1, s + score)
    });
    println!("\nAverage via fold: {} / {} = {}", sum2, count2, sum2 / count2);

    // .fold() — stringing numbers together
    let nums = vec![1, 2, 3, 4, 5];
    let formatted = nums.iter().fold(String::new(), |mut acc, &n| {
        if !acc.is_empty() {
            acc.push_str(" → ");
        }
        acc.push_str(&n.to_string());
        acc
    });
    println!("Joined: {}", formatted);

    // .reduce() — finding the longest string
    let words = vec!["cat", "elephant", "dog", "hippopotamus"];
    let longest = words
        .iter()
        .reduce(|a, b| if a.len() >= b.len() { a } else { b });
    println!("\nThe longest word: {:?}", longest);

    // .reduce() returns Option (the empty-iterator case)
    let empty: Vec<i32> = vec![];
    let result = empty.into_iter().reduce(|a, b| a + b);
    println!("reduce of an empty Vec: {:?}", result);
}
```

6.12.4 Recap

- `.count()` counts the elements.
- `.sum()` and `.product()` compute total and product; annotate the return type.
- `.min()` and `.max()` return `Option`, since the iterator may be empty.
- `.fold(init, |acc, x| ...)` is the most general aggregation — accumulating step by step from an initial value and a closure.

- `.reduce(|acc, x| ...)` is like `fold` but seeds with the first element, returning `Option`.
- Aggregation methods consume the whole iterator, producing one single value.

6.13 Combining and Slicing

6.13.1 Goal of This Episode

Learn to combine and trim iterators with `zip`, `enumerate`, `chain`, `take`, `skip`, and `flatten`.

6.13.2 Concept

6.13.2.1 `.zip(iter)` — Pairing Two Iterators

`zip` pairs two iterators "zipper-style," producing tuples:

```
let names = vec!["Alice", "Bob", "Charlie"];
let scores = vec![90, 85, 92];
let paired: Vec<_> = names.iter().zip(scores.iter()).collect();
// [("Alice", 90), ("Bob", 85), ("Charlie", 92)]
```

If the two iterators differ in length, `zip` stops when the shorter one ends.

6.13.2.2 `.enumerate()` — Bringing the Index Along

```
let names = vec!["Alice", "Bob", "Charlie"];
for (i, name) in names.iter().enumerate() {
    println!("Number {}: {}", i, name);
}
```

`enumerate` wraps each element in an `(index, element)` tuple, indices starting at 0.

6.13.2.3 `.chain(iter)` — Joining Two Iterators

`chain` connects two iterators end to end:

```
let first = vec![1, 2, 3];
let second = vec![4, 5, 6];
let all: Vec<i32> = first.into_iter().chain(second.into_iter()).collect();
// [1, 2, 3, 4, 5, 6]
```

6.13.2.4 `.take(n)` — Only the First n

```
let first_three: Vec<i32> = (1..=100).into_iter().take(3).collect();
// [1, 2, 3]
```

6.13.2.5 `.skip(n)` — Skipping the First n

```
let after_skip: Vec<i32> = (1..=10).into_iter().skip(7).collect();
// [8, 9, 10]
```

6.13.2.6 `.flatten()` — Squashing Nested Structures

If an iterator's elements are themselves iterators (or `Options`, `Vecs`, etc.), `flatten` squashes one layer:

```
    let nested = vec![vec![1, 2], vec![3, 4], vec![5]];
    let flat: Vec<i32> = nested.into_iter().flatten().collect();
    // [1, 2, 3, 4, 5]
```

`Option` can be `flattened` too — `Some(value)` is taken out; `None` is ignored:

```
    let options = vec![Some(1), None, Some(3), None, Some(5)];
    let values: Vec<i32> = options.into_iter().flatten().collect();
    // [1, 3, 5]
```

That works because `Option` also implements `IntoIterator`.

6.13.3 Example Code

```
fn main() {
    // zip — pairing names with scores
    let students = vec!["Ming", "Hua", "Mei"];
    let grades = vec![88, 95, 72];
    println!("--- zip ---");
    for (name, grade) in students.iter().zip(grades.iter()) {
        println!("{}: {} points", name, grade);
    }

    // enumerate — with indices
    println!("\n--- enumerate ---");
    let fruits = vec!["apple", "banana", "cherry"];
    for (i, fruit) in fruits.iter().enumerate() {
        println!("Number {}: {}", i + 1, fruit);
    }

    // chain — joining two Vecs
    let morning = vec!["meeting", "writing the report"];
    let afternoon = vec!["coding", "code review"];
    let all_tasks: Vec<&&str> = morning.iter().chain(afternoon.iter()).collect();
    println!("\nToday's schedule: {:?}", all_tasks);

    // take and skip
    let numbers: Vec<i32> = (1..=20).into_iter().collect();
    let first_five: Vec<&i32> = numbers.iter().take(5).collect();
    let last_five: Vec<&i32> = numbers.iter().skip(15).collect();
    println!("\nFirst 5: {:?}", first_five);
    println!("After skipping 15: {:?}", last_five);

    // take + skip combined: the middle stretch
    let middle: Vec<&i32> = numbers.iter().skip(5).take(5).collect();
    println!("Numbers 6~10: {:?}", middle);

    // flatten — squashing a nested Vec
    let matrix = vec![
        vec![1, 2, 3],
        vec![4, 5, 6],
        vec![7, 8, 9],
    ];
    let flat: Vec<i32> = matrix.into_iter().flatten().collect();
    println!("\nFlattened matrix: {:?}", flat);
```

```
    // flatten — filtering Options
    let maybe_values = vec![Some(10), None, Some(30), None, Some(50)];
    let real_values: Vec<i32> = maybe_values.into_iter().flatten().collect();
    println!("The ones with values: {:?}", real_values);

    // zip + map combined — the iterator's map arrives next episode
    println!("\n--- zip + map ---");
    let prices = vec![100, 200, 300];
    let quantities = vec![2, 1, 4];
    let grand_total: i32 = prices.iter()
        .zip(quantities.iter())
        .map(|(p, q)| p * q)
        .sum();
    println!("Grand total: {}", grand_total);
}
```

6.13.4 Recap

- `.zip(iter)` pairs two iterators into tuples, going by the shorter one.
- `.enumerate()` attaches a 0-based index to each element.
- `.chain(iter)` joins two iterators end to end.
- `.take(n)` keeps only the first n elements; `.skip(n)` skips the first n.
- `.flatten()` squashes one layer of nesting (`Vec<Vec<T>>` → `Vec<T>`; works on `Option` too).
- These methods combine freely into powerful data-processing pipelines.

6.14 Transforming and Filtering

6.14.1 Goal of This Episode

Learn the iterator's most-used transformation and filtering methods, and how chained calls build powerful data pipelines.

6.14.2 Concept

6.14.2.1 `.map(f)` — Transforming Each Element

`map` applies a closure to each element, producing transformed new elements:

```
    let doubled: Vec<i32> = vec![1, 2, 3].iter().map(|x| x * 2).collect();
    // [2, 4, 6]
```

Careful! `.iter()` produces `&T`, so the closure's parameter is `&i32`. If you'd rather not deal with references, pair it with `.copied()` (coming right up).

6.14.2.2 `.flat_map(f)` — `map` + `flatten`

`flat_map` equals `map` followed by `flatten` (last episode's). Each element becomes an iterator via the closure, and everything gets squashed flat:

```
    let words = vec!["abc", "de", "f"];
    let chars: Vec<char> = words.iter().flat_map(|s| s.chars()).collect();
```

```
    // ['a', 'b', 'c', 'd', 'e', 'f']
```

Remember `and_then` on `Option` and `Result` from Episode 7? What `flat_map` does on iterators is essentially the same — "transform, and since the result is itself a container, flatten."

6.14.2.3 `.filter(pred)` — Filtering Elements

`filter` keeps only the elements for which the closure returns `true`:

```
    let numbers = vec![1, 2, 3, 4, 5];
    let evens: Vec<&i32> = numbers.iter().filter(|&&x| x % 2 == 0).collect();
    // [&2, &4]
```

`filter`'s closure receives `&&T` (`.iter()` already gives `&T`, and `filter` borrows once more, making `&&T`). This trips up beginners regularly, but it becomes second nature with practice.

6.14.2.4 `.copied()` and `.cloned()`

When an iterator produces references (`&T`) but you want values (`T`), these two methods copy each element out:

- `.copied()` — requires `T: Copy`; copies each `&T` into a `T`.
- `.cloned()` — requires `T: Clone`; calls `.clone()` on each `&T` to get a `T`.

```
    let numbers = vec![1, 2, 3];
    let owned: Vec<i32> = numbers.iter().copied().collect();
    // From &i32 to i32
```

`.copied()` often pairs with `.filter()`, dodging the `&&T` annoyance:

```
    let evens: Vec<i32> = vec![1, 2, 3, 4, 5]
        .iter()
        .copied()
        .filter(|x| x % 2 == 0)
        .collect();
    // [2, 4] — much cleaner!
```

6.14.2.5 `.rev()` — Reversing the Iteration Order

```
    let reversed: Vec<i32> = (1..=5).into_iter().rev().collect();
    // [5, 4, 3, 2, 1]
```

`.rev()` requires the iterator to implement the `DoubleEndedIterator` `trait` — meaning it can take elements from both ends. `Vec`, arrays, and the like support it, but iterators from `from_fn` don't (no concept of a "tail end").

6.14.2.6 The Power of Chaining

`Iterator` methods chain freely into data-processing pipelines:

```
    let result: Vec<String> = names
        .iter()
        .enumerate()
        .filter(|(_, name)| name.len() > 3)
        .map(|(i, name)| format!("#{}: {}", i + 1, name))
        .collect();
```

Each step does one small thing; strung together, they accomplish very complex operations. And because iterators are lazy (next episode), no extra `Vec`s materialize along the way.

6.14.3 Example Code

```
fn main() {
    let scores = vec![55, 82, 91, 47, 73, 88, 69, 95];

    // map — 5 bonus points per score (a curve adjustment)
    let adjusted: Vec<i32> = scores.iter().map(|s| s + 5).collect();
    println!("After the bonus: {:?}", adjusted);

    // flat_map — splitting each word into characters
    let words = vec!["Rust", "rocks"];
    let all_chars: Vec<char> = words.iter().flat_map(|w| w.chars()).collect();
    println!("All the characters: {:?}", all_chars);

    // flat_map resembling and_then — keep successful parses, drop failures
    let inputs = vec!["42", "not_a_number", "7"];
    let parsed: Vec<i32> = inputs.iter().flat_map(|s| s.parse::<i32>()).collect();
    println!("Successfully parsed: {:?}", parsed);

    // filter — sifting out the passing scores
    let passing: Vec<i32> = scores.iter().copied().filter(|&s| s >= 60).collect();
    println!("Passing: {:?}", passing);

    // copied — from &i32 to i32
    let max_score: Option<i32> = scores.iter().copied().max();
    println!("\nHighest score: {:?}", max_score);

    // cloned — from &String to String
    let names = vec![String::from("Alice"), String::from("Bob")];
    let cloned_names: Vec<String> = names.iter().cloned().collect();
    println!("cloned: {:?}", cloned_names);
    println!("The originals remain: {:?}", names);

    // rev — reversing
    let countdown: Vec<i32> = (1..=5).into_iter().rev().collect();
    println!("\nCountdown: {:?}", countdown);

    // Chained combinations
    println!("\n--- Chained combinations ---");
    let long_words: Vec<&str> = vec!["hi", "hello", "hey", "howdy", "greetings"]
        .into_iter()
        .filter(|w| w.len() >= 4)
        .collect();
    println!("4+ letters: {:?}", long_words);

    // filter + map combined
    let words = vec!["hello", "hi", "hey", "howdy", "greetings"];
    let long_upper: Vec<String> = words
        .iter()
        .filter(|w| w.len() >= 4)
        .map(|w| w.to_uppercase())
```

```
        .collect();
    println!("\n4+ letters, uppercased: {:?}", long_upper);
}
```

6.14.4 Recap

- `.map(f)` transforms each element; `.filter(pred)` drops the non-qualifying ones.
- `.flat_map(f)` = `.map(f)` + `.flatten()` — conceptually like `and_then` on `Option` / `Result`.
- `.copied()` turns each `&T` into `T` (requires `T: Copy`); `.cloned()` is similar but uses `Clone`.
- `.rev()` reverses the iteration order, requiring `DoubleEndedIterator`.
- These methods chain freely into clear data-processing pipelines.
- Pairing with `.copied()` dodges `filter`'s pesky `&&T` problem.

6.15 Lazy Evaluation

6.15.1 Goal of This Episode

Understand the lazy nature of iterators — `.map(f)` and `.filter(pred)` don't run immediately; they build nested structures that `.collect()` or `for` later pulls through one by one.

6.15.2 Concept

6.15.2.1 Iterators Are Lazy

This may be the most important idea in all of Chapter 6: **an iterator's transformation methods don't execute immediately**.

```
fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let iter = v.iter().map(|x| {
        println!("Processing {}", x);
        x * 2
    });
    // Up to this point, nothing has been printed!
}
```

`map` hasn't "run through" the elements. It merely built a new iterator structure recording "what to do later." Only when someone calls a "consuming" method — `collect()`, `for`, `sum()` — do elements get pulled through one at a time.

6.15.2.2 Russian Nesting Dolls

Each call to `.map(f)` or `.filter(pred)` really "wraps another layer" around the iterator. Like Russian nesting dolls:

```
    v.iter()                // Innermost: the original iterator
        .filter(|x| **x > 2) // Second layer: a Filter struct holding inner + closure
        .map(|x| x * 10);    // Third layer: a Map struct holding inner + closure
```

Each layer is a `struct` holding the inner iterator and its own closure. The standard library's `Map` and `Filter` look roughly like:

```
struct Map<I, F> {
    iter: I, // The inner iterator
    f: F,    // The closure to apply
}

struct Filter<I, P> {
    iter: I,      // The inner iterator
    predicate: P, // The filtering-condition closure
}
```

Their `.next()` implementations are intuitive too:

```
// Map's next(): fetch one element from inside, apply the closure
impl<B, I: Iterator, F: FnMut(I::Item) -> B> Iterator for Map<I, F> {
    type Item = B;
    fn next(&mut self) -> Option<B> {
        let x = self.iter.next()?; // Ask the inner layer for an element
        Some((self.f)(x))          // Apply the closure and return
    }
}

// Filter's next(): keep fetching from inside until something qualifies
impl<I: Iterator, P: FnMut(&I::Item) -> bool> Iterator for Filter<I, P> {
    type Item = I::Item;
    fn next(&mut self) -> Option<I::Item> {
        loop {
            let x = self.iter.next()?; // Ask the inner layer for an element
            if (self.predicate)(&x) {
                return Some(x);        // Qualifies — return it
            }
            // Doesn't qualify; ask for the next one
        }
    }
}
```

So the whole chain is a stack of `structs` wrapped together — call the outermost `.next()`, it asks the layer inside, which asks the layer further in, all the way to the bottom.

6.15.2.3 Pull-based: One Element at a Time

When you call `.collect()` or run a `for` loop, the outermost iterator starts "pulling":

1. The outermost (`Map`) asks the second layer (`Filter`): "Give me the next element."
2. Filter asks the innermost (the original iterator): "Give me the next element."
3. The innermost returns `Some(&1)`.
4. Filter checks: `1 > 2`? Fails. Ask again.
5. The innermost returns `Some(&2)`.
6. Filter checks: `2 > 2`? Fails. Ask again.
7. The innermost returns `Some(&3)`.
8. Filter checks: `3 > 2`? Passes! Hand it to Map.
9. Map applies the closure: `3 * 10 = 30`, returning `Some(30)`.

Each element is processed **all the way through** — not "all the `filters` first, then all the `maps`." Which means **no intermediate `Vec` is ever needed**.

6.15.2.4 Infinite Iterators

Thanks to laziness, iterators can be **infinite**. Both `std::iter::repeat` and `std::iter::from_fn` can produce iterators that never return `None`:

```
use std::iter;

fn main() {
    // Forever producing 1, 2, 3, 4, 5, ...
    let mut n = 0;
    let naturals = iter::from_fn(move || {
        n += 1;
        Some(n)
    });
}
```

This doesn't loop forever, because iterators are lazy — with nobody calling `.next()`, nothing happens.

6.15.2.5 Taming Infinite Iterators with `.take(n)`

`.take(n)` extracts finitely many elements from an infinite iterator:

```
    let first_ten: Vec<i32> = naturals.take(10).collect();
    // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

That's the power of lazy evaluation — describe a "conceptually infinite" computation first, and decide how much to take at the end.

6.15.2.6 Accidentally Forgot to Consume?

Because iterators are lazy, writing `.map(f)` but forgetting `.collect()` or `for` means nothing happens. The Rust compiler warns you:

```
warning: unused `Map` that must be used
note: iterators are lazy and do nothing unless consumed
```

Seeing this warning tells you: you forgot to consume the iterator.

6.15.3 Example Code

```
use std::iter;

fn main() {
    // The laziness demo: map doesn't run immediately
    println!("--- The laziness demo ---");
    let v = vec![1, 2, 3];
    let iter = v.iter().map(|x| {
        println!("  Processing {}", x);
        x * 2
    });
    println!("map is built, but hasn't run yet...");
    println!("Now collecting:");
    let result: Vec<i32> = iter.collect();
    println!("Result: {:?}", result);

    // Pull-based: filter + map handling one element at a time
```

```
    println!("\n--- The pull-based demo ---");
    let data = vec![1, 2, 3, 4, 5, 6];
    let processed: Vec<i32> = data
        .iter()
        .filter(|&&x| {
            println!("  filter checking {}", x);
            x % 2 == 0
        })
        .map(|&x| {
            println!("  map processing {}", x);
            x * 10
        })
        .collect();
    println!("Result: {:?}", processed);
    // Watch the printed order! filter and map run interleaved

    // Building an infinite iterator with from_fn (the first 10 primes)
    let mut candidate = 1;
    let primes: Vec<i32> = iter::from_fn(move || {
        loop {
            candidate += 1;
            let is_prime = (2..candidate).into_iter().all(|d| candidate % d != 0);
            if is_prime {
                return Some(candidate);
            }
        }
    })
    .take(10)
    .collect();
    println!("\nThe first 10 primes: {:?}", primes);

    // No intermediate Vecs — everything in one pipeline
    println!("\n--- Zero intermediate Vecs ---");
    let sum_of_even_squares: i32 = (1..=100)
        .into_iter()
        .filter(|x| x % 2 == 0)
        .map(|x| x * x)
        .sum();
    println!("Sum of squares of the evens in 1~100: {}", sum_of_even_squares);
    // No intermediate Vec was ever built; every element was processed one at a time
}
```

6.15.4 Recap

- `Iterator` methods like `.map(f)` / `.filter(pred)` are **lazy** — nothing runs immediately.
- Each transformation call "wraps another `struct` layer" outside (Russian nesting dolls).
- Consumption (`.collect()`, `for`, `.sum()`, etc.) is what triggers execution.
- Execution is **pull-based** — one element pulled at a time, passing through every layer, no intermediate `Vec`s.
- Thanks to laziness, iterators can be **infinite**.
- Use `.take(n)` to extract finitely many elements from an infinite iterator.
- Forget to consume an iterator, and the compiler warns you.

Congratulations on finishing Chapter 6! 🎉 From function pointers through the three `Fn` `traits` of closures to the lazy evaluation of iterators — this chapter combined ownership, `traits`, generics, and everything before it, showing off the power of functional programming in Rust. You can now write clean, efficient data-processing pipelines with no intermediate staging. Next chapter: Cargo, `crates`, and the `mod` system — taking your code from a single file to a real project structure!

Chapter 7

Cargo, Crates, and the Module System

This chapter's topics are on the easier side: how to split up and manage code, restrict its visibility, and ultimately build a project of your own. These features aren't theoretically difficult, but project-level modularity makes a software engineer's life much more convenient.

7.1 Cargo and crates.io

7.1.1 Goal of This Episode

Get to know more of Cargo's features and how to use community `crates` via crates.io.

7.1.2 Concept

We've been using `cargo new` and `cargo run` since Chapter 1. Actually, `cargo run` does two things: first it **compiles** your code, then it **runs** the compiled executable. To compile without running, use `cargo build` — it just produces the executable, placed in the `target/debug/` folder.

This episode covers more of Cargo's features, especially bringing in external `crates`.

7.1.2.1 Debug Build vs Release Build

`cargo build` and `cargo run` default to **debug mode** — fast to compile but slow to run (no optimizations). When it's time to ship your program, add `--release`:

```
cargo build --release
```

This produces an optimized executable, placed in `target/release/` instead of `target/debug/`. The difference can be enormous — some programs run several times faster in release mode.

7.1.2.2 Cargo.toml

Every Cargo project's root has a `Cargo.toml`. TOML is a configuration format designed to be easy to read and write.

A typical `Cargo.toml`:

```
[package]
name = "my_project"
version = "0.1.0"
edition = "2024"

[dependencies]
```

- `[package]`: the project's basic information (name, version, Rust edition)

- `[dependencies]`: the external `crates` this project uses

The `edition` here is a Rust **version number** — not the compiler's version, but the **language specification's version**. Rust publishes a new edition every few years (2015, 2018, 2021, 2024), each possibly adjusting some syntax or default behaviors. `crates` written using different editions interoperate fine, so compatibility isn't a worry. `cargo new` sets the newest edition for you automatically.

7.1.2.3 Adding External `crates`

Want to use `crates` others have written? The simplest way:

```
cargo add rand
```

This automatically adds a line like the following to `Cargo.toml`'s `[dependencies]`:

```
[dependencies]
rand = "0.10"
```

The actual version number depends on the latest release when you run `cargo add` — it may differ from what's written here.

7.1.2.4 crates.io

crates.io is Rust's official `crate` registry. You can search for `crates`, check download counts, and read documentation. Every `crate` page has:

- Usage instructions and version history
- A link to auto-generated documentation on docs.rs.
- Download counts (a rough gauge of popularity).

7.1.2.5 Version Semantics for Dependencies

There are several ways to specify a `crate`'s version in `[dependencies]`:

- `"^1.0"` (or simply `"1.0"`): any version compatible with `1.x.y`, but never up to `2.0`.
- `"=1.0.0"`: locked to **exactly** this version.
- `">=1.2, <1.5"`: a range.

Most of the time the default `^` is fine; Cargo picks a suitable version for you. For more detail, see the official documentation.

7.1.2.6 Cargo Features

Some `crates` offer optional functionality, enabled via `features`:

```
[dependencies]
serde = { version = "1.0", features = ["derive"] }
```

Now `serde`'s `#[derive(Serialize, Deserialize)]` becomes usable, while unneeded features stay out of the compilation.

7.1.3 Example Code

Generating random numbers with the `rand` `crate`:

```rust
// First run: cargo add rand
extern crate rand;

use rand::RngExt;

fn main() {
    let mut rng = rand::rng();

    let n: u32 = rng.random_range(1..=100);
    println!("Random number: {}", n);

    let coin: bool = rng.random();
    if coin {
        println!("Heads!");
    } else {
        println!("Tails!");
    }
}
```

7.1.4 Recap

- `cargo build --release` produces an optimized executable, suited for shipping.
- `Cargo.toml` uses TOML format; `[package]` holds project info, `[dependencies]` the external `crates`.
- `edition` is the Rust language specification's version (2015, 2018, 2021, 2024); `crates` written using different editions interoperate.
- `cargo add <crate>` is the fastest way to add an external `crate`.
- crates.io is Rust's official registry; docs.rs hosts auto-generated docs.
- The version `"1.0"` equals `"^1.0"`, allowing compatible upgrades; `"=1.0.0"` requires exactly that version.
- `features` switch on a `crate`'s optional functionality.

7.2 `mod`

7.2.1 Goal of This Episode

Learn to organize code into a layered structure with `mod`.

7.2.2 Concept

As programs grow longer, cramming everything into one `main.rs` becomes hard to maintain. We need to group related functions, `structs`, and `enums` — and in Rust, that grouping mechanism is the **module (`mod`)**.

7.2.2.1 Defining a `mod` in the Same File

The simplest usage: create a block right in the file with the `mod` keyword.

```rust
mod math {
    pub fn add(a: i32, b: i32) -> i32 {
        a + b
    }
```

```
    pub fn multiply(a: i32, b: i32) -> i32 {
        a * b
    }
}
```

Call a `mod`'s functions with the `::` path syntax:

```
    let result = math::add(3, 5);
```

Note that `pub` — things inside a `mod` are **private by default**. Without `pub`, the outside can't see or use them. The full rules for `pub` come in Episode 4; for now, remember: want outside access, add `pub`.

7.2.2.2 Nested `mod`s

`mod`s can nest, layer within layer:

```
mod math {
    pub mod basic {
        pub fn add(a: i32, b: i32) -> i32 {
            a + b
        }
    }

    pub mod advanced {
        pub fn power(base: i32, exp: u32) -> i32 {
            let mut result = 1;
            for _ in 0..exp {
                result *= base;
            }
            result
        }
    }
}
```

Calls then use the full path:

```
    let sum = math::basic::add(2, 3);
    let p = math::advanced::power(2, 10);
```

It's like a filesystem's folder structure — `math` holds two sub-`mod`s, `basic` and `advanced`.

7.2.3 Example Code

```
mod geometry {
    pub struct Rectangle {
        pub width: f64,
        pub height: f64,
    }

    impl Rectangle {
        pub fn new(width: f64, height: f64) -> Rectangle {
            Rectangle { width, height }
        }
```

```
        pub fn area(&self) -> f64 {
            self.width * self.height
        }
    }

    pub mod utils {
        pub fn describe_shape(name: &str, area: f64) {
            println!("The area of the {} is {}", name, area);
        }
    }
}

fn main() {
    let rect = geometry::Rectangle::new(10.0, 5.0);
    let area = rect.area();
    geometry::utils::describe_shape("rectangle", area);
}
```

7.2.4 Recap

- `mod name { ... }` creates a `mod` in the same file.
- Things inside a `mod` are called with the `mod_name::item` path syntax.
- Everything inside a `mod` is **private by default**; external use requires `pub`.
- `mod`s can nest, making paths ever longer: `a::b::c::func()`.
- `mod` is Rust's basic unit of code organization — like folders organizing files.

7.3 File `mod`s

7.3.1 Goal of This Episode

Learn to split `mod`s into separate files, and understand Rust's file-to-`mod` correspondence rules.

7.3.2 Concept

Last episode we wrote `mod`s inside one file, but real projects can't stuff everything together. Rust provides rules for splitting `mod`s into standalone files.

7.3.2.1 The Basic Split: `mod` + a Standalone File

Suppose you have a `math` `mod` and want to move it into its own file. The recipe is simple:

1. In `main.rs` (or `lib.rs`) write `mod math;` (note the trailing semicolon, not braces).
2. Create `math.rs` and put the `mod`'s contents in it.

```
src/
├── main.rs
└── math.rs
```

main.rs:

```
mod math;

fn main() {
    let result = math::add(3, 5);
```

```
    println!("3 + 5 = {}", result);
}
```

math.rs:

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

pub fn subtract(a: i32, b: i32) -> i32 {
    a - b
}
```

Note that inside `math.rs` you **don't write `mod math { ... }` again** — the file itself is that `mod`.

7.3.2.2 Folder Structures for Sub-mods

If the `math` mod has sub-mods of its own, there are two ways to organize:

Way 1: with `mod.rs` (the traditional style)

```
src/
├── main.rs
└── math/
    ├── mod.rs
    ├── basic.rs
    └── advanced.rs
```

`math/mod.rs` is the `math` mod's entry point, declaring the sub-mods:

```
// math/mod.rs
pub mod basic;
pub mod advanced;
```

Way 2: a same-named file + folder (recommended)

```
src/
├── main.rs
├── math.rs          ← The math mod's entry point
└── math/
    ├── basic.rs
    └── advanced.rs
```

```
// math.rs
pub mod basic;
pub mod advanced;
```

Both ways work identically — pick whichever you like. Newer projects lean toward Way 2, avoiding a pile of files all named `mod.rs` that are hard to tell apart in an editor.

7.3.2.3 `lib.rs` vs `main.rs`

A Rust project can contain one or more `crates`. A `crate` comes in two types:

- **Binary `crate`**: has `src/main.rs`, compiling to an executable.
- **Library `crate`**: has `src/lib.rs`, a library for others to use.

One project can contain **both** `main.rs` and `lib.rs`. `main.rs` is the binary `crate`'s root; `lib.rs`

is the library `crate`'s root.

```
src/
├── main.rs    ← binary crate root
├── lib.rs     ← library crate root
├── math.rs
└── math/
    ├── basic.rs
    └── advanced.rs
```

Inside `main.rs`, refer to things in `lib.rs` via the `crate`'s name:

```rust
// main.rs
// Assuming Cargo.toml's [package] name = "my_project"
use my_project::math;

fn main() {
    let result = math::basic::add(1, 2);
    println!("{}", result);
}
```

7.3.3 Example Code

Since file `mod`s span multiple files, a single-file demo isn't possible. Below is a complete multi-file example — create the corresponding file structure and run it with `cargo run`:

```
src/
├── main.rs
├── math.rs
└── math/
    ├── basic.rs
    └── advanced.rs
```

main.rs:

```rust
mod math;

fn main() {
    let sum = math::basic::add(10, 20);
    println!("10 + 20 = {}", sum);

    let p = math::advanced::power(2, 8);
    println!("2 ^ 8 = {}", p);
}
```

math.rs:

```rust
pub mod basic;
pub mod advanced;
```

math/basic.rs:

```rust
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

math/advanced.rs:

```rust
pub fn power(base: i32, exp: u32) -> i32 {
    let mut result = 1;
    for _ in 0..exp {
        result *= base;
    }
    result
}
```

7.3.4 Recap

- `mod math;` (semicolon-terminated) tells Rust to go find the sub-`mod`.
- The split-out file **doesn't** contain another `mod math { ... }` — the file itself is the `mod`.
- Sub-`mod`s can use `math/mod.rs` (traditional) or `math.rs` + a `math/` folder (recommended).
- `main.rs` is the binary `crate`'s root; `lib.rs` is the library `crate`'s root.
- One project can contain a binary `crate` and a library `crate` at the same time.

7.4 `pub` Visibility

7.4.1 Goal of This Episode

Fully understand Rust's visibility rules, and master `pub`'s various usages.

7.4.2 Main Text

Episode 2 mentioned that a `mod`'s contents are private by default. This episode lays out the visibility rules completely.

7.4.2.1 Private by Default

Rust's philosophy is **closed by default** — everything starts private, and you must open it explicitly with `pub`. The exact opposite of languages that default to public.

```rust
mod secrets {
    fn hidden() {
        // The outside can't see me
    }

    pub fn visible() {
        // The outside may call me
        hidden(); // Calls within the same mod are fine
    }
}

fn main() {
    secrets::visible(); // OK
    secrets::hidden();  // Compile error! hidden is private
}
```

You might wonder: neither `fn main()` nor `mod secrets` has `pub`, so why can `main` see `secrets`? Because both are defined in the root `mod` — members of the same `mod` see each other, no `pub` required. `pub` exists to let **other `mod`s** see your things.

7.4.2.2 `pub fn`

A function with `pub` is publicly exposed. Nothing more to say.

7.4.2.3 `pub struct` — Fields Marked Individually

`pub` on a `struct` only makes the **type** public — the fields stay private! Each field needs its **own** `pub`:

```
mod user {
    pub struct Profile {
        pub name: String,  // Externally readable/writable
        pub email: String, // Externally readable/writable
        age: u32,          // Private! Invisible outside
    }

    impl Profile {
        pub fn new(name: String, email: String, age: u32) -> Profile {
            Profile { name, email, age }
        }

        pub fn age(&self) -> u32 {
            self.age // Read-only access exposed via a method
        }
    }
}

fn main() {
    let p = user::Profile::new(
        String::from("Yaju"),
        String::from("yaju@senpai.com"),
        24,
    );
    println!("Name: {}", p.name); // OK, name is pub
    println!("Age: {}", p.age()); // OK, accessed via the method
    println!("{}", p.age);        // Compile error! The age field is private
}
```

This design matters — it lets you control which fields to expose and which to hide. If a `struct` has any private field, outsiders can't construct it directly with `StructName { ... }`; they must go through a constructor you provide.

Tuple `structs` are the same — fields default to private, each needing its own `pub`:

```
mod geometry {
    pub struct Point(pub f64, pub f64); // Both fields public
    pub struct Id(u64);                 // The field is private!
}

fn main() {
    let p = geometry::Point(1.0, 2.0); // OK, the fields are pub
    println!("x = {}", p.0);

    let id = geometry::Id(42); // Compile error! Id's field is private
}
```

7.4.2.4 `pub enum` — Variants Automatically Public

enums differ from `structs`: once the `enum` itself is `pub`, all variants are **automatically public**.

```
mod status {
    pub enum Color {
        Red,
        Green,
        Blue,
    }
}

fn main() {
    let c = status::Color::Red; // Every variant is available
    match c {
        status::Color::Red => println!("Red"),
        status::Color::Green => println!("Green"),
        status::Color::Blue => println!("Blue"),
    }
}
```

Which makes sense — publishing an `enum` while hiding some variants would make correct `matching` impossible; better not to publish at all.

7.4.2.5 `pub trait` and `impl`

Once a `trait` has `pub`, the `fns` inside **neither need nor may** take individual `pubs` — their visibility follows the `trait`. A public `trait` means public `fns`; a private `trait`, private `fns`. Sensible: a `trait` is a "contract," and publishing the contract means publishing all its clauses — how else would anyone implement it?

```
mod animal {
    pub trait Speak {
        fn speak(&self); // No pub needed; follows the trait
    }

    pub struct Dog;

    impl Speak for Dog {
        fn speak(&self) {
            println!("Woof!");
        }
    }
}

fn main() {
    use animal::Speak; // The trait must be in scope to call its methods
    let d = animal::Dog;
    d.speak();
}
```

Note the line `use animal::Speak;` — even though `Dog` implements `Speak`, you still must bring the `Speak` `trait` into scope to call its methods. Remove that line and `d.speak()` fails to compile. That's Rust's rule: **when using the `.method()` syntax, the `trait` that provides the method must be in scope.**

```rust
mod animal {
    pub trait Speak {
        fn speak(&self); // No pub needed; follows the trait
    }

    pub struct Dog;

    impl Speak for Dog {
        fn speak(&self) {
            println!("Woof!");
        }
    }
}

fn main() {
    // No use animal::Speak;
    let d = animal::Dog;
    d.speak(); // Compile error! Speak isn't in scope
}
```

The `impl` block itself **neither needs nor may take `pub`**. For `impl Type` (not `impl Trait for Type`), each `fn` inside controls its own visibility with `pub`:

```rust
mod shapes {
    pub struct Circle {
        pub radius: f64,
    }

    impl Circle {
        pub fn area(&self) -> f64 {
            std::f64::consts::PI * self.radius * self.radius
        }

        // A private method, usable only within the mod
        fn internal_check(&self) -> bool {
            self.radius > 0.0
        }
    }
}
```

7.4.2.6 `pub(crate)`, `pub(super)`, `pub(in path)`

Sometimes you don't want full publicity, yet other `mod`s within the `crate` should have access. Rust offers fine-grained control:

- `pub(crate)`: visible throughout the `crate`, invisible outside (to other `crate`s).
- `pub(super)`: visible within the parent `mod`.
- `pub(in crate::some::path)`: visible within the named ancestor `mod` — the finest control.

```rust
mod database {
    // Callable anywhere within the crate, but if this is a library,
    // users of your library can't see this function
    pub(crate) fn connect() -> String {
        String::from("connected")
    }
```

```
    // queries is private: database can access it, but main cannot.
    mod queries {
        // pub(super) lets the parent database mod call this function.
        pub(super) fn raw_query() -> String {
            String::from("SELECT * FROM users")
        }
    }

    pub(crate) fn safe_query() -> String {
        let raw = queries::raw_query(); // OK: database is queries' parent
        format!("SAFE: {}", raw)
    }
}

// A pub(in path) example
mod app {
    pub mod api {
        pub mod internal {
            // Visible within app::api.
            pub(in crate::app::api) fn secret_key() -> &'static str {
                "super-secret"
            }
        }

        pub fn get_key() -> &'static str {
            internal::secret_key() // OK, we're inside app::api
        }
    }
}

// app::api::internal::secret_key() is invisible here,
// since pub(in crate::app::api) makes it visible only within app::api

// Note: pub(in path) must name a mod that contains you
// (one of the layers outward), not an unrelated path:
//     pub(in crate::some_unrelated_mod) fn foo() {}
// The compiler errors; you cannot open visibility to a mod that does not contain you.

fn main() {
    let conn = database::connect(); // OK, we're in the same crate
    let q = database::safe_query(); // OK, pub(crate)
    println!("{}, {}", conn, q);
    database::queries::raw_query(); // Error: queries is private
}
```

7.4.2.7 Everything You Make Public, Taken Together, Is Your API

The set of things you open up with `pub` — functions, types, methods, `traits` — together form the `crate`'s **API**. API (application programming interface) means "**the interface a piece of code exposes for others to call**": others see, and should depend on, only your public face; the private implementation details hidden behind `pub` are beyond their reach, and yours to change freely later. From Chapter 1 to now, every `String::new()`, `vec.push(x)`, and `iter.map(...)` you wrote was a call into the standard library's API — the standard library marked those func-

tions and methods `pub` for you, hiding its internals completely. The only difference: before, you were the API's **user**; from this chapter on, you're also an API **designer**.

7.4.3 Recap

- Rust makes **everything private by default**; publicity requires an explicit `pub`.
- `pub struct` publicizes only the type name; every field needs its **own** `pub` (tuple `structs` too).
- A `struct` with private fields can't be constructed directly from outside; provide a constructor.
- A `pub enum`'s variants are **automatically public**.
- In `impl Trait for T`, the `fn`s' visibility follows the `trait` — no `pub`; in `impl T`, each `fn` takes its own `pub`.
- When using the `.method()` syntax, the `trait` that provides the method must be in scope.
- `pub(crate)`: visible within the `crate`, not outside.
- `pub(super)`: visible within the parent `mod`.
- `pub(in path)`: visible within the named ancestor `mod`.
- Everything `pub`, taken together, **is your API**; the rest is implementation detail. The standard library is itself an API — this chapter turns you from the API's "user" into its "designer."

7.5 use

7.5.1 Goal of This Episode

Learn to simplify paths with `use`, and understand Rust's path-resolution rules and the various import styles.

7.5.2 Concept

We've had a first taste of `use` before; here we lay out all its usages and the path rules in full.

7.5.2.1 Why `use` Is Needed

Writing the full path at every call site gets tiring:

```
    let sum = crate::math::basic::add(1, 2);
    let diff = crate::math::basic::subtract(5, 3);
```

Bring the path in with `use`, and short names work from then on:

```
use crate::math::basic::add;
use crate::math::basic::subtract;

fn main() {
    let sum = add(1, 2);
    let diff = subtract(5, 3);
}
```

7.5.2.2 Absolute vs Relative Paths

Rust paths have two starting points:

Absolute paths — starting from the `crate` root:

```
use crate::math::add; // The math mod within this very crate
```

Relative paths — starting from the current `mod`'s position:

```
use math::add; // The math sub-mod under the current mod
```

7.5.2.3 Paths for External `crate`s

After adding an external `crate` in `Cargo.toml`, use the `crate`'s name as the path's head:

```
use std::collections::HashMap;
use rand::RngExt;
```

`std` is Rust's **standard library** — a built-in toolkit including the `Vec`, `String`, `Option`, `Result`, `println!` we've already used, plus much more: file operations, networking, collections, and so on. No Cargo.toml dependency is needed, since every Rust program links `std` automatically. Its paths read like an external `crate`'s — `std::collections::HashMap`, `std::fmt::Display`, etc. And not only is `std` linked automatically — `std`'s **prelude** is imported automatically too, meaning the most common types and `traits` (`Vec`, `String`, `Option`, `Result`, `Clone`, `Copy`...) work with no `use` at all. That's why the early chapters never needed `use`.

To emphasize "this is an external `crate`" explicitly, start with `::`:

```
use ::rand::RngExt; // Explicitly: rand is an external crate, not a local mod
```

Especially useful when your own `crate` also has a `mod` named `rand` — it removes the ambiguity.

7.5.2.4 super:: and self::

- `super::` one level up, to the **parent `mod`**.
- `self::` the **current `mod`** (usually omitted, but occasionally useful within `use`).

```
mod outer {
    pub fn greet() -> String {
        String::from("Hello from outer")
    }

    pub mod inner {
        pub fn call_parent() -> String {
            super::greet() // Calling the parent mod's greet
        }
    }
}
```

7.5.2.5 `use`-ing Several Things at Once

Importing several items under one path can be merged with braces:

```
use std::io::{self, Read, Write};
// Equivalent to:
// use std::io;
// use std::io::Read;
// use std::io::Write;
```

`self` here stands for `std::io` itself — so you've imported the `io` `mod` along with the `Read` and `Write` inside it.

7.5.2.6 `use ... as` (Aliases)

When two different places have same-named things, alias with `as`:

```
use std::fmt::Result as FmtResult;
use std::io::Result as IoResult;

fn format_something() -> FmtResult {
    Ok(())
}

fn read_something() -> IoResult<()> {
    Ok(())
}
```

7.5.2.7 Name Collisions with `use`

`use`-ing two same-named things into one scope makes Rust error outright:

```
mod a {
    pub fn hello() -> &'static str { "from a" }
}

mod b {
    pub fn hello() -> &'static str { "from b" }
}

use a::hello;
use b::hello; // Compile error! hello is already defined
```

That's when `as` aliases save the day.

But across **different scopes**, an inner `use` shadows the outer — just like `let` shadowing:

```
mod a {
    pub fn hello() -> &'static str { "from a" }
}

mod b {
    pub fn hello() -> &'static str { "from b" }
}

use a::hello;

fn main() {
    println!("{}", hello());     // "from a"

    {
        use b::hello;            // Shadows the outer hello within this scope
        println!("{}", hello()); // "from b"
    }

    println!("{}", hello());     // "from a" (back to the outer)
}
```

7.5.2.8 Glob Imports (the Asterisk)

`*` brings in every name accessible from a `mod` at the current location:

```
use std::collections::*; // HashMap, HashSet, BTreeMap... all available
```

Generally not recommended in production code — it's unclear what came in, inviting collisions. But it's very common in **tests** — `use super::*;` brings everything from the parent `mod` into the test `mod`. Next episode covers `cargo test`, where you'll see this in action.

7.5.2.9 `use`-ing `enum` Variants

`use` isn't just for things under a `mod` — `enum` variants can be imported too:

```
use std::cmp::Ordering::{Less, Equal, Greater};

fn compare(a: i32, b: i32) {
    match a.cmp(&b) {
        Less => println!("Less than"),
        Equal => println!("Equal"),
        Greater => println!("Greater than"),
    }
}
```

No writing `Ordering::Less` every time — plain `Less` suffices. Especially handy when a `match` has many variants.

7.5.3 Example Code

```
mod math {
    pub mod basic {
        pub fn add(a: i32, b: i32) -> i32 {
            a + b
        }

        pub fn subtract(a: i32, b: i32) -> i32 {
            a - b
        }
    }

    pub mod advanced {
        pub fn power(base: i32, exp: u32) -> i32 {
            let mut result = 1;
            for _ in 0..exp {
                result *= base;
            }
            result
        }

        pub fn factorial(n: u64) -> u64 {
            let mut result: u64 = 1;
            for i in 1..=n {
                result *= i;
            }
            result
        }
```

```
    }
}

// The various flavors of use
use math::basic::add;
use math::basic::subtract;
use math::advanced::{power, factorial};

fn main() {
    println!("3 + 5 = {}", add(3, 5));
    println!("10 - 4 = {}", subtract(10, 4));
    println!("2 ^ 10 = {}", power(2, 10));
    println!("10! = {}", factorial(10));
}
```

7.5.4 Recap

- `use` brings a path into scope, sparing you the full path each time.
- Absolute paths start with `crate::`; relative paths start from the current `mod`.
- External `crates` start with their name; a `::` prefix marks one explicitly as external.
- `std` is the standard library — usable without a dependency; the prelude lives there too.
- `super::` points to the parent `mod`; `self::` to the current one.
- `use a::b::{self, X, Y};` imports several things at once.
- `use X as Alias;` aliases, resolving name collisions.
- Same-scope same-name `use`s error; different scopes shadow (inner over outer).
- `use something::*;` — the glob import: common in tests, rare in production code.
- `enum` variants can be `use`d too.

7.6 `cargo test`

7.6.1 Goal of This Episode

Learn to write tests with `#[test]`, verify results with the `assert!` family of macros, and run tests with `cargo test`.

7.6.2 Concept

7.6.2.1 Why Write Tests?

Once code is written, how do you know it's right? Run it by hand? Then you'll be running it again after every change. **Automated tests** let you write once and verify at any time — one command tells you whether anything broke.

7.6.2.2 The Simplest Test

Add `#[test]` above a function, and it becomes a test function:

```
#[test]
fn it_works() {
    assert_eq!(2 + 2, 4);
}
```

Run `cargo test`, and Rust automatically finds and executes every function marked `#[test]`. If a test function panics, that test counts as failed.

7.6.2.3 The `assert` Family of Macros

- `assert!(condition)` — panics if `condition` is `false`.
- `assert_eq!(left, right)` — panics if `left != right`.
- `assert_ne!(left, right)` — panics if `left == right`.

On failure, `assert_eq!` and `assert_ne!` print both values in `Debug` format, so you can see exactly what went wrong.

The `assert!` family isn't only for tests — you can check conditions in ordinary code with them too. But beware: `assert!` runs in **both** debug and release mode; even a shipped program panics when the condition fails. If you want checks only during development, automatically removed for release, use `debug_assert!`, `debug_assert_eq!`, `debug_assert_ne!` — the compiler ignores them entirely in release mode.

Inside **tests**, though, plain `assert!` is fine — tests run on debug builds by default anyway.

7.6.2.4 Testing an Expected Panic

Sometimes you want the opposite: confirming a piece of code **does** panic — say, accessing an out-of-range index. Use `#[should_panic]`:

```
#[test]
#[should_panic]
fn test_out_of_bounds() {
    let v = vec![1, 2, 3];
    let _ = v[10]; // This panics
}
```

If the function panics, the test passes; if it **doesn't**, the test fails instead.

The `expected` parameter can require the panic message to contain a given string, ensuring the panic happened for the right reason:

```
#[test]
#[should_panic(expected = "index out of bounds")]
fn test_out_of_bounds_message() {
    let v = vec![1, 2, 3];
    let _ = v[10];
}
```

7.6.2.5 The Idiomatic Test `mod` Structure

Last episode introduced `use super::*;` — tests use it most of all. The convention is a test `mod` at the bottom of the file:

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn multiply(a: i32, b: i32) -> i32 {
    a * b
}
```

```rust
#[cfg(test)]
mod tests {
    use super::*; // Bring in everything from the parent mod

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }

    #[test]
    fn test_multiply() {
        assert_eq!(multiply(3, 4), 12);
    }
}
```

Key points:

- `#[cfg(test)]` tells the compiler: this `mod` **compiles only when running tests**. Shipped programs contain no test code.
- `mod tests` is an ordinary `mod`; `tests` is just the customary name.
- `use super::*;` brings in everything from the parent `mod` (the file's outermost level), so tests can call `add`, `multiply`, and friends directly.

7.6.2.6 `cargo test`

```
cargo test
```

This command:

1. Compiles your code (tests included).
2. Executes every `#[test]` function.
3. Reports which passed and which failed.

7.6.2.7 Testing Private Functions

Since `mod tests` is a child of the surrounding `mod`, Rust's privacy rules let it access private items declared in its parent. Tests can therefore test **private functions** directly, no `pub` needed.

7.6.3 Example Code

```rust
fn is_even(n: i32) -> bool {
    n % 2 == 0
}

fn abs(n: i32) -> i32 {
    if n >= 0 { n } else { -n }
}

fn clamp(value: i32, min: i32, max: i32) -> i32 {
    if value < min {
        min
    } else if value > max {
        max
```

```
    } else {
        value
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_is_even() {
        assert!(is_even(4));
        assert!(!is_even(7));
        assert!(is_even(0));
    }

    #[test]
    fn test_abs() {
        assert_eq!(abs(5), 5);
        assert_eq!(abs(-3), 3);
        assert_eq!(abs(0), 0);
    }

    #[test]
    fn test_clamp() {
        assert_eq!(clamp(5, 0, 10), 5);   // Within range; unchanged
        assert_eq!(clamp(-3, 0, 10), 0);  // Below the floor; becomes min
        assert_eq!(clamp(15, 0, 10), 10); // Above the ceiling; becomes max
    }

    #[test]
    fn test_not_equal() {
        assert_ne!(abs(-5), -5); // abs(-5) should be 5, not -5
    }

    // Testing an expected panic
    #[test]
    #[should_panic(expected = "already borrowed")]
    fn test_refcell_double_borrow() {
        use std::cell::RefCell;
        let cell = RefCell::new(42);
        let _r = cell.borrow();
        let _w = cell.borrow_mut(); // An immutable borrow exists; this panics
    }
}

fn main() {
    // main can stay empty — tests run via cargo test
    println!("Run the tests with cargo test!");
}
```

7.6.4 Recap

- `#[test]` marks test functions; `cargo test` finds and runs them all automatically.

- `assert!(condition)`, `assert_eq!(a, b)`, `assert_ne!(a, b)` verify results (running in both debug and release).
- `debug_assert!`, `debug_assert_eq!`, `debug_assert_ne!` run only in debug mode, ignored in release.
- `#[should_panic]` tests expected panics; add `expected = "..."` to check the panic message.
- `#[cfg(test)]` compiles the test `mod` only during testing.
- `use super::*;` imports everything from the parent `mod` — the test idiom.
- Tests can exercise private functions directly (the test `mod` being a child `mod`).

7.7 `pub use`

7.7.1 Goal of This Episode

Learn to re-export internal items with `pub use`, so users never need to know your `mod` structure.

7.7.2 Concept

Suppose you've written a library whose internals look like:

```
src/
├── lib.rs
├── math.rs
└── math/
    ├── basic.rs
    └── advanced.rs
```

With no further arrangement, users of your library must write:

```
use your_crate::math::basic::add;
use your_crate::math::advanced::power;
```

Cumbersome — users couldn't care less how you divide folders internally; they just want `add` and `power`.

7.7.2.1 The Magic of `pub use`

`pub use` "re-exports" internal things into the current `mod`, giving the outside a shorter path:

```
// lib.rs
mod math;

// Re-export, sparing users the math::basic:: path
pub use math::basic::add;
pub use math::advanced::power;
```

Now users of your library need only:

```
use your_crate::add;
use your_crate::power;
```

Much cleaner.

Note: `pub use` can only export **things that were already `pub`**. Attempting to `pub use` a private

item makes the compiler complain — you can't publicize what someone else has hidden.

7.7.2.2 Re-exporting from Other crates

pub use isn't limited to your own mods — it can export things from **other crates** too:

```
// lib.rs
pub use rand::RngExt; // Users write use your_crate::RngExt — no rand dependency of their own
```

Common in library design — your library depends on some crate, and you want users to reach those types through your crate without adding the dependency to their own Cargo.toml.

7.7.2.3 Layered Re-exports

You can also re-export at intermediate mod levels, building a more layered public library:

```
// math.rs
pub mod basic;
pub mod advanced;

// Promote the common functions up to the math level
pub use basic::add;
pub use basic::subtract;
pub use advanced::power;
```

Now the outside can use your_crate::math::add, never needing to know about the basic layer.

7.7.2.4 Real-world Cases

Many famous Rust libraries re-export heavily. When you write use std::io::Read;, Read may well be defined somewhere deeper — merely re-exported up to std::io.

7.7.3 Example Code

```
mod shapes {
    pub mod circle {
        pub struct Circle {
            pub radius: f64,
        }

        impl Circle {
            pub fn new(radius: f64) -> Circle {
                Circle { radius }
            }

            pub fn area(&self) -> f64 {
                std::f64::consts::PI * self.radius * self.radius
            }
        }
    }

    pub mod rectangle {
        pub struct Rectangle {
            pub width: f64,
            pub height: f64,
```

```
        }

        impl Rectangle {
            pub fn new(width: f64, height: f64) -> Rectangle {
                Rectangle { width, height }
            }

            pub fn area(&self) -> f64 {
                self.width * self.height
            }
        }
    }

    // Re-exports: users needn't know about the circle and rectangle sub-mods
    pub use circle::Circle;
    pub use rectangle::Rectangle;
}

// Taken straight from shapes; no shapes::circle::Circle needed
use shapes::{Circle, Rectangle};

fn main() {
    let c = Circle::new(5.0);
    println!("Circle area: {}", c.area());

    let r = Rectangle::new(4.0, 6.0);
    println!("Rectangle area: {}", r.area());
}
```

7.7.4 Recap

- `pub use path::Item;` re-exports internal things, giving the outside a shorter path.
- It can export your own `mod`s' contents, or things from other `crate`s.
- A library's `lib.rs` commonly uses `pub use` to lift important types to the `crate`'s top level.

7.8 The Orphan Rule

7.8.1 Goal of This Episode

Understand Rust's orphan rule, and what to do when you want to implement an external `trait` for an external type.

7.8.2 Concept

In Chapter 5 we learned `traits` — you can implement any `trait` for your own types. But have you ever tried this:

```
use std::fmt;

impl fmt::Display for Vec<i32> {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "my vec")
    }
```

```
}
```

The compiler refuses flat out. Why?

7.8.2.1 The Orphan Rule

Rust has a rule:

> **To `impl` a `trait`, at least one of the `trait` or the type must be defined in your own `crate`.**

Put differently: **the `trait` is yours, or the type is yours** — at least one must hold.

In the example above, `Display` is defined by the standard library, and so is `Vec<i32>` — neither is yours, so no.

7.8.2.2 Why This Restriction Exists

Imagine there were no orphan rule:

- `crate A` implements `Display` for `Vec<i32>`, printing `[1, 2, 3]`.
- `crate B` also implements `Display` for `Vec<i32>`, printing `1 | 2 | 3`.
- Your program uses both `A` and `B`... which should the compiler pick?

That's a conflict. The orphan rule prevents the problem at its root.

7.8.2.3 The Legal Cases

All of these are legal:

```
// Case 1: your type + an external trait
struct MyPoint {
    x: f64,
    y: f64,
}

impl std::fmt::Display for MyPoint {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

// Case 2: an external type + your trait
trait Describable {
    fn describe(&self) -> String;
}

impl Describable for Vec<i32> {
    fn describe(&self) -> String {
        format!("A Vec with {} elements", self.len())
    }
}
```

7.8.2.4 Adding Methods to an External Type with Your Own `trait`

Case 2 above is especially useful. Although `Vec<i32>` is a standard-library type, `Describable` is a `trait` we defined in the current `crate`, so the orphan rule allows us to write `impl Describ-`

able for Vec<i32>. After that implementation, a Vec<i32> value can call .describe() with method syntax.

7.8.2.5 Why Can't You Write impl Vec<i32> Directly?

If the goal is merely to add a method, you might wonder why we cannot skip the trait and write this instead:

```
impl Vec<i32> {
    fn describe(&self) -> String {
        format!("A Vec with {} elements", self.len())
    }
}
```

Rust does not allow this either. **The Type in impl Type { ... } must be defined in the current crate.** Vec is defined by the standard library; bringing it into scope with use does not make it your type.

The reason is the one behind the orphan rule — two crates colliding — except that with a trait the collision stays distinguishable. A trait method is only callable when its trait is in scope, so even if crate A and crate B each define their own trait and implement describe for Vec<i32>, the decision is still yours: whichever trait you use, that is the .describe() you get.

A method written directly inside impl Vec<i32> { ... } has no such layer. It belongs to no trait, and calling it requires no use at all: as long as that crate is among your dependencies, v.describe() simply exists. If two crates each added one, you would have no way to say which one you meant. That is why a method attached directly to a type can only be added by the crate that defines that type.

The boundary here is the crate, not a particular file or mod. As long as a type is defined in the current crate, its impl Type { ... } block may live in another mod within that same crate.

Therefore, when you do not want to wrap an external type but do want to add a method to it, you can define your own trait and implement that trait for the external type, as above.

7.8.2.6 The Newtype Pattern (the Workaround)

If you truly need to implement an external trait for an external type, use the **newtype pattern** — a tuple struct wrapping the external type:

```
use std::fmt;

struct MyVec(Vec<i32>);

impl fmt::Display for MyVec {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        let items: Vec<String> = self.0.iter()
            .map(|x| x.to_string())
            .collect();
        write!(f, "[{}]", items.join(", "))
    }
}
```

`MyVec` is a type you defined, so implementing `Display` for it is allowed. `self.0` reaches the inner `Vec<i32>`.

7.8.3 Example Code

```
use std::fmt;

// The newtype pattern: wrapping an external type in your own struct
struct Scores(Vec<i32>);

impl Scores {
    fn new() -> Scores {
        Scores(Vec::new())
    }

    fn add(&mut self, score: i32) {
        self.0.push(score);
    }

    fn total(&self) -> i32 {
        self.0.iter().sum()
    }
}

// Now Display can be implemented for "your type"
impl fmt::Display for Scores {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        let items: Vec<String> = self.0.iter()
            .map(|x| x.to_string())
            .collect();
        write!(f, "Scores: [{}], total: {}", items.join(", "), self.total())
    }
}

fn main() {
    let mut scores = Scores::new();
    scores.add(85);
    scores.add(92);
    scores.add(78);
    scores.add(95);

    // With Display implemented, println works directly
    println!("{}", scores);
}
```

7.8.4 The Multi-parameter `trait` Case

The rule above is the simplest version. For multi-parameter `traits` (like Chapter 5's `From<T>`), the rules are actually more intricate. In brief:

```
// OK: your type appears among the parameters
impl From<MyType> for String { ... }

// Not allowed: both sides are external
```

```
impl From<String> for Vec<i32> { ... }
```

The complete rules involve concepts like "covered type parameters," beyond this tutorial's scope. The curious can consult the official documentation.

7.8.5 Recap

- **The orphan rule**: in `impl Trait for Type`, at least one of the `trait` or the type must be defined in your `crate`.
- "Your type + an external `trait`" ✓ legal.
- "An external type + your `trait`" ✓ legal.
- "An external type + an external `trait`" ✗ illegal.
- Defining your own `trait` and implementing it for an external type adds methods to that type.
- The `Type` in `impl Type { ... }` must be defined in the current `crate`, though the `impl` and the type may live in different files or `mod`s.
- The orphan rule exists to prevent `impl` conflicts between `crate`s.
- **The newtype pattern**: wrap the external type in `struct MyWrapper(OriginalType)`, and it becomes your type.
- The orphan rule for multi-parameter `trait`s is far subtler — see the official documentation.

7.9 Doc Comments

7.9.1 Goal of This Episode

Learn to write documentation comments, understand that documentation examples are tests (doctests), and generate professional HTML docs with `cargo doc`.

7.9.2 Concept

Rust treats documentation as a first-class citizen of the language — not squeezed out by external tools, but built into the syntax. Better still: the example code in your docs gets executed as tests by `cargo test`, so Rust documentation examples never silently go stale.

7.9.2.1 /// Item Doc Comments

Three slashes `///` document **the item that follows** (a function, `struct`, `enum`, `trait`, etc.):

```
/// Computes the greatest common divisor of two integers.
///
/// Uses the Euclidean algorithm, with O(log(min(a, b))) efficiency.
///
/// # Examples
///
/// ```
/// use my_math_lib::gcd;
///
/// let result = gcd(12, 8);
/// assert_eq!(result, 4);
```

```
/// ```
pub fn gcd(mut a: u64, mut b: u64) -> u64 {
    while b != 0 {
        let temp = b;
        b = a % b;
        a = temp;
    }
    a
}
```

/// supports full **Markdown syntax** — headings, bold, code blocks, lists, all of it.

7.9.2.2 `//!` `mod`/`crate`-level Docs

Two slashes plus a bang, `//!`, documents **the item containing it**, usually placed at the very top of a file:

```
//! # Math Library
//!
//! This library provides basic mathematical functions.
//!
//! ## Features
//!
//! - Basic arithmetic
//! - Greatest common divisor computation
//! - Exponentiation

pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

At the top of `lib.rs` it documents the whole `crate`; at the top of some `mod`'s file, that `mod`.

7.9.2.3 The Customary Documentation Sections

The Rust community has some conventional section names:

- `# Examples` — usage examples (the most important one!)
- `# Panics` — under what circumstances it panics.
- `# Errors` — if it returns a `Result`, when it's an `Err`.

7.9.2.4 Documentation Examples Are Tests (doctests)

Here's the point. The code blocks under `# Examples` aren't just for reading — **`cargo test` extracts, compiles, and runs every documentation example**. These are **doctests**. The `cargo test` from Episode 6 actually runs all doctests in addition to `#[test]` functions.

Each documentation example compiles as a **standalone little program** — it lives **outside** your library, like a program written by one of your library's users. So the example must say `use my_math_lib::gcd;`, exactly as a real user would. Forget the `use` and the doctest fails to compile — and **a compile failure counts as a test failure**. Incidentally, examples don't need `fn main()`; rustdoc wraps one around automatically.

This design yields something beautiful: **the examples are always right**. Rename a function or change its signature while forgetting the docs, and `cargo test` throws the error in your face

at once. In many languages, doc examples silently rot as the code evolves; in Rust, a rotten example blocks your tests.

One caveat: only a **library `crate`**'s doctests execute. Doc comments in a binary `crate` still generate documentation, but their examples **won't** run as tests.

7.9.2.5 `cargo doc`

With doc comments written, one command produces beautiful HTML documentation:

```
cargo doc --open
```

This:

1. Compiles your `crate` (without running it).
2. Generates HTML docs from **public** items' `///` and `//!`.
3. Opens them in your browser automatically.

The generated docs are exactly what you see on docs.rs.

7.9.3 Example Code

A complete example this time. Assuming `Cargo.toml`'s `[package]` has `name = "temperature"`, here's `src/lib.rs`:

```
//! # Temperature Conversion Tools
//!
//! Provides conversion functions between Celsius and Fahrenheit.

/// Celsius to Fahrenheit.
///
/// # Formula
///
/// `F = C × 9/5 + 32`
///
/// # Examples
///
/// ```
/// use temperature::celsius_to_fahrenheit;
///
/// let f = celsius_to_fahrenheit(100.0);
/// assert!((f - 212.0).abs() < 0.001);
/// ```
pub fn celsius_to_fahrenheit(c: f64) -> f64 {
    c * 9.0 / 5.0 + 32.0
}

/// Fahrenheit to Celsius.
///
/// # Formula
///
/// `C = (F - 32) × 5/9`
///
/// # Examples
///
/// ```
```

```rust
/// use temperature::fahrenheit_to_celsius;
///
/// let c = fahrenheit_to_celsius(32.0);
/// assert!((c - 0.0).abs() < 0.001);
/// ```
pub fn fahrenheit_to_celsius(f: f64) -> f64 {
    (f - 32.0) * 5.0 / 9.0
}

/// A representation of temperature.
///
/// Supports both Celsius and Fahrenheit units.
pub enum Temperature {
    /// A Celsius temperature
    Celsius(f64),
    /// A Fahrenheit temperature
    Fahrenheit(f64),
}

impl Temperature {
    /// Converts any temperature to Celsius.
    ///
    /// # Examples
    ///
    /// ```
    /// use temperature::Temperature;
    ///
    /// let body = Temperature::Fahrenheit(98.6);
    /// assert!((body.to_celsius() - 37.0).abs() < 0.001);
    /// ```
    pub fn to_celsius(&self) -> f64 {
        match self {
            Temperature::Celsius(c) => *c,
            Temperature::Fahrenheit(f) => fahrenheit_to_celsius(*f),
        }
    }

    /// Converts any temperature to Fahrenheit.
    pub fn to_fahrenheit(&self) -> f64 {
        match self {
            Temperature::Celsius(c) => celsius_to_fahrenheit(*c),
            Temperature::Fahrenheit(f) => *f,
        }
    }
}
```

7.9.4 Recap

- `///` documents the item that follows (`fn`, `struct`, `enum`, etc.).
- `//!` documents the containing item (`mod`, `crate`), usually at the file's top.
- Doc comments support full Markdown syntax.
- `# Examples` is the most important section — a good example beats a thousand words.
- **Documentation examples are doctests**: `cargo test` compiles and runs every one; com-

pile failures and `assert` failures both count as test failures.
- Doctests compile as a "library user," so examples must write `use your_crate::...`.
- Doctests run only for library `crates`.
- `cargo doc --open` generates and opens HTML docs in one step.
- The docs you see on docs.rs are produced by this very mechanism.

7.10 `cargo publish`

7.10.1 Goal of This Episode

Learn to publish your library to crates.io, making it available to Rust developers worldwide.

7.10.2 Concept

So far we've learned to organize code, write documentation, and use other people's `crates`. This episode flips the direction — publishing a project of your own.

7.10.2.1 Account Setup

First, you need a crates.io account:

1. Go to crates.io and log in with a GitHub account.
2. On the account settings page, generate an **API Token**.
3. In the terminal, run:

```
cargo login
```

After pressing enter, the terminal prompts you to paste the token — paste it, press enter again, done. The token is stored locally and used automatically for future publishes.

7.10.2.2 Preparing `Cargo.toml`

Before publishing, `Cargo.toml` needs some required metadata:

```toml
[package]
name = "my-awesome-lib"
version = "0.1.0"
edition = "2024"
description = "A wonderful math utility library"
license = "MIT"
repository = "https://github.com/yourname/my-awesome-lib"
readme = "README.md"
keywords = ["math", "utility"]
categories = ["mathematics"]
```

Per the official documentation, fill in before publishing:

- `license` (or `license-file`): the open-source license (e.g. `MIT`, `Apache-2.0`, `MIT OR Apache-2.0`).
- `description`: a one-line summary
- `homepage`: the project homepage URL
- `repository`: the source repository URL
- `readme`: the README file's path

Recommended but not required:

- `keywords`: search keywords (up to 5)
- `categories`: categories (must match crates.io's category list)

7.10.2.3 Pre-publish Checks

Before publishing, `cargo package` checks for problems:

```
cargo package
```

This simulates the packaging process, checking for missing required fields and other issues.

7.10.2.4 Publish!

Once everything's ready:

```
cargo publish
```

Done! Your project is now on crates.io, and anyone can `cargo add my-awesome-lib`.

7.10.2.5 The Version Update Flow

After publishing, to release an update:

1. Modify the code.
2. Bump `version` in `Cargo.toml`, following SemVer (semantic versioning).
3. `cargo publish` again.

SemVer's rules:

- **Before 1.0** (`0.x.y`): the whole API is considered unstable; any release may break things.
- **After 1.0**:
  - Bug fixes: `1.0.0` → `1.0.1` (patch).
  - New features (backward compatible): `1.0.1` → `1.1.0` (minor).
  - Breaking changes: `1.1.0` → `2.0.0` (major) — the first number changes.

Why does SemVer fuss so much over "breaking changes"? Because once you've published, **your public API (especially the `pub` things) is no longer just your own business** — other people's programs `use` your functions and depend on your type and method declarations. Your public API becomes **a promise to your users**: the surface they depend on isn't yours to change on a whim.

The promise **isn't limited to `pub` things: documented behavior can be part of it too**. Private implementation details remain yours to change as long as those promises still hold. The question most worth asking before publishing or updating is: "Do I really want to maintain this `pub` long-term?" The more you publish, the more you promise, and the less room remains for changing things without breaking someone. Keeping the unnecessary private (or `pub(crate)`) preserves your future freedom to change.

Note: published versions **can't be deleted or overwritten**. If a version turns out badly broken, `cargo yank` marks it as discouraged — but those already using it are unaffected:

```
cargo yank --version 0.1.0
```

7.10.2.6 Best Done Before Publishing

- Write a good `README.md` (shown on the `crate`'s crates.io page).
- Run `cargo test` and confirm all tests pass.
- Write doc comments with `///` (last episode's lesson).
- Make sure there's example code.
- Check the docs look right with `cargo doc --open`.

7.10.3 Example Code

The complete structure of a small library ready for publishing:

```
my-math-lib/
├── Cargo.toml
├── README.md
└── src/
    └── lib.rs
```

Cargo.toml:

```toml
[package]
name = "my-math-lib"
version = "0.1.0"
edition = "2024"
description = "Simple math utility functions"
license = "MIT"
homepage = "https://example.com/my-math-lib"
repository = "https://github.com/example/my-math-lib"
readme = "README.md"
keywords = ["math", "utility"]
categories = ["mathematics"]
```

src/lib.rs:

```rust
//! # My Math Lib
//!
//! Provides simple, handy math functions.

/// Computes the greatest common divisor.
///
/// # Examples
///
/// ```
/// use my_math_lib::gcd;
///
/// assert_eq!(gcd(12, 8), 4);
/// ```
pub fn gcd(mut a: u64, mut b: u64) -> u64 {
    while b != 0 {
        let temp = b;
        b = a % b;
        a = temp;
    }
    a
}
```

```
/// Computes the least common multiple.
///
/// # Examples
///
/// ```
/// use my_math_lib::lcm;
///
/// assert_eq!(lcm(4, 6), 12);
/// ```
pub fn lcm(a: u64, b: u64) -> u64 {
    if a == 0 || b == 0 {
        return 0;
    }
    a / gcd(a, b) * b
}

/// Determines whether a number is prime.
///
/// # Examples
///
/// ```
/// use my_math_lib::is_prime;
///
/// assert!(is_prime(7));
/// assert!(!is_prime(4));
/// ```
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    let mut i: u64 = 2;
    while i * i <= n {
        if n % i == 0 {
            return false;
        }
        i += 1;
    }
    true
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_gcd() {
        assert_eq!(gcd(12, 8), 4);
        assert_eq!(gcd(7, 3), 1);
        assert_eq!(gcd(0, 5), 5);
    }

    #[test]
    fn test_lcm() {
        assert_eq!(lcm(4, 6), 12);
        assert_eq!(lcm(0, 5), 0);
```

```
    }

    #[test]
    fn test_is_prime() {
        assert!(!is_prime(0));
        assert!(!is_prime(1));
        assert!(is_prime(2));
        assert!(is_prime(17));
        assert!(!is_prime(15));
    }
}
```

The publishing command sequence:

```
cargo test       # Confirm the tests pass
cargo doc --open # Check the docs
cargo package    # Simulate packaging
cargo publish    # Publish for real!
```

7.10.4 Recap

- Log into crates.io with GitHub, generate an API token, then configure with `cargo login`.
- Before publishing, `Cargo.toml` should have `license`, `description`, `homepage`, `repository`, `readme`.
- `cargo package` checks for problems before publishing.
- `cargo publish` publishes to crates.io for real.
- Bump the `version` field for updates, following SemVer (semantic versioning).
- Your public API (especially the `pub` things) is a **promise to your users**; removing or incompatibly changing a public item is a breaking change (major). Backward-compatible additions belong in a minor release, but not every addition is backward compatible. Documented behavior can also be part of the promise; private implementation details may change as long as the promises still hold.
- Published versions can't be deleted; `cargo yank` merely marks them as discouraged.
- Writing the README, doc comments, and tests before publishing is basic respect for your users.

Congratulations on finishing Chapter 7! 🎉 By this point, we've covered Rust's major concepts — ownership, borrowing, generics, `traits`, lifetimes, closures, iterators, plus the module system and how to build and publish Cargo projects. You can now stand on your own. If there's an idea in your head, now is a great time to build it!

Even so, Rust has many more distinctive and powerful features. The chapters ahead continue with important topics not yet covered, aiming to give you a more complete, well-rounded understanding of Rust.

Appendix I

A few gems that never quite found room for discussion in the main chapters.

8.1 Number Literal Formats

8.1.1 Goal of This Episode

Learn underscore separators, different bases, type suffixes, and the various ways of writing floating-point literals.

> This episode supplements **Chapter 1**.

8.1.2 Concept

Chapter 1 taught basic number notation like `42` and `3.14`. But Rust's numeric literals actually come in many forms, making your numbers more readable and more precise.

8.1.2.1 Underscore Separators

With a big number, which reads better — `1000000` or `1_000_000`? Rust lets you insert underscores _ at certain positions in a numeric literal; they do not affect its value:

```
let million = 1_000_000;
let weird_but_legal = 1_00_00_00; // Legal, but don't write this
```

8.1.2.2 Different Bases

Beyond decimal, Rust supports three base prefixes:

- `0x` — hexadecimal (e.g. `0xff` = 255)
- `0b` — binary (e.g. `0b1010` = 10)
- `0o` — octal (e.g. `0o77` = 63)

Especially practical for bitwise operations, color values, and the like. Underscores combine with bases too: `0b1111_0000`.

8.1.2.3 Type Suffixes

You can append the type directly to a number:

```
let byte = 0xFFu8;      // Hexadecimal + u8
let big = 1_000_000i64; // Underscores + i64
let pi = 3.14f32;       // Float + f32
```

Without a suffix, integers default to `i32` and floats to `f64`.

8.1.2.4 Floating-point Literals

Floats can be written several ways:

```
    let a = 3.14;        // An ordinary decimal, defaulting to f64
    let b = 3.14f32;     // Specifying f32
    let c = 1.0e10;      // Scientific notation: 1.0 × 10^10
    let d = 2.5E-3;      // Scientific notation: 2.5 × 10^-3 = 0.0025
    let e = 1_234.567_8; // Underscores work in floats too
```

8.1.3 Example Code

```
fn main() {
    // Underscore separators
    let population = 23_000_000;
    println!("Taiwan's population is roughly {}", population);

    // Hexadecimal
    let hex_color = 0xFF5733;
    println!("Color value: {}", hex_color);

    // Binary
    let bits = 0b1010_1100;
    println!("Bit value: {}", bits);

    // Octal
    let octal = 0o755;
    println!("Octal 0o755 = {}", octal);

    // Type suffixes
    let byte_max = 0xFFu8;
    println!("u8's maximum: {}", byte_max);

    // Floats
    let pi = 3.14_159_265f64;
    println!("Pi is roughly {}", pi);

    // Scientific notation
    let speed_of_light = 3.0e8;
    println!("The speed of light is roughly {} m/s", speed_of_light);

    let tiny = 1.6e-19;
    println!("The electron charge is roughly {} C", tiny);
}
```

8.1.4 Recap

- _ can go in certain positions in a numeric literal to aid reading; the compiler ignores it.
- `0x` hexadecimal, `0b` binary, `0o` octal.
- Type suffixes like `u8`, `i64`, `f32` attach directly to numbers.
- Floats support scientific notation (`1.0e10`, `2.5E-3`).

8.2 Short-circuiting of && and ||

8.2.1 Goal of This Episode

Understand that && and || don't necessarily evaluate both sides — sometimes the left side alone settles the result.

> This episode supplements **Chapter 1**.

8.2.2 Concept

Chapter 1 taught && (and) and || (or). One detail went unmentioned: they **short-circuit** (short-circuit evaluation).

8.2.2.1 &&'s Short Circuit

If &&'s left side is `false`, the right side never runs — whatever it is, the overall result must be `false`:

```
fn main() {
    let x = 0;
    // The left side x != 0 is false, so the right side 10 / x never runs
    // If it did run, 10 / 0 would panic!
    if x != 0 && 10 / x > 2 {
        println!("Greater than 2");
    }
}
```

8.2.2.2 ||'s Short Circuit

If ||'s left side is `true`, the right side never runs — whatever it is, the overall result must be `true`:

```
fn check() -> bool {
    println!("check was called");
    true
}

fn main() {
    // The left side is already true; check() is never called
    if true || check() {
        println!("The result is true");
    }
    // Only "The result is true" prints — never "check was called"
}
```

8.2.2.3 Why Know This

Most of the time you needn't think about short-circuiting. But when the right-hand expression has **side effects** (printing, modifying variables) or **can fail** (dividing by zero), knowing the right side may never run becomes important.

8.2.3 Example Code

```rust
fn is_even(n: i32) -> bool {
    println!("  Checking whether {} is even", n);
    n % 2 == 0
}

fn is_positive(n: i32) -> bool {
    println!("  Checking whether {} is positive", n);
    n > 0
}

fn main() {
    // &&: a false left side means the right is never looked at
    println!("--- && short-circuiting ---");
    let n = -3;
    if is_even(n) && is_positive(n) {
        println!("{} is a positive even number", n);
    } else {
        println!("{} is not a positive even number", n);
    }
    // is_even(-3) returns false; is_positive is never called

    // ||: a true left side means the right is never looked at
    println!("\n--- || short-circuiting ---");
    let n = 4;
    if is_even(n) || is_positive(n) {
        println!("{} is even or positive", n);
    }
    // is_even(4) returns true; is_positive is never called

    // A practical scenario: avoiding division by zero
    println!("\n--- A practical scenario ---");
    let divisor = 0;
    if divisor != 0 && 100 / divisor > 10 {
        println!("The quotient exceeds 10");
    } else {
        println!("The divisor is zero, or the quotient doesn't exceed 10");
    }
}
```

8.2.4 Recap

- &&: a `false` left side skips the right; the whole result is immediately `false`.
- `||`: a `true` left side skips the right; the whole result is immediately `true`.
- This is called short-circuit evaluation.
- It matters most when the right side has side effects or can fail.

8.3 `break` with a Value

8.3.1 Goal of This Episode

Learn to return a value from a `loop` with `break`, using the loop as an expression.

This episode supplements **Chapter 1**.

8.3.2 Concept

Remember how "almost everything in Rust is an expression"? The `loop` is no exception — `break` can carry a value out, turning the whole `loop` into an expression.

8.3.2.1 Basic Syntax

```
    let result = loop {
        break 42;
    };
```

Here `loop { break 42; }` has type `i32`, since `break` carried out `42`.

8.3.2.2 Why Can't `while` and `for` Do This?

You might ask: why not `while` and `for`?

The reason: `while` and `for` can finish normally when their condition becomes false or their iterator runs out, without ever reaching a `break`. In that case, the loop produces `()` rather than a value carried by `break`.

`loop` differs because it has no condition that ends it normally. If a `loop` finishes, it must be through `break`; if no `break` is reached, it keeps running and produces no result. That's why the value carried by `break` can become the value of the entire `loop`.

8.3.2.3 Practical Scenarios

The most common use is "searching for something inside a loop, carrying it out when found":

```
    let found = loop {
        // Do some searching...
        if condition {
            break some_value;
        }
    };
```

Much cleaner than declaring a variable first, assigning inside the loop, then `break`ing.

8.3.3 Example Code

```
fn main() {
    // Basic usage: a loop returning a value
    let lucky_number = loop {
        break 7;
    };
    println!("Lucky number: {}", lucky_number);

    // A practical example: the first square number exceeding 100
    let mut n = 1;
    let result = loop {
        let square = n * n;
        if square > 100 {
            break square;
        }
```

```
        n += 1;
    };
    println!("The first square exceeding 100: {}", result);
    println!("It's the square of {}", n);
}
```

8.3.4 Using Labels with `break` Values

A label such as `'search:` can be placed before a loop (`loop`, `while`, or `for`) or an ordinary block expression `{ ... }`. The latter creates a labeled block. Here `'search` is a label, not a lifetime.

`break 'label value` exits the labeled `loop` or block and makes that expression evaluate to `value`. When breaking out of the innermost `loop`, the label can be omitted (`break value`); in a labeled block, the label is required.

```
fn main() {
    let from_loop = 'search: loop {
        loop {
            break 'search 7;
        }
    };

    let from_block = 'answer: {
        let n = 7;
        if n > 5 {
            break 'answer n * 2;
        }
        0
    };

    println!("From loop: {}", from_loop);
    println!("From block: {}", from_block);
}
```

Here `break 'search 7` exits both `loops` directly, making the outer `loop` labeled `'search` evaluate to 7. Meanwhile, `break 'answer n * 2` makes the labeled block evaluate to 14.

8.3.5 Recap

- `let x = loop { break value; };` makes the `loop` an expression, returning the value `break` carries.
- `while` and `for` can't return a value with `break`, because they can finish normally without reaching one.
- `break 'label value` can return a value from a labeled `loop` or labeled block; the label is required for a labeled block.
- The common use: search inside a loop and carry the result out with `break`.

8.4 Multiline Strings & Raw String Literals

8.4.1 Goal of This Episode

Learn to write multiline strings, line-continuation backslashes, and raw strings free of escape characters in Rust.

> This episode supplements **Chapter 1**.

8.4.2 Concept

Programming regularly involves multiline text, file paths, or strings full of special characters. Rust provides some handy syntax for these situations.

8.4.2.1 Multiline Strings

In Rust, string literals can span lines directly:

```
    let poem = "Moonlight before my bed,
like frost upon the ground.";
```

The newlines get included in the string as-is.

8.4.2.2 The Line Continuation \

If you want a long string split across source lines but **without** newlines in the result, end the line with `\`. It swallows the newline and the next line's leading whitespace:

```
    let long = "This is a very long sentence, \
                yet really just one line.";
    // Result: "This is a very long sentence, yet really just one line."
```

8.4.2.3 Raw String Literals

Sometimes strings hold lots of backslashes (Windows paths, say), and escaping each is a chore. The `r"..."` syntax skips escaping entirely:

```
    let path = r"C:\Users\test\documents";
    // No need for "C:\\Users\\test\\documents"
```

8.4.2.4 Raw Strings Containing Quotes

What if the raw string needs double quotes inside? Use the `r#"..."#` syntax:

```
    let json = r#"{"name": "Andy", "age": 29}"#;
```

What if the string even contains `"#`? Add more # layers:

```
    let tricky = r##"There's a "#" symbol here"##;
```

You can use up to 255 # characters, as long as the opening and closing counts match.

8.4.3 Example Code

```
fn main() {
    // A multiline string
    let haiku = "An old pond—
```

```
a frog leaps in,
the sound of water";
    println!("Haiku:\n{}", haiku);
    println!("---");

    // The line continuation: \ swallows the newline and leading whitespace
    let sentence = "Rust is a programming language focused on safety, \
                    performance, and concurrency.";
    println!("{}", sentence);
    println!("---");

    // Raw strings: no escape processing
    let win_path = r"C:\Users\Andy\Desktop\project";
    println!("Path: {}", win_path);

    // Great for regular expressions and similar
    let pattern = r"\d+\.\d+";
    println!("Regex: {}", pattern);

    // A raw string containing double quotes
    let json = r#"{"name": "Ming", "score": 95}"#;
    println!("JSON: {}", json);

    // Multiple #s — for when the string contains "#
    let code_sample = r##"
        let s = r#"hello"#;
        println!("{}", s);
    "##;
    println!("Code sample: {}", code_sample);

    // Raw strings can span lines too
    let html = r#"
<html>
    <body>
        <h1>Hello, Rust!</h1>
    </body>
</html>
"#;
    println!("{}", html);
}
```

8.4.4 Recap

- String literals can span lines directly; the newlines are preserved.
- A \ at line's end continues to the next line, dropping the newline and the next line's leading whitespace.
- `r"..."` is a raw string, processing no escapes at all (`\n`, `\\`, etc. stay verbatim).
- `r#"..."#` lets a raw string contain double quotes.
- A raw string can use up to 255 # characters (`r##"..."##`, `r###"..."###`, and so on), as long as the opening and closing counts match.
- Raw strings shine for Windows paths, regular expressions, JSON, embedded code, and the like.

8.5 Format Strings in Depth

8.5.1 Goal of This Episode

Learn `println!`'s assorted formatting tricks: the variable-capture shorthand, positional parameters, width, precision control, alignment, and base display.

> This episode supplements **Chapter 2**.

8.5.2 Concept

We've been printing with `println!("{}", x)` all along, but Rust's format strings are far more powerful. This episode covers the most-used tricks without covering everything — for the full formatting syntax, see the official documentation.

8.5.2.1 The Variable-capture Shorthand

You can write a variable's name straight into the `{}`:

```
fn main() {
    let name = "Andy";
    println!("{name}"); // Equivalent to println!("{}", name)
}
```

This can be more convenient than writing `{}` and matching variables afterward, especially with many variables. Note only variable names go inside — no expressions (`"{x + 1}"` won't work).

8.5.2.2 Positional Parameters

`{}` matches the following arguments in order by default, but you can write a number in the braces to name which argument explicitly (counting from 0):

```
fn main() {
    println!("{0} {1}", "hel", "lo"); // hel lo
    println!("{1} {0}", "hel", "lo"); // lo hel
}
```

The same argument can be reused without passing it twice:

```
fn main() {
    println!("{0}, {0}!", "wait"); // wait, wait!
}
```

When is this useful? Most commonly when one value appears several times in the string, or when you want to reorder the output without reordering the arguments.

8.5.2.3 Decimal Precision

Control the digits after the decimal point with `:.N`:

```
fn main() {
    let pi = 3.14159265;
    println!("{pi:.2}"); // Prints 3.14
}
```

8.5.2.4 Width

Specify a minimum width with `:N` — shortfalls get padded with spaces:

```
fn main() {
    let x = 42;
    println!("{x:5}"); // "   42" (width 5, right-aligned, space-padded)
}
```

8.5.2.5 Alignment

Control right, left, and center alignment explicitly with `:>N`, `:<N`, `:^N`:

```
fn main() {
    let name = "Andy";
    println!("[{name:>10}]"); // Right-aligned, width 10
    println!("[{name:<10}]"); // Left-aligned
    println!("[{name:^10}]"); // Centered
}
```

8.5.2.6 The Fill Character

Padding defaults to spaces, but another character can be specified:

```
fn main() {
    let id = 42;
    println!("{id:0>5}"); // Prints 00042 (padded with 0s)
}
```

8.5.2.7 Base Display

`:b`, `:x`, `:o` display numbers in binary, hexadecimal, and octal respectively:

```
fn main() {
    let n = 255;
    println!("{n:b}"); // 11111111
    println!("{n:x}"); // ff
    println!("{n:o}"); // 377
}
```

These formats combine — e.g. `{:0>8b}` is "binary, zero-padded to 8 digits."

8.5.2.8 Escaping Braces

To print a literal { or } in a format string, use {{ and }}:

```
fn main() {
    println!("These are braces: {{}}"); // Prints: These are braces: {}
}
```

8.5.3 Example Code

```
fn main() {
    let name = "Ming";
    let score = 87.5678;

    // The variable-capture shorthand
    println!("Student: {name}");
```

```
    println!("Score: {score}");

    // Positional parameters: reordering, reuse
    println!("{1}'s score is {0}", score, name);
    println!("{0}! {0}! {0}!", "Go");

    // Decimal precision
    println!("Rounded to two places: {score:.2}");

    // Width
    println!("[{name:10}]"); // Strings left-align by default
    let x = 42;
    println!("[{x:10}]");    // Numbers right-align by default

    // Alignment
    println!("[{name:>10}]"); // Right
    println!("[{name:<10}]"); // Left
    println!("[{name:^10}]"); // Centered

    // Zero padding
    let id = 42;
    println!("ID: {id:0>5}");

    // Base display
    let value = 255;
    println!("Decimal: {value}");
    println!("Binary: {value:b}");
    println!("Hexadecimal: {value:x}");
    println!("Octal: {value:o}");

    // Combo: zero-pad + right-align + width 2 + hexadecimal
    let byte = 10;
    println!("0x{byte:0>2x}"); // Prints 0x0a

    // Combo: zero-pad + right-align + width 8 + binary
    println!("{byte:0>8b}"); // Prints 00001010

    // Printing braces themselves takes {{ and }}
    println!("Here's a brace pair: {{}}"); // Prints: Here's a brace pair: {}
}
```

8.5.4 Recap

- `println!("{x}")` puts the variable name right in the braces — variables only, no expressions.
- `{0}`, `{1}` pick arguments by number (from 0); the same argument can be reused.
- `{:.2}` controls digits after the decimal point.
- `{:5}` sets a minimum width.
- `{:>10}`, `{:<10}`, `{:^10}` align right, left, and center.
- `{:0>5}` pads with `0` to width 5.
- `{:b}`, `{:x}`, `{:o}` display in binary, hexadecimal, octal.
- Format options combine, e.g. `{:0>8b}` = zero-pad + right + width 8 + binary.

- Print literal { and } by escaping as {{ and }}.

8.6 `structs` / `enums` inside `fns`

8.6.1 Goal of This Episode

Learn that "items" like `fn`, `struct`, and `enum` can be defined inside functions, and the fundamental ordering difference between them and `let` bindings.

> This episode supplements **Chapter 3**.

8.6.2 Concept

You're probably used to defining `structs` and `enums` outside `fn main()`, but putting them inside is perfectly legal too:

```
fn main() {
    struct Point {
        x: i32,
        y: i32,
    }
    let p = Point { x: 1, y: 2 };
    println!("{}", p.x);
}
```

This code compiles just fine.

8.6.2.1 The Limitation: Visible Only within That Function

A type defined inside a function is visible only inside that `fn`. Functions outside it can't use it. So by convention, type definitions still go outside — unless you're sure the type is used in just one `fn`.

8.6.2.2 The Important Difference: Items Aren't Order-sensitive

Here's a point many don't know. In Rust, **items** — including `fn`, `struct`, `enum`, `trait`, `impl`, and so on — **are unaffected by definition order**. They can be used before they are defined:

```
fn main() {
    let p = Point { x: 1, y: 2 }; // Used first
    println!("{}", p.x);

    struct Point {                // Defined later
        x: i32,
        y: i32,
    }
}
```

Completely unlike `let`! A `let` binding must appear before its use, or the compiler errors. But item definitions are "globally visible" (within their scope), regardless of which line they sit on.

8.6.2.3 Why Is It Like This?

Because items are static definitions settled at compile time. The compiler scans all items first, builds the full type information, and only then processes runtime statements like `let`.

8.6.3 Example Code

```rust
fn main() {
    // Called first, defined later — perfectly legal
    greet();

    // The struct used first, defined later
    let p = Point { x: 3.0, y: 4.0 };
    println!("Coordinates: ({}, {})", p.x, p.y);

    // The enum used first, defined later
    let color = Color::Red;
    describe(color);

    // These items are all defined after their use
    struct Point {
        x: f64,
        y: f64,
    }

    enum Color {
        Red,
        Green,
        Blue,
    }

    fn describe(c: Color) {
        match c {
            Color::Red => println!("Red"),
            Color::Green => println!("Green"),
            Color::Blue => println!("Blue"),
        }
    }

    fn greet() {
        println!("Hi there!");
    }

    // But let bindings must precede their use!
    // Uncommenting the following fails to compile:
    // println!("{}", not_yet);
    let not_yet = 42;
    println!("A let binding must be declared first: {}", not_yet);
}
```

8.6.4 Recap

- Items like `struct`, `enum`, and `fn` can legally be defined inside functions.
- An item defined inside an `fn` is visible only inside that `fn` (scope restriction).
- Type definitions conventionally still go outside `fn`s, unless a single `fn` uses them.
- **Items are unaffected by definition order** — they can be used either before or after their definitions.
- **`let` bindings must appear before use** — the fundamental difference between items and

`let`.

- The reason: items are compile-time static definitions; the compiler scans all items before handling runtime code.

8.7 `struct` Update Syntax

8.7.1 Goal of This Episode

Learn to build a new `struct` instance quickly from an existing one with the `..` syntax, and understand how `Copy` fields differ from moved fields.

This episode supplements **Chapter 3**.

8.7.2 Concept

Remember writing out every field when creating a `struct`? If you only want one or two fields changed with the rest as-is, spelling everything out each time is tedious. Rust provides **`struct` update syntax** — `..` "fills in the remaining fields."

8.7.2.1 Basic Syntax

```
    let p2 = Point { x: 10, ..p1 };
```

Meaning: `p2`'s `x` becomes `10`, and the remaining fields move over from `p1`.

`..p1` must go last, preceded by a comma (when other fields come before it).

8.7.2.2 The copy vs move Difference

An important detail here. `..p1` is not "`clone` the whole `struct`" — it works **field by field**:

- Fields whose type implements `Copy` (like `i32`, `f64`, `bool`) get **copied**.
- Fields whose type **lacks** `Copy` (like `String`) get **moved**.

Meaning: if `..p1` moves some of `p1`'s non-`Copy` fields, those fields can no longer be accessed through `p1` afterward.

8.7.2.3 Pairing with `Default`

If your `struct` implements the `Default` `trait`, `..` paired with its `default()` builds an instance "specifying just a few fields, defaults for the rest":

```
    let config = Config { debug: true, ..Config::default() };
```

Especially nice for `structs` with many fields.

8.7.3 Example Code

```
#[derive(Debug)]
struct Config {
    width: u32,
    height: u32,
    fullscreen: bool,
    title: String,
```

```
}

impl Default for Config {
    fn default() -> Self {
        Config {
            width: 800,
            height: 600,
            fullscreen: false,
            title: String::from("My App"),
        }
    }
}

#[derive(Debug, Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

fn main() {
    // Basic usage: changing just one field
    let p1 = Point { x: 1.0, y: 2.0 };
    let p2 = Point { x: 10.0, ..p1 };
    println!("p1 = {:?}", p1); // p1 still works, since f64 is Copy
    println!("p2 = {:?}", p2);

    // With Default: specifying only the fields you want changed
    let custom = Config {
        width: 1920,
        height: 1080,
        ..Config::default()
    };
    println!("Custom settings: {:?}", custom);

    // All defaults
    let default_config = Config { ..Config::default() };
    println!("Default settings: {:?}", default_config);

    // Watch the move semantics!
    let c1 = Config {
        width: 1024,
        height: 768,
        fullscreen: true,
        title: String::from("Game"),
    };
    let c2 = Config {
        fullscreen: false,
        ..c1 // title (String) gets moved!
    };
    // println!("{}", c1.title); // Compile error! title has been moved
    println!("c1.width = {}", c1.width);  // But the Copy fields remain usable
    println!("c2 = {:?}", c2);
}
```

8.7.4 Recap

- `let p2 = Point { x: 1, ..p1 };` fills `p2`'s remaining fields from `p1`.
- `..source` must come last.
- `Copy`-typed fields are copied; non-`Copy` fields are moved.
- If every field is `Copy`, the original `struct` stays usable.
- If a non-`Copy` field was moved, that field of the original `struct` is off-limits.
- `..Config::default()` suits "mostly defaults, a few changes" nicely.

8.8 `ref` Patterns and `match` Ergonomics

8.8.1 Goal of This Episode

Learn what the `ref` keyword does in pattern matching, and why modern Rust hardly ever needs a hand-written `ref`.

> This episode supplements **Chapter 3**.

8.8.2 Concept

This episode covers syntax you may have seen in older code but will hardly ever use in modern Rust: `ref`. Understanding its existence and mechanics helps you read other people's code.

8.8.2.1 What Is `ref`?

In a pattern, `ref` makes the bound variable a reference instead of taking ownership:

```
    let val = String::from("hello");
    let ref r = val; // r's type is &String
    // Equivalent to: let r = &val;
```

You might think: why not just write `&val`? Right — in a `let` binding, the two are fully equivalent. `ref` earns its keep mainly inside `match`.

8.8.2.2 `ref` in `match`

In the old days (before Rust 1.26), borrowing instead of moving inside a `match` required a hand-written `ref`:

```
fn main() {
    let opt = Some(String::from("hello"));
    match opt {
        Some(ref s) => println!("{}", s), // Borrowed, not moved
        None => println!("nothing"),
    }
    // opt remains usable, since we only borrowed the value inside
}
```

Without the `ref`, `s` would take the `String`'s ownership, and `opt` would be unusable afterward.

8.8.2.3 `match` Ergonomics (Rust 1.26+)

Starting with Rust 1.26, the compiler got smarter. When you `match` a **reference**, the bindings inside automatically become references:

```
fn main() {
    let opt = Some(String::from("hello"));
    match &opt {     // Note the &opt here
        Some(s) => { // s is automatically &String; no ref needed
            println!("{}", s);
        }
        None => println!("nothing"),
    }
    // opt remains usable!
}
```

This is what's called **match ergonomics**: the effect you used to have to spell out with `ref`, the compiler now gives you automatically once it sees you `match` a reference (`&opt`).

8.8.2.4 So Is `ref` Still Needed?

Hardly ever. In 99% of cases, just `match` a reference (`match &value`) and the compiler handles the rest. But when reading old code, you should at least know what a `ref` is doing.

8.8.3 Example Code

```
fn main() {
    // ===== ref basics =====
    let name = String::from("Rust");
    let ref r = name; // r: &String
    println!("The ref binding: {}", r);
    println!("The original remains usable: {}", name);

    // ===== The old style: ref in match to avoid the move =====
    let data = Some(String::from("important data"));

    match data {
        Some(ref s) => println!("Old-style borrow: {}", s),
        None => println!("Empty"),
    }
    println!("data remains: {:?}", data); // No move, thanks to ref

    // ===== The new style: match ergonomics =====
    let data2 = Some(String::from("a new world"));

    match &data2 {   // Matching a reference
        Some(s) => { // s is automatically &String
            println!("New-style borrow: {}", s);
        }
        None => println!("Empty"),
    }
    println!("data2 remains: {:?}", data2);

    // ===== A more elaborate example =====
    let pairs = vec![
        (String::from("Taipei"), 25),
        (String::from("Tokyo"), 10),
        (String::from("New York"), 5),
    ];
```

```
    // match ergonomics also makes destructuring in for loops natural
    for (city, temp) in &pairs {
        // city: &String, temp: &i32 (automatically borrowed)
        println!("{} is {} degrees", city, temp);
    }
    println!("pairs remains, {} entries in total", pairs.len());
}
```

8.8.4 Recap

- `let ref x = val;` equals `let x = &val;` — identical in a `let`.
- In a `match`, `Some(ref x)` borrows the inner value rather than moving it.
- **`match` ergonomics (Rust 1.26+)**: matching a reference makes the pattern's variables automatically references.
- Modern Rust hardly needs a manual `ref` — `match &value` suffices.
- `for (k, v) in &collection` benefits from `match` ergonomics too: `k` and `v` are automatically references.
- Knowing `ref` is mostly for reading older code.

8.9 `panic!` / `todo!` / `unimplemented!` / `unreachable!`

8.9.1 Goal of This Episode

Meet four macros that trigger a panic, and learn when each is appropriate.

> This episode is a general supplement, tied to no particular chapter.

8.9.2 Concept

You've probably noticed the `!` after names like `println!()` and `format!()`. In Rust, things with a `!` in their name are **macros** — not quite functions, though for now knowing how to use them is enough; how macros work comes later.

This episode introduces four commonly used macros that trigger a panic as soon as execution reaches them. A panic interrupts normal execution; in the programs we've written so far, an unhandled panic ends the program. All four panic, but their semantics differ — and so does the message they send to whoever reads the code.

8.9.2.1 `panic!("message")`

The most basic "something went wrong; panic now." For errors you can't handle:

```
    panic!("Something that shouldn't happen just happened!");
```

Formatted messages work: `panic!("Couldn't find id: {}", id);`

8.9.2.2 `todo!()`

"Not finished yet — placeholder for now." The development favorite: scaffold the program's structure first, fill in details later:

```rust
fn calculate_tax(income: f64) -> f64 {
    todo!() // To be implemented later
}
```

It compiles, but execution reaching it panics with “not yet implemented.”

8.9.2.3 `unimplemented!()`

“This isn’t implemented.” Similar to `todo!()`, but with different semantics — `todo!()` clearly signals “will do later,” while `unimplemented!()` makes **no promise it ever will be**. Maybe there’s no plan, no current need, or it’s a `trait`-required method meaningless for this type:

```rust
trait Foo {
    fn bar(&self) -> u8;
    fn baz(&self);
}

struct MyStruct;

impl Foo for MyStruct {
    fn bar(&self) -> u8 {
        1 + 1
    }

    fn baz(&self) {
        // baz is meaningless for MyStruct, but the trait demands a definition
        unimplemented!()
    }
}

fn main() {}
```

8.9.2.4 `unreachable!()`

“This line should never execute.” When you’re certain some logic can’t be reached, mark it:

```rust
fn main() {
    let direction = "north";
    match direction {
        "north" | "south" | "east" | "west" => println!("A valid direction"),
        _ => unreachable!("There are only four directions; this can't be reached"),
    }
}
```

If it does get reached, your assumption was wrong — and the panic surfaces that bug for you.

8.9.2.5 The Four Compared

- `panic!` — something’s wrong. For unhandleable errors.
- `todo!` — not written yet; will be implemented. A development placeholder.
- `unimplemented!` — not implemented, no promise it will be. Maybe unneeded; maybe `trait`-required but meaningless.
- `unreachable!` — shouldn’t get here. Marks logically impossible branches.

8.9.3 Example Code

```
enum Shape {
    Circle(f64),
    Rectangle(f64, f64),
    Triangle(f64, f64, f64),
}

fn area(shape: &Shape) -> f64 {
    match shape {
        Shape::Circle(r) => 3.14159 * r * r,
        Shape::Rectangle(w, h) => w * h,
        Shape::Triangle(_, _, _) => todo!("Triangle area to be implemented later"),
    }
}

fn describe_score(score: u32) -> &'static str {
    match score {
        90..=100 => "Excellent",
        80..=89 => "Good",
        70..=79 => "Fair",
        60..=69 => "Passing",
        0..=59 => "Failing",
        _ => unreachable!("Scores should be between 0-100"),
    }
}

trait Storage {
    fn save(&mut self, data: &str);
    fn load(&self) -> String;
}

struct LocalStorage;

impl Storage for LocalStorage {
    fn save(&mut self, data: &str) {
        println!("Saving locally: {}", data);
    }

    fn load(&self) -> String {
        // The trait demands a definition, but LocalStorage doesn't need this feature
        unimplemented!()
    }
}

fn main() {
    // todo! — the development placeholder
    let circle = Shape::Circle(5.0);
    println!("Circle area: {}", area(&circle));

    let rect = Shape::Rectangle(3.0, 4.0);
    println!("Rectangle area: {}", area(&rect));

    // Uncommenting the next line panics with the todo! message
    // let tri = Shape::Triangle(3.0, 4.0, 5.0);
```

```
    // println!("Triangle area: {}", area(&tri));

    // unreachable! — the branch that shouldn't be reached
    let grade = describe_score(85);
    println!("The grade for 85 points: {}", grade);

    // unimplemented! — the unimplemented feature
    let mut storage = LocalStorage;
    storage.save("hello");
    // storage.load(); // Uncommenting panics: not implemented

    // panic! — trigger a panic
    // panic!("Panicking on purpose!");
    println!("The program ended normally");
}
```

8.9.4 Recap

- `panic!("msg")` is the basic way to trigger a panic, for unhandleable errors.
- `todo!()` is the development placeholder, clearly saying "will implement later."
- `unimplemented!()` says "not implemented," promising nothing — maybe unneeded, maybe `trait`-required but meaningless for the type.
- `unreachable!()` marks logically unreachable code paths.
- All four panic; the difference lies in the **intent** conveyed — picking the right one makes code more expressive.

8.10 `let` Chains

8.10.1 Goal of This Episode

Meet `let` chains — stringing together multiple `let`s and boolean conditions with `&&` inside `if` and `while` conditions.

> This episode supplements **Chapter 3**.

8.10.2 Concept

8.10.2.1 The Problem: Nested `if let`

Chapter 3 taught `if let`. But needing several pattern matches in a row means nested `if let`s:

```
enum Wrapper {
    Value(i32),
    Empty,
}

fn get_a() -> Wrapper { Wrapper::Value(10) }
fn get_b(x: i32) -> Wrapper { Wrapper::Value(x + 1) }

fn main() {
    if let Wrapper::Value(a) = get_a() {
        if a > 0 {
            if let Wrapper::Value(b) = get_b(a) {
```

```
                println!("a = {}, b = {}", a, b);
            }
        }
    }
}
```

Every extra condition adds a level of indentation, and the code sinks deeper and deeper.

8.10.2.2 `let` Chains Flatten It

You can string multiple `let`s and boolean conditions into one `if` with &&:

```
enum Wrapper {
    Value(i32),
    Empty,
}

fn get_a() -> Wrapper { Wrapper::Value(10) }
fn get_b(x: i32) -> Wrapper { Wrapper::Value(x + 1) }

fn main() {
    if let Wrapper::Value(a) = get_a()
        && a > 0
        && let Wrapper::Value(b) = get_b(a)
    {
        println!("a = {}, b = {}", a, b);
    }
}
```

The &&-chained conditions are checked left to right in order. Variables bound by earlier `let`s are usable in later conditions (like a above). If any condition fails, the rest don't run — just like &&'s short-circuiting.

8.10.2.3 Works in `while` Too

```
while let Some(item) = next_item()
    && item.value > 0
{
    // ...
}
```

8.10.3 Example Code

```
enum Command {
    Run { speed: i32 },
    Stop,
}

fn get_command() -> Command {
    Command::Run { speed: 5 }
}

fn get_boost() -> Command {
    Command::Run { speed: 3 }
}
```

```rust
fn main() {
    // The nested style
    if let Command::Run { speed: s } = get_command() {
        if s > 0 {
            if let Command::Run { speed: boost } = get_boost() {
                println!("Nested: speed {} + boost {} = {}", s, boost, s + boost);
            }
        }
    }

    // The let-chains style — same logic, flatter
    if let Command::Run { speed: s } = get_command()
        && s > 0
        && let Command::Run { speed: boost } = get_boost()
    {
        println!("Flat: speed {} + boost {} = {}", s, boost, s + boost);
    }
}
```

8.10.4 Recap

- `let` chains string multiple `let`s and boolean conditions together with `&&` inside `if` and `while`.
- They replace nested `if let`, flattening the code.
- Variables bound earlier are usable later.
- Consistent with `&&` short-circuiting: an earlier failure stops the rest.

8.11 `Rc` Cycles and `Weak`

8.11.1 Goal of This Episode

Understand how `Rc` reference cycles leak memory, and learn to break cycles with `Weak`.

> This episode supplements **Chapter 5**.

8.11.2 Concept

Remember `Rc<T>` from Chapter 5? It manages memory through reference counting — each extra `Rc` pointing at the same data adds one to the count; each fewer subtracts one; hitting zero releases the memory.

Sounds perfect, but it has one fatal weakness: the **reference cycle**.

8.11.2.1 What's a Reference Cycle?

Picture two nodes A and B: A holds an `Rc` to B, and B holds an `Rc` to A. When the outside no longer holds either:

1. A's external `Rc` gets `dropped` → A's count decrements, but B still points at A → the count isn't zero → A isn't released.
2. B's external `Rc` gets `dropped` → B's count decrements, but A still points at B → the count isn't zero → B isn't released.

Result: A and B are **never released** — a memory leak. A ring invisible from outside, holding itself up — that's the essence of the cycle problem.

What makes a leak awkward is that it has no visible symptom: nothing panics, nothing fails to compile, that memory simply never comes back. So let's make it visible with `Drop` — give the node a destructor that prints, and a message that never appears means a value that was never released:

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    name: &'static str,
    other: Option<Rc<RefCell<Node>>>,
}

impl Drop for Node {
    fn drop(&mut self) {
        println!("{} released", self.name);
    }
}

fn main() {
    {
        let a = Rc::new(RefCell::new(Node { name: "A", other: None }));
        let b = Rc::new(RefCell::new(Node { name: "B", other: None }));

        a.borrow_mut().other = Some(b.clone());
        b.borrow_mut().other = Some(a.clone());

        println!("before leaving the scope, A's strong count = {}", Rc::strong_count(&a));
    }

    println!("the scope has been left");
}
```

The output is only two lines:

```
before leaving the scope, A's strong count = 2
the scope has been left
```

That `strong count = 2` is exactly the "B still points at A" from above, and neither "released" message shows up — the variables `a` and `b` are gone, yet the nodes they pointed at are still holding each other up. Comment out the `b.borrow_mut().other = ...` line and run it again: the ring is broken, and both messages appear.

8.11.2.2 What Is `Weak`

`Weak<T>` is a "weak reference" — it points at the same data but **doesn't increase the strong count**.

```
    let strong = Rc::new(42);
    let weak: Weak<i32> = Rc::downgrade(&strong);
```

`Rc::downgrade` demotes an `Rc` to a `Weak`. Internally, `Rc` keeps **two** counters: the strong count

and the weak count. `.clone()` bumps the strong count; `Rc::downgrade()` bumps only the weak count. `Rc` decides "release or not" solely on the strong count — hitting zero releases, whatever the weak count says.

Since data a `Weak` points at may already be gone, direct access isn't allowed. You must `.upgrade()` first:

```
use std::rc::{Rc, Weak};

fn main() {
    let strong = Rc::new(42);
    let weak: Weak<i32> = Rc::downgrade(&strong);

    match weak.upgrade() {
        Some(rc) => println!("Still here: {}", rc),
        None => println!("Already released"),
    }
}
```

`upgrade` returns `Option<Rc<T>>` — an `Rc` if the data survives, `None` if it's been released.

8.11.2.3 Breaking Cycles with `Weak`

Back to the example. The crux: the graph formed by strong counts contains a ring. Flip one direction to `Weak`, and the strong-count graph has no ring — `Weak` contributes nothing to strong counts.

A concrete illustration: suppose we're building a **doubly linked list** — each node points to both its predecessor and successor, so walking head-to-tail or tail-to-head is easy. With `Rc` in both directions, adjacent nodes form cycles.

The fix: `next` (forward) uses `Rc`; `prev` (backward) uses `Weak`:

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node<T> {
    value: T,
    next: Option<Rc<RefCell<Node<T>>>>,
    prev: Option<Weak<RefCell<Node<T>>>>,
}
```

Why does this avoid cycles? Look at the strong-count graph:

```
outside ──Rc──→ A ──Rc──→ B ──Rc──→ C
                ←─ • Weak • ←─    ←─ • Weak • ←─
```

The `Weak` edges don't count toward strong counts. The strong-count graph has only the left-to-right arrows — a chain, no ring.

The outside releases A → A's strong count hits zero → A is `dropped` → A's `next` gets `dropped` along with it → B's strong count hits zero → B is `dropped` → … a chain reaction all the way down. No node gets propped up by a `prev`, because `prev` is `Weak` and contributes no strong count.

8.11.2.4 Do `Rc`s from `upgrade` Cause Problems?

You might wonder: "If I `upgrade` a `Weak`, get an `Rc`, and hold onto it, isn't that one more strong count?"

Correct — an `upgrade`d `Rc` does add one to the strong count. But that `Rc` is an **independent variable** — its strong-count contribution is charged to "the variable holding that `Rc`," not to the original `Weak` field. The `Weak` field's contribution to the strong count is forever 0.

The cycle question was settled — or not — the moment the data structure was built; how you `upgrade` afterward is entirely beside the point.

8.11.3 Example Code

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node<T> {
    value: T,
    next: Option<Rc<RefCell<Node<T>>>>,
    prev: Option<Weak<RefCell<Node<T>>>>,
}

impl<T> Node<T> {
    fn new(value: T) -> Rc<RefCell<Node<T>>> {
        Rc::new(RefCell::new(Node { value, next: None, prev: None }))
    }
}

/// Attach b after a
fn link<T>(a: &Rc<RefCell<Node<T>>>, b: &Rc<RefCell<Node<T>>>) {
    a.borrow_mut().next = Some(b.clone());
    b.borrow_mut().prev = Some(Rc::downgrade(a));
}

fn main() {
    let a = Node::new(1);
    let b = Node::new(2);
    let c = Node::new(3);

    link(&a, &b);
    link(&b, &c);

    // Walking forward (via Rc)
    print!("Walking forward: ");
    let mut current = Some(a.clone());
    while let Some(node) = current {
        print!("{} ", node.borrow().value);
        // next is Option<Rc<...>>; as_ref gives Option<&Rc<...>>, then map clones a new Rc
        current = node.borrow().next.as_ref().map(|rc| rc.clone());
    }
    println!();

    // Walking backward (via Weak; upgrade needed)
    print!("Walking backward: ");
```

```
    let mut current = Some(c.clone());
    while let Some(node) = current {
        print!("{} ", node.borrow().value);
        current = node.borrow().prev.as_ref().and_then(|w| w.upgrade());
    }
    println!();

    // Checking the counts
    // Strong counts are charged to the node pointed at:
    // a.next points at b → b's strong +1; b.next points at c → c's strong +1
    // Weak counts are charged to the node pointed at too:
    // b.prev points at a → a's weak +1; c.prev points at b → b's weak +1
    // a: strong=1 (variable a), weak=1 (b.prev)
    // b: strong=2 (variable b + a.next), weak=1 (c.prev)
    // c: strong=2 (variable c + b.next), weak=0 (no node's prev points at c)
    println!("a strong={}, weak={}", Rc::strong_count(&a), Rc::weak_count(&a));
    println!("b strong={}, weak={}", Rc::strong_count(&b), Rc::weak_count(&b));
    println!("c strong={}, weak={}", Rc::strong_count(&c), Rc::weak_count(&c));
}
```

8.11.4 Recap

- `Rc` reference cycles leak memory — the strong count can never hit zero.
- `Weak` doesn't raise the strong count, so it never blocks a release.
- `Rc::downgrade(&rc)` creates a `Weak<T>`; `weak.upgrade()` returns `Option<Rc<T>>`.
- Breaking cycles with `Weak`: keep the strong-count graph ring-free.
- The doubly-linked-list recipe: `next` uses `Rc` (owning the successor); `prev` uses `Weak` (observing the predecessor).
- A `Weak` field's contribution to the strong count is forever 0; `upgrade`d `Rc`s are independent variables.
- `Rc::strong_count()` and `Rc::weak_count()` inspect the current counts.

8.12 Fully Qualified Syntax

8.12.1 Goal of This Episode

Learn the three levels of method-call syntax, and how to disambiguate when `trait` method names collide.

> This episode supplements **Chapter 5**.

8.12.2 Concept

In Rust, calling a method actually has three notations, from simple to complete:

8.12.2.1 The First: Method Syntax

```
    dog.speak();
```

The most common form. The compiler finds the matching method automatically.

8.12.2.2 The Second: Naming the `trait` or Type

```
    Animal::speak(&dog);
```

Explicitly telling the compiler "I'm calling the `speak` on the `Animal` `trait`." The `&dog` is what would have been `&self`.

8.12.2.3 The Third: Fully Qualified Syntax

```
    <Dog as Animal>::speak(&dog);
```

The most explicit form: "On the `Animal` `trait` as implemented by `Dog`, call the `speak` method, passing `&dog`."

8.12.2.4 When Is It Needed?

Mostly the first form suffices. But when **several `traits` define same-named methods**, the compiler can't tell which you mean, and more explicit syntax is required:

```
trait Animal {
    fn name(&self) -> &str;
}

trait Robot {
    fn name(&self) -> &str;
}
```

If some type implements both `Animal` and `Robot`, calling `.name()` errors. That's when the second or third form disambiguates.

8.12.2.5 Associated Functions Need It More Often

For **associated functions** without a `self` parameter, there's no receiver for the compiler to infer from, so fully qualified syntax is needed more often:

```
    // When several traits have the create() associated function
    let x = <MyType as TraitA>::create();
```

8.12.2.6 Accessing Associated Types

Fully qualified syntax also reaches a type's **associated type** on a particular `trait`:

```
// The IntoIterator trait has an associated type named Item
// Fully qualified syntax retrieves its concrete type:
type MyItem = <Vec<i32> as IntoIterator>::Item; // i32
```

Some places allow plain `Type::TypeName`, but under ambiguity or failed inference, fully qualified syntax makes the type explicit.

8.12.3 Example Code

```
trait Animal {
    fn speak(&self);
    fn category() -> &'static str;
}
```

```rust
trait Robot {
    fn speak(&self);
    fn category() -> &'static str;
}

struct CyberDog {
    name: String,
}

impl Animal for CyberDog {
    fn speak(&self) {
        println!("{} goes woof! (animal)", self.name);
    }

    fn category() -> &'static str {
        "mammal"
    }
}

impl Robot for CyberDog {
    fn speak(&self) {
        println!("{} goes beep! (robot)", self.name);
    }

    fn category() -> &'static str {
        "artificial intelligence"
    }
}

// CyberDog has a speak of its own too
impl CyberDog {
    fn speak(&self) {
        println!("{} goes woof-beep! (itself)", self.name);
    }
}

fn main() {
    let dog = CyberDog {
        name: String::from("Snowy"),
    };

    // Level one: method syntax — the type's own method wins
    dog.speak(); // "Snowy goes woof-beep! (itself)"

    // Level two: naming the trait
    Animal::speak(&dog); // "Snowy goes woof! (animal)"
    Robot::speak(&dog);  // "Snowy goes beep! (robot)"

    // Level three: fully qualified syntax
    <CyberDog as Animal>::speak(&dog); // "Snowy goes woof! (animal)"
    <CyberDog as Robot>::speak(&dog);  // "Snowy goes beep! (robot)"

    // Associated functions (no self) — fully qualified syntax needed all the more
    // Animal::category(); // Compile error! The compiler doesn't know whose implementation
    let animal_cat = <CyberDog as Animal>::category();
```

```
    let robot_cat = <CyberDog as Robot>::category();
    println!("Animal category: {}", animal_cat);
    println!("Robot category: {}", robot_cat);

    // Accessing an associated type
    // Vec<i32> implements IntoIterator, whose Item is i32
    // Fully qualified syntax retrieves the associated type:
    let _: <Vec<i32> as IntoIterator>::Item = 42; // The type is i32
    println!("Vec<i32>'s IntoIterator::Item is i32");
}
```

8.12.4 Recap

- Method calls have three levels: `object.method()` → `Trait::method(&object)` → `<Type as Trait>::method(&object)`.
- Use the simplest; escalate only on conflict.
- With same-named methods across `traits`, name whose version you're calling.
- Associated functions (no `self`) need fully qualified syntax more often.
- The fully-qualified format: `<Type as Trait>::function(args)`.
- It also reaches associated types: `<Type as Trait>::TypeName`.

8.13 A Brief Introduction to DSTs

8.13.1 Goal of This Episode

Understand what dynamically sized types (DSTs) are, and what `Sized` and `?Sized` mean in generics.

> This episode supplements **Chapter 5**.

8.13.2 Concept

(The pointer sizes mentioned in this episode assume a 64-bit system — as nearly all computers now are.)

In Rust's type system, most types have compile-time-known sizes — `i32` is 4 bytes, `bool` 1 byte, `(i32, i32)` 8 bytes. But some types' sizes are **unknown at compile time** — these are **DSTs (Dynamically Sized Types)**.

8.13.2.1 The Common DSTs

You've actually met them already:

- **`str`**: the "content" type of string slices. `"hello"` is 5 bytes, `"哈囉"` is 6 — no fixed length.
- **`[T]`**: the "content" type of array slices. A `[i32]` might have 3 elements or 100.

Because their sizes aren't fixed, you **can't** use them directly as values:

```
fn main() {
    let s: str = "hello";       // Compile error!
    let arr: [i32] = [1, 2, 3]; // Compile error!
}
```

8.13.2.2 How to Use Them? Through Pointers!

A DST must hide behind some kind of pointer:

- `&str`, `&[T]` — references
- `Box<str>`, `Box<[T]>` — pointers to the heap

Pointers to `str` and `[T]` are **fat pointers** — storing an address and a length:

```
Ordinary pointer: [address]         (8 bytes)
Fat pointer:      [address][length] (16 bytes)
```

So an `&str` actually occupies 16 bytes: 8 pointing at the string data, 8 recording the length.

8.13.2.3 The `Sized` trait

Rust has a special `trait` called `Sized`, meaning "this type's size is known at compile time." **The vast majority of types implement `Sized` automatically.**

Furthermore — and many don't know this — **generic parameters carry a default `Sized` bound**:

```rust
fn print_it<T>(val: T) { ... }
// Is actually equivalent to
fn print_it<T: Sized>(val: T) { ... }
```

Sensible, since if T's size were unknown, the function couldn't know how much stack space to allocate.

8.13.2.4 `?Sized`: Loosening the Restriction

Sometimes you want a generic parameter to accept DSTs. That's when `?Sized` loosens the bound:

```rust
fn print_it<T: ?Sized>(val: &T) { ... }
//                     ^^^^^^^ Note: must go through a reference
```

`?Sized` means "T may be `Sized`, or may not." Since the size may be unknown, T is usually usable only through references or smart pointers.

8.13.2.5 `Self` in a `trait` Defaults to `?Sized`

We said generic parameters T default to a `Sized` bound. But a `trait`'s `Self` is the exception — it defaults to `?Sized`; that is, `Self` needn't be `Sized`.

Remember `Clone` from Chapter 4 Episode 3? Its method is `fn clone(&self) -> Self` — returning `Self` outright. To restrict this operation to `Sized` types, the bound can be placed on the whole `trait` or just on the method with `where Self: Sized`. `Clone` places it on the whole `trait`:

```rust
trait Clone: Sized {
    fn clone(&self) -> Self;
}
```

8.13.2.6 Looking Back at Chapter 5's `Cow`

When Chapter 5's last episode taught `Cow`, we used a simplified definition too:

```rust
// The simplified version from Chapter 5
pub enum Cow<'a, B>
where
    B: 'a + ToOwned,
{
    Borrowed(&'a B),
    Owned(B::Owned),
}
```

If you'd tried putting `str` or `[T]` into that `Cow` — writing `Cow<'_, str>`, say — it wouldn't compile. The generic parameter `B` demands `Sized` by default, and `str` isn't `Sized`.

Adding `?Sized` fixes it:

```rust
pub enum Cow<'a, B>
where
    B: 'a + ToOwned + ?Sized,
{
    Borrowed(&'a B),
    Owned(B::Owned),
}
```

The `B` in `Borrowed(&'a B)` already sits behind a reference, so `B` being a DST is fine — the fat pointer takes care of it.

8.13.2.7 `&mut [T]` and `&mut str`

DSTs can take mutable references too. `&mut [T]` is quite useful — you can modify the slice's elements:

```rust
    let mut arr = [1, 2, 3, 4, 5];
    let slice: &mut [i32] = &mut arr[1..4];
    slice[0] = 99;  // arr becomes [1, 99, 3, 4, 5]
```

But `&mut str` is nearly useless. Syntactically legal, yet there's almost nothing you can do with it. The reasons:

First, `&mut str`, like `&mut [T]`, can't change the length. `str` is a DST; `&mut str` is a fat pointer (address + length), the length being part of the reference. An `&mut str` is only a borrow — you don't own that memory's allocation, so you can't grow or shrink it. Changing length requires the memory-owning `String`.

Second, even changing the contents is restricted. In UTF-8, one character may take 1~4 bytes:

- `'a'` → 1 byte
- `'é'` → 2 bytes
- `'哈'` → 3 bytes

Suppose you have `"哈囉"` (6 bytes) and want to change `'哈'` into `'a'` — `'a'` is 1 byte while `'哈'` occupies 3; in-place replacement is impossible with mismatched lengths. Forcing the first byte changed without handling the rest breaks the UTF-8 multi-byte sequence. And Rust's `str` guarantees its content is always valid UTF-8 — violating that guarantee causes undefined behavior.

Hence the standard library's methods on `&mut str` are pitifully few — basically just `make_ascii_uppercase()` and `make_ascii_lowercase()`, operations that "never change byte length" (ASCII case conversion happens to be 1 byte for 1 byte). For string modification, stick with `String`.

8.13.2.8 DSTs and `Deref`

Chapter 5 also introduced the `Deref` `trait`. `String` and `Vec<T>` implement `Deref` too, and their `Deref` targets are exactly DSTs:

- `String` implements `Deref`; `Deref::deref(&String)` returns `&str`.
- `Vec<T>` implements `Deref`; `Deref::deref(&Vec<T>)` returns `&[T]`.

That is, `String`'s target is `str`, and `Vec<T>`'s target is `[T]`. DSTs can't live in variables directly, but `deref` coercion happens at the **reference level**: `&String` becomes `&str`, `&Vec<T>` becomes `&[T]`. The result of the conversion is a fat pointer carrying address and length — no need to know the DST's actual size.

That's why a function accepting `&str` takes an `&String` directly, and one accepting `&[T]` takes an `&Vec<T>` — the mechanism underneath is exactly DSTs + `Deref` combined.

8.13.2.9 Pointers Still Fuzzy?

If concepts like "pointer," "fat pointer," and "address" remain hazy, don't worry — the next chapter's first episode formally introduces what pointers really are.

8.13.3 Example Code

```rust
use std::fmt::Display;

// The default: T must be Sized
fn print_sized<T: Display>(val: T) {
    println!("A Sized value: {}", val);
}

// Loosened: T may be a DST, but must come through a reference
fn print_unsized<T: Display + ?Sized>(val: &T) {
    println!("Possibly a DST: {}", val);
}

// Showing fat pointer sizes on a 64-bit machine
fn show_pointer_sizes() {
    use std::mem::size_of;

    println!("--- Pointer size comparison ---");
    println!("&i32     = {} bytes", size_of::<&i32>());     // 8
    println!("&[i32]   = {} bytes", size_of::<&[i32]>());   // 16 (fat pointer)
    println!("&str     = {} bytes", size_of::<&str>());     // 16 (fat pointer)
    println!("Box<i32> = {} bytes", size_of::<Box<i32>>()); // 8
    println!("Box<str> = {} bytes", size_of::<Box<str>>()); // 16 (fat pointer)
}

fn main() {
    // Sized values: ordinary types
```

```
    print_sized(42);
    print_sized(String::from("hello"));

    // ?Sized: accepts &str (str being a DST)
    print_unsized("hello");                // T = str (a DST)
    print_unsized(&42);                    // T = i32 (Sized works too)
    print_unsized(&String::from("world")); // T = String (Sized)

    // &str and &[T] are fat pointers
    show_pointer_sizes();

    // str and [T] can't be used directly as values
    // let s: str = *"hello";    // Compile error!
    // let a: [i32] = *&[1,2,3]; // Compile error!

    // But through references, no problem
    let s: &str = "hello";
    let a: &[i32] = &[1, 2, 3];
    println!("\n&str = {}", s);
    println!("&[i32] length = {}", a.len());

    // Box<str> works as well
    let boxed: Box<str> = String::from("boxed string").into_boxed_str();
    println!("Box<str> = {}", boxed);
}
```

8.13.4 Recap

- **DSTs (Dynamically Sized Types)**: types with compile-time-unknown sizes, like `str` and `[T]`.
- DSTs can't be used directly as values; they need pointers: `&str`, `&[T]`, `Box<str>`, etc.
- Pointers to `str` and `[T]` are **fat pointers** containing an address and a length; they occupy 16 bytes on 64-bit machines.
- **`Sized`**: the type's size is compile-time known; generic parameters default to the `T: Sized` bound.
- **`?Sized`**: loosens the bound so generics can accept DSTs.
- A `trait`'s `Self` defaults to `?Sized`. `Clone` adds `Sized` to the whole `trait`; a method can instead add `where Self: Sized` so the restriction applies only to that method.
- The `B: ?Sized` in `Cow<'a, B>` exists precisely so `B` can be a DST like `str` or `[T]`.
- `String`'s and `Vec<T>`'s `Deref` targets are the DSTs `str` and `[T]`; `deref` coercion makes `&String` → `&str` and `&Vec<T>` → `&[T]` possible.

8.14 `extern crate`

8.14.1 Goal of This Episode

Understand the difference between `extern crate` and `use`, and why some examples in this tutorial still contain `extern crate`.

This episode supplements **Chapter 7**.

8.14.2 Main Text

Chapter 7 introduced adding external crates with Cargo. For example, to use rand, first run:

```
cargo add rand
```

Cargo adds rand to Cargo.toml. In newer Rust editions, that is enough to use it directly in your program:

```
use rand::RngExt;

fn main() {
    let mut rng = rand::rng();
    let n = rng.random_range(1..=100);
    println!("{}", n);
}
```

An ordinary Cargo project does not need an additional extern crate rand; line.

8.14.2.1 What Does extern crate Do?

extern crate explicitly tells the compiler to load an external crate:

```
extern crate rand;
```

It does not download or install rand, however, and it cannot replace the dependency in Cargo.toml. Cargo or another build tool must still make the external crate available first.

8.14.2.2 extern crate Is Not the Same as use

These two lines do different jobs:

```
extern crate rand;

use rand::RngExt;
```

- extern crate rand; explicitly loads the external crate named rand.
- use rand::RngExt; brings the RngExt trait from rand into the current scope.

In other words, extern crate deals with the external crate itself, while use deals with how names are used in the program.

8.14.2.3 Aliasing an External crate with as

extern crate can also give the external crate a name to use in the current scope:

```
extern crate rand as random;
```

The general form is:

```
extern crate a as b;
```

Here:

- a is the name of the external crate.
- b is the name introduced into the current scope by this declaration.

For example, after aliasing rand as random, you can use it through the name random:

```rust
extern crate rand as random;

use random::RngExt;

fn main() {
    let mut rng = random::rng();
    let n = rng.random_range(1..=100);
    println!("{}", n);
}
```

In an ordinary Cargo project using a newer Rust edition, however, if you only want an alias, you can usually use the `use ... as ...` syntax introduced in Chapter 7:

```rust
use rand as random;
```

Although both forms use `as`, they still serve different purposes: `extern crate rand as random;` explicitly loads the external `crate` and introduces the name `random`, while `use rand as random;` aliases a `crate` that is already available.

8.14.2.4 Why Does This Tutorial Still Use It?

You may already have seen this in the tutorial's examples:

```rust
extern crate rand;
```

That does not mean ordinary Cargo projects written with newer Rust editions still require it. **This tutorial includes `extern crate` so that its internal tests pass.**

The tutorial uses `mdbook test` to compile and test the Rust code in the book automatically. Before running the tests, it compiles the external `crates` required by the examples, then uses `-L` to tell the test tool where to find the compiled output.

However, `-L` only provides a search path. Unlike Cargo, it does not pass complete information for every dependency. The examples therefore use `extern crate rand;` to tell the compiler explicitly to load `rand`, allowing the internal tests to find and use it.

This is a special requirement of the tutorial's testing setup, not the usual style in newer Rust editions. If you copy an example into your own Cargo project and have already added the dependency with `cargo add rand`, you can usually remove `extern crate rand;`.

8.14.3 Recap

- `extern crate name;` explicitly tells the compiler to load an external `crate`.
- `extern crate` does not download a package and cannot replace the dependency in `Cargo.toml`.
- `extern crate` and `use` have different purposes: the former deals with an external `crate`, while the latter brings names into scope.
- `extern crate a as b;` explicitly loads external `crate a` and introduces it as `b` in the current scope; in newer Rust editions, use `use a as b;` when only an alias is needed.
- Ordinary Cargo projects using newer Rust editions usually do not need `extern crate`.
- This tutorial includes `extern crate` so that its internal tests using `mdbook test -L` can find external `crates`.

Part II

Chapter 9

Multithreading

This chapter covers using multiple `Threads` to write programs that run several tasks at once.

9.1 Pointers

9.1.1 Goal of This Episode

Understand the concept of memory addresses, and what a pointer is at the low level.

9.1.2 Concept

In earlier chapters, using `&T`, `Box<T>`, and `Rc<T>`, we cared about "who owns the data" and "who's borrowing." This episode switches angles — what are these things actually in memory?

The DST introduction in Appendix I touched on DSTs and fat pointers. If that felt hazy at the time, that's normal — we hadn't formally introduced pointers yet. This episode fills in that foundation.

> This episode uses a simplified model of memory and addresses. What compilers and hardware do during actual execution is much more complex, but we will not go into those details here.

9.1.2.1 Memory Addresses

While a program runs, every variable sits somewhere in memory, and every location has a number — its **address**. What &x obtains is x's address. The `{:p}` format prints it out for inspection:

```
fn main() {
    let x: i32 = 42;
    println!("{:p}", &x); // e.g. 0x7ffd5e8a3b4c
}
```

That hexadecimal number is x's address in memory.

9.1.2.2 The True Face of `&T`

The value &x produces is, generally speaking, x's memory address. What the &T type stores is, at bottom, that address-number. When you pass &x into a function, what's passed isn't x's contents — it's x's address.

9.1.2.3 Pointer Sizes

In most cases, an &T occupies 8 bytes — the size of one address on a 64-bit system. Verify with `std::mem::size_of`:

```rust
use std::mem::size_of;

fn main() {
    println!("{}", size_of::<i32>());          // 4
    println!("{}", size_of::<[i32; 1000]>()); // 4000
    println!("{}", size_of::<&i32>());         // 8
    println!("{}", size_of::<&[i32; 1000]>()); // 8
    println!("{}", size_of::<Box<i32>>());     // 8
}
```

The `&T` and `Box<T>` values above all point to `Sized` types. Under this condition, they are the same size because they only store addresses. Data an `&T` points at may be on the stack or the heap, while an owning `Box<T>` always points into the heap. Wherever they point, the address itself is one size. So when `T` is large, passing an address is lighter than copying the whole `T` — at the cost of an extra layer of indirection on every access.

9.1.2.4 Dereferencing

With an address in hand, what can we do? The `*` operator **dereferences**, fetching the contents at the address:

```rust
fn main() {
    let x = 42;
    let r = &x;
    println!("{}", *r); // Fetching the value via the address: 42
}
```

Dereferencing isn't free. Most of the time the cost is tiny, but knowing it exists is worthwhile.

9.1.2.5 Fat Pointers

The DST introduction in Appendix I explained that `[T]` and `str` are types of indeterminate size, unable to sit directly in variables, usually handled through `&[T]`, `&str`, `Box<[T]>`, and the like. But with no fixed size, an address alone isn't enough. Picture it: you're handed an address and told a contiguous run of `i32` data starts there — but where does it end? Memory itself won't say; an address is only a starting point. So besides the address, a length must also be recorded to know how far the data extends. Hence `&[T]` and `&str` occupy 16 bytes:

```rust
use std::mem::size_of;

fn main() {
    println!("{}", size_of::<&i32>());   // 8 (address)
    println!("{}", size_of::<&[i32]>()); // 16 (address + length)
    println!("{}", size_of::<&str>());   // 16 (address + length)
}
```

9.1.3 Example Code

```rust
use std::mem::size_of;

fn main() {
    let x: i32 = 42;
    let r: &i32 = &x;
```

```
    // Printing the address
    println!("x's address: {:p}", &x);
    println!("The value r stores: {:p}", r); // Same as above

    // Dereferencing
    println!("x's value obtained through r: {}", *r);

    // Smart pointers dereference too
    let b = Box::new(99);
    println!("The value in the Box: {}", *b);

    // Pointer sizes
    println!("--- Ordinary pointers ---");
    println!("i32 size: {} bytes", size_of::<i32>());
    println!("&i32 size: {} bytes", size_of::<&i32>());
    println!("[i32; 1000] size: {} bytes", size_of::<[i32; 1000]>());
    println!("&[i32; 1000] size: {} bytes", size_of::<&[i32; 1000]>());
    println!("Box<i32> size: {} bytes", size_of::<Box<i32>>());

    // Fat pointers
    println!("--- Fat pointers ---");
    println!("&[i32] size: {} bytes", size_of::<&[i32]>());
    println!("&str size: {} bytes", size_of::<&str>());
    println!("Box<[i32]> size: {} bytes", size_of::<Box<[i32]>>());
}
```

9.1.4 Recap

- At the low level, `&T` is a memory address — essentially a number.
- On 64-bit systems, in most cases `&T` and `Box<T>` are 8 bytes — one address's size.
- `*` dereferences, fetching the contents at an address, with one layer of indirection as its cost.
- `&[T]` and `&str` are fat pointers, occupying 16 bytes (address + length), since DSTs have no fixed size.

9.2 `thread::spawn`

9.2.1 Goal of This Episode

Learn to create `Threads`, letting a program do several things at once.

9.2.2 Concept

Until now, our programs have had a single flow of execution, doing one thing at a time. But sometimes you want a program doing several things **at once** — downloading a file while updating a progress bar, say. That's what **`Threads`** are for.

9.2.2.1 Creating a `Thread`

`std::thread::spawn` takes a closure and runs it on a new `Thread`:

```rust
use std::thread;

fn main() {
    thread::spawn(|| {
        println!("I'm on another thread!");
    });
}
```

9.2.2.2 No join, No Survival

Something important: when the main function ends, the whole program ends — whether or not other Threads have finished.

```rust
use std::thread;

fn main() {
    thread::spawn(|| {
        for i in 0..10 {
            println!("Child thread: {}", i);
        }
    });

    println!("main is done");
    // The child thread may have printed only part — or nothing at all
}
```

9.2.2.3 JoinHandle

thread::spawn returns a JoinHandle. Calling .join() waits for that Thread to finish:

```rust
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        for i in 0..5 {
            println!("Child thread: {}", i);
        }
    });

    handle.join().expect("thread panicked"); // Wait for the child thread
    println!("All done");
}
```

.join() isn't only waiting — it also retrieves the closure's return value. Whatever the closure returns, .join().expect("thread panicked") receives:

```rust
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        let answer = 21 * 2;
        answer // The closure's return value
    });

    let result = handle.join().expect("thread panicked");
    println!("The result received from the other thread: {}", result); // 42
```

```
}
```

The simplest way to pass a computation result back from another `Thread`.

9.2.2.4 `move` Closures

Using outside variables in the closure generally requires `move`:

```
use std::thread;

fn main() {
    let name = String::from("Rust");

    let handle = thread::spawn(move || {
        println!("Hello, {}!", name);
    });

    println!("{}", name); // Compile error! name was moved into the closure

    handle.join().expect("thread panicked");
}
```

Why is `move` needed? Because the new `Thread` may outlive the function that called `spawn`. If the closure merely borrowed `name`, and that function ended first, discarding `name`, the closure would be left holding a dangling reference. With `move`, `name`'s ownership travels into the closure, and however the original scope ends, the closure keeps its `name`.

9.2.2.5 Interleaved Output

When several `Threads` run at once, their output interleaves — differently on each run, perhaps:

```
use std::thread;

fn main() {
    let h1 = thread::spawn(|| {
        for _ in 0..5 {
            println!("AAA");
        }
    });

    let h2 = thread::spawn(|| {
        for _ in 0..5 {
            println!("BBB");
        }
    });

    h1.join().expect("thread panicked");
    h2.join().expect("thread panicked");
}
```

Run it a few times and you'll see AAA and BBB in varying orders. That's the nature of multithreading — execution order is nondeterministic.

9.2.3 Example Code

```rust
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4, 5];

    let handle = thread::spawn(move || {
        let sum: i32 = data.iter().sum();
        println!("The sum the child thread computed: {}", sum);
        sum
    });

    // data has been moved; unusable here
    // println!("{:?}", data); // Compile error

    let result = handle.join().expect("thread panicked");
    println!("The main thread received the result: {}", result);
}
```

9.2.4 Recap

- `thread::spawn(|| { ... })` creates a new `Thread`.
- All `Threads` die when `main` ends; wait for a `Thread` with `.join()`.
- `.join()` also retrieves the closure's return value — the simplest way to pass results back.
- Closures using outside variables generally need `move`, since the new `Thread`'s lifespan is uncertain.
- Execution order across `Threads` is nondeterministic; output may interleave.

9.3 Send / Sync

9.3.1 Goal of This Episode

Understand how Rust guarantees at compile time that types are safe to use across `Threads`.

9.3.2 Concept

9.3.2.1 Why Extra Protection Is Needed

Remember the keychain analogy that opened Chapter 4? By now you can see that the key is a pointer.

One reason Rust has ownership rules and borrowing rules — no two `&muts` at once, for example — is to keep one address's value from being read and written simultaneously, causing the **data race** mentioned before. A concrete example: suppose an `i32` holds 0, and `Threads` A and B each add 1 to it through pointers. You expect 2, but reality might go:

1. `Thread` A reads the value: 0.
2. `Thread` B reads it too: 0.
3. `Thread` A writes back 0 + 1 = 1.
4. `Thread` B also writes back 0 + 1 = 1.

The result is 1, not 2. Two increments, only one took effect.

Note the crux: two `Threads` reading and writing the same data simultaneously — in fact, whenever data is shared and someone is writing, trouble looms. Even a party that's merely reading might read data whose write hasn't fully finished.

Rust's ownership and borrowing rules prevent many problems — no two `&muts` at once, no `&` alongside `&mut`. But under multithreading, they alone don't suffice. Take the example above: merely passing an `i32`'s value to another `Thread` is fine — `i32` is `Copy`, a duplicate goes over, and each side works on its own copy. But some types aren't so simple — after moving one over, the original `Thread` might still hold shared data. Which types can cross `Threads` safely? Which can't? Rust answers with two `traits` — `Send` and `Sync`.

9.3.2.2 What `spawn` Actually Does

Last episode, creating `Threads` with `thread::spawn`, we passed a closure. Closures capture outside variables — and `spawn` in effect **ships those captured variables to another `Thread`**. That's the real question: what can be shipped safely?

9.3.2.3 `Send`

A type implementing `Send` means its values can safely move to another `Thread`. Most types are `Send` — `i32`, `String`, `Vec<T>` (when `T` is `Send`), and so on.

9.3.2.4 `Sync`

A type implementing `Sync` means its `&T` (shared reference) can safely be shared among `Threads`. In other words:

> `T: Sync` is equivalent to `&T: Send`

If `&T` can be shipped safely to another `Thread`, `T` is `Sync`.

9.3.2.5 `Sync` Usually Implies `Send`

If something can be read by many `Threads` at once without trouble (`Sync`), then moving it wholesale to another `Thread` — eliminating even the possibility of simultaneous reads — can usually only be safer. So most `Sync` types are `Send` too, with a few exceptions.

9.3.2.6 `auto traits`: `traits` the Compiler Implements for You

You normally don't need to implement `Send` or `Sync` by hand. They're **`auto traits`** — the compiler implements them for your types automatically. The rule is simple: if everything a type stores is `Send`, the type itself defaults to `Send`. Same for `Sync`.

```
struct MyData {
    x: i32,    // Send + Sync
    s: String, // Send + Sync
}
// MyData is automatically Send + Sync
```

9.3.2.7 No Memorizing Required

You needn't memorize which types are `Send` or `Sync`. Toss an unsafe type into `thread::spawn` and the compiler tells you outright:

```rust
use std::rc::Rc;
use std::thread;

fn main() {
    let data = Rc::new(42);
    thread::spawn(move || {
        println!("{}", data);
    });
    // Compile error! Rc<i32> is not Send
}
```

9.3.2.8 Back to `spawn`'s Type Signature

Knowing `Send` and `Sync`, we can revisit `thread::spawn`'s signature:

```rust
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T + Send + 'static,
    T: Send + 'static,
```

The closure `F` must be `Send` — a closure's type includes whatever it captured, so a non-`Send` capture makes the closure non-`Send` and `spawn` fails to compile. The return value `T` must be `Send` too, since the result travels back from the child `Thread`.

And that `'static` — why? Because we have no idea how long a `spawn`ed `Thread` lives. You might `join` it; you might not, letting it run until `main` ends and it's forcibly terminated. Rust's type system can't guarantee you'll `join` at any particular moment, so it demands the most conservative guarantee: nothing in the closure or return value may hold a reference that could expire. Chapter 5 Episode 29 taught lifetime bounds — `T: 'a` means every reference in `T` outlives `'a`. `F: 'static` is that concept's extreme: references inside the closure must live as long as the whole program. In practice, the usual answer is to use `move` to move owned values into the closure. However, if a captured variable is itself a reference, `move` only moves or copies that reference into the closure; the closure still contains that reference. With `thread::spawn`, move the original owned data instead of its reference, or `clone` the data first and move that owned `clone`. If you really need a `Thread` to hold references to local data, Episode 13 later in this chapter introduces `thread::scope`, which is designed for that.

9.3.3 Example Code

```rust
use std::thread;

// All this struct's fields are Send + Sync,
// so it's automatically Send + Sync
struct Config {
    name: String,
    max_retries: u32,
}

fn main() {
    let config = Config {
        name: String::from("my_app"),
        max_retries: 3,
```

```
    };

    // Config is Send; it can safely move to another thread
    let handle = thread::spawn(move || {
        println!("Config name: {}", config.name);
        println!("Max retries: {}", config.max_retries);
    });

    handle.join().expect("thread panicked");
}
```

9.3.4 Recap

- Data race: several `Threads` accessing the same data at once with at least one writing — unpredictable results.
- `thread::spawn`'s closure ships its captures to another `Thread`, so those variables must be `Send`.
- `Send` = the value can safely move to another `Thread`.
- `Sync` = `&T` can safely be shared among `Threads` (`T: Sync` equals `&T: Send`).
- `Sync` usually implies `Send` — what many `Threads` may read at once is only safer moved.
- The compiler usually implements `Send` / `Sync` automatically; no manual marking is normally needed.

9.4 `RefCell` under Multithreading

9.4.1 Goal of This Episode

Understand why interior mutability is dangerous under multithreading, and `RefCell`'s `Send` / `Sync` characteristics.

9.4.2 Concept

9.4.2.1 Interior Mutability Is a Major Multithreading Threat

Chapter 5 taught that `RefCell` can modify its inner value through `&T` (a shared reference). In the single-threaded world, `RefCell` checks the borrowing rules at runtime and stays out of trouble.

The multithreaded world is different. `&T` looks "read-only," and `Sync`'s very definition is that `&T` can safely be shared among `Threads`. If a type can sneak modifications through `&T`, several `Threads` doing so at once can go wrong.

9.4.2.2 `RefCell`'s Borrow Count Isn't Atomic

`RefCell` tracks its current borrow state (how many immutable borrows; any mutable borrow) with an ordinary integer. Operations on that counter aren't **atomic**. An atomic operation is indivisible — no other `Thread` can ever see a halfway state. `RefCell`'s counter doesn't provide this guarantee, so two `Threads` could both read the same old value before either updates it. If two `Threads` call `borrow_mut()` through `&RefCell<T>` simultaneously, this can happen:

1. `Thread` A calls `borrow_mut()`, reads the counter, sees 0 (nobody borrowing).

2. Thread B calls borrow_mut() too, reads the counter, also sees 0.
3. Thread A concludes "nobody's borrowing; the mutable borrow is mine," setting the counter to "mutably borrowed."
4. Thread B also concludes "nobody's borrowing" — its step-2 read was the stale value — and takes a mutable borrow too.

Result: two Threads holding mutable borrows at once. RefCell's runtime check was bypassed entirely.

9.4.2.3 RefCell Is Not Sync

For the reason above, RefCell isn't Sync — &RefCell<T> can't be shared among Threads. Try, and the compiler blocks you.

9.4.2.4 RefCell<T> Is Send When T: Send

RefCell<T> can be **moved** to another Thread as long as T itself is Send. After the move, that one Thread alone owns the RefCell — no multiple Threads operating simultaneously.

```
use std::cell::RefCell;
use std::thread;

fn main() {
    let data = RefCell::new(vec![1, 2, 3]);

    // OK: RefCell<Vec<i32>> is Send; it can move to another thread
    let handle = thread::spawn(move || {
        data.borrow_mut().push(4);
        println!("{:?}", data.borrow());
    });

    handle.join().expect("thread panicked");
}
```

9.4.3 Example Code

```
use std::cell::RefCell;
use std::thread;

fn main() {
    // RefCell<String> can move to another thread (Send)
    let data = RefCell::new(String::from("hello"));

    let handle = thread::spawn(move || {
        // Within this thread, the RefCell works normally
        data.borrow_mut().push_str(" world");
        println!("Child thread: {}", data.borrow());
    });

    handle.join().expect("thread panicked");

    // But &RefCell can't be shared among threads (not Sync)
    // Try to have two threads share one RefCell, and the compiler stops you.
}
```

9.4.4 Recap

- Interior mutability lets `&T` modify contents — dangerous under multithreading.
- Atomic operation = an indivisible operation: no other `Thread` can see a halfway state.
- `RefCell`'s `borrow` count is an ordinary integer, not atomic; simultaneous multithreaded operations can bypass the check.
- `RefCell` is not `Sync` — `&RefCell<T>` can't be shared among `Threads`.
- `RefCell<T>` is `Send` when `T: Send` — it can then move to another `Thread`, which owns it alone.

9.5 `Rc` under Multithreading

9.5.1 Goal of This Episode

Understand why `Rc` can't cross `Threads` at all — neither `Send` nor `Sync`.

9.5.2 Concept

9.5.2.1 `Rc` Is Not `Sync`

`Rc`'s reference count, like `RefCell`'s `borrow` count, is an ordinary integer — no atomic operations. If several `Threads` `clone` or `drop` through `&Rc<T>` simultaneously, the count's increments and decrements can trample each other, corrupting the count — releasing the data early, or never releasing it.

So `Rc` isn't `Sync`, for the same reason as `RefCell`.

9.5.2.2 `Rc` Isn't Even `Send`

Last episode said `RefCell` is `Send`, since after a move one `Thread` alone owns it. `Rc` is different.

`Rc`'s whole design is multiple `Rcs` pointing at one piece of data. Move one `Rc` to another `Thread`, and its `clones` may remain on the original `Thread`. Both sides operating on the reference count simultaneously can wreck the counter.

The problem isn't the move itself, but that **after the move, two `Threads` still share one counter**.

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(42);
    let b = a.clone(); // a and b share the data and the counter

    // If a moved to another thread,
    // b would remain on the original — both sides touching the counter at once, boom
    std::thread::spawn(move || {
        println!("{}", a);
    });
    // Compile error! Rc<i32> is not Send
}
```

9.5.2.3 Rc Can't Cross Threads, Period

Rc is neither Send nor Sync. It can't move to other Threads, nor share references among them. Sharing data across Threads takes a different tool.

9.5.3 Example Code

```
use std::rc::Rc;
use std::thread;

fn main() {
    // Rc works normally in a single thread
    let a = Rc::new(String::from("hello"));
    let b = a.clone();
    println!("a = {}, b = {}", a, b);
    println!("Count = {}", Rc::strong_count(&a));

    // But it can't cross threads — the following won't compile:

    // let data = Rc::new(42);
    // thread::spawn(move || {
    //     println!("{}", data);
    // });
    // Compile error: Rc<i32> is not Send

    println!("Rc is single-threaded only");
}
```

9.5.4 Recap

- Rc's reference count is an ordinary integer, not atomic — so not Sync.
- Rc isn't even Send: after moving an Rc to another Thread, its clones may remain behind, and both sides touching the counter at once breaks it.
- In short: Rc cannot cross Threads at all.

9.6 Arc<T>

9.6.1 Goal of This Episode

Learn to share data safely among Threads with Arc<T>.

9.6.2 Concept

9.6.2.1 The Problem, Recapped

We've said Rc can't cross Threads — its reference count isn't atomic. Yet we genuinely need shared data across Threads — what now?

9.6.2.2 Arc: Atomic Reference Counting

Arc<T> is Rc with its reference count swapped for **atomic operations**. Atomic operations guarantee that even simultaneous counter updates from multiple Threads never trample each other.

Usage is almost identical to `Rc`:

```
use std::sync::Arc;

fn main() {
    let a = Arc::new(String::from("hello"));
    let b = Arc::clone(&a); // Like Rc: clone the pointer; data is shared
    println!("Count = {}", Arc::strong_count(&a)); // 2
}
```

9.6.2.3 Sharing across `Threads`

Move an `Arc::clone` into another `Thread`:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let data_clone = Arc::clone(&data);
    let handle = thread::spawn(move || {
        println!("Child thread: {:?}", data_clone);
    });

    println!("Main thread: {:?}", data);
    handle.join().expect("thread panicked");
}
```

9.6.2.4 `Arc<T>: Send` and `Arc<T>: Sync` Require `T: Send + Sync`

`Arc<T>` itself does not require `T: Send + Sync`. But to send and share an `Arc<T>` between `Threads` as above, `T` must satisfy both `traits`. Why?

`Sync`: multiple `Threads` access the same `T` simultaneously through their own `Arcs`. Chapter 5 taught `Deref` — `Arc` implements it, so `T`'s contents are reachable straight through the `Arc`. That amounts to multiple `Threads` holding shared references to `T` at once, so `T` must be `Sync`.

`Send`: when the last `Arc` gets `dropped`, `T` gets `dropped` too. Which `Thread` holds the last `Arc` is indeterminate, so `T`'s `drop` may happen on any `Thread` — `T` is effectively "shipped" to that `Thread` for destruction, so `T` must be `Send`.

9.6.3 Example Code

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3, 4, 5]);

    let mut handles = vec![];

    for i in 0..3 {
        let data_clone = Arc::clone(&data);
        let handle = thread::spawn(move || {
            let sum: i32 = data_clone.iter().sum();
```

```
            println!("The sum thread {} computed: {}", i, sum);
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().expect("thread panicked");
    }

    println!("Final count = {}", Arc::strong_count(&data)); // 1
}
```

9.6.4 Recap

- `Arc<T>` is `Rc<T>`'s multithreaded version, with atomic reference counting.
- Usage nearly matches `Rc`: `Arc::new()`, `Arc::clone()`.
- `Arc::clone` and move the `clone` into other `Threads` to share data.
- `Arc<T>: Send` and `Arc<T>: Sync` both require `T: Send + Sync`: `Sync` for simultaneous multithreaded access, `Send` because the `drop` may happen on any `Thread`.

9.7 Atomic Types

9.7.1 Goal of This Episode

Learn to read and write simple values safely across `Threads` with atomic types.

9.7.2 Concept

9.7.2.1 What's an Atomic Operation

Last episode's `Arc` counts references with atomic operations. What exactly is atomic?

Suppose two `Threads` run `count += 1` on one variable simultaneously. It looks like one step, but it's really three: read the current value, add 1, write it back. With both `Threads` doing those three steps at once, this can happen:

1. `Thread` A reads `count` = 0.
2. `Thread` B reads `count` = 0.
3. `Thread` A writes `count` = 1.
4. `Thread` B writes `count` = 1.

Each side added once, yet the result is 1, not 2.

Some **atomic operations** fuse read, modify, and write into one indivisible act — no `Thread` can ever see a halfway state. Do `count += 1` atomically, and two simultaneous `Threads` always yield 2.

9.7.2.2 `AtomicI32` and `AtomicBool`

The standard library provides several atomic types in `std::sync::atomic`, integers and booleans being the most used:

```rust
use std::sync::atomic::{AtomicI32, AtomicBool, Ordering};

fn main() {
    let counter = AtomicI32::new(0);
    let flag = AtomicBool::new(false);
}
```

9.7.2.3 Basic Operations

```rust
use std::sync::atomic::{AtomicI32, Ordering};

fn main() {
    let counter = AtomicI32::new(0);

    counter.store(10, Ordering::Relaxed);              // Write
    let val = counter.load(Ordering::Relaxed);         // Read: 10
    let old = counter.fetch_add(5, Ordering::Relaxed); // Add 5, returning the old value 10
    // counter is now 15
}
```

Every operation takes an `Ordering` parameter. Why?

Modern processors, for performance, may **reorder instruction execution**. Single-threaded, that's harmless — the processor guarantees results identical to in-order execution. But multi-threaded, one `Thread`'s reordering can show another `Thread` an inconsistent state.

`Ordering` tells the processor "instructions around this operation may not be freely reordered" — generally, the stricter the restriction, the higher the performance cost.

An example: `Thread` A writes data into a `Vec`, then sets an atomic flag `true`; `Thread` B, seeing the flag `true`, reads the `Vec`:

```rust
// Thread A
data.push(42);                          // Step 1: write the data
ready.store(true, Ordering::Relaxed); // Step 2: set the flag

// Thread B
if ready.load(Ordering::Relaxed) {    // Sees true
    println!("{}", data[0]);          // But the data may not be written yet!
}
```

With `Relaxed`, the processor may reorder `Thread` A's steps 1 and 2 — `Thread` B sees the flag already `true` while the data isn't in yet. The processor dares to reorder because, from `Thread` A's own perspective, flag-then-data and data-then-flag give identical results — it doesn't know another `Thread` is watching. `SeqCst` prevents this problem by preserving the two orders written in the code: `Thread` A writes the data before setting the flag, and `Thread` B checks the flag before reading the data. Therefore, if `Thread` B sees `true`, it is guaranteed to also see the data that `Thread` A wrote first.

The details run deep; as a beginner, remember two:

- `Ordering::Relaxed`: guarantees only this atomic operation itself; no restrictions on other instructions' order. Fine for plain counters.
- `Ordering::SeqCst`: the strictest. In this example, it preserves "write data → set flag" and

"check flag → read data," so when `Thread` B sees `ready == true`, it is guaranteed to also see the data that `Thread` A wrote first.

When unsure, `SeqCst` is safest.

9.7.2.4 Interior Mutability

Look at the code above — `store` and `fetch_add` clearly modify the value, yet need no `&mut self`; `&self` suffices. Like Chapter 5's `Cell`, this is interior mutability.

Why must it be designed so? If modification required `&mut self`, only one `Thread` could hold the `&mut`, and no other `Thread` could touch the value at all — what cross-thread anything would that be? Atomics exist precisely so multiple `Threads` access one value simultaneously through `&`, so interior mutability is mandatory.

`Cell` has interior mutability too, but `Cell` isn't `Sync` (no cross-thread sharing). Atomics are `Sync` — the underlying hardware guarantees the operations' atomicity, so simultaneous modification through `&` from many `Threads` stays sound.

9.7.2.5 Pairing with `Arc`

Atomics most commonly pair with `Arc`, letting several `Threads` update one counter together:

```
use std::sync::Arc;
use std::sync::atomic::{AtomicI32, Ordering};
use std::thread;
fn main() {
    let counter = Arc::new(AtomicI32::new(0));

    let mut handles = vec![];

    for _ in 0..10 {
        let counter_clone = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            for _ in 0..1000 {
                counter_clone.fetch_add(1, Ordering::Relaxed);
            }
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().expect("thread panicked");
    }

    println!("Final result: {}", counter.load(Ordering::Relaxed)); // Always 10000
}
```

Ten `Threads` adding 1000 each — the result is always 10000, never undercounted.

9.7.2.6 Atomic Types vs Locks

Atomic operations apply only to simple types — such as integers (`AtomicI32`, `AtomicU64`, `AtomicUsize`, etc.) and booleans (`AtomicBool`). To protect a `Vec`, `String`, or any complex structure, atomics can't; you need next episode's locks.

But for simple counters and flags, atomics beat locks — every `Thread` operates directly, no queueing for someone else to finish.

9.7.3 Example Code

```
use std::sync::Arc;
use std::sync::atomic::{AtomicI32, Ordering};
use std::thread;

fn main() {
    let counter = Arc::new(AtomicI32::new(0));

    let mut handles = vec![];

    // Three threads, each counting to a different limit
    for limit in [100, 200, 300] {
        let counter_clone = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            for _ in 0..limit {
                counter_clone.fetch_add(1, Ordering::Relaxed);
            }
            println!("Added {} times; now: {}", limit, counter_clone.load(Ordering::Relaxed));
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().expect("thread panicked");
    }

    // 100 + 200 + 300 = 600, whatever the execution order
    println!("Final result: {}", counter.load(Ordering::Relaxed));
}
```

9.7.4 Recap

- Some atomic operations fuse read-modify-write into one indivisible act, safe under simultaneous `Threads`.
- Common types: `AtomicI32`, `AtomicUsize`, `AtomicBool`.
- Common methods: `load` (read), `store` (write), `fetch_add` (add, returning the old value).
- `Ordering` controls memory ordering; when unsure, `SeqCst`.
- Atomic types have interior mutability — modifying through `&self` — and are `Sync` (shareable across `Threads`).
- Simple types only; complex data needs locks.

9.8 `Mutex<T>`

9.8.1 Goal of This Episode

Learn to let multiple `Threads` safely modify shared data with `Mutex<T>`.

9.8.2 Concept

9.8.2.1 What about Modifying Complex Shared Data?

Last episode's atomics apply only to simple types like integers and booleans. What if several `Threads` should modify a `Vec`, a `String`, or any complex structure?

9.8.2.2 `Mutex`: Multithreaded Interior Mutability

`Mutex<T>` somewhat resembles `RefCell` — both provide interior mutability, modifying values without `&mut`. The difference:

- **`RefCell`**: single-threaded, borrow-checking with an ordinary integer.
- **`Mutex`**: multithreaded, guarding the data with an operating-system lock.

9.8.2.3 `lock` and `MutexGuard`

Acquire the lock with `mutex.lock().expect("lock failed")`. It returns a `MutexGuard`:

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(42);
    {
        let mut guard = m.lock().expect("lock failed");
        *guard += 1; // Modify the value through the guard
        println!("{}", *guard); // 43
    } // The guard is dropped; automatic unlock
}
```

`MutexGuard` implements `Deref` and `DerefMut` (from Chapter 5), making it a smart pointer too — usable directly as `&T` or `&mut T`.

Only one `Thread` can `lock` successfully at a time. Other `Threads` calling `.lock()` **block** (wait) until the lock-holding `Thread` `drops` its guard.

9.8.2.4 `Arc` + `Mutex`

In practice they usually pair up — `Arc` lets several `Threads` share the `Mutex`; the `Mutex` guards the data inside:

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().expect("lock failed");
            *num += 1;
        });
        handles.push(handle);
    }
```

```
    for handle in handles {
        handle.join().expect("thread panicked");
    }

    println!("Result: {}", *counter.lock().expect("lock failed")); // 10
}
```

9.8.2.5 Don't Let the `MutexGuard` Live Long

While the guard lives, the lock stays held, and every other `Thread` waits. So keep the guard's lifespan short:

```
// Bad: the guard lives to scope's end; the lock is held too long
let mut guard = mutex.lock().expect("lock failed");
*guard += 1;
// ... lots of work that doesn't need the lock ...
// The guard only drops way down here

// Good: release when done
{
    let mut guard = mutex.lock().expect("lock failed");
    *guard += 1;
} // The guard drops immediately; the lock releases immediately
// ... other work ...
```

9.8.2.6 `Mutex` Turns `Send` into `Sync`

Episode 3 taught `Send` and `Sync`. Some types are `Send` but not `Sync` — Episode 4's `RefCell<T>`, say: safely movable to another `Thread` (`Send`), but not accessible by several `Threads` at once through `&RefCell<T>` (not `Sync`).

`Mutex` solves this. `Mutex<T>` guarantees that only one `Thread` accesses `T` at a time — even with many `Threads` sharing one `&Mutex<T>`, only the lock-holder touches the inner `T`. So `Mutex<T>` requires only `T: Send` for `Mutex<T>` itself to be `Sync`.

Put differently: `T` not being `Sync` is fine — the `Mutex`'s locking already rules out simultaneous access. `T` needs `Send` because: `Thread` A takes the lock, works on `T`, releases; the next lock-taker might be `Thread` B. From `T`'s perspective, it was A's exclusively, now it's B's exclusively — effectively `T` was "shipped" from A to B. Hence `T` must be `Send`.

9.8.3 Example Code

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for i in 0..5 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            // Shrinking the guard's scope
            {
```

```
                let mut num = counter.lock().expect("lock failed");
                *num += 1;
                println!("Thread {} set the counter to {}", i, *num);
            } // The guard drops right here

            // The lock is no longer held here
            println!("Thread {} is done", i);
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().expect("thread panicked");
    }

    println!("Final result: {}", *counter.lock().expect("lock failed"));
}
```

9.8.4 Recap

- `Mutex<T>` is multithreaded interior mutability, guarding data with a lock.
- `.lock().expect(...)` returns a `MutexGuard`, usable directly as `&mut T` via `DerefMut`.
- Only one `Thread` holds the lock at a time; the rest wait.
- Dropping the guard unlocks automatically.
- The common pairing: `Arc<Mutex<T>>` — `Arc` for sharing, `Mutex` for safe modification.
- Don't let the `MutexGuard` live long; while locked, every other `Thread` waits.
- `Mutex<T>` needs only `T: Send` to be `Sync` — its locking lets non-`Sync` types be safely shared among `Threads`.

9.9 RwLock<T>

9.9.1 Goal of This Episode

Learn the read-write-separated lock `RwLock<T>`, and how it compares to `Mutex`.

9.9.2 Concept

9.9.2.1 `Mutex`'s Limitation

`Mutex` locks whether you're reading or writing. But often many `Threads` only want to read — reads don't conflict with reads, so locking everything is wasteful.

9.9.2.2 `RwLock`: Separating Reads from Writes

`RwLock<T>` distinguishes read locks from write locks:

- **Read lock** (`.read().expect(...)`): several `Threads` may hold read locks **simultaneously**.
- **Write lock** (`.write().expect(...)`): exclusive — while a write lock is held, no read locks nor other write locks may exist.

"Several readers at once" is not something a single `Thread` can demonstrate; this episode's

example code at the end will show it with three `Threads`. For now, just how the two locks are taken:

```
use std::sync::RwLock;

fn main() {
    let lock = RwLock::new(42);

    // Read lock: look, don't touch
    {
        let r = lock.read().expect("read lock failed");
        println!("read {}", *r);
    } // r drops here, releasing the read lock

    // Write lock: exclusive, and you may modify
    {
        let mut w = lock.write().expect("write lock failed");
        *w += 1;
    } // w drops here, releasing the write lock

    println!("now {}", *lock.read().expect("read lock failed"));
}
```

9.9.2.3 The Guards' Behavior

The read lock returns an `RwLockReadGuard`; the write lock, an `RwLockWriteGuard`. Like `MutexGuard`, they're smart pointers — operate on the contents directly, unlocking automatically on `drop`.

The same caution applies: don't let guards live long.

9.9.2.4 Compared with `RefCell`

	`RefCell`	`RwLock`
`Threads`	Single-threaded	Multithreaded
Rule	Many `borrow()`s or one `borrow_mut()`	`read()`s from many `Threads`, or one `write()`
Enforcement	Runtime; violations panic	The OS's lock; violations block and wait

There is one trap `RefCell` doesn't have, though: `RefCell` lets you `borrow()` several times on the same `Thread`, but `RwLock`'s "many readers" means **many `Threads`**. Taking a second read lock on the same `RwLock` from the same `Thread` **may panic** — the standard library says so outright — and on some platforms it can hang outright.

9.9.2.5 `Mutex` vs `RwLock`

Which when?

- **`Mutex`**: simple, low overhead. Suits frequent reads-and-writes, or very short lock holds. `Mutex` suffices most of the time.
- **`RwLock`**: advantageous when reads far outnumber writes, since readers proceed simultaneously. But the lock itself costs more than a `Mutex`, and there's the risk of **writer starvation** — with readers streaming in endlessly, a writer may never get the lock.

9.9.3 Example Code

```
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));

    let mut handles = vec![];

    // Launch 3 readers
    for i in 0..3 {
        let data = Arc::clone(&data);
        let handle = thread::spawn(move || {
            let read_guard = data.read().expect("read lock failed");
            println!("Reader {}: {:?}", i, *read_guard);
            // Several readers may hold read locks at once
        });
        handles.push(handle);
    }

    // Launch 1 writer
    {
        let data = Arc::clone(&data);
        let handle = thread::spawn(move || {
            let mut write_guard = data.write().expect("write lock failed");
            write_guard.push(4);
            println!("Writer: write complete; it's now {:?}", *write_guard);
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().expect("thread panicked");
    }

    println!("Final result: {:?}", *data.read().expect("read lock failed"));
}
```

9.9.4 Recap

- `RwLock<T>` separates read and write locks: many `Threads` may read simultaneously, one exclusive writer.
- `.read().expect(...)` takes the read lock; `.write().expect(...)` the write lock.
- Guards operate on contents via `Deref`, unlocking automatically on `drop`.
- Against `RefCell`: `RefCell` is the single-threaded version; `RwLock` the multithreaded one.
- Don't take a second read lock on the same `RwLock` from the same `Thread` — it isn't `RefCell`'s `borrow()`; it may panic or hang.
- `Mutex` is simple and cheap — usually enough; `RwLock` suits read-heavy workloads, at higher cost and with writer-starvation risk.

9.10 Poisoning

9.10.1 Goal of This Episode

Understand what lock poisoning is, and how to handle it.

9.10.2 Concept

9.10.2.1 Why `.lock()` Returns a `Result`

Learning `Mutex` and `RwLock` in recent episodes, we always wrote `.lock().expect("lock failed")`. But when can acquiring the lock "fail"? The answer: **poisoning**.

9.10.2.2 What Is Poisoning

If a `Thread` panics while holding a `Mutex` lock or an `RwLock` write lock, the lock gets marked "poisoned." Every later attempt to take the lock — `Mutex::lock`, or the `RwLock`'s `read` or `write` — receives `Err(PoisonError)`.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let data = Arc::new(Mutex::new(vec![1, 2, 3]));
    let data2 = Arc::clone(&data);

    let handle = thread::spawn(move || {
        let mut guard = data2.lock().expect("lock failed");
        guard.push(4);
        panic!("Oops!"); // The guard is alive at the panic → the lock is poisoned
    });

    let _ = handle.join(); // Collect the panic; don't let it propagate

    // A later lock → Err
    match data.lock() {
        Ok(guard) => println!("Normal: {:?}", *guard),
        Err(_poisoned) => println!("The lock is poisoned!"),
    }
}
```

9.10.2.3 Why Poisoning Exists

A panic usually means an unexpected error. If a `Thread` panics halfway through modifying data, that data may be a half-finished product — a `Vec` mid-`push`, or two fields with only one updated. Poisoning is a safety mechanism: it tells you something went wrong, and lets you decide whether to keep using the data.

9.10.2.4 Three Ways to Handle It

1. Panic outright (the most common)

```
use std::sync::Mutex;

fn main() {
    let data = Mutex::new(Vec::<i32>::new());
```

```
    let guard = data.lock().expect("lock failed");
}
```

If the lock is poisoned, your `Thread` panics too. Usually that's fine — the previous `Thread` panicking generally means the whole program should end.

2. Ignore the poison and carry on

```
use std::sync::{Mutex, PoisonError};

fn main() {
    let data = Mutex::new(Vec::<i32>::new());
    let guard = data.lock().unwrap_or_else(PoisonError::into_inner);
}
```

`PoisonError::into_inner` hands back the guard, skipping the poison warning. If you're sure the data's state is fine, or don't care, this works.

Note that this is a "**write it this way at every lock site**" approach: the lock itself stays poisoned throughout; you simply ignore it every time. If one place in the program forgets and reaches for a plain `lock().expect(...)`, that place panics.

3. Repair the data, then continue

```
use std::sync::{Mutex, PoisonError};

fn main() {
    let data = Mutex::new(Vec::<i32>::new());
    let guard = match data.lock() {
        Ok(g) => g,
        Err(poisoned) => {
            let mut g = poisoned.into_inner();
            *g = vec![];         // Reset to a known-safe state
            data.clear_poison(); // Clear the poison so later locks work again
            g
        }
    };
}
```

Take the guard, restore the data to a sensible value, then proceed.

Option 2 can ignore the poison indefinitely because every lock site is written the same way. Option 3 means something different: repair the data, then go back to running normally. Repairing the data isn't enough for that — every plain `lock()` elsewhere in the program still comes back `Err` — so the repair is followed by `clear_poison()`, which is what actually clears it (`RwLock` has a method of the same name).

9.10.2.5 Why `.into_inner()` Is Safe

You might wonder: the data in a poisoned lock may be half-finished — is touching it really okay?

From memory's standpoint, yes. Poisoned or not, the data inside is valid memory — no touching memory that's no longer usable, no type confusion, no data races. Poisoning protects **logical consistency**, not **memory safety**. The data may be logically wrong, yet perfectly legal

from memory's perspective. Hence `.into_inner()` can be called safely.

9.10.2.6 `RwLock`'s Poisoning

`RwLock` poisons only when a **write lock** panics. A panicking read lock doesn't poison — reading modifies nothing and leaves no inconsistent state behind. But once poisoned, both `read` and `write` return `Err`.

9.10.3 Example Code

```rust
use std::sync::{Arc, Mutex, PoisonError};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));

    // Launch a thread that panics
    let counter2 = Arc::clone(&counter);
    let handle = thread::spawn(move || {
        let mut guard = counter2.lock().expect("lock failed");
        *guard += 1;
        panic!("Uh-oh, something broke!");
    });

    // Wait for that thread (it panics, but let _ ignores it)
    let _ = handle.join();

    // Try to take the lock — a PoisonError arrives
    match counter.lock() {
        Ok(guard) => {
            println!("Lock acquired normally, value = {}", *guard);
        }
        Err(poisoned) => {
            println!("The lock is poisoned!");

            // Take a look at the data
            let guard = poisoned.into_inner();
            println!("The value inside = {}", *guard);
        }
    }

    // Or ignore the poison in one line
    let guard = counter.lock().unwrap_or_else(PoisonError::into_inner);
    println!("Ignoring the poison, value = {}", *guard);
}
```

9.10.4 Recap

- A `Thread` panics while holding a `Mutex` lock or an `RwLock` write lock → the lock is poisoned.
- Afterward `lock` / `read` / `write` all return `Err(PoisonError)`.
- `RwLock` poisons only on a write-lock panic; read-lock panics don't.
- `PoisonError::into_inner` recovers the guard — memory safety is intact; only logical consistency is in question.

- Three handling options:
  - Panic (`.unwrap()` or `.expect()`).
  - Ignore (`.unwrap_or_else(PoisonError::into_inner)`).
  - Repair the data and continue (`into_inner` doesn't clear the poison; you also need `clear_poison()` for a real recovery).

9.11 mpsc

9.11.1 Goal of This Episode

Learn to make Threads communicate by passing messages through channels, and how this compares to shared memory.

9.11.2 Concept

9.11.2.1 A Different Line of Thought

The earlier Mutex and RwLock follow the "shared memory" approach — several Threads access one piece of data, with locks preventing conflicts.

Channels take a completely different approach: **Threads communicate by passing messages**. Data gets sent straight over — no sharing.

9.11.2.2 Creating a Channel

`std::sync::mpsc::channel()` creates a sender (tx) and receiver (rx) pair:

```
use std::sync::mpsc;

fn main() {
    let (tx, rx) = mpsc::channel::<i32>();
}
```

mpsc stands for **multiple producer, single consumer** — many senders allowed, but only one receiver.

9.11.2.3 Sending and Receiving

`tx.send(value)` sends the value out (moving it); `rx.recv()` receives on the other end (blocking until something arrives):

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        tx.send(String::from("hello")).expect("send failed");
    });

    let received = rx.recv().expect("receive failed");
    println!("Received: {}", received);
}
```

9.11.2.4 Multiple Senders

`tx.clone()` produces additional senders:

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    for i in 0..3 {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(format!("From thread {}", i)).expect("send failed");
        });
    }

    drop(tx); // The original tx must be dropped too, or rx never finishes

    for received in rx {
        println!("Received: {}", received);
    }
}
```

9.11.2.5 When Does It End

Once every `tx` is `dropped`, `rx.recv()` first drains all unreceived messages; only calls to `.recv()` after that return `Err`. The `for msg in rx` loop behaves likewise — it runs through the remaining messages, then ends. That's how "nobody will send again, and every message has been handled" gets determined.

Note the `drop(tx)` in the example above — if you `cloned` `tx` but never `dropped` the original, the receiver believes a sender still lives and never finishes.

9.11.2.6 Channels vs Shared Memory

Which when?

- **Several `Threads` repeatedly reading and writing one piece of data** (a shared counter, a shared cache) → `Mutex` / `RwLock` is more direct.
- **A produce-on-one-side, consume-on-the-other** pipeline → channels are more natural. Ownership of the data transfers outright: no locks, and no forgetting to release one.

9.11.3 Example Code

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    // Launch 3 workers, each computing and sending its result back
    for i in 0..3 {
        let tx = tx.clone();
        thread::spawn(move || {
```

```
            let result = i * i;
            println!("Thread {} finished computing: {}", i, result);
            tx.send((i, result)).expect("send failed");
        });
    }

    // Drop the original tx so the rx loop ends once all clones finish
    drop(tx);

    // Receive every result
    let mut total = 0;
    for (id, result) in rx {
        println!("Main thread received: thread {}'s result is {}", id, result);
        total += result;
    }

    println!("The sum of all results: {}", total);
}
```

9.11.4 Recap

- Channels let `Threads` communicate by message passing; data is sent over, not shared.
- `mpsc::channel()` creates the sender `tx` and receiver `rx`.
- `tx.send(value)` moves `value`; `rx.recv()` blocks until something arrives.
- `tx.clone()` makes multiple senders, but there's only one receiver (`mpsc`).
- Once every `tx` is `dropped`, the `rx` loop ends automatically.
- Pipelines take channels; repeated access to one piece of data takes `Mutex` / `RwLock`.

9.12 Deadlocks

9.12.1 Goal of This Episode

Understand what a deadlock is, why Rust's compiler can't stop it, and how to avoid it.

9.12.2 Concept

9.12.2.1 What's a Deadlock

A deadlock is two or more `Threads` waiting for each other to release locks — nobody can move, and the program hangs forever.

The classic case: `Thread` A holds lock 1 while waiting for lock 2; `Thread` B holds lock 2 while waiting for lock 1. Both wait forever.

9.12.2.2 A Code Demonstration

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let lock1 = Arc::new(Mutex::new(0));
    let lock2 = Arc::new(Mutex::new(0));
```

```
    let l1 = Arc::clone(&lock1);
    let l2 = Arc::clone(&lock2);

    let a = thread::spawn(move || {
        let _g1 = l1.lock().expect("lock failed"); // Takes lock 1
        // Imagine some delay here...
        let _g2 = l2.lock().expect("lock failed"); // Waits for lock 2
    });

    let l1 = Arc::clone(&lock1);
    let l2 = Arc::clone(&lock2);

    let b = thread::spawn(move || {
        let _g2 = l2.lock().expect("lock failed"); // Takes lock 2
        // Imagine some delay here...
        let _g1 = l1.lock().expect("lock failed"); // Waits for lock 1
    });

    // With unlucky timing, the program hangs here forever
    a.join().expect("thread panicked");
    b.join().expect("thread panicked");
}
```

`Thread` A takes lock 1 first, then wants lock 2. But lock 2 belongs to `Thread` B, which is waiting for lock 1 — nobody can advance.

9.12.2.3 The Compiler Doesn't Block Deadlocks

`Send` and `Sync` protect against **data races** — undefined behavior from simultaneous data access. A deadlock is a **logic problem**: nothing breaks and nothing is undefined; the program just hangs forever. Rust's compiler can't detect deadlocks.

9.12.2.4 One `Thread` Can Deadlock Alone

Even with a single `Thread`, calling `lock` twice on the same `Mutex` can deadlock — if the first `lock` hasn't been released, the second may wait forever:

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(42);
    let _g1 = m.lock().expect("lock failed");
    let _g2 = m.lock().expect("lock failed"); // Possible deadlock: _g1 still holds the lock
}
```

9.12.2.5 How to Avoid It

- **All `Threads` take locks in the same order**: if everyone takes lock 1 before lock 2, nobody jams anybody.
- **Hold fewer locks at once**: if one lock suffices, don't use two.
- **Don't let guards live long**: `drop` promptly when done, shortening lock-hold time.

9.12.3 Example Code

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let lock1 = Arc::new(Mutex::new(String::from("resource A")));
    let lock2 = Arc::new(Mutex::new(String::from("resource B")));

    // The correct way: both threads take the locks in the same order

    let l1 = Arc::clone(&lock1);
    let l2 = Arc::clone(&lock2);
    let a = thread::spawn(move || {
        let g1 = l1.lock().expect("lock failed"); // Lock 1 first
        let g2 = l2.lock().expect("lock failed"); // Then lock 2
        println!("Thread A: {} and {}", *g1, *g2);
    });

    let l1 = Arc::clone(&lock1);
    let l2 = Arc::clone(&lock2);
    let b = thread::spawn(move || {
        let g1 = l1.lock().expect("lock failed"); // Also lock 1 first
        let g2 = l2.lock().expect("lock failed"); // Then lock 2
        println!("Thread B: {} and {}", *g1, *g2);
    });

    a.join().expect("thread panicked");
    b.join().expect("thread panicked");
    println!("No deadlock!");
}
```

9.12.4 Recap

- Deadlock: `Threads` waiting on each other's locks; the program hangs forever.
- Rust's compiler doesn't block deadlocks — `Send` / `Sync` guard against data races; deadlocks are logic problems.
- One `Thread` locking the same `Mutex` twice can deadlock too, the first lock never having been released.
- Avoidance: a uniform lock order, fewer simultaneous locks, prompt guard `drops`.

9.13 A Brief Introduction to `thread::scope`

9.13.1 Goal of This Episode

Learn to create bounded-lifetime `Threads` with `thread::scope`, borrowing outside data without `move` or `Arc`.

9.13.2 Concept

9.13.2.1 `thread::spawn`'s Limitation

Using `thread::spawn` earlier, outside variables had to be `moved` into the closure or wrapped in `Arc`. That's because a `spawned` `Thread` may outlive the function that called it — Rust can't guarantee the data survives until the `Thread` finishes.

9.13.2.2 Why `spawn` Can't Borrow

Episode 3 examined `thread::spawn`'s type signature: the closure and return value both demand `'static` — living as long as the whole program. That's why local variables can't be borrowed: references to locals aren't `'static`.

9.13.2.3 `thread::scope`

`thread::scope` solves this. It guarantees every `Thread` `spawned` inside gets `joined` before the `scope` ends:

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4, 5];

    thread::scope(|s| {
        s.spawn(|| {
            println!("Child thread: {:?}", data); // Borrowed directly — no move needed
        });
    }); // Every scoped thread is guaranteed finished by here

    // data remains usable
    println!("Main thread: {:?}", data);
}
```

Since `scope` guarantees all `Threads` finish before the `}`, `data` can't be discarded early — the closure borrows it safely, needing neither `move` nor `Arc`.

9.13.2.4 Compared with the `spawn` + `Arc` Style

The same job with `thread::spawn` reads:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3, 4, 5]);
    let data_clone = Arc::clone(&data);

    let handle = thread::spawn(move || {
        println!("{:?}", data_clone);
    });

    handle.join().expect("thread panicked");
}
```

With `thread::scope`, far cleaner:

```rust
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4, 5];

    thread::scope(|s| {
        s.spawn(|| {
            println!("{:?}", data);
        });
    });
}
```

No Arc, no clone, no move, no manual join.

9.13.3 Example Code

```rust
use std::thread;

fn main() {
    let mut results = vec![];
    let input = vec![1, 2, 3, 4, 5];

    thread::scope(|s| {
        // Several threads borrowing input simultaneously (immutable borrows)
        let h1 = s.spawn(|| {
            let sum: i32 = input.iter().sum();
            sum
        });

        let h2 = s.spawn(|| {
            let max = input.iter().max().expect("empty input");
            *max
        });

        let h3 = s.spawn(|| {
            let min = input.iter().min().expect("empty input");
            *min
        });

        // Inside the scope, join retrieves return values too
        results.push(h1.join().expect("thread panicked"));
        results.push(h2.join().expect("thread panicked"));
        results.push(h3.join().expect("thread panicked"));
    });

    println!("input is still usable: {:?}", input);
    println!("Sum = {}, max = {}, min = {}", results[0], results[1], results[2]);
}
```

9.13.4 Recap

- thread::spawn demands 'static, so its closure can't borrow locals.
- thread::scope guarantees every scoped Thread joins before the scope ends, so outside data can be borrowed safely.

- No `move`, no `Arc`, no manual `join` — far cleaner code.
- When multithreading is needed only within one region, `thread::scope` beats `thread::spawn` for convenience.

Congratulations on finishing the multithreading chapter! 🎉 Starting from the low-level notion of pointers, this chapter worked through `Threads`, `Send` / `Sync`, `Arc`, `Mutex`, `RwLock`, poisoning, channels, and `thread::scope`. In many languages, multithreaded programming is headache territory, but Rust's type system blocks data races at compile time — no relying on experience and intuition to dodge bugs; the compiler is your best teammate. Next chapter: advanced language features!

Chapter 10

Advanced Language Features

This chapter discusses advanced language features that the previous chapters haven't had a chance to cover.

10.1 `dyn Trait` Basics

10.1.1 Goal of This Episode

Learn to use `dyn Trait` to store values of different types in the same place, and understand how dynamic dispatch works.

10.1.2 Concept

10.1.2.1 The Problem: Different Types in the Same Place

In Chapter 5 we learned `impl Trait`, which lets you write `fn print_it(x: &impl Display)` so a function accepts any type that implements `Display`. But what if you want to put values of different types into the same `Vec`?

```
trait Describe {
    fn describe(&self) -> String;
}

struct Cat;
struct Dog;

impl Describe for Cat {
    fn describe(&self) -> String {
        String::from("a cat")
    }
}

impl Describe for Dog {
    fn describe(&self) -> String {
        String::from("a dog")
    }
}
```

`Cat` and `Dog` are different types — you can't write `Vec<impl Describe>` to put them together. `impl Trait` decides on a concrete type at compile time, and every element in a `Vec` must be the same type.

10.1.2.2 Enter `dyn Trait`

`dyn Describe` means "some type that implements `Describe`, but I don't know which one specifically."

But since we don't know what it actually is, the size of `dyn Describe` isn't fixed — `Cat` might take 1 byte while `Dog` takes 100 bytes, and the compiler can't know at compile time which one it'll be. So `dyn Describe` is a DST (which we learned about in Appendix I's DST introduction) and must live behind a pointer:

- `&dyn Describe` — borrowed.
- `Box<dyn Describe>` — owned

```
trait Describe {
    fn describe(&self) -> String;
}

struct Cat;
struct Dog;

impl Describe for Cat {
    fn describe(&self) -> String {
        String::from("a cat")
    }
}

impl Describe for Dog {
    fn describe(&self) -> String {
        String::from("a dog")
    }
}

fn main() {
    let animals: Vec<Box<dyn Describe>> = vec![
        Box::new(Cat),
        Box::new(Dog),
    ];

    for animal in &animals {
        println!("{}", animal.describe());
    }
}
```

By the same logic, function return types can use `dyn Trait` too:

```
fn make_animal(is_cat: bool) -> Box<dyn Describe> {
    if is_cat {
        Box::new(Cat)
    } else {
        Box::new(Dog)
    }
}
```

`impl Trait` can't do this — the two branches of the `if` return different types, and the compiler can't decide at compile time which one it would be.

10.1.2.3 Fat Pointers: Address + vtable

In Appendix I's DST introduction we learned that `&[T]` is a fat pointer (address + length). `&dyn Trait` is also a fat pointer, but it stores something different:

```
&[T]         = [data address][length]
&dyn Trait   = [data address][vtable pointer]
```

The vtable (virtual method table) is a table holding function pointers to all of this concrete type's methods for this `trait`. `Cat`'s vtable has a pointer to `Cat::describe`; `Dog`'s vtable has a pointer to `Dog::describe`.

When you call `animal.describe()`, Rust looks up "which function is `describe`" in the vtable, then calls it.

```rust
use std::mem::size_of;

trait Describe {
    fn describe(&self) -> String;
}

fn main() {
    println!("{}", size_of::<&i32>());          // 8
    println!("{}", size_of::<&dyn Describe>()); // 16 (address + vtable pointer)
    println!("{}", size_of::<&[i32]>());        // 16 (address + length)
}
```

10.1.2.4 Dynamic Dispatch vs Static Dispatch

Static dispatch (`impl Trait` / generics): the compiler knows the concrete type and generates a separate copy of the function's code for each type. This is called **monomorphization**. Method calls jump straight to the right function — fast, but if there are many types, the code gets bigger.

```rust
use std::fmt::Display;

fn print_it(x: &impl Display) {
    println!("{}", x);
}

fn main() {
    print_it(&42);      // the compiler generates print_it::<i32>
    print_it(&"hello"); // the compiler generates print_it::<&str>
}
```

Dynamic dispatch (`dyn Trait`): the compiler generates only one copy of the code, and at runtime the function to call is looked up through the vtable. There's only one copy of the code, but every call pays an extra vtable lookup.

	Static dispatch (`impl Trait` / generics)	Dynamic dispatch (`dyn Trait`)
Decided at	Compile time	Runtime
Amount of code	One copy per type	Just one copy
Call speed	Fast (direct call)	Slightly slower (vtable lookup)
Can mix different types	No	Yes

Most of the time, static dispatch is all you need. Reach for `dyn Trait` only when you need to put different types in the same place.

10.1.2.5 `Box<dyn Fn()>` vs `impl Fn()`

Chapter 6 covered closures. `Box<dyn Fn()>` lets you unify different closures into a single type:

```
fn main() {
    let callbacks: Vec<Box<dyn Fn()>> = vec![
        Box::new(|| println!("hello")),
        Box::new(|| println!("world")),
    ];

    for cb in &callbacks {
        cb();
    }
}
```

`Vec<impl Fn()>` can't do this, because every closure is its own distinct anonymous type.

10.1.2.6 Lifetime Bounds on `dyn Trait`

`dyn Trait` can take a lifetime bound, written `dyn Trait + 'a` and read as `dyn (Trait + 'a)` — it means the same thing as `T: Trait + 'a` in generics; `dyn` turns that bound into a type.

In certain positions, if you don't write a lifetime bound, the compiler fills in a default. The default for `Box<dyn Trait>` is `'static`, so the full spelling is `Box<dyn Trait + 'static>`. The `+ 'static` means the concrete type inside can't contain any non-`'static` references. Take a look at this example:

```
struct Foo<'a>(&'a str);

impl<'a> Describe for Foo<'a> {
    fn describe(&self) -> String { String::from(self.0) }
}

// This function doesn't compile!
// Box<dyn Describe> = Box<dyn Describe + 'static>
// but Foo borrows s, and s isn't 'static
fn make_box(s: &str) -> Box<dyn Describe> {
    Box::new(Foo(s))
}
```

If you need to store a type that holds references, write the lifetime explicitly to override the default `'static`:

```
fn make_box<'a>(s: &'a str) -> Box<dyn Describe + 'a> {
    Box::new(Foo(s))
}
```

`&'a dyn Trait` defaults to `&'a (dyn Trait + 'a)` — that one rarely needs special handling.

10.1.2.7 `trait` Upcasting

If `trait B` is a subtrait of `trait A` (`trait B: A`), then `dyn B` can be converted to `dyn A`:

```
trait Animal {
    fn name(&self) -> &str;
}

trait Pet: Animal {
    fn owner(&self) -> &str;
}

fn print_animal_name(a: &dyn Animal) {
    println!("{}", a.name());
}

fn example(pet: &dyn Pet) {
    print_animal_name(pet); // dyn Pet → dyn Animal, OK
}
```

A `Pet` is always an `Animal`, so of course a `dyn Pet` can be used as a `dyn Animal`.

10.1.3 Example Code

```
trait Describe {
    fn describe(&self) -> String;
}

struct Cat { name: String }
struct Dog { name: String }

impl Describe for Cat {
    fn describe(&self) -> String {
        format!("the cat {}", self.name)
    }
}

impl Describe for Dog {
    fn describe(&self) -> String {
        format!("the dog {}", self.name)
    }
}

fn make_animal(is_cat: bool, name: &str) -> Box<dyn Describe> {
    if is_cat {
        Box::new(Cat { name: String::from(name) })
    } else {
        Box::new(Dog { name: String::from(name) })
    }
}

fn main() {
    let animals: Vec<Box<dyn Describe>> = vec![
        Box::new(Cat { name: String::from("Mimi") }),
        Box::new(Dog { name: String::from("Blackie") }),
        make_animal(true, "Kitty"),
        make_animal(false, "Rex"),
    ];
```

```
    for animal in &animals {
        println!("{}", animal.describe());
    }

    println!(
        "size of &dyn Describe: {} bytes",
        std::mem::size_of::<&dyn Describe>()
    );
}
```

10.1.4 Recap

- `dyn Trait` means "some type that implements `Trait`; which one specifically is unknown."
- `dyn Trait` is a DST and must live behind a pointer: `&dyn Trait`, `Box<dyn Trait>`.
- `&dyn Trait` is a fat pointer: data address + vtable pointer.
- Dynamic dispatch (`dyn Trait`) looks up methods through the vtable; static dispatch (`impl Trait`) is decided at compile time.
- Most of the time use static dispatch; use `dyn Trait` only when mixing different types.
- `Box<dyn Fn()>` can unify different closures into one type.
- `Box<dyn Trait>` implicitly defaults to `+ 'static` in some positions; `dyn Trait + 'a` reads as `dyn (Trait + 'a)` — `dyn` turns a `trait` bound into a type.
- `dyn SubTrait` can be converted to `dyn SuperTrait` (`trait` upcasting).

10.2 `dyn` Compatibility

10.2.1 Goal of This Episode

Understand which `traits` can be used with `dyn` and which can't, and the reasons behind it.

10.2.2 Concept

10.2.2.1 Not Every `trait` Works with `dyn`

Last episode we learned `dyn Trait`. But if you try to write `dyn Clone`, you get a compile error. That's because `Clone` is not **`dyn` compatible**.

10.2.2.2 The Core Idea: `impl Trait for dyn Trait`

To understand `dyn` compatibility, first think about how `dyn Trait` works. The compiler automatically generates an:

```
impl Trait for dyn Trait {
    fn method(&self, ...) {
        // look up the vtable, call the actual implementation
    }
}
```

In this auto-generated `impl`, `Self` = `dyn Trait`. And `dyn Trait` is a DST — its size isn't fixed; it's not `Sized`.

If some of a `trait`'s methods can't work when `Self = dyn Trait`, that `trait` isn't `dyn` compatible. Concretely, there are a few situations:

10.2.2.3 Restriction 1: `Self` Can't Appear in Types Other Than `self`

```
trait Compare {
    fn compare(&self, other: &Self) -> bool;
}

impl Compare for Cat {
    fn compare(&self, other: &Cat) -> bool { ... }
}

impl Compare for Dog {
    fn compare(&self, other: &Dog) -> bool { ... }
}
```

`compare`'s second parameter is `&Self`. When you use `dyn Compare`, the concrete type has been erased — you don't know whether it's a `Cat` or a `Dog` inside. But `Cat::compare` expects a `&Cat`, and `Dog::compare` expects a `&Dog`.

If someone passes in a `Dog`, but the function found through the vtable is `Cat::compare`, the function would treat the `Dog`'s data as a `Cat` — the types get mixed up.

To guarantee no mix-up, the compiler would need a runtime check that "the concrete type of the `y` being passed in matches the concrete type of `x`." But the whole point of `dyn` is that the concrete type has been erased — the compiler no longer knows what it originally was, so it can't do that check. So Rust simply forbids it.

10.2.2.4 Restriction 2: Methods Can't Have Generic Parameters

```
trait Converter {
    fn convert<U>(&self) -> U;
}
```

A vtable is a fixed-size table of function pointers. But a generic method is a different function for each different `U` — `convert::<i32>` and `convert::<String>` are two different function pointers. A vtable can't hold infinitely many versions.

The main issue is that the vtable has to be built by whoever compiles the `impl` — because only that side knows the concrete type of `Self`. But when the `impl` is compiled, you don't know which `U`s users will pick later, so the vtable can't possibly prepare every version in advance.

10.2.2.5 Restriction 3: The `trait` Itself Can't Require `Self: Sized`

Back to the opening question — why doesn't `dyn Clone` work? Recall from Appendix I's DST introduction that, besides returning `Self`, `Clone` has `Sized` as a supertrait:

```
trait Clone: Sized {
    fn clone(&self) -> Self;
}
```

For `dyn Clone` to work, the compiler would have to generate the `impl Clone for dyn Clone` described above. But `Clone: Sized` requires the implementing type to be `Sized`, while `dyn Clone` would be a DST. That `impl` is therefore impossible, so `Clone` is not `dyn` compatible and `dyn Clone` can't be formed.

10.2.2.6 The Escape Hatch: `where Self: Sized`

If only some of a `trait`'s methods are `dyn` compatible and others aren't, you can add `where Self: Sized` to all the others to opt them out:

```
trait MyTrait {
    fn normal(&self) -> String; // callable on dyn MyTrait
    fn special(&self) -> Self   // returns Self, not dyn compatible
        where Self: Sized;      // add this to opt it out
}
```

`where Self: Sized` means "this method can only be called when `Self` is `Sized`." `dyn MyTrait` isn't `Sized`, so this method can't be called on `dyn MyTrait` — but the `trait` itself remains `dyn` compatible, and the other methods can still be used through `dyn MyTrait`.

```
let x: &dyn MyTrait = &something;
x.normal(); // OK
// x.special(); // compile error: dyn MyTrait is not Sized
```

The full rules of `dyn` compatibility are actually more intricate than this episode covers, but these get you most of the way there.

10.2.3 Example Code

```
// A dyn compatible trait
trait Greet {
    fn greet(&self) -> String;
}

struct Alice;
struct Bob;

impl Greet for Alice {
    fn greet(&self) -> String { String::from("Hi, I'm Alice!") }
}

impl Greet for Bob {
    fn greet(&self) -> String { String::from("Hey, I'm Bob!") }
}

// A trait that mixes in where Self: Sized
trait Animal {
    fn name(&self) -> &str;

    // This method isn't dyn compatible (returns Self); opt out with where Self: Sized
    fn duplicate(&self) -> Self
    where
        Self: Sized + Clone;
}

#[derive(Clone)]
struct Cat { name: String }

impl Animal for Cat {
    fn name(&self) -> &str { &self.name }
```

```
    fn duplicate(&self) -> Self
    where
        Self: Sized + Clone,
    {
        self.clone()
    }
}

fn main() {
    // dyn Greet: different types in the same Vec
    let greeters: Vec<Box<dyn Greet>> = vec![
        Box::new(Alice),
        Box::new(Bob),
    ];

    for g in &greeters {
        println!("{}", g.greet());
    }

    // dyn Animal: name() works, duplicate() doesn't
    let cat = Cat { name: String::from("Mimi") };
    let animal: &dyn Animal = &cat;
    println!("animal: {}", animal.name());  // OK
    // animal.duplicate(); // compile error: dyn Animal is not Sized

    // But duplicate can be called on the concrete type
    let cat2 = cat.duplicate();
    println!("copy: {}", cat2.name());
}
```

10.2.4 Recap

- Not every `trait` can be used with `dyn` — it must be `dyn` compatible.
- The core idea: the compiler auto-generates `impl Trait for dyn Trait`, where `Self` = `dyn Trait` (a DST).
- `Self` can't appear in types other than `self` — the concrete type has been erased.
- Methods can't have generic parameters — the vtable is fixed-size and can't hold infinitely many versions.
- The `trait` can't require `Self: Sized` — `dyn Trait` is a DST, not `Sized`.
- Adding `where Self: Sized` to individual methods opts them out of `dyn`, keeping the `trait` itself `dyn` compatible.

10.3 `const fn`

10.3.1 Goal of This Episode

Learn to use `const fn` to define functions that can also run at compile time, plus the `const { }` block.

10.3.2 Concept

10.3.2.1 The Problem: Computing a `const` Value with a Function

Chapter 2 covered `const` — compile-time constants. But a `const`'s value can only use simple expressions:

```
const MAX: i32 = 100;        // OK
const DOUBLE: i32 = MAX * 2; // OK
```

What if you want to compute it with a function?

```
fn square(x: i32) -> i32 { x * x }
const VALUE: i32 = square(5); // compile error! ordinary functions can't run at compile time
```

10.3.2.2 `const fn`

Put `const` in front of a function and it becomes a function that can also run at compile time:

```
const fn square(x: i32) -> i32 { x * x }
const VALUE: i32 = square(5); // OK! 25 is computed at compile time
```

A `const fn` is not "compile-time only" — it can be called at runtime just fine, like a regular function. It just has one extra ability: **it can run at compile time**.

```
const fn max(a: i32, b: i32) -> i32 {
    if a > b { a } else { b }
}

const BIGGER: i32 = max(10, 20); // compile time: 20

fn main() {
    let x = max(3, 7); // runtime: works too, just a regular function
    println!("{}", x);
    println!("{}", BIGGER);
}
```

10.3.2.3 Restrictions

You can't do everything inside a `const fn`. The basic principle: **the compiler must be able to simulate running this code inside itself**.

What you can do:

- Arithmetic, comparison, and logical operations
- `if`, `match`, `loop`, `while`.
- `let` bindings (including `let mut`).
- Creating tuples, `structs`, `enums`.
- Calling other `const fn`s.
- `panic!` (a compile-time panic becomes a compile error).

What you can't do:

- Call non-`const` functions.
- Input/output (`println!` and the like).
- Interact with the operating system
- inline assembly

Every Rust release relaxes the restrictions a little more; the list of things you can do in a `const fn` keeps growing.

10.3.2.4 `const` Blocks

`const { ... }` lets you insert a piece of compile-time computation anywhere, without defining a `const` variable or a `const fn`:

```
fn main() {
    let x = const { 1 + 2 + 3 };
    println!("{}", x); // 6, computed at compile time
}
```

This is handy when you want compile-time computation "in place," without defining a separate `const`.

10.3.3 Example Code

```
const fn factorial(n: u64) -> u64 {
    if n <= 1 {
        1
    } else {
        n * factorial(n - 1)
    }
}

const fn clamp(value: i32, min: i32, max: i32) -> i32 {
    if value < min {
        min
    } else if value > max {
        max
    } else {
        value
    }
}

const FACT_10: u64 = factorial(10);
const CLAMPED: i32 = clamp(150, 0, 100);

fn main() {
    println!("10! = {}", FACT_10);
    println!("clamp(150, 0, 100) = {}", CLAMPED);

    // callable at runtime too
    let n = factorial(5);
    println!("5! = {}", n);

    // const block
    let size = const { std::mem::size_of::<[i32; 100]>() };
    println!("size of 100 i32s: {} bytes", size);
}
```

10.3.4 Recap

- A `const fn` can run at compile time and at runtime.

- Its main use is initializing `const` values.
- Restrictions: it can't call non-`const fn`s or do I/O, but the restrictions loosen with each release.
- A `const { ... }` block inserts compile-time computation anywhere.

10.4 Associated `const`s

10.4.1 Goal of This Episode

Learn to define constants inside `trait`s and `impl`s.

10.4.2 Concept

10.4.2.1 Associated `const` in a `trait`

Besides methods and associated types, a `trait` can also define constants:

```
trait HasLimit {
    const LIMIT: i32;
}

impl HasLimit for u8 {
    const LIMIT: i32 = 255;
}

impl HasLimit for i8 {
    const LIMIT: i32 = 127;
}
```

The implementation must specify the value. To use it, write `Type::CONST`:

```
trait HasLimit {
    const LIMIT: i32;
}

impl HasLimit for u8 {
    const LIMIT: i32 = 255;
}

impl HasLimit for i8 {
    const LIMIT: i32 = 127;
}
fn main() {
    println!("u8: {}", u8::LIMIT); // 255
    println!("i8: {}", i8::LIMIT); // 127
}
```

10.4.2.2 Associated `const`s Can Have Defaults

Just like a `trait`'s default methods, an associated `const` can have a default value:

```
trait Config {
    const TIMEOUT: u64 = 30;
    const RETRIES: u32 = 3;
}
```

```
struct MyApp;

impl Config for MyApp {
    const TIMEOUT: u64 = 60; // override the default
    // RETRIES uses the default of 3
}

fn main() {}
```

10.4.2.3 Associated `const` in an `impl`

An associated `const` doesn't have to live in a `trait` — you can also define type-bound constants directly in an `impl` block:

```
struct Circle;

impl Circle {
    const PI: f64 = 3.14159265358979;
}

fn main() {
    println!("PI = {}", Circle::PI);
}
```

Just like an associated function, you access it with `::`.

10.4.3 Example Code

```
trait Bounded {
    const LOWER: i32;
    const UPPER: i32;

    fn is_in_range(&self, value: i32) -> bool {
        value >= Self::LOWER && value <= Self::UPPER
    }
}

struct Percentage;

impl Bounded for Percentage {
    const LOWER: i32 = 0;
    const UPPER: i32 = 100;
}

struct Temperature;

impl Bounded for Temperature {
    const LOWER: i32 = -273;
    const UPPER: i32 = 1000;
}

// associated const in an impl
struct Grid;
```

```
impl Grid {
    const WIDTH: usize = 80;
    const HEIGHT: usize = 24;
    const TOTAL: usize = Self::WIDTH * Self::HEIGHT;
}

fn main() {
    let p = Percentage;
    println!("is 50 in range? {}", p.is_in_range(50));
    println!("is 150 in range? {}", p.is_in_range(150));

    println!("temperature range: {} ~ {}", Temperature::LOWER, Temperature::UPPER);

    println!("Grid size: {}x{} = {}", Grid::WIDTH, Grid::HEIGHT, Grid::TOTAL);
}
```

10.4.4 Recap

- A `trait` can define `const NAME: Type;`, with the value given in the `impl`.
- An associated `const` can have a default value, which the `impl` may override.
- An `impl` block (outside any `trait`) can also define associated `const`s, accessed as `Type::CONST`.

10.5 `const` Generics

10.5.1 Goal of This Episode

Learn to use constant values as generic parameters and handle arrays of any length.

10.5.2 Concept

10.5.2.1 The Problem: A Function over Arrays of Any Length

`[i32; 3]` and `[i32; 5]` are different types — the length is part of the type. If you want a function that prints an array of any length, surely you don't have to write one per length?

10.5.2.2 `const` Generics

Generic parameters aren't limited to types — they can also be **constant values**:

```
fn print_array<const N: usize>(arr: [i32; N]) {
    for x in arr {
        println!("{}", x);
    }
}

fn main() {
    print_array([1, 2, 3]);        // N = 3
    print_array([10, 20, 30, 40]); // N = 4
}
```

`<const N: usize>` declares a constant generic parameter `N` of type `usize`. Like a type parameter `<T>`, the compiler generates one copy of the code for each distinct `N`.

10.5.2.3 How It Differs from Slices

You might think: why not just pass `&[i32]`? True — if all you need is to read a sequence of data, slices are more flexible. But `const` generics can do things slices can't:

Returning a fixed-length array:

```
fn zeros<const N: usize>() -> [i32; N] {
    [0; N]
}

fn main() {
    let a: [i32; 3] = zeros();
    let b: [i32; 10] = zeros();
}
```

A slice can't be returned as `[T]` (a DST), but `[T; N]` can.

Guaranteeing lengths at the type level:

```
fn add_arrays<const N: usize>(a: [i32; N], b: [i32; N]) -> [i32; N] {
    let mut result = [0; N];
    for i in 0..N {
        result[i] = a[i] + b[i];
    }
    result
}
```

The two parameters are guaranteed at compile time to have the same length. Slices can't do that.

10.5.2.4 On `structs`

```
struct Matrix<const ROWS: usize, const COLS: usize> {
    data: [[f64; COLS]; ROWS],
}
```

10.5.2.5 Expression Syntax

If the value in a `const` generic position isn't a simple literal or path, wrap it in `{}`:

```
fn example<const N: usize>() -> [i32; N] { [0; N] }

fn main() {
    let a = example::<3>();         // literal, no {} needed
    let b = example::<{ 1 + 2 }>(); // expression, needs {}
}
```

10.5.2.6 Combined with `const fn`

The `const fn` we just learned can also supply a `const` generic's value:

```
const fn double(n: usize) -> usize { n * 2 }

fn zeros<const N: usize>() -> [i32; N] { [0; N] }

fn main() {
    let c = zeros::<{ double(3) }>(); // [i32; 6], a const fn as the value
```

```
}
```

10.5.3 Example Code

```
fn sum<const N: usize>(arr: [i32; N]) -> i32 {
    let mut total = 0;
    for i in 0..N {
        total += arr[i];
    }
    total
}

fn filled<T: Copy, const N: usize>(value: T) -> [T; N] {
    [value; N]
}

fn main() {
    println!("sum([1, 2, 3]) = {}", sum([1, 2, 3]));
    println!("sum([10, 20]) = {}", sum([10, 20]));

    let ones: [i32; 5] = filled(1);
    println!("{:?}", ones);

    let hellos: [&str; 3] = filled("hello");
    println!("{:?}", hellos);

    // expression syntax
    let zeros = filled::<i32, { 2 + 3 }>(0);
    println!("{:?}", zeros);
}
```

10.5.4 Recap

- Generic parameters can be constant values: `<const N: usize>`.
- The most common use: handling arrays of any length, `[T; N]`.
- Compared to slices: `const` generics can return fixed-length arrays and guarantee lengths at the type level.
- Wrap expressions in {}: `Foo::<{ 1 + 2 }>`.
- They combine nicely with `const fn`.

10.6 Default Parameters

10.6.1 Goal of This Episode

Learn where type parameters and `const` generic parameters can have defaults, and how to define them.

10.6.2 Concept

10.6.2.1 Default Type Parameters

On declarations for `structs`, `enums`, `unions`, type aliases, and `traits`, a type parameter that is almost always the same type can have a default. When that argument is omitted while using the type or `trait`, the default applies. Generic parameters on functions and methods cannot have defaults.

Take the standard library's `PartialEq` as an example:

```
trait PartialEq<Rhs = Self> {
    fn eq(&self, other: &Rhs) -> bool;
}
```

`Rhs = Self` means: if you don't specify `Rhs`, it defaults to `Self`. So `impl PartialEq for Point` is the same as `impl PartialEq<Point> for Point` — by default, you compare against your own type.

If you occasionally want to compare against a different type, just override it:

```
impl PartialEq<(i32, i32)> for Point {
    fn eq(&self, other: &(i32, i32)) -> bool {
        self.x == other.0 && self.y == other.1
    }
}
```

10.6.2.2 Defining Your Own

Use = in the generic definition to give a default:

```
struct Container<T = String> {
    value: T,
}

fn main() {
    let c: Container = Container { value: String::from("hello") }; // T defaults to String
    let c2: Container<i32> = Container { value: 42 };              // specified manually
}
```

10.6.2.3 Defaults for `const` Generics

`const` generic parameters can also have defaults on the same kinds of type and `trait` declarations. They cannot have defaults on functions or methods either.

```
struct Buffer<const N: usize = 1024> {
    data: [u8; N],
}

fn main() {
    let buf: Buffer = Buffer { data: [0; 1024] };     // N defaults to 1024
    let small: Buffer<64> = Buffer { data: [0; 64] }; // specified manually
}
```

10.6.2.4 Parameters with Defaults Must Come Last

```
struct Pair<T, U = T> { // OK: U has a default and comes after T
    first: T,
    second: U,
}
```

10.6.3 Example Code

```
struct Pair<T, U = T> {
    first: T,
    second: U,
}

impl<T: std::fmt::Debug, U: std::fmt::Debug> Pair<T, U> {
    fn show(&self) {
        println!("({:?}, {:?})", self.first, self.second);
    }
}

fn main() {
    // U uses the default (= T = i32)
    let p1: Pair<i32> = Pair { first: 1, second: 2 };
    p1.show();

    // specify U manually
    let p2: Pair<i32, &str> = Pair { first: 42, second: "hello" };
    p2.show();
}
```

10.6.4 Recap

- Type parameters can have defaults on `struct`, `enum`, `union`, type alias, and `trait` declarations.
- `const` generic parameters can have defaults in the same places.
- Function and method generic parameters cannot have defaults.
- The syntax is `<T = String>`, `<Rhs = Self>`, or `<const N: usize = 1024>`.
- Leave it out and the default applies; specify it and it's overridden.
- `PartialEq<Rhs = Self>` is the standard library's classic example.
- Parameters with defaults must come after parameters without them.

10.7 Operator Overloading

10.7.1 Goal of This Episode

Learn to implement operators like `+` and `-` for your own types.

10.7.2 Concept

10.7.2.1 Operators Are `trait` Methods

In Rust, `a + b` is really shorthand for `a.add(b)` — `+` corresponds to the `std::ops::Add` `trait`. Implement `Add` for your type and you can use `+`.

10.7.2.2 The Definition of the `Add` `trait`

```
trait Add<Rhs = Self> {
    type Output;
    fn add(self, rhs: Rhs) -> Self::Output;
}
```

Three things to note:

- `Rhs = Self`: the default parameter from last episode — the right-hand side of the addition defaults to the same type as the left.
- `type Output`: the associated type from Chapter 5 — the result of an addition isn't necessarily the same type as the inputs.
- `self`, not `&self`: `add` consumes the left-hand value (types that are `Copy` are unaffected).

10.7.2.3 Implementing `Add` for `Point`

```
use std::ops::Add;

#[derive(Debug)]
struct Point { x: i32, y: i32 }

impl Add for Point {
    type Output = Point;
    fn add(self, rhs: Point) -> Point {
        Point {
            x: self.x + rhs.x,
            y: self.y + rhs.y,
        }
    }
}
```

10.7.2.4 Common Operators

Commonly used `traits` in `std::ops`:

Operator	`trait`	Method
`+`	`Add`	`add(self, rhs)`
`-`	`Sub`	`sub(self, rhs)`
`*`	`Mul`	`mul(self, rhs)`
`/`	`Div`	`div(self, rhs)`
`%`	`Rem`	`rem(self, rhs)`
`-x`	`Neg`	`neg(self)`
`!x`	`Not`	`not(self)`
`&`	`BitAnd`	`bitand(self, rhs)`
`|`	`BitOr`	`bitor(self, rhs)`
`^`	`BitXor`	`bitxor(self, rhs)`

Operator	trait	Method
<<	Shl	shl(self, rhs)
>>	Shr	shr(self, rhs)
+=	AddAssign	add_assign(&mut self, rhs)
&=	BitAndAssign	bitand_assign(&mut self, rhs)
[]	Index	index(&self, idx)
[] mutable	IndexMut	index_mut(&mut self, idx)

The bitwise operators (&, |, ^, <<, >>, !) come up a lot in systems programming — flags, masks, bit fields, and so on. If you're not yet familiar with bitwise operations, it's worth looking them up on your own.

Every binary operator listed above has a corresponding assign version (e.g. &= corresponds to `BitAndAssign`, <<= to `ShlAssign`), used just like the += or -= you learned earlier.

10.7.2.5 `AddAssign` vs `Add`

`a += b` and `a = a + b` aren't necessarily implemented the same way in Rust:

- `Add::add(self, rhs)` consumes a and produces a new value.
- `AddAssign::add_assign(&mut self, rhs)` modifies a in place.

For an `i32` the difference barely matters, but for a non-`Copy` type (like `String`), `s1 += &s2` only needs a mutable borrow of `s1`, while `s1 = s1 + &s2` has to give up ownership of `s1` and assign the result back. They ask for different things from you, and for some types the efficiency differs too, which is why they're separate `traits`.

`Add` and `AddAssign` are completely independent — implementing `Add` doesn't automatically make += work, nor vice versa. Without the implementation, it's a compile error.

10.7.2.6 `Index` / `IndexMut`

`Vec` supports `v[i]` precisely because it implements `Index`:

```
use std::ops::Index;

struct MyVec(Vec<i32>);

impl Index<usize> for MyVec {
    type Output = i32;
    fn index(&self, idx: usize) -> &i32 {
        &self.0[idx]
    }
}
```

10.7.2.7 Adding Different Types

Override the default `Rhs`:

```
use std::ops::Add;

struct Meters(f64);
struct Centimeters(f64);
```

```
impl Add<Centimeters> for Meters {
    type Output = Meters;
    fn add(self, rhs: Centimeters) -> Meters {
        Meters(self.0 + rhs.0 / 100.0)
    }
}
```

10.7.3 Example Code

```
use std::ops::{Add, Neg};

#[derive(Debug, Clone, Copy)]
struct Vec2 { x: f64, y: f64 }

impl Add for Vec2 {
    type Output = Vec2;
    fn add(self, rhs: Vec2) -> Vec2 {
        Vec2 { x: self.x + rhs.x, y: self.y + rhs.y }
    }
}

impl Neg for Vec2 {
    type Output = Vec2;
    fn neg(self) -> Vec2 {
        Vec2 { x: -self.x, y: -self.y }
    }
}

fn main() {
    let a = Vec2 { x: 1.0, y: 2.0 };
    let b = Vec2 { x: 3.0, y: 4.0 };
    let c = a + b;
    println!("a + b = {:?}", c);
    println!("-a = {:?}", -a);
}
```

10.7.4 Recap

- `a + b` is shorthand for `Add::add(a, b)`; likewise for the other operators.
- `Add`'s signature uses a default parameter (`Rhs = Self`) and an associated type (`Output`).
- `AddAssign` (`+=`) modifies in place (`&mut self`); `Add` (`+`) produces a new value (`self`).
- `Index` / `IndexMut` let your type use the `[]` operator.
- Overriding `Rhs` enables operations between different types.

10.8 Type Conversion with as

10.8.1 Goal of This Episode

Learn to convert between numeric types with as, and the safer alternatives.

10.8.2 Concept

10.8.2.1 Basic Usage

In Chapter 1 we learned that Rust doesn't convert types automatically. When you need a conversion, use as:

```
fn main() {
    let x: i32 = 42;
    let y: f64 = x as f64;

    let a: f64 = 3.99;
    let b: i32 = a as i32; // 3 — the decimal part is truncated, not rounded
}
```

10.8.2.2 Between Integers

Small to large — the value is unchanged:

```
    let x: u8 = 200;
    let y: u32 = x as u32; // 200
```

Large to small — **silent truncation**, no error:

```
    let x: u32 = 300;
    let y: u8 = x as u8; // 44! 300 = 256 + 44; only the lowest 8 bits are kept
```

Between signed and unsigned there can also be surprises:

```
    let x: i32 = -1;
    let y: u32 = x as u32; // 4294967295
```

10.8.2.3 `From` / `Into`: The Safer Choice

Chapter 5 covered `From` and `Into`. By convention, these `traits` should only be implemented for infallible conversions:

```
fn main() {
    let x: i32 = 42;
    let y: f64 = f64::from(x); // OK
    let z: i32 = i32::from(3.14_f64); // compile error! conversion may lose information
}
```

10.8.2.4 `TryFrom` / `TryInto`

For conversions that can fail, use `TryFrom` — it returns a `Result`:

```
use std::convert::TryFrom;

fn main() {
    let x: u32 = 300;
    let result = u8::try_from(x); // Err
    let y: u32 = 42;
    let result = u8::try_from(y); // Ok(42)
}
```

10.8.2.5 Which One to Use When

- Use `From` / `Into` for conversions intended to be infallible.

- Use `TryFrom` / `TryInto` when the conversion can fail — it returns a `Result`.
- Use `as` only when you truly know what you're doing.

10.8.3 Example Code

```
use std::convert::TryFrom;

fn main() {
    // basic as conversion
    let x: i32 = 42;
    let y: f64 = x as f64;
    println!("i32 {} → f64 {}", x, y);

    let a: f64 = 3.99;
    let b: i32 = a as i32;
    println!("f64 {} → i32 {} (truncated, not rounded)", a, b);

    // dangerous silent truncation
    let big: u32 = 300;
    let small: u8 = big as u8;
    println!("u32 {} → u8 {} (silent truncation!)", big, small);

    // From: safe
    let safe: f64 = f64::from(42_i32);
    println!("From: {}", safe);

    // TryFrom: can fail
    match u8::try_from(300_u32) {
        Ok(v) => println!("TryFrom succeeded: {}", v),
        Err(e) => println!("TryFrom failed: {}", e),
    }

    match u8::try_from(42_u32) {
        Ok(v) => println!("TryFrom succeeded: {}", v),
        Err(e) => println!("TryFrom failed: {}", e),
    }
}
```

10.8.4 Recap

- as converts between numeric types: float to integer truncates the decimals; large integer to small integer silently truncates.
- `From` / `Into`: by convention, for infallible conversions.
- `TryFrom` / `TryInto`: for conversions that can fail; they return a `Result`.
- Prefer `From`, then `TryFrom`, and only then `as`.

10.9 `enum` Discriminants

10.9.1 Goal of This Episode

Understand the integer value behind each `enum` variant, and how to customize it.

10.9.2 Concept

10.9.2.1 Every Variant Has an Integer Value

Chapter 3 covered C-style `enum`s. Behind every variant there is an integer, called the **discriminant**. Rust uses it to tell which variant a value currently is.

```
enum Color {
    Red,   // 0
    Green, // 1
    Blue,  // 2
}
```

By default it starts at 0 and increments by 1 per variant.

10.9.2.2 Getting the Discriminant with `as`

Last episode covered `as`. A C-style `enum` can be converted to an integer with `as` to reveal its discriminant:

```
enum Color {
    Red,   // 0
    Green, // 1
    Blue,  // 2
}

fn main() {
    println!("{}", Color::Red as i32);   // 0
    println!("{}", Color::Green as i32); // 1
    println!("{}", Color::Blue as i32);  // 2
}
```

10.9.2.3 Custom Discriminants

Specify values manually:

```
enum HttpStatus {
    Ok = 200,
    NotFound = 404,
    InternalError = 500,
}

fn main() {
    println!("{}", HttpStatus::NotFound as i32); // 404
}
```

Unspecified variants continue from the previous one +1:

```
enum Level {
    Low = 1,
    Medium,    // 2
    High,      // 3
    Critical = 10,
    Emergency, // 11
}
```

10.9.2.4 Controlling the Underlying Type with `#[repr]`

By default, the underlying type is up to the compiler. Use `#[repr]` to specify it explicitly:

```
#[repr(u8)]
enum Direction {
    North, // 0_u8
    South, // 1_u8
    East,  // 2_u8
    West,  // 3_u8
}
```

Common choices include `u8`, `u16`, `u32`, `i32`, and so on.

10.9.2.5 `enum`s with Data Have Discriminants Too

An `enum` carrying data also has an internal discriminant to distinguish variants, but you **can't get it with `as`**:

```
enum Shape {
    Circle(f64),
    Rectangle(f64, f64),
}

fn main() {
    Shape::Circle(3.0) as i32; // compile error!
}
```

10.9.3 Example Code

```
#[repr(u8)]
enum Command {
    Quit = 0,
    Move = 1,
    Write = 2,
    ChangeColor = 3,
}

enum Season {
    Spring = 1,
    Summer, // 2
    Autumn, // 3
    Winter, // 4
}

fn main() {
    println!("Quit = {}", Command::Quit as u8);
    println!("Write = {}", Command::Write as u8);

    println!("Spring = {}", Season::Spring as i32);
    println!("Winter = {}", Season::Winter as i32);
}
```

10.9.4 Recap

- Every `enum` variant has an integer discriminant, starting at 0 by default and incrementing.

- A C-style `enum` can be converted to an integer with `as` to see the discriminant.
- Specify values manually with `= number`; unspecified ones continue from the previous +1.
- `#[repr(u8)]` and friends control the underlying type.
- `enums` with data have discriminants too, but they can't be obtained with `as`.

10.10 An Overview of Attributes

10.10.1 Goal of This Episode

Survey Rust's common attributes and understand the difference between outer and inner ones.

10.10.2 Concept

10.10.2.1 outer vs inner

- **Outer attribute** `#[...]`: goes above an item and decorates that item.
- **Inner attribute** `#![...]`: goes inside an item (usually at the top of a file) and decorates the enclosing item as a whole.

```
#![allow(dead_code)] // inner: applies to the whole mod

#[derive(Debug)]     // outer: applies to the struct below
struct Point { x: i32, y: i32 }
```

The difference is one exclamation mark `!`.

10.10.2.2 derive

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct Color(u8, u8, u8);
```

10.10.2.3 Warning Control

```
#[allow(dead_code)]        // don't warn about unused code
#[allow(unused_variables)] // don't warn about unused variables
#[warn(missing_docs)]      // turn on the "missing docs" warning
#[deny(unsafe_code)]       // upgrade "uses unsafe" to an error
```

10.10.2.4 Conditional Compilation

```
#[cfg(target_os = "windows")]
fn windows_only() { /* ... */ }

#[cfg(test)]
mod tests { /* ... */ }
```

10.10.2.5 Testing

```
#[test]
fn test_add() { assert_eq!(1 + 1, 2); }

#[test]
```

```rust
#[should_panic]
fn test_panic() { panic!("on purpose"); }

#[test]
#[ignore]
fn slow_test() { /* skip for now */ }
```

10.10.2.6 Performance Hints

When a function is called, the program has to jump to the function's location, run it, then jump back. **`inline`** is an optimization: the compiler "pastes" the function's code directly into the call site, saving the jumping around.

```rust
#[inline]         // suggest the compiler inline this function
#[inline(always)] // suggest always inlining
#[inline(never)]  // suggest never inlining
```

Most of the time you don't need to write these by hand — the compiler decides on its own. They're only needed for small functions called across `crates`, or in performance-critical spots.

10.10.2.7 Memory Layout

Rust's compiler freely rearranges a `struct`'s fields in memory and adjusts alignment to save space. But if you're interoperating with C, C `structs` have fixed layout rules — `#[repr(C)]` tells Rust "lay this out by C's rules":

```rust
#[repr(C)]  // use C's memory layout
#[repr(u8)] // enum underlying type (from last episode)
```

10.10.2.8 Other Common Ones

`#[must_use]` on a function or type makes the compiler warn if a caller receives the return value but doesn't use it. `Result` carries `#[must_use]` — that's why you see a warning when you don't handle a `Result`.

```rust
#[must_use]
fn compute() -> i32 { 42 }

fn main() {
    compute(); // warning: unused return value
    let _ = compute(); // OK: explicitly ignore with let _
}
```

```rust
#[non_exhaustive] // tell other crates this enum / struct may gain new items later
#[deprecated]     // mark as deprecated
#[deprecated(since = "2.0", note = "use new_function instead")]
```

10.10.2.9 Doc Comments Are Attribute Shorthand

```rust
/// This is a function
fn foo() {}

// is the same as
#[doc = "This is a function"]
fn foo() {}
```

`///` is just shorthand for `#[doc = "..."]`. Likewise, `//!` is shorthand for `#![doc = "..."]` — used at the top of a file to document a whole `mod` or `crate`.

10.10.3 Example Code

```rust
#![allow(dead_code)]

#[derive(Debug, Clone, PartialEq)]
struct Config {
    name: String,
    value: i32,
}

#[must_use]
fn create_config(name: &str, value: i32) -> Config {
    Config { name: String::from(name), value }
}

#[deprecated(note = "use create_config instead")]
fn make_config() -> Config {
    create_config("default", 0)
}

#[cfg(target_os = "linux")]
fn linux_only() {
    println!("only runs on Linux");
}

fn main() {
    let c = create_config("test", 42);
    println!("{:?}", c);
}
```

10.10.4 Recap

- `#[...]` (outer) decorates the item below it; `#![...]` (inner) decorates the item containing it.
- `#[derive(...)]`: auto-implement `traits`.
- `#[allow/warn/deny(...)]`: control warnings.
- `#[cfg(...)]`: conditional compilation.
- `#[test]` / `#[should_panic]` / `#[ignore]`: test-related
- `#[must_use]`: warn when the return value is ignored.
- `#[deprecated]`: mark as deprecated.
- `///` is shorthand for `#[doc = "..."]`; `//!` is shorthand for `#![doc = "..."]`.

10.11 The `cfg!` Macro

10.11.1 Goal of This Episode

Learn to use `cfg!` to evaluate a condition at compile time and obtain a `bool` result, and how it differs from `#[cfg]`.

10.11.2 Concept

10.11.2.1 `cfg!` Returns a `bool`

Last episode covered `#[cfg(...)]` — conditional compilation, where code that doesn't match the condition is removed wholesale. But sometimes you just want to take different branches based on a condition, without removing whole blocks of code. That's what `cfg!` does:

```
fn main() {
    if cfg!(target_os = "windows") {
        println!("you're on Windows");
    } else {
        println!("you're not on Windows");
    }
}
```

10.11.2.2 How It Differs from `#[cfg]`

	`#[cfg(...)]`	`cfg!(...)`
Effect	Conditional compilation: whole block kept or removed	Expands to `true` or `false` at compile ti
Code checking	Non-matching code is removed and not checked	Both branches remain and are checked

`cfg!` is often used inside an ordinary `if`, but its condition is already determined at compile time rather than waiting until runtime. Both branches remain and must pass the compiler's checks.

This is an important difference: with `#[cfg]`, the non-matching block doesn't exist at all — even calls to nonexistent functions inside it won't error. But with `cfg!`, if one side has a compile error, it errors regardless of whether the condition holds.

```
// #[cfg] version: on Windows, linux_only() isn't compiled — no error
#[cfg(target_os = "linux")]
fn linux_only() { /* Linux-specific functionality */ }

// cfg! version: both sides get compiled
if cfg!(target_os = "linux") {
    // linux_only(); // if the function doesn't exist, this is a compile error even on Windows!
}
```

10.11.2.3 Common Conditions

`#[cfg]` and `cfg!` accept the same conditions:

- `target_os = "windows"` / `"linux"` / `"macos"`
- `target_arch = "x86_64"` / `"aarch64"`.
- `debug_assertions` — `true` in debug mode
- `feature = "my_feature"` — Cargo features
- `test` — `true` during `cargo test`

10.11.3 Example Code

```rust
fn main() {
    if cfg!(debug_assertions) {
        println!("debug mode");
    } else {
        println!("release mode");
    }

    let os = if cfg!(target_os = "windows") {
        "Windows"
    } else if cfg!(target_os = "linux") {
        "Linux"
    } else if cfg!(target_os = "macos") {
        "macOS"
    } else {
        "other"
    };
    println!("operating system: {}", os);
}
```

10.11.4 Recap

- `cfg!(...)` expands to a constant `bool` at compile time; both branches remain and are checked.
- `#[cfg(...)]` is conditional compilation; non-matching code is removed wholesale.
- Both accept the same conditions: `target_os`, `debug_assertions`, `feature`, `test`, etc.

10.12 `macro_rules!`

10.12.1 Goal of This Episode

Learn to define your own declarative macros with `macro_rules!`.

10.12.2 Concept

10.12.2.1 Macros vs Functions

We've been using macros since Chapter 1 — `println!`, `vec!`, `format!`, `assert_eq!`. The exclamation mark `!` in the call is what distinguishes a macro from a function.

The most fundamental difference between macros and functions: macros **expand into code at compile time**. The macro call you write gets replaced during compilation by its expanded code, and the compiler then compiles that expanded result. Macros can produce arbitrary code — new functions, `struct`s, even other macro calls. They can also accept things that aren't values as arguments — type names, patterns, and so on.

But macros are also harder to write, harder to read, and give worse error messages. If a function will do, don't use a macro.

10.12.2.2 Basic Syntax

```rust
macro_rules! say_hello {
    () => {
```

```
        println!("Hello!");
    };
}

fn main() {
    say_hello!(); // prints Hello!
}
```

The structure is `(pattern) => { expansion }` — match on the left, expand on the right.

10.12.2.3 With Parameters

Capture parameters with `$name:kind`:

```
macro_rules! say {
    ($msg:expr) => {
        println!("{}", $msg);
    };
}

fn main() {
    say!("hi");
    say!(1 + 2);
}
```

Common kinds:

- `expr`: an expression
- `ty`: a type
- `ident`: an identifier (like a variable name)
- `tt`: a token tree (the most flexible)

There are other kinds too; look them up if you ever need them.

10.12.2.4 Multiple Arms

```
macro_rules! log {
    ($val:expr) => {
        println!("value: {}", $val);
    };
    ($name:expr, $val:expr) => {
        println!("{} = {}", $name, $val);
    };
}

fn main() {
    log!(42);           // value: 42
    log!("score", 100); // score = 100
}
```

10.12.2.5 Repetition

The `$( ... ),*` syntax matches repeated items. Broken down:

- `$( ... )` holds the pattern to repeat.

- `,` is the separator — each repetition is separated by a comma. The separator doesn't have to be a comma; you can use `;` or other symbols, or omit it.
- `*` means zero or more. You can also use `+` for one or more.

```
macro_rules! make_vec {
    ($($element:expr),*) => {
        {
            let mut v = Vec::new();
            $( v.push($element); )*
            v
        }
    };
}

fn main() {
    let v = make_vec![1, 2, 3];
}
```

Expansion also uses `$( ... )*` — `$( v.push($element); )*` expands once per captured element, becoming:

```
v.push(1);
v.push(2);
v.push(3);
```

10.12.2.6 Three Kinds of Brackets

Macros can be called with three kinds of brackets, with identical effect:

- `macro!(...)` — parentheses, like a function call.
- `macro![...]` — square brackets, like an array (`vec![1,2,3]` uses these).
- `macro!{...}` — curly braces, like a code block.

The difference is purely convention.

10.12.2.7 Macro Scope

A macro defined with `macro_rules!` can only be used after its definition (unlike functions — functions aren't restricted by definition order).

To make a macro usable by other `crates`, add `#[macro_export]` in front. When referring to items from the defining `crate` inside the macro, use the `$crate` path — that way the path resolves correctly no matter what name the user's `crate` gives yours:

```
// in the my_lib crate

pub fn _log_impl(msg: &str) {
    println!("[LOG] {}", msg);
}

#[macro_export]
macro_rules! log_msg {
    ($msg:expr) => {
        $crate::_log_impl($msg);
    };
}
```

A crate that depends on my_lib can call the macro as my_lib::log_msg!("hello"), or import it with use my_lib::log_msg; and then write log_msg!("hello"). $crate automatically resolves to the crate where the macro was defined.

10.12.3 Example Code

```
macro_rules! max {
    ($a:expr, $b:expr) => {{
        let a = $a;
        let b = $b;
        if a > b { a } else { b }
    }};
}

macro_rules! print_all {
    ($($item:expr),*) => {
        $(
            println!("{}", $item);
        )*
    };
}

// stringify! is a built-in macro that turns whatever you pass in into a string verbatim
// stringify!(hello) becomes "hello"
macro_rules! create_fn {
    ($name:ident) => {
        fn $name() {
            println!("called the function {}", stringify!($name));
        }
    };
}

create_fn!(hello);
create_fn!(world);

fn main() {
    println!("max(3, 7) = {}", max!(3, 7));

    print_all!["a", "b", "c"];

    hello();
    world();
}
```

10.12.4 Recap

- If a function will do, don't use a macro.
- macro_rules! defines declarative macros: (pattern) => { expansion }.
- Receive parameters with $name:expr etc.; common kinds: expr, ty, ident, tt.
- $(...),* matches repeated items; $(...)* in the expansion repeats per item.
- The three bracket styles () / [] / {} behave identically.
- Macros are usable only after their definition (unlike functions).
- #[macro_export] makes a macro usable from other crates.

10.13 Proc Macros

10.13.1 Goal of This Episode

Meet the three kinds of proc macros and understand how they work. This episode is only a rough introduction to the concept and skeleton of proc macros — it won't walk you through writing a complete one. If you need that, search for a dedicated tutorial.

10.13.2 Concept

10.13.2.1 What Is a Proc Macro

Last episode's `macro_rules!` expands code via pattern matching. **Proc macros** (procedural macros) let you write the expansion logic in ordinary Rust: you can use conditionals, run loops, and call functions to decide what code to generate.

A proc macro receives your code as input (a stream of tokens) and produces new code (also a stream of tokens).

For example, it can parse each field declaration in a `struct` and generate different handling code based on the type syntax: one approach for `String`, another for `Option<T>`.

10.13.2.2 `TokenStream`

A proc macro's input and output are both `TokenStreams` — sequences of Rust code tokens. When `struct Foo { x: i32 }` comes in, the proc macro sees a stream of tokens: `struct`, `Foo`, `{`, `x`, `:`, `i32`, `}`.

10.13.2.3 The Three Kinds of Proc Macros

1. `derive` macros

Used with `#[derive(...)]` — the most common kind.

```
#[proc_macro_derive(MyDerive)]
pub fn my_derive(input: TokenStream) -> TokenStream {
    // input: the code of the struct / enum marked with #[derive(MyDerive)]
    // return: new code to "attach" alongside (the original struct / enum isn't replaced)
    TokenStream::new()
}
```

Usage: `#[derive(MyDerive)] struct Foo { x: i32 }`

2. attribute macros

Custom attributes.

```
#[proc_macro_attribute]
pub fn my_attr(attr: TokenStream, item: TokenStream) -> TokenStream {
    // attr: the attribute's arguments
    // item: the entire item being marked
    // return: "replaces" the original item
    item
}
```

Usage: `#[my_attr(some_arg)] fn my_function() { ... }`

3. function-like macros

Look like function calls.

```
#[proc_macro]
pub fn my_macro(input: TokenStream) -> TokenStream {
    // input: whatever is inside the parentheses
    // return: the expanded code
    input
}
```

Usage: `my_macro!(any tokens);`

10.13.2.4 How the Three Differ

- **`derive`**: **attaches** new code; doesn't replace the original `struct` / `enum`.
- **Attribute**: **replaces** the marked item.
- **Function-like**: the contents in the brackets get **expanded** into new code.

10.13.2.5 A Separate `crate`

Proc macros must be defined in their own `crate`, with this in `Cargo.toml`:

```
[lib]
proc-macro = true
```

10.13.2.6 `syn` and `quote`

In practice, two community `crates` almost always come along:

- **`syn`**: parses a `TokenStream` into structured data (e.g. knowing "this is a `struct` with one field named x").
- **`quote`**: conveniently generates a `TokenStream` from structured data.

Without them you'd be handling tokens one by one — very painful.

10.13.3 Example Code

Here are minimal skeletons for the three kinds of proc macros (they need to live in a separate proc-macro `crate`):

```
use proc_macro::TokenStream;

// 1. derive macro
#[proc_macro_derive(MyDerive)]
pub fn my_derive(input: TokenStream) -> TokenStream {
    // parse input with syn, generate code with quote
    TokenStream::new() // generates nothing
}

// 2. attribute macro
#[proc_macro_attribute]
pub fn my_attr(_attr: TokenStream, item: TokenStream) -> TokenStream {
    item // returned untouched
}

// 3. function-like macro
```

```rust
#[proc_macro]
pub fn my_macro(input: TokenStream) -> TokenStream {
    input // returned untouched
}
```

And here's the usage side (in another `crate`):

```rust
// suppose the proc-macro crate is called my_macros
use my_macros::{MyDerive, my_attr, my_macro};

#[derive(MyDerive)]
struct Foo { x: i32 }

#[my_attr]
fn hello() {
    println!("hello");
}

fn main() {
    hello();
    my_macro!(any tokens can go here);
}
```

10.13.4 Recap

- Proc macros come in three kinds: `derive`, attribute, and function-like.
- At heart they are compile-time functions that take a `TokenStream` and return a `TokenStream`.
- You can write code-generation logic in ordinary Rust, such as choosing different handling based on a field's type syntax.
- `derive` attaches code, attribute replaces the item, function-like expands its contents.
- They must be defined in a separate `crate` (`proc-macro = true`).
- `syn` (parsing) and `quote` (generation) are the usual companions.

10.14 `unsafe`

10.14.1 Goal of This Episode

Understand what `unsafe` means, what it lets you do, and what to watch out for when writing `unsafe` code.

10.14.2 Concept

10.14.2.1 Why `unsafe` Exists

Rust's safety guarantees rest on certain **assumptions** — for example, that a `&mut T` is always exclusive, or that a reference always points to valid data. The compiler checks those assumptions for you.

But some operations are impossible for the compiler to verify. Rust doesn't forbid you from doing them — it asks you to explicitly say "I take responsibility for this part." That's `unsafe`.

10.14.2.2 Rust's Safety Guarantees

Safe Rust guarantees the following things **can never happen**, no matter how your code is written:

- No access to memory that has already been freed.
- No data races (multiple `Threads` reading and writing simultaneously with at least one writer).
- No dangling references.
- No value getting `dropped` twice.
- No reads of uninitialized memory.
- No type confusion (e.g. reading the bytes of an `isize` as a pointer).

Violate any one of these and you have **undefined behavior** (UB for short). It is far worse than "the result is unpredictable": the compiler optimizes on the assumption that these things never happen, so once one does, it isn't only that line that breaks — the whole program loses every guarantee, and it may compute nonsense, crash, or go wrong somewhere entirely unrelated.

The responsibility of `unsafe` code is this: even while bypassing the compiler's checks, it must ensure that **all** of these guarantees still hold.

10.14.2.3 `unsafe` Blocks

Wrap code needing `unsafe` operations in `unsafe { }`. `unsafe` is not "turn off all checks" — borrowing rules and type checking still apply inside an `unsafe` block. It merely unlocks a few specific extra operations.

10.14.2.4 The Five `unsafe` Operations

1. **Dereferencing raw pointers** (`*const T`, `*mut T`)
2. **Calling `unsafe` functions**
3. **Manually implementing an `unsafe trait`**
4. **Accessing `static mut` variables**
5. **Accessing a `union`'s fields**

10.14.2.5 Raw Pointers

A raw pointer is a pointer with no borrow-rule protection. **Creating** one doesn't require `unsafe`; **using** it (dereferencing) does:

```
fn main() {
    let x = 42;
    let ptr: *const i32 = &raw const x; // creating: no unsafe needed

    let value = unsafe { *ptr }; // dereferencing: unsafe needed
    println!("{}", value); // 42
}
```

You can also convert a reference into a raw pointer with `as`:

```
    let x = 42;
    let ptr = &x as *const i32; // &i32 to *const i32
```

But `&raw const x` and `&raw mut x` are better — they take a raw pointer straight from the

variable without creating a reference first. Sometimes merely creating the reference can itself break the rules (e.g. taking & of uninitialized memory); &raw sidesteps that problem.

10.14.2.6 unsafe fn

If a function's safety must be guaranteed by its caller, mark it unsafe fn:

```
unsafe fn dangerous(ptr: *const i32) -> i32 {
    unsafe { *ptr }
}

fn main() {
    let x = 42;
    let value = unsafe { dangerous(&raw const x) };
}
```

Note: since the Rust 2024 edition, unsafe operations require an unsafe { } block even inside an unsafe fn — so every unsafe operation is explicitly marked.

10.14.2.7 unsafe trait

Some traits can only be implemented correctly by satisfying conditions the compiler can't check automatically:

```
unsafe trait MyGuarantee {
    fn check(&self) -> bool;
}

unsafe impl MyGuarantee for i32 {
    fn check(&self) -> bool { *self >= 0 }
}
```

An unsafe trait means: "implementing this trait requires satisfying conditions the compiler cannot check." You implement it with unsafe impl, signaling that you guarantee those conditions hold.

Send and Sync are unsafe traits — the compiler implements them automatically when appropriate, but if you implement them manually, you must guarantee Thread safety yourself.

Note: **calling** an unsafe trait's methods doesn't require unsafe — the danger lies in the implementation, not the use.

10.14.2.8 The unsafe Boundary

unsafe code must guarantee: **no matter what safe code calls it, it can never cause undefined behavior.**

Take the standard library's Vec: it uses unsafe internally to manage memory, but exposes a safe API. No matter how you use Vec's safe API, you cannot trigger undefined behavior.

10.14.2.9 Guidelines for Writing unsafe Code

- **Keep unsafe blocks as small as possible** — wrap only the lines that truly need it.
- **Write // SAFETY: comments** — explain why this unsafe operation is correct.
- **Mind the borrowing rules** — even with raw pointers, rules like "&mut must be exclusive" still hold semantically.

- **Uphold type invariants** — e.g. a `String` is always valid UTF-8; a `bool` is always 0 or 1.
- **Consider panic safety** — if the `unsafe` block contains operations that can panic, make sure the data structure remains valid after a panic.
- **Test with Miri** — `cargo +nightly miri test` can detect many `unsafe` problems.

10.14.2.10 Common Uses

- Implementing data structures (linked lists, the internals of `Vec`)
- Interoperating with C
- Performance-critical sections

10.14.3 Example Code

```
fn main() {
    // raw pointers
    let mut x = 42;
    let ptr_const: *const i32 = &raw const x;
    let ptr_mut: *mut i32 = &raw mut x;

    unsafe {
        println!("read: {}", *ptr_const);
        *ptr_mut = 100;
        println!("after write: {}", *ptr_mut);
    }

    // unsafe fn
    unsafe fn add_one(ptr: *mut i32) {
        unsafe { *ptr += 1; }
    }

    let mut val = 10;
    // SAFETY: ptr points to a valid, initialized i32, and no other references exist
    unsafe { add_one(&raw mut val); }
    println!("val = {}", val);
}
```

10.14.4 Recap

- `unsafe` lets you do things the compiler can't verify, but it doesn't turn off all checks.
- The five `unsafe` operations: dereferencing raw pointers, calling `unsafe fn`s, implementing `unsafe trait`s, accessing `static mut`, accessing `union` fields.
- Raw pointers `*const T` / `*mut T`: pointers without borrow-rule protection, not guaranteed to point to valid data. Creating them needs no `unsafe`; dereferencing does.
- `&raw const x` / `&raw mut x`: take a raw pointer directly, without going through a reference.
- Since the 2024 edition, `unsafe fn` bodies also require `unsafe { }` blocks.
- With `unsafe trait`s the danger is in implementing, not using (calling methods needs no `unsafe`).
- **Undefined behavior (UB)**: violating safe Rust's guarantees. The compiler optimizes on the assumption that it never happens, so once it does, the whole program loses its guarantees.

- The `unsafe` boundary: no matter what safe code calls it, it must never cause undefined behavior.

10.15 `static` Variables

10.15.1 Goal of This Episode

Understand the difference between `static` and `const`, and why you should almost never use `static mut`.

10.15.2 Concept

10.15.2.1 `static` vs `const`

Chapter 2 covered `const` — compile-time constants whose values get embedded directly wherever they're used. `static` looks similar, but there's one fundamental difference: **a `static` variable has a fixed memory address**.

```
static GREETING: &str = "Hello, world!";
static MAX_SIZE: usize = 1024;
```

Most of the time `const` is enough. Use `static` only when you need a fixed memory address (e.g. to hand to a C function).

10.15.2.2 `static mut`

Rust allows mutable `static`s — but both reading and writing require `unsafe`:

```
static mut COUNTER: i32 = 0;

fn increment() {
    unsafe { COUNTER += 1; }
}
```

Why the `unsafe`? Because a `static` is globally shared — multiple `Thread`s reading and writing it at once is a data race.

`static mut` should almost never be used. Modern Rust has better alternatives:

- A simple counter → `AtomicI32`, `AtomicBool`.
- Complex mutable global state → `Mutex<T>` (paired with `static`).
- Lazy initialization → `LazyLock` (next episode).

10.15.3 Example Code

```
use std::sync::atomic::{AtomicI32, Ordering};

// const: value embedded at use sites
const MAX: i32 = 100;

// static: has a fixed address
static GREETING: &str = "Hello!";

// atomic instead of static mut
```

```rust
static COUNTER: AtomicI32 = AtomicI32::new(0);

fn increment() {
    COUNTER.fetch_add(1, Ordering::Relaxed);
}

fn main() {
    println!("{}", GREETING);
    println!("MAX = {}", MAX);

    increment();
    increment();
    increment();
    println!("COUNTER = {}", COUNTER.load(Ordering::Relaxed));
}
```

10.15.4 Recap

- A `static` has a fixed memory address; one copy is shared by the whole program.
- A `const` has no fixed address; its value is embedded at use sites — use `const` most of the time.
- `static mut` requires `unsafe` for both reads and writes and should almost never be used.
- Alternatives: `AtomicXxx`, `Mutex<T>`, `LazyLock`.

10.16 LazyLock

10.16.1 Goal of This Episode

Learn to lazily initialize global variables with `LazyLock`.

10.16.2 Concept

Strictly speaking, `LazyLock` is a standard-library utility, not a language feature. But since we just learned `static`, and it's the thing most often paired with `static`, we cover it here.

10.16.2.1 The Problem: A `static`'s Value Must Be Known at Compile Time

A `static`'s value must be computable at compile time. An empty `Vec::new()` is fine (it's a `const fn` and allocates no memory), but what if you want a `Vec` that **already has contents**?

```rust
// the vec! macro and String::from both need to allocate memory at runtime
static NAMES: Vec<String> = vec![String::from("Alice"), String::from("Bob")];
```

So what now? If we can't provide the value at compile time, then **don't provide it yet** — initialize it the first time it's used at runtime. That's lazy initialization.

10.16.2.2 `LazyLock`

`std::sync::LazyLock` does exactly this — you give it a closure, it runs the closure to produce the value only on first access, and every access after that uses the cached result. `LazyLock` implements `Deref`, so you can treat it directly as the value inside, just like `Box`, `Rc`, and the other smart pointers:

```rust
use std::sync::LazyLock;

static NAMES: LazyLock<Vec<String>> = LazyLock::new(|| {
    vec![String::from("Alice"), String::from("Bob")]
});

fn main() {
    println!("{:?}", *NAMES); // first time: runs the closure
    println!("{}", NAMES[0]); // afterwards: uses the cache
}
```

10.16.2.3 Why It's Called `LazyLock`

- **Lazy**: doesn't initialize until it's needed.
- **Lock**: there's a lock inside, so concurrent access from multiple `Threads` initializes only once (thread-safe).

10.16.3 Example Code

```rust
use std::sync::LazyLock;

static NAMES: LazyLock<Vec<String>> = LazyLock::new(|| {
    println!("initializing NAMES!");
    vec![String::from("Alice"), String::from("Bob"), String::from("Charlie")]
});

fn print_first() {
    println!("first name: {}", NAMES[0]);
}

fn main() {
    println!("program start");
    print_first(); // first access — initialization happens now
    print_first(); // second access — straight from the cache
    println!("{} names total", NAMES.len());
}
```

10.16.4 Recap

- A `static` initializer must be computable at compile time; operations such as building a populated `Vec` or calling `String::from` require runtime initialization.
- `LazyLock` postpones initialization to the first access and caches afterwards.
- `LazyLock` is thread-safe and can be used safely in a `static`.

10.17 `extern` Blocks

10.17.1 Goal of This Episode

Learn to call C functions and let C call Rust functions. This episode is only a rough tour of FFI. If you want a complete FFI example (from building a C library to calling it from Rust), search for a dedicated tutorial.

10.17.2 Concept

10.17.2.1 What Is FFI

FFI (Foreign Function Interface) is the mechanism that lets different programming languages call each other's functions. Rust can call functions written in C, and C can call functions written in Rust. Since nearly every language can interoperate with C, Rust can talk to most languages by using C as the bridge.

10.17.2.2 Calling C Functions

Declare external C functions with an `unsafe extern "C"` block:

```
unsafe extern "C" {
    fn fabs(x: f64) -> f64;
}

fn main() {
    let result = unsafe { fabs(-42.0) };
    println!("fabs(-42.0) = {}", result);
}
```

Calling an external function requires `unsafe` — Rust has no way to check whether the function on the C side is safe.

Since the Rust 2024 edition, the `extern` block itself also requires `unsafe` — because Rust can't verify that the function signatures you wrote in the declaration (parameter types, return type, etc.) are correct. If a signature doesn't match what's actually on the C side, that's undefined behavior.

10.17.2.3 `safe fn`

If you're certain an external function is safe, you can mark it `safe`:

```
unsafe extern "C" {
    safe fn fabs(x: f64) -> f64; // fabs is safe for every f64 input
}

fn main() {
    let result = fabs(-42.0); // callable without unsafe!
    println!("fabs(-42.0) = {}", result);
}
```

10.17.2.4 What the `"C"` Means

The `"C"` in `extern "C"` refers to the **ABI** (Application Binary Interface) — how functions are called at the binary level. `"C"` is the most common ABI; nearly every language can interoperate with the C ABI.

10.17.2.5 Letting C Call Rust

```
#[unsafe(no_mangle)]
pub extern "C" fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

- `extern "C"`: use the C ABI.
- `#[unsafe(no_mangle)]`: don't mangle the function name, so C can find it as `add`. In the 2024 edition, `no_mangle` is an `unsafe` attribute, because it changes how the function is linked, which can affect safety.

10.17.2.6 `extern` Blocks Can Declare `static` Variables Too

```
unsafe extern "C" {
    static daylight: i32; // a global variable on the C side
}
```

10.17.3 Example Code

```
unsafe extern "C" {
    safe fn fabs(x: f64) -> f64;
    fn sqrt(x: f64) -> f64;
}

#[unsafe(no_mangle)]
pub extern "C" fn rust_add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    // functions marked safe don't need unsafe
    println!("fabs(-10.0) = {}", fabs(-10.0));

    // unmarked ones do
    let root = unsafe { sqrt(25.0) };
    println!("sqrt(25.0) = {}", root);

    // Rust's extern "C" functions can also be called directly from Rust
    println!("rust_add(3, 4) = {}", rust_add(3, 4));
}
```

10.17.4 Recap

- `unsafe extern "C" { ... }` declares external C functions.
- Calling external functions requires `unsafe`, except those marked `safe fn`.
- `"C"` is the ABI — how functions are called at the binary level.
- `#[unsafe(no_mangle)] pub extern "C" fn` lets C call Rust.
- An `extern` block can declare `static` variables too.

10.18 `union`

10.18.1 Goal of This Episode

Meet `union` — where all fields share the same block of memory.

10.18.2 Concept

10.18.2.1 What Is a `union`

Each field of a `struct` occupies its own block of memory. A `union` is different — **all fields share the same block of memory**:

```
union IntOrBool {
    i: i32,
    b: bool,
}
```

`IntOrBool` is 4 bytes, enough to hold either its 4-byte `i32` or its 1-byte `bool`. `i` and `b` occupy the same memory — writing to `i` overwrites the contents of `b`.

10.18.2.2 Writing Needs No `unsafe`; Reading Does

```
    let u = IntOrBool { i: 1 };
    let value = unsafe { u.i }; // reading requires unsafe
```

Why does reading require `unsafe`? Because Rust doesn't know which field you last wrote. A `bool` in memory **must be 0 or 1**. If you write 42 through `i` and read it back through `b`, that memory contains 42 — not a valid value for a `bool`. That's **undefined behavior**. When reading a `union` field, you must guarantee yourself that the memory's contents are valid for the type you're reading — the compiler can't check this, hence the `unsafe`.

10.18.2.3 How It Differs from `enum`

	`enum`	`union`
Knows the current variant	Has a discriminant	Doesn't know; you track it yourself
Reading	Safe	Requires `unsafe`

10.18.2.4 The Use Case: FFI

In pure Rust you'll almost never need a `union` — `enum`s are safer and nicer. The main reason `union` exists is interoperating with C: C has `union`s, and you need Rust's `union` to match their memory layout.

10.18.3 Example Code

```
union IntOrBool {
    i: i32,
    b: bool,
}

fn main() {
    // writing needs no unsafe
    let u = IntOrBool { b: true };

    // reading does
    unsafe {
        // wrote b, reading b — fine
        println!("b = {}", u.b);
```

```
    }

    let v = IntOrBool { i: 42 };
    unsafe {
        println!("i = {}", v.i);
        // never do this:
        // println!("b = {}", v.b);
        // a bool must be 0 or 1, but this memory holds 42 → undefined behavior!
    }

    // IntOrBool is 4 bytes
    println!("size: {} bytes", std::mem::size_of::<IntOrBool>()); // 4
}
```

10.18.4 Recap

- All of a `union`'s fields share the same block of memory.
- Writing needs no `unsafe`; reading does — Rust doesn't know which field is stored inside.
- When reading, you must guarantee the memory is valid for the type — a `bool` must be 0 or 1; writing 42 then reading `b` is undefined behavior.
- Unlike an `enum`: a `union` has no discriminant and doesn't track the current variant.
- Its main use is FFI (matching C's `union`s).

10.19 The Never Type `!`

10.19.1 Goal of This Episode

Meet the `!` type — the type of things that never produce a value.

10.19.2 Concept

10.19.2.1 Functions That Never Return

Most functions finish and return a value. But some functions **never return**:

```
fn forever() -> ! {
    loop {
        // runs forever
    }
}
```

`-> !` means this function cannot possibly return.

10.19.2.2 What Has Type `!`

- `panic!("...")` — panics the current `Thread` instead of returning normally.
- `std::process::exit(0)` — the program ends.
- `loop {}` (with no break) — runs forever.
- A `return` expression itself
- A `break` expression itself
- A `continue` expression itself

10.19.2.3 ! Coerces into Any Type

This is !'s most useful property. An expression that never produces a value can sit anywhere a value is expected without contradiction — it's never actually going to produce one anyway.

This is why code like the following compiles:

```
    let x: i32 = match option {
        Some(v) => v,
        None => panic!("shouldn't be None"),
    };
```

Every arm of a `match` must return the same type. `Some(v) => v` returns `i32`, and `None => panic!(...)` returns `!`. Since `!` can convert into any type, it's treated as `i32`, and the `match`'s types line up.

`return`, `break`, and `continue` work the same way:

```
    let x: i32 = match option {
        Some(v) => v,
        None => return, // return has type !
    };
```

```
    for item in list {
        let value: i32 = match item.parse::<i32>() {
            Ok(n) => n,
            Err(_) => continue, // continue has type !
        };
        println!("{}", value);
    }
```

10.19.3 Example Code

```
fn exit_with_error(msg: &str) -> ! {
    println!("error: {}", msg);
    std::process::exit(1);
}

fn parse_or_exit(input: &str) -> i32 {
    match input.parse::<i32>() {
        Ok(n) => n,
        Err(_) => exit_with_error("please enter a valid number"), // ! treated as i32
    }
}

fn main() {
    let value = parse_or_exit("42");
    println!("parsed successfully: {}", value);

    // let bad = parse_or_exit("abc"); // this would call exit_with_error and end the program
}
```

10.19.4 Recap

- ! is the never type — it never produces a value.

- A `-> !` function never returns.
- `panic!`, `process::exit`, `return`, `break`, and `continue` all have type `!`.
- `!` coerces into any type — that's how a `match` can have one arm return a value and another panic.

Congratulations on finishing the advanced language features chapter! 🎉 This chapter covered Rust's advanced language features — from `dyn Trait`, compile-time computation, type conversion, attributes, and the macro system, to `unsafe`, `static`, FFI, `union`, and the never type. Most of these won't come up every day, but knowing they exist means you can reach for them when the need arises. In the next chapter we'll look at more practical tools in the standard library.

Chapter 11

Advanced Standard Library Topics

This chapter is a little different from the ones before it: it steps away from language features and instead introduces some of what's in the standard library. A tutorial like this can't be exhaustive, but knowing a bit in this area still makes life a lot easier.

11.1 `AsRef<T>` / `AsMut<T>`

11.1.1 Goal of This Episode

Learn to use `AsRef` and `AsMut` so a function can accept multiple types.

11.1.2 Main Text

11.1.2.1 Motivation

Suppose you wrote a function that takes a `&str`:

```
fn print_length(s: &str) {
    println!("length: {}", s.len());
}
```

If the caller holds a `String`, `&String` converts to `&str` automatically thanks to `Deref`, so that's fine. But what if you want a function that accepts `String`, `&str`, and maybe other types all at once?

11.1.2.2 `AsRef`

The `AsRef<T>` `trait` means "I can be cheaply borrowed as a `&T`":

```
fn print_length(s: impl AsRef<str>) {
    println!("length: {}", s.as_ref().len());
}

fn main() {
    let text = String::from("hello");

    print_length("hi");                    // &str
    print_length(&text);
    print_length(String::from("welcome")); // String

    println!("{text}"); // still usable
}
```

The standard library already implements `AsRef` for many types:

- `String: AsRef<str>`

- `String: AsRef<[u8]>`
- `Vec<T>: AsRef<[T]>`

11.1.2.3 `AsMut`

`AsMut<T>` is the mutable version — borrow as `&mut T`:

In most cases, we use `AsMut` to modify a value the caller already owns, not to take ownership of it. Therefore, the parameter is usually written as `&mut impl AsMut<T>` rather than `impl AsMut<T>`.

```
fn fill_zeros(buf: &mut impl AsMut<[u8]>) {
    for byte in buf.as_mut() {
        *byte = 0;
    }
}

fn main() {
    let mut v = vec![1, 2, 3];
    fill_zeros(&mut v);
    println!("{:?}", v); // [0, 0, 0]
}
```

11.1.2.4 Ownership and When to Use It

`AsRef` and `AsMut` only describe which kind of reference a type can provide. They do not determine whether a function takes ownership of its argument; that depends on the parameter type and what the caller passes in.

Although `s: impl AsRef<str>` is a by-value parameter, the concrete type passed in can itself be a reference. `print_length(&text)` only borrows `text`, so it remains usable afterward; passing `text` directly would move it. This lets the caller choose between an owned value and a reference.

In `fill_zeros`, the outer `&mut` means that the function only borrows the buffer, while `AsMut<[u8]>` means that the buffer can provide a `&mut [u8]`.

11.1.2.5 How It Differs from `Deref`

`Deref` is used automatically in places like `deref` coercion and method calls: Rust borrows through the value for you. `AsRef` is a manual `.as_ref()` call.

The more important difference: each type can have only one `Deref` target (`String`'s target is `str`), but it can implement multiple `AsRefs` (`String` is both `AsRef<str>` and `AsRef<[u8]>`). Same for `AsMut`.

Use `AsRef<T>` or `AsMut<T>` when a function needs a common way to borrow several possible input types.

11.1.3 Recap

- `AsRef<T>` and `AsMut<T>` let a function borrow multiple input types as `&T` and `&mut T`, respectively.
- `AsRef` / `AsMut` do not decide ownership: `impl AsRef<T>` can receive an owned value or a reference, while `&mut impl AsMut<T>` borrows and modifies the original value.

- Conversions require an explicit `.as_ref()` or `.as_mut()` call; Rust can use `Deref` automatically.
- A type can implement multiple `AsRefs` / `AsMuts`, but it can have only one `Deref` target.

11.2 `Ordering` and Sorting

11.2.1 Goal of This Episode

Meet `Ordering`, the `min`/`max` family of functions, the sorting methods, and how `Reverse` works.

11.2.2 Concept

11.2.2.1 `Ordering`

Chapter 5 covered the `Ord trait` — types implementing `Ord` can be compared. `Ord`'s core method is `cmp`, which compares two values and returns `std::cmp::Ordering` — an `enum` with just three values:

```
use std::cmp::Ordering;

fn main() {
    match 5.cmp(&3) {
        Ordering::Less => println!("smaller"),
        Ordering::Equal => println!("equal"),
        Ordering::Greater => println!("greater"),
    }
}
```

11.2.2.2 `min` / `max`

`std::cmp::min(a, b)` and `std::cmp::max(a, b)` return the smaller or larger of two values, requiring the type to implement `Ord`:

```
use std::cmp;

fn main() {
    println!("{}", cmp::min(3, 7)); // 3
    println!("{}", cmp::max(3, 7)); // 7
}
```

11.2.2.3 The Trouble with Floats

`f64` doesn't implement `Ord` (Chapter 5 mentioned why: `NAN` compares as ordered against nothing at all), so you can't use `cmp::min` on it directly.

`f64` only has `PartialOrd`, whose method `partial_cmp` returns `Option<Ordering>` instead of `Ordering` — when `NAN` shows up there's no way to compare, so it can only return `None`.

In that case use `min_by` / `max_by` with custom comparison logic:

```
use std::cmp;

fn main() {
    let smaller = cmp::min_by(3.0_f64, 2.5, |a, b| {
        a.partial_cmp(b).unwrap() // if you're sure NAN can't appear, unwrap the Ordering
```

```
    });
    println!("{}", smaller); // 2.5

    let bigger = cmp::max_by(3.0_f64, 2.5, |a, b| {
        a.partial_cmp(b).unwrap()
    });
    println!("{}", bigger); // 3
}
```

The closure passed to `min_by` / `max_by` returns an `Ordering` — you decide how to compare.

11.2.2.4 `min_by_key` / `max_by_key`

Compare by some key:

```
use std::cmp;

fn main() {
    let short = cmp::min_by_key("hello", "hi", |s| s.len());
    println!("{}", short); // "hi"
}
```

11.2.2.5 Sorting

`Vec` and slices provide several sorting methods:

```
fn main() {
    let mut nums = vec![3, 1, 4, 1, 5];

    // sort: ascending, requires Ord
    nums.sort();
    println!("{:?}", nums); // [1, 1, 3, 4, 5]

    // sort_by: custom comparison; pass a closure returning Ordering
    nums.sort_by(|a, b| b.cmp(a));
    println!("{:?}", nums); // [5, 4, 3, 1, 1]

    // sort_by_key: sort by a key
    let mut words = vec!["banana", "apple", "fig"];
    words.sort_by_key(|w| w.len());
    println!("{:?}", words); // ["fig", "apple", "banana"]
}
```

11.2.2.6 `Reverse`

`std::cmp::Reverse` flips the sort order:

```
use std::cmp::Reverse;

fn main() {
    let mut nums = vec![3, 1, 4, 1, 5];
    nums.sort_by_key(|&x| Reverse(x));
    println!("{:?}", nums); // [5, 4, 3, 1, 1]
}
```

How does that work? `Reverse` is really just a newtype:

```
pub struct Reverse<T>(pub T);
```

Its `Ord` implementation flips the comparison around:

```
impl<T: Ord> Ord for Reverse<T> {
    fn cmp(&self, other: &Reverse<T>) -> Ordering {
        other.0.cmp(&self.0) // note: other compared against self — reversed
    }
}
```

Normally `5.cmp(&3)` returns `Greater`, but `Reverse(5).cmp(&Reverse(3))` internally does `3.cmp(&5)` and returns `Less`. `sort_by_key` orders by the keys' `cmp`, and once the key is wrapped in `Reverse`, the comparison logic reverses automatically.

Compared to `sort_by(|a, b| b.cmp(a))`, the `Reverse` spelling makes the intent clearer.

11.2.3 Example Code

```
use std::cmp::{self, Reverse};

fn main() {
    // min / max
    println!("min(10, 20) = {}", cmp::min(10, 20));
    println!("max(10, 20) = {}", cmp::max(10, 20));

    // floats need min_by / max_by
    let smaller = cmp::min_by(1.5_f64, 2.3, |a, b| {
        a.partial_cmp(b).unwrap()
    });
    println!("min_by(1.5, 2.3) = {}", smaller);

    // sorting
    let mut scores = vec![85, 92, 78, 95, 88];
    scores.sort();
    println!("ascending: {:?}", scores);

    scores.sort_by_key(|&s| Reverse(s));
    println!("descending: {:?}", scores);

    // sort by string length
    let mut names = vec!["Alice", "Bob", "Charlie", "Dave"];
    names.sort_by_key(|n| n.len());
    println!("by length: {:?}", names);
}
```

11.2.4 Recap

- `Ordering` has three values: `Less`, `Equal`, `Greater`.
- `cmp::min` / `cmp::max` take the smaller/larger of two values and require `Ord`.
- `f64` has no `Ord`; use `min_by` / `max_by` with custom comparison.
- `min_by_key` / `max_by_key` compare by a key.
- `sort()` sorts ascending, `sort_by()` takes custom comparison, `sort_by_key()` sorts by key.

- `Reverse` is a tuple `struct` whose `Ord` implementation flips the comparison, so sort results reverse with it.

11.3 `HashMap<K, V>`

11.3.1 Goal of This Episode

Learn to store and look up key-value data with `HashMap`.

11.3.2 Concept

11.3.2.1 Motivation

If you want to look up a score by name or a user by ID, a `Vec` can certainly do it — store a pile of `(name, score)` tuples and walk from the start until you find the matching name. But the more data, the slower that gets.

`HashMap<K, V>` solves this. It uses a hash function to narrow down where a key may be stored, so it can usually find the value without walking through every entry.

11.3.2.2 Creation and Basic Operations

```
use std::collections::HashMap;

fn main() {
    let mut scores = HashMap::new();
    scores.insert("Alice", 95);
    scores.insert("Bob", 80);

    println!("{:?}", scores.get("Alice")); // Some(&95)
    println!("{:?}", scores.get("Eve"));   // None
}
```

`insert` puts a pair in; `get` looks up and returns `Option<&V>` (`None` if the key doesn't exist); `remove` deletes and returns `Option<V>` (`Some(removed value)` if the key existed, `None` otherwise).

Calling `insert` again with the same key overwrites the old value.

11.3.2.3 Building from an `Iterator` with `collect`

```
use std::collections::HashMap;

fn main() {
    let scores: HashMap<&str, i32> = vec![("Alice", 95), ("Bob", 80)]
        .into_iter()
        .collect();
}
```

11.3.2.4 Iterating

```
use std::collections::HashMap;

fn main() {
    let scores: HashMap<&str, i32> = vec![("Alice", 95), ("Bob", 80)]
```

```
        .into_iter()
        .collect();
    for (name, score) in &scores {
        println!("{}: {}", name, score);
    }
}
```

Note that the iteration order is **not fixed** — it can differ between runs. If you need a fixed order, use `BTreeMap` (introduced later).

11.3.2.5 What `Hash` Is

A `HashMap` needs to find the value for a key quickly. It feeds the key into a **hash function** to compute a number called a hash value. The map uses that number to decide where to start looking in its internal table. If multiple keys lead to the same area, it may examine several candidate locations until it finds the key or determines that it isn't there. This lets it locate keys without going through every stored entry.

So the key type must implement the `Hash` `trait` — which tells Rust how to feed values of that type into a hasher.

11.3.2.6 Key Requirements: `Eq + Hash`

Besides `Hash`, keys also need `Eq`. A hash value only narrows down the search; it does not uniquely identify a key. Different keys can lead to the same area, so the `HashMap` uses `==` to confirm that a candidate is the key you asked for.

Most basic types (integers, `bool`, `char`, `&str`, `String`) already implement `Eq + Hash`. `f64` implements neither `Eq` nor `Hash`, so it can't be used directly as a key (`NaN` is one reason).

11.3.2.7 Implementing `Hash` for Your Own Types

`Hash` can be derived:

```
use std::collections::HashMap;

#[derive(Debug, PartialEq, Eq, Hash)]
struct Student {
    name: String,
    grade: i32,
}

fn main() {
    let mut map = HashMap::new();
    map.insert(Student { name: String::from("Alice"), grade: 90 }, "honors");
}
```

Note that you need `PartialEq`, `Eq`, and `Hash` all together — since `Eq: PartialEq`, all three are required.

As a rule of thumb, whenever you `derive` `PartialEq` and `Eq`, it's a good idea to `derive` `Hash` along with them. It costs nothing extra, and your type won't need revisiting later when it has to serve as a `HashMap` key.

11.3.2.8 The `entry` API

"Leave it if present, insert if not" is a very common need:

```
use std::collections::HashMap;

fn main() {
    let mut scores = HashMap::new();
    scores.insert("Alice", 95);

    scores.entry("Alice").or_insert(0); // Alice exists — untouched
    scores.entry("Eve").or_insert(0);   // Eve doesn't — insert 0
}
```

`or_insert` returns a `&mut V`, so you can modify it directly. This is especially handy for counting:

```
    let words = vec!["hello", "world", "hello", "rust"];
    let mut counts = HashMap::new();

    for word in words {
        let count = counts.entry(word).or_insert(0);
        *count += 1;
    }
    // {"hello": 2, "world": 1, "rust": 1}
```

11.3.2.9 Other Common Methods

`HashMap` has a few more methods you'll use often:

- `.contains_key(&key)`: checks whether a key exists, returns `bool`.
- `.len()`: how many key-value pairs there are.
- `.is_empty()`: whether it's empty.
- `.keys()`: an iterator over all keys
- `.values()`: an iterator over all values

11.3.3 Example Code

```
use std::collections::HashMap;

fn main() {
    // count how many times each character appears
    let text = "hello world";
    let mut char_counts = HashMap::new();

    for c in text.chars() {
        if c == ' ' { continue; }
        let count = char_counts.entry(c).or_insert(0);
        *count += 1;
    }

    // print the results (order not fixed)
    for (ch, count) in &char_counts {
        println!("'{}': {} times", ch, count);
    }

    // find the most frequent character
```

```
    if let Some((ch, count)) = char_counts.iter().max_by_key(|(_, count)| *count) {
        println!("most frequent is '{}', {} times", ch, count);
    }
}
```

11.3.4 Recap

- `HashMap<K, V>` uses a hash to find values by key without walking through every entry.
- `insert` adds, `get` looks up (returns `Option<&V>`), `remove` deletes.
- Keys must implement `Eq + Hash`; `Hash` can be `derived`.
- `f64` can't be a key (no `Eq`).
- `.entry(k).or_insert(v)` is the idiom for "insert only if absent"; it returns `&mut V`.
- Iteration order is not fixed.

11.4 HashSet<T>

11.4.1 Goal of This Episode

Learn to work with set operations using `HashSet`.

11.4.2 Concept

11.4.2.1 Motivation

A `HashMap` stores key-value pairs, but sometimes you only care about "is it there or not" and not any associated value — say, tracking which users are online, or which words have appeared. That's what `HashSet` is for.

11.4.2.2 The Essence

A `HashSet` is really just a `HashMap` with only keys and no values. So its elements likewise require `Eq + Hash`.

11.4.2.3 Basic Operations

```
use std::collections::HashSet;

fn main() {
    let mut fruits = HashSet::new();
    fruits.insert("apple");
    fruits.insert("banana");
    fruits.insert("apple"); // duplicate — won't be added

    println!("{}", fruits.contains("apple")); // true
    println!("{}", fruits.len());             // 2

    fruits.remove("banana");
}
```

11.4.2.4 Building from an `Iterator`

```
use std::collections::HashSet;

fn main() {
    let nums: HashSet<i32> = vec![1, 2, 3, 2, 1].into_iter().collect();
    println!("{:?}", nums); // {1, 2, 3} — duplicates removed automatically
}
```

11.4.2.5 Set Operations

This is where `HashSet` shines:

```
use std::collections::HashSet;

fn main() {
    let a: HashSet<i32> = [1, 2, 3].into_iter().collect();
    let b: HashSet<i32> = [2, 3, 4].into_iter().collect();

    // intersection: in both
    let intersection: HashSet<_> = a.intersection(&b).copied().collect();
    // {2, 3}

    // union: everything combined
    let union_set: HashSet<_> = a.union(&b).copied().collect();
    // {1, 2, 3, 4}

    // difference: in a but not in b
    let diff: HashSet<_> = a.difference(&b).copied().collect();
    // {1}

    // symmetric difference: in exactly one side
    let sym_diff: HashSet<_> = a.symmetric_difference(&b).copied().collect();
    // {1, 4}
}
```

11.4.2.6 Operators

The advanced language features chapter covered operator overloading — `HashSet` puts it to use. You can apply `&` `|` `-` `^` to references of two `HashSet`s for set operations:

```
    let intersection = &a & &b; // intersection
    let union_set    = &a | &b; // union
    let diff         = &a - &b; // difference
    let sym_diff     = &a ^ &b; // symmetric difference
```

11.4.2.7 Other Relations

```
use std::collections::HashSet;

fn main() {
    let small: HashSet<i32> = [1, 2].into_iter().collect();
    let big: HashSet<i32> = [1, 2, 3, 4].into_iter().collect();

    println!("{}", small.is_subset(&big));   // true
    println!("{}", big.is_superset(&small)); // true
```

```
    println!("{}", small.is_disjoint(&big)); // false (they intersect)
}
```

11.4.2.8 Iterating

Like HashMap, iteration order isn't fixed:

```
    for fruit in &fruits {
        println!("{}", fruit);
    }
```

11.4.3 Example Code

```
use std::collections::HashSet;

fn main() {
    let class_a: HashSet<&str> = ["Alice", "Bob", "Charlie", "Dave"].into_iter().collect();
    let class_b: HashSet<&str> = ["Charlie", "Dave", "Eve", "Frank"].into_iter().collect();

    println!("Class A: {:?}", class_a);
    println!("Class B: {:?}", class_b);

    // people in both classes
    let both = &class_a & &class_b;
    println!("in both: {:?}", both);

    // everyone
    let all = &class_a | &class_b;
    println!("everyone: {:?}", all);

    // people only in class A
    let only_a = &class_a - &class_b;
    println!("only in A: {:?}", only_a);

    // removing duplicates
    let words = vec!["hello", "world", "hello", "rust", "world"];
    let unique: HashSet<_> = words.into_iter().collect();
    println!("unique words: {:?}", unique);
}
```

11.4.4 Recap

- HashSet<T> is a keys-only HashMap; elements don't repeat.
- Elements must implement Eq + Hash.
- insert adds, contains checks, remove removes.
- Set operations: intersection, union, difference, symmetric_difference.
- Operators work too: & (intersection), | (union), - (difference), ^ (symmetric difference).
- is_subset, is_superset, is_disjoint test the other relations.

11.5 A Brief Tour of Other Collections

11.5.1 Goal of This Episode

Meet `BTreeMap`, `BTreeSet`, and `VecDeque`.

11.5.2 Concept

`HashMap` and `HashSet` are the most commonly used collections, but the standard library has other options.

11.5.2.1 `BTreeMap`

The difference from `HashMap`: the keys are **ordered**. Iteration follows the keys' sort order, not a random one:

```
use std::collections::BTreeMap;

fn main() {
    let mut scores = BTreeMap::new();
    scores.insert("Charlie", 70);
    scores.insert("Alice", 90);
    scores.insert("Bob", 85);

    for (name, score) in &scores {
        println!("{}: {}", name, score);
    }
    // always alphabetical: Alice, Bob, Charlie
}
```

The cost: keys must implement `Ord` (rather than `Hash + Eq`). On lookup speed, `HashMap` is nearly constant regardless of size; `BTreeMap` gets slightly slower with more data, but it's still fast.

11.5.2.2 `BTreeSet`

`BTreeSet` is a keys-only `BTreeMap`, in the same relationship as `HashSet` is to `HashMap`. Elements are ordered, and iteration outputs them in order:

```
use std::collections::BTreeSet;

fn main() {
    let mut set = BTreeSet::new();
    set.insert(3);
    set.insert(1);
    set.insert(2);

    for x in &set {
        print!("{} ", x);
    }
    // 1 2 3
}
```

All of `HashSet`'s set operations (intersection, union, etc.) exist on `BTreeSet` too.

11.5.2.3 Which One When

- Don't care about order → `HashMap` / `HashSet` (faster).
- Need ordered iteration, or need the smallest/largest key → `BTreeMap` / `BTreeSet`.

11.5.2.4 `VecDeque`

A `Vec` can only `push` / `pop` efficiently at the tail. `insert` or `remove` at the head means shifting every later element over by one — the more data, the slower.

`VecDeque` (a double-ended queue) is efficient at both the head and the tail, with nearly constant speed no matter the size:

```
use std::collections::VecDeque;

fn main() {
    let mut deque = VecDeque::new();
    deque.push_back(1);
    deque.push_back(2);
    deque.push_front(0);

    println!("{:?}", deque); // [0, 1, 2]

    deque.pop_front(); // removes 0
    deque.pop_back();  // removes 2
    println!("{:?}", deque); // [1]
}
```

11.5.2.5 When to Use `VecDeque`

When you need a first-in-first-out (FIFO) queue, or frequent operations at both ends. If you only touch the tail, `Vec` is enough.

11.5.3 Example Code

```
use std::collections::{BTreeMap, VecDeque};

fn main() {
    // BTreeMap: ordered key-value
    let mut scores = BTreeMap::new();
    scores.insert("Charlie", 70);
    scores.insert("Alice", 90);
    scores.insert("Bob", 85);
    scores.insert("Dave", 60);

    // always prints alphabetically
    for (name, score) in &scores {
        println!("{}: {}", name, score);
    }

    // VecDeque: double-ended queue
    let mut queue = VecDeque::new();
    queue.push_back("first");
    queue.push_back("second");
    queue.push_back("third");
```

```
    // take from the front — first in, first out
    while let Some(item) = queue.pop_front() {
        println!("processing: {}", item);
    }
}
```

11.5.4 Recap

- `BTreeMap`: iteration follows key order; keys must implement `Ord`.
- `BTreeSet`: iteration follows element order; elements must implement `Ord`.
- Use the `BTree` family for ordered iteration; otherwise the `Hash` family (faster).
- `VecDeque`: double-ended queue, fast at both ends.
- `Vec` is only fast at the tail; head operations are slow (all elements shift).

11.6 `std::env` / `std::process`

11.6.1 Goal of This Episode

Learn to read command-line arguments and environment variables, and to control how the program exits.

11.6.2 Concept

11.6.2.1 Command-line Arguments

A program can be given arguments when run, e.g. `cargo run -- hello world`. Get them with `std::env::args()`:

```
use std::env;

fn main() {
    for arg in env::args() {
        println!("{}", arg);
    }
}
```

The first one is the path of the program itself; your arguments come after. Usually you `collect` them into a `Vec`:

```
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
    if args.len() < 2 {
        println!("please provide an argument");
        return;
    }
    println!("you entered: {}", args[1]);
}
```

11.6.2.2 Environment Variables

Environment variables are a set of key-value settings provided by the operating system; programs can read them to get information about the system. If you're not familiar with environment variables, look them up on your own.

```
use std::env;

fn main() {
    match env::var("HOME") {
        Ok(val) => println!("HOME = {}", val),
        Err(_) => println!("HOME is not set"),
    }
}
```

`env::var` returns `Result<String, VarError>`. If the environment variable doesn't exist, it returns `Err`.

11.6.2.3 `process::exit`

```
use std::process;

fn main() {
    process::exit(1); // end the program immediately with error code 1
}
```

Returning 0 conventionally means success; nonzero means failure. As we learned in the advanced language features chapter, `process::exit`'s return type is `!` (the never type).

11.6.2.4 `eprintln!`

```
fn main() {
    eprintln!("this is an error message");
    println!("this is normal output");
}
```

`println!` writes to **`stdout`** (standard output); `eprintln!` writes to **`stderr`** (standard error). They look the same in a terminal, but they can be redirected to different places. Error messages should use `eprintln!`.

11.6.3 Example Code

```
use std::env;
use std::process;

fn main() {
    let args: Vec<String> = env::args().collect();

    if args.len() < 2 {
        eprintln!("usage: {} <name>", args[0]);
        process::exit(1);
    }

    let name = &args[1];
    println!("hello, {}!", name);
```

```
    // print some environment variables
    if let Ok(home) = env::var("HOME") {
        println!("your HOME directory: {}", home);
    }

    if let Ok(path) = env::var("PATH") {
        let preview: String = path.chars().take(50).collect();
        println!("first 50 characters of PATH: {}", preview);
    }
}
```

11.6.4 Recap

- `env::args()` returns an iterator over the command-line arguments; the first is the program path.
- `env::var("NAME")` returns a `Result`.
- `process::exit(code)` ends the program immediately; its return type is `!`.
- `eprintln!` writes to `stderr` — use it for error messages.

11.7 `std::path`

11.7.1 Goal of This Episode

Learn to handle cross-platform paths with `Path` and `PathBuf`.

11.7.2 Concept

11.7.2.1 Motivation

Programs constantly read and write files — for example, reading config files or writing logs. We'll learn how to do that soon. But first, we need a way to express "where the file is."

Path formats differ across operating systems — Windows uses \, while Linux / macOS use /. If you glue paths together from raw strings, cross-platform code can break. `std::path` handles those differences for you.

11.7.2.2 `Path` and `PathBuf`

The same relationship as `str` and `String`:

- `Path` corresponds to `str` — a DST you can't hold directly; usually used as `&Path`.
- `PathBuf` corresponds to `String` — the owned, modifiable version.

```
use std::path::{Path, PathBuf};

fn main() {
    let p = Path::new("/home/user/file.txt");

    let mut buf = PathBuf::from("/home/user");
    buf.push("documents");
    buf.push("file.txt");
    println!("{}", buf.display()); // /home/user/documents/file.txt
}
```

push automatically inserts the correct path separator.

11.7.2.3 Common Methods

```
use std::path::Path;

fn main() {
    let p = Path::new("/home/user/notes.txt");

    println!("{:?}", p.parent());    // Some("/home/user")
    println!("{:?}", p.file_name()); // Some("notes.txt")
    println!("{:?}", p.extension()); // Some("txt")
    println!("{:?}", p.file_stem()); // Some("notes")
    println!("{}", p.exists());      // does the path exist
    println!("{}", p.is_file());     // is it a file
    println!("{}", p.is_dir());      // is it a directory
}
```

`file_name`, `extension`, and `file_stem` return `Option<&OsStr>`, not `Option<&str>` — because on some operating systems a file name isn't necessarily valid UTF-8. Most of the time you can convert to `&str` with `.to_str().unwrap()`.

11.7.2.4 `join`

`join` is like `push`, but instead of modifying the original `Path` or `PathBuf`, it returns a new `PathBuf`:

```
use std::path::Path;

fn main() {
    let dir = Path::new("/home/user");
    let file = dir.join("documents").join("file.txt");
    println!("{}", file.display()); // /home/user/documents/file.txt
}
```

11.7.2.5 Converting To and From Strings

```
use std::path::{Path, PathBuf};

fn main() {
    // &str → &Path
    let p = Path::new("hello.txt");

    // &str → PathBuf
    let buf = PathBuf::from("/some/path");

    // PathBuf → String (possibly lossy; non-UTF-8 characters get replaced)
    let s: String = buf.to_string_lossy().into_owned();
}
```

11.7.3 Example Code

```
use std::path::{Path, PathBuf};

fn show_info(path: &Path) {
```

```
    println!("path: {}", path.display());

    if let Some(parent) = path.parent() {
        println!("  parent: {}", parent.display());
    }
    if let Some(name) = path.file_name() {
        println!("  file name: {:?}", name);
    }
    if let Some(ext) = path.extension() {
        println!("  extension: {:?}", ext);
    }
    println!("  exists: {}", path.exists());
}

fn main() {
    show_info(Path::new("/home/user/notes.txt"));

    // building a path with PathBuf
    let mut config_path = PathBuf::from("/home/user");
    config_path.push(".config");
    config_path.push("app");
    config_path.push("settings.toml");
    show_info(&config_path);

    // join leaves the original Path unchanged
    let base = Path::new("/var/log");
    let log_file = base.join("app.log");
    show_info(&log_file);
}
```

11.7.4 Recap

- `Path` is a DST (corresponding to `str`); `PathBuf` is the owned version (corresponding to `String`).
- `push` / `join` insert the correct path separator automatically.
- `parent`, `file_name`, `extension`, `file_stem` take paths apart.
- `exists`, `is_file`, `is_dir` check a path's status.

11.8 String Methods

11.8.1 Goal of This Episode

Meet the most commonly used methods on `&str` and `String`, and Rust strings' relationship with UTF-8.

11.8.2 Concept

When you read and write files, the content usually comes as strings, and you'll need all kinds of methods to work with them — searching, splitting, trimming, replacing, and so on. We've used `.trim()`, `.parse()`, and `.chars()` before, but `&str` and `String` carry a great many more useful methods. This episode covers the most common ones.

11.8.2.1 Searching

```
    let s = "hello, world!";

    s.contains("world");    // true
    s.starts_with("hello"); // true
    s.ends_with("!");       // true
    s.find("world");        // Some(7) — position of the first occurrence (byte index)
```

11.8.2.2 Trimming and Replacing

```
    "  hello  ".trim();       // "hello"
    "  hello  ".trim_start(); // "hello  "
    "  hello  ".trim_end();   // "  hello"

    "hello world".replace("world", "Rust"); // "hello Rust"
```

11.8.2.3 Splitting

```
    let parts: Vec<&str> = "a,b,c".split(',').collect();
    // ["a", "b", "c"]

    let words: Vec<&str> = "hello  world".split_whitespace().collect();
    // ["hello", "world"]
```

`split` returns an iterator, usually paired with `collect`.

11.8.2.4 Iterating Character by Character

```
fn main() {
    for c in "hello".chars() {
        println!("{}", c);
    }
}
```

`.chars()` returns an iterator of Unicode characters. There's also `.bytes()` for raw bytes.

11.8.2.5 Case

```
    "Hello".to_uppercase(); // "HELLO"
    "Hello".to_lowercase(); // "hello"
```

11.8.2.6 `len` Counts Bytes

```
    "hello".len();         // 5
    "hello".is_empty();    // false
    "hello".repeat(3);     // "hellohellohello"

    // note: .len() returns the byte count, not the character count
    "你好".len();           // 6 (UTF-8 bytes)
    "你好".chars().count(); // 2 (characters)
```

11.8.3 Example Code

```rust
fn main() {
    let sentence = "  Hello, Rust World!  ";

    // trim whitespace
    let trimmed = sentence.trim();
    println!("trimmed: '{}'", trimmed);

    // search
    println!("contains Rust: {}", trimmed.contains("Rust"));
    println!("position of Rust: {:?}", trimmed.find("Rust"));

    // split
    let words: Vec<&str> = trimmed.split_whitespace().collect();
    println!("word count: {}", words.len());
    for word in &words {
        println!("  {}", word);
    }

    // replace
    let replaced = trimmed.replace("Rust", "World");
    println!("after replacing: {}", replaced);

    // UTF-8
    let chinese = "你好世界";
    println!("bytes: {}", chinese.len());                // 12
    println!("characters: {}", chinese.chars().count()); // 4

    for (i, c) in chinese.chars().enumerate() {
        println!("character {}: {}", i + 1, c);
    }
}
```

11.8.4 Recap

- `contains`, `starts_with`, `ends_with`, `find`: searching
- `trim`, `trim_start`, `trim_end`: trimming whitespace
- `replace`: replacing
- `split`, `split_whitespace`: splitting; they return iterators.
- `chars`: iterate by character; `bytes`: iterate by byte.
- `len` returns the byte count; use `.chars().count()` for characters.

11.9 I/O: `stdin` and File Access

11.9.1 Goal of This Episode

Meet Rust's I/O (input / output) methods and learn to read and write files.

11.9.2 Concept

We've learned to express file locations with `Path` and to work with string content — this episode covers actually reading and writing files.

11.9.2.1 `stdin` Revisited

In Chapter 1 we copied these three lines verbatim to read user input:

```
    let mut input = String::new();
    std::io::stdin().read_line(&mut input).expect("read failed");
    let name = input.trim();
```

You've since learned `String`, `&mut`, and `.expect()`, so this should all make sense now. What we haven't mentioned: `std::io::stdin()` returns a `Stdin` `struct`, and `read_line` returns an `io::Result<usize>` — a type alias for `Result<usize, io::Error>` that many I/O functions in the standard library use.

11.9.2.2 The Simplest File Reading and Writing

`fs::read_to_string`'s parameter type is `impl AsRef<Path>` — the `AsRef` from Episode 1 pays off here. You can pass a `&str`, `String`, `&Path`, or `PathBuf`, no manual conversion needed.

Reading a whole file into a string:

```
use std::fs;

fn main() {
    let content = fs::read_to_string("hello.txt").expect("read failed");
    println!("{}", content);
}
```

Writing to a file (created if missing, overwritten if present):

```
use std::fs;

fn main() {
    fs::write("output.txt", "Hello, file!").expect("write failed");
}
```

These two functions are wonderfully simple, but `read_to_string` reads the entire file into memory at once — not a good fit for very large files.

11.9.2.3 `File` + `BufReader`: Reading Line by Line

```
use std::fs::File;
use std::io::{BufRead, BufReader};

fn main() {
    let file = File::open("data.txt").expect("open failed");
    let reader = BufReader::new(file);

    for line in reader.lines() {
        let line = line.expect("failed to read line");
        println!("{}", line);
    }
}
```

`File::open` opens the file, `BufReader` wraps it to provide a buffer, and `.lines()` reads line by line — each line is an `io::Result<String>`.

11.9.2.4 Writing to a File

```
use std::fs::File;
use std::io::Write;

fn main() {
    let mut file = File::create("output.txt").expect("failed to create");
    writeln!(file, "first line").expect("write failed");
    writeln!(file, "second line").expect("write failed");
}
```

`File::create` creates (or overwrites) a file, and `writeln!` is a lot like `println!`, just with the output going to the file instead of the screen. Using `writeln!` requires importing the `Write` `trait`.

11.9.2.5 `Read`, `Write`, `BufRead`

The standard library abstracts I/O with `traits`:

- `Read`: things bytes can be read from (`File`, `Stdin`, `TcpStream`, etc.).
- `Write`: things bytes can be written to (`File`, `Stdout`, `TcpStream`, etc.).
- `BufRead`: buffered reading, providing `lines()` and other methods. `BufReader` can turn any `Read` into a `BufRead`.

That's why many functions declare parameters as `impl Read` or `impl Write` — whether you pass a file, `stdin`, or a network connection, anything implementing the right `trait` works.

11.9.3 Example Code

```
use std::fs::{self, File};
use std::io::{self, BufRead, BufReader, Write};

fn main() -> io::Result<()> {
    // write a file
    let mut file = File::create("names.txt")?;
    writeln!(file, "Alice")?;
    writeln!(file, "Bob")?;
    writeln!(file, "Charlie")?;

    // read the whole file at once
    let all = fs::read_to_string("names.txt")?;
    println!("whole file:\n{}", all);

    // read line by line
    let file = File::open("names.txt")?;
    let reader = BufReader::new(file);
    for (i, line) in reader.lines().enumerate() {
        println!("line {}: {}", i + 1, line?);
    }

    Ok(())
```

```
}
```

11.9.4 Recap

- io::Result<T> is a type alias for Result<T, io::Error>.
- fs::read_to_string / fs::write: the simplest one-line read/write
- File::open + BufReader: read big files line by line
- File::create + writeln!: write line by line.
- Read, Write, and BufRead are the core I/O traits, giving different sources (files, stdin, the network) one shared interface.

11.10 The Error trait

11.10.1 Goal of This Episode

Learn to define custom error types, and to handle errors of different kinds uniformly with Box<dyn Error>.

11.10.2 Concept

11.10.2.1 Recap: Result and ?

Chapter 5 covered Result<T, E> and the ? operator. But the error types back then were simple — one function produced one kind of error. Real programs often face several: reading a file can fail (io::Error), and parsing a number can fail too (ParseIntError). If both can happen inside one function, what do you put for the E in the returned Result?

11.10.2.2 The Error trait

The standard library defines the std::error::Error trait, the common interface of all error types:

```
pub trait Error: std::fmt::Display + std::fmt::Debug {
    fn source(&self) -> Option<&(dyn Error + 'static)> { None }
}
```

To implement Error, your type must first implement Display and Debug. .source() returns the underlying cause of this error, defaulting to None.

11.10.2.3 Custom Error Types

Wrap all the possible errors together in an enum:

```
use std::fmt;

#[derive(Debug)]
enum AppError {
    Io(std::io::Error),
    Parse(std::num::ParseIntError),
}

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
```

```
        match self {
            AppError::Io(_) => write!(f, "I/O operation failed"),
            AppError::Parse(_) => write!(f, "failed to parse an integer"),
        }
    }
}

impl std::error::Error for AppError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            AppError::Io(e) => Some(e),
            AppError::Parse(e) => Some(e),
        }
    }
}
```

For example, `AppError::Parse` displays "failed to parse an integer," while `.source()` returns the original `ParseIntError` so callers can inspect the detailed parsing error when needed. This preserves both the outer message and the underlying cause without printing the same error text twice.

Chapter 5 said `?` returns early when it meets an `Err`. Actually `?` does one more thing: it calls `From::from(e)` to convert the error into the `E` of the function's return type. So as long as you implement `From` for the underlying errors, `?` converts automatically:

```
impl From<std::io::Error> for AppError {
    fn from(e: std::io::Error) -> Self {
        AppError::Io(e)
    }
}

impl From<std::num::ParseIntError> for AppError {
    fn from(e: std::num::ParseIntError) -> Self {
        AppError::Parse(e)
    }
}
```

Now one function can use `?` on both kinds of errors:

```
fn read_number(path: &str) -> Result<i32, AppError> {
    let content = std::fs::read_to_string(path)?; // io::Error → AppError
    let num = content.trim().parse::<i32>()?;     // ParseIntError → AppError
    Ok(num)
}
```

11.10.2.4 The Problem: All That Every Time?

A custom error type + `impl Display` + `impl Error` + a `From` for each kind... quite a mouthful. Is there a simpler way?

11.10.2.5 `Box<dyn Error>`

If your return type doesn't need to expose a fixed set of error kinds that callers can match exhaustively, use `Box<dyn Error>` as a catch-all error type:

```
use std::error::Error;

fn read_number(path: &str) -> Result<i32, Box<dyn Error>> {
    let content = std::fs::read_to_string(path)?;
    let num = content.trim().parse::<i32>()?;
    Ok(num)
}
```

Concrete error types implementing `Error + 'static` can be converted automatically into `Box<dyn Error>`, so these errors work directly with `?` without a hand-written `From` implementation.

`Box<dyn Error>` erases the concrete error's static type, so callers cannot exhaustively `match` it like a custom error `enum`. The error information is not completely lost, however: callers can inspect the underlying cause through `.source()` or check a known concrete type with `.downcast_ref::<T>()`.

For example, a caller can check whether a `Box<dyn Error>` contains a `std::io::Error`:

```
fn main() {
    if let Err(error) = read_number("missing.txt") {
        if let Some(io_error) = error.downcast_ref::<std::io::Error>() {
            println!("this is an I/O error of kind {:?}", io_error.kind());
        } else {
            println!("this is some other error: {}", error);
        }
    }
}
```

If the `Box` directly contains a `std::io::Error`, `.downcast_ref::<std::io::Error>()` returns `Some(&std::io::Error)`; for a different type it returns `None`. For example, if the `Box` contains an `AppError`, then `error.downcast_ref::<std::io::Error>()` still returns `None` even when the value is `AppError::Io` wrapping a `std::io::Error`, because the type stored directly in the `Box` is `AppError`. In that case, call `.source()` first to obtain the wrapped error, then call `.downcast_ref::<T>()` on it.

`Box<dyn Error>` by itself does not guarantee that the error can cross `Thread` boundaries. If an error must be sent to another `Thread`, or an API requires thread-safe errors, a common type is `Box<dyn Error + Send + Sync>`; the contained error must implement `Send + Sync` as well.

11.10.2.6 Which One When

- **Quick prototypes, scripts, the `main` function**: `Box<dyn Error>` is the least effort.
- **Libraries, or when callers must handle errors precisely**: a custom error `enum` + `impl Error` + `impl From`.

Next episode we'll see how community `crates` dramatically simplify writing custom error types.

11.10.3 Example Code

```
use std::error::Error;
use std::fs;
```

```rust
fn first_line_number(path: &str) -> Result<i32, Box<dyn Error>> {
    let content = fs::read_to_string(path)?;
    let first_line = content.lines().next().ok_or("the file is empty")?;
    let num = first_line.trim().parse::<i32>()?;
    Ok(num)
}

fn main() {
    match first_line_number("number.txt") {
        Ok(n) => println!("number read: {}", n),
        Err(e) => println!("error: {}", e),
    }
}
```

11.10.4 Recap

- The `Error` `trait` requires `Display + Debug` and is the common interface of all error types.
- Custom errors: define an `enum` → `impl Display` → `impl Error` (preserving causes with `.source()`) → `impl From` for each underlying error.
- With `From` in place, `?` automatically converts underlying errors into your custom error.
- `Box<dyn Error>` erases the concrete error's static type, but errors remain inspectable through `.source()` or downcasting.
- To send errors across `Thread` boundaries, `Box<dyn Error + Send + Sync>` is commonly used.
- `Box<dyn Error>` suits rapid development; custom error `enums` suit libraries.

11.11 A Brief Introduction to `thiserror` / `anyhow`

11.11.1 Goal of This Episode

Meet the community's two most popular error-handling `crates`.

11.11.2 Concept

This episode covers not the standard library, but two community `crates`. They're practically standard equipment in the Rust ecosystem and extremely useful, so we introduce them here.

Install them before use:

```
cargo add thiserror
cargo add anyhow
```

11.11.2.1 Background

Last episode we saw how much boilerplate a custom error takes (`enum` + `Display` + `Error` + a `From` for each kind). `thiserror` and `anyhow` solve exactly that.

11.11.2.2 `thiserror`: For Libraries

`thiserror` auto-generates `Display`, `Error`, and `From` with a `derive` macro:

```rust
use thiserror::Error;

#[derive(Debug, Error)]
enum AppError {
    #[error("I/O operation failed")]
    Io(#[from] std::io::Error),

    #[error("failed to parse an integer")]
    Parse(#[from] std::num::ParseIntError),

    #[error("custom error: {0}")]
    Custom(String),
}
```

- `#[error("...")]` auto-generates the `Display` implementation.
- `#[from]` auto-generates the `From` implementation and treats the same field as the underlying cause returned by `.source()`.
- What took dozens of hand-written lines last episode is now a few lines.

Usage is the same as last episode — `?` converts automatically:

```rust
fn read_number(path: &str) -> Result<i32, AppError> {
    let content = std::fs::read_to_string(path)?;
    let num = content.trim().parse::<i32>()?;
    Ok(num)
}
```

Callers can still `match` and handle each error kind precisely.

11.11.2.3 anyhow: For Applications

If callers don't need to distinguish error kinds (say, in the `main` function or a CLI tool), `anyhow` is even simpler:

```rust
use anyhow::{Context, Result};

fn read_number(path: &str) -> Result<i32> {
    let content = std::fs::read_to_string(path)
        .context("failed to read the file")?;
    let num = content.trim().parse::<i32>()
        .context("failed to parse the number")?;
    Ok(num)
}
```

- `anyhow::Result<T>` is just `Result<T, anyhow::Error>`.
- `anyhow::Error` is similar to `Box<dyn Error + Send + Sync>`, but more convenient to use.
- `.context("...")` adds extra explanation to an error, handy for debugging.
- No new error type to define; errors implementing `Error + Send + Sync + 'static` convert automatically.

11.11.2.4 How the Two Relate

- **`thiserror`**: helps you define precise error types without the repetitive hand-written code. For libraries — users can `match` on your errors.

- **anyhow**: no separate error type to define; errors meeting the bounds above are handled uniformly through one type. For applications — you just need to report errors, not let others handle them programmatically.

They combine well: libraries define errors with `thiserror`, applications receive them all through anyhow.

11.11.3 Example Code

```
// this example shows anyhow in action
extern crate anyhow;

use anyhow::{Context, Result};
use std::fs;

fn read_config(path: &str) -> Result<(String, i32)> {
    let content = fs::read_to_string(path)
        .context("couldn't read the config file")?;

    let mut lines = content.lines();

    let name = lines.next()
        .context("the config file is empty")?
        .to_string();

    let value = lines.next()
        .context("missing second line")?
        .trim()
        .parse::<i32>()
        .context("the second line isn't a valid number")?;

    Ok((name, value))
}

fn main() -> Result<()> {
    let (name, value) = read_config("config.txt")?;
    println!("name: {}, value: {}", name, value);
    Ok(())
}
```

11.11.4 Recap

- `thiserror`: auto-generates `Display`, `Error`, and `From` via `derive`; suits libraries.
- `#[error("...")]` generates `Display`; `#[from]` generates `From` and marks the field as the underlying `source`.
- anyhow: handles errors implementing `Error + Send + Sync + 'static` uniformly without an error `enum`; suits applications.
- `.context("...")` adds extra explanation to errors.
- Libraries use `thiserror`, applications use anyhow, and the two combine well.

11.12 catch_unwind

11.12.1 Goal of This Episode

Learn to intercept panics with `catch_unwind`, and understand when it should be used.

11.12.2 Concept

11.12.2.1 Basic Usage

Normally, an uncaught panic eventually terminates the current `Thread`. `catch_unwind` lets you put a boundary around a closure and catch a panic that leaves it:

```
use std::panic;

fn main() {
    let result = panic::catch_unwind(|| {
        println!("running normally");
        42
    });
    println!("{:?}", result); // Ok(42)

    let result = panic::catch_unwind(|| {
        panic!("something went wrong!");
    });
    println!("{:?}", result); // Err(...)
}
```

If the closure returns normally, you get `Ok(value)`; if it panics, you get `Err`, and the program can continue afterward.

You may still see a panic message in the terminal even when the panic is caught. The important point is that the program continues and `catch_unwind` returns `Err`.

11.12.2.2 Why Catch a Panic?

One special use is inside a Rust function exposed to C. If a panic is not caught inside an `extern "C"` function, the whole program terminates before control can return to C.

To avoid the abort, use `catch_unwind` inside the Rust function and turn `Err` into an error code. The example at the end of this episode demonstrates this pattern. If aborting the process is acceptable, you do not need `catch_unwind`.

11.12.2.3 `UnwindSafe`

`catch_unwind` requires its closure to be `UnwindSafe`. The reason is simple: a panic may interrupt a modification halfway through, and the program might continue using that half-updated data after catching it.

`&mut T` does not pass this check. If a closure modifies data through a mutable reference and then panics, the data outside the closure may be left half-updated.

Most shared references, such as `&i32` and `&String`, pass the check—but not every `&T` does. For example, `&Cell<T>` and `&RefCell<T>` do not, because `Cell` and `RefCell` allow modification through a shared reference.

`UnwindSafe` is only a reminder to think about the state left behind after a panic. It does not prove that the data is logically correct.

11.12.2.4 `AssertUnwindSafe`

If you have considered the possible state and know how to handle it, `AssertUnwindSafe` lets you explicitly ask Rust to accept the closure:

```
use std::panic::{catch_unwind, AssertUnwindSafe};

fn main() {
    let mut data = vec![1, 2, 3];
    let original_len = data.len();

    let result = catch_unwind(AssertUnwindSafe(|| {
        data.push(4);
        panic!("the update stopped halfway");
    }));

    if result.is_err() {
        data.truncate(original_len);
    }

    println!("{:?}", data); // [1, 2, 3]
}
```

`AssertUnwindSafe` does not repair the data for you. It only tells Rust that you accept responsibility for checking or restoring the state afterward.

11.12.2.5 `panic = "abort"`

`Cargo.toml` can set:

```
[profile.release]
panic = "abort"
```

Under this setting, a panic immediately terminates the whole program. `catch_unwind` cannot catch it.

11.12.2.6 Not Ordinary Error Handling

`catch_unwind` is not a general-purpose `try/catch`. Expected failures should use `Result`. Use `catch_unwind` only when you deliberately need to contain a panic, such as returning an error code from an FFI function instead of terminating the program.

11.12.3 Example Code

```
use std::panic;

// Simulates code called by an FFI function that we cannot fully control.
fn library_task(mode: i32) -> i32 {
    if mode == 0 {
        panic!("library task panicked");
    }
    100 / mode
}
```

```rust
extern "C" fn ffi_entry(mode: i32) -> i32 {
    match panic::catch_unwind(|| library_task(mode)) {
        Ok(value) => value,
        Err(_) => -1, // Turn the panic into an error code
    }
}

fn main() {
    println!("success: {}", ffi_entry(4)); // 25
    println!("failure: {}", ffi_entry(0)); // -1; the program continues
}
```

Here the panic is caught inside `ffi_entry`, so it never escapes the `extern "C"` function. The function returns `-1` normally instead.

11.12.4 Recap

- `catch_unwind` runs a closure and returns `Ok(value)` or `Err`.
- A caught panic may still print a message, but the program can continue.
- If a panic is not caught inside an `extern "C"` Rust function, the whole program terminates before control can return to C. Catch it first only when you want a different result, such as an error code.
- `&mut T` does not pass the `UnwindSafe` check. Most shared references do, but `&Cell<T>` and `&RefCell<T>` are exceptions.
- `AssertUnwindSafe` asks Rust to accept your judgment; handling half-updated data is still your responsibility.
- Under `panic = "abort"`, `catch_unwind` cannot catch a panic.
- Use `Result` for expected failures, not `catch_unwind`.

Congratulations on finishing the advanced standard library chapter! 🎉 This chapter toured practical tools from the standard library and the community — from `AsRef`, sorting, and collections, to I/O, string methods, and error handling, all the way to `catch_unwind`. In the next chapter, we enter the world of async!

Chapter 12

Async

With the foundations from the previous chapters in place, we can now enter the hardest chapter of this tutorial: async. Besides all sorts of new concepts, this chapter also asks you to read a lot of code; I didn't design it specifically to train readers in that, but judging by the outcome, it may well do so. In the first half of the chapter, we'll design a complete API-like set of tools. If you can make it through that part, I believe you'll have a much better sense of how abstractions are built in Rust; in the second half, we introduce the tools you'll actually use when writing async programs in practice.

12.1 Your First `async` Program

12.1.1 Goal of This Episode

Write a tiny server with Tokio that responds to a browser, to get a first impression of what `async` programs look like.

12.1.2 Main Text

Welcome to the world of async! In this chapter we'll put in a lot of work slowly peeling open the workings of asynchrony (`async`), layer by layer. But this first episode skips the theory — we'll write a runnable program straight away so you get a feel for what `async` code looks like. Having read this far, you've learned a lot; just reading the code, you can probably guess what it does.

Let's start from the word itself. **Synchronous** means "everyone moves in lockstep": until one thing finishes, the next thing waits. **Asynchronous** means "no need to wait in lockstep": while one thing is waiting on a result, the program can push something else forward first. In a server, that means waiting for a browser to connect, or for a response to be sent, doesn't force every other connection to sit frozen.

12.1.2.1 Rust's `async` Needs a Runtime

Unlike many other languages, Rust's standard library has **no** built-in engine for executing async work (we'll call it a runtime from now on). The standard library only defines async's "specification"; how the async work actually gets run is left to third-party `crates`. That sounds odd, but this design lets Rust's `async` serve everything from big servers to small embedded devices.

The most widely used runtime today is **Tokio**. In the second half of this chapter, we'll dig into Tokio's features. To use it, first add the dependency in `Cargo.toml`:

```
[dependencies]
tokio = { version = "1", features = ["full"] }
```

Or by command:

```
cargo add tokio --features full
```

The program below writes `.await` directly in `main`. Note this syntax rule now: **`.await` can only appear in an `async` context**. A plain `fn main()` can't `.await` directly, so we'll write it as `async fn main()`.

However, `async fn main()` can't serve as the program's entry point by itself the way a plain `fn main()` does. The `#[tokio::main]` attribute is the helper Tokio provides: it sets up the runtime for us so this `async fn main()` can actually be executed.

12.1.2.2 A Server That Counts

The program below opens a little server on your machine; whenever someone connects, it replies "this is request number N." All connections share one counter, so as you refresh your browser, the number keeps climbing:

```
use std::sync::Arc;
use std::sync::atomic::{AtomicU64, Ordering};
use tokio::io::AsyncWriteExt;
use tokio::net::TcpListener;

#[tokio::main]
async fn main() {
    // the counter shared by all connections
    let counter = Arc::new(AtomicU64::new(0));

    // listen on local port 8080
    let listener = TcpListener::bind("127.0.0.1:8080").await.expect("bind failed");
    println!("server started — open http://127.0.0.1:8080 in your browser");

    loop {
        // wait for the next connection to come in
        let (mut socket, _) = listener.accept().await.expect("accept failed");

        // hand a share of the counter's ownership to the upcoming background job
        let counter = Arc::clone(&counter);

        // toss this connection to the background; the main loop goes right back to waiting
        tokio::spawn(async move {
            let n = counter.fetch_add(1, Ordering::SeqCst) + 1;
            let body = format!("this is request number {}\n", n);
            let response = format!(
                "HTTP/1.1 200 OK\r\nContent-Length: {}\r\n\r\n{}",
                body.len(),
                body,
            );
            socket.write_all(response.as_bytes()).await.expect("failed to respond");
        });
    }
}
```

Once it's running on your machine, open your browser to `http://127.0.0.1:8080` and refresh a few times — you'll see the number keep going up.

> When you open this URL in a browser, the browser may request `/favicon.ico` in addition to the page itself. This simplified example counts every incoming TCP connection, so the number may sometimes increase by 2. Only the response for the page itself is displayed; the favicon response does not appear on the page.

12.1.2.3 What `.await` Means

Several `.await`s appear in the program — this is the very heart of `async` code. For now, understand it like this:

> `.await` means "this may take a while to be ready; if we do have to wait, please try to find something else to do in the meantime."

Take `listener.accept().await`: accepting a new connection means waiting until someone actually connects, which could be a few milliseconds or several seconds. `.await` marks this "might have to wait" spot; while waiting, this `async` job can be paused, yielding its turn to run.

12.1.2.4 It Really Does Handle Many Connections at Once

Notice we used `tokio::spawn` to toss "handling a single connection" into the background. The main loop only accepts new connections; for each one it spawns a background job to write the response, then immediately goes back to waiting for the next connection.

So even if some connection is in the middle of `socket.write_all(response.as_bytes()).await`, the main loop doesn't wait for the write to finish. Other connections keep being accepted and processed. This ability to split many things apart and advance them interleaved is exactly `async`'s selling point.

This episode showed you the look and effect of an `async` program. Next episode we spell out the motivation: what scenarios it suits, why not just open lots of `Thread`s, and what exactly separates "concurrency" from "parallelism."

12.1.3 Recap

- Rust's standard library only defines `async`'s specification; actual execution relies on a third-party **runtime**, most commonly **Tokio**.
- `.await` can only be written in an `async` context; `#[tokio::main]` lets `main` be an `async fn`, sets up the runtime for you, and drives it.
- `.await` means "wait for this to be ready, and meanwhile go do other things" — not "sit and block."
- Paired with `tokio::spawn` to push work into the background, an `async` program can advance many connections at once.

12.2 Why We Need async

12.2.1 Goal of This Episode

Understand what kinds of programs async suits, and clearly separate two often-confused words: "concurrency" and "parallelism."

12.2.2 Main Text

12.2.2.1 Why async Exists

Last episode's server had one defining characteristic: it spends most of its time **waiting**. Waiting for someone to connect, for data to arrive, for a response to be written out. The time actually spent computing on the CPU is pitifully small.

Programs that "mostly wait" are in fact everywhere:

- Web servers: waiting for clients to send requests, for the database to answer.
- Crawlers: firing off a pile of network requests, then waiting for replies.
- Database queries: sending the query, waiting for results.
- Chat rooms: waiting for each user to type a message.
- All sorts of background jobs: waiting on timers, files, other programs.

The bottleneck in these programs isn't "the CPU can't compute fast enough" — it's "too much time spent waiting." async was born for this situation: it lets your program, while waiting on one thing, put its precious Threads to work advancing other jobs.

12.2.2.2 Why Not Just Open Lots of Threads

You might think: didn't earlier chapters teach multithreading? To handle ten thousand connections at once, just open ten thousand Threads?

The problem is cost. Operating system Threads (OS Threads) are **memory-hungry** — for each one, the OS has to set aside stack space, often several MB. Ten thousand Threads could eat several GB of memory, before even counting the OS's overhead of switching among that many Threads. For a program that "mostly waits," opening ten thousand Threads and letting nearly all of them sleep is extravagant indeed.

async takes a different approach: it can use **just a few** Threads to advance **thousands upon thousands** of waiting jobs in turn. Each job no longer corresponds to a heavyweight OS Thread, but to something lightweight (we'll gradually see its true face later). That's how async supports huge numbers of connections.

12.2.2.3 Concurrency vs Parallelism

Time to nail down two easily confused words, because they're key to understanding async.

Concurrency: *handling* many things at once, by way of "interleaved switching." Picture one barista covering several tables alone: he takes table A's order, and while the coffee machine is brewing, goes to take table B's order, then returns to finish up with table A. At any single instant he's really doing only one thing, but because he knows to switch to something else during the "waiting" gaps, it looks like he's serving many tables at once. One person — one Thread — is enough for concurrency.

Parallelism: at the same instant, many things truly *executing* together. This needs multiple CPU cores — like a coffee shop with several baristas, each covering their own table, genuinely working at the same time.

These are **two independent dimensions**, freely combinable:

- Single-threaded `async`: concurrency, no parallelism (one barista serving tables in turn).
- Pure computation thrown onto multiple cores: parallelism (several baristas), no need for `async`'s concurrency.
- A multithreaded runtime (Tokio's default): both (several baristas, each of whom also switches tables during the gaps).

12.2.2.4 What `async` Provides Is Concurrency

Here comes the key conclusion: **`async` itself provides concurrency** — it lets a few `Threads` advance a large pile of waiting jobs, interleaved. Whether you also get **parallelism** is decided by how many `Threads` the runtime uses to run those jobs.

So remember: `async` **does not make your CPU computations faster**. If your program is grinding through a resource-hungry math computation, `async` can't help — that's a problem only parallelism (spreading over multicore processors) can solve. `async` solves a different problem — **making sure waiting time isn't wasted**, switching to other work during the gaps.

Starting next episode, we take "what exactly is `async`" apart, layer by layer.

12.2.3 Recap

- `async` suits programs that "spend most of their time waiting on I/O": servers, crawlers, databases, chat rooms, background jobs.
- OS `Threads` are memory-hungry; "one `Thread` per connection" can't sustain huge connection counts — `async` advances masses of work on just a few `Threads`.
- **Concurrency** is interleaved switching, *handling* many things at once — one `Thread` suffices (the barista analogy); **parallelism** is multiple cores truly *executing* at the same instant.
- `async` provides concurrency; parallelism depends on how many `Threads` the runtime uses.
- `async` doesn't speed up computation; it just lets waiting time be spent on other work.

12.3 `async fn` Returns a `Future`

12.3.1 Goal of This Episode

Build a key mental model: calling an `async fn` doesn't execute it — you merely get a `Future` that hasn't started running.

12.3.2 Main Text

12.3.2.1 Calling an `async fn` Doesn't Run It

This is the pit beginners fall into most often, so let's prove it by experiment. Start with an ordinary `async fn`:

```
async fn say_hello() {
    println!("hello");
}

#[tokio::main]
async fn main() {
    say_hello(); // note: this line does NOT print hello!
}
```

Intuitively you'd expect calling `say_hello()` to print `hello`, but in fact **nothing happens**. The `println!` in the function body never runs. Not only that, the compiler gives you a warning:

```
warning: unused implementer of `Future` that must be used
note: futures do nothing unless you `.await` or poll them
```

This warning (courtesy of `#[must_use]`) already spills the truth: calling `say_hello()` gives you a **`Future`** — a "job that hasn't run yet." You've only described the job; nobody executed it, so it got thrown away.

To actually run it, add `.await`:

```
async fn say_hello() {
    println!("hello");
}

#[tokio::main]
async fn main() {
    say_hello().await; // this time hello gets printed
}
```

12.3.2.2 Making the Compiler Confirm It's a `Future`

Still not convinced? We can force the compiler to tell the truth another way: deliberately annotate the wrong return type and see how it complains.

```
async fn say_hello() {
    println!("hello");
}

#[tokio::main]
async fn main() {
    let x: () = say_hello(); // compile error
}
```

`say_hello`'s body returns nothing, so it "should" return `()`, and we deliberately write `let x: () = ...`. But the compiler errors:

```
expected `()`, found future
```

It tells you plainly: the type of `say_hello()` is **not** `()` but a future. Confirmed — calling an `async fn` gets you a `Future`, not the result of the body's execution.

12.3.2.3 `Futures` Are Lazy

The two sections above showed two things: calling `say_hello()` doesn't execute the body immediately; and `say_hello()`'s return type isn't `()` but a `Future`. Put together, they give this

episode's most important sentence:

> Calling an `async fn` just gets you a `Future`, and that `Future` is **lazy**.

"Lazy" should sound familiar. Recall Chapter 6's **iterators**: when you write `v.iter().map(...).filter(...)` those methods haven't processed a single element — they only describe "what to do later"; the real running starts the moment you `.collect()` or walk it with `for`.

`Future` and `Iterator` share the same design philosophy at heart: **describe first, execute later**. An `Iterator` describes "how a sequence of values gets computed" and does work only when you ask it for values; a `Future` describes "what an async job will do" and makes progress only when the runtime pushes it forward.

Next episode we switch angles and set `.await` beside the `?` you've long known — you'll find they're the same kind of thing.

12.3.3 Recap

- Calling an `async fn` does **not** execute the body; you only get a `Future`.
- Without a `.await` in `async fn main`, the called `async` function won't run a single line, and you'll get a `#[must_use]` warning.
- Annotating the return value as `()` makes the compiler report `expected (), found future`, proving it really is a `Future`.
- `Futures` are **lazy** — like Chapter 6's `Iterator`, the design is "describe first, execute later."

12.4 async as an Effect

12.4.1 Goal of This Episode

Look at `async` from another angle: set `.await` beside the `?` you already know, and discover they're the same kind of thing.

12.4.2 Main Text

12.4.2.1 Two Little Tails

Think back to Chapter 5's `?`. When an expression's type is `Option` / `Result`, sticking a `?` on the end pulls out the "success value" for you to use, while "what if it failed" gets handled automatically by the compiler:

```
let x = a.parse::<i32>()?; // ? pulls the value out of the Result
```

`.await` does something very similar. When an expression's type is `Future`, sticking a `.await` on the end pulls out the "value that will be computed later" for you to use, while "what if it's not ready yet" gets handled automatically by the runtime:

```
let x = some_async_thing().await; // .await pulls the value out of the Future
```

See it? `?` and `.await` are both **little tails stuck onto expressions** that pull "a value wrapped in some special world" into your hands.

12.4.2.2 Two Worlds, Each with Its Own Rules

Think of it this way: some values don't live in the "ordinary world" but in a wrapped-up special world.

- **The `Option` / `Result` world**: the value might not be computable. This world's rule is "may fail."
- **The `Future` world**: the value might not be ready yet; you have to wait. This world's rule is "may not be ready."

When you pull values out with `?` or `.await`, your code reads just like ordinary code — line after line, using values in computations. But behind the scenes the compiler is doing something for you: **chaining** these "wrapped values" together according to each world's rules. Every `?` or `.await` is a seam where the rules get applied: the `?` seam auto-returns early on error; the `.await` seam auto-pauses when things aren't ready and yields the `Thread`.

12.4.2.3 Why `.await` Needs Its Own `async` Syntax

`?`'s rule is fairly simple — the compiler only inserts an "on error, `return` early" check. But `.await`'s rule is far more involved: when "not ready," it must **pause** the whole function, remember where it got to, hand the `Thread` to someone else, and resume from that spot once ready.

To pull that off, the compiler must **heavily rewrite** your `async` function into something called a "state machine" (explained later in this chapter — just remember the term for now). It's precisely because the rewrite is so extensive that Rust needs the dedicated `async` keyword — it effectively tells the compiler: "please rewrite this part into a pausable, resumable form."

12.4.2.4 `async` Is "Contagious"

`?`'s restriction is that you can only use it inside "functions that return `Option` / `Result`." Likewise, `.await` can only be used in an `async` context. `.await`ing directly in an ordinary function fails to compile:

```
async fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn normal_function() {
    let sum = add(3, 4).await; // compile error: can't .await in a regular function
}
```

In other words, you can only pull values out "from inside the world." To use `.await`, the function you're in must itself be `async` — and so `async` "infects" its way up the call chain. This is the same story as `?` requiring "the caller must itself be able to handle errors."

12.4.2.5 Effects Must Eventually "Land"

Whichever world it is, at some point you must return to the ordinary world — unwrap the packaging and get a concrete value. First look at how error handling "lands"; it has two routes.

Route one: let `main` itself return a `Result`, handing things to the compiler at the program's boundary.

```
fn parse_and_add(a: &str, b: &str) -> Result<i32, std::num::ParseIntError> {
    let x = a.parse::<i32>()?;
    let y = b.parse::<i32>()?;
    Ok(x + y)
}

fn main() -> Result<(), std::num::ParseIntError> {
    let sum = parse_and_add("3", "4")?;
    println!("the result is {}", sum);
    Ok(())
}
```

Route two: take the `Result` apart yourself with `match`, handling it in ordinary code.

```
fn parse_and_add(a: &str, b: &str) -> Result<i32, std::num::ParseIntError> {
    let x = a.parse::<i32>()?;
    let y = b.parse::<i32>()?;
    Ok(x + y)
}

fn main() {
    match parse_and_add("3", "4") {
        Ok(sum) => println!("the result is {}", sum),
        Err(e) => println!("something went wrong: {}", e),
    }
}
```

12.4.2.6 `async` Lands the Same Way, in Perfect Correspondence

The `Future` world lands by the same two routes, and they line up one to one:

Route one: **`#[tokio::main]`**, corresponding to "a `main` that returns `Result`." You just make `main` `async`, letting the Tokio framework handle things at the program's boundary:

```
extern crate tokio;

async fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[tokio::main]
async fn main() {
    let sum = add(3, 4).await;
    println!("the result is {}", sum);
}
```

Route two: **`block_on`**, corresponding to "`match` it yourself." Inside an ordinary `main`, you ask the runtime on the spot to run a `Future` to completion, settling it into an ordinary value:

```
extern crate tokio;

async fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
```

```
    let runtime = tokio::runtime::Runtime::new().expect("failed to create the runtime");
    let sum = runtime.block_on(add(3, 4)); // run the Future into a plain value now
    println!("the result is {}", sum);
}
```

Pair them up: a `Result`-returning `main` ↔ `#[tokio::main]` (let the framework settle things at the boundary), `match` ↔ `block_on` (settle it yourself on the spot in synchronous code). Keep this correspondence in mind, and `async` stops being something brand new — it's "the `?` you already know, with a more complicated set of rules."

12.4.3 Recap

- `?` and `.await` are both little tails on expressions that pull out "values from a special world."
- The `Result` world's rule is "may fail"; the `Future` world's rule is "may not be ready yet"; the compiler chains the wrapped values together by each world's rules.
- `.await`'s rule is complex — it rewrites the function into a **state machine** — hence the dedicated `async` syntax.
- Like `?`, `async`'s `.await` is "contagious": to `.await`, the enclosing function must itself be `async`.
- Two landing routes in perfect correspondence: `Result`-returning `main` ↔ `#[tokio::main]`, `match` ↔ `block_on`.

12.5 async Blocks

12.5.1 Goal of This Episode

Learn to make a `Future` on the spot inside a function with `async { ... }`, and understand its relationship to `async fn`.

12.5.2 Main Text

12.5.2.1 Making a `Future` on the Spot

Besides `async fn`, Rust also lets you create a `Future` **on the spot**, mid-program, with `async { ... }`:

```
#[tokio::main]
async fn main() {
    // this async block is itself a Future
    let fut = async {
        println!("I'm inside an async block");
        42
    };

    // just like an async fn, it only runs when .awaited
    let value = fut.await;
    println!("got {}", value);
}
```

Note: exactly like an `async fn`, merely writing `async { ... }` doesn't execute its contents — you've only made a lazy `Future`, and it makes progress only when `.await`ed.

12.5.2.2 How `async fn` and `async` blocks Relate

A rough first cut at telling them apart:

- An `async fn` is a **named `Future` factory** — define it once, call it repeatedly, each call producing a fresh `Future`.
- An `async` block is **an anonymous `Future` created on the spot** — right here, just this one, no name.

That is, looking only at "named and reusable" versus "anonymous and on the spot," they're a bit like the difference between ordinary functions and closures. But that's just a first-impression analogy — don't read too much into it: `async fn`s and `async` blocks both produce `Future`s, but an `async` block is not a closure; you don't call it with `()`. Once created, it's driven by `.await` or the runtime.

12.5.2.3 `async move`

An `async` block can also be written as `async move { ... }`. The `move` here is similar to `move` on a closure: it moves the outside variables used by the block into the `Future`. What gets moved is the variable itself; if that variable is already a reference, then the reference is what gets moved.

```
#[tokio::main]
async fn main() {
    let name = String::from("Ferris");

    let fut = async move {
        println!("hello, {}", name);
    };

    fut.await;

    // println!("{}", name); // compile error: name has been moved into the async block
}
```

Without `move`, an `async` block usually tries to borrow outside variables. With `async move`, the outside variables it uses are moved into the resulting `Future`. This is common when you want to store a `Future`, hand it to a runtime, or let it leave the current scope.

12.5.2.4 In the `Result` World, This Needs No New Syntax

Here's an interesting contrast. In the `Result` world, if you want "a block right here where I can use `?`," you need **no new syntax at all** — an immediately invoked closure does it:

```
fn main() {
    // define a closure, then call it immediately with ()
    let result: Result<i32, std::num::ParseIntError> = (|| {
        let x = "3".parse::<i32>()?;
        let y = "4".parse::<i32>()?;
        Ok(x + y)
    })();

    println!("{:?}", result);
}
```

The `(|| { ... })()` here means "define a closure and call it immediately." The closure's body

can use `?` because the closure itself returns a `Result`; after the call, the outer `main` simply receives that `Result` value.

12.5.2.5 Why the `Future` World Can't Copy That Trick

You might wonder: can the `Future` world just do the same? Stuff the `.await` into an immediately invoked closure?

```
async fn get_number() -> i32 {
    42
}

#[tokio::main]
async fn main() {
    let value = (|| {
        get_number().await // compile error: can't .await in an ordinary closure
    })();
}
```

No. The reason goes back to the previous episodes: `.await` requires the whole stretch of code to be **rewritten into a state machine** so it can "pause to allow concurrency." But an ordinary closure compiles into an ordinary function, which has **no** notion of "pause now, resume later" — it can't express that rewrite. So the `Result` world's trick doesn't carry over.

This is exactly why `async` blocks exist. Writing `async { ... }` explicitly tells the compiler: "rewrite this block into a `Future`." With that dedicated syntax in place, `.await` becomes legal inside:

```
extern crate tokio;

async fn get_number() -> i32 {
    42
}

#[tokio::main]
async fn main() {
    let value = async {
        get_number().await // works this time, because this is an async block
    }.await;
    println!("{}", value);
}
```

With that, the first five episodes have laid down the basic syntax and mental models — `async fn`, `.await`, `async` blocks. Starting next episode, we roll up our sleeves and take the internals of `Future` apart with our own hands.

12.5.3 Recap

- `async { ... }` creates an anonymous `Future` on the spot, mid-function; it likewise runs only when `.await`ed.
- An `async fn` is a named, reusable `Future` factory; an `async` block is one anonymous `Future` made in place.
- `async move { ... }` moves the outside variables it uses into the resulting `Future`; this is common when the `Future` needs to leave the current scope or be handed to a runtime.

- An immediately invoked closure that returns `Result` can use `?` inside; the outer code just receives the closure call's `Result` value.
- `.await` can't copy that trick — it may only appear inside `async` constructs, and an ordinary closure can't pause and resume — hence the dedicated `async` block syntax.

12.6 The `Future` `trait` and the Most Bare-bones Executor

12.6.1 Goal of This Episode

Read and understand the formal definition of the `Future` `trait`, and hand-write the dumbest executor that actually runs.

12.6.2 Main Text

12.6.2.1 What the `Future` `trait` Looks Like

We've been saying "`Future`" for several episodes; time to see its real definition. It's a `trait` in the standard library:

```
pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
```

Piece by piece:

- `type Output` is the type of the value this `Future` will yield once complete.
- `poll` is the core method. It asks the `Future`: "are you done yet?"
- The return value is `Poll`, an `enum` with just two states:

```
pub enum Poll<T> {
    Ready(T), // done — here's the result
    Pending,  // not yet — ask again later
}
```

So the way to advance a `Future` is to `poll` it repeatedly: `Pending` means not done yet, `Ready(value)` means finished and the result can be taken.

12.6.2.2 Why `poll`'s `self` Is `Pin<&mut Self>`

You've probably noticed something odd: `poll`'s first parameter isn't the familiar `self` / `&self` / `&mut self`, but `self: Pin<&mut Self>`.

Don't panic — later in this chapter we'll spend several episodes on the details of `Pin`. For now, accept one thing: **`Pin` is a very special type**. Rust decrees that besides the `self` / `&self` / `&mut self` we already know, only a small handful of "smart pointers" may sit in the `self` position:

- `Box<Self>`, `Rc<Self>`, `Arc<Self>`.
- And `Pin<...>`.

Your own custom types generally **can't** be used in the `self` position like that. `poll` can be written `self: Pin<&mut Self>` precisely because `Pin` is special enough. For the moment, think of `Pin<&mut Self>` as "a restricted `&mut Self`" — it lets you modify the `Future`'s contents

but forbids moving the whole thing away. Why that restriction exists comes later.

12.6.2.3 The Most Bare-bones Executor

`poll` is the `Future`'s engine, but someone has to crank it — the role that "keeps `poll`ing until completion" is called the **executor**. Rust's standard library ships **no** executor, so let's write the dumbest possible one ourselves:

```
use std::future::Future;
use std::task::{Context, Poll, Waker};

fn block_on<F: Future>(future: F) -> F::Output {
    // put the Future on the heap and "pin" it, getting a Pin<Box<F>>
    let mut future = Box::pin(future);

    // make a Context wrapping a do-nothing Waker — what it's for comes later
    let mut cx = Context::from_waker(Waker::noop());

    loop {
        // .as_mut() borrows the Pin<Box<F>> as Pin<&mut F>, exactly the type poll wants
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value, // done — return the result
            Poll::Pending => {
                // not ready; this dumbest executor just polls again (busy-spins)
            }
        }
    }
}

fn main() {
    let value = block_on(async {
        println!("the async block is running");
        1 + 2
    });
    println!("the result is {}", value);
}
```

12.6.2.4 Two Little Value-shuffling Tools

This executor used two `Pin`-related tools; a quick introduction:

`Box::pin(x)` has type `fn pin(x: T) -> Pin<Box<T>>` — it puts the value on the heap and pins it with `Pin`. For now, just treat it as "a restricted pointer."

`as_mut` on `Pin<Ptr>` has type `fn as_mut(&mut self) -> Pin<&mut <Ptr as Deref>::Target>`, which for `Pin<Box<T>>` means `-> Pin<&mut T>` — exactly the `self: Pin<&mut Self>` that `poll` needs. The key point is that `as_mut` is only a mutable borrow; it doesn't give `future` away, which is why our `loop` can `poll` the same `future` over and over.

12.6.2.5 Honestly: Nothing Has Actually Been Waiting So Far

Time for an honest confession. From Episode 3 through this one, the `async fn`s and `async` blocks we wrote haven't **really waited for anything** — none of them contain a `.await` that could stall. For such `Future`s, the very first `poll` returns `Ready`, and our `Pending` branch never runs at all.

In other words, the examples so far were purely demonstrations of the `Future` and executor machinery — not yet programs that "really use `async`." Next episode we hand-write a `Delay` — a `Future` that genuinely returns `Pending` and needs a stretch of time to finish. That will be our first more respectable piece of async work.

12.6.2.6 Executors Come in Many Designs

One last idea to keep: Rust's standard library only defines the `Future trait`; **how to implement an executor is left entirely up to the runtime**. What we wrote this episode is the dumb version that "busy-spins re-polling on `Pending`" — a colossal waste of CPU. Real runtimes are much smarter: they sleep when there's nothing to do and get woken when there is.

Precisely because the standard library doesn't dictate how executors are written, we have Tokio, smol, and other runtimes each with their own character. Over the coming episodes, we'll evolve this dumbest version step by step toward something resembling a real runtime.

12.6.3 Recap

- The heart of the `Future trait` is `poll`, returning `Poll::Ready(value)` (done) or `Poll::Pending` (not yet).
- `poll`'s `self` is `Pin<&mut Self>`; `Pin` is one of the few special types allowed directly in the `self` position — for now, "a restricted `&mut Self`."
- The **executor** keeps `poll`ing a `Future` until `Ready`; the standard library ships none, so you build one or use a runtime's.
- `Box::pin` heap-allocates and pins the value; `as_mut` lends out `Pin<&mut T>`; together they let the `loop` repeatedly `poll` the same `Future`.
- The earlier episodes' `async` never waited on anything — one `poll` and it's `Ready`; next episode's `Delay` will genuinely go `Pending`.
- The standard library defines only `Future`; executors are the runtime's business — which is why Tokio, smol, and friends exist.

12.7 Writing a `Delay` `Future` by Hand

12.7.1 Goal of This Episode

Hand-write your first `Future` that genuinely returns `Pending` — a timer called `Delay` — and run it with last episode's executor.

12.7.2 Main Text

12.7.2.1 Why Build a `Delay`

Last episode we promised a `Future` that "really needs to wait." But the real world's waitable events — network packets, disks, databases — all drag in a pile of operating system concepts, far too complex for a first encounter with `Pending`.

So we'll prop things up with the simplest possible thing: a **timer**. The rules are plain:

- Not yet expired → return `Pending` (not ready).
- Expired → return `Ready` (done).

This Delay will star in the next several episodes: whenever we need "an event that takes time to become ready," we'll use it as the stand-in for studying .await, join, and the Waker.

12.7.2.2 Writing Delay

Delay remembers an "expiry moment" when, and each time it's polled, it checks whether the current time has passed it:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant, // the moment it's scheduled to complete
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration, // expires duration from now
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        if Instant::now() >= self.when {
            println!("Delay finished");
            Poll::Ready(()) // expired
        } else {
            Poll::Pending // not yet — ask again later
        }
    }
}
```

poll's logic is that direct: time's up, return Ready(()); otherwise Pending. Output is () because this timer has no value to give when done — it's purely the event "the time has arrived."

12.7.2.3 Running It on Our Executor

Carry over last episode's dumbest block_on, and our own Delay runs:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
```

```
        Delay {
            when: Instant::now() + duration
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        if Instant::now() >= self.when {
            println!("Delay finished");
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

fn main() {
    println!("start");
    block_on(Delay::new(Duration::from_secs(1)));
    println!("one second has passed");
}
```

Run it and you'll see a one-second pause after "start" before "one second has passed" prints. Our first `Future` that genuinely returns `Pending` works! During that second, `block_on`'s loop frantically `poll`s and keeps getting `Pending`, until the time finally arrives and it gets `Ready`.

> **Note**: the web version's code sandbox doesn't always show time delays clearly. There are more timing and waiting examples later in this chapter; if you want to actually feel effects like "pause one second" or "wait on different jobs at once," copy the programs onto your own machine and run them there.

12.7.2.4 Honestly: This `Delay` Is Oversimplified

This version runs, but it's actually **cutting corners** — `poll`'s `cx` parameter is written `_cx` and never used.

Inside `cx` lives something called a `Waker`. Before returning `Pending`, a proper `Future` should use it to tell the executor "call me when I'm ready." Our `Delay` does no such thing. Then why does it still work? Because the executor we paired it with is equally dumb — it never sleeps; on `Pending` it immediately `poll`s again, so nobody needs to notify it anyway.

In other words, this Delay only functions because it's **bound** to this dumb executor. Drop it onto a real executor — one that sleeps and continues only when woken by a Waker — and it would return Pending without ever notifying anyone. The executor would sleep forever; this Delay would effectively never complete.

We'll fix this corner-cutting later. But before that, we'll use this Delay to build up .await and some concurrency concepts. Next episode: what happens when you .await this Delay inside async.

12.7.3 Recap

- Real I/O is too complex, so a bare-bones timer serves as the first encounter with Pending.
- Delay uses a timer to simulate "an event that takes time": Pending before expiry, Ready after — it stands in for the real thing over the next episodes.
- A custom Future — impl Future plus a poll implementation — runs fine with last episode's block_on.
- This Delay is oversimplified: poll ignores the Waker in cx, and it only happens to work because the paired executor never sleeps; on a sleeping executor it would break. We'll fix it later.

12.8 Waiting for Delay with .await

12.8.1 Goal of This Episode

Wait on last episode's Delay with .await, and watch with your own eyes — via println! — how a Future "pauses and resumes."

12.8.2 Main Text

12.8.2.1 Printing Around the .awaits

We have Delay, and we have block_on. Now put Delay inside an async block, wait on it with .await, and add println! before and after every .await to observe the order of execution:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration
        }
    }
}

impl Future for Delay {
```

```
    type Output = ();

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        if Instant::now() >= self.when {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

fn main() {
    block_on(async {
        println!("start");

        println!("waiting for the first delay...");
        Delay::new(Duration::from_secs(1)).await;
        println!("first delay done, moving on");

        println!("waiting for the second delay...");
        Delay::new(Duration::from_secs(1)).await;
        println!("second delay done, moving on");
    });
}
```

Run it, and the output appears step by step, like this:

```
start
waiting for the first delay...
(one-second pause)
first delay done, moving on
waiting for the second delay...
(one-second pause)
second delay done, moving on
```

12.8.2.2 How It “Pauses and Resumes”

This output order reveals how a `Future` operates. Remember, the whole `async` block is itself a `Future`, and `block_on` keeps `poll`ing it:

1. First `poll`: it runs from the top, prints “start” and “waiting for the first delay...”, then hits the first `.await`. The `Delay` hasn’t expired, so it returns `Pending` — and the whole `async` block returns `Pending` along with it, **pausing right here**.
2. The executor `poll`s again and again, but the `Delay` still isn’t due; each time it gets stuck

at that first `.await` returning `Pending`, unable to move on.
3. A second later, the `Delay` duly returns `Ready(())`. This `poll` gets past the first `.await`, prints "first delay done" and "waiting for the second delay...", hits the second `.await`, and returns `Pending` again — **paused at a new spot**.
4. One more second, the second `Delay` duly returns `Ready(())`; it clears the second `.await`, prints the final line, the whole `async` block returns `Ready`, and `block_on` finishes.

The crux: **each time it's `polled`, the `Future` picks up from where it last paused**, running until the next not-yet-ready `.await` where it may stop. This ability to "remember progress, pause, and resume from the same spot" is delivered by the "state machine" mentioned earlier — but this episode, just watch the phenomenon.

12.8.2.3 `.await` Doesn't Give You Concurrency for Free

Note something important: the two `Delay`s above were waited on **one after the other**, taking two seconds in total. The second `Delay` started its countdown only after the first finished.

This trips up beginners a lot. `.await` means "wait for this to be ready" — it does **not** automatically make your program concurrent. Two `.await`s in a row wait dutifully in sequence; there's no cleverness that "waits on both together."

So what if I do want both `Delay`s timing simultaneously, one second total? That's next episode's topic — we'll build, by hand, a tool that advances multiple `Future`s concurrently.

12.8.3 Recap

- Waiting on `Delay` with `.await` inside `async`, plus `println!`, lets you watch execution step forward.
- Each `poll` resumes the `Future` from where it last paused, until the next unfinished `.await` returns `Pending`.
- A `Future` remembers its progress and resumes in place — the state machine behind it deserves the credit.
- `.await` does **not** give you concurrency for free: two consecutive `.await`s wait in sequence; concurrency needs other tools (one comes next episode).

12.9 Writing join by Hand

12.9.1 Goal of This Episode

Write a `Future` of your own that wraps several `Future`s into one, advancing them **concurrently**.

12.9.2 Main Text

12.9.2.1 The Goal: Waiting on Several `Future`s Together

Last episode ended with a question: two consecutive `.await`s wait in sequence. If I want several jobs going **at the same time**, waiting until they all finish, what do I do?

The answer is to write a `Future` ourselves — call it `JoinAll`. It takes in a whole `Vec` of `Future`s, and each time it's `poll`ed, it runs a `for` loop `poll`ing **each** unfinished `Future` inside once, nudging it forward. Only when all of them are done does it return `Ready` itself.

12.9.2.2 Writing `JoinAll`

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        if Instant::now() >= self.when {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

type BoxFuture = Pin<Box<dyn Future<Output = ()>>>;

// wrap a Vec of Futures, each held in a Some (swapped to None once done)
struct JoinAll {
    futures: Vec<Option<BoxFuture>>,
}

fn boxed<F>(future: F) -> BoxFuture
where
    F: Future<Output = ()> + 'static,
{
    Box::pin(future)
}
```

```
fn join_all(futures: Vec<BoxFuture>) -> JoinAll {
    JoinAll {
        futures: futures.into_iter().map(Some).collect(),
    }
}

impl Future for JoinAll {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut(); // JoinAll is Unpin, so we can get a plain &mut back
        let mut all_done = true;

        for slot in &mut this.futures {
            // temporarily take the Future out (slot becomes None) and poll it once
            if let Some(mut fut) = slot.take() {
                match fut.as_mut().poll(cx) {
                    Poll::Ready(_) => {
                        // done — don't put it back; the slot stays None
                    }
                    Poll::Pending => {
                        *slot = Some(fut); // not ready — put it back to poll next round
                        all_done = false;
                    }
                }
            }
        }

        if all_done {
            Poll::Ready(()) // everything finished
        } else {
            Poll::Pending // some remain unfinished
        }
    }
}

// a job with "two .awaits", so it takes multiple polls to complete
async fn worker(id: u32) {
    println!("worker {} starting", id);
    Delay::new(Duration::from_secs(1)).await;
    println!("worker {} past the first second", id);
    Delay::new(Duration::from_secs(1)).await;
    println!("worker {} done", id);
}

fn main() {
    block_on(async {
        let workers = vec![
            boxed(worker(1)),
            boxed(worker(2)),
            boxed(worker(3)),
        ];
        join_all(workers).await;
        println!("all workers are done");
```

```
    });
}
```

Here `type BoxFuture = Pin<Box<dyn Future<Output = ()>>>` gives the type a short name; `BoxFuture` is only a type alias and adds no extra wrapper. `dyn Future<Output = ()>` means: "I don't care which concrete kind of `Future` this is, as long as it returns `()` when done." `boxed(...)` calls `Box::pin`, producing a `Pin<Box<F>>`; its declared return type then erases the concrete `F` behind `dyn Future<Output = ()>`. The results therefore all have the same `BoxFuture` type, so the `Vec` inside `JoinAll` can hold them all.

You may notice this line:

```
let this = self.get_mut(); // JoinAll is Unpin, so we can get a plain &mut back
```

The `self` that `poll` receives has type `Pin<&mut JoinAll>`, not a plain `&mut JoinAll`. But in some situations, Rust lets us strip that outer `Pin` and recover the original mutable reference inside. That's what `get_mut()` does: it turns `Pin<&mut JoinAll>` back into `&mut JoinAll`. The formal justification comes later; for now, know only this: with a plain `&mut JoinAll` in hand, we can modify the `Vec` inside in the familiar ways.

Also worth a look:

```
if let Some(mut fut) = slot.take() { ... }
```

`slot` has type `&mut Option<BoxFuture>`. `Option::take` **takes the value out** of the `Option` (gaining ownership) and leaves `None` in its place. So if `slot` was `Some(fut)`, after calling `take()` we hold that `Some(fut)` while `slot` temporarily becomes `None`.

That's exactly what we want: take the child `Future` out and `poll` it once. If it finished, don't put it back — the `slot` stays `None`; if it hasn't, put it back with `*slot = Some(fut)` and keep `poll`ing next round.

12.9.2.3 Why This Is Concurrent

Run it and you'll find the three `worker`s start nearly together and finish nearly together, taking **two seconds** in total rather than six.

The reason: one round of `JoinAll`'s `poll` nudges all three `worker`s once each. The three `Delay`s are timing simultaneously, so two seconds later all three `worker`s come due. That's concurrency — over the same stretch of time, three "all waiting" jobs get pushed forward together. Compare last episode: writing `worker(1).await; worker(2).await; worker(3).await;` runs one to completion before the next, six seconds in total.

12.9.2.4 Even `Future`s Needing Many `poll`s Get Pushed Along Fine

Note that we deliberately chose `worker` — a job with **two `.await`s** — to put inside. This kind of `Future` isn't done in one `poll`; it takes many, many `poll`s (each `Delay` waits a second, during which the executor polls furiously) to walk through.

And `JoinAll` needn't worry about any of that — its only job is "`poll` each unfinished `Future` once per round." Which `.await` a given `Future` is stuck at internally, and how many more `poll`s it needs, is remembered by that `Future` itself (remember? `Future`s remember their own progress). `JoinAll` just keeps `poll`ing round after round, and each `Future` naturally steps

forward until they all return Ready. This is exactly the power of the poll design: whoever merely composes Futures need not understand the internals of what's being composed.

Still, our executor remains that furiously busy-spinning dumb version. Next episode we fix that — letting the executor sleep when idle and get woken when it's time.

12.9.3 Recap

- The way to advance multiple Futures concurrently is to write a Future yourself (JoinAll) whose poll uses a for loop to poll each child Future once.
- Finished children get swapped to None; only when all are None (done) does JoinAll return Ready.
- JoinAll needn't handle "this Future takes many polls" — each child remembers its own progress; just keep polling round after round.

12.10 Waking the Executor with Threads and Wakers

12.10.1 Goal of This Episode

Teach the executor to sleep: park when there's nothing to do, and get woken by a Waker when an event completes. Along the way, nail down poll's two important contracts.

12.10.2 Main Text

12.10.2.1 No More Busy-spinning

So far our executor has a nasty habit: on Pending, it immediately polls again, burning an entire Thread on a job that's still just waiting. A real runtime doesn't do this — it goes to **sleep** when idle and gets woken when there's actual progress.

The waking tool is the Waker we've been neglecting for several episodes. cx.waker() yields a Waker; before returning Pending, a Future should hand that Waker to "whoever is responsible for announcing it's ready." When the event completes, that party calls waker.wake(), rousing the sleeping executor.

This episode we make **another Thread** responsible for timing: on Delay's first poll, spawn a Thread that sleeps, and once well rested, wakes the executor.

12.10.2.2 Making a Waker of Our Own

First, how a Waker is born. The standard library provides a Wake trait: implement its wake method to describe "what should happen on wakeup," then convert with Waker::from into a Waker.

We want "waking" to mean rousing the executor's Thread, so make a small type that remembers it:

```
use std::sync::Arc;
use std::task::Wake;
use std::thread::{self, Thread};

struct ThreadWaker {
```

```
    thread: Thread, // the executor's Thread
}

impl Wake for ThreadWaker {
    fn wake(self: Arc<Self>) {
        self.thread.unpark(); // waking = unparking that Thread
    }
}
```

Note that wake's `self` is `Arc<Self>` (another of those special types allowed in the `self` position, as mentioned in Episode 6). `Waker::from(Arc::new(...))` turns it into a `Waker`.

12.10.2.3 An Executor That Sleeps

With `ThreadWaker`, the executor can switch to "`park` and sleep on `Pending`":

```
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);

    // make a Waker that unparks this executor Thread
    let waker = Waker::from(Arc::new(ThreadWaker {
        thread: thread::current(),
    }));
    let mut cx = Context::from_waker(&waker);

    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => thread::park(), // nothing to do — sleep until unparked
        }
    }
}
```

12.10.2.4 A `Delay` That Wakes Others Itself

Finally, rewrite `Delay`: before returning `Pending`, `spawn` a `Thread` to sleep, waking on schedule:

```
use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};
use std::time::{Duration, Instant};

struct ThreadWaker {
    thread: Thread,
}

impl Wake for ThreadWaker {
    fn wake(self: Arc<Self>) {
        self.thread.unpark();
    }
```

```
}

struct Delay {
    when: Instant,
    started: bool, // has the timing Thread been started
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration,
            started: false,
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        if Instant::now() >= this.when {
            Poll::Ready(())
        } else {
            if !this.started {
                this.started = true;
                let waker = cx.waker().clone(); // clone a Waker for the timing Thread
                let when = this.when;
                thread::spawn(move || {
                    let now = Instant::now();
                    if now < when {
                        thread::sleep(when - now);
                    }
                    waker.wake(); // time's up — wake the executor
                });
            }
            Poll::Pending
        }
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let waker = Waker::from(Arc::new(ThreadWaker {
        thread: thread::current(),
    }));
    let mut cx = Context::from_waker(&waker);
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => thread::park(),
        }
    }
}
```

```
fn main() {
    block_on(async {
        println!("start");
        Delay::new(Duration::from_secs(1)).await;
        println!("one second later");
        Delay::new(Duration::from_secs(1)).await;
        println!("two seconds later");
    });
}
```

This time the executor no longer burns CPU spinning — one `poll` yields `Pending`, it `park`s and sleeps a full second, and only after the timing `Thread`'s `wake` rouses it does it `poll` again.

12.10.2.5 If `wake` Happens Before `park`, Do We Sleep Forever?

There's a timing concern worth raising. Between the executor's `poll` returning `Pending` and it actually executing `thread::park()`, there's a small gap. What if the timing `Thread` happens to `wake` → `unpark` inside that gap? Wouldn't the executor be "woken first, then go to sleep," the `unpark` land on nothing, and the executor never wake?

No. `unpark` is designed so that if the `Thread` isn't parked yet, it **leaves a permit**. The next time that `Thread` calls `park()`, it sees the permit and **returns immediately** — no sleeping at all. So whether `wake` (i.e. `unpark`) lands before or after `park()`, nothing is missed. It's precisely because `park` / `unpark` carry this guarantee that we dare use them directly as our "sleep / wake" tools.

12.10.2.6 `poll`'s Two Contracts

With this `poll` / `wake` logic freshly assembled, let's spell out the standard library's two important contracts on `Future::poll`:

Contract one: only the `Waker` from the most recent `poll` counts. On each `poll`, the `Waker` from `cx.waker()` **may differ** (e.g. the `Task` got moved to another `Thread`). So a correct `Future` should re-store the latest `Waker` on every `poll` and wake with the newest one.

Our `Delay` cheats — thanks to the `started` flag, it grabs the `Waker` once on the first `poll` and never again. That causes no harm here because every `Waker` our executor hands out **wakes the same target**, so the one it grabbed first stays valid. If the target changed, this `Delay` would fail to wake it — and a `Future` shouldn't bet on that. When we hand-write the I/O `Futures` in Episode 14, we'll honestly re-store the `Waker` on every `poll`; `Delay` itself keeps cheating the way it does now.

Contract two: after `Ready`, never `poll` again. Once a `Future` returns `Ready`, it must **not** be polled again; otherwise behavior is unguaranteed (it might panic, might wedge). So the executor must remember: once a `Future` finishes, remove it and don't touch it. Our current `block_on` returns the moment it gets `Ready`, so it can't offend; but when we're managing many `Futures` at once, this needs real care (we'll do that next episode).

12.10.2.7 One `Thread` per `Future`? Not Acceptable

Finally, some cold water: right now, "every waiting `Delay` spawns a `Thread`." That's clearly no good — remember Episode 2? `Threads` are memory-hungry. Ten thousand waiting connec-

tions would mean ten thousand Threads — **exactly the problem async set out to avoid**, and we've circled right back into it.

The next several episodes solve this for good. First we'll wrap each woken Future into something called a **Task** that can queue itself back onto the executor's "ready queue" (to-do queue); after that we can introduce the **reactor**, using one or a few Threads to watch large amounts of I/O — escaping "one job, one Thread" once and for all.

12.10.3 Recap

- Before returning Pending, a Future should hand cx.waker() to "whoever will notify it"; on completion, waker.wake() rouses the executor.
- DIY Waker: implement the Wake trait's wake method, then convert via Waker::from(Arc::new(...))
- The executor sleeps with thread::park() and the Waker wakes it with unpark(); unpark leaves a permit, so wake before or after park is never missed.
- **Contract one**: the Waker may differ per poll; a correct Future re-stores the latest one each time (Delay storing it once is an oversimplification).
- **Contract two**: no polling after Ready; the executor must remove finished Futures.
- "One Thread per Future" is too costly; starting next episode we switch to Task + ready queue, and later a reactor, to fix it.

12.11 spawn and the Ready Queue

12.11.1 Goal of This Episode

Introduce the concept of a Task, letting the executor keep many Futures at once, managed through a ready queue (to-do queue).

12.11.2 Main Text

12.11.2.1 Why Tasks Are Needed

The executors of the past few episodes always held exactly **one** Future, polling it repeatedly in a loop. But a real runtime keeps **many** Futures at once.

Here's the problem: when some Future's Waker shouts "I'm ready!", if the executor holds a pile of bare Futures, how does it know **which one** is ready — which to poll? A Future by itself carries no such information.

Our solution is to give each Future a set of "carry-on data," wrapping it into a **Task**. A Task holds:

- Its own Future
- **Which** ready queue it should requeue onto
- **Which** executor Thread to wake
- A flag to avoid queuing itself twice
- A flag marking that it has already finished

From now on the executor manages Tasks, not Futures directly. And spawn simply means "wrap a Future into a Task and hand it to the executor."

12.11.2.2 The Ready Queue and "Waking"

The executor will keep a **ready queue**: the `Task`s that "should be `poll`ed now." The executor's job is to take `Task`s off the queue and `poll` their `Future`s; when the queue is empty, it sleeps.

When a `Task` gets `wake`d, it puts **itself** back onto the ready queue, then `unpark`s the sleeping executor. Note that this `unpark` is only an **alarm bell** — it says "there's work, get up!" without pointing at which `Task` is ready. The real information — "which `Task`s should be `poll`ed" — lives in the ready queue.

12.11.2.3 Writing It Out

This episode's program is longer, but the skeleton is just the sentences above. See how a `Task` requeues itself (that's its `Wake` implementation), and how the `Executor` explicitly provides `spawn` and `block_on`:

```
use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant,
    started: bool,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration,
            started: false,
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        if Instant::now() >= this.when {
            Poll::Ready(())
        } else {
            if !this.started {
                this.started = true;
                let waker = cx.waker().clone();
                let when = this.when;
                thread::spawn(move || {
                    let now = Instant::now();
                    if now < when {
                        thread::sleep(when - now);
```

```
                    }
                    waker.wake();
                });
            }
            Poll::Pending
        }
    }
}

type Queue = Arc<Mutex<VecDeque<Arc<Task>>>>;

// a Future + the carry-on data needed to reschedule it
struct Task {
    future: Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>,
    queue: Queue,
    executor_thread: Thread,
    queued: AtomicBool, // am I currently in the queue?
    done: AtomicBool,   // am I finished?
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        // grab the old queued value while leaving true in its place
        if !self.queued.swap(true, Ordering::SeqCst) {
            self.queue.lock().expect("lock failed").push_back(self.clone());
            self.executor_thread.unpark(); // wake the executor
        }
    }
}

struct Executor {
    queue: Queue,
    executor_thread: Thread,
    remaining: usize, // number of unfinished Tasks
}

impl Executor {
    fn new() -> Executor {
        Executor {
            queue: Arc::new(Mutex::new(VecDeque::new())),
            executor_thread: thread::current(),
            remaining: 0,
        }
    }

    // spawn: wrap a Future into a Task and put it on the executor's queue
    fn spawn(&mut self, future: impl Future<Output = ()> + Send + 'static) {
        let task = Arc::new(Task {
            future: Mutex::new(Box::pin(future)),
            queue: self.queue.clone(),
            executor_thread: self.executor_thread.clone(),
            queued: AtomicBool::new(false),
            done: AtomicBool::new(false),
        });
```

```
        self.remaining += 1;
        task.wake(); // a new task needs its first trip into the ready queue
    }

    fn block_on(&mut self, future: impl Future<Output = ()> + Send + 'static) {
        // spawn the incoming Future as a Task too
        self.spawn(future);

        while self.remaining > 0 {
            // first, drain the ready queue
            loop {
                let task = self.queue.lock().expect("lock failed").pop_front();
                let Some(task) = task else { break };

                if task.done.load(Ordering::SeqCst) {
                    continue; // a stale wakeup queued after completion — skip it
                }

                task.queued.store(false, Ordering::SeqCst); // release the flag before polling
                let waker = Waker::from(task.clone());
                let mut cx = Context::from_waker(&waker);
                let mut future = task.future.lock().expect("lock failed");

                if future.as_mut().poll(&mut cx).is_ready() {
                    task.done.store(true, Ordering::SeqCst); // all later wakeups are ignored
                    self.remaining -= 1; // finished
                }
            }

            // the queue is empty. is every Task done?
            if self.remaining > 0 {
                // some remain — sleep until someone wakes us
                thread::park();
            }
        }
    }
}

fn main() {
    let mut executor = Executor::new();

    executor.spawn(async {
        println!("task A: starting");
        Delay::new(Duration::from_secs(1)).await;
        println!("task A: one second is up");
    });

    executor.block_on(async {
        println!("task B: starting");
        Delay::new(Duration::from_secs(2)).await;
        println!("task B: two seconds are up");
    });

    println!("executor finished");
}
```

Run it and the two Tasks (A and B) advance concurrently: A comes due at second one, B at second two; when each expires, it requeues only **itself** to get polled once, without disturbing the other. block_on waits until every Task in the executor completes before returning, so "executor finished" prints last.

12.11.2.4 Why the queued Flag Uses swap

The queued.swap(true, ...) in wake closely resembles Episode 9's Option::take: it's not simply "reading a value" — it **grabs the old value while leaving a new one in its place**.

Episode 9's slot.take() was "take the Some(fut) out, leave None in its place." Here, queued.swap(true, ...) is "take the old queued out, leave true in its place." So:

- Getting false means this Task was **not** in the queue — so we push it in.
- Getting true means it's already queued, and this wake needn't queue it again.

Why not load then store? Because wake can come from different Threads. To be safe, swap binds "look at the old value" and "leave the new value" into a single atomic operation, so two Threads can't both see false at once and push the same Task into the queue twice.

12.11.2.5 The done Flag: Honoring Contract Two

Last episode's contract two said: once a Future returns Ready, it must never be polled again. Back then, block_on simply returned the moment it got Ready, so it couldn't offend; but now that the executor keeps many Tasks at once, things aren't so simple anymore.

The threat comes from the Waker copies scattered outside. After a Task completes, the executor itself certainly won't requeue it; but a copy like the one Delay handed to its timing Thread is beyond the executor's recall. If someone holding such a stale Waker calls wake **after** the Task has completed, the finished Task gets requeued onto the ready queue and then polled again — contract two is broken, and remaining -= 1 gets subtracted one extra time.

So a Task also needs a done flag, to invalidate stale wakeups. The defense sits on the executor's side: after popping a Task, check done first — if it's true, just continue past it. That way a stale wakeup at worst puts the Task into the queue one extra time; it can never get it polled again. And since only the executor Thread ever polls the Futures, and only it sets done to true, "never poll again once done is set" holds strictly.

Honestly, this episode's examples can't actually trigger the problem — every timer fires exactly once, and always before its Task completes. But the executor's correctness can't rest on that kind of luck.

12.11.2.6 Why the Future Field Must Be Send

You may also notice the Task's future field is typed Mutex<Pin<Box<dyn Future<Output = ()> + Send>>> — why Send?

Follow the chain and it makes sense: the Future goes inside the Task, and the Task also impl Wake, doubling as the Waker (in theory the Task needn't be its own Waker, but this is the most economical way to write it). The conversion Waker::from(Arc<Task>) requires Task: Send + Sync + 'static. For a type to be Send + Sync, **every field** must be Send + Sync — including that Future.

Hence the `dyn Future` gains `+ Send` (so it can be moved to another `Thread`), wrapped in a `Mutex` (a `Mutex<T>` is automatically `Sync` when `T: Send`). Last episode's `Waker` was simple enough in construction that we didn't have to fret over these bounds; this episode, with the `Task` serving as its own `Waker`, they must be taken seriously.

Next episode we build on this and let `spawn` return results — adding a `JoinHandle`.

12.11.3 Recap

- Wrap each `Future` into a **`Task`** (`Future` + scheduling carry-on data); the executor manages `Task`s, not bare `Future`s.
- The **ready queue** holds the `Task`s due for polling; a `waked` `Task` requeues itself, then `unpark`s the executor.
- `unpark` is only the "get up" alarm — it doesn't say which `Task` is ready; that information lives in the ready queue.
- `spawn` is an `Executor` method: wrap the `Future` into a `Task` and put it on its ready queue.
- `queued.swap(true, ...)` is like `Option::take`: grab the old value, leave the new — one atomic operation, preventing duplicate queue entries.
- After a `Task` completes, unretrievable `Waker` copies may still deliver **stale wakeups**; the `done` flag upholds contract two — the executor checks `done` right after popping a `Task`, sets it on `Ready`, and every wakeup thereafter is void.
- With `Task` as its own `Waker`, `Waker::from(Arc<Task>)` demands `Task: Send + Sync + 'static`, so the `Future` field needs `+ Send` and a `Mutex` around it.

12.12 `spawn` and `JoinHandle`

12.12.1 Goal of This Episode

Let `spawn`ed `Task`s hand their results back, by adding a `JoinHandle` — an `.await`able waiting end.

12.12.2 Main Text

12.12.2.1 Only Three Things Different from Last Episode

Last episode's `spawn` had a limitation: it only accepted `Future<Output = ()>` — once the work finished, that was that; no way to return the result. This episode fills that in.

The good news: the core scheduling logic **doesn't change at all**. We add just three things on top:

1. A new shared state `Shared<T>`, plus a `JoinHandle<T>` (which is itself a `Future`).
2. `Executor::spawn` upgrades from accepting only `Future<Output = ()>` to accepting `Future<Output = T>` and returning `JoinHandle<T>`.
3. `Executor::block_on` upgrades from returning `()` to returning the passed-in `Future`'s value, `T`.

12.12.2.2 How the Finishing Side Notifies the Waiting Side

The core question: when the background Task finishes, how does it deliver the result to "whoever is .awaiting it"?

The answer: **through a piece of shared state, Shared<T>** — not one Future notifying another directly. Shared<T> holds two things: the computed result, and the Waker of the Task polling the JoinHandle.

The flow underneath goes like this:

- A JoinHandle<T> is itself a Future, but creating one does not automatically schedule it as another Task. In this example, it is polled when the executor reaches handle.await while polling the async block passed to block_on. If passed directly to block_on, block_on would wrap it in a Task, just like any other input Future.
- When a Task polls the JoinHandle and the result isn't ready, the JoinHandle stores cx.waker() — the current Task's Waker — into Shared<T> and returns Pending.
- When the background Task finishes, it puts the result into Shared<T>, then calls wake on the stored Waker. This requeues the Task that polled the JoinHandle and unparks the executor; the next time that Task is polled, it can retrieve the result.

```
use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};
use std::time::{Duration, Instant};

struct Delay {
    when: Instant,
    started: bool,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration,
            started: false,
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        if Instant::now() >= this.when {
            Poll::Ready(())
        } else {
            if !this.started {
                this.started = true;
```

```
                let waker = cx.waker().clone();
                let when = this.when;
                thread::spawn(move || {
                    let now = Instant::now();
                    if now < when {
                        thread::sleep(when - now);
                    }
                    waker.wake();
                });
            }
            Poll::Pending
        }
    }
}

type Queue = Arc<Mutex<VecDeque<Arc<Task>>>>;

struct Task {
    future: Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>,
    queue: Queue,
    executor_thread: Thread,
    queued: AtomicBool,
    done: AtomicBool,
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        if !self.queued.swap(true, Ordering::SeqCst) {
            self.queue.lock().expect("lock failed").push_back(self.clone());
            self.executor_thread.unpark();
        }
    }
}

// state shared between a background Task and its JoinHandle
struct Shared<T> {
    state: Mutex<(Option<T>, Option<Waker>)>, // (result, current Task's Waker)
}

struct JoinHandle<T> {
    shared: Arc<Shared<T>>,
}

impl<T> Future for JoinHandle<T> {
    type Output = T;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<T> {
        let mut state = self.shared.state.lock().expect("lock failed");
        if let Some(value) = state.0.take() {
            Poll::Ready(value) // the result is ready
        } else {
            state.1 = Some(cx.waker().clone()); // not yet — store the current Task's Waker
            Poll::Pending
        }
    }
```

```
}

struct Executor {
    queue: Queue,
    executor_thread: Thread,
    remaining: usize,
}

impl Executor {
    fn new() -> Executor {
        Executor {
            queue: Arc::new(Mutex::new(VecDeque::new())),
            executor_thread: thread::current(),
            remaining: 0,
        }
    }

    // spawn<T>: accept a Future<Output = T>, return a JoinHandle<T>
    fn spawn<T, F>(&mut self, future: F) -> JoinHandle<T>
    where
        F: Future<Output = T> + Send + 'static,
        T: Send + 'static,
    {
        let shared = Arc::new(Shared { state: Mutex::new((None, None)) });
        let shared_for_task = shared.clone();

        // wrap the Future<Output = T> into a Future<Output = ()> the executor understands
        let task_future = async move {
            let value = future.await; // actually run the job
            let mut state = shared_for_task.state.lock().expect("lock failed");
            state.0 = Some(value); // deposit the result
            if let Some(waker) = state.1.take() {
                waker.wake(); // wake whoever is waiting
            }
        };

        let task = Arc::new(Task {
            future: Mutex::new(Box::pin(task_future)),
            queue: self.queue.clone(),
            executor_thread: self.executor_thread.clone(),
            queued: AtomicBool::new(false),
            done: AtomicBool::new(false),
        });

        self.remaining += 1;
        task.wake();

        JoinHandle { shared }
    }

    fn block_on<T, F>(&mut self, future: F) -> T
    where
        F: Future<Output = T> + Send + 'static,
        T: Send + 'static,
    {
```

```
        let handle = self.spawn(future); // spawn it as a Task; keep its JoinHandle

        // run until every Task completes (the loop is identical to last episode)
        while self.remaining > 0 {
            loop {
                let task = self.queue.lock().expect("lock failed").pop_front();
                let Some(task) = task else { break };

                if task.done.load(Ordering::SeqCst) {
                    continue;
                }

                task.queued.store(false, Ordering::SeqCst);
                let waker = Waker::from(task.clone());
                let mut cx = Context::from_waker(&waker);
                let mut future = task.future.lock().expect("lock failed");

                if future.as_mut().poll(&mut cx).is_ready() {
                    task.done.store(true, Ordering::SeqCst);
                    self.remaining -= 1;
                }
            }

            if self.remaining > 0 {
                thread::park();
            }
        }

        // pull the result out of the Shared and return it
        handle.shared.state.lock().expect("lock failed").0.take().expect("result not ready")
    }
}

fn main() {
    let mut executor = Executor::new();

    // spawn a background Task that returns an i32
    let handle = executor.spawn(async {
        Delay::new(Duration::from_secs(1)).await;
        println!("background task: computed");
        21 * 2
    });

    let result = executor.block_on(async move {
        // .await the background Task's JoinHandle here to get the result
        let value = handle.await;
        println!("main task: got the background result {}", value);

        value + 100 // return a value of our own
    });
    println!("block_on returned: {}", result);
}
```

12.12.2.3 Walking Through It Step by Step

Say A is the background Task above: it waits one second, then computes 42. B is the Task passed to block_on: it .awaits A's JoinHandle and, once it has the result, returns 142.

1. executor.spawn(A): spawn first builds a wrapper task_future, responsible for awaiting A, writing the result into Shared<T>, and waking the waiter. What actually enters the ready queue is this wrapped Task; spawn then immediately returns a JoinHandle<i32>.
2. executor.block_on(B): B is also spawned as a Task and queued; block_on keeps B's JoinHandle for itself, to extract B's return value at the end.
3. The executor polls Task A first. What's actually polled is the outer task_future; it reaches let value = future.await and only then starts polling the real inner A. Inner A reaches Delay::new(...).await and polls the Delay; the Delay isn't done, so it returns Pending. That Pending propagates back out through the task_future, and A's poll is over for now.
4. After A's Pending, the executor doesn't sleep — the ready queue still holds B. It immediately polls Task B. Likewise, B's outer task_future is polled first; it reaches let value = future.await and starts polling the async block that was passed to block_on.
5. B's inner async block reaches handle.await, so it polls A's JoinHandle. A's result isn't ready yet, so the JoinHandle stores B's Waker into Shared<T> and returns Pending. That Pending propagates back out through B's task_future, and B pauses too.
6. The ready queue is empty; the executor falls asleep via thread::park().
7. About a second later, A's timing Thread calls A's Waker; A is requeued and the executor is unparked awake.
8. The executor polls A again. As before, A's outer task_future is polled first and continues polling inner A; the Delay has completed, so A resumes past the .await, first printing background task: computed, then computing 42.
9. A's outer task_future receives the 42, deposits it into Shared<T>, then takes out B's stored Waker and wakes it. This doesn't directly resume B — it requeues B onto the ready queue.
10. The executor next polls B. B's outer task_future continues polling the inner async block; this time handle.await retrieves 42 from Shared<T>, prints main task: got the background result 42, and B returns 142.
11. B's own outer task_future writes 142 into B's own Shared<T>. All Tasks are done; block_on extracts 142 from B's JoinHandle and returns it, finally printing block_on returned: 142.

12.12.2.4 Whose Waker Is cx.waker(), Exactly

Having walked that through, we can add a point you may not have noticed but which matters. When the executor polls a Task, it first builds a Waker from that Task and puts it in the Context; that Context is then passed down through the outer task_future, the inner async block, and on to the Future before each .await. In other words, every Future polled along the way within a Task shares that same Task's Waker.

This is close to the meaning of Task as the unit of scheduling: what gets requeued and repolled by the executor is the Task, not any individual Future inside. So when an inner Future registers how it wants to be woken, there's no more sensible Waker to use than the current

`Task`'s.

Mapping back onto the walkthrough, you can see the two different `Waker`s at play: when A is `poll`ed (step 3), the `Delay` gets **A's** `Waker` from `cx.waker()` and hands it to the timing `Thread`, which on expiry wakes "A, the doer" (step 7); when B is `poll`ed (step 5), the `JoinHandle` gets **B's** `Waker` from `cx.waker()` and stores it into `Shared<T>`, which A uses on completion to wake "B, the result-waiter" (step 9). Different origins, but both end down the same road: requeue the corresponding `Task`, then `unpark` the executor.

12.12.2.5 Not `Future`-to-`Future` Notification

Please note: there is no direct line between the `JoinHandle` and the background `Task`; they only share a `Shared<T>`. The waiting side leaves its `Waker` in the shared state; the finishing side, when done, takes that `Waker` out of the shared state and `wake`s it. All waking ultimately returns to the same old road: "requeue onto the ready queue + `unpark` the executor."

At this point, our hand-written executor is looking respectable: it can `spawn`, sleep, and be woken. But one big puzzle piece is missing — "waiting" still relies on opening a `Thread` per `Delay`. Starting next episode, we bring in `mio` and the reactor, watching real I/O with just a few `Thread`s.

12.12.3 Recap

- `JoinHandle<T>` is a `Future`; `.await` it to get the background `Task`'s return value.
- The scheduling core is unchanged; only three additions: `Shared<T>` + `JoinHandle<T>`, an `Executor::spawn` returning `JoinHandle<T>`, and an `Executor::block_on` returning `T`.
- When the executor `poll`s a `Task`, the `Context` flows down to the inner `Future`s; so the `cx.waker()` an inner `Future` sees is the current `Task`'s `Waker`.
- A `JoinHandle` has no `Waker` of its own; when its `poll` returns `Pending`, it stores the current `Task`'s `Waker` in `Shared<T>`.
- On completion, the background `Task` deposits the result into `Shared<T>`, then `wake`s that `Waker`, requeuing the `Task` that `poll`ed the `JoinHandle`.
- Waking is not `Future` notifying `Future` directly — the finisher wakes the waiter through shared state.

12.13 `mio`

12.13.1 Goal of This Episode

Meet `mio` — the tool that makes "one `Thread` watching a big pile of I/O sources" possible, and the foundation of next episode's reactor.

12.13.2 Main Text

12.13.2.1 The Reactor's Role in the Runtime

Let's step back and look at the full picture of the runtime we're building. A runtime really has two roles, each with its own duty:

- **Executor**: takes `Task`s off the ready queue and `poll`s them — "running `Task`s." It knows

nothing about the outside world — not whether a network packet has arrived, nor whether a file read is done.
- **Reactor**: watches all the I/O sources, and wakes the corresponding `Task` when one becomes ready — "waiting for events." It does **not** `poll` `Futures` and is not a `Task`; it only watches external event sources.

In recent episodes our "waiting" relied on one `Thread` per `Delay` — far too wasteful. The reactor's mission is to watch **many** I/O sources with **one** `Thread`. The way to do that is this episode's star: `mio`.

12.13.2.2 `mio`'s Two Protagonists

`mio` is the low-level `crate` in the Rust ecosystem for cross-platform I/O event notification (Tokio uses it internally too). Add the dependency before use:

```
[dependencies]
mio = { version = "1", features = ["os-poll", "net"] }
```

Note one thing first: `mio` itself is **not** `async`. It doesn't know about `async fn`, won't build `Futures` for you, and won't `.await` on your behalf. What it provides is lower-level: for instance, setting a socket to non-blocking, registering it with a `Poll`, and letting "the `Thread` that calls `poll.poll(...)`" sleep while waiting on "is the socket ready."

This episode we only need to meet two of its pieces:

- **`mio::Poll`**: a place where you can "sleep waiting for I/O events." After one `Thread` registers many I/O sources with it, a single `poll.poll(...)` watches them all at once, waking whenever any shows activity.
- **`Token`**: an event source's "name tag." When registering an I/O source, you give it a `Token`; later, when `Poll` notifies you "there's an event," it hands that `Token` back, so you know which source is calling.

12.13.2.3 Watching a `TcpListener` with `mio`

The example below registers a `TcpListener` (the thing that accepts connections) with a `Poll`, then opens another `Thread` that connects to it after one second. The main `Thread` sleeps on `poll.poll()` and wakes when the listener reports readiness:

```
extern crate mio;

use mio::net::TcpListener;
use mio::{Events, Interest, Poll, Token};
use std::time::Duration;

// the listener's name tag
const SERVER: Token = Token(0);

fn main() {
    let mut poll = Poll::new().expect("Poll creation failed");
    let mut events = Events::with_capacity(128); // receive at most 128 events at a time

    let addr = "127.0.0.1:8080".parse().expect("failed to parse the address");
    let mut listener = TcpListener::bind(addr).expect("bind failed");
```

```
    // register the listener with the Poll: name tag SERVER, interested in "readable" events
    // (someone connecting counts as readable)
    poll.registry()
        .register(&mut listener, SERVER, Interest::READABLE)
        .expect("register failed");

    // another thread connects after one second
    std::thread::spawn(|| {
        std::thread::sleep(Duration::from_secs(1));
        let _ = std::net::TcpStream::connect("127.0.0.1:8080");
    });

    println!("sleeping on poll, waiting for I/O events…");
    loop {
        // poll sleeps here until a registered source has an event
        poll.poll(&mut events, None).expect("poll failed");

        for event in events.iter() {
            match event.token() {
                SERVER => {
                    // token matches: the listener reported readable, so try accepting
                    match listener.accept() {
                        Ok((_stream, addr)) => {
                            println!("someone connected: {}", addr);
                            return; // it's an example, so call it a day
                        }
                        // readiness events may be spurious; just wait for the next one
                        Err(e) if e.kind() == std::io::ErrorKind::WouldBlock => {}
                        Err(e) => panic!("accept failed: {}", e),
                    }
                }
                _ => {}
            }
        }
    }
}
```

12.13.2.4 Walking the Flow

1. `Poll::new()` makes a `Poll`.
2. `registry().register(&mut listener, SERVER, Interest::READABLE)` registers the `listener`, gives it the name tag `SERVER`, and states our interest — "readable" (`Interest::READABLE`). To wait on "writable," use `Interest::WRITABLE`.
3. `poll.poll(&mut events, None)` puts this `Thread` **to sleep** until a registered source has an event (`None` means no timeout — sleep until something happens).
4. On waking, check the `events` one by one. `event.token()` hands back the name tag from registration; matching `SERVER` tells us the `listener` reported readiness, so we try `accept()`:
    - Success means we really got a connection.
    - `WouldBlock` means it still can't accept one right now, so we go back to waiting for the next event. Readiness notifications may be **spurious**, so this is normal rather than a failure.

- Other errors mean something actually went wrong; this simplified example panics.

`mio`'s sockets are **non-blocking**: calling `accept` or `read` does not make the `Thread` wait for a connection or data. If the operation can't proceed yet, it returns immediately with `WouldBlock`. In this episode that means going back to `poll.poll()`; next episode, after we wrap I/O in a `Future`, the same situation maps to `Poll::Pending`.

The crux: even if you register **a hundred** I/O sources, only **one** `Thread` sleeps on that same `poll.poll()`. Whichever source acts up, `Poll` hands you its name tag. This is exactly the secret weapon a reactor uses to watch masses of I/O with just a few `Threads`.

Next episode, we hook `mio` up to the runtime we hand-wrote earlier, building a real reactor — the first time our runtime can handle real network I/O.

12.13.3 Recap

- A runtime has two roles: the **executor** runs `Tasks` (`poll`), the **reactor** waits for events (watching I/O and `wake`ing the right `Task`); the reactor isn't a `Task` and doesn't `poll` `Futures`.
- `mio` itself is not an `async` runtime: it only does event notification for non-blocking I/O — no `Futures`, no `.await`, no `Task` scheduling.
- `mio::Poll` is the "sleep waiting for I/O events" place; one `Thread` can watch many I/O sources at once.
- A `Token` is a source's name tag: you give it at registration, and `Poll` returns it when the event fires so you can identify the source.
- Register with `registry().register(&mut source, token, Interest::READABLE)`, sleep on `poll.poll()`, identify by `event.token()`, then try the I/O operation. If it returns `WouldBlock`, wait for another event; inside a `Future`, that maps to `Poll::Pending`.

12.14 Writing a Reactor by Hand

12.14.1 Goal of This Episode

Hook the waking machinery from recent episodes up to **real I/O** — build a reactor, so our runtime can handle network connections for the first time.

12.14.2 Main Text

12.14.2.1 Not One Line of the Executor Changes

Here's something reassuring about this episode: **the executor carries over from Episode 12 unchanged**. `Task`, `Executor::spawn<T>`, `JoinHandle<T>`, `Shared<T>`, `Executor::block_on` — not a line needs touching.

The only thing we swap out is "who does the `wake`ing." Before, each `Delay` opened its own timing `Thread` to `wake`; now we switch to a single **reactor `Thread`**, sleeping on a `mio::Poll` waiting for real I/O, and on waking it finds the matching `Waker` and `wake()`s it.

What we add is a `Reactor`, plus two I/O `Futures` (`Accept` and `Read`).

12.14.2.2 The `Reactor` and the I/O `Futures`

The `Reactor` runs on its own `Thread`, asleep on a `mio::Poll`. So how do the `Futures` running on the executor `Thread` talk to it? The answer: **through shared state, not messages**. Three things are shared via `Arc`:

- **`Registry`** (from `mio`): `Futures` use it directly to register / deregister sockets.
- **`AtomicUsize`**: the reactor uses it to self-allocate a unique `Token` per source.
- **`Mutex<HashMap<Token, Waker>>`**: `Futures` write their `Waker` in as they run (keyed by `Token`); when the reactor receives an event it fetches by `Token` and `wakes`.

```
use std::collections::{HashMap, VecDeque};
use std::future::Future;
use std::io::Read as _;
use std::pin::Pin;
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};
use mio::event::Source;
use mio::net::{TcpListener, TcpStream};
use mio::{Events, Interest, Poll as MioPoll, Registry, Token};

type Queue = Arc<Mutex<VecDeque<Arc<Task>>>>;

struct Task {
    future: Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>,
    queue: Queue,
    executor_thread: Thread,
    queued: AtomicBool,
    done: AtomicBool,
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        if !self.queued.swap(true, Ordering::SeqCst) {
            self.queue.lock().expect("lock failed").push_back(self.clone());
            self.executor_thread.unpark();
        }
    }
}

struct Shared<T> {
    state: Mutex<(Option<T>, Option<Waker>)>,
}

struct JoinHandle<T> {
    shared: Arc<Shared<T>>,
}

impl<T> Future for JoinHandle<T> {
    type Output = T;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<T> {
        let mut state = self.shared.state.lock().expect("lock failed");
```

```
        if let Some(value) = state.0.take() {
            Poll::Ready(value)
        } else {
            state.1 = Some(cx.waker().clone());
            Poll::Pending
        }
    }
}

struct Executor {
    queue: Queue,
    executor_thread: Thread,
    remaining: usize,
}

impl Executor {
    fn new() -> Executor {
        Executor {
            queue: Arc::new(Mutex::new(VecDeque::new())),
            executor_thread: thread::current(),
            remaining: 0,
        }
    }

    fn spawn<T, F>(&mut self, future: F) -> JoinHandle<T>
    where
        F: Future<Output = T> + Send + 'static,
        T: Send + 'static,
    {
        let shared = Arc::new(Shared { state: Mutex::new((None, None)) });
        let shared_for_task = shared.clone();

        let task_future = async move {
            let value = future.await;
            let mut state = shared_for_task.state.lock().expect("lock failed");
            state.0 = Some(value);
            if let Some(waker) = state.1.take() {
                waker.wake();
            }
        };

        let task = Arc::new(Task {
            future: Mutex::new(Box::pin(task_future)),
            queue: self.queue.clone(),
            executor_thread: self.executor_thread.clone(),
            queued: AtomicBool::new(false),
            done: AtomicBool::new(false),
        });

        self.remaining += 1;
        task.wake();

        JoinHandle { shared }
    }
```

```
    fn block_on<T, F>(&mut self, future: F) -> T
    where
        F: Future<Output = T> + Send + 'static,
        T: Send + 'static,
    {
        let handle = self.spawn(future);

        while self.remaining > 0 {
            loop {
                let task = self.queue.lock().expect("lock failed").pop_front();
                let Some(task) = task else { break };

                if task.done.load(Ordering::SeqCst) {
                    continue;
                }

                task.queued.store(false, Ordering::SeqCst);
                let waker = Waker::from(task.clone());
                let mut cx = Context::from_waker(&waker);
                let mut future = task.future.lock().expect("lock failed");

                if future.as_mut().poll(&mut cx).is_ready() {
                    task.done.store(true, Ordering::SeqCst);
                    self.remaining -= 1;
                }
            }

            if self.remaining > 0 {
                thread::park();
            }
        }

        handle.shared.state.lock().expect("lock failed").0.take().expect("result not ready")
    }
}

struct Reactor {
    registry: Registry, // Futures use it to register / deregister sockets
    next_token: AtomicUsize, // self-allocated Tokens
    wakers: Mutex<HashMap<Token, Waker>>, // Token -> waiting Waker
}

impl Reactor {
    fn unique_token(&self) -> Token {
        Token(self.next_token.fetch_add(1, Ordering::Relaxed))
    }

    fn register(&self, source: &mut impl Source, token: Token, interest: Interest) {
        self.registry.register(source, token, interest).expect("register failed");
    }

    fn deregister(&self, source: &mut impl Source) {
        self.registry.deregister(source).expect("deregister failed");
    }
```

```
    fn set_waker(&self, token: Token, waker: Waker) {
        self.wakers.lock().expect("lock failed").insert(token, waker);
    }

    fn clear_waker(&self, token: Token) {
        self.wakers.lock().expect("lock failed").remove(&token);
    }

    // runs on its own Thread: sleeps on poll, wakes and looks up Wakers by Token
    fn run(&self, mut poll: MioPoll) {
        let mut events = Events::with_capacity(128);
        loop {
            poll.poll(&mut events, None).expect("poll failed");
            for event in events.iter() {
                let waker = self
                    .wakers
                    .lock()
                    .expect("lock failed")
                    .remove(&event.token());

                if let Some(waker) = waker {
                    waker.wake();
                }
            }
        }
    }
}

fn start_reactor() -> Arc<Reactor> {
    let poll = MioPoll::new().expect("Poll creation failed");
    let registry = poll.registry().try_clone().expect("failed to clone the Registry");
    let reactor = Arc::new(Reactor {
        registry,
        next_token: AtomicUsize::new(0),
        wakers: Mutex::new(HashMap::new()),
    });
    // the reactor runs on its own Thread
    let reactor_for_thread = reactor.clone();
    std::thread::spawn(move || reactor_for_thread.run(poll));
    reactor
}

// now for the new Futures

struct Accept {
    reactor: Arc<Reactor>,
    listener: TcpListener,
    listener_token: Token,
}

impl Accept {
    fn new(reactor: Arc<Reactor>, mut listener: TcpListener) -> Accept {
        let listener_token = reactor.unique_token();
        reactor.register(&mut listener, listener_token, Interest::READABLE);
        Accept { reactor, listener, listener_token }
```

```
    }
}

impl Future for Accept {
    type Output = TcpStream;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<TcpStream> {
        let this = self.get_mut();
        // the order is deliberately "register the Waker first, then try accept".
        // if we tried accept first, got WouldBlock, and only then went to register the Waker,
        // a connection might arrive in between; the reactor would find no Waker to wake,
        // and the executor could oversleep.
        this.reactor.set_waker(this.listener_token, cx.waker().clone());
        match this.listener.accept() {
            Ok((stream, _addr)) => {
                // this poll may have "registered first, then immediately succeeded".
                // after success there is no I/O event to wait for; clear the stored Waker.
                this.reactor.clear_waker(this.listener_token);
                this.reactor.deregister(&mut this.listener);
                Poll::Ready(stream)
            }
            Err(e) if e.kind() == std::io::ErrorKind::WouldBlock => Poll::Pending,
            Err(e) => panic!("accept failed: {}", e),
        }
    }
}

struct Read<'a> {
    reactor: Arc<Reactor>,
    stream: &'a mut TcpStream,
    buf: &'a mut [u8],
    stream_token: Token,
}

impl<'a> Future for Read<'a> {
    type Output = usize;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<usize> {
        let this = self.get_mut();
        this.reactor.set_waker(this.stream_token, cx.waker().clone()); // register first
        match this.stream.read(this.buf) { // then try
            Ok(n) => {
                // clear the Waker
                this.reactor.clear_waker(this.stream_token);
                Poll::Ready(n)
            }
            Err(e) if e.kind() == std::io::ErrorKind::WouldBlock => Poll::Pending,
            Err(e) => panic!("read failed: {}", e),
        }
    }
}

// accept one connection; read/print requests (simplified, no timeout)
async fn serve(reactor: Arc<Reactor>, listener: TcpListener) {
    let mut stream = Accept::new(reactor.clone(), listener).await;
```

```
    let stream_token = reactor.unique_token();
    reactor.register(&mut stream, stream_token, Interest::READABLE);

    for i in 1..=3 {
        let mut buf = vec![0u8; 1024];
        let n = Read {
            reactor: reactor.clone(),
            stream: &mut stream,
            buf: &mut buf,
            stream_token,
        }
        .await;
        if n == 0 {
            println!("the connection closed");
            break;
        }
        println!("request {}: {}", i, String::from_utf8_lossy(&buf[..n]).trim());
    }

    reactor.deregister(&mut stream);
}

fn main() {
    let reactor = start_reactor();
    let addr = "127.0.0.1:8080".parse().expect("failed to parse the address");
    let listener = TcpListener::bind(addr).expect("bind failed");

    let mut executor = Executor::new();
    executor.block_on(serve(reactor, listener));
}
```

> Note: this program listens on `127.0.0.1:8080` locally; you need a tool like `nc` to connect to it (e.g. `nc 127.0.0.1 8080`) to see the effect. The web sandbox isn't suited to this kind of interactive network program; to experience the full result, run the code on your own machine.

12.14.2.3 `Token`s Are Bound to I/O Sources

`Accept` and `Read` don't share one `Token`. The `listener_token` inside `Accept` belongs to the `TcpListener`; after a connection is accepted, `serve` creates a separate `stream_token` and registers it for that `TcpStream`.

The three later `Read`s share the same `stream_token`, deliberately: a `Token` is the name tag of an I/O source — you don't swap tags for every `.await`. This simplified example only ever waits on one `read` on this stream at a time, so one stream `Token` mapping to one waiting `Waker` suffices.

After the I/O succeeds, `Accept` / `Read` call `clear_waker`, removing this wait's `Waker` from the `HashMap`. That way the reactor holds no waiters who "no longer need waking."

12.14.2.4 Why "Register First, Then Try" Matters

Notice that both `Accept`'s and `Read`'s `poll` do `set_waker` **first**, and **then** try `accept` / `read` once. The order is deliberate.

That "try once" doesn't mean this round will succeed. If it's still `WouldBlock`, this `poll` returns `Pending`; later, when the reactor receives the event and calls the `Waker` we just stored, the executor `poll`s this `Future` again next round, and only then does it retry the I/O.

Imagine the reverse: try `read` first, get `WouldBlock` (no data), and just as you're about to register the `Waker` — in that gap, the data arrives. The reactor wakes wanting to `wake`, but finds no `Waker` for this `Token` in the `HashMap`; the wakeup is **missed**, and this `Future` is never `poll`ed again.

Flipping the order — put the `Waker` in place first, then try the I/O once — plugs that gap: if the data already arrived, this `accept` / `read` simply succeeds and returns `Ready`; if it truly hasn't, the `Waker` is already in position, and the reactor's notification triggers the next round. Success $\rightarrow$ `Ready`; `WouldBlock` $\rightarrow$ `Pending`. And precisely because we "register first, then try," if this `accept` / `read` does succeed immediately, the `Waker` just placed in the `HashMap` is no longer needed. That's when `Accept` / `Read` call `clear_waker` before returning `Ready` to remove it. In other words, `set_waker` prevents "missing a wakeup by registering too late," and `clear_waker` prevents "finishing but leaving behind an unneeded waiter."

12.14.2.5 The Wake Path Is Completely Unchanged

Compare this episode with Episode 12 and you'll find the wake path ends at exactly the same place. The reactor may run on its own `Thread`, but the `waker.wake()` it calls is still some `Task`'s `Waker` — and `wake` still requeues that `Task` and `unpark`s the executor. We merely replaced "the party responsible for waking the `Thread`" — timing `Thread` out, reactor `Thread` in; everything downstream is untouched.

And with that, our from-scratch, hand-written runtime is complete as a teaching project! It can `spawn`, sleep, and be woken by timers or real I/O.

This simplified implementation still has many bugs in its details and leaves plenty of edge cases unhandled, so it is far from a production-ready runtime. Even so, it is enough to show the rough shape of what an async runtime does behind the scenes: schedule `Task`s, wait for events, and wake `Future`s so the executor can `poll` them again.

With that overall picture in place, in the coming episodes we turn back to finally open up the "state machine" behind `async fn` that we've kept mentioning but never dissected.

12.14.3 Recap

- The reactor connects waking to real I/O: **the executor carries over from Episode 12 unchanged**; only "who `wake`s" switches from timing `Thread`s to the reactor `Thread`.
- The reactor runs on its own `Thread`, sleeps on `mio::Poll`, and on waking fetches `Waker`s from the `HashMap` by `Token` to `wake`.
- `Future`s and the reactor communicate through `Arc`-shared `Registry`, `AtomicUsize`, and `Mutex<HashMap<Token, Waker>>` — shared state, not messages.
- A `Token` is an I/O source's name tag: the listener has its `listener_token`, the stream its `stream_token`; in our code, multiple `Read`s on one stream can share the same stream `Token`.
- `WouldBlock` is the normal state of non-blocking I/O — "can't `accept` / `read` yet, try later" — mapping to `Poll::Pending` in a `Future`.

- I/O `Futures`' `poll` always "**`set_waker` first, then try the I/O**" to avoid missed wakeups; on immediate success, `clear_waker` before returning `Ready`.
- Whether the wakeup comes from a timer or I/O, it takes the same road: "requeue onto the ready queue + `unpark` the executor."

12.15 The State Machine behind `async fn`

12.15.1 Goal of This Episode

Unmask `async fn`: the compiler rewrites it into a **state machine** that can pause and resume.

12.15.2 Main Text

12.15.2.1 `.await` Doesn't Open a New `Thread`

First, let's dispel a possible misconception. Seeing `.await`, you might imagine it "secretly opens a `Thread` in the background to wait." **Absolutely not.** From Episode 6 until now, the executor of our hand-written runtime has been one `Thread` `poll`ing over and over, start to finish — `.await` conjured no new `Threads`.

So what does `.await` actually do? It **cuts your function into segments** — every `.await` is a cut point. The function can pause at a cut point, hand control back to the executor, and later resume from that same cut point.

The compiler achieves this by rewriting the whole `async fn` into a **state machine**: "which state am I in" records the progress, and the next `poll` picks up from that state and continues.

12.15.2.2 What an `async fn` Gets Rewritten Into

Suppose we have this `async fn` that waits twice:

```
async fn two_delays() {
    Delay::new(Duration::from_secs(1)).await;
    println!("one second is up");
    Delay::new(Duration::from_secs(1)).await;
    println!("two seconds are up");
}
```

On seeing it, the compiler rewrites it into an `enum` — each "state" standing for "which segment am I stuck in":

- `Start`: not yet begun.
- `FirstDelay`: waiting on the first `Delay` (the unfinished `Delay` itself must be stored in here too).
- `SecondDelay`: waiting on the second `Delay`.
- `Done`: finished.

It then implements `Future` for this `enum`, with `poll` using a `match` on the current state to decide what to do. Let's write this rewrite out **by hand**, and you'll see what an `async fn` looks like underneath:

```
use std::future::Future;
use std::pin::Pin;
```

```
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};
use std::thread::{self, Thread};
use std::time::{Duration, Instant};

struct ThreadWaker {
    thread: Thread,
}

impl Wake for ThreadWaker {
    fn wake(self: Arc<Self>) {
        self.thread.unpark();
    }
}

struct Delay {
    when: Instant,
    started: bool,
}

impl Delay {
    fn new(duration: Duration) -> Delay {
        Delay {
            when: Instant::now() + duration,
            started: false,
        }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        if Instant::now() >= this.when {
            Poll::Ready(())
        } else {
            if !this.started {
                this.started = true;
                let waker = cx.waker().clone();
                let when = this.when;
                thread::spawn(move || {
                    let now = Instant::now();
                    if now < when {
                        thread::sleep(when - now);
                    }
                    waker.wake();
                });
            }
            Poll::Pending
        }
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
```

```
    let mut future = Box::pin(future);
    let waker = Waker::from(Arc::new(ThreadWaker {
        thread: thread::current(),
    }));
    let mut cx = Context::from_waker(&waker);
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => thread::park(),
        }
    }
}

// this is roughly what the two_delays async fn looks like underneath
enum TwoDelays {
    Start,
    FirstDelay(Delay), // waiting on the first Delay — keep it stored
    SecondDelay(Delay), // waiting on the second Delay
    Done,
}

impl Future for TwoDelays {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        loop {
            match this {
                TwoDelays::Start => {
                    // enter first segment: create first Delay and switch state
                    *this = TwoDelays::FirstDelay(Delay::new(Duration::from_secs(1)));
                }
                TwoDelays::FirstDelay(delay) => match Pin::new(delay).poll(cx) {
                    Poll::Ready(()) => {
                        println!("one second is up");
                        *this = TwoDelays::SecondDelay(Delay::new(Duration::from_secs(1)));
                    }
                    Poll::Pending => return Poll::Pending, // stuck in this segment — pause
                },
                TwoDelays::SecondDelay(delay) => match Pin::new(delay).poll(cx) {
                    Poll::Ready(()) => {
                        println!("two seconds are up");
                        *this = TwoDelays::Done;
                        return Poll::Ready(());
                    }
                    Poll::Pending => return Poll::Pending,
                },
                TwoDelays::Done => panic!("shouldn't be polled after Ready"),
            }
        }
    }
}

fn main() {
    println!("start");
```

```
    block_on(TwoDelays::Start); // equivalent to block_on(two_delays())
}
```

The bare-bones `Delay` and `block_on` from earlier are included above only so this hand-written state machine actually runs. The star of this example isn't the executor but `TwoDelays`: it demonstrates the kind of state machine an `async fn` might be rewritten into.

12.15.2.3 Side by Side

Compare this hand-written state machine with the original `async fn`:

- The **progress** through the original `async fn` becomes **a variant** of the `enum`.
- **Local variables** still needed across an `.await` (here, the unfinished `Delay`) get stored inside the variant and carried along.
- Each `.await` becomes "`poll` the child `Future`: on `Ready`, switch to the next state and continue; on `Pending`, `return Poll::Pending` and pause."
- On the next `poll`, the `match` jumps straight to the state where it last stopped and continues from there — that's "resuming in place."

This exactly explains the phenomena of the past few episodes: why a `Future` remembers where it got to on every `poll`, and why it resumes from the same place after pausing. Because it simply is a state machine that remembers "which state I'm in."

When you write `async fn` day to day, all of this is generated automatically by the compiler — you never hand-write such an `enum`. But knowing its true face is what makes the upcoming episodes on `Pin` meaningful — because this auto-generated state machine hides a danger related to "moving memory around." Next episode we look at that danger.

12.15.3 Recap

- `.await` does **not** open a new `Thread`; it cuts the function into pausable, resumable segments.
- The compiler rewrites an `async fn` / `async` block into a **state machine** (conceptually an `enum`): progress becomes variants; locals crossing an `.await` are stored in the variant.
- `poll` uses `match` on the current state: child `Future` `Ready` → switch to the next state; `Pending` → return `Pending` and pause.
- The next `poll` jumps back to the last state and resumes in place — that's how a `Future` "remembers its progress."
- The rewrite is normally done by the compiler automatically, but understanding it is the prerequisite for understanding `Pin` later.

12.16 Self-referential `Futures`

12.16.1 Goal of This Episode

Understand why an `async` state machine can become a structure that "points at itself," and why moving such a structure spells trouble.

12.16.2 Main Text

12.16.2.1 Moving a Value Changes Its Address

Start with an ordinary program containing **no** `async` at all. Using `{:p}` (the address-printing format), we look at a value's address before and after a move:

```
fn main() {
    let p1 = String::from("hello");
    println!("p1's address: {:p}", &p1);

    let p2 = p1; // move: relocate p1 into p2
    println!("p2's address: {:p}", &p2);
}
```

The two addresses differ. Which makes sense — `p1` and `p2` are two different local variables living at different spots on the stack, and a move relocates the value from one place to the other.

For ordinary values this is perfectly fine: after the move, the old variable `p1` can't be used anymore (Chapter 4's ownership rules), so "the old address is dead" bothers no one.

12.16.2.2 But What If the Value Stores "an Address Pointing Into Itself"?

The trouble comes with a special kind of value: **one of its fields stores the address of another of its own fields**.

Imagine such a value being moved to a new location. The address stored inside **doesn't update itself** — it still points at the **old** spot. But what lived there has moved away, so the pointer becomes a **dangling pointer** (pointing into memory that's no longer valid). The moment anyone follows it, that's undefined behavior — the program might read garbage, or blow up outright.

Do such "pointing at itself" values actually come up? They do — **a self-referential `Future` state machine is exactly such a value**. Recall last episode: an `async fn` gets rewritten into a state machine, and locals needed across an `.await` get stored inside it. If one of those locals is "a reference to another local," then the state machine holds a field pointing at another of its own fields — a textbook self-referential structure.

```
async fn borrows() {
    let s = String::from("hello");
    let r = &s; // r borrows s
    other().await; // crossing an .await — both s and r must be preserved by the state machine
    println!("{}", r); // r is used after the .await
}
```

This `async fn`'s state machine, in the state at that `.await`, stores both `s` and `r`, with `r` pointing at `s`. That's self-reference. Move it while in that state, and `r` becomes a dangling pointer. Hence the conclusion: **once a `Future` has been `polled` into a possibly self-referential state, moving it is dangerous**.

12.16.2.3 First, Prove "create → `poll` → move → `poll`" Is Achievable

Before discussing defenses, though, let's confirm one thing: a `Future` really can be "moved after being `poll`ed, then `poll`ed again." Here's a minimal `Future` — `Counter` — that bumps a

count on every `poll` and prints `self`'s address with `{:p}`:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};

struct Counter {
    count: u32,
}

impl Future for Counter {
    type Output = ();

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        let this = self.get_mut();
        this.count += 1;
        println!("poll number {}, self at address {:p}", this.count, this);
        Poll::Pending
    }
}

fn main() {
    let mut cx = Context::from_waker(Waker::noop());

    let mut counter = Counter { count: 0 };
    let _ = Pin::new(&mut counter).poll(&mut cx); // poll once

    let mut moved = counter; // move to a new location
    let _ = Pin::new(&mut moved).poll(&mut cx); // poll again
}
```

Run it and the two `poll`s print **different** addresses — proof that the "`poll` → move → `poll` again" sequence really can happen, with the `Future` at a new address by the second `poll`. `Counter` has no self-references, so moving it is harmless; but swap in the self-referential state machine above, and that move is a disaster.

12.16.2.4 Rust's Line of Defense: If Moving Breaks It, You Don't Even Get in the Door

So how does Rust stop self-referential `Future`s from being moved about? Let's apply the same sequence to the "borrow across `.await`" `async fn` from before:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Waker};

async fn other() {}

async fn borrows() {
    let s = String::from("hello");
    let r = &s;
    other().await;
    println!("{}", r);
}

fn main() {
    let mut cx = Context::from_waker(Waker::noop());
```

```
    let mut fut = borrows();

    // try to do what Counter did: poll once
    let _ = Pin::new(&mut fut).poll(&mut cx); // compile error!

    // then move to a new location
    let mut moved = fut;

    // and poll again
    let _ = Pin::new(&mut moved).poll(&mut cx); // this wouldn't be allowed either
}
```

The compiler blocks it flat:

```
error[E0277]: `{async fn body of borrows()}` cannot be unpinned
```

The code spells out the whole "poll once, move, poll again" routine, but the compiler actually stops it at the very first `Pin::new(&mut fut)`.

`Pin::new` requires the type to be `Unpin` (meaning "safe to move" — details soon). `Counter` is `Unpin`, so it passes; but this self-referential `async fn` state machine is **not** `Unpin`, so `Pin::new` bars the way **before you've actually polled it or actually moved it**.

Comparing the two examples, Rust's line of defense is clear: things that survive moving (like `Counter`) get the convenience — move them at will; things that break when moved (self-referential state machines) don't even get through the `Pin::new` door. How `Pin` builds this defense out of the type system is the subject of what comes next.

12.16.3 Recap

- Moving a value changes its address; for ordinary values that's fine, since the old variable can't be used anymore.
- If a value stores "an address pointing into itself," a move leaves that address un-updated — a dangling pointer. Dangerous.
- The state machines produced by `async fn` / `async` blocks can be such values: if a borrow crosses an `.await`, the machine may hold both the borrowed value and the reference — one field pointing at another.
- The `Counter` example proves "`poll` → move → `poll` again" is achievable (two different addresses).
- Rust's defense is `Unpin`: `Counter` is `Unpin` and passes `Pin::new`; a self-referential `async` state machine is not `Unpin`, and `Pin::new` fails to compile.

12.17 Why `poll` Needs `Pin`

12.17.1 Goal of This Episode

Figure out why `poll` writes its `self` as `Pin<&mut Self>`, and by what means `Pin` actually turns "no moving allowed" into reality.

12.17.2 Main Text

12.17.2.1 What `poll` Wants

Recap: once a `Future` in a self-referential state gets moved, its internal self-pointing pointer dangles.

Remember the sequence we ran on `Counter` last episode?

```
let _ = Pin::new(&mut counter).poll(&mut cx); // poll once
let mut moved = counter; // move the whole thing to a new spot with let
let _ = Pin::new(&mut moved).poll(&mut cx); // poll again
```

The two `poll`s printed different addresses — proving a `Future` really can be "polled, moved away, polled again." `Counter` has no self-references, so the move doesn't matter; but run the same routine on a self-referential `Future`, and by the second `poll` it lies at a new address with its self-pointing pointer dangling.

So whoever advances the `Future` (the executor) must keep one rule: **between successive `poll`s, this `Future` must not be moved**. The question is: what kind of `self` should `poll` take to help enforce that rule? From `poll`'s point of view, it wants two properties at once:

1. **Hands-on access**: every `poll` must modify the `Future`'s internals (advancing the state machine, carrying progress forward), so it needs some kind of "mutable" access.
2. **But no relocation**: it must not let anyone seize the chance to move the whole `Future` off its address, or the self-references are ruined.

The ready-made tool — a plain `&mut Self` — satisfies only the first. If `poll` took `&mut Self`, then to the executor this `Future` would be just another ordinary value in hand; between two `poll`s the executor could `let moved = ...` it away exactly as above, with nothing to stop it.

So Rust needs a "hands-on but no-moving" flavor of `&mut`. That's `Pin<&mut T>` — you can read it as "**a `&mut T` tied down in place, not allowed to be moved away**."

Note, though, that "not allowed to move" doesn't mean the value could never move from the moment of its creation. Before being pinned, a value may be moved around under ordinary Rust rules just fine; what `Pin` guarantees is that once you've pinned a value needing pinning, the compiler can no longer let it be moved. After all, before `poll`ing starts, an `async` state machine holds no self-references yet; the real danger is the state machine establishing self-references after a `poll`, then getting relocated.

12.17.2.2 The Key: Leave No Route for Moving the Value Out

`Pin<&mut T>` claims "no moving" — but **on what grounds** can it deliver?

The two real keys:

1. When the `Pin` is created, no alternative route may remain by which you could later move the value.
2. Once you hold a `Pin`, the safe API must never hand you back an ordinary pointer capable of moving the value inside.

The first point first.

For `Pin` to keep its "no move" promise, it isn't enough for `Pin` itself to be leak-free; you must

also ask whether, after the Pin is created, the outside still holds another road to move the value away.

That's what makes Pin::new(&mut value) suspicious. A Pin<&mut T> is only a temporary borrow: once the borrow ends, the original variable outside still exists and can still be moved. If Pin::new(&mut value) were allowed for any T, then the Counter-style "poll, move, poll again" routine could be applied verbatim to a self-referential Future.

So for types that "break when moved," Pin::new(&mut value) ought not be freely available. Right — that's the principle; it's just that some types "don't break when moved," and Rust is happy to let those take this road. Next episode explains that part.

Now the second point.

To a Pin<&mut T>, the dangerous thing is a plain &mut T. Because with a &mut T you can do things like Option::take:

```
let old = option.take();
```

It doesn't just turn Some(value) into None — it moves the value out and returns it as Some(value). So if you have a pinned Option<Future> and someone can obtain a plain &mut Option<Future> from it, they can .take() it, moving that Future off its original address.

Hence, for an unknown, arbitrary T, a Pin<&mut T> won't give you a plain &mut T. More generally, Pin<P> carefully guards the pointer layer P. P might be &mut T, Box<T>, or another smart pointer. If a Pin<Box<T>> casually handed back the Box<T> inside, you'd again hold an ordinary owner of T, and could proceed to move T out. So for arbitrary T, Pin's safe API doesn't return pointers to T directly; it offers only a few operations that can't break the pin guarantee.

12.17.2.3 Pin Has Only a Few Moves

Precisely because its job is "blocking moves," Pin doesn't let you do much. The usual repertoire:

Read-only — a Pin<P<T>> can always be dereferenced to T (reading can't move the value out; no risk), courtesy of Deref:

```
impl<Ptr: Deref> Deref for Pin<Ptr> {
    type Target = Ptr::Target;
    fn deref(&self) -> &Ptr::Target { /* ... */ }
}
```

Re-lending a pinned reference — as_mut borrows something like a &mut Pin<Box<T>> into a Pin<&mut T>. as_mut can be called again and again, because it only borrows — and what it lends out is still a Pin<&mut T>, not a plain &mut T.

And of course, holding a Pin<&mut F>, you can do the one thing that matters most — call its poll. For a Future produced by an async fn or async block, you never implement poll yourself; the compiler generates the implementation for you. Episode 6's executor running future.as_mut().poll(...) over and over is exactly this: as_mut re-lends a Pin<&mut F> and feeds it to F's own poll — and when that F comes from an async fn / async block, what runs is precisely the compiler-generated poll.

12.17.2.4 `Pin` Pins the "Value," Not the "Pointer"

Next, a point that's very easy to get wrong:

> What `Pin<P>` pins is **the value `P` points to** — not "the `Pin<P>` pointer variable itself."

So a `Pin<Box<T>>` can **itself** be moved around freely. Shift it from one variable to another, stuff it into a `Vec`, take it back out — all fine, because what you're moving is only the pointer; the value it points to stays put at its original spot on the heap. Below we print the pointed-to value's address with `{:p}` (`&*` obtains a `&T` from `Pin<P<T>>`) and let the facts speak:

```
use std::pin::Pin;

struct Data {
    value: i32,
}

fn main() {
    let mut queue: Vec<Pin<Box<Data>>> = Vec::new();

    let boxed = Box::pin(Data { value: 7 });
    println!("before entering the queue, the value is at {:p}", &*boxed);

    queue.push(boxed); // the Pin<Box<Data>> pointer moves into the Vec
    let popped = queue.pop().unwrap(); // and moves back out

    println!("after leaving the queue, the value is at {:p}", &*popped); // identical address
}
```

The two printed addresses are exactly the same: the pointer went in and out of the `Vec`, but the `Data` on the heap was never moved. The only thing `Pin` forbids is the single act of "**using it to move the pointed-to value off its address**."

12.17.2.5 `Pin` Usually Stays Behind the Scenes

Finally, a reassurance: when you write everyday `async fn` + `.await` code on a ready-made runtime, `Pin` usually stays behind the scenes. The compiler generates the `Future` state machine, and the runtime pins and `poll`s it. We meet `Pin` directly in this chapter because we are exploring that machinery ourselves. So don't fret if the details of these episodes feel hazy — they are here to show you what happens underneath, not because you will routinely handle it all by hand.

Should the day come when you hand-roll a low-level `Future` and need to extract a field's `Pin<P<Inner>>` from an outer `Pin<P<Outer>>` (an operation called projection), the community's `pin-project` `crate` does it safely for you, no hand-written `unsafe` required. Knowing the tool exists is enough; we won't go deeper here.

And if you want to "get the value in a `Pin` back as a plain `&mut T`," next episode covers the trick that's often available when "moving wouldn't break it anyway."

12.17.3 Recap

- `poll` wants both "can modify the innards" and "can't be moved away"; a plain `&mut Self` can't block moves (the executor could still `let moved = ...` between `poll`s), so it won't

do.

- `Pin<&mut T>` is "a `&mut` tied down in place"; hence `poll` takes `Pin<&mut Self>`.
- Before being pinned, a value moves freely under normal Rust rules; `Pin` governs what happens "after pinning" — the value can't be moved thereafter.
- Blocking moves requires two things: creation must leave no external route to later move the value; and the safe API must never hand out inner pointers that could move the value.
- `Pin`'s repertoire is small: read via `Deref`, re-lend via `as_mut`, and of course feed it to `poll`.
- `Pin<P>` pins the pointed-to value, not the pointer itself — so a `Pin<Box<T>>` moves freely (even in and out of a `Vec`), which is why the executor can shuffle `Pin<Box<Fut>>`s around.
- For everyday `async fn` + `.await` on a ready-made runtime, `Pin` usually stays behind the scenes: the compiler generates the `Future` state machine, and the runtime pins and `poll`s it. In this chapter, we meet `Pin` directly because we are exploring that machinery ourselves.

12.18 `Unpin`

12.18.1 Goal of This Episode

Meet `Unpin`, the "doesn't break when moved" label, and see why, with it, a pinned `&mut` can turn back into a plain `&mut`.

12.18.2 Main Text

12.18.2.1 First, a Question: Who Does "No Moving" Actually Protect?

Last episode `Pin` went to great lengths to block post-pinning moves. But step back: who is this rule really guarding against?

The answer — apart from rare special cases like **self-referential `Future` state machines**, the types you use daily (`i32`, `String`, `Vec`, your own `struct`s…) don't break at all when moved; to them a move is just storing a few bytes somewhere else. Forcing "no moving" onto these types is pure meddling.

Rust separates the two camps with a label, and that label is **`Unpin`**: a type being `Unpin` means "**moving me doesn't break me; `Pin` needn't bother about me**."

12.18.2.2 Almost Everything Is `Unpin`

Like the `Send` / `Sync` introduced in the multithreading chapter, `Unpin` is an **`auto trait`** — if everything a type stores is `Unpin`, the type itself is `Unpin` by default. Skipping to the punchline: **the overwhelming majority of types are `Unpin`**.

A small gadget verifies this. The `assert_unpin` below only accepts `Unpin` types, and all the common values pass:

```
fn assert_unpin<T: Unpin>(_: T) {}

fn main() {
    assert_unpin(42);
    assert_unpin(String::from("hi"));
    assert_unpin(vec![1, 2, 3]);
```

```
    println!("these are all Unpin");
}
```

Even our hand-written `Delay`, `Counter`, `JoinAll`, and `JoinHandle` are all `Unpin` — their fields are ordinary movable things.

So what isn't `Unpin`? Run the same check on an `async fn` that borrows a local variable across an `.await` — the kind of self-referential state machine we saw in Episode 16 — and the compiler rejects it:

```
fn assert_unpin<T: Unpin>(_: T) {}

async fn other() {}

async fn demo() {
    let s = String::from("hi");
    let r = &s;
    other().await; // s and r both cross the .await
    println!("{}", r);
}

fn main() {
    assert_unpin(demo()); // compile error: demo()'s Future is not Unpin
}
```

The compiler says `... cannot be unpinned`. And rightly so: the `Future` of an `async fn` / async block **cannot be assumed `Unpin`**, because it might be exactly that move-breaks-it self-referential state machine.

12.18.2.3 `Unpin` Types Can Ask for the Value Back

Knowing who's `Unpin`, we can now pay off last episode's foreshadowing: "turning a pinned value back into a plain `&mut T`" is open to `Unpin` types.

The logic is direct: since this type doesn't break when moved, `Pin`'s protection was superfluous for it anyway — so you may as well have the plain `&mut T` back. Concretely, only when `T: Unpin` does `Pin<&mut T>` offer `get_mut` to turn back into `&mut T`, and only then does `Pin<P<T>>` implement `DerefMut`:

```
use std::pin::Pin;

fn main() {
    let mut n = 10;
    let mut pinned: Pin<&mut i32> = Pin::new(&mut n);

    // i32 is Unpin, so Pin<&mut i32> implements DerefMut
    *pinned = 100;
    println!("{}", pinned);

    // get_mut also recovers a plain &mut i32
    let back: &mut i32 = pinned.get_mut();
    *back += 5;
    println!("{}", back);
}
```

That's why every one of our custom Futures could open its poll with let this = self.get_mut(); without trouble. Those types are all Unpin, so of course get_mut works. If some day your Future isn't Unpin, that line fails to compile, forcing you to handle things carefully through Pin's methods instead.

12.18.2.4 Two Actions, Both Demanding Unpin

Last episode said Pin's guarantee rests on two things:

1. When the Pin is created, no alternative route may remain for later moving the value.
2. Once you hold a Pin, the safe API must never hand back an ordinary pointer that could move the value inside.

Now set Pin::new and get_mut side by side, and you'll see they each relax one of those restrictions — and both demand **the same condition**: Unpin:

```
// method one: create a Pin from an existing pointer
// not allowed if the pointed-to value isn't Unpin
impl<P: Deref> Pin<P> {
    pub fn new(pointer: P) -> Pin<P> where P::Target: Unpin { /* ... */ }
}

// method two: turn a pinned value back into a plain &mut
// also not allowed unless T is Unpin
impl<T: ?Sized> Pin<&mut T> {
    pub fn get_mut(self) -> &mut T where T: Unpin { /* ... */ }
}
```

Pin::new relaxes the first restriction. It lets you take an existing pointer — say a &mut T — and wrap it straight into a Pin<&mut T>. For move-breaks-it types this is dangerous, because Pin<&mut T> is only a temporary borrow; once the borrow ends, the original variable T outside can still be moved. So Pin::new is only permitted for T: Unpin.

get_mut relaxes the second restriction. It turns Pin<&mut T> back into a plain &mut T. That too is only safe for T: Unpin, since a plain &mut T can do things like Option::take that move values off their address. DerefMut likewise.

Unpin is the statement: "this type doesn't break when moved, so these actions are safe on it." Episode 16's Counter is Unpin — Pin::new(&mut counter), get_mut, and DerefMut all sail through; the self-referential async state machine is not Unpin, and both actions are denied it.

So the practical judgment is simple: is the Future in your hand Unpin? If yes, Pin::new, get_mut, DerefMut are yours to use. If not (typically the Future born of an async fn / async block), you must pin it in a way that upholds Pin's guarantee — Box::pin onto the heap, or the pin! macro debuting next episode, onto the stack.

12.18.3 Recap

- Apart from rare cases like self-referential async state machines, most everyday types survive moves just fine.
- Unpin is the "doesn't break when moved" label — an auto trait, implemented automatically by the compiler; **the vast majority of types are Unpin**.
- The Future of an async fn / async block can't be assumed Unpin (it may be a self-

referential state machine).

- Only when `T: Unpin` can `Pin<&mut T>` use `get_mut` to recover a plain `&mut T`, and only then does `Pin<P<T>>` implement `DerefMut` — which is why our hand-written `Future`s could call `self.get_mut()`.
- `Pin::new` and `get_mut` / `DerefMut` relax last episode's two restrictions respectively — creating a `Pin` from an existing pointer, and recovering a plain `&mut` — and both are open only to `Unpin`.
- When a `Future` isn't `Unpin`, pin it with `Box::pin` or next episode's `pin!`.

12.19 `pin!`

12.19.1 Goal of This Episode

Learn to pin a `Future` on the stack with `pin!`, and understand why it absolutely has to be a macro.

12.19.2 Main Text

12.19.2.1 stack pinning

So far, to pin a `Future` we've always used `Box::pin` — putting it on the heap. But sometimes you'd rather not pay for a heap allocation just to pin a `Future` (there is a cost to it), especially when the `Future` is only used within the current scope and never passed out.

For that, use `std::pin::pin!`. It pins a value within the current scope and gives you a `Pin<&mut T>`:

```
use std::future::Future;
use std::pin::pin;
use std::task::{Context, Poll, Waker};

async fn hello() -> i32 {
    42
}

fn main() {
    // pin this future on the stack, getting a Pin<&mut _>
    let mut future = pin!(hello());

    let mut cx = Context::from_waker(Waker::noop());
    match future.as_mut().poll(&mut cx) {
        Poll::Ready(value) => println!("done: {}", value),
        Poll::Pending => println!("not ready yet"),
    }
}
```

Here's an easy point of confusion: the compiler currently takes the conservative stance that `Future`s produced by `async fn`s like `hello()` — and `async` blocks — are all treated as **not** `Unpin`, even this `hello`, which has no `.await` and couldn't possibly be self-referential (the compiler doesn't want to judge case by case, so it simply implements `Unpin` for none of them). So last episode's `Pin::new` won't work on it — but `pin!` will: `pin!` performs **stack pinning**, and its

way of pinning doesn't require Unpin.

12.19.2.2 Why pin! Must Be a Macro

This is a delightful question. Why is pin! a macro rather than an ordinary function?

Grab the key fact first: the Pin<&mut T> that pin! hands you is a **reference**, and a reference must point at a value that's still alive. As long as you hold that Pin<&mut T>, the value it borrows must not disappear.

So the problem isn't merely "how to produce a Pin<&mut T>"; it's that the value this Pin<&mut T> borrows has to live long enough.

Written as an ordinary function, it would look like:

```
fn pin<T>(value: T) -> Pin<&mut T> { /* ??? */ }
```

That can't work. value is a local variable of the pin function itself. The moment the function returns, its locals are cleaned up and value vanishes — the returned Pin<&mut T> instantly becomes a dangling reference into invalidated memory. In fact the compiler flat-out won't let you return a reference to "a function's own local variable."

pin! isn't an ordinary function, so it doesn't have the "returning a reference to my own stack variable" problem. The Pin<&mut T> that pin! produces borrows a value in the scope where pin! is used, not some ordinary function's own temporary. Hence the borrow doesn't dangle the moment a function returns, and you can use it with confidence.

12.19.2.3 Contrast with Box::pin

Then why can Box::pin be an ordinary function? Because it takes an entirely different road: Box::pin puts the value on the **heap** and hands you ownership of that heap memory wrapped in a Pin<Box<T>>. Something owned on the heap outlives "this function call" — returning doesn't discard it — so returning an owning Pin<Box<T>> is perfectly fine.

In short, here's how they differ:

- pin!: **stack borrowing** — obtain a Pin<&mut T> from a value in the current block without heap allocation; since ordinary functions can't return references to their own stack variables, this must be done by a macro at the call site.
- Box::pin: **heap owning** — the value goes on the heap, ownership is handed over, it can be passed around freely, so an ordinary function works. The cost is one heap allocation.

12.19.3 Recap

- pin! does **stack pinning**: a Pin<&mut T> valid only within the current block, no heap allocation — good when the pinned value needn't leave the scope.
- pin! must be a **macro**, not a function: an ordinary function can't return a Pin<&mut T> into its own stack; the reference would dangle on return (and the compiler forbids it anyway).
- Box::pin hands over ownership with the value on the heap (outliving the call), so it can be an ordinary function; the difference is "stack borrowing vs heap owning."

12.20 async Recursion

12.20.1 Goal of This Episode

Understand why an `async fn` can't call itself directly, and how to fix it with `Box::pin`.

12.20.2 Main Text

12.20.2.1 Direct Recursion Fails to Compile

Let's try an async factorial — an `async fn` that `.await`s itself:

```
async fn factorial(n: u64) -> u64 {
    if n == 0 {
        1
    } else {
        n * factorial(n - 1).await // compile error
    }
}

fn main() {}
```

The compiler refuses outright:

```
error[E0733]: recursion in an async fn requires boxing
```

12.20.2.2 Why? Because the `Future` Type's Size Can't Be Determined

Recall Episode 15: an `async fn` is rewritten into a state machine, and anything used across an `.await` gets stored inside it.

The key here isn't whether the recursion terminates at runtime. `n == 0` is of course the base case, and execution would stop; but before the program ever runs, the compiler must first determine how big the `Future` returned by `factorial` is.

Roughly picture what it would need to look like:

```
enum FactorialFuture {
    Start { n: u64 },
    Waiting {
        n: u64,
        child: FactorialFuture,
    },
    Done,
}
```

The `Waiting` state must store the `factorial(n - 1)` being `.await`ed — and `factorial(n - 1)` returns the very same `FactorialFuture`. The type directly contains itself, and when the compiler tries to compute how much space the `child` field takes, no fixed answer ever comes out.

You've seen this situation before. Chapter 5's discussion of recursive types hit exactly the same problem: a `struct` directly containing itself has infinite size. The fix back then was `Box`, putting the recursive part on the heap — no matter how big `T` is, a `Box<T>` itself is always just pointer-sized.

12.20.2.3 The Fix: Wrap the Recursive Call in `Box::pin`

The fix for `async` recursion is the same: wrap the `Future` produced by the recursive call in `Box::pin`. Now the state machine stores only a fixed-size pointer instead of directly embedding another state machine of the same type:

```
use std::future::Future;
use std::task::{Context, Poll, Waker};

async fn factorial(n: u64) -> u64 {
    if n == 0 {
        1
    } else {
        n * Box::pin(factorial(n - 1)).await
    }
}

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(v) => return v,
            Poll::Pending => {}
        }
    }
}

fn main() {
    let result = block_on(factorial(5));
    println!("5! = {}", result);
}
```

> The most bare-bones `block_on` from earlier is attached above (this `factorial` has no `.await` that genuinely waits, so that version suffices).

`Box` and `Pin` each handle a different job here: `Box` lets the state machine store only a fixed-size pointer, while `Pin` makes the `Future` inside safely `poll`able. Writing only `Box::new(factorial(n - 1)).await` isn't enough, because the `Future` returned by an `async fn` isn't guaranteed to implement `Unpin`, and `Box<F>` implements `Future` only when `F: Unpin`. So we use `Box::pin` to get a `Pin<Box<F>>`; as long as `F: Future`, `Pin<Box<F>>` itself also implements `Future` and can be `.await`ed directly.

At this point, we've walked the whole of `async`'s underlying machinery — `Future`, executor, reactor, state machines, `Pin` — from start to finish. From next episode on, we return to Tokio to see what conveniences a truly mature runtime offers for writing `async` code.

12.20.3 Recap

- An `async fn` calling itself directly fails to compile, because the compiler can't determine the state machine type's size.
- A base case only settles whether execution stops at runtime, not the type's size at compile time; it's the same problem as Chapter 5's recursive types — self containing self, infinite

size.
- The fix is wrapping the recursive call in `Box::pin`, so the state machine stores only a fixed-size pointer.

12.21 Back to Tokio

12.21.1 Goal of This Episode

Return from the hand-written runtime to Tokio: revisit `tokio::spawn` and `JoinHandle`, compare Tokio's `block_on` with our hand-written one, and learn the runtime's multithreaded / single-threaded flavors.

12.21.2 Main Text

12.21.2.1 You Already Understand the Underneath

Congratulations on surviving the hardest episodes! We hand-wrote an executor, reactor, `Task`, and `JoinHandle` from scratch, and dissected state machines and `Pin`. Tokio's real implementation is of course far more sophisticated, but looking back at its API now, most of the terms and design trade-offs should feel familiar.

12.21.2.2 `tokio::spawn` and `JoinHandle`

`tokio::spawn` is the `spawn` we hand-wrote: wrap a `Future` into a `Task`, hand it to the runtime's scheduler, and get back a `JoinHandle`:

```
#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        21 * 2
    });
    let result = handle.await.expect("the background task panicked");
    println!("result: {}", result);
}
```

(`.await`ing Tokio's `JoinHandle` returns a `Result`, since the background `Task` might panic — hence the `expect` here.)

12.21.2.3 How Tokio's `block_on` Differs from Ours

Unlike our hand-written version, `tokio::runtime::Runtime::block_on` requires **neither** `Send` nor `'static`. It simply runs the `Future` you give it to completion on the current calling `Thread`, never moving it elsewhere, so `Send` isn't a concern.

There is also an important semantic difference: the `block_on` we hand-wrote from Episode 11 onward waits until **every** `Task` in the ready queue completes before returning. Tokio's `block_on` instead "**returns as soon as the `Future` you passed it completes**," without waiting for other background `Tasks` opened via `tokio::spawn`. Unfinished background `Tasks` stay on the runtime.

The one-line contrast: the hand-written version "finishes the whole batch before moving on"; Tokio "finishes the one I specified, then moves on." So in Tokio, `block_on` returning only means your `Future` finished; `Tasks` you `spawn`ed may still be running. If the runtime then shuts down,

those background Tasks never get to finish.

12.21.2.4 The Most Common Beginner Compile Error: Holding a Non-`Send` Value Across `.await`

`tokio::spawn` requires `Future: Send`, and whether a `Future` is `Send` depends on **what it stores across `.await`s**. Holding a non-`Send` value such as `Rc` across an `.await` makes the whole `Future` non-`Send`, so it can't be spawned:

```
use std::rc::Rc;

async fn some_async() {}

#[tokio::main]
async fn main() {
    tokio::spawn(async {
        let rc = Rc::new(5);
        some_async().await; // rc is held across the .await, and Rc isn't Send
        println!("{}", rc);
    });
}
```

The compiler says `future cannot be sent between threads safely` and points out that `Rc<i32>` is used across an `.await`.

Several fixes:

Use a `Send` substitute. Here, swap `Rc<i32>` for `Arc<i32>`, which is `Send`:

```
use std::sync::Arc;

async fn some_async() {}

#[tokio::main]
async fn main() {
    tokio::spawn(async {
        let arc = Arc::new(5);
        some_async().await;
        println!("{}", arc);
    });
}
```

Dispose of the non-`Send` value before the `.await`. Shrink its scope with `{}` so it's `dropped` before the `.await`, and the state machine never holds it across:

```
use std::rc::Rc;

async fn some_async() {}

#[tokio::main]
async fn main() {
    tokio::spawn(async {
        let n = {
            let rc = Rc::new(5);
            *rc
        }; // rc is dropped at the end of this block — it never crosses the .await
```

```
        some_async().await;
        println!("{}", n);
    });
}
```

12.21.2.5 `#[tokio::main]` flavors

Finally: `#[tokio::main]` defaults to the multithreaded runtime, but you can change it:

```
extern crate tokio;

// single-threaded runtime
#[tokio::main(flavor = "current_thread")]
async fn main() {
    println!("I run on a single thread");
}
```

Or specify the number of worker `Threads`:

```
extern crate tokio;

// multithreaded, with 4 workers
#[tokio::main(flavor = "multi_thread", worker_threads = 4)]
async fn main() {
    println!("I have 4 worker threads");
}
```

The single-threaded runtime's upside is that it pays no cross-`Thread` cost; the downside is no true parallelism.

12.21.3 Recap

- `tokio::spawn` hands a `Future` to the runtime and returns a `JoinHandle` (`.await` yields a `Result`, since the `Task` may panic).
- Unlike our hand-written `block_on`, Tokio's requires neither `Send` nor `'static` and returns as soon as **the specified `Future`** completes instead of waiting for **all** `Tasks`.
- Holding a non-`Send` value such as `Rc` across an `.await` makes the `Future` non-`Send` and unspawnable; fix with `Arc`, or use `{}` to shrink the value's scope so it is dropped before the `.await`.
- `#[tokio::main]` defaults to multithreaded, adjustable via `flavor = "current_thread"` or `worker_threads = N`; either way, `tokio::spawn` still requires its `Future` and output to be `Send + 'static`.

12.22 `spawn_blocking`

12.22.1 Goal of This Episode

Learn the discipline of "don't block the `Thread`," and how to house must-block work properly with `spawn_blocking`.

12.22.2 Main Text

12.22.2.1 An Iron Rule: Don't Block the Thread

async can advance masses of work on a few Threads because everyone **takes turns**. An .await is a **possible** yield point: if the Future being awaited isn't ready and returns Pending, the Task yields the Thread, letting someone else run. If the Future is already ready, execution continues immediately without yielding.

That leads to an iron rule: **a Task must not go long without .awaiting**. If a Task hogs the Thread — maybe doing an expensive computation (seconds of math), maybe calling some **synchronous** blocking function (std::thread::sleep, synchronous file reads, a slow synchronous database call) — it monopolizes that Thread. Until the blocking work finishes, **that Thread cannot poll any other Task**.

A bad example:

```
#[tokio::main]
async fn main() {
    // heavy synchronous computation inside an async task — bad!
    let sum: u64 = (0..2_000_000_000u64).sum(); // no .await anywhere in this stretch
    println!("sum: {}", sum);
}
```

This computation contains no .await from start to finish, so the Future running it cannot yield until the computation is done. If code like this runs in a spawned Task, it occupies one worker Thread for the duration and prevents that Thread from advancing other Tasks, though other worker Threads can continue running.

12.22.2.2 The Fix: spawn_blocking

For this kind of "must block" work, the fix is tokio::task::spawn_blocking. It tosses the work onto a **dedicated blocking Thread pool** (whose Threads are designed to be tied up), returning an awaitable handle:

```
#[tokio::main]
async fn main() {
    let handle = tokio::task::spawn_blocking(|| {
        // heavy computation goes to the dedicated blocking pool
        (0..2_000_000_000u64).sum::<u64>()
    });

    // if the work isn't done yet, awaiting here yields the thread until it is
    let sum = handle.await.expect("the blocking task failed");
    println!("sum: {}", sum);
}
```

The crux: because you wait on the handle with .await, your own Task **yields while the handle is not ready**, and the runtime can use the Thread to advance other Tasks; when the blocking pool finishes the computation, you're woken. The slow computation is quarantined in its dedicated pool, never dragging down the Threads doing async work.

(Incidentally: to "sleep a bit" in async, don't use std::thread::sleep — that blocks the Thread. Use tokio::time::sleep(...).await, which is async and duly yields.)

12.22.2.3 Why Not Just `std::thread::spawn`

You might ask: to push work onto another `Thread`, doesn't the multithreading chapter give us `std::thread::spawn`?

The problem is "how to get the result back." The `JoinHandle` from `std::thread::spawn` requires calling `.join()` for the result — and `.join()` is **blocking**; it isn't `async` and can't be `.awaited`. Call `.join()` inside `async` and you've jammed the `Thread` again, right back at the original problem.

`spawn_blocking`'s value is that it packages up "when the synchronous work finishes in the blocking pool, notify the `.awaiting` `async` `Task` to continue." You don't `.join()` a `std::thread::JoinHandle` yourself, nor wire up a `Waker`; just `.await` the returned handle, and while the result isn't ready your `Task` yields the `Thread`, then gets woken when the result is ready.

12.22.2.4 But Long-lived Background `Thread`s Still Belong to `thread::spawn`

One last point worth making: `spawn_blocking` suits one-off work that **will finish**. If what you want is a **long-lived, independent background `Thread`** (say, a listener spinning an infinite loop for the program's whole lifetime), then `std::thread::spawn` is still the right tool.

Why? Because the blocking pool has limited capacity. Toss an infinite loop into `spawn_blocking` and it **permanently occupies** a slot in the pool, never giving it back — a misuse. Over time the pool fills up, and the short jobs that truly need it can't get in.

12.22.3 Recap

- The iron rule: `Thread`s are yielded only at `.await`, so a `Task` mustn't go long without one — otherwise it hogs the `Thread` and stalls every other `Task` on it.
- Expensive computation and synchronous blocking calls (`std::thread::sleep`, sync I/O, slow sync DB calls) all block the `Thread`.
- `tokio::task::spawn_blocking` sends such work to a dedicated blocking pool and returns an `.await`able handle, letting your `Task` yield the `Thread` while the result isn't ready.
- We avoid `std::thread::spawn` because its `.join()` blocks and can't be `.awaited`; `spawn_blocking` builds the "done → wake the `Task`" bridge for you.
- But long-lived independent background `Thread`s still belong to `std::thread::spawn`; an infinite loop in `spawn_blocking` permanently eats a pool slot — a misuse.

12.23 `join!`

12.23.1 Goal of This Episode

Learn to wait on multiple `Future`s at once within a single `Task` using `join!`, and understand why it's a macro.

12.23.2 Main Text

12.23.2.1 Concurrency Within One Task

In Episode 9 we hand-wrote JoinAll, advancing several Futures together. Tokio provides a ready-made join! that does the same thing:

```
extern crate tokio;

use tokio::time::{sleep, Duration};

async fn fetch_a() -> i32 {
    sleep(Duration::from_secs(1)).await;
    1
}

async fn fetch_b() -> &'static str {
    sleep(Duration::from_secs(1)).await;
    "hello"
}

#[tokio::main]
async fn main() {
    // both Futures wait simultaneously — about one second total — returning a tuple
    let (a, b) = tokio::join!(fetch_a(), fetch_b());
    println!("a = {}, b = {}", a, b);
}
```

join! waits for **all** branches to complete before moving on, handing back each branch's result packed into a tuple. The two fetches above each wait one second, but because they're concurrent, the total is about one second, not two.

12.23.2.2 The Difference Between spawn and join!

Both spawn and join! give you concurrency, but by different means:

- tokio::spawn turns each job into an **independent Task** handed to the runtime, possibly run on different Threads — hence Send + 'static.
- join! polls its branches in turn **within the same Task**; they do **not** become independent Tasks.

Because the branches stay inside the current Task and join! waits for all of them to complete, they never become independent Tasks that can outlive the current scope. That makes join! a good fit for a **fixed number** of concurrent I/O operations that should all complete within the current scope — calling three APIs at once, reading two files at once.

12.23.2.3 join!'s Concurrency Is Not CPU Parallelism

An important limitation to clear up. join!'s branches are **polled in turn** on **the same Task**, which means its concurrency is the "interleaved switching" kind — it **cannot** be CPU parallelism.

The consequence is practical: if one branch goes a long time without .awaiting (doing lengthy computation, or calling a synchronous blocking function), it hogs the Thread — and since everyone takes turns on the same Task, **even the other branches within the same join! go unpolled**. The illusion of concurrency shatters on the spot.

This is exactly last episode's "don't block the `Thread`" iron rule playing out in `join!`. If some branch really has heavy lifting to do, use `spawn_blocking` — don't let it wedge inside the `join!`.

12.23.2.4 Why `join!` Is a Macro

You've probably noticed `join!` is also a macro, not a function. Why must it be, this time?

Because it has to swallow any number of `Futures` of mutually different types, then return a tuple shaped to match. `join!(a, b)` and `join!(a, b, c, d)` both work, each branch's `Future` type entirely its own; the return type changes accordingly to `(A::Output, B::Output)` or `(A::Output, B::Output, C::Output, D::Output)`.

An ordinary Rust function has a fixed number of parameters. We could write separate generic functions for two, three, or four `Futures`, each returning a tuple whose element types match those `Futures`' outputs, but no single function can cover every possible arity. A macro can instead generate, at **compile time**, code with exactly the right tuple shape for each invocation.

The contrast with Episode 9's `JoinAll` sharpens the picture: `JoinAll` handles "**same output type, dynamic count**" — the count settled at runtime, while which concrete `Future` each one is got erased behind `dyn Future<Output = ()>`, the only requirement being that they all output `()`. `join!` is the reverse: "**mixed output types, fixed count**" — the count and each `Future`'s output type locked in as you write the code, so a macro can unroll them at compile time into a tuple that matches exactly.

12.23.3 Recap

- `join!` waits on multiple `Futures` at once **within one `Task`**, returning the results as a tuple once all complete.
- Unlike `spawn`: `join!`'s branches don't become independent `Tasks` — suited to a fixed number of concurrent I/O operations that should all complete within the current scope.
- `join!`'s concurrency isn't CPU parallelism: branches take turns being `polled` on one `Task`, and one stuck branch starves the rest.
- `join!` is a macro because each invocation can take a different number of differently typed `Futures` and produce a correspondingly shaped output tuple; an ordinary function would need a separate version for each arity.
- Against our own `JoinAll` (same output type, dynamic count), `join!` is mixed output types, fixed count.

12.24 `Semaphore` and Backpressure

12.24.1 Goal of This Episode

Learn to cap "how many things happen at once" with a `Semaphore`, and understand the idea of backpressure.

12.24.2 Main Text

12.24.2.1 Capping Simultaneity

Some things you don't want happening "all together without limit." For instance: don't download too many files at once (or your bandwidth chokes), the number of simultaneously open files has a ceiling, and requests to some API must be throttled (or they'll block you).

`tokio::sync::Semaphore` manages exactly this. Its core is a fixed number of **permits**: you set how many exist in total; whoever wants to work must first take one, returning it when done. When permits run out, latecomers wait until someone returns one.

```
extern crate tokio;

use std::sync::Arc;
use tokio::sync::Semaphore;
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    // only 3 permits total, so at most 3 tasks can work at once
    let semaphore = Arc::new(Semaphore::new(3));

    let mut handles = vec![];
    for i in 0..10 {
        let semaphore = Arc::clone(&semaphore);
        handles.push(tokio::spawn(async move {
            // take a permit, .awaiting if none is available
            let _permit = semaphore.acquire().await.expect("the semaphore was closed");
            println!("task {} got a permit, starting work", i);
            sleep(Duration::from_millis(100)).await;
            // _permit leaves scope here, automatically returning the slot
        }));
    }

    for h in handles {
        h.await.expect("a task failed");
    }
}
```

We spawned 10 tasks, but with only 3 permits, at most 3 are working at any moment; the rest queue up dutifully waiting for a permit.

12.24.2.2 Permits Return Themselves via `Drop`

Notice that after taking the permit above, we **never manually returned it** — how did it come back on its own?

Because the permit implements `Drop`. When `_permit` leaves scope, its `Drop` implementation automatically gives the slot back to the `Semaphore`. So as long as the permit leaves scope at the "right moment to finish," the return happens automatically — impossible to forget. That's also why we bound it to a variable with `let _permit = ...` — to keep it **alive until the work ends** before being `dropped`. Writing `let _ = ...` would `drop` it immediately, returning the permit at once — no cap enforced at all.

12.24.2.3 Backpressure

`Semaphore` leads into a more general idea: **backpressure**.

Picture an assembly line: upstream keeps sending items in; downstream processes them slowly. If upstream sends faster than downstream can process, items pile up — a blown-out memory is only a matter of time. Backpressure means: **when downstream can't keep up, there must be a way to make upstream "slow down and wait,"** instead of stuffing without limit.

A `Semaphore` can build that backpressure: permits represent "capacity," and once capacity is full, would-be entrants are held at `acquire().await`, naturally slowing down.

Next episode's **bounded channel** works on the same principle — limited capacity, and when it's full, `send().await` waits, forcing upstream to ease off. So you can understand all the backpressure tools through one lens: "**finite capacity — when full, you wait**."

12.24.3 Recap

- `tokio::sync::Semaphore` expresses capacity as a fixed number of **permits**, capping "how many at once": simultaneous downloads, open files, `Task`s inside some section, etc.
- `acquire().await` takes a permit, waiting if none is free; permits implement `Drop` and return their slot automatically on leaving scope.
- Use `let _permit = ...` so the permit lives until the work finishes; don't write `let _ = ...` (instant `drop`).
- **Backpressure**: make upstream wait when downstream can't keep up, avoiding unbounded pileups; understand `Semaphore` (and next episode's bounded channel) as "finite capacity, wait when full."

12.25 `mpsc`

12.25.1 Goal of This Episode

Learn to pass work between `Task`s with the `async` version of the `mpsc` channel, and understand the bounded channel's backpressure.

12.25.2 Main Text

12.25.2.1 A Work Queue Between `Task`s

In the multithreading chapter we used `std::sync::mpsc` to pass messages between `Thread`s. The `async` world's counterpart is `tokio::sync::mpsc`, the most common queue between `Task`s: one side (the producer) `send`s work in, the other (the consumer) `recv`s it out for processing. It's likewise **multi-producer single-consumer** — many senders allowed, but only one receiver.

```
extern crate tokio;

use tokio::sync::mpsc;
```

```rust
#[tokio::main]
async fn main() {
    // create a bounded channel with capacity 32
    let (tx, mut rx) = mpsc::channel::<i32>(32);

    // producer: spawned off to send 5 jobs
    tokio::spawn(async move {
        for i in 0..5 {
            tx.send(i).await.expect("the receiver was closed");
            println!("sent {}", i);
        }
        // tx drops here; once the remaining messages are received, recv returns None
    });

    // consumer: keep receiving until the channel closes
    while let Some(value) = rx.recv().await {
        println!("received {}", value);
    }
    println!("the channel closed — done");
}
```

`rx.recv().await` returns an `Option`: a message is `Some(value)`; once every sender has been dropped and the channel's leftover messages have all been received, it returns `None`, and the `while let` ends naturally.

12.25.2.2 Bounded Channels and Backpressure

Notice we gave the channel a capacity of 32 — this is a **bounded** channel. That capacity ceiling is precisely last episode's backpressure: when the messages piling up in the channel **fill** all 32 slots (meaning the consumer can't keep up), the producer's `tx.send(value).await` **waits**, resuming only after the consumer clears some space.

That also explains why `send` needs `.await` — because it **may have to wait** (for a free slot). Contrast the synchronous `send` introduced in the multithreading chapter, which never waits (it's unbounded); the `.await` here is backpressure incarnate. Tokio also has `unbounded_channel`, whose `send` needs no `.await` — but then there's no backpressure, so use it with care.

12.25.3 Recap

- `tokio::sync::mpsc` is the most common work queue between `async` Tasks: many senders, one receiver.
- `rx.recv().await` returns an `Option`: `Some` while there are messages, `None` after all senders drop and the leftovers are drained.
- A **bounded channel** has a capacity ceiling; when full, `send().await` waits — that's backpressure, forcing producers to match the consumer's pace.
- `send` requires `.await` exactly because it may wait for a slot; `unbounded_channel` never waits but has no backpressure.

12.26 `oneshot`, `watch`, and `broadcast`

12.26.1 Goal of This Episode

Meet three more kinds of channels and learn to judge which to use.

12.26.2 Main Text

Last episode's `mpsc` was "many senders, one receiver." Tokio has three more channels, each suiting different situations. The most basic distinction is how many **senders** and **receivers** each has.

12.26.2.1 `oneshot`: One Value, Once

`oneshot` is "**one sender, one receiver, one value only**." Perfect for one-time returns of the "compute a result in the background, send it back when done" kind.

```
extern crate tokio;

use tokio::sync::oneshot;

#[tokio::main]
async fn main() {
    let (tx, rx) = oneshot::channel::<i32>();

    tokio::spawn(async move {
        // compute a result and send it back (send is single-use and needs no .await)
        tx.send(42).expect("the receiver disappeared");
    });

    // rx is itself a Future — .await it to get the value
    let result = rx.await.expect("the sender disappeared");
    println!("got the result: {}", result);
}
```

Note that `oneshot`'s receiving end `rx` is itself a `Future`; just `rx.await`.

12.26.2.2 `watch`: Only the "Latest State" Matters

`watch` is "**many senders, many receivers, with only the latest value retained**." Its `Sender` can be cloned, so multiple tasks can update the same channel. It's not a queue delivering every message — it's more like a "bulletin board": senders can update what's on it at any time, and receivers only care about "what does the board say now." Old values missed in between are not made up to you.

It's best for broadcasting state like "what's the current configuration."

```
extern crate tokio;

use tokio::sync::watch;

#[tokio::main]
async fn main() {
    let (tx, mut rx) = watch::channel("starting up");
```

```
    tokio::spawn(async move {
        tx.send("running").expect("no receivers");
        tx.send("finished").expect("no receivers");
    });

    // changed().await waits for an update; borrow() reads the current latest value
    while rx.changed().await.is_ok() {
        println!("latest state: {}", *rx.borrow());
    }
}
```

12.26.2.3 broadcast: Deliver Events to Every Subscriber

broadcast is "**many senders, many receivers, each receiver with its own progress**." Unlike watch, it doesn't give only the latest value — it delivers every message to all currently subscribed receivers. Suits "one event must notify every subscriber."

```
extern crate tokio;

use tokio::sync::broadcast;

#[tokio::main]
async fn main() {
    let (tx, mut rx1) = broadcast::channel::<i32>(16);
    let mut rx2 = tx.subscribe(); // open another receiver

    tx.send(1).expect("no receivers");
    tx.send(2).expect("no receivers");

    // rx1 and rx2 both receive 1 and 2
    println!("rx1 got: {}", rx1.recv().await.expect("receive failed"));
    println!("rx1 got: {}", rx1.recv().await.expect("receive failed"));
    println!("rx2 got: {}", rx2.recv().await.expect("receive failed"));
    println!("rx2 got: {}", rx2.recv().await.expect("receive failed"));
}
```

That said, broadcast isn't an unlimited historical record. The 16 given at creation is the capacity; if some receiver goes too long without receiving and falls behind by more than the capacity, old messages get discarded. Its recv().await then returns Lagged(n), telling you how many you missed:

```
match rx.recv().await {
    Ok(value) => println!("got: {}", value),
    Err(broadcast::error::RecvError::Lagged(n)) => {
        println!("too slow — missed {} messages", n);
    }
    Err(broadcast::error::RecvError::Closed) => {
        println!("all senders closed");
    }
}
```

So, more precisely: broadcast broadcasts messages to all receivers, but each receiver must keep up on its own; fall behind and you get Lagged, not a guarantee of every old message forever.

12.26.3 Recap

- Channels differ in their "number of senders / receivers."
- `oneshot`: one-to-one, a single value once; the receiver is itself a `Future` (`rx.await`) — good for returning results.
- `watch`: many-to-many, latest value only — good for broadcasting current state; use `.changed().await` + `.borrow()`.
- `broadcast`: many-to-many, notifying every subscriber of each event; each receiver keeps its own progress, but falling behind the capacity yields `Lagged`.
- Contrast with last episode's `mpsc` (many-to-one, every message, a queue).

12.27 `AsyncRead` and `AsyncWrite`

12.27.1 Goal of This Episode

Meet the `async` versions of the I/O operations, and make first contact with an `async`-specific concept: **cancellation**.

12.27.2 Main Text

12.27.2.1 Reading and Writing, the `async` Way

In the advanced standard library chapter, we used the synchronous `Read` / `Write` `traits`. The `async` world has corresponding `AsyncRead` / `AsyncWrite` — same idea, except the reads and writes become `.await`able.

One important property up front: the true core methods underlying the `AsyncRead` / `AsyncWrite` `traits` are `poll_read` / `poll_write`. They only promise to "**try to make progress once**"; `poll_read` fills what it read into the buffer, and `poll_write` reports how many bytes this attempt actually wrote. Neither **guarantees** filling your whole buffer in one go, nor writing all the data at once. Say you want 100 bytes: some `poll_read` might fill in only 30 — the rest must be read later.

12.27.2.2 The Convenience Helpers in `AsyncReadExt` / `AsyncWriteExt`

Handling "didn't read enough, didn't finish writing" yourself every time is tedious. So Tokio's extension `traits` `AsyncReadExt` / `AsyncWriteExt` provide many helpers that wrap the loop for you. Internally they too repeatedly drive the underlying `poll_read` / `poll_write`. Two of them:

- `.read_exact(&mut buf)`: keeps reading until `buf` is **completely filled**.
- `.write_all(buf)`: keeps writing until `buf` is **entirely written out**.

```
use tokio::io::{AsyncReadExt, AsyncWriteExt};
use tokio::net::TcpStream;

#[tokio::main]
async fn main() {
    let mut stream = TcpStream::connect("127.0.0.1:8080").await.expect("connect failed");

    // write_all: writes the whole buffer (may call poll_write more than once)
    stream.write_all(b"GET / HTTP/1.0\r\n\r\n").await.expect("write failed");
```

```
    // read_exact: reads 16 bytes (may call poll_read more than once)
    let mut buf = [0u8; 16];
    stream.read_exact(&mut buf).await.expect("read failed");
    println!("read 16 bytes: {:?}", buf);
}
```

12.27.2.3 First Contact with "Cancellation"

Helpers like `read_exact` happen to bring us to a concept that's vital in `async` yet easy to overlook: **cancellation**.

Remember that `Futures` are lazy? They only move when `polled`. Flip that around — if you **stop `polling`** one and just `drop` it, that `async` job has effectively been **called off**; the code after that point will never run. That's `async` cancellation: **`dropping a Future` is cancelling it**.

This is an ability unique to `async`. Ordinary `Threads` can't be stopped this cleanly — there's no safe way to halt a running `Thread` midway from the outside. But an `async` job is just an unfinished `Future`; ignore it, discard it, and it stops.

12.27.2.4 `read_exact` Is Not Cancellation Safe

Convenient as cancellation is, there's a trap. Operations like `read_exact` — "**spanning several advances, accumulating state along the way**" — call for care.

Imagine preparing a 100-byte buffer and handing it to `read_exact(&mut buf)`. Its goal is to return only after filling the whole `buf`. But the underlying `poll_read` might read just 30 bytes the first time, so `read_exact` remembers "30 read so far, 70 to go" and continues `.awaiting`.

Here's the problem: if this `read_exact` gets cancelled midway (`dropped`), the progress it remembered vanishes with it. Continuing the "30 bytes on the first read" scenario: those 30 bytes have already been taken off the socket and written into the front of `buf`; but `read_exact` never returned successfully, so it never handed back the fact that "30 bytes have been read so far." In other words, the "fill 100 bytes" operation stopped mid-road, and the remaining 70 bytes won't complete themselves.

Cancellation can strike at other moments too: earlier, and perhaps 0 bytes had been read; later, and maybe 80; land exactly on 100 and `read_exact` may complete normally. The nuisance is: whenever it's discarded before completing normally, you lose the "where exactly did I get to" progress. For I/O that must be parsed in order, bytes already consumed can't be un-read and retried; without separately saving the progress yourself, safely continuing the read becomes very hard.

We say `read_exact` and `write_all` are **not cancellation safe**: cancelled midway, they leave a mess (some data may already be consumed, yet the whole "fill the buffer" operation never finished). So you should **not** place operations like `read_exact` or `write_all` anywhere they "might get discarded midway."

And where might that be? The classic case is next episode's `select!` — by its very nature, when one branch completes, it `drops` (i.e. cancels) the other unfinished branches. So next episode returns to cancellation safety and how to avoid this pit inside `select!`.

12.27.3 Recap

- `AsyncRead` / `AsyncWrite` are the `async` versions of `Read` / `Write`; the core is `poll_read` / `poll_write`, each attempting one advance: `poll_read` fills the buffer, `poll_write` reports bytes written — neither guarantees a full read or complete write.
- `AsyncReadExt` / `AsyncWriteExt` provide helpers like `read_exact` and `write_all` that wrap the "fill / finish" loop for you.
- **Cancellation**: `Futures` are lazy; `dropping` one (never `polling` again) cancels the `async` job — unique to `async`, impossible with `Threads`.
- Operations like `read_exact` that "span multiple advances and accumulate state" are **not cancellation safe**: cancelled midway, data may be partially consumed with the "fill the buffer" operation unfinished — keep them out of places that may get `dropped` midway.

12.28 `select!`

12.28.1 Goal of This Episode

Learn to use `select!` to wait for "the first of several branches to finish with an output that matches its pattern," and understand its close ties to cancellation.

12.28.2 Main Text

12.28.2.1 Waiting for "Whoever Arrives First"

`join!` waits for "**all** done." `select!` is in a sense its opposite: it waits on several branches at once, and when one completes with an output that matches the pattern on the left, the handler for that branch runs and the whole `select!` ends — **the other unfinished branches get `dropped`**.

12.28.2.2 Basic Syntax

Each `select!` branch looks roughly like:

```
tokio::select! {
    pattern = future => {
        // when future completes, its output is caught by pattern
    }
    _ = other_future => {
        // we don't care about other_future's output
    }
}
```

The `pattern` before the equals sign catches the output of the `Future` after it; variables bound in the pattern are available inside the braces on the right. What goes after the equals sign is just the `Future` to wait on — do **not** add `.await` yourself. `select!` takes care of `polling` these `Futures` simultaneously, waiting for one to finish with an output that matches its pattern.

If you don't need some `Future`'s output, ignore it with `_`, just like an ordinary `match` pattern:

```
tokio::select! {
    value = compute() => {
        println!("computed: {}", value);
    }
```

```
    _ = shutdown.recv() => {
        println!("got the shutdown signal");
    }
}
```

If the output is itself an `Option<T>` or `Result<T, E>`, the most intuitive style is to catch the whole value, then `match` it inside the handler:

```
tokio::select! {
    message = receiver.recv() => {
        match message {
            Some(message) => println!("got a message: {}", message),
            None => println!("the channel closed"),
        }
    }
    _ = shutdown.recv() => {
        println!("preparing to shut down");
    }
}
```

`select!` can itself have a return value — the last expression in the winning branch's braces. Much like `match`: every branch must return the same type.

```
let status = tokio::select! {
    value = compute() => {
        println!("computed: {}", value);
        "done"
    }
    _ = shutdown.recv() => {
        println!("got the shutdown signal");
        "shutdown"
    }
};

println!("status: {}", status);
```

The most classic use of `select!` is **timeout**: `select!` on "the real work" and "a timer" together and see which arrives first.

```
extern crate tokio;

use tokio::time::{sleep, Duration};

async fn do_work() {
    sleep(Duration::from_secs(5)).await; // pretend the work takes five seconds
    println!("work finished");
}

#[tokio::main]
async fn main() {
    tokio::select! {
        _ = do_work() => {
            println!("the work completed fine");
        }
        _ = sleep(Duration::from_secs(1)) => {
```

```
            println!("timeout! the work took too long — not waiting");
        }
    }
}
```

The timer fires at one second, beating the five-second job, so `select!` takes the timer branch, prints "timeout," and **drops the `do_work()` `Future`** — the work is thereby cancelled.

`select!` shines in these situations:

- **Timeout** (the example above).
- **Receiving on multiple channels at once**: whichever channel has a message first gets handled.
- **Waiting for a shutdown signal**: doing normal work while also listening for "time to wrap up," responding to whichever comes first.

12.28.2.3 Watch Out for Cancellation Safety with `select!` in Loops

We just mentioned `drop` — this is exactly last episode's **cancellation**: `dropping` a `Future` cancels it. And `select!`, by design, `drops` all the other branches when one wins. Grasping this keeps later `select!` usage out of the minefield.

`select!` is often placed inside a `loop` and run repeatedly (e.g. a server loop: each round `select!`s on "new work" or "the shutdown signal"). Such code demands special care about last episode's **cancellation safety**.

Recall: operations like `read_exact` that "span multiple advances and accumulate state" are **not cancellation safe** — cancelled midway, some data may already be consumed with the "fill the buffer" operation unfinished. And every round of `select!` may `drop` (cancel) this branch's `Future` because **another** branch finished first. Put a `read_exact` in a `select!` branch inside a `loop`, and it may well be cancelled mid-read, leaving a half-done state that's hard to resume.

Losing branches never run their braces — that's `select!`'s normal behavior and not the problem. The real thing to watch: before being discarded, the losing `Future` may already have produced external effects — bytes read off a socket, part of the data written out.

So the risk isn't "the handler didn't run"; it's "the `Future` got cancelled with half-done work never properly wrapped up." If the operation needs to accumulate progress across steps, keep the progress outside the `select!`, and let the branch wait only on a single safely cancellable small step. Later in this chapter we demonstrate designs following this principle.

12.28.2.4 A Few Practical Extras

`select!` has some further commonly used features:

Branch preconditions: append `, if condition` to a branch. The condition can use variables that already exist before entering the `select!`, such as `accepting_jobs` below, but it cannot use variables that will be bound by the pattern on the left. `job` becomes available inside the handler only after `jobs.recv()` completes and `Some(job)` matches successfully.

```
tokio::select! {
    Some(job) = jobs.recv(), if accepting_jobs => {
        handle(job).await;
```

```
    }
    _ = shutdown.recv() => {
        accepting_jobs = false;
    }
}
```

The relevant steps in one call to `select!` occur in this order:

1. Evaluate every branch's `if` precondition. A branch whose condition is false is disabled for this call to `select!`.
2. Evaluate every `async` expression on the right of an equals sign, including expressions in disabled branches. When a condition is false, the expression is still evaluated to create a `Future`, but that `Future` is not `poll`ed. Here, "evaluated" means creating the `Future`, not running the `async` work inside it; ordinary expressions such as argument calculations performed before creating the `Future` still run.
3. `poll` the `Future`s of the remaining branches.
4. When a `Future` completes, try to match its output against the pattern on the left. If it matches, run the handler and finish the `select!`; if it does not, disable that branch and continue waiting for the others. The first branch to complete is therefore not necessarily the winner; the winner is a branch that completes and whose pattern matches.
5. When all branches are disabled, run `else`; if there is no `else`, `select!` panics.

The `else` branch: for example, if `Some(job) = jobs.recv()` encounters a closed channel, `.recv()` returns `None`, so `Some(job)` fails to match and that branch is disabled. If every other branch is also disabled, the `else` branch runs.

```
tokio::select! {
    Some(job) = jobs.recv(), if accepting_jobs => {
        handle(job).await;
    }
    Some(msg) = messages.recv(), if accepting_messages => {
        handle_message(msg).await;
    }
    else => {
        break; // no branch could run this round
    }
}
```

Fairness and `biased;`: by default, `select!` randomly chooses which branch to `poll` first. This matters mainly when `select!` runs repeatedly — typically in a `loop` — and multiple branches remain ready. Varying the starting branch reduces the risk that one branch wins every round simply because it appears earlier.

Adding `biased;` makes `select!` always `poll` from top to bottom. The branches are still `poll`ed one at a time, but `select!` stops as soon as one returns `Ready` and its output matches the pattern. An always-ready branch near the top can therefore prevent later branches from ever being `poll`ed:

```
loop {
    tokio::select! {
        biased;
```

```
        // If messages are always waiting, this branch is always Ready.
        Some(message) = messages.recv() => {
            handle_message(message).await;
        }
        // This branch may never be polled, even after shutdown arrives.
        _ = shutdown.recv() => {
            break;
        }
    }
}
```

If `messages.recv()` is immediately ready on every iteration, it wins before `shutdown.recv()` is reached. This is starvation. With `biased;`, put a branch that must not be delayed first:

```
loop {
    tokio::select! {
        biased;

        _ = shutdown.recv() => {
            break;
        }
        Some(message) = messages.recv() => {
            handle_message(message).await;
        }
    }
}
```

12.28.3 Recap

- `select!` waits on several branches at once; the first branch to complete with an output matching its pattern runs its handler, and the rest get `dropped` (cancelled).
- Basic syntax is `pattern = future => { ... }`; don't write `.await` on the left side of `=>`; use `_ = future` when the output isn't needed; `select!` can return the winning branch's value.
- `select!` is thus the place in a program that **manufactures the most cancellations**; great for timeouts, multi-channel receives, and shutdown signals.
- Using `select!` in a `loop` demands cancellation-safety care: keep non-cancellation-safe `Futures` like `read_exact` out of branches that may be `dropped`.
- Extras: branch `if` (preconditions), branches disabled on pattern mismatch, `else` (when all branches are disabled), and `biased;` for fixed top-down polling.

12.29 `async` `Mutex`, `RwLock`, and `Notify`

12.29.1 Goal of This Episode

Figure out why you sometimes need Tokio's locks, when the standard library's are fine, and meet the wakeup tool `Notify`.

12.29.2 Main Text

12.29.2.1 Starting from an Exception to Send / Sync

Back to the multithreading chapter's Send / Sync. Everyday types follow a pattern: if a type is Sync (borrowable by many Threads at once), it's usually also Send (movable to another Thread).

But there are rare exceptions: the **guards** of std::sync::Mutex and RwLock (the MutexGuard / RwLockReadGuard / RwLockWriteGuard returned by .lock()) are **Sync but not Send**. Why? Because on some operating systems, a lock **must be unlocked by the same Thread that locked it**; moving the guard to another Thread before dropping it (unlocking) would misbehave. So the standard library simply forbids these guards from being Send.

12.29.2.2 These Exceptions Reach into async

This non-Send property turns into a bewildering compile error in async. Recall Episode 21: a Future holding something non-Send across an .await is itself non-Send, hence un-tokio::spawn-able. And the standard library's guards are exactly non-Send — so **holding a std guard across an .await** gets you hit:

```
use std::sync::{Arc, Mutex};

async fn do_io() {}

#[tokio::main]
async fn main() {
    let data = Arc::new(Mutex::new(0));
    tokio::spawn(async move {
        let mut guard = data.lock().expect("lock failed"); // std's MutexGuard is not Send
        do_io().await; // holding the guard across the .await
        *guard += 1;
    }); // compile error: the future isn't Send and can't be spawned
}
```

This error is really a **helpful warning** — it flags a violation of an important discipline: **a Mutex guards shared mutable state; keep lock scopes as short as possible, and try not to hold a lock while waiting on I/O.** Hold a lock while waiting on I/O, and everyone else stays shut out of the lock the whole time — concurrency can collapse.

So the best fix is usually not "find a way to hold the lock across the .await," but **shortening the lock scope**: finish the changes before the .await and let the guard leave scope:

```
use std::sync::{Arc, Mutex};

async fn do_io() {}

#[tokio::main]
async fn main() {
    let data = Arc::new(Mutex::new(0));
    tokio::spawn(async move {
        {
            let mut guard = data.lock().expect("lock failed");
            *guard += 1;
        } // the guard drops right here — it never crosses the .await
        do_io().await; // no lock in hand while waiting on I/O
```

```
    });
}
```

12.29.2.3 Reach for Tokio's Locks Only When Necessary

But sometimes you truly **need** to hold a lock across an `.await` (say, performing an async operation while holding the lock, with logic that can't be split). Only then switch to `tokio::sync::Mutex` — its guard is `Send` and can safely cross `.await`s:

```
use std::sync::Arc;
use tokio::sync::Mutex; // note: tokio's Mutex

#[tokio::main]
async fn main() {
    let data = Arc::new(Mutex::new(0));
    let d = data.clone();
    tokio::spawn(async move {
        let mut guard = d.lock().await; // note that .lock() takes .await
        *guard += 1; // this guard is Send and may cross .awaits
    });
}
```

But remember: **the standard library's locks are faster than Tokio's** (Tokio's pay extra to be able to cross `.await`s). So default to `std`'s locks with short scopes; deploy Tokio's `Mutex` only when "holding the lock across an `.await`" is unavoidable.

Like the standard library, Tokio also has an `RwLock` separating reads from writes: `.read().await` admits many readers at once, `.write().await` is exclusive to a single writer. The usage principles match Tokio's `Mutex`.

12.29.2.4 `Notify`: Wakeups Without Data

Finally, `tokio::sync::Notify`. It's a **wakeup tool without a payload (no data)** — it lets one `Task` sleep in wait (`.notified().await`) and another give it a poke to wake up (`.notify_one()`), but **transmits no value**.

```
use std::sync::Arc;
use tokio::sync::Notify;

#[tokio::main]
async fn main() {
    let notify = Arc::new(Notify::new());
    let n = notify.clone();

    let handle = tokio::spawn(async move {
        n.notified().await; // sleep awaiting notification
        println!("notified — waking up to work");
    });

    notify.notify_one(); // poke one waiter awake
    handle.await.expect("task panicked");
}
```

`Notify` usually pairs with **shared state you manage yourself under a `Mutex`**: after changing the shared state, give a `notify`, and the awakened `Task` checks the state itself. It is **not a**

queue — multiple calls to `.notify_one()` may merge into one. If no one is waiting, `Notify` stores at most one permit.

12.29.2.5 `Notify` vs `watch`

`Notify` is easily confused with Episode 26's `watch`, but their roles differ:

- `Notify`: **carries no data and stores at most one permit**. It only handles "poking people awake"; what to look at upon waking is yours to manage with a `Mutex` or similar.
- `watch`: **carries the "latest value"**. It stores the latest state itself, and receivers read it directly upon waking.

12.29.3 Recap

- The standard library's `Mutex` / `RwLock` guards are `Sync` but not `Send` (some OSes require unlocking on the locking `Thread`); holding one across an `.await` makes the `Future` non-`Send` and unspawnable.
- That compile error is a useful warning: keep `Mutex` lock scopes short; don't hold a lock while waiting on I/O — usually just shorten the scope (`drop` the guard before the `.await`).
- Use `tokio::sync::Mutex` only when holding a lock across an `.await` is a must (its guard is `Send`; `.lock().await`), but prefer the faster std locks.
- Tokio's `RwLock` splits reads and writes: `.read().await` for many readers, `.write().await` for one writer.
- `Notify` is a data-free wakeup tool that stores at most one permit; multiple calls to `.notify_one()` may merge, while `watch` stores the latest value.

12.30 `Stream`

12.30.1 Goal of This Episode

Meet `Stream` — the `async` version of `Iterator` — and learn how to walk through one.

12.30.2 Main Text

12.30.2.1 `Stream` Is the `async` Version of `Iterator`

Chapter 6's `Iterator` is "a sequence of values, taken one at a time." Its `.next()` is a **synchronous** call that returns the next value (or `None`) when it completes. If fetching the value requires expensive computation or blocking I/O, the call must wait for that work to finish.

`Stream` is its `async` counterpart: still a sequence of values taken one at a time, but you can **wait asynchronously for the next value**, such as the next piece of data arriving over the network. Its `.next()` returns a `Future`, and `.next().await` obtains the next value; while the data isn't ready, it can yield control so the runtime can work on other `Tasks`.

The side-by-side makes it easy to remember:

- `iterator.next()` → a synchronous call that returns `Option<Item>` when it completes.
- `stream.next().await` → waits asynchronously through `.await`, producing `Option<Item>` when it completes.

Both use "None means the end."

These examples use the `tokio-stream` crate (it's not part of Tokio proper), so add the dependency first:

```
[dependencies]
tokio-stream = "0.1"
```

One small thing to watch: the crate name is written `tokio-stream` (hyphen) in `Cargo.toml`, but `tokio_stream` (underscore) in code — a `-` in a crate name always becomes `_` in code.

12.30.2.2 Walking Through a `Stream`

An `Iterator` can be walked with `for`, but a `Stream` can't (`for` has no way to `.await`). The standard way to walk a `Stream` is **`while let Some(x) = stream.next().await`** — take values one by one, stopping at `None`:

```
extern crate tokio;
extern crate tokio_stream;

use tokio_stream::StreamExt;

#[tokio::main]
async fn main() {
    // build the simplest stream from a Vec
    let mut stream = tokio_stream::iter(vec![1, 2, 3]);

    // take values one at a time, until None
    while let Some(value) = stream.next().await {
        println!("got {}", value);
    }
}
```

12.30.2.3 `Stream` Isn't in the Standard Library

One thing deserves special mention: unlike `Future`, `Stream` is **currently not in the standard library**. The `Stream` trait is defined in the `futures-core` crate; `tokio-stream` re-exports it and provides its own `StreamExt`. To use this episode's `next`, `map`, and `filter` methods, import `tokio_stream::StreamExt`:

```
extern crate tokio;
extern crate tokio_stream;

use tokio_stream::StreamExt;

#[tokio::main]
async fn main() {
    // just like Iterator, you can chain tools like map / filter
    let mut stream = tokio_stream::iter(1..=5)
        .map(|x| x * 2)
        .filter(|x| x % 3 == 0);

    while let Some(value) = stream.next().await {
        println!("{}", value);
    }
}
```

You'll notice map, filter, and friends are nearly identical to Chapter 6's Iterator — because Stream really is Iterator's async twin. If you've learned Iterator, Stream is just that plus .await.

In practice, Stream is a great fit for "data that keeps arriving over time" — network connections coming in one by one, database query results row by row, or events fired on a schedule. tokio_stream provides a whole toolkit for working with them.

12.30.3 Recap

- Stream is the async version of Iterator: a sequence of values taken one at a time, using .next().await to wait asynchronously for the next value.
- Side by side: iterator.next() returns an Option synchronously; stream.next().await needs .await to return an Option; both end with None.
- Walk it with **while let Some(x) = stream.next().await** (for doesn't work on a Stream).
- Stream isn't in the standard library; it's defined in futures, and tokio_stream::StreamExt provides next, map, filter, etc. (used almost exactly like Iterator).

12.31 JoinSet and FuturesUnordered

12.31.1 Goal of This Episode

Learn to handle "lots of dynamically generated concurrent work, processed in whatever order it finishes," and understand the trade-off between JoinSet and FuturesUnordered.

12.31.2 Main Text

12.31.2.1 Where join! Falls Short

join! is great, but it has two limitations: the number of branches is **fixed** (you must list them all when writing the program), and it waits for **all** of them to finish.

Yet often your work is "**a large amount, generated dynamically, and whoever finishes first gets processed first**" — crawling a thousand web pages, say. join! can't handle that; you need different tools. There are two routes, and the difference is whether each job becomes an independent Task.

12.31.2.2 Route 1: JoinSet (the Dynamic Version of spawn)

Think of tokio::task::JoinSet as "**the dynamic version of spawn**." You spawn any number of jobs into it, each an **independent Task**. On a multi-thread runtime, Tokio can schedule them on different worker Threads, allowing them to run in parallel (and, like spawn, they need Send + 'static). Then collect the finished results one by one with join_next().await — **whoever finishes first is received first**:

```
use tokio::task::JoinSet;
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let mut set = JoinSet::new();
```

```
    // dynamically spawn five jobs, deliberately with different delays
    for i in 0..5 {
        set.spawn(async move {
            sleep(Duration::from_millis(100 * (5 - i))).await;
            i
        });
    }

    // received in completion order (not spawn order)
    while let Some(result) = set.join_next().await {
        let value = result.expect("task panicked or was cancelled");
        println!("done: {}", value);
    }
}
```

`join_next()` returns `Option<Result<T, JoinError>>`:

- `None`: no `Task`s left; you've collected them all.
- `Some(Ok(value))`: a `Task` finished successfully.
- `Some(Err(...))`: that `Task` panicked or was cancelled (so handle this `Err`).

`JoinSet` also supports `.abort_all()` to cancel all the work at once, and when a `JoinSet` is dropped it **automatically cancels** every unfinished `Task` inside — very convenient for graceful shutdown (next episode uses this).

12.31.2.3 Route 2: `FuturesUnordered` (the Dynamic Version of `join!`)

`futures::stream::FuturesUnordered` is "**the dynamic version of `join!`**." It advances a pile of `Future`s in turn **within a single `Task`** — it does **not** make them independent `Task`s and does **not necessarily** cross `Thread`s. Both the cost and the benefit flow from that:

- Since it doesn't `spawn` them as independent `Task`s, `FuturesUnordered` itself **doesn't require those `Future`s to be `Send + 'static`** — it can hold `Future`s that borrow local variables (`JoinSet` can't, because it has to `spawn`).
- But since everyone takes turns on the same `Task`, if one `Future`'s `poll` blocks or runs for too long, `poll`ing the others is delayed (that "don't block the `Thread`" iron rule again).

`FuturesUnordered` is defined in the `futures` `crate`, so add the dependency first (last episode's `tokio-stream` is used here too):

```
[dependencies]
futures = "0.3"
tokio-stream = "0.1"
```

`FuturesUnordered` is itself really just a `Stream` — it tracks wake-ups and `poll`s only newly added or woken `Future`s; it doesn't `spawn` them as `Task`s. So it **doesn't depend on a particular runtime**, a big advantage over `JoinSet` (whose `spawn` is tied to the Tokio runtime). Walk it the `Stream` way:

```
extern crate futures;
extern crate tokio;
extern crate tokio_stream;

use futures::stream::FuturesUnordered;
```

```rust
use tokio_stream::StreamExt;

#[tokio::main]
async fn main() {
    let mut futures = FuturesUnordered::new();

    // dynamically push in a pile of Futures (they don't become independent Tasks)
    for i in 0..5 {
        futures.push(async move { i * 2 });
    }

    // it's a Stream — results pop out in completion order
    while let Some(value) = futures.next().await {
        println!("done: {}", value);
    }
}
```

12.31.2.4 How to Choose

Both produce results in completion order, and both suit crawlers, batch requests, and the like. The differences:

- Want **independent `Task`s that can run in parallel, with greater scheduling isolation** → use **`JoinSet`** (but it needs `Send + 'static` and is tied to Tokio).
- Want to **borrow local variables in place, keep jobs lightweight, avoid depending on a particular runtime** → use **`FuturesUnordered`** (multiplexed within one `Task`, no `Send` needed, but one `Future`'s blocking or long-running `poll` delays `poll`ing the others).

Next episode, we assemble the tools learned so far — `select!`, channels, `JoinSet` — into a complete graceful shutdown flow.

12.31.3 Recap

- For "large, dynamic, first-done-first-served" work, `join!` isn't enough; use `JoinSet` or `FuturesUnordered`.
- **`JoinSet`** (dynamic `spawn`): each job is an independent `Task`, can run in parallel on a multi-thread runtime, needs `Send + 'static`, tied to Tokio; `join_next()` returns `Option<Result<T, JoinError>>`, supports `.abort_all()`, and cancels whatever is left on `drop`.
- **`FuturesUnordered`** (dynamic `join!`): multiplexes within one `Task`, doesn't spawn independent `Task`s, and doesn't itself require its `Future`s to be `Send + 'static` (so they can borrow local variables when the surrounding context allows), but one `Future`'s blocking or long-running `poll` delays polling the others; it's itself a runtime-agnostic `Stream`.
- Independent `Task`s, possible parallel execution, and greater scheduling isolation → `JoinSet`; in-place borrowing, lightweight work, runtime-agnostic → `FuturesUnordered`.

12.32 Graceful Shutdown

12.32.1 Goal of This Episode

Assemble the tools from earlier episodes into a complete graceful shutdown flow.

12.32.2 Main Text

12.32.2.1 What Is Graceful Shutdown

When a server needs to stop, the crudest approach is to just kill it — but then work in progress is cut off mid-flight, possibly leaving corrupted data and unanswered requests. **graceful shutdown** is the politer way: on receiving a stop request, don't hard-cut — instead, "**tell everyone to wrap up → wait for work in hand to finish; on timeout, request cancellation of the remaining work**."

Break it into three ingredients:

1. **Signal source**: how to know "it's time to stop."
2. **Broadcasting shutdown**: how to tell every worker "we're wrapping up."
3. **Waiting for the drain**: how to wait for all workers to finish up.

We'll match each one to a tool we've already learned.

12.32.2.2 Assembling the Three Ingredients

- The **signal source** is `tokio::signal::ctrl_c()` — it's a `Future`; `.awaiting` it waits until the user presses Ctrl-C (in practice you'd also listen for SIGTERM).
- **Broadcasting shutdown** uses Episode 26's `watch` as a shutdown flag: one-to-many, and workers who subscribe late can still read the current state.
- **Waiting for the drain** uses Episode 31's `JoinSet` — `join_next()` until it's empty.

Each worker internally uses `select!` to wait for "the next job" and "the shutdown signal" **at the same time**. If a job arrives first, leave the `select!` and process it; if shutdown arrives first, stop taking new jobs:

```
use std::time::Duration;
use tokio::sync::watch;
use tokio::task::JoinSet;
use tokio::time::{sleep, timeout};

async fn wait_next_job(id: u32, next_job: &mut u32) -> u32 {
    // here sleep stands in for "waiting for the next job to come in".
    // this wait may be cancelled by shutdown; actual processing happens outside the select!.
    sleep(Duration::from_millis(500)).await;
    let job = *next_job;
    *next_job += 1;
    println!("worker {} got job {}", id, job);
    job
}

async fn process_job(id: u32, job: u32) {
    // this stands in for "actually processing the job".
    // it's deliberately outside the select!, so shutdown can't cancel it midway.
    sleep(Duration::from_millis(300)).await;
    println!("worker {} finished job {}", id, job);
}

async fn worker(id: u32, mut shutdown: watch::Receiver<bool>) {
    let mut next_job = 0;
```

```
    loop {
        let job = tokio::select! {
            // waiting for the next job: this can be cancelled by shutdown
            job = wait_next_job(id, &mut next_job) => job,
            // the wrap-up signal
            _ = shutdown.changed() => {
                println!("worker {} got the shutdown signal, exiting", id);
                break;
            }
        };

        // process outside select!, so shutdown cannot drop it midway
        process_job(id, job).await;
    }
}

async fn drain_workers(workers: &mut JoinSet<()>) {
    while let Some(result) = workers.join_next().await {
        if let Err(error) = result {
            eprintln!("worker ended abnormally: {}", error);
        }
    }
}

#[tokio::main]
async fn main() {
    // the watch flag used to broadcast shutdown
    let (shutdown_tx, shutdown_rx) = watch::channel(false);

    // manage all workers with a JoinSet
    let mut workers = JoinSet::new();
    for id in 0..3 {
        workers.spawn(worker(id, shutdown_rx.clone()));
    }

    // 1. wait for the signal
    tokio::signal::ctrl_c().await.expect("failed to listen for Ctrl-C");
    println!("got Ctrl-C, starting graceful shutdown");

    // 2. broadcast the wrap-up
    shutdown_tx.send(true).expect("no worker is listening");

    // 3. wait for all workers to drain, with a 5-second deadline
    match timeout(Duration::from_secs(5), drain_workers(&mut workers)).await {
        Ok(()) => println!("all workers have ended"),
        Err(_) => {
            println!("drain timed out; requesting cancellation of the remaining workers");
            workers.abort_all();
        }
    }
}
```

`timeout(Duration::from_secs(5), future)` sets a five-second deadline for waiting on this `future`.

It is itself a `Future`. When the wait completes, `.await` yields `Ok(the inner output)`; if the wait times out, it yields `Err(_)`. In this example, the inner `future` is:

```
drain_workers(&mut workers)
```

That is, "check each worker's completion result until the `JoinSet` is empty." `drain_workers` prints any `JoinError` it receives and keeps waiting for the other workers, so `Ok(())` only means the wait completed, not that every worker ended normally. On timeout, we call `abort_all()` to request cancellation of the remaining `Tasks`.

12.32.2.3 The Cancellation Safety Design Point

Here's a key design choice echoing Episodes 27 and 28: **place the `select!` deliberately**. In the worker above, the `select!` waits on "the next job" and "shutdown"; once a job is actually in hand, we leave the `select!` and only then call `process_job`. So when shutdown wins, what gets `dropped` (cancelled) is the resumable wait for the next job — not work that has already started.

If instead you put the real processing inside a branch that can lose to shutdown, operations that aren't safely cancellable — like `read_exact` — could be cut off midway, and the data lost with them. This is the cancellation safety we emphasized earlier, applied concretely to shutdown.

12.32.2.4 Set a Deadline for Wrapping Up

These five seconds are the waiting period for workers to finish the work in hand on their own. On timeout, we request cancellation of the remaining `Tasks`; once cancellation is requested, in-flight work can be interrupted even though `process_job` is outside the `select!`.

This is not a guaranteed time limit for the whole program to exit. Tokio needs `Tasks` to yield control so it can handle scheduling and cancellation; a loop or blocking call that never yields cannot be interrupted immediately by `timeout` or `abort_all()`. A `spawn_blocking` `Task` that has already started cannot be stopped by `abort_all()` either. Returning from `abort_all()` only means cancellation has been requested.

12.32.2.5 A Better-fitting Tool: `CancellationToken`

Using `watch` as a shutdown flag works, but it feels a bit like "borrowing" a state-broadcast tool to serve as a switch. `tokio-util` provides a tool designed for "cancellation" from the ground up — `CancellationToken`, with semantics that fit better. `tokio-util` isn't part of Tokio proper, so add the dependency first:

```
[dependencies]
tokio-util = "0.7"
```

(As with `tokio-stream` in Episode 30, the `-` in the `crate` name becomes `_` in code: `use tokio_util::...`.)

Swapping it in for the `watch` above:

```
use std::time::Duration;
use tokio::task::JoinSet;
use tokio::time::{sleep, timeout};
use tokio_util::sync::CancellationToken;
```

```rust
async fn wait_next_job(id: u32, next_job: &mut u32) -> u32 {
    sleep(Duration::from_millis(500)).await;
    let job = *next_job;
    *next_job += 1;
    println!("worker {} got job {}", id, job);
    job
}

async fn process_job(id: u32, job: u32) {
    sleep(Duration::from_millis(300)).await;
    println!("worker {} finished job {}", id, job);
}

async fn worker(id: u32, token: CancellationToken) {
    let mut next_job = 0;

    loop {
        let job = tokio::select! {
            job = wait_next_job(id, &mut next_job) => job,
            _ = token.cancelled() => { // wait directly on "being cancelled"
                println!("worker {} got cancelled, exiting", id);
                break;
            }
        };

        process_job(id, job).await;
    }
}

async fn drain_workers(workers: &mut JoinSet<()>) {
    while let Some(result) = workers.join_next().await {
        if let Err(error) = result {
            eprintln!("worker ended abnormally: {}", error);
        }
    }
}

#[tokio::main]
async fn main() {
    let token = CancellationToken::new();

    let mut workers = JoinSet::new();
    for id in 0..3 {
        workers.spawn(worker(id, token.clone())); // each worker gets a clone
    }

    tokio::signal::ctrl_c().await.expect("failed to listen for Ctrl-C");
    token.cancel(); // one command, everyone cancelled

    match timeout(Duration::from_secs(5), drain_workers(&mut workers)).await {
        Ok(()) => println!("all workers have ended"),
        Err(_) => {
            println!("drain timed out; requesting cancellation of the remaining workers");
            workers.abort_all();
```

```
        }
    }
}
```

`token.cancelled()` is a `Future` that waits for "being cancelled"; one call to `token.cancel()` wakes every worker holding a `clone`. It reads as what it is — cancellation — and fits the need better than borrowing `watch` as a switch.

12.32.3 Recap

- Graceful shutdown: signal the wrap-up and wait for work in hand to finish; on timeout, request cancellation of the remaining work.
- Three ingredients: signal source (`tokio::signal::ctrl_c()`), broadcasting shutdown (a `watch` flag), waiting for the drain (check each result from `join_next()` until the `JoinSet` is empty).
- `select!` is a good fit for waiting on "the next job" and "shutdown" at once; if the real work can't be safely cancelled, use `select!` only to obtain the job, then leave the `select!` to process it, so shutdown can't `drop` in-flight work midway (cancellation safety).
- Use `timeout` to set the deadline for workers to wrap up on their own; on timeout, request cancellation with `abort_all()`.
- The better-fitting tool is `tokio_util`'s `CancellationToken`: `token.cancel()` gives the order and every `token.cancelled()` wakes up — semantically a better match than borrowing `watch`.

12.33 Testing async Code

12.33.1 Goal of This Episode

Learn to write `async` tests with `#[tokio::test]`, and use virtual time to make delay-related tests fast and reliable.

12.33.2 Main Text

12.33.2.1 `#[tokio::test]`

Chapter 7 taught testing with `#[test]` and `cargo test`. But `#[test]` marks an ordinary function, which can't `.await`. To test `async` code, Tokio provides `#[tokio::test]` — it automatically wraps your test function in a runtime, so you don't `block_on` yourself:

```
async fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[tokio::test]
async fn test_add() {
    let result = add(2, 3).await;
    assert_eq!(result, 5);
}
```

That's all there is to it. `#[tokio::test]` equals "`#[test]` + automatic runtime setup + `async`

allowed." Everything else matches Chapter 7: put tests in `#[cfg(test)] mod tests`, run them with `cargo test`, check results with macros like `assert_eq!`.

12.33.2.2 What About Time-dependent Tests

async programs constantly involve time — timeouts, delays, scheduled retries. Tested literally, a "times out after 5 seconds" behavior means the test really waits 5 seconds — slow and annoying.

Tokio's answer is **virtual time**: time in the test is advanced **manually by you**, with no real waiting. Two key functions:

- `tokio::time::pause()`: "pause" time; from then on it doesn't flow by itself.
- `tokio::time::advance(duration)`: manually fast-forward time by a stretch.

```rust
use tokio::time::{self, Duration};

#[tokio::test]
async fn test_with_virtual_time() {
    time::pause(); // pause time

    let start = time::Instant::now();

    // push virtual time forward 10 seconds — instant, no real waiting
    time::advance(Duration::from_secs(10)).await;

    assert_eq!(start.elapsed(), Duration::from_secs(10));
}
```

This test finishes **instantly**, even though logically "10 seconds passed." Time is virtual, and `advance` just jumps over it. If you want time paused from the very start, write `#[tokio::test(start_paused = true)]` and skip the manual `pause()` call.

With virtual time, any test involving timeouts, delays, or retry intervals becomes **deterministic (same result every run)** and fast — you fully control how time moves, no longer at the mercy of the real clock.

(Small reminder: the virtual-time tools `pause` / `advance` require Tokio's `test-util` feature — just add `"test-util"` to Tokio's features in `Cargo.toml`.)

12.33.3 Recap

- `#[tokio::test]` wraps the test function in a runtime and allows `.await` — "`#[test]` + runtime + `async`"; everything else works like Chapter 7's `cargo test`.
- Don't use real time in time-dependent tests (slow and flaky); use Tokio's virtual time.
- `tokio::time::pause()` stops time, `tokio::time::advance(duration)` fast-forwards manually, making timeout/delay tests instant and deterministic.
- `#[tokio::test(start_paused = true)]` pauses time from the start; the virtual-time tools need Tokio's `test-util` feature.

12.34 Runtimes Other Than Tokio

12.34.1 Goal of This Episode

Get to know `async` runtimes besides Tokio, and learn to tell which parts of your code are tied to a specific runtime and which aren't.

12.34.2 Main Text

12.34.2.1 The Standard Library Only Defines the Language-level Abstractions

This chapter took great pains to hand-write a runtime from scratch. By now it should be clear: the standard library defines only the **language-level abstractions** — the `Future` `trait`, `Poll`, `Context`, `Waker`, `Pin`. But "how a `Future` actually gets run" — how the executor schedules, how the reactor watches I/O, how timers are implemented — the standard library stays entirely out of, leaving it to third-party runtimes to do as they please. The things we hand-wrote earlier (the executor, reactor, timer, the design of `Task`) are exactly the parts a runtime should contain.

12.34.2.2 Not Just Tokio

Tokio is currently the most mainstream runtime, but not the only choice. Since the standard library doesn't dictate how a runtime is written, the community has grown several runtimes, each with its own character:

- **Tokio**: the general-purpose runtime with the most complete features and the largest ecosystem — multithreaded, has everything. It's what the second half of this chapter used.
- **smol**: a runtime on the lightweight, minimal path — small core, easy to understand.
- **monoio / glommio**: specialized runtimes on the **thread-per-core** path, often paired with Linux's io_uring, built for extreme I/O performance.
- **Embassy**: a runtime for **embedded** devices, able to run `async` on microcontrollers with no operating system and no standard library.

These runtimes can differ in every dimension: how many `Threads`, how scheduling works, how I/O is done, how timers are implemented, the details and restrictions of `spawn`. Which to pick depends on your situation — for ordinary network services Tokio is the least hassle; for embedded you'd use Embassy.

12.34.2.3 Runtime-agnostic vs Runtime-specific

With this many runtimes around, it's worth keeping one question in mind as you write code: is this piece of code **tied** to a specific runtime or not?

- **The runtime-agnostic parts (not tied)**: pure `Future` composition logic. For example your own `impl Futures`, chains of `async` / `.await`, combinations via `join!` / `select!`, `FuturesUnordered` — these depend only on the standard library's `Future` abstraction and usually work unchanged on another runtime.
- **The runtime-specific parts (tied)**: the things that actually touch the outside world or the scheduler. For example `tokio::net::TcpStream` (I/O), `tokio::time::sleep` (timers), `tokio::spawn` (scheduling) — these come from Tokio, and switching runtimes means swapping in that runtime's equivalents.

In practice there's no need to hamstring yourself for "runtime neutrality" — most projects pick Tokio and use it all the way. But knowing where the line sits helps when you "want to switch runtimes" or "want to write a library for others without locking them into a runtime": you'll know exactly which code can stay untouched and which must be swapped.

12.34.3 Recap

- Rust's standard library defines only the **language-level abstractions** like `Future`; there's no built-in runtime — executor, reactor, timers, I/O, and `Task` design all come from the runtime (exactly the parts we hand-wrote).
- Tokio is the mainstream general-purpose runtime; there's also lightweight smol, thread-per-core monoio / glommio, embedded Embassy, and more — designs can differ in every dimension.
- As you write code, note: pure `Future` composition logic (custom `Futures`, `join!`, `select!`, `FuturesUnordered`) is mostly **runtime-agnostic**; I/O, timers, and `spawn` are **runtime-specific** and must be swapped when switching runtimes.

Congratulations on finishing the async chapter! 🎉 This chapter started from the first `async fn` and took everything apart — `Future`, `poll`, `Waker`, executor, reactor, `Pin` — then returned to Tokio's `spawn`, I/O, channels, `select!`, graceful shutdown, and testing. Having come this far, you've seen the complete skeleton of "how lazy `Future`s get driven forward by a runtime" behind `async`. From now on, when using Tokio or any other runtime, you won't just be looking at async APIs — you'll also know roughly what those APIs are arranging on your behalf.

Appendix II

Here we collect advanced language features that draw on several earlier chapters at once, yet never fit neatly into any of the main chapters. Every episode assumes you have finished the rest of Part II.

13.1 `'a: 'b`

13.1.1 Goal of This Episode

Understand what `'a: 'b` actually solves: when a generic API juggles long-lived data and short-lived operations at once, how to tell the compiler explicitly which lifetime has to live longer.

> This episode supplements **Chapter 5's lifetimes**.

13.1.2 Main Text

Chapter 5 showed this kind of bound:

```
T: 'a
```

It says every reference inside `T` must outlive `'a`. Both sides of the bound can be lifetimes too:

```
'a: 'b
```

This reads "`'a` **outlives** `'b`," meaning `'a` lasts at least as long as `'b`.

The real purpose of this notation isn't to work out which of two local variables goes out of scope first — it's to **hand generic code a guarantee it can use**. If `'a` and `'b` are two unrelated lifetime parameters, a function or type can't assume on its own which is longer; once `'a: 'b` is in effect, the code can safely take data tied to `'a` and place it somewhere that only needs to last for `'b`.

13.1.2.1 A Practical Problem: A Long-lived Setting as a Short-lived Fallback

Suppose a service has long-lived configuration but also lets each request name a language temporarily:

```
struct Config {
    default_language: String,
}
```

We want `language_for_request` to return the request's language when there is one, and otherwise borrow the default from the config:

```
struct Config {
    default_language: String,
}
```

```
fn language_for_request<'config, 'request>(
    config: &'config Config,
    requested: Option<&'request str>,
) -> &'request str {
    match requested {
        Some(language) => language,
        None => &config.default_language,
    }
}
```

This doesn't compile. The `Some` branch returns an `&'request str`, but the `None` branch returns an `&'config str` borrowed from `config`. The signature declares two lifetimes and says nothing about how their lengths relate.

As far as the compiler knows, the caller might hand over a very short-lived `config` while demanding the function return an `&'request str` that lives longer. So it can't simply treat an `&'config str` as an `&'request str`.

This is exactly what `'config: 'request` is for:

```
struct Config {
    default_language: String,
}

fn language_for_request<'config, 'request>(
    config: &'config Config,
    requested: Option<&'request str>,
) -> &'request str
where
    'config: 'request,
{
    match requested {
        Some(language) => language,
        None => &config.default_language,
    }
}

fn main() {
    let config = Config {
        default_language: String::from("zh-TW"),
    };

    {
        let request_language = String::from("en");
        let selected = language_for_request(&config, Some(&request_language));
        println!("Specified by the request: {selected}");
    }

    {
        let selected = language_for_request(&config, None);
        println!("Using the default: {selected}");
    }
}
```

`'config: 'request` guarantees the configuration data stays valid throughout `'request`. The `None` branch can therefore **shorten** its `&'config str` into an `&'request str`, letting both branches return the same type.

That is the most direct practical use of lifetime-outlives-lifetime:

- The input data comes from two lifetimes playing different roles.
- The API picks one of them as the lifetime of the return value or of the result of an operation.
- If the other piece of data may also become a source for that result, it must be guaranteed to outlive that lifetime.

13.1.2.2 Which APIs Have This Relationship?

Don't bother memorizing a pile of API names yet. This relationship usually just describes the following situation:

> There is a longer-lived piece of data; we borrow it temporarily to produce a result or a tool that will only be used for a short while.

The `Config` above is exactly that:

- `config.default_language` lives longer.
- `language_for_request` only needs to return an `&str` valid for the duration of this request.
- Hence the configuration's lifetime must outlive the request's lifetime.

The `DebugStruct` coming up is the same thing:

- The `Formatter` and the output target it writes to live longer.
- `DebugStruct` only borrows the `Formatter` temporarily, to help assemble this one piece of output.
- Hence the lifetime of the data inside the `Formatter` must cover the lifetime of the reference `DebugStruct` holds.

In ordinary code the compiler can often infer this relationship on its own, so you won't see `'long: 'short` scattered everywhere. It shows up mostly in library type signatures. For now, just remember: **when you temporarily borrow long-lived data, that data can't expire before the reference you obtained from it.**

13.1.2.3 A Real Case: `DebugStruct<'a, 'b>`

The standard library's `DebugStruct`, which helps implement `Debug`, is defined roughly like this:

```
pub struct DebugStruct<'a, 'b: 'a> {
    fmt: &'a mut Formatter<'b>,
    // Other fields omitted
}
```

`'b: 'a` is the same as writing the `where` clause below:

```
pub struct DebugStruct<'a, 'b>
where
    'b: 'a,
{
    fmt: &'a mut Formatter<'b>,
```

```
}
```

The two lifetimes here each have a clear role:

- `'a` is the period during which `DebugStruct` temporarily borrows the `Formatter`.
- `'b` is the lifetime of the output target inside the `Formatter`.

During `'a`, `DebugStruct` writes to that output target through `Formatter<'b>`, so the target mustn't expire while the builder still exists. `'b: 'a` is precisely that requirement: the data borrowed inside the `Formatter` must outlive the reference held by the outer builder.

Users normally never have to write these two lifetimes themselves:

```
use std::fmt;

struct Account {
    id: u64,
    token: String,
}

impl fmt::Debug for Account {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        formatter
            .debug_struct("Account")
            .field("id", &self.id)
            .field("token", &"<hidden>")
            .finish()
    }
}

fn main() {
    let account = Account {
        id: 7,
        token: String::from("secret-token"),
    };

    println!("{account:?}");
}
```

This code really does use `DebugStruct`, but `Formatter::debug_struct` has already baked the correct lifetime relationship into the API — the caller only has to satisfy it. You typically encounter or write `'b: 'a` directly only when reading standard library documentation, building a generic wrapper around this sort of builder, or designing your own type containing several layers of references.

13.1.2.4 What's the Benefit of Separating the Two Lifetimes?

Why doesn't `DebugStruct` just use a single lifetime? We can write a simplified pair to compare.

Let `FormatterLike` stand in for `Formatter`. It borrows the `String` that actually holds the output text:

```
struct FormatterLike<'buffer> {
    output: &'buffer mut String,
}
```

If the builder used only one `'a`, the outer reference to `FormatterLike` and the inner reference to the `String` would be forced onto the same lifetime:

```
struct FormatterLike<'buffer> {
    output: &'buffer mut String,
}

struct BadBuilder<'a> {
    formatter: &'a mut FormatterLike<'a>,
}

impl BadBuilder<'_> {
    fn field(&mut self, text: &str) {
        self.formatter.output.push_str(text);
    }
}

fn write_two_parts<'buffer>(formatter: &mut FormatterLike<'buffer>) {
    let mut first = BadBuilder { formatter };
    first.field("first part");

    formatter.output.push(' ');

    let mut second = BadBuilder { formatter };
    second.field("second part");
}
```

The `String` inside `FormatterLike<'buffer>` may be borrowed for a long time, but `BadBuilder` really only needs to borrow `formatter` briefly. Writing both as `'a` forces the compiler to tie the outer mutable reference and the inner data to the same length, so `formatter` remains unusable even after the first builder is done.

Splitting them into two lifetimes describes the requirement precisely:

```
struct FormatterLike<'buffer> {
    output: &'buffer mut String,
}

struct Builder<'borrow, 'buffer: 'borrow> {
    formatter: &'borrow mut FormatterLike<'buffer>,
}

impl Builder<'_, '_> {
    fn field(&mut self, text: &str) {
        self.formatter.output.push_str(text);
    }
}

fn write_two_parts<'buffer>(formatter: &mut FormatterLike<'buffer>) {
    let mut first = Builder { formatter };
    first.field("first part");

    formatter.output.push(' ');

    let mut second = Builder { formatter };
```

```
    second.field("second part");
}

fn main() {
    let mut output = String::new();
    let mut formatter = FormatterLike {
        output: &mut output,
    };
    write_two_parts(&mut formatter);

    println!("{output}");
}
```

Here `'buffer` is how long the inner `String` is borrowed, while `'borrow` is how long one particular builder temporarily borrows the `FormatterLike`. `'buffer: 'borrow` only demands that the inner output outlive this short-lived reference; it doesn't extend the short-lived reference in return.

The variables and scopes are identical in both versions — the only difference is the builder's lifetime design. In the correct version, once `first` is used for the last time the compiler can end the `formatter` reference it holds; after that you can write to `formatter` directly and build a second `Builder`. Using two lifetimes in `DebugStruct<'a, 'b: 'a>` buys exactly this flexibility: **the underlying output can live a long time, while each formatting builder borrows the `Formatter` only briefly, when it needs to.**

13.1.2.5 It Doesn't Work in Reverse

An outlives bound only lets you use a longer guarantee as a shorter one; it can't make short-lived data live longer out of thin air:

```
fn extend<'short, 'long>(short: &'short str, _long: &'long str) -> &'long str
where
    'long: 'short,
{
    short
}
```

`'long: 'short` says `'long` is longer than `'short`, yet the function wants to return a reference guaranteed only to live for `'short` as an `&'long str`. The bound provides no such guarantee, so this doesn't compile.

13.1.3 Recap

- Two lifetime parameters don't necessarily have any fixed length relationship; generic code can only use the guarantees the signature provides.
- `'long: 'short` provides the guarantee "`'long` lasts at least as long as `'short`."
- With that guarantee, an `&'long T` can be shortened to an `&'short T` and used as the return value of, or an internal reference in, a short-lived operation.
- A long-lived setting acting as the fallback for a short-lived request is a direct application of this bound.
- Ordinary call sites mostly rely on inference; you meet `'a: 'b` head-on mainly when designing or reading generic APIs that keep several lifetime roles apart.

- An outlives bound can only prove an existing relationship — it can't extend anything's lifetime.

13.2 HRTB

13.2.1 Goal of This Episode

Understand the `for<'a>` in a higher-ranked `trait` bound (HRTB), learn to tell "handles one particular lifetime" apart from "handles any lifetime," and read how the `Fn` trio elides the lifetimes of its inputs and return values.

This episode supplements **Chapter 5's lifetimes** and **Chapter 6's closures**.

13.2.2 Concept

Suppose we have two sets of product data and want to pick one featured product from each using the same rule:

```
fn select_from_both<'first, 'second, T, F>(
    first: &'first [T],
    second: &'second [T],
    choose: F,
) -> (Option<&'first T>, Option<&'second T>)
where
    F: Fn(&[T]) -> Option<&T>,
{
    let from_first = choose(first);
    let from_second = choose(second);
    (from_first, from_second)
}
```

`first` and `second` are two different parameters, with lifetimes `'first` and `'second` respectively. Inside the function the same `choose` is called twice:

- The first call receives an `&'first [T]` and returns an `Option<&'first T>`.
- The second receives an `&'second [T]` and returns an `Option<&'second T>`.

The return type keeps both lifetimes: the first result follows `first`, the second follows `second`. So we can't simply shrink both inputs down to one shared lifetime.

The lifetimes in the bound are elided, but the full idea is this: whatever lifetime the slice `choose` receives has, it returns an element borrowed from that slice. Spelled out in full, that's an HRTB:

```
F: for<'a> Fn(&'a [T]) -> Option<&'a T>
```

On the first call `'a` can be `'first`; on the second it can switch to `'second`.

13.2.2.1 The `Fn` Trio Has Lifetime Elision Too

Chapter 5 showed that functions can elide common lifetimes:

```
fn first<T>(values: &[T]) -> Option<&T>
```

Because there's only one input lifetime, the compiler knows the `&T` inside the `Option` must be borrowed from that `&[T]`. Written out in full:

```
fn first<'a, T>(values: &'a [T]) -> Option<&'a T>
```

The same lifetime elision applies to the parameters and return values of `Fn`, `FnMut`, and `FnOnce`. So all three bounds below have elided lifetimes:

```
F: Fn(&[T]) -> Option<&T>
F: FnMut(&[T]) -> Option<&T>
F: FnOnce(&[T]) -> Option<&T>
```

Their full forms are, respectively:

```
F: for<'a> Fn(&'a [T]) -> Option<&'a T>
F: for<'a> FnMut(&'a [T]) -> Option<&'a T>
F: for<'a> FnOnce(&'a [T]) -> Option<&'a T>
```

The difference between the three `traits` is still how the closure may be called, and whether calling it modifies or consumes the captured values; the lifetime elision rules they use are the same.

13.2.2.2 What `for<'a>` Means

`for<'a>` says "**this holds for every possible `'a`**." So:

```
for<'a> F: Fn(&'a [T]) -> Option<&'a T>
```

Or equivalently:

```
F: for<'a> Fn(&'a [T]) -> Option<&'a T>
```

Both say that `F` accepts an `&[T]` of any lifetime, and that the `&T` it returns uses the same lifetime as that call's input.

HRTB is short for **higher-ranked `trait` bound**. The name sounds forbidding, but for now there's only one reading that matters: "this `trait` bound must hold for all `'a`."

13.2.2.3 Who Gets to Pick `'a`?

Back to the three lifetimes in `select_from_both`:

```
fn select_from_both<'first, 'second, T, F>(
    first: &'first [T],
    second: &'second [T],
    choose: F,
) -> (Option<&'first T>, Option<&'second T>)
where
    F: for<'a> Fn(&'a [T]) -> Option<&'a T>,
{
    /* ... */
}
```

`'first` and `'second` are the function's own generic parameters, fixed by the caller when they pass in the two sets of data.

`for<'a>`, on the other hand, sits inside `F`'s bound. `select_from_both` picks `'a = 'first` for the first call to `choose`, then `'a = 'second` for the second. So it's **the side using `choose`** that picks `'a` for each call, and `F` has to accept them all.

13.2.2.4 Why Isn't an Ordinary Lifetime Parameter Enough?

If we tie `choose` to `'first` alone, the first call is fine and the second fails:

```
fn select_from_both<'first, 'second, T, F>(
    first: &'first [T],
    second: &'second [T],
    choose: F,
) -> (Option<&'first T>, Option<&'second T>)
where
    F: Fn(&'first [T]) -> Option<&'first T>,
{
    let from_first = choose(first);
    let from_second = choose(second);
    (from_first, from_second)
}
```

This bound only guarantees that `choose` can take an `&'first [T]` and return an `Option<&'first T>`. The second call needs it to take an `&'second [T]` and return an `Option<&'second T>`. Unless `'first` and `'second` happen to be identical, the guarantee falls short.

Switch to an HRTB and the same `choose` can use different lifetimes across the two calls:

```
fn select_from_both<'first, 'second, T, F>(
    first: &'first [T],
    second: &'second [T],
    choose: F,
) -> (Option<&'first T>, Option<&'second T>)
where
    F: for<'a> Fn(&'a [T]) -> Option<&'a T>,
{
    let from_first = choose(first);
    let from_second = choose(second);
    (from_first, from_second)
}
```

In practice you can usually lean on lifetime elision and write the bound as:

```
F: Fn(&[T]) -> Option<&T>
```

You still need to recognize the full form with `for<'a>`, because more complicated APIs spell it out.

13.2.2.5 The Key Test for HRTB

HRTB isn't needed only when the same closure gets called two or more times with different lifetimes. Even a single call may need it. First consider building and calling a closure directly in the same scope:

```
fn main() {
    let values = [10, 20, 30];
    let length = |items: &[i32]| items.len();

    println!("Length: {}", length(&values));
}
```

There's no generic API taking an `F` here, so there's no bound for you to write `for<'a>` in. In fact, because the parameter is annotated `&[i32]`, the signature the compiler infers for `length` is higher-ranked all along: `for<'a> Fn(&'a [i32]) -> usize`. It's just that the compiler handles this `for<'a>` for you throughout, so you never see it.

The real difference appears when you have to write the bound yourself. A generic API takes a function or closure, then hands it a reference created only inside the API:

```
struct Report {
    entries: usize,
}

fn inspect_report<F>(inspect: F)
where
    F: for<'a> Fn(&'a Report),
{
    let report = Report { entries: 3 };
    inspect(&report);
}

fn main() {
    inspect_report(|report| {
        println!("{} entries in total", report.entries);
    });
}
```

The `&Report` in the bound needs a lifetime, and it could come from two places: `inspect_report`'s own generic parameter, or `for<'a>`. The former is fixed by the caller when they call `inspect_report`, and `report` is only created after entering the function body — it can't outlive any `'a` the caller could possibly pick:

```
fn inspect_report<'a, F>(inspect: F)
where
    F: Fn(&'a Report),
{
    let report = Report { entries: 3 };
    inspect(&report);
}
```

That leaves only `for<'a>`: letting `inspect_report` pick `'a` itself on each call to `inspect`. You could also lean on lifetime elision here and write `F: Fn(&Report)`, but that only elides the HRTB — it doesn't make the requirement go away.

A closure **capturing** references to outer local variables is a separate matter. Capturing constrains how long the closure itself can live, which is a different question from whether `F` has to accept multiple lifetimes.

When reading this sort of API, start by asking:

Is the lifetime of the reference passed to `F` decided in advance by the caller, or by the API each time it calls `F`?

If it's the latter, `F` usually needs higher-ranked capability.

13.2.3 Example Code

```
#[derive(Debug)]
struct Product {
    name: String,
    price: u32,
}

fn select_from_both<'first, 'second, F>(
    first: &'first [Product],
    second: &'second [Product],
    choose: F,
) -> (Option<&'first Product>, Option<&'second Product>)
where
    F: for<'a> Fn(&'a [Product]) -> Option<&'a Product>,
{
    let from_first = choose(first);
    let from_second = choose(second);
    (from_first, from_second)
}

fn main() {
    let first_catalog = vec![
        Product {
            name: String::from("Keyboard"),
            price: 2_500,
        },
        Product {
            name: String::from("Mouse"),
            price: 1_200,
        },
    ];
    let second_catalog = vec![
        Product {
            name: String::from("Monitor"),
            price: 8_000,
        },
        Product {
            name: String::from("Speakers"),
            price: 3_000,
        },
    ];

    let (first_featured, second_featured) = select_from_both(
        &first_catalog,
        &second_catalog,
        |products: &[Product]| {
            products.iter().max_by_key(|product| product.price)
        },
    );

    if let (Some(first), Some(second)) = (first_featured, second_featured) {
        println!("Featured in the first: {}, price: {}", first.name, first.price);
        println!("Featured in the second: {}, price: {}", second.name, second.price);
    }
```

```
}
```

13.2.4 Recap

- Lifetime elision applies to the parameters and return values of `Fn`, `FnMut`, and `FnOnce` too.
- With only one input lifetime, `Fn(&[T]) -> Option<&T>` amounts to `for<'a> Fn(&'a [T]) -> Option<&'a T>`.
- `for<'a>` says the `trait` bound that follows holds for every `'a`.
- An outer `fn foo<'a>` usually lets the caller pick `'a`; the `for<'a>` in an HRTB lets the side using `F` pick `'a` on each call.
- When the same function or closure must be called once per reference of differing lifetimes while keeping each output's lifetime separate, an HRTB expresses that requirement directly.
- A generic API that hands `F` a reference created inside the function body also needs `F` to be higher-ranked; when you build and call a closure directly, the compiler infers this and you never write `for<'a>`.
- In practice lifetime elision usually hides this sort of HRTB, but you'll still meet `for<'a>` head-on when reading advanced signatures.

13.3 Variance

13.3.1 Goal of This Episode

Understand covariance, invariance, and contravariance, and know how the original subtype relationship shifts once a type is wrapped in a reference, a `Cell`, or a function.

> This episode supplements **Chapter 5's lifetimes** and the first two episodes of this appendix.

13.3.2 Concept

We already know that if `'long: 'short`, an `&'long T` can be shortened into an `&'short T`. That means lifetimes stand in a **subtype** relationship to one another: a reference with the longer guarantee can be used wherever only the shorter guarantee is required.

But once a type is placed inside another type, that relationship isn't necessarily preserved as-is. Studying "how the subtype relationship changes across a wrapper" is variance.

13.3.2.1 Covariance: The Direction Stays the Same

Start with shared references:

```
fn use_short<'short>(value: &'short str, _marker: &'short ()) {
    println!("{value}");
}

fn main() {
    let forever: &'static str = "I live until the program ends";

    {
```

```
        let marker = ();
        use_short(forever, &marker);
    }
}
```

An `&'static str` has a very long lifetime, but `use_short` only asks for one as short as `marker`'s. Rust can use the long reference as a short one.

We say `&'a T` is **covariant** in `'a`: if `'long` is a subtype of `'short`, then `&'long T` is likewise a subtype of `&'short T` — the direction is unchanged.

Common owning types such as `Box<T>`, `Vec<T>`, and `Option<T>` are usually covariant in `T` as well. If `'long: 'short`, for instance, a `Vec<&'long str>` can be used where a `Vec<&'short str>` is needed. Doing so hands over ownership of the whole `Vec`, so the original location can no longer reach that same `Vec` through the longer-lifetime type.

13.3.2.2 Invariance: The Inner Type Can't Be Converted Along

Mutable references are a special case. An `&'a mut T` can still be shortened in the lifetime `'a`, but it is **invariant** in the `T` inside.

The reason is that `&mut T` doesn't just let you read the `T` — it lets you put a new `T` in.

Suppose we want to perform this assignment:

```
*slot = short;
```

`short`'s type is `&'short str`. For the assignment to work, the `*slot` on the left must also be an `&'short str`. And dereferencing `slot` gives `*slot`, so `slot`'s type must be:

```
&mut &'short str
```

So the function responsible for storing the short-lived reference would be written:

```
fn replace_with_short<'short>(
    slot: &mut &'short str,
    short: &'short str,
) {
    *slot = short;
}
```

The assignment inside the function is fine, since `*slot` and `short` are both `&'short str`. The problem shows up at the call site:

```
fn replace_with_short<'short>(
    slot: &mut &'short str,
    short: &'short str,
) {
    *slot = short;
}

fn main() {
    let mut forever: &'static str = "the original long-lived data";

    {
        let temporary = String::from("short-lived data");
```

```
        replace_with_short(&mut forever, &temporary);
    }

    println!("{forever}");
}
```

`forever`'s type is `&'static str`, so `&mut forever` has type `&mut &'static str`. But `temporary` can only supply an `&'short str`, so `replace_with_short`'s first parameter needs an `&mut &'short str`.

Allowing the call would force the compiler to perform this conversion:

```
&mut &'static str → &mut &'short str
```

And then `*slot = short` inside the function would write the short-lived reference into `forever`. Once the mutable borrow ends, `forever`'s type still claims to be `&'static str` while in reality it may already dangle. So what Rust forbids is the conversion at the call site, not the assignment inside the function, where both sides have the same type.

Hence, even though an `&'static str` can serve as a shorter `&str`, an `&mut &'static str` cannot freely follow along into an `&mut &'short str`. Wrapped in `&mut`, the inner `T`'s subtype relationship is locked down.

Types offering interior mutability — `Cell<T>`, `RefCell<T>`, `Mutex<T>` — are invariant in `T` across the board. Even though what you hold may only be an `&Cell<T>`, they can still swap out the value inside.

13.3.2.3 Contravariance: The Direction Reverses

A third case shows up in function parameters: **contravariance**.

To isolate variance alone, the outer function below fixes `'short` and `'long` up front:

```
fn use_as_long_only<'short, 'long: 'short>(
    can_accept_short: fn(&'short str),
) -> fn(&'long str) {
    // fn(&'short str) is converted into fn(&'long str) here.
    can_accept_short
}
```

If `'long: 'short`, an `&'long str` can be used as an `&'short str`. But wrapped inside a function parameter, the direction of the permitted conversion reverses:

```
&'long str      → &'short str
fn(&'short str) → fn(&'long str)
```

The function `use_as_long_only` returns will only ever receive `&'long str`. The original `can_accept_short` can handle anything whose reference lives at least as long as `'short`; an `&'long str`, living even longer, certainly meets that requirement, so the conversion is safe.

The relationship for a function's **input** type therefore runs opposite to intuition: a function that demands less and accepts a wider range of inputs can be placed where a narrower input is expected. Function parameters are contravariant in their type.

Function **return values**, by contrast, behave like ordinary reads and are covariant: a function

that can return a reference with the longer guarantee also satisfies a caller who only asks for a shorter one.

13.3.2.4 A Quick Reference Table

Type position	Variance in the parameter	Intuition
`&'a T`	Covariant in `'a` and `T`	Shared reads only, so the guarantee can be shortened
`&'a mut T`	Covariant in `'a`, invariant in `T`	Can swap the `T`, so the inner promise can't be rewritten
`*const T`	Covariant in `T`	The `T` can only be read through the pointer; direction unchanged
`*mut T`	Invariant in `T`	The `T` can be swapped through the pointer, so the inner promise can't be rewritten
`Box<T>`, `Vec<T>`, `Option<T>`	Usually covariant in `T`	They own the value; no arbitrary `T` gets swapped in through a shared entry point
`Cell<T>`, `RefCell<T>`	Invariant in `T`	The contents can still be swapped while shared
`fn(T) -> U`	Contravariant in `T`, covariant in `U`	Can accept wider input, and can return a value with a stronger guarantee

This table is for looking things up, not memorizing. When the compiler rejects a lifetime shortening that looks perfectly reasonable, start by asking: "can a new value be written into this type?" If it can, invariance is usually the reason.

13.3.2.5 Variance Doesn't Change Runtime Results

Variance is entirely a compile-time typing rule. It inserts no conversions at runtime and doesn't actually rebuild any reference. The compiler is merely deciding whether one type can safely be used where another is expected.

13.3.3 Example Code

```
fn choose_shorter<'short>(
    long: &'static str,
    short: &'short str,
    use_long: bool,
) -> &'short str {
    if use_long {
        long
    } else {
        short
    }
}

fn print_text(text: &str) {
    println!("The function received: {text}");
```

```
}

fn use_as_long_only<'short, 'long: 'short>(
    can_accept_short: fn(&'short str),
) -> fn(&'long str) {
    // fn(&'short str) is converted into fn(&'long str) here.
    can_accept_short
}

fn main() {
    let local = String::from("a local string");

    // Shared references are covariant: 'static can shorten to local's lifetime.
    let selected = choose_shorter("long-lived data", &local, true);
    println!("Selected: {selected}");

    // Function inputs are contravariant: the parameter type converts the other way.
    let print_long: fn(&'static str) = use_as_long_only(print_text);
    print_long("from a string literal");
}
```

13.3.4 Recap

- Variance describes how the original subtype relationship changes once a type is placed inside another type.
- Covariant preserves the direction; shared references, function return values, and many owning types fall into this group.
- Invariant permits no conversion along the subtype relationship; `&mut T`, `Cell<T>`, and others that can write the inner value are the common cases.
- Contravariant reverses the direction; function parameters are the main example.
- Variance is a compile-time rule — it never extends a lifetime or produces a runtime conversion.

13.4 `PhantomData` / `PhantomPinned`

13.4.1 Goal of This Episode

Understand how a marker type tells the compiler about a type designer's promise without storing any actual data, and tell apart the different jobs of `PhantomData` and `PhantomPinned`.

> This episode supplements **Chapter 5's generics**, this appendix's variance episode, and `Pin` from the **async chapter**.

13.4.2 Concept

Some standard library types carry no useful runtime data of their own; they exist purely to influence type checking. Such types are commonly called **marker types**.

This episode's two protagonists have similar names, and both affect whether the enclosing type automatically implements certain `auto traits`, but they express different things:

- `PhantomData<T>`: says "this type logically uses `T`," so `T`'s properties may affect the enclosing type.
- `PhantomPinned`: a marker whose sole purpose is to stop the enclosing type from automatically implementing `Unpin`.

13.4.2.1 The Problem: A Generic Parameter That Never Appears in a Field

Suppose we want to tag database IDs differently, making `Id<User>` and `Id<Article>` distinct types at compile time so user IDs and article IDs never get mixed up:

```
struct Id<T> {
    value: u64,
}
```

We only want `T` to serve as a type tag; there's no need to actually store a `User` or an `Article`. But Rust doesn't allow declaring a generic parameter that goes entirely unused in the fields, so the code above won't compile. The compiler can't tell whether this is a deliberate tag or an accidental omission of `T`.

This is where `PhantomData<T>` comes in, stating explicitly "I really do want this type to be related to `T`":

```
use std::marker::PhantomData;

struct User;
struct Article;

struct Id<T> {
    value: u64,
    _kind: PhantomData<T>,
}

fn show_user(id: Id<User>) {
    println!("User ID: {}", id.value);
}

fn main() {
    let user_id = Id::<User> {
        value: 7,
        _kind: PhantomData,
    };
    show_user(user_id);

    let _article_id = Id::<Article> {
        value: 7,
        _kind: PhantomData,
    };
    // show_user(_article_id); // Id<Article> is not Id<User>
}
```

`PhantomData<T>` doesn't actually put a `T` inside, so `Id<T>`'s runtime content is still just a `u64`. But in the type system's eyes, `Id<User>` and `Id<Article>` are already different types.

13.4.2.2 "Logically Uses" Is About More Than Silencing an Error

`PhantomData<T>` tells the compiler: when analyzing this enclosing type, please treat it as related to `T`. That affects:

- **Variance**: `PhantomData<&'a T>`, for instance, carries the relationship between `'a` and `T`.
- **`auto traits`**: whether `T` is `Send`, `Sync`, and so on may affect the enclosing type.

Different spellings express different relationships:

```
PhantomData<T>     // As if it logically owns a T
PhantomData<&'a T> // As if it logically borrows an &'a T
PhantomData<fn(T)> // T sits in function input position
```

These distinctions matter a great deal in low-level libraries. In ordinary applications the most common case is the first `Id<T>` example: using a type tag to keep identical underlying data from being mixed up.

13.4.2.3 `PhantomPinned`: Blocking the Automatic `Unpin` Implementation

As the async chapter taught, `Unpin` is an `auto trait` just like `Send` and `Sync`. As long as every field is `Unpin`, the enclosing `struct` normally implements `Unpin` automatically too.

But when designing an address-sensitive type, we may need to tell the compiler explicitly: "even if every other field can move, this type must not be moved once pinned." On stable Rust, if all of a custom type's fields implement `Unpin`, we can't opt out of `Unpin` with a single direct declaration; we have to include a field that doesn't implement `Unpin`. The `Futures` produced by `async fn` and `async` blocks usually don't implement `Unpin`, but their concrete types are anonymous, so no type name can be written into a field. `PhantomPinned` is exactly the named zero-sized marker the standard library provides for this.

Putting `PhantomPinned` in a field stops the enclosing type from automatically implementing `Unpin`:

```
use std::marker::PhantomPinned;

struct AddressSensitive {
    name: String,
    _pin: PhantomPinned,
}

fn assert_unpin<T: Unpin>(_: T) {}

fn main() {
    let value = AddressSensitive {
        name: String::from("don't assume I can be moved"),
        _pin: PhantomPinned,
    };

    assert_unpin(value);
}
```

The gist of the error is that `PhantomPinned` doesn't implement `Unpin`, so `AddressSensitive`, which contains it, doesn't implement `Unpin` automatically either.

But adding `PhantomPinned` **is not the same as having pinned the value**. It only changes whether the type automatically implements `Unpin`; actually creating a `Pin<&mut T>` or a `Pin<Box<T>>` still requires `pin!`, `Box::pin`, or the like.

13.4.2.4 Moving Is Still Allowed at Construction Time

`PhantomPinned` doesn't make a value completely immovable from birth. The rule from the async chapter still holds: **moving is fine before pinning; the address only has to stay put afterward.**

```
use std::marker::PhantomPinned;
use std::pin::Pin;

struct AddressSensitive {
    name: String,
    _pin: PhantomPinned,
}

fn show(value: Pin<&AddressSensitive>) {
    println!("{} is at {:p}", value.name, &*value);
}

fn main() {
    let value = AddressSensitive {
        name: String::from("fixed position"),
        _pin: PhantomPinned,
    };

    // Until now value is still an ordinary value and can move into Box::pin.
    let pinned = Box::pin(value);
    show(pinned.as_ref());
}
```

`Box::pin` moves the value into its final position on the heap and creates a `Pin<Box<T>>`. The `Pin<Box<T>>` pointer itself can be moved afterward, but no safe API can move the `AddressSensitive` inside out of its address.

13.4.2.5 A Practical Case: Keeping an Executor on the Thread That Created It

The executor built from Episode 11 of the async chapter onward records the current `Thread` with `thread::current()` inside `Executor::new()`. Later, `Task::wake` calls `.unpark()` through that `Thread` handle, while an executor with no work parks the `Thread` currently running it via `thread::park()`.

Hidden here is a constraint the type system doesn't spot on its own: the `Executor` has to stay on the `Thread` that created it. Here's a simplified version:

```
use std::thread::{self, Thread};

struct Task {
    executor_thread: Thread,
}

impl Task {
    fn wake(self) {
```

```
        self.executor_thread.unpark();
    }
}

struct Executor {
    executor_thread: Thread,
}

impl Executor {
    fn new() -> Self {
        Self {
            executor_thread: thread::current(),
        }
    }

    fn run_once(&self) {
        let task = Task {
            executor_thread: self.executor_thread.clone(),
        };

        thread::spawn(move || task.wake())
            .join()
            .expect("the thread failed");
        thread::park();
    }
}

fn main() {
    let executor = Executor::new(); // Remembers the main thread

    // Executor's fields are all Send, so this compiles — and may hang forever.
    thread::spawn(move || executor.run_once())
        .join()
        .expect("the thread failed");
}
```

A `Thread` handle can be sent across threads, but moving the handle doesn't change which thread it points at. After the example above moves `executor` to a new `Thread`:

1. `task.wake()` calls `.unpark()` on the **main thread**, the one that created the executor.
2. `run_once()`, meanwhile, calls `thread::park()` on the **new `Thread`**.

So `.unpark()` isn't ineffective — it delivers the wake token to the wrong `Thread`. The new `Thread` actually running the executor never receives a token and may block indefinitely.

This executor can only stay on the `Thread` that created it, and other `Threads` shouldn't be able to call `.run()` on it through a shared reference either. So we can add a zero-sized marker that blocks the automatic `Send` and `Sync` implementations:

```
use std::marker::PhantomData;
use std::rc::Rc;
use std::thread::{self, Thread};

struct Executor {
    executor_thread: Thread,
```

```
    _not_send_sync: PhantomData<Rc<()>>,
}

impl Executor {
    fn new() -> Self {
        Self {
            executor_thread: thread::current(),
            _not_send_sync: PhantomData,
        }
    }

    fn run(&self) {
        thread::park();
    }
}

fn main() {
    let executor = Executor::new();

    // Compile error: Executor can no longer be sent across threads.
    thread::spawn(move || executor.run());
}
```

`Rc<()>` is neither `Send` nor `Sync`, and `PhantomData<Rc<()>>` makes the `auto trait` analysis treat `Executor` as logically containing an `Rc<()>`. So `Executor` is likewise neither `Send` nor `Sync`: not being `Send` keeps it from being moved to another `Thread`, and not being `Sync` keeps other `Threads` from calling `.run()` through an `&Executor`. What the `move` example above demonstrates directly is that the executor is now non-`Send`; an attempt to share an `&Executor` across `Threads` would be rejected for being non-`Sync`. This field never actually allocates or stores an `Rc`; the correct usage is to create and run the executor on the same `Thread`, while `Wakers` can still call `.unpark()` on that executor's `Thread` from other `Threads`.

13.4.3 Example Code

```
use std::marker::{PhantomData, PhantomPinned};
use std::pin::Pin;

struct Meters;
struct Seconds;

struct Measurement<Unit> {
    value: f64,
    _unit: PhantomData<Unit>,
}

struct AddressSensitive {
    name: String,
    _pin: PhantomPinned,
}

fn print_record(record: Pin<&AddressSensitive>) {
    println!("Record: {}, address: {:p}", record.name, &*record);
}
```

```
fn main() {
    let distance = Measurement::<Meters> {
        value: 12.5,
        _unit: PhantomData,
    };
    let time = Measurement::<Seconds> {
        value: 3.0,
        _unit: PhantomData,
    };

    println!("Distance: {}, time: {}", distance.value, time.value);

    let record = AddressSensitive {
        name: String::from("distance measurement"),
        _pin: PhantomPinned,
    };
    let pinned = Box::pin(record);
    print_record(pinned.as_ref());
}
```

13.4.4 Recap

- A marker type can affect type checking without storing any runtime data.
- `PhantomData<T>` says the enclosing type logically uses, owns, or borrows some `T`.
- `PhantomData<T>` affects variance and `auto traits`.
- `PhantomPinned` blocks the enclosing type's automatic `Unpin` implementation.
- `PhantomPinned` doesn't pin a value by itself; you still create the `Pin` through `pin!`, `Box::pin`, and the like.
- `PhantomPinned` doesn't forbid moves before pinning; the real address guarantee begins once the value is pinned.
- A type that must stay on the `Thread` that created it can use `PhantomData<Rc<()>>` to block the automatic `Send` and `Sync` implementations, keeping it from being moved to another `Thread` or used across `Threads` through a shared reference.

13.5 GAT

13.5.1 Goal of This Episode

Understand generic associated types (GATs), and learn to express "the return type changes with each borrow" using an associated type that takes a lifetime parameter.

> This episode supplements **Chapter 5's associated types** and this appendix's HRTB episode.

13.5.2 Concept

Chapter 5 introduced associated types, and `Iterator` is built the same way:

```
trait Iterator {
    type Item;
```

```
    fn next(&mut self) -> Option<Self::Item>;
}
```

Each implementor picks one `Item`. If some iterator's `Item` is `i32`, for instance, every subsequent call to `next` returns the same type, `Option<i32>`.

But what if `next` wants to return a value **borrowed from the iterator itself**? Each `&mut self` borrow may have a different lifetime, so `Item` has to change along with that borrow. An ordinary associated type has nowhere to put this lifetime parameter.

13.5.2.1 Associated Types Can Take Generic Parameters Too

GATs let an associated type carry generic parameters of its own:

```
trait LendingIterator {
    type Item<'a>
    where
        Self: 'a;

    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>>;
}
```

GAT is short for **generic associated type**. `Item` here no longer stands for a single type but for a whole family of types:

```
Self::Item<'a_short_borrow>
Self::Item<'a_long_borrow>
```

On each call to `next`, the `'a` in `&'a mut self` decides which `Item<'a>` this particular call uses.

13.5.2.2 What Is `where Self: 'a`?

`Item<'a>` may borrow data inside `Self`. For that borrow to be valid throughout `'a`, `Self` itself must of course outlive `'a`:

```
type Item<'a>
where
    Self: 'a;
```

This is exactly Chapter 5's lifetime bound. It doesn't say `Self` must always be `'static`; it says that whenever an `'a` is chosen, every lifetime contained in the `Self` used on that occasion must live at least until `'a` ends.

13.5.2.3 An Iterator That Lends Out Its Own Data

The `Lines` below owns a batch of `String`s and returns one `&str` from them on each `next`:

```
trait LendingIterator {
    type Item<'a>
    where
        Self: 'a;

    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>>;
}

struct Lines {
    data: Vec<String>,
```

```
    position: usize,
}

impl LendingIterator for Lines {
    type Item<'a> = &'a str
    where
        Self: 'a;

    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>> {
        let position = self.position;
        self.position += 1;
        self.data.get(position).map(String::as_str)
    }
}

fn main() {
    let mut lines = Lines {
        data: vec![
            String::from("first line"),
            String::from("second line"),
        ],
        position: 0,
    };

    while let Some(line) = lines.next() {
        println!("{line}");
    }
}
```

In this implementation `Item<'a>` is `&'a str`. The returned string slice borrows `lines.data`, and the duration of that borrow is tied precisely to this particular `&'a mut self`.

13.5.2.4 Why Can't an Ordinary `Iterator` Express This?

Forcing this into an ordinary `Iterator`, you'd want to write:

```
struct Lines {
    data: Vec<String>,
    position: usize,
}

impl Iterator for Lines {
    type Item = &str;

    fn next(&mut self) -> Option<Self::Item> {
        let position = self.position;
        self.position += 1;
        self.data.get(position).map(String::as_str)
    }
}
```

`type Item = &str` has no source for its lifetime. `Iterator::Item` is a fixed type chosen once at implementation time; it can't express "this `&str`'s lifetime comes from the `&mut self` of each call."

A standard `Iterator` can of course return references to data **outside** the iterator — `slice.iter()`'s `Item`, for instance, is an `&'data T` already determined when the iterator is created. GATs solve a different problem: an output that borrows directly from the `self` of each call.

13.5.2.5 Each Lent Value Has to Be Done With First

Because `next`'s output borrows `&mut self`, the iterator can't be mutably borrowed again while that output is still in use:

```
    let mut lines = Lines {
        data: vec![String::from("one"), String::from("two")],
        position: 0,
    };

    let first = lines.next().expect("there should be a first line");
    let second = lines.next().expect("there should be a second line");
    println!("{first}, {second}");
```

`first` is used all the way through the final `println!`, so its borrow of `lines` hasn't ended yet and the second `next()` can't borrow `lines` again. This isn't a flaw in GATs — it's them faithfully expressing a lending API's borrow relationships.

13.5.2.6 GATs Aren't Limited to Lifetimes

The "generic" in the name is meant seriously: an associated type can carry type or `const` parameters as well, such as `type Buffer<T>` or `type Array<const N: usize>`. Lifetime parameters, though, best showcase what ordinary associated types can't do, and they're the form you'll most often need to read in library APIs.

13.5.3 Example Code

```
trait View {
    type Output<'a>
    where
        Self: 'a;

    fn view<'a>(&'a self) -> Self::Output<'a>;
}

struct Document {
    title: String,
    body: String,
}

impl View for Document {
    type Output<'a> = (&'a str, &'a str)
    where
        Self: 'a;

    fn view<'a>(&'a self) -> Self::Output<'a> {
        (&self.title, &self.body)
    }
}
```

```rust
fn print_view<T: View>(value: &T)
where
    for<'a> T::Output<'a>: std::fmt::Debug,
{
    println!("{:?}", value.view());
}

fn main() {
    let document = Document {
        title: String::from("GAT"),
        body: String::from("associated types can take generic parameters too"),
    };

    print_view(&document);
}
```

13.5.4 Recap

- A GAT is an associated type that takes generic parameters.
- `type Item<'a>` stands for a whole family of types varying with `'a`, not a single fixed type.
- `where Self: 'a` guarantees that when the output borrows `Self`, `Self` is still valid throughout `'a`.
- A lending iterator lets each output borrow that call's `&mut self`; an ordinary `Iterator::Item` can't express this relationship.
- While a lent output is still in use, the borrow of `self` is still live as well.
- GATs often turn up alongside HRTBs in advanced `trait` bounds.

13.6 async Closures

13.6.1 Goal of This Episode

Learn the `async |...|` syntax, understand why an `async` closure can borrow the environment it captures, and see what `AsyncFnOnce`, `AsyncFnMut`, and `AsyncFn` are for.

> This episode supplements **Chapter 6's closures** and the **async chapter**.

13.6.2 Concept

The async chapter already used `async fn` and `async` blocks. To hand a piece of asynchronous work to another function, the common approach used to be an ordinary closure returning an `async` block:

```rust
|name| async move {
    println!("Hello, {name}");
}
```

Modern Rust lets you write an `async` closure directly:

```rust
async |name| {
    println!("Hello, {name}");
}
```

Like an ordinary closure it can capture its environment, but calling it doesn't run the body immediately — it produces a `Future`. Only once that `Future` is `polled` does the body make progress.

The runnable examples below call `block_on`. It isn't a standard library API but the most bare-bones executor from Episode 6 of the async chapter; this episode merely borrows it to run immediately-completable `Futures`, so demonstrating language syntax doesn't tie us to a particular runtime.

13.6.2.1 Why Do Ordinary Closures So Often Pair With `async move`?

When an ordinary closure is called, it runs the closure body first and then returns the `Future` the `async` block produced. That `Future` is `.awaited` later, so it may only really run after this call to the ordinary closure has finished.

Suppose the ordinary closure takes a `String` and lets the `async` block use it:

```
fn main() {
    let greet = |name: String| async {
        println!("Hello, {name}");
    };

    let _future = greet(String::from("Ming"));
}
```

There's no `move` here, so the `async` block tries to borrow the parameter `name`. But as soon as the ordinary closure returns the `Future`, this call's `name` parameter should go out of scope; if the `Future` were still borrowing it, the reference could later be invalid when it runs, so Rust rejects this code.

Adding `move` moves `name` into the `Future`:

```
fn main() {
    let greet = |name: String| async move {
        println!("Hello, {name}");
    };

    let _future = greet(String::from("Ming"));
}
```

Now, when the ordinary closure returns, `name` isn't left behind in a call that has already finished — it's owned by the returned `Future` and lives alongside it until the `Future` completes or is dropped. That's why `|param| async move { ... }` is more common than `|param| async { ... }`.

That said, `async move` isn't mandatory in every situation. If the `async` block borrows no parameter or local variable from this closure call, `move` isn't necessarily needed. The point is: **a `Future` must not borrow data that expires when the ordinary closure's call ends.**

13.6.2.2 Your First `async` Closure

```
use std::future::Future;
use std::task::{Context, Poll, Waker};
```

```
fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

fn main() {
    let greet = async |name: &str| {
        std::future::ready(()).await;
        println!("Hello, {name}");
    };

    block_on(async {
        greet("Ming").await;
        greet("Mei").await;
    });
}
```

`greet("Ming")` produces a `Future`; only `.await` runs the `ready` and the `println!` inside. The same closure can be called twice, much like an ordinary `Fn` closure.

13.6.2.3 So Why Not Always Use `|x| async move { ... }`?

An ordinary closure and the `async move` block it returns are two different things. The `async move` block below has to own `prefix`, so each time the outer ordinary closure is called it must move the `prefix` it captured into the newly produced `Future`:

```
fn main() {
    let prefix = String::from("message");

    let make_future = || async move {
        println!("{prefix}");
    };

    let _first = make_future();
    let _second = make_future();
}
```

`String` doesn't implement `Copy`. The first call to the outer closure moves `prefix` into the returned `Future`, which means moving the captured value out of its own environment, so this closure can only implement `FnOnce`. The first call consumes `make_future`, and there can be no second call.

An `async` closure lets the returned `Future` **borrow data held inside the closure**. To make this capability really visible — rather than just copying a shared reference — the closure below holds a mutable reference to a counter:

```
use std::future::Future;
use std::task::{Context, Poll, Waker};
```

```
fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

fn main() {
    let mut count = 0;

    let mut increment = async || {
        std::future::ready(()).await;
        count += 1;
        println!("Call number {count}");
    };

    block_on(async {
        increment().await;
        increment().await;
    });
}
```

`increment` holds a mutable reference to `count` internally. The `Future` produced by each call to `increment()` obtains a shorter-lived mutable reference through that one and holds it until the `Future` completes or is dropped. Once the first `.await` finishes, that short-lived mutable reference is gone, so `increment()` can be called again. This closure implements `AsyncFnMut`, which is why the variable itself must be declared `mut` too.

An ordinary closure that returns an `async` block still holding such a mutable reference won't compile:

```
fn main() {
    let mut count = 0;

    let mut increment = || async {
        std::future::ready(()).await;
        count += 1;
    };

    let _future = increment();
}
```

This `Future` would have to go on holding a mutable reference obtained from inside the closure after the closure call ends, and that reference's lifetime comes from this particular `&mut self`. But an ordinary `FnMut`'s fixed `Self::Output` can't carry a lifetime produced anew on each call.

13.6.2.4 The `AsyncFn` Trio

Chapter 6 taught that an ordinary closure implements `FnOnce`, `FnMut`, or `Fn` depending on how its body uses the captured values. `async` closures have a matching set of `traits`:

Ordinary closure	async closure	How it's called
`FnOnce`	`AsyncFnOnce`	Callable once; may consume captured values
`FnMut`	`AsyncFnMut`	Callable repeatedly through a mutable reference
`Fn`	`AsyncFn`	Callable repeatedly through a shared reference

13.6.2.5 Roughly What Does the Compiler Implement?

Chapter 6 Episode 3 had us picture an ordinary closure as "an anonymous `struct` storing the captured values, with `FnOnce`, `FnMut`, or `Fn` implemented for it." The first half is the same for async closures; the difference is that the call method doesn't compute a result directly but returns a `Future` state machine.

With some implementation details omitted, the three `traits` relate roughly as follows. This is a simplified version meant to explain the structure, not a definition from the standard library you can implement yourself:

```
use std::future::Future;

trait AsyncFnOnce<Args> {
    type Output;
    type CallOnceFuture: Future<Output = Self::Output>;

    fn async_call_once(self, args: Args) -> Self::CallOnceFuture;
}

trait AsyncFnMut<Args>: AsyncFnOnce<Args> {
    type CallRefFuture<'a>: Future<Output = Self::Output>
    where
        Self: 'a;

    fn async_call_mut(&mut self, args: Args) -> Self::CallRefFuture<'_>;
}

trait AsyncFn<Args>: AsyncFnMut<Args> {
    fn async_call(&self, args: Args) -> Self::CallRefFuture<'_>;
}
```

The `self` and return type of the three calling conventions:

`trait`	`self`	Type returned by the call
`AsyncFnOnce`	`self`	`CallOnceFuture`
`AsyncFnMut`	`&mut self`	`CallRefFuture<'_>`
`AsyncFn`	`&self`	The same `CallRefFuture<'_>`

`Output` is the final result obtained after awaiting the `Future`, not the `Future` itself. `CallOnceFuture` comes from a call that consumes the closure, so it can own the captured values moved out of it directly and needs no lifetime parameter.

An ordinary `FnMut`'s `call_mut` returns a fixed `Self::Output`; that associated type can't vary with the lifetime of each `&mut self` call. So an ordinary closure can't return a value that still

holds "a mutable reference obtained from this `&mut self`." The crux is whether the return type can carry the reference lifetime of `&mut self`, not whether the work completes synchronously.

An `async` closure itself can be seen as an anonymous `struct` made of the captured values. The `Futures` returned by `AsyncFnMut` and `AsyncFn` may hold references into that anonymous `struct`'s data, so the `Future` type has to carry the lifetime of its borrow of the closure. `CallRefFuture<'a>` uses a GAT to write that lifetime into the associated type.

`AsyncFn` declares no associated type of its own and reuses `AsyncFnMut`'s `CallRefFuture<'a>`. Neither `async_call` nor `async_call_mut` consumes the closure, and the `Futures` they return may both go on borrowing the captured environment, so a single lifetime-carrying `Future` type suffices for both.

They can be used to write bounds for APIs that take an asynchronous handler:

```
use std::future::Future;
use std::task::{Context, Poll, Waker};

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

async fn run_twice<F>(job: F)
where
    F: AsyncFn(&str),
{
    job("first time").await;
    job("second time").await;
}

fn main() {
    let prefix = String::from("Running");

    block_on(run_twice(async |name| {
        std::future::ready(()).await;
        println!("{prefix}: {name}");
    }));
}
```

`AsyncFn(&str)` reads as: "callable through a shared reference, takes an `&str`, and the call yields asynchronous work that can be awaited."

Just as with `Fn(&str)`, the elided parameter lifetime here has the effect of an HRTB: each time `run_twice` calls `job`, it can pass in the `&str` borrowed on that occasion.

13.6.2.6 `async move |...|`

`async` closures can take `move` as well:

```
use std::future::Future;
use std::task::{Context, Poll, Waker};

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

fn main() {
    let label = String::from("background job");

    let job = async move || {
        println!("{label}");
    };

    block_on(async {
        job().await;
        job().await;
    });
}
```

The `move` here controls how the closure captures `label` from the **enclosing environment**: the closure takes ownership of `label`. It doesn't mean every call has to move `label` back out of the closure; the body only reads it via sharing, so `job` still satisfies `AsyncFn`, which is what lets the code above call it twice in a row.

Keep the two layers distinct:

- `async move || { ... }`: moves outer values into the closure when the closure is created.
- Calling `job()`: produces a `Future` that may borrow that closure.

13.6.3 Example Code

```
use std::future::Future;
use std::task::{Context, Poll, Waker};

fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => {}
        }
    }
}

async fn for_each_async<F>(items: &[String], action: F)
```

```
where
    F: AsyncFn(&str),
{
    for item in items {
        action(item).await;
    }
}

fn main() {
    let items = vec![
        String::from("alpha"),
        String::from("beta"),
        String::from("gamma"),
    ];
    let heading = String::from("Processing");

    block_on(for_each_async(&items, async |item| {
        std::future::ready(()).await;
        println!("{heading}: {item}");
    }));

    println!("{} items processed in total", items.len());
}
```

13.6.4 Recap

- `async |params| { ... }` creates an `async` closure; calling it produces a `Future`.
- Compared with an ordinary closure returning an `async move` block, an `async` closure naturally expresses a `Future` borrowing the closure's captured environment.
- `AsyncFnOnce`, `AsyncFnMut`, and `AsyncFn` correspond to ordinary closures' `FnOnce`, `FnMut`, and `Fn`.
- `async move` controls how outer values are captured when the closure is created; it doesn't mean every call consumes those values.

13.7 `use` Bounds

13.7.1 Goal of This Episode

Understand which generic parameters the hidden type behind a return-position `impl Trait` captures, learn to restrict the capture set explicitly with `use<...>`, and get to know RPITIT.

> This episode supplements **Chapter 5's `impl Trait` and lifetimes**.

13.7.2 Concept

Chapter 5 showed `impl Trait` used in return position:

```
fn numbers() -> impl Iterator<Item = i32> {
    [1, 2, 3].into_iter()
}
```

The `impl Iterator<Item = i32>` the caller sees is an **opaque type**: the interface deliberately

doesn't reveal its concrete identity. The concrete type the function body picks for it is called the **hidden type**; in this example that's `std::array::IntoIter<i32, 3>`. A hidden type isn't necessarily nameless — it's simply concealed by the opaque return type.

If the function has generic parameters, may the hidden type use them? That's the question "capturing" answers.

13.7.2.1 The Hidden Type Really Does Borrow the Parameter

```
fn words<'a>(text: &'a str) -> impl Iterator<Item = &'a str> {
    text.split_whitespace()
}

fn main() {
    let text = String::from("Rust lifetime capture");

    for word in words(&text) {
        println!("{word}");
    }
}
```

What's actually returned is a `SplitWhitespace<'a>`, holding a reference to `text` inside. So the hidden type must be able to use — that is, **capture** — `'a`.

In Rust's 2024 edition, a return-position `impl Trait` captures every lifetime, type, and `const` generic parameter available in scope by default.

13.7.2.2 Capturing More Than Actually Needed

But "permitted to capture" shows up in the caller's borrow checking. Even if the current hidden type never uses a parameter, a public signature that lets it capture one forces the caller to treat the return value as though it might have:

```
fn greeting<'a>(_name: &'a str) -> impl Fn() {
    || println!("Hello!")
}

fn main() {
    let name = String::from("Ming");
    let greet = greeting(&name);

    drop(name);
    greet();
}
```

The closure doesn't use `name` at all, yet under the 2024 edition's default rule the `impl Fn()` is permitted to capture `'a`. So a caller looking only at the signature conservatively assumes `greet` may still borrow `name`, and won't let us `drop(name)` first.

13.7.2.3 `use<...>` Specifies the Capture Set

A `use<...>` bound lists exactly which generic parameters the hidden type may capture:

```
fn greeting<'a>(_name: &'a str) -> impl Fn() + use<> {
    || println!("Hello!")
}
```

```
fn main() {
    let name = String::from("Ming");
    let greet = greeting(&name);

    drop(name);
    greet();
}
```

`use<>` is empty, meaning the hidden type may capture no generic parameter at all. The signature now guarantees explicitly that `greet` has nothing to do with `'a`, so the caller can release `name` first.

The `use` here is nothing like the `use std::...` that imports names. It appears in an `impl Trait`'s bound list, and its job is to list the capture set.

13.7.2.4 List What You Need to Capture

If the hidden type really does use `'a` but the signature says `use<>`, the function itself won't compile:

```
fn text_length<'a>(text: &'a str) -> impl Fn() -> usize + use<> {
    move || text.len()
}
```

The closure holds `text`, so the hidden type uses `'a`. Just add `'a` to the capture set:

```
fn text_length<'a>(text: &'a str) -> impl Fn() -> usize + use<'a> {
    move || text.len()
}

fn main() {
    let text = String::from("the captured data");
    let get_length = text_length(&text);
    println!("{}", get_length());
}
```

`use<'a>` says the hidden type may use `'a`. This time `get_length` really does borrow `text`, so `text` can't be released while it's alive.

13.7.2.5 Type and `const` Generics Can Be Captured Too

`use<...>` isn't just for lifetimes; type parameters and `const` parameters go in it as well:

```
fn repeat<T, const N: usize>(value: T) -> impl Iterator<Item = T> + use<T, N>
where
    T: Clone,
{
    std::iter::repeat_n(value, N)
}

fn main() {
    let values: Vec<_> = repeat::<_, 3>(String::from("Rust")).collect();
    println!("{values:?}");
}
```

For now an `impl Trait` may have at most one `use<...>`, and every type and `const` generic

parameter in scope must be listed. Furthermore, if another part of the return type already uses some lifetime, that must go into `use<...>` too. In `impl Iterator<Item = &'a str> + use<'a>`, for instance, `Item = &'a str` already uses `'a`, so it can't be rewritten as `use<>`. Lifetimes in the list come before type and `const` generic parameters. If the list is incomplete or misordered, the compiler will say so outright.

13.7.2.6 `trait` Methods Can Return `impl Trait` Too: RPITIT

Methods in `trait` definitions and `trait` implementations can use `impl Trait` in return position as well. This is called return-position `impl Trait` in `trait`, abbreviated **RPITIT**:

```
trait Words {
    fn words<'a>(
        &'a self,
    ) -> impl Iterator<Item = &'a str> + use<'a, Self>;
}

struct Sentence(String);

impl Words for Sentence {
    fn words<'a>(
        &'a self,
    ) -> impl Iterator<Item = &'a str> + use<'a> {
        self.0.split_whitespace()
    }
}

fn main() {
    let sentence = Sentence(String::from("return position impl trait"));
    let words: Vec<_> = sentence.words().collect();

    println!("{words:?}");
}
```

The `impl Iterator` returned by `Words::words` is an opaque type; the compiler treats an RPITIT in a `trait` definition as an anonymous associated type. Each `trait` implementation gets to pick its own hidden concrete type for it; the `Sentence` above picks `SplitWhitespace<'a>`, while the caller relies only on the promise `Iterator<Item = &'a str>`.

The `use<'a, Self>` in the `trait` definition captures the `'a` with which the method borrows `self`, plus the `trait`'s implicit type parameter `Self`. As things stand, writing `use<...>` on the RPITIT of a `trait`'s associated function requires listing all of that `trait`'s generic parameters, `Self` included. By the time we reach `impl Words for Sentence`, `Self` is definitively the concrete `Sentence`, so the implementation side only needs `use<'a>`.

Even when the return value doesn't use `self` at all, `use<...>` in a `trait` definition can't omit `Self`:

```
trait FixedNumbers {
    // Compile error: the trait's `Self` isn't listed in `use<...>`.
    fn numbers(&self) -> impl Iterator<Item = i32> + use<>;
}
```

An RPITIT's callers can only use the bounds the `trait` signature has promised. Even if the iter-

ator some implementation actually returns also implements `DoubleEndedIterator`, a generic caller still can't call `.rev()` on it directly; if that capability is needed, the `trait` should use a `DoubleEndedIterator` bound from the start, or switch to a named associated type.

A method returning an opaque type can't be dynamically dispatched through `dyn Trait`, so a `trait` containing such a method usually isn't `dyn` compatible. Adding `where Self: Sized` to the method keeps the `trait`'s other methods usable through `dyn Trait`, though this RPITIT method itself still won't be. `async fn` in `traits` is built on the same mechanism and can be understood conceptually as returning `impl Future<Output = T>`.

13.7.3 Example Code

```
fn borrowed_words<'a>(
    text: &'a str,
) -> impl Iterator<Item = &'a str> + use<'a> {
    text.split_whitespace()
}

fn fixed_numbers<'a>(
    _label: &'a str,
) -> impl Iterator<Item = i32> + use<> {
    [1, 2, 3].into_iter()
}

fn main() {
    let text = String::from("one two three");
    let words: Vec<_> = borrowed_words(&text).collect();
    println!("Borrowed text: {words:?}");

    let label = String::from("never used by the iterator");
    let numbers = fixed_numbers(&label);

    // use<> guarantees numbers didn't capture label's lifetime.
    drop(label);
    println!("Fixed numbers: {:?}", numbers.collect::<Vec<_>>());
}
```

13.7.4 Recap

- A return-position `impl Trait` is an opaque type at the interface, backed by a hidden concrete type the function body decides on.
- Capturing a generic parameter means the hidden type is permitted to use it.
- Rust's 2024 edition captures every lifetime, type, and `const` generic parameter in scope by default.
- `impl Trait + use<'a, T, N>` lists exactly which parameters may be captured; `use<>` captures none.
- If the hidden type really uses a parameter that wasn't listed, the function itself fails to compile.
- A return-position `impl Trait` in a `trait` method is called an RPITIT; using `use<...>` there requires listing `Self` and all of that `trait`'s generic parameters.
- Precisely ruling out unnecessary lifetime capture keeps callers' reference lifetimes from

being conservatively extended.

Afterword

Finishing this tutorial does not mean that you have mastered computer science; it only means that you have learned the broad outlines of Rust's design and how to write Rust programs. In fact, this book simplifies many concepts. If you would like to study programming in greater depth, my first recommendation would be to learn **data structures**, whether by taking a university course, reading books, or finding other tutorials online. Because this tutorial is aimed primarily at people with no prior programming experience, it greatly simplifies its explanations of data structures and computational complexity. It also devotes very little space to `unsafe`; on this subject, I recommend *The Rustonomicon*. Although The Rustonomicon is still incomplete, and may never be complete, it is already the best resource for learning how Rust interacts with hardware at a low level.

If you want to get closer to low-level hardware, you may also need to learn another programming language such as C, since most widely used programming languages offer interoperability with C through an FFI. Perhaps, however, you are like me and find programming languages more interesting than hardware. In that case, you may want to explore languages used more often in academia, such as the purely functional Haskell, or even Agda or Lean, which can be used to construct mathematical proofs. Although the design philosophies of those high-level languages differ greatly from Rust's, Rust has borrowed quite a few ideas from them, so the time you spent learning Rust has not gone to waste. Who knows? Perhaps one of those languages will become popular in the future.

It may be a little late to say this, but at the very end, I would like to thank my family for giving me the opportunity to learn what I wanted to learn and to complete this tutorial.